

Joachim Goll
Tobias Stamm

C als erste Programmier- sprache

Nach den Standards C11 und C23

9. Auflage

IT
DESIGNERS
GRUPPE

MOREMEDIA



Springer Vieweg

C als erste Programmiersprache

Joachim Goll · Tobias Stamm

C als erste Programmiersprache

Nach den Standards C11 und C23

9. Auflage

Joachim Goll
Fakultät Informationstechnik
Hochschule Esslingen
Esslingen, Deutschland

Tobias Stamm
Lenzburg, Schweiz

ISBN 978-3-658-45208-7 ISBN 978-3-658-45209-4 (eBook)
<https://doi.org/10.1007/978-3-658-45209-4>

Die Deutsche Nationalbibliothek verzeichnet diese Publikation in der Deutschen Nationalbibliografie; detaillierte bibliografische Daten sind im Internet über <https://portal.dnb.de> abrufbar.

© Der/die Herausgeber bzw. der/die Autor(en), exklusiv lizenziert an Springer Fachmedien Wiesbaden GmbH, ein Teil von Springer Nature 1998, 2000, 2003, 2005, 2008, 2011, 2014, 2024

Das Werk einschließlich aller seiner Teile ist urheberrechtlich geschützt. Jede Verwertung, die nicht ausdrücklich vom Urheberrechtsgesetz zugelassen ist, bedarf der vorherigen Zustimmung des Verlags. Das gilt insbesondere für Vervielfältigungen, Bearbeitungen, Übersetzungen, Mikroverfilmungen und die Einspeicherung und Verarbeitung in elektronischen Systemen.

Die Wiedergabe von allgemein beschreibenden Bezeichnungen, Marken, Unternehmensnamen etc. in diesem Werk bedeutet nicht, dass diese frei durch jede Person benutzt werden dürfen. Die Berechtigung zur Benutzung unterliegt, auch ohne gesonderten Hinweis hierzu, den Regeln des Markenrechts. Die Rechte des/der jeweiligen Zeicheninhaber*in sind zu beachten.

Der Verlag, die Autor*innen und die Herausgeber*innen gehen davon aus, dass die Angaben und Informationen in diesem Werk zum Zeitpunkt der Veröffentlichung vollständig und korrekt sind. Weder der Verlag noch die Autor*innen oder die Herausgeber*innen übernehmen, ausdrücklich oder implizit, Gewähr für den Inhalt des Werkes, etwaige Fehler oder Äußerungen. Der Verlag bleibt im Hinblick auf geografische Zuordnungen und Gebietsbezeichnungen in veröffentlichten Karten und Institutionsadressen neutral.

Planung/Lektorat: Leonardo Milla

Springer Vieweg ist ein Imprint der eingetragenen Gesellschaft Springer Fachmedien Wiesbaden GmbH und ist ein Teil von Springer Nature.

Die Anschrift der Gesellschaft ist: Abraham-Lincoln-Str. 46, 65189 Wiesbaden, Germany

Wenn Sie dieses Produkt entsorgen, geben Sie das Papier bitte zum Recycling.

Vorwort

Die Programmiersprache C wird seit über 50 Jahren in allen grundlegenden Bereichen der technisierten Welt eingesetzt. Sei es Betriebssystem, Hardwaresteuerung oder Applikationsentwicklung, C ist die Basis moderner Informatik.

Seit der ersten Standardisierung von C im Jahre 1989 hat sich die Programmierwelt bedeutend verändert. Neue Paradigmen und entsprechend auch neue Programmiersprachen wurden entwickelt, und einige davon haben sich in bestimmten Bereichen auch durchgesetzt.

Das grundlegende Verständnis, wie all diese Programmiersprachen arbeiten, wird jedoch abgebildet durch C. Welche Konzepte auch immer in modernen Sprachen eingebaut werden, sie finden ihren Ursprung in der Behandlung von Adressen im Speicher mit einem zentralen Rechenkern. Für genau diese Aufgabe war C von Anfang an entwickelt worden. Die Sprache erlaubt es, auf intuitive und strukturierte Weise hardwarenahe zu programmieren und dennoch selbst die komplexesten Projekte durch Modularisierung im Griff zu haben. Die Sprache C hat sich weltweit als Standard für viele Anwendungen durchgesetzt.

Das Studium von C ist somit von großem Wert, sei es zum Erlernen der ersten Programmiersprache, oder als Ergänzung zu einer anderen, bereits beherrschten Sprache. Die Einblicke, die C in die Steuerung eines Computers erlaubt, sind für alle Bereiche des Programmierens wichtig.

Aufbau des Buches

Lerninhalte werden in diesem Buch strukturiert aufgebaut, von einfachen Programmen über Basiselemente der Sprache bin hin zu erweiterten Konzepten. Auch werden Inhalte wie Flussdiagramme, Suchen und Sortieren, oder die Handhabung von Threads in einem Betriebssystem angesprochen. Diese Inhalte haben nicht direkt mit der Programmiersprache C selbst zu tun, bilden jedoch ein wichtiges Fundament zum besseren Verständnis moderner Informatik.

Folgende Symbole helfen beim Lesen des Buches:



Lernkästchen, auf die grafisch durch eine kleine Glühlampe aufmerksam gemacht wird, weisen auf für das Verständnis wichtige Aspekte eines Kapitels hin und bringen das Wissen auf den Punkt.



Warnkästchen weisen zum einen auf gerne begangene Fehler hin. Zum anderen warnen sie vor potentiellen Fehlinterpretationen beim Lesen des Textes.



Überspringbare Kapitel deuten an, dass die Inhalte nicht grundlegend für das weitere Vorankommen sind, somit nur ein Detailwissen darstellen und deshalb auch noch zu einem späteren Zeitpunkt vertieft werden können.

Des Weiteren finden sich bei den meisten Kapiteln am Ende Übungsaufgaben zur selbständigen Vertiefung des gelernten Stoffs. Alle Dateien inklusive Musterlösung können von der folgenden Website heruntergeladen werden, damit Sie diese bequem selbst bearbeiten können. Eine mögliche Errata-Liste mit Korrekturen zum Buch wird ebenfalls dort zu finden sein.

<https://link.springer.com/>

Zum Inhalt

Die Programmiersprache C wurde bereits mehrfach standardisiert. Die in diesem Buch vorgestellten Inhalte orientieren sich nach dem neuesten Standard, welcher zum Zeitpunkt des Verfassens C11 war. Die Inhalte des nächsten Standards, welcher aktuell als C23 erwartet wird, werden jedoch ebenfalls bereits erwähnt.

Die älteren Standards C90 und C99 sind implizit in diesem Buch mit enthalten, jedoch werden Inhalte, welche als überholt oder für einen Einstieg in die Programmierung als zu weitreichend gelten, nur noch in den Anhängen behandelt oder es wird auf eine genaue Behandlung verzichtet.

Die vorgestellten Programmbeispiele sind minimalistisch aufgebaut, um den Überblick zu erleichtern. Dies bedeutet, dass an manchen Stellen heutzutage übliche Sicherheits-Prüfungen fehlen. Dort, wo dies für den tatsächlichen Programmieralltag kritisch ist, wird darauf hingewiesen. Die Programmbeispiele finden sich übrigens genauso wie die Übungsaufgaben und Lösungen unter obengenanntem Webaufttritt.

Eine Programmiersprache wird am besten erlernt, indem Dinge ausprobiert werden. Man soll sich nicht davor scheuen, Fehler zu machen oder scheinbar zu viel Zeit an einem Thema zu verbringen. Jede Erfahrung kann zum Verständnis beitragen. Und für den Fall, dass man sich in einer Sackgasse befindet, darf auch gerne zur Ergänzung das Internet zu Rate gezogen werden.

Danksagung

Wir bedanken uns an dieser Stelle herzlich für die wertvolle Korrekturarbeit und unterstützende Erfahrung von Prof. Dr. Manfred Dausmann.

Lenzburg und Esslingen, im Dezember 2023

Tobias Stamm, Joachim Goll

Inhaltsverzeichnis

1	Einführung in die Programmiersprache C	1
1.1	Hello World	2
1.2	Die Entwicklung von C	6
1.3	Abstraktion	12
1.4	Vorgänger und Nachfolger von C	16
1.5	Standardisierung von C	18
2	Einfache Beispielprogramme	21
2.1	Quadratzahlen	21
2.2	Celsius und Fahrenheit	28
2.3	Zinsberechnung	34
3	Entwurf von Programmen	39
3.1	Vom Problem zum Programm	39
3.2	Entwurf mit Flussdiagrammen nach UML	46
3.3	Pseudocode	54
3.4	Das finale Programm von Euklid in C	56
3.5	Übungsaufgaben	57
4	Aufbau eines Programmes in C	59
4.1	Zeichen und Codierungen	59
4.2	Variablen und Werte	64
4.3	Funktionen in prozeduralen Programmen	68
4.4	Schrittweise Verfeinerung beim Top-Down-Design	73
4.5	Funktionen in C	77
4.6	Struktur einer Quelldatei in C	79
5	Programmerzeugung und -ausführung	85
5.1	Compiler	87
5.2	Linker	93
5.3	Lader	95
5.4	Integrierte Entwicklungsumgebungen und Debugger	97
6	Lexikalische Konventionen	99
6.1	Zeichenvorrat von C	99
6.2	Lexikalische Einheiten	101
6.3	Reservierte Wörter (Schlüsselwörter)	103
6.4	Bezeichner	105
6.5	Literale Konstanten	107
6.6	Operatoren und Interpunktionszeichen	115
6.7	Übungsaufgaben	117

7	Datentypen und Variablen in C	119
7.1	Typkategorien	120
7.2	Elementare Datentypen in C	121
7.3	Modifikatoren und andere Besonderheiten	132
7.4	Variablen in C	135
7.5	Qualifikatoren	141
7.6	Übungsaufgaben	144
8	Einführung in Pointer und Arrays	145
8.1	Pointer-Typen und Pointer-Variablen	145
8.2	Pointer auf void	158
8.3	Arrays	159
8.4	Der Qualifikator restrict	166
8.5	Übungsaufgaben	168
9	Ausdrücke, Anweisungen und Operatoren	171
9.1	Ausdrücke und Anweisungen	171
9.2	Operatoren und Operanden	180
9.3	Einstellige arithmetische Operatoren	187
9.4	Zweistellige arithmetische Operatoren	189
9.5	Zuweisungs-Operatoren	192
9.6	Relationale Operatoren	194
9.7	Logische Operatoren	197
9.8	Bit-Operatoren	201
9.9	Sonstige Operatoren	206
9.10	Implizite Typumwandlung	217
9.11	Konvertierungsvorschriften	220
9.12	Übungsaufgaben	223
10	Kontrollstrukturen	229
10.1	Blöcke – Kontrollstruktur für die Sequenz	229
10.2	if und else – Einfache Selektion	230
10.3	switch – Fallunterscheidung	234
10.4	while – Bedingte Schleife	241
10.5	for – Zählschleife	242
10.6	do while – Annehmende bedingte Schleife	249
10.7	Endlosschleifen und leere Ausdrücke bei Schleifen	250
10.8	break – Abbruch-Anweisung	253
10.9	continue – Fortsetzungs-Anweisung	254
10.10	return – Rücksprung-Anweisung	255
10.11	goto – Sprunganweisung und Marken	255
10.12	Übungsaufgaben	257

11	Blöcke und Funktionen	259
11.1	Struktur und Schachtelung von Blöcken	260
11.2	Lebensdauer, Gültigkeit und Sichtbarkeit von Variablen	262
11.3	Definition von Funktionen	267
11.4	Rücksprung aus einer Funktion – die return-Anweisung	273
11.5	Konventionen bei der Parameterübergabe	275
11.6	Vorwärtsdeklaration von Funktionen	280
11.7	Die Ellipse . . . – variable Parameteranzahl	284
11.8	Iteration und Rekursion	287
11.9	inline-Funktionen	298
11.10	Funktionen ohne Wiederkehr in C11	300
11.11	Übungsaufgaben	301
12	Fortgeschrittene Programmierung mit Pointern	305
12.1	Gleichheit und Unterschiede von Arrays und Pointer	305
12.2	Pointerarithmetik	308
12.3	Initialisierung von Arrays	312
12.4	Übergabe von Arrays an Funktionen	319
12.5	Unterschiede zwischen char-Arrays und Pointern auf char	322
12.6	Das Schlüsselwort const bei Pointern und Arrays	324
12.7	Beispiel für Strings und Pointerarithmetik	326
12.8	Funktionen zur Speicherbearbeitung	329
12.9	Standardfunktionen zur Stringverarbeitung	335
12.10	Weitere Funktionen in <string.h>	343
12.11	Beispiele für Arrays und Pointer	344
12.12	Pointer auf Funktionen	350
12.13	Pointer auf void für Strukturen	354
12.14	Übungsaufgaben	357
13	Strukturen, Unionen und Bitfelder	361
13.1	Strukturen	362
13.2	Anonyme Strukturen	381
13.3	Unionen	384
13.4	Bitfelder	388
13.5	Übungsaufgaben	391
14	Komplexere Vereinbarungen, eigene Typnamen und Namensräume	393
14.1	Komplexere Vereinbarungen	393
14.2	Vereinbarung eigener Typnamen mittels typedef	396
14.3	Namensräume	401
15	Speicherklassen	403
15.1	Der Linker	405

15.2	Der Lader und die Segmente des Adressraums eines Programms	408
15.3	Die Speicherklasse <code>extern</code> bei Programmen aus mehreren Dateien	411
15.4	Die Speicherklasse <code>static</code> bei Programmen aus mehreren Dateien	415
15.5	Die Speicherklasse <code>auto</code> bei lokalen Variablen	417
15.6	Die Speicherklasse <code>register</code> bei lokalen Variablen	418
15.7	Die Speicherklasse <code>static</code> bei lokalen Variablen	420
15.8	Automatische und nicht automatische Initialisierung	422
15.9	Überblick über die Speicherklassen	423
15.10	Übungsaufgaben	424
16	Ein- und Ausgabe	427
16.1	Speicherung von Daten in Dateisystemen	428
16.2	Das Ein-/Ausgabe-Konzept von C	432
16.3	Standardeingabe und Standardausgabe in C	434
16.4	High- und Low-Level-Bibliotheksfunktionen zur Ein- und Ausgabe	438
16.5	Der File-Pointer bei High-Level-Funktionen	440
16.6	Einfaches Schreiben in Dateien mit High-Level-Funktionen	450
16.7	Schreiben von formatierten Strings	453
16.8	Lesen von Dateien mit High-Level-Funktionen	461
16.9	Lesen von formatierten Strings	465
16.10	Alternative Funktionen für mehr Sicherheit nach C11	470
16.11	Übungsaufgaben	474
17	Programmaufruf und -beendigung	477
17.1	Übergabe von Argumenten beim Programmaufruf	477
17.2	Beendigung von Programmen	481
18	Dynamische Speicherzuweisung	489
18.1	Reservierung von Speicher	491
18.2	Freigabe von Speicher mittels <code>free()</code>	497
18.3	Dynamisch erzeugte Arrays	500
18.4	Übungsaufgaben	502
19	Dynamische Datenstrukturen	505
19.1	Verkettete Listen	505
19.2	Baumstrukturen	515
19.3	Übungsaufgaben	528
20	Sortieren und Suchen	531
20.1	Interne Sortiervverfahren	532
20.2	Einfache Suchverfahren	545
20.3	Suchen nach dem Hashverfahren	551
20.4	Suchen mit Hilfe von Backtracking	567
20.5	Übungsaufgaben	573

21	Präprozessor	577
21.1	Einfügen von Dateien	579
21.2	Symbolische Konstanten und Makros mit Parametern	581
21.3	Bedingte Compilierung	587
21.4	Informationen über den Übersetzungskontext	590
21.5	String-Umwandlungs- und Verknüpfungs-Operatoren	592
21.6	Präprozessor-Erweiterungen unter C99 und C11	594
22	Modulares Design in C	599
22.1	Konzept des Strukturierten Designs	599
22.2	Konzept des Modularen Designs	600
22.3	Umsetzung des Modularen Designs in C	606
22.4	Realisierung eines Stacks mit Modularem Design in C	611
22.5	Übungsaufgaben	619
23	Multithreading	621
23.1	Betriebssystemprozesse und Threads	621
23.2	Verwendung der Standard-Bibliothek <threads.h>	626
23.3	Synchronisation von Threads	633
23.4	Deadlocks, Livelocks und Starvation	651
23.5	Fortgeschrittene Programmierung mit Threads	653
A	Standardbibliotheksfunktionen	665
A.1	Klassifizierung und Konvertierung von Zeichen (ctype.h)	666
A.2	Mathematische Funktionen (math.h)	667
A.3	Ein- und Ausgabe (stdio.h)	668
A.4	Allgemeine Funktionen (stdlib.h)	671
A.5	String- und Speicherbearbeitung (string.h)	672
A.6	Datum und Uhrzeit (time.h)	673
B	Low-Level-Dateizugriffsfunktionen	675
B.1	Operationen auf dem Dateisystem	676
B.2	Ein- und Ausgabe, Positionieren innerhalb einer Datei	678
B.3	Beispiel zur Dateibearbeitung mit Low-Level-Funktionen	680
C	Wandlungen zwischen Zahlensystemen	683
C.1	Vorstellung der Zahlensysteme	683
C.2	Umwandlung von Dezimal in Binär/Hexadezimal	685
C.3	Umwandlung von Binär in Hexadezimal und umgekehrt	687
D	ASCII und Unicode	689
D.1	ASCII	690
D.2	Unicode	692

E	Arithmetische Typen, Wertebereiche und Codierungen	697
E.1	Modifikatoren short und long	697
E.2	Suffixe von arithmetischen Literalen	698
E.3	Wertebereiche von Integer-Typen und Gleitpunkt-Typen	699
E.4	Zweierkomplement-Codierung	701
E.5	Gleitpunkt-Codierung	702
Literatur		705
Index		707

1 Einführung in die Programmiersprache C



Die Programmiersprache C hat in der Praxis eine sehr große Bedeutung für die hardwarenahe Programmierung. Hardwarenahe Programmierung bedeutet unter anderem, dass die Programmiersprache direkte Zugriffe auf den Arbeitsspeicher des Rechners (Computers), auf dem das Programm läuft, und auch auf Register des Prozessors gestatten muss. Damit ist C ideal für die Systemprogrammierung, bei der betriebssystemnah beziehungsweise hardwarenah programmiert wird. Wegen seiner Hardwarenähe hat C auch für die Programmierung von eingebetteten Systemen – wie beispielsweise den Steuergeräten eines Kraftfahrzeugs – eine herausragende Bedeutung. Die Programmiersprache C liegt schon seit langem in einer von ANSI/ISO standardisierten Form vor.

Das Ziel dieses Kapitels ist es, ein erstes Programmierbeispiel zu zeigen, danach jedoch sogleich auf die Entwicklung und die Eigenschaften der Programmiersprache C einzugehen, die Sprache mit anderen Programmiersprachen zu vergleichen sowie den Stand der Standardisierung von C aufzuzeigen.

Diejenigen Personen, welche so schnell wie möglich mit praktischen Programmierbeispielen beginnen wollen, können die Kapitel über die Entwicklung, andere Sprachen sowie die Standardisierung getrost überspringen und zu einem späteren Zeitpunkt wieder hervorholen.

Ergänzende Information Die elektronische Version dieses Kapitels enthält Zusatzmaterial, auf das über folgenden Link zugegriffen werden kann https://doi.org/10.1007/978-3-658-45209-4_1.

© Der/die Autor(en), exklusiv lizenziert an
Springer Fachmedien Wiesbaden GmbH, ein Teil von Springer Nature 2024
J. Goll und T. Stamm, *C als erste Programmiersprache*,
https://doi.org/10.1007/978-3-658-45209-4_1

1.1 Hello World

Programmieren bedeutet, dem Computer Anweisungen zu geben, um ein bestimmtes Problem zu lösen. Diese Anweisungen werden dem Computer mittels einer Programmiersprache verständlich gemacht, was als „**Implementation**“ bezeichnet wird. Implementieren bedeutet Realisieren, Umsetzen, Verwirklichen.

Wie in der Programmiersprache C Anweisungen geschrieben werden, wird hier anhand des weltberühmten „Hello World“-Programms von Kernighan und Ritchie, den Vätern der Programmiersprache C [KR78], gezeigt.

Dieses einfache Programm wird in vielen Programmiersprachen als erstes Beispiel angegeben. Es weist den Computer an, einfach nur den Text Hello, world! auf dem Bildschirm auszugeben. In der Programmiersprache C sieht das Programm folgendermaßen aus:

helloWorld.c

```
#include <stdio.h>

int main() {
    printf("Hello, world!");
    return 0;
}
```

Das „Hello World“-Programm in C besteht aus einer `#include`-Zeile, welche später erklärt wird, sowie einer sogenannten „Funktion“, welche den Namen `main()` trägt und die eigentlichen Anweisungen beinhaltet, welche ausgeführt werden sollen.

In der Programmiersprache C werden Anweisungen in Funktionen geschrieben.



In diesem Buch werden Funktionen, welche im Text auftauchen, immer mit einem leeren Paar runder Klammern versehen, um sie besser kenntlich zu machen. Hier im Beispiel wird die Funktion `main()` betrachtet.

In C muss jedes Programm eine `main()`-Funktion besitzen.



Wenn ein Programm ausgeführt wird, wird mit der Ausführung der `main()`-Funktion begonnen. Die `main()`-Funktion wird auch als Startpunkt eines Programms bezeichnet. Ein Programm darf nur eine einzige `main()`-Funktion besitzen.

Innerhalb der geschweiften Klammern `{ }` werden der Reihe nach die Anweisungen der Funktion `main()` notiert. Unter dem Begriff „**Anweisung**“ wird in C ein mit einem Semikolon `;` abgeschlossener Ausdruck verstanden, welcher verschiedene Sprachelemente eindeutig miteinander verknüpft. Geschweifte Klammern und Anweisungen werden in Kapitel [2](#) ausführlicher behandelt.

Während das Programm läuft, werden die Anweisungen nacheinander, also sequenziell hintereinander, ausgeführt. Es ist üblich, mit einer neuen Anweisung in einer neuen Zeile zu beginnen.

Im Falle des „Hello World“-Programms wird als erste Anweisung die Funktion `printf()` aufgerufen. Der Name der Funktion `printf()` steht für „print formatted“. Diese Funktion schreibt Hello, world! auf den Bildschirm. Mit der zweiten Anweisung, der `return`-Anweisung, wird die Funktion `main()` auch schon wieder beendet.

Hier wird bereits ein wichtiges Konzept von C sichtbar: Funktionen können wiederum Funktionen aufrufen.

Alle Programme in C basieren von ihrem Aufbau her komplett auf Funktionen.



1.1.1 Schreiben, Übersetzen und Ausführen eines Programmes

Das oben stehende Programm wird in eine Textdatei geschrieben. Hierfür kann ein einfacher Text-Editor oder eine **Entwicklungsumgebung** (auf Englisch „Integrated Development Environment“, kurz **IDE**) verwendet werden. Die Datei wird unter dem Dateinamen `hello.c` gespeichert.

Beim Eintippen des Programms muss auf die Groß- und Kleinschreibung geachtet werden, da in C zwischen Groß- und Kleinbuchstaben unterschieden wird.



Um das Programm auszuführen, muss es zunächst mittels eines sogenannten „Compilers“ und „Linkers“ in ein lauffähiges Programm umgewandelt werden. Ein **Compiler** ist ein Programm, welches Programme aus einer Sprache in eine andere Sprache übersetzt. Ein C-Compiler übersetzt beispielsweise ein in C geschriebenes Programm in Anweisungen der sogenannten „**Maschinensprache**“, die der Prozessor eines Computers direkt versteht. Genauer wird in Kapitel [→ 5.1](#) behandelt.

Ein **Linker** verknüpft ein übersetztes Programm mit bereits existierenden übersetzten Programmteilen. Genauer kann in Kapitel [→ 5.2](#) nachgelesen werden. Auch für das „Hello World“-Programm wird der Linker benötigt:

Die Programmiersprache C selbst hat keine eigenen Funktionen für die Ein- und Ausgabe. Hierfür wird die Funktion einer sogenannten „**Bibliothek**“ verwendet. Bibliotheken sind vorgefertigte Sammlungen von Funktionen und weiteren benötigten Programmierelementen. Um Bibliotheksfunktionen nutzen zu können, muss in C eine passende „Schnittstelle“ mittels einer **#include**-Angabe eingebunden, sprich bekannt gemacht werden.

Hier im Beispiel des „Hello World“-Programmes wird die Funktion `printf()` benötigt. Damit das Programm `hello.c` somit compiliert und gelinkt werden kann, muss die passende Schnittstelle mittels `#include <stdio.h>` eingebunden werden.

Durch das Einbinden einer Bibliothek werden normalerweise eine Vielzahl an Bibliotheksfunktionen bereitgestellt. Auf die Funktionen der `<stdio.h>`-Bibliothek wird im Detail in Kapitel [→ 16](#) eingegangen.

Compilieren und Linken erfolgt bei den heute gängigen Compilern durch einen einzigen Aufruf des Compilers. Wenn Sie mit einer IDE arbeiten, gibt es entsprechende Menübefehle oder anderweitige Bedienelemente, um eine Compilation zu starten. Wenn Sie mit einer **Konsole** (auch „Terminal“, „Eingabeaufforderung“, „Shell“, „Prompt“, „Command Line“ oder „Kommandozeile“ genannt) arbeiten, können Sie mit einem der folgenden Befehle das Programm compilieren. Jede Zeile steht hierbei für einen anderen Compiler.

```
cl hello.c -o hello.exe
gcc hello.c -o hello.exe
clang hello.c -o hello.exe
```

Hier bedeutet `-o hello.exe`, dass das ausführbare Programm `hello.exe` als Output erzeugt werden soll. Unter Unix würde die `.exe` Endung entfallen. Alle Compiler erlauben eine Vielzahl an erweiterten Optionen, auf welche in diesem Buch jedoch nicht eingegangen wird.

Durch Aufruf von `hello.exe` – beziehungsweise unter Unix `hello` – kann das Programm einfach gestartet werden. Wenn Sie mit einer IDE arbeiten, können Sie das Programm wiederum ganz einfach durch Menübefehle oder ein anderes Bedienelement starten.

Nach dem Starten von `hello.exe` wird die gewünschte Ausgabe auf dem Bildschirm angezeigt:

```
Hello, world!
```

Aus der Ausgabe ist ersichtlich, dass die Anführungszeichen `"` nicht mit ausgegeben werden. Sie dienen nur dazu, den Anfang und das Ende eines sogenannten „**Strings**“ (siehe Kapitel [→ 6.5.4](#)) zu markieren. Ein String wird im Deutschen auch als **Zeichenkette** bezeichnet und beschreibt wörtlich eine Hintereinanderreihung von einzelnen Zeichen.

1.2 Die Entwicklung von C

Das Betriebssystem UNIX wurde bei den Bell Laboratorien der Fa. AT&T in den USA entwickelt. Die erste Version von UNIX lief auf einer PDP-7, einem Rechner der Fa. DEC. Diese Version war zunächst in Assembler geschrieben. Assembler ist grundsätzlich die direkte Umsetzung von Maschinensprache in eine für den Menschen verständliche Sprache. Um das System auf einen anderen Rechner portieren zu können, hätte es somit in der für den neuen Rechner gültigen Maschinensprache komplett neu geschrieben werden müssen.

Um das neue Betriebssystem somit einfacher portieren zu können, sollte es in einer sogenannten „höheren Programmiersprache“ neu geschrieben werden. Es sollte aber dennoch sehr schnell sein und Zugriffe auf die Hardware eines Rechners wie den Arbeitsspeicher und Register gestatten.

Eine höhere Programmiersprache zeichnet sich durch eine Expressivität der Sprache aus, welche – mehr als nur einem vorgegebenen Formalismus (**Syntax**) gerecht zu werden – direkt auf die Bedeutung (**Semantik**) des Codes rück-schließen lässt. Eine höhere Programmiersprache verwendet Sprachkonstrukte, welche für Menschen intuitiv erscheinen. So steht beispielsweise in C das englische Wort *while* für eine Iteration (Schleife) und das Wort *if* für eine Selektion (Auswahl).

Durch die höhere Expressivität ergibt sich konsequenterweise eine höhere Produktivität und auch weniger Aufwand für die Fehlersuche, da ein solches Programm rascher verstanden wird.

Ebenfalls eine Eigenschaft höherer Programmiersprachen ist die Anwendung erweiterter „Paradigmen“, die über das bloße Abarbeiten von sequenziellen Anweisungen – wie es von Assembler her bekannt ist – hinaus geht.

Durch die gesammelten Erfahrungen mit anderen Programmiersprachen kristallisierten sich zur Zeit der Entwicklung von C zwei Paradigmen heraus: Die sogenannte „Strukturierte Programmierung“ sowie die „Prozedurale Programmierung“. Spätere, noch modernere Sprachen bauten dann auf diesen Konzepten auf und haben „objektorientierte“ und „funktionale“ Paradigmen entwickelt, wie sie heute bekannt sind.

Gesucht war zu Beginn der Entwicklung von C also eine neue Programmiersprache von der Art eines „Super-Assemblers“, der in Form einer höheren Programmiersprache die folgenden Ziele erfüllen sollte:

- Möglichkeiten einer hardwarenahen Programmierung vergleichbar mit Assembler.
- Eine Performance des Laufzeitcodes vergleichbar mit Assembler.
- Eine Unterstützung der Sprachmittel der Strukturierten Programmierung → 1.2.2 sowie der Prozeduralen Programmierung → 4.3.
- Die Möglichkeiten einer maschinenspezifischen Implementierung.

Da eine Programmiersprache aber nicht nur ein Mittel zum Zweck sein sollte, sondern auch ein praktisches und mächtiges Werkzeug, wurden zudem folgende qualitative Eigenschaften für den Charakter der Programmiersprache C definiert [JTC03]:

- Trust the programmer.
- Don't prevent the programmer from doing what needs to be done.
- Keep the language small and simple.
- Provide only one way to do an operation.
- Make it fast, even if it is not guaranteed to be portable.

Eine solche Programmiersprache stand damals noch nicht zur Verfügung. Deshalb entwarf und implementierte Thompson, einer der Väter von UNIX, die Programmiersprache B, beeinflusst von der Programmiersprache BCPL. B musste interpretiert werden und war langsam. Um diese Schwächen zu beseitigen, entwickelte Ritchie 1971/72 die Programmiersprache B zu C weiter. Die Programmiersprache C musste nicht interpretiert werden, sondern für C-Programme konnte von einem Compiler effizienter Code erzeugt werden. Im Jahre 1973 wurde UNIX dann neu in C realisiert, nur rund 1/10 blieb in Assembler geschrieben.

Die Sprache C wurde dann in mehreren Schritten von Kernighan und Ritchie festgelegt. Mit dem Abschluss ihrer Arbeiten erschien 1978 das lange Zeit als „Sprach-Bibel“ betrachtete grundlegende Werk „The C Programming Language“ [KR78]. Mit der Verbreitung von Unix nahm dann C einen rasanten Aufstieg.

In den folgenden Unterkapiteln werden nun die Eigenschaften von C behandelt:

- Hardwarenahe Programmierung
- Strukturierte und Prozedurale Programmierung
- Abstraktion

1.2.1 Hardwarenahe Programmierung

C ist eine relativ „maschinennahe“ Sprache. Das bedeutet, dass mit C der Prozessor angewiesen wird, Daten aus dem Arbeitsspeicher zu holen, zu verarbeiten und sie wieder dort zu speichern.

Die Speicherzellen des Arbeitsspeichers sind durchnummeriert. Die Nummern der Speicherzellen werden **Adressen** genannt.



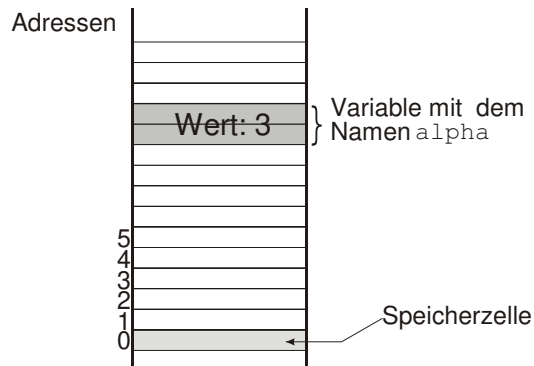
Die Sprache C erlaubt eine **hardwarenahe Programmierung** unter anderem durch direkte Zugriffe auf Adressen im Arbeitsspeicher.



C arbeitet somit mit denselben Objekten wie der Prozessor, nämlich mit Zahlen und Speicheradressen.

In der Regel ist eine Speicherzelle 1 **Byte** groß. Ein Byte stellt in der Regel eine Folge von 8 zusammengehörigen **Bits** dar.

Eine **Variable** ist ein Bereich im Arbeitsspeicher, der über einen Namen angesprochen werden kann. Eine Variable kann auch mehrere Speicherzellen einnehmen. Die Adresse der Variablen ist dabei die Adresse der Speicherzelle, in der die Variable beginnt:



Eine Variable besitzt stets einen sogenannten „**Typ**“, welcher definiert, wieviele Speicherzellen er belegt. Auf das Typkonzept von C wird in Kapitel [→ 7](#) genauer eingegangen, hier jedoch eine kurze Auflistung der Basis-Typen, welche in der Programmiersprache C verfügbar sind:

- Integer-Typen
- Gleitpunkt-Typen
- Aufzählungs-Typen
- Strukturen
- Unionen
- Arrays
- Bitfelder
- Adressen

Was scheinbar fehlt sind Zeichen, Strings, sowie boolesche Variablen. Zeichen werden in der Programmiersprache C ebenfalls als Zahlen gespeichert. Strings sind eine Hintereinanderreihung von mehreren solcher Zeichen, was in C mittels eines Arrays passiert (siehe Kapitel [→ 6.5.4](#)). Boolesche Variablen mit den Wahrheitswerten `false` und `true` gab es im ursprünglichen Sprachkern von C nicht, sondern wurden mit den Zahlen 0 und 1 abgebildet. Für Operationen auf einzelnen Bits innerhalb eines Bytes gibt es in C spezielle Operatoren. Auch die Aufzählungs-Konstanten der sogenannten Aufzählungs-Typen entsprechen in C Integer-Werten.

C-Compiler unterstützen zudem oftmals auch den Zugriff auf Hardware-Register entweder durch spezifische Funktionen, wie beispielsweise durch die Bibliotheksfunktionen `_inp()` und `_outp()` beim Visual C++ Compiler oder durch die Anweisung `asm`. Diese Funktionen wie auch die `asm`-Anweisungen werden jedoch je nach Compiler, Dialekt und System anders verwendet und sind nicht Teil dieses Buches.

1.2.2 Strukturierte und Prozedurale Programmierung

Strukturierte Programmierung erlaubt es, anstelle eines rein sequenziellen Ablaufs von unzähligen Anweisungen einen intuitiven Kontrollfluss zu ermöglichen.



C nutzt hierfür Kontrollstrukturen wie `if`, `while` und `for`. Diese werden im Kapitel [→ 10](#) ausführlich behandelt.

Wilde Sprünge im Programm sind bei der Strukturierten Programmierung nicht erlaubt. C enthält aber als hardwarenahe Sprache noch die Sprunganweisung `goto` [→ 10.11](#).

Es ist möglich, dass der in C geschriebene **Quellcode** (auch „Quelltext“, oder auf Englisch **source code** genannt) eines Programms aus mehreren Dateien bestehen kann. Jede dieser Dateien kann getrennt in Maschinensprache übersetzt werden. Dabei ist Maschinensprache eine Sprache, die ein bestimmter Prozessor versteht.



Der Aufbau eines Programms aus getrennt compilierbaren Dateien, den Modulen, wird in Kapitel [→ 15](#) unter dem Thema „Speicherklassen“ behandelt.

Die separate Compilierung der verschiedenen Module bietet große Vorteile, zum einen für die Verwaltung der Programme, zum anderen für den Vorgang des Compilierens:

Bei Änderungen im Code muss gegebenenfalls nur eine einzige Datei als kleiner Baustein des Programms geändert werden. Die anderen Dateien bleiben in diesem Fall stabil. Bei komplexen Programmen kann der Compilierlauf des gesamten Programms wesentlich länger als der Compilierlauf einzelner Dateien dauern. Es ist also günstig, wenn nur einzelne Dateien neu compiliert werden müssen.

Prozedurale Programmierung ermöglicht es, Teile des Codes in kleinere Einheiten zu verpacken. In C werden hierfür die sogenannten „**Funktionen**“ verwendet.



Durch Hinzufügen von Parametern zu solchen Funktionen können beliebige „komplexe Anweisungen“ programmiert werden, welche sodann an beliebiger Stelle aufgerufen werden können. Durch die Benennung solcher Funktionen mit eingängigen Namen wird zudem der Code lesbarer.

Eine prozedurale Sprache wie C stellt Techniken für die Definition von Funktionen und deren Parametrisierung sowie für den Aufruf von Funktionen, die Argumentübergabe und die Rückgabe von Ergebnissen bereit.



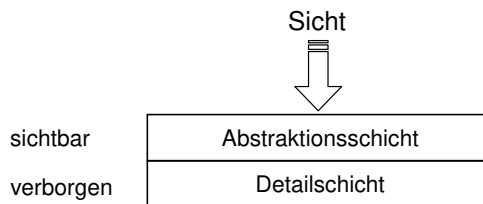
Die Konzepte der Prozeduralen Programmierung werden im Detail in Kapitel [→ 4.3](#) abgehandelt.

1.3 Abstraktion

Abstraktion bedeutet grundsätzlich, dass bei komplexen Sachverhalten unwesentliche Dinge unsichtbar gemacht werden, sodass nur das Wesentliche sichtbar bleibt.



Das symbolisiert das folgende Bild:



Durch die allmähliche Entwicklung weg von der rein elektronischen Datenverarbeitung (EDV) hin zur Informationsverarbeitung (auf Englisch „Information Technology“, oder kurz IT) mussten somit Abstraktionskonzepte auch auf Programmiersprachen angewendet werden.

Bei der Entwicklung der Programmiersprachen kann im Nachhinein festgestellt werden, dass es drei große Fortschritte im Abstraktionsgrad gab [Bar83]:

- Abstraktion bei Ausdrücken
- Abstraktion bei Kontrollstrukturen
- Abstraktion von Daten

1.3.1 Abstraktion bei Ausdrücken

Allgemein ist ein **Ausdruck** eine Verknüpfung von Operanden, Operatoren und runden Klammern wie beispielsweise $3 * (4 + 7)$.

Den ersten Fortschritt in der Abstraktion von Ausdrücken brachte die Programmiersprache FORTRAN (**FOR**mula **TRAN**slation). Die Programmiersprache C hat diese Eigenschaften übernommen.

Während in Assembler für komplexere Ausdrücke mehrere Instruktionen geschrieben werden müssen, welche zudem direkt auf die Maschinenregister zugreifen, können in C direkt mathematische Ausdrücke wie beispielsweise $3 * (4 + 7)$ hingeschrieben werden.



Die Umsetzung auf die Maschinenregister wird durch den Compiler vorgenommen und bleibt vollständig verborgen.

1.3.2 Abstraktion bei Kontrollstrukturen

Unter einer **Kontrollstruktur** wird die Beeinflussung der Abarbeitungsreihenfolge von Anweisungen aufgrund von Bedingungen verstanden.



Unter Assembler und selbst FORTRAN mussten noch manuell Sprungmarken definiert werden, an welche der Prozessor springen soll:

```
      IF (A-B) 100, 200, 300
100   ...
      GOTO 400
200   ...
      GOTO 400
300   ...
400   ...
```

Mit der Programmiersprache ALGOL 60 (**ALGO**rithmic **L**anguage **60**) wurde zum ersten Mal die Iteration und Selektion in abstrakter Form zur Verfügung gestellt, ohne dass einzelne Punkte im Programmablauf mit Marken benannt und dorthin gesprungen werden musste. Die Programmiersprache C hat diese Eigenschaft übernommen, sodass derselbe Ausdruck in C folgendermaßen aussieht:

```
if (A - B < 0) {  
    ...  
} else if (A - B == 0) {  
    ...  
} else {  
    ...  
}
```

1.3.3 Datenabstraktion

Ein großer Fortschritt in der Geschichte der Programmiersprachen war das Konzept der Datenabstraktion.

Mit dem Konzept der **Datenabstraktion** wurde das Ziel verfolgt, die Einzelheiten der Datendarstellung von den Beschreibungen der Operationen auf den Daten zu trennen, um eine gesteigerte Übertragbarkeit und Wartbarkeit sowie eine höhere Sicherheit zu erreichen. Bei diesem Konzept bleibt die Darstellung der Daten als Bits und Bytes verborgen, bekannt sind nur die Operationen zum Zugriff und zur Manipulation der Daten.



In der Programmiersprache C wird diese Abstraktion durch die Verwendung eines klaren Typsystems unterstützt. Jede Variable und selbst jede Funktion besitzt einen eindeutigen, statischen Typ, welcher vom Compiler ermittelt wird. Eine **Signatur** gibt einer Funktion einen Namen und definiert die Übergabeparameter. Welche Operationen mit welchen Daten und welchen Funktionen möglich sind, ist somit durch die Vergabe der Typen definiert. Weicht irgend eine Benutzung einer Variablen oder Funktion von der durch die Sprache vorgegebenen Typregeln ab, so meldet der Compiler einen Fehler.

Die Nutzung ausführlicher und verständlicher Bezeichner (Namen) von zu verarbeitenden Daten und aufzurufenden Funktionen lässt zusätzlich auf deren Bedeutung schließen. So muss man sich in C nicht mehr um die korrekte Speicherung einer Variablen an den entsprechenden Speicherzellen kümmern, sondern kann einfach sie mit ihrem Namen ansprechen oder etwas ihr zuweisen.

Eine Variable mit dem Namen `gewicht` beispielsweise lässt bereits errahnen, dass in dieser Variablen ein Gewicht abgespeichert wird. In der modernen Programmierung sind nicht-aussagekräftige Namen unerwünscht. Eine Ausnahme von dieser Regelung bilden Bezeichner wie `i` und `j`, die häufig als einfache Zählvariablen für einen Schleifenindex verwendet werden.

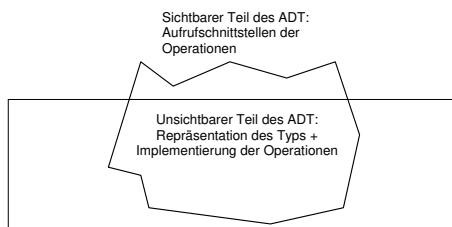
Auch ermöglicht es C, Daten mittels Strukturen zu größeren Informationseinheiten zu verkapseln, um sie so geordnet ansprechen zu können. Die Verwendung des strukturierten Typs wird in Kapitel [13.1](#) besprochen.

Das grundlegende Konzept der Datenabstraktion wird durch den Begriff des sogenannten „abstrakten Datentyps“ verkörpert.

Ein abstrakter Datentyp (ADT) basiert auf der Formulierung abstrakter Operationen auf abstrakt beschriebenen Daten. Um die erlaubten Operationen zu spezifizieren, benötigen sie jeweils eine Signatur und eine Semantik (Bedeutung).



Bertrand Meyer [Mey97] symbolisiert einen abstrakten Datentyp durch einen Eisberg, von dem nur der Teil über Wasser – sprich die Aufrufchnittstellen der Operationen – sichtbar ist. „Unter Wasser“ und damit im Verborgenen liegen die Repräsentation der Daten und die Implementierung der Operationen:



C unterstützt dieses Konzept nicht vollumfänglich. Erst das Klassenkonzept der objektorientierten Programmiersprachen wie beispielsweise C++ ermöglichte es, dass Daten und die Operationen, die mit diesen Daten arbeiten, zu einem gemeinsamen Datentyp – der Klasse – zusammengefasst werden können.

1.4 Vorgänger und Nachfolger von C

C wird zu den **imperativen Sprachen** gezählt.



Imperative Sprachen sind geprägt durch die von-Neumann-Architektur eines Rechners, bei der Befehle im Speicher die Daten im gleichen Speicher bearbeiten.

Bei imperativen Sprachen besteht ein **Programm** aus Variablen, die Speicherstellen darstellen, und einer Folge von Befehlen, die Daten verarbeiten.



Der **Algorithmus** ist bei den imperativen Programmiersprachen der zentrale Ansatzpunkt. Ein Algorithmus ist eine eindeutige Handlungsvorschrift zur Lösung eines Problems. Die Verarbeitungsschritte und ihre Reihenfolge müssen also im Detail festgelegt werden, um zu einem gewünschten Ergebnis zu gelangen.

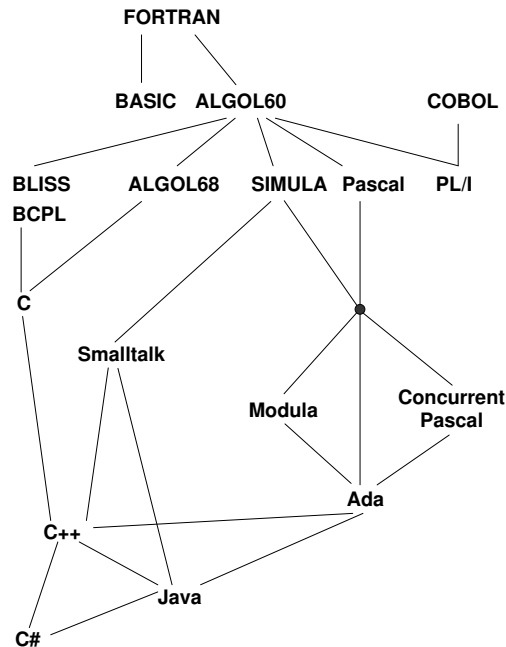
Weitere Beispiele für imperative Sprachen neben C sind FORTRAN, COBOL und Pascal, aber auch Java und C#.

Im Gegensatz dazu stehen die „deklarativen Sprachen“. Bei ihnen werden nicht mehr die Verarbeitungsschritte angegeben, sondern das gewünschte Ergebnis wird direkt beschrieben, also „deklariert“. Ein Übersetzer beziehungsweise Interpreter muss daraus die Verarbeitungsschritte ableiten. Zu den deklarativen Sprachen zählen sogenannte „funktionale Sprachen“ wie zum Beispiel LISP, sogenannte „logikbasierte Sprachen“ wie PROLOG und sogenannte „regelbasierte Sprachen“ wie OPS5.

Bei den imperativen Sprachen werden folgende Sprachen unterschieden:

- Maschinenorientierten Sprachen wie Assembler.
- Prozeduralen Sprachen wie FORTRAN, ALGOL, Pascal, C.
- Objektorientierten Sprachen wie Smalltalk, Eiffel, C++, Java und C#.

Das folgende Bild zeigt einen Stammbaum imperativer Programmiersprachen:



Anzumerken ist, dass C zwar schon etwas älter ist, dass aber viele moderne objektorientierten Programmiersprachen wie C++, Java und C# in ihrer Syntax auf C basieren, wobei allerdings das Konzept der Objektorientierung eine neue Dimension darstellt.

1.4.1 Die Sprache C++

Während C als „Super-Assembler“ für hardwarenahe Software entwickelt wurde, liegt die Zielrichtung bei der Entwicklung von **C++** darin, neue Sprachmittel wie beispielsweise Klassen bereitzustellen, um Anwendungsprobleme noch intuitiver zu formulieren.



C++ baut auf dem Funktionsumfang von C auf und erweitert diesen um das Paradigma der „objektorientierten Programmierung“. Die grundlegende Idee hierbei ist, dass eigene Datentypen definiert werden können, für welche genau festgelegt werden kann, welche Operationen zulässig sind.

C++ gilt als der direkte Nachkomme von C und hat sich über die Jahre stark weiterentwickelt. Mittlerweile sind auch Konzepte der funktionalen Programmierung sowie moderner Design-Paradigmen eingeflossen.

Die fest verankerte Rückwärtskompatibilität zu C macht C++ zu einer sehr mächtigen Partnersprache von C: Während C beispielsweise für die Ansteuerung von Hardware verwendet wird, kann C++ die Präsentation mittels eines User Interfaces (GUI) übernehmen. In C geschriebene Funktionalität kann direkt aus einem Programm geschrieben in C++ aufgerufen werden. Mit anderen Sprachen ist diese Verknüpfung nicht so einfach möglich.

1.5 Standardisierung von C

Wie bei UNIX selbst, so entwickelten sich anfänglich auch bei C-Compilern viele verschiedenartige Dialekte, was zu einer erheblichen Einschränkung der Portabilität von C-Programmen führte.

Im Jahre 1989 wurde C durch das ANSI-Komitee X3J11 normiert mit dem Ziel, die Portabilität von C zu ermöglichen. Programme, die nach **ANSI-C** (X3.159-1989) geschrieben wurden, konnten von jedem Compiler auf jedem Rechner kompiliert werden, vorausgesetzt, der Compiler war ein ANSI-C-Compiler oder enthielt ANSI-C als Teilmenge.

Der ANSI-Standard normierte nicht nur die Sprache C, sondern auch die Standardbibliotheken, ohne die C nicht auskommt, da C selbst – wie bereits erwähnt – beispielsweise keine Einrichtungen für die Ein- und Ausgabe hat.

Bibliotheken sind im Unterschied zu Programmen nicht selbstständig ablauffähig. Bibliotheken enthalten Hilfsmodule, welche von Programmen angefordert werden können.



In der Folgezeit wurde der Standard ANSI X3.159-1989 mit kleinen Änderungen von der ISO als internationaler Standard ISO/IEC 9899 [C90] übernommen. Diese Sprachversion wird nach dem Veröffentlichungsjahr **C90** genannt. C90 wird bis heute von fast allen Compilern unterstützt und ist in der Industrie immer noch verbreitet.

1995 erschien die erste Erweiterung zur C-Norm. Der unter dem Namen ISO/IEC 9899/AMD1:1995 (oder auch **C95** genannt) veröffentlichte Standard enthielt neben Fehlerbehebungen nur kleine Änderungen am Sprachumfang.

In einer weiteren Überarbeitung wurden auch Sprachkonzepte aus C++ übernommen. Der Standard ISO/IEC 9899 [C99] erschien 1999 und wird als **C99** bezeichnet.

C11 ist der informelle Name des im Dezember 2011 veröffentlichten ISO-Standards ISO/IEC 9899:2011 [C11] für die Programmiersprache C. Er ersetzt den vorigen Standard C99.

Der Standard **C17** gilt als ein Zwischenstandard, welcher kleinere Fehler des C11 Standards korrigierte. Da er selbst jedoch keine neuen Spracheigenschaften einführte, wird umgangssprachlich immer noch C11 als der aktuelle Standard bezeichnet.

Zum Zeitpunkt des Schreibens dieses Buches ist ein neuer Standard in Bearbeitung, welcher voraussichtlich mit **C23** benannt werden wird. Die wichtigsten neuen Spracheigenschaften werden in diesem Buch bereits beschrieben.

Ein neuer C-Standard ersetzt immer seinen Vorgänger. Es kann zur selben Zeit immer nur einen einzigen gültigen Standard geben.



Die grundsätzlichen Unterschiede zwischen den Standards werden in diesem Buch nicht beschrieben. Stattdessen werden nur vereinzelt Besonderheiten erwähnt werden, die für das Studium der Sprache interessant sind.

2 Einfache Beispielprogramme



Mit dem „Hello World“-Programm in Kapitel [→ 1.1](#) wurden bereits erste Erfahrungen im Programmieren gesammelt. Programmieren kann viel Spaß bereiten. Außerdem funktioniert Lernen iterativ. Im Folgenden sollen deshalb weitere kurze und aussagekräftige Programme vorgestellt werden, damit Sie sich spielerisch voranarbeiten, um dann auch Augen und Ohren für die erforderliche Theorie zu haben. Alle Programme des Buches finden sich auch im begleitenden Webaufttritt, sodass Sie die Programme nicht abzutippen brauchen.

<https://link.springer.com/>

Es ist in C möglich, bereits mit wenigen einfachen Mitteln sinnvolle Programme zu schreiben. Als Einstieg sollen in den folgenden Kapiteln einfache Programmbeispiele vorgestellt werden, um mit Konstanten, Variablen, Funktionen, Ausdrücken, Schleifen und der Ein- und Ausgabe in C vertraut zu werden.

2.1 Quadratzahlen

Es soll ein C-Programm geschrieben werden, das die ersten zehn ganzzahligen Quadratzahlen auf der Konsole ausgibt:

square.c

```
#include <stdio.h>

int main(void) {
    int max = 10;

    printf("Die ersten %d ganzzahligen Quadratzahlen sind:\n", max);

    int zahl = 1;
    while (zahl <= max) {
        printf("%d ", zahl * zahl);
        zahl = zahl + 1;
    }

    return 0;
}
```

Ergänzende Information Die elektronische Version dieses Kapitels enthält Zusatzmaterial, auf das über folgenden Link zugegriffen werden kann https://doi.org/10.1007/978-3-658-45209-4_2.

Dieses Programm gibt aus:

```
Die ersten 10 ganzzahligen Quadratzahlen sind:  
1 4 9 16 25 36 49 64 81 100
```

Kurz zusammengefasst passiert in diesem Programm das Folgende:

Zuerst wird genauso wie im „Hello World“ Beispiel aus Kapitel [→ 1.1](#) die Bibliothek `<stdio.h>` eingebunden, womit die Ausgabe auf der Konsole ermöglicht wird. Dann beginnt das Hauptprogramm mit der Funktion `main()`, welche als erstes den Wert `max` definiert. Dann wird ein Text auf der Konsole ausgegeben. Danach startet eine Schleife, innerhalb welcher eine Zahl sukzessive von 1 bis `max` hochgezählt und der quadratische Wert dieser Zahl auf der Konsole ausgegeben wird. Nach Ablauf der Schleife wird das Programm beendet.

Anhand dieses einfachen Beispiels werden hier nun einige grundlegende Elemente der Sprache C erläutert:

2.1.1 Leerzeichen und Leerzeilen

Leerzeichen wie Space und Tab, sowie Leerzeilen mit Enter werden im Englischen als „**whitespace characters**“ bezeichnet und haben keine spezielle Bedeutung in C. Sie können aber dazu benutzt werden, ein Programm optisch zu gliedern.

Die Verwendung von Leerzeichen zur Gliederung des Codes am linken Rand wird „**Einrückung**“, oder auf Englisch „**indentation**“, genannt. Üblich sind 2 oder 4 Leerzeichen pro Einrückungsebene.

Leere Zeilen werden üblicherweise einfach für mehr Abstand zwischen zusammengehörigen Codeteilen verwendet.

2.1.2 Variablen, Definitionen und Anweisungen

Damit ein Programm Berechnungen ausführen kann, benötigt es zum einen einen Platz, um Werte abzurufen und Resultate zu speichern, zum anderen benötigt es Anweisungen, was genau berechnet werden soll.

Werte finden in C in sogenannten „**Variablen**“ Platz, welche mittels **Definitionen** erstellt werden. Im obigen Beispiel wird die Variable `max` als Ganzzahl definiert. In der Programmierung wird eine **Ganzzahl** als ein sogenannter „**Integer**“ (`int`) bezeichnet. Der Integer `max` wird sodann auch gleich mit dem Wert `10` initialisiert:

```
int max = 10;
```

Danach folgen **Anweisungen** wie die Ausgabe auf dem Bildschirm und die Durchführung der Schleife.

Da Leerzeichen in C keine spezielle Bedeutung haben, benötigt die Sprache ein Trennzeichen, um das Ende einer Anweisung oder einer Definition anzuzeigen. Diese Aufgabe übernimmt in der Sprache C das **Semikolon** (zu Deutsch Strichpunkt).

Eine Anweisung oder eine Definition wird mittels eines Semikolons ; abgeschlossen.



Beachten Sie, dass das Zeichen `=` in der Programmiersprache C ein Zuweisungs-Operator ist, sprich der Wert auf der rechten Seite des Gleichheitszeichens wird der Variable auf der linken Seite zugewiesen. Für den Vergleichs-Operator „ist gleich“ muss in C die Notation `==` verwendet werden.

2.1.3 while-Schleife

Im Beispiel wird die folgende Schleife ausgeführt:

```
int zahl = 1;
while (zahl <= max) {
    printf("%d ", zahl * zahl);
    zahl += 1;
}
```

Umgangssprachlich könnte dies in etwa so ausgedrückt werden:

„Setze vor dem ersten Durchlauf der Schleife die Variable `zahl` auf den Wert 1. Berechne innerhalb der Schleife das Quadrat von `zahl` und gib es auf der Konsole aus. Danach wird `zahl` um 1 erhöht. Führe dies solange durch, wie die Zahl kleiner oder gleich ist wie `max`.“

Da bei dieser Schleife mehrere Anweisungen während eines Durchlaufs ausgeführt werden müssen, werden geschweifte Klammern benutzt.

2.1.4 Blöcke und geschweifte Klammern {}

Blöcke werden im Detail im Kapitel [→ 10.1](#) behandelt. Hier genügt es zu sagen, dass mittels geschweiffter Klammern mehrere Anweisungen und Definitionen zusammengefasst werden können.

Geschweifte Klammern umschließen in C einen sogenannten **Block**, der die Definition beliebig vieler lokaler Variablen und Anweisungen beinhalten kann. Lokale Variablen sind nur innerhalb des umschließenden Blocks gültig.



So werden im obigen Beispiel die geschweiften Klammern nicht nur für die `while`-Schleife genutzt, sondern auch für die `main()` Funktion. Die Klammern der `main()` Funktion umfassen sämtliche Anweisungen und Definitionen, welche bei Aufruf ausgeführt werden sollen.

2.1.5 Die Funktion main()

Ein **Hauptprogramm** muss immer vorhanden sein. Mit dem Hauptprogramm beginnt ein Programm seine Ausführung. In C heißt das Hauptprogramm stets main().



Die Funktion **main()** wird im Detail in Kapitel [→ 17.1](#) behandelt. In diesem Kapitel wird der folgende, einfache Kopf der Funktion main() benutzt:

```
int main()
```

Der Rückgabetyt `int` der Funktion `main()` kennzeichnet, dass die Funktion `main()` einen Integer-Wert an den Aufrufer zurückgeben soll. Tatsächlich findet dies in der letzten Zeile der Funktion mittels `return 0` statt. Die Funktion gibt mit der Zahl `0` an, dass bei ihrer Abarbeitung alles glatt gegangen ist. Weitere Informationen über diesen sogenannten „Status“-Wert können in Kapitel [→ 17.2](#) nachgelesen werden.

Es ist auch möglich, beim Aufruf eines Programms Argumente an das Hauptprogramm `main()` zu übergeben. Innerhalb der runden Klammern kann eine Funktion eine Liste von **Parametern**, die sogenannte „Parameterliste“, definieren. Diese Parameter werden beim Aufruf der Funktion mit den übergebenen **Argumenten** gefüllt.

Um in C einen Name als Namen einer Funktion zu markieren, sind runde Klammern erforderlich. Diese Klammern müssen geschrieben werden, selbst wenn es keine Parameter gibt. Falls keine Parameter vorhanden sind, kann einfach die Parameterliste leergelassen werden.



Um dem Compiler explizit mitzuteilen, dass keine Parameter erwartet werden, kann auch das Schlüsselwort `void` in die Klammern geschrieben werden:

```
int main(void)
```

Das Fehlen dieser Angabe wurde bisher von vielen Compilern mit einer Warnung vermerkt. Ab dem Standard C23 ist dies jedoch unproblematisch.

2.1.6 Die Funktion printf()

Wie bereits in Kapitel [→ 1.1](#) erwähnt, bedeutet printf „print formatted“, also formatierte Ausgabe. Eine Ausgabe wird formatiert, indem in dem übergebenen String encodiert wird, welche Werte wo und auf welche Weise ausgegeben werden sollen.

Der an printf() übergebene String wird **Format-String** genannt.



Hier in diesem Beispiel treten zwei **Formatelemente** auf: Die Zeichenfolgen `\n` und `%d`.

Mittels der Zeichenfolge `\n` wird eine neue Zeile (auf Englisch **newline**) begonnen, sprich die nachfolgenden Ausgaben werden auf der nächsten Bildschirmzeile links in der Konsole stehen. Eine Zeichenfolge, welche mit einem Backslash `\` beginnt, wird auch „**Ersatzdarstellung**“ (auf Englisch „**escape sequence**“) genannt, was in Kapitel [→ 6.5.5](#) genauer besprochen werden wird.

Die Zeichenfolge `%d` veranlasst die Konsole, eine Dezimalzahl (d) auszugeben. Die gewünschte Zahl wird als zusätzliches Argument an die Funktion printf() in den runden Klammern angegeben: `zahl * zahl`. Mit Hilfe des Aufrufes der Funktion printf() wird somit die Zahl ausgegeben, die der Quadratzahl der Variablen `zahl` während des aktuellen Schleifendurchlaufs entspricht.

Beachten Sie die Reihenfolge: printf() muss als erstes Argument den Format-String und dann anschließend die anderen auszugebenden Ausdrücke erhalten. Für jedes Formatelement im Format-String muss eins-zu-eins ein entsprechender Ausdruck existieren. Dabei müssen die Formatelemente und die auszugebenden Ausdrücke im Typ übereinstimmen.



Weitere Beispiele und Erklärungen zu der Funktion printf() und den Formatelementen sind in den folgenden Beispielprogrammen zu finden. Eine ausführliche Beschreibung findet sich in Kapitel [→ 16.7](#).

2.1.7 Inkludieren von Bibliotheksfunktionen

Die Sprache C selbst besitzt keine eingebauten Funktionen für die Ein- und Ausgabe am Bildschirm. In C werden hierfür Funktionen mit standardisierten Schnittstellen in sogenannten Standardbibliotheken bereitgestellt. Die Schnittstellen der Funktionen stehen in sogenannten „**Header-Dateien**“ (auf Englisch **header files** oder kurz **h-files**). Durch Einbinden der entsprechenden Header-Datei in das eigene Programm ist es möglich, die **Ein- und Ausgabefunktionen** der Bibliotheken zu nutzen.

Mit Hilfe der **#include**-Anweisung ist es möglich, eine externe Datei für die Dauer des Übersetzungslaufs in den Quellcode zu kopieren. Eine Header-Datei enthält alle Angaben zu Funktionen, Typen und externen Variablen, welche in einer Bibliothek enthalten sind.



Der C Standard umfasst mehrere Standard-Bibliotheken wie hier im Beispiel `stdio`, in welcher sich unter anderem die Schnittstelle der Funktion `printf()` befindet. Erst mithilfe der passenden `#include`-Anweisung wird somit ein Aufruf der Funktion `printf()` überhaupt erst möglich.

Da eine Bibliotheksfunktion vor ihrem Aufruf dem Compiler bekannt gemacht werden muss, sollte man die `#include`-Anweisungen stets an den Anfang einer Quelldatei stellen.



2.2 Celsius und Fahrenheit

Folgendes Programm erzeugt eine Temperaturtabelle zur Umrechnung von der Einheit Fahrenheit in die Einheit Celsius:

fahrenheit.c

```
#include <stdio.h>

// Konstanten
#define UPPER      200
#define LOWER      0
#define STEP_SIZE  20

/* Diese Funktion gibt eine Tabelle aus,
 * in welcher Fahrenheit-Werte in die passenden
 * Celsius-Werte umgerechnet wurden.
 */
int main(void) {

    // Schleife von LOWER bis UPPER
    for (int fahr = LOWER; fahr <= UPPER; fahr = fahr + STEP_SIZE) {
        int celsius = 5 * (fahr - 32) / 9;
        printf("%3d\t%3d\n", fahr, celsius);
    }

    return 0; // Ende des Programmes
}
```

Dieses Programm gibt aus:

0	-17
20	-6
40	4
60	15
80	26
100	37
120	48
140	60
160	71
180	82
200	93

2.2.1 Kommentare

Das erste, was in diesem Programm auffällt, sind die neu hinzugefügten Kommentare.

Ein **Kommentar** dient zur Dokumentation eines Programms und hat keinen Einfluss auf dessen Ablauf. Kommentare sollten immer dann geschrieben werden, wenn ein Programm nicht selbsterklärend ist.



In C gibt es zwei Varianten von Kommentaren: Ein einzelner Kommentar und ein Kommentarblock.

Ein **einzeiliger Kommentar** wird durch zwei Slash-Zeichen `//` eingeführt. Alles, was in derselben Zeile hinter diesen beiden Zeichen steht, wird vom Compiler ignoriert.



Alles, was zwischen den Zeichenfolgen `/*` und `*/` steht, stellt einen sogenannten „**Kommentarblock**“ dar. Kommentarblöcke können über mehrere Zeilen gehen, dürfen aber nicht in einander verschachtelt werden.



Der einzeilige Kommentar wird heute in vielen Programmiersprachen bevorzugt und wird auch durch Entwicklungsumgebungen standardmäßig unterstützt. Tatsächlich war aber in C ursprünglich nur der Kommentarblock möglich. Erst seit dem Standard C99 sind einzeilige Kommentare erlaubt.

2.2.2 Makros

Die Größen LOWER, UPPER und STEP_SIZE sind sogenannte **Makros** und werden auch „**symbolische Konstanten**“ genannt.

Makros werden üblicherweise mit Großbuchstaben und dem Underscore-Zeichen als Trenner geschrieben.



Mithilfe der Anweisung **#define** kann definiert werden, was ein bestimmter Name innerhalb des Codes representieren soll. Dies bedeutet, dass diese Namen keine Variablen darstellen, sondern einfach nur vom Compiler durch den Wert ersetzt werden, mit welchem sie definiert wurden.

Diese Ersetzung übernimmt der sogenannte „**Präprozessor**“ (auf Englisch „**pre-processor**“). Dieser bearbeitet die Dateien vor dem eigentlichen Compilieren. Anweisungen an den Präprozessor beginnen stets mit einem Nummernzeichen #. Somit ist auch die **#include** Anweisung eine Präprozessor-Anweisung.

Mehr dazu und auch wieso diese Konstanten „Makros“ genannt werden, kann im Kapitel [→ 21](#) nachgelesen werden.

2.2.3 for-Schleife

In diesem Beispiel wurde keine while-Schleife verwendet, sondern eine for-Schleife. Eine for-Schleife eignet sich besonders gut für **Zählschleifen**. Hierbei wird stets eine Zählvariable (auch „**Schleifenvariable**“ genannt) von einem Initialwert bis zu einem Zielwert gezählt.

```
for (int fahr = LOWER; fahr <= UPPER; fahr = fahr + STEP)
```

Die for-Schleife definiert drei Elemente, jeweils getrennt durch ein Semikolon:

- Initialisierung der Zählvariable.
- Bedingung, unter welcher die Schleife ausgeführt wird.
- Schritt, mit welchem die Zählvariable verändert wird.

In der for-Schleife stellt somit `fahr = LOWER` die Initialisierung dar, `fahr <= UPPER` die **Bedingung**, wie lange die Schleife durchgeführt wird, und `fahr = fahr + STEP` berechnet den nächsten Wert, für den die Schleife durchgeführt wird.

Vergleicht man dieses Beispiel mit dem Beispiel aus Kapitel [→ 2.1](#), so wird klar, dass auch dort bereits eine for-Schleife hätte verwendet werden können.

Die verschiedenen Schleifen werden in Kapitel [→ 10](#) behandelt.

2.2.4 Formatierung von Dezimalzahlen bei `printf()`

Für die Formatierung der Zahlen wurden bei `printf()` zwei neue Formatelemente verwendet: `\t` und `%3d`.

Mit der Zeichenfolge `\t` wird auf der Konsole ein Tabulator-Schritt ausgegeben.



Durch die Verwendung des Tabulator-Zeichens als Trennzeichen zwischen den Werten werden die Celsius-Werte auf der Konsole mit einem größeren Abstand ausgegeben. Viele Programme erkennen solche Tabulatoren und ordnen die Werte dann automatisch in Spalten an.

Mit der Zeichenfolge `%3d` wird eine Dezimalzahl rechtsbündig mit 3 Ziffern Platz ausgegeben.



Eine ausführliche Beschreibung aller Formatelemente kann in Kapitel [→ 16.7](#) nachgelesen werden.

2.2.5 Ausdrücke

Die Berechnung der Temperatur lässt sich wie ein mathematischer **Ausdruck**:

```
int celsius = 5 * (fahr - 32) / 9;
```

Genau darauf baut die Sprache C auf: Wie in der Mathematik können Operanden wie Variablen und Zahlen mit Operatoren wie Addition, Subtraktion, Multiplikation und Division verknüpft werden. Genauso wie in der Mathematik werden auch die runden Klammern verwendet, um anzuzeigen, welche Teil-Ausdrücke priorisiert behandelt werden sollen.

Die Länge eines Ausdrucks ist unbegrenzt, es ist jedoch üblich, Ausdrücke so einfach wie möglich zu gestalten und gegebenenfalls Zwischenresultate wiederum in Variablen zwischenzuspeichern.

In jedem Zusammenhang, in dem der Wert einer Variablen eines bestimmten Typs stehen kann, kann auch ein komplizierter Ausdruck von diesem Typ stehen.



2.2.6 int-Zahlen und double-Zahlen

In dem Beispiel wurde bislang stets mit Integer (Ganzzahlen), sprich dem Typ `int` gerechnet. Dies funktionierte bislang ganz gut, birgt jedoch einige Gefahren. Man könnte den Ausdruck zur Berechnung des Celsius-Wertes beispielsweise auch so schreiben:

```
int celsius = 5 / 9 * (fahr - 32);
```

Rein mathematisch würde dies dasselbe Resultat ergeben. In der Programmierung jedoch entsteht dadurch ein Problem: Die beiden Zahlen 5 und 9 sind Integer, weswegen der Compiler automatisch eine sogenannte „Integer-Division“ (eine Division ohne Rest) durchführt. Diese Division 5/9 ergibt 0. Das Resultat sämtlicher Werte wird somit null sein.

Um dieses Problem zu lösen, muss der Ausdruck mit sogenannten „**Gleitpunkt-Typen**“ (Zahlen mit Nachkommastellen) geschrieben werden:

```
double celsius = 5. / 9. * (fahr - 32.);
```

Als erstes wird die Variable `celsius` nicht mehr als `int` definiert, sondern als `double`. Dies ist in C der Standard-Typ für Gleitpunkt-Typen. Die Variable `celsius` kann somit auch Zahlen mit Nachkommastellen speichern.

Des Weiteren wurden die Zahlen des Ausdrucks mit einem Punkt versehen. Damit wird der Compiler angewiesen, diese Zahlen mit dem Typ `double` zu verarbeiten. Entsprechend wird keine Integer-Division mehr ausgeführt, sondern eine normale Division mit Nachkommastellen.

Die Konvertierung der `int`-Variablen `fahr` in eine `double`-Variable erfolgt hier implizit (siehe auch Kapitel [→ 9.10](#)).

Man spricht von einer **impliziten Typumwandlung**, wenn eine Konvertierung vorgenommen wird, ohne dass dies explizit in der Programmiersprache formuliert werden muss.



Die Ausgabe einer Zahl vom Typ `double` auf der Konsole benötigt ein neues Formatelement: `%f`

```
printf("%3d\t%f\n", fahr, celsius);
```

Fügt man diese beiden Änderungen in das Beispiel ein, so ergibt sich folgende Ausgabe:

```
0      -17.777778
20     -6.666667
40      4.444444
60     15.555556
80     26.666667
100    37.777778
120    48.888889
140    60.000000
160    71.111111
180    82.222222
200    93.333333
```

2.3 Zinsberechnung

Das folgende Beispiel berechnet die jährliche Entwicklung eines Grundkapitals über eine vorgegebene Laufzeit. Die Bank gewährt zwei unterschiedliche Zinsen über zwei Laufzeiten. Die Zinsen sollen jeweils nicht ausgeschüttet, sondern mit dem Kapital wieder angelegt werden. Es wird eine Tabelle mit folgenden Angaben erzeugt: Laufendes Jahr und angesammeltes Kapital (in EUR).

Gegeben seien das Grundkapital (1000 EUR), der Zins (5%) über die erste Laufzeit (3 Jahre) und der Zins (2.5%) über die zweite Laufzeit (5 Jahre).

zins.c

```
#include <stdio.h>

#define ZINS_1      5.0
#define ZINS_2      2.5
#define LAUFZEIT_1  3
#define LAUFZEIT_2  5
#define GRUNDKAPITAL 1000.00

void printKapital(int jahr, double kapital) {
    printf("Jahr: %2d\t", jahr);
    printf("Kapital: %7.2f EUR\n", kapital);
}

int main(void) {
    double kapital = GRUNDKAPITAL;
    printKapital(0, kapital);

    int totalLaufzeit = LAUFZEIT_1 + LAUFZEIT_2;
    for (int jahr = 1; jahr <= totalLaufzeit; jahr = jahr + 1) {
        if (jahr <= LAUFZEIT_1) {
            kapital = kapital * (1. + ZINS_1 / 100.);
        } else {
            kapital = kapital * (1. + ZINS_2 / 100.);
        }
        printKapital(jahr, kapital);
    }

    printf("\n");
    printf("Aus %7.2f EUR Grundkapital\n", GRUNDKAPITAL);
    printf("wurden in %d Jahren %7.2f EUR\n", totalLaufzeit, kapital);
    return 0;
}
```

Genauso wie in den vorangegangenen Beispielen können auch hier wieder folgenden Punkte beobachtet werden:

- Grundkapital, Laufzeiten und Zinssätze werden als konstante Größen angesehen. Entsprechend wurden sie als symbolische Konstanten (Makros) definiert.
- Die Variable `kapital` ist eine Variable vom Datentyp `double`. Dies ist notwendig, um die Zinsberechnung mit korrekten Nachkommastellen durchzuführen. Die Zählvariable `jahr` hingegen ist vom Datentyp `int`.
- Um Schreibarbeit zu sparen, wurde die Gesamtlaufzeit einmal berechnet und in der Variable `totalLaufzeit` gespeichert.
- In der `for`-Schleife stellt der Initialisierungsausdruck `jahr = 1` den Beginn der Schleife dar. `jahr <= totalLaufzeit` ist die Bedingung, wie lange die Schleife durchzuführen ist, und `jahr = jahr + 1` berechnet den nächsten Wert, für den die Schleife durchgeführt wird.
- Die Funktion `printf()` wird wiederum benutzt, um Text und Werte formatiert auszugeben. Mit dem Steuerzeichen `'\n'` wird der Cursor an den Beginn der nächsten Bildschirmzeile positioniert und mit `'\t'` ein Tabulatorschritt eingefügt.
- Neu hinzugekommen ist in diesem Beispiel das Formatelement `%7.2f`, was bedeutet, dass die Zahl mit insgesamt 7 Zeichen formatiert werden soll. Hierbei sollen 2 Ziffern für die Nachkommastellen benötigt werden. Ein Zeichen wird für den Dezimalpunkt benötigt, somit bleiben noch 4 Ziffern für die Zahl vor dem Dezimalpunkt übrig.

Das Programm gibt aus:

```
Jahr: 0    Kapital: 1000.00 EUR
Jahr: 1    Kapital: 1050.00 EUR
Jahr: 2    Kapital: 1102.50 EUR
Jahr: 3    Kapital: 1157.62 EUR
Jahr: 4    Kapital: 1186.57 EUR
Jahr: 5    Kapital: 1216.23 EUR
Jahr: 6    Kapital: 1246.64 EUR
Jahr: 7    Kapital: 1277.80 EUR
Jahr: 8    Kapital: 1309.75 EUR
```

```
Aus 1000.00 EUR Grundkapital
wurden in 8 Jahren 1309.75 EUR
```

2.3.1 if-Struktur

Ein sehr wichtiges Element der Programmierung ist die if-Struktur.

Die if-Struktur wird auch als **einfache Selektion** bezeichnet. Es werden zwei Alternativen ausprogrammiert, wobei die Entscheidung, welche der beiden Alternativen schlussendlich durchgeführt wird, auf einer Bedingung basiert, welche während des Programmablaufes ausgewertet wird.



Hier werden mit der if-Struktur die beiden Laufzeiten unterschieden. Falls das aktuelle Jahr kleiner oder gleich ist wie LAUFZEIT_1, so wird der Zinssatz ZINS_1 für die Berechnung verwendet, ansonsten der Zinssatz ZINS_2.

Die if-Struktur wird im Detail in Kapitel [→ 10.2](#) behandelt.

2.3.2 Funktionsaufruf

In diesem Beispiel wurde ein weiteres, wichtiges Element benutzt: Eine **Funktion**.

Funktionen verkapseln eine Funktionalität und können an beliebigen Stellen aufgerufen werden.



Hier wurde die Funktion `printKapital()` definiert, welche eine einzige Zeile der Tabelle schön formatiert ausgibt. Um dies zu tun, erwartet die Funktion zwei sogenannte „**Parameter**“: Das Jahr und das Kapital. Innerhalb dieser Funktion können diese beiden Angaben als Variablen angesprochen werden.

Um die Funktion aufzurufen, wird einfach der Funktionsname geschrieben und innerhalb der runden Klammern werden die Werte angegeben, welche als sogenannte „**Argumente**“ an die Funktion übergeben werden sollen.

Diese Argumente können beliebige Ausdrücke sein. Insbesondere sind Variablen erlaubt wie `jahr` und `kapital` aber auch feste Werte wie `0` oder symbolische Konstanten wie `GRUNDKAPITAL`.

Auch die `printf()`-Funktion ist eine Funktion. Entsprechend können auch dort beliebige Ausdrücke übergeben werden. Im Beispiel [→ 2.1](#) wurde der Ausdruck `zahl * zahl` übergeben.

Das Schlüsselwort `void` bedeutet, dass diese Funktion keinen Wert zurückgibt. Diese und andere Eigenschaften von Funktionen werden im Detail in Kapitel [→ 4.3](#) behandelt. Hier an dieser Stelle genügt es, einmal gesehen zu haben, wie eine Funktion geschrieben und aufgerufen wird.

3 Entwurf von Programmen



Bevor man mit einer Programmiersprache umzugehen lernt, muss man wissen, was ein Programm prinzipiell ist und wie man Programme entwirft. Damit wird sich unter anderem dieses Kapitel befassen.

3.1 Vom Problem zum Programm

Der Begriff Programm ist eng mit dem Begriff Algorithmus verbunden. Algorithmen werden in Programmen umgesetzt.

Algorithmen sind Vorschriften für die Lösung eines Problems, welche die Handlungen und ihre Abfolge – also die Handlungsweise – beschreiben.



Im Alltag begegnet man Algorithmen in Form von Bastelanleitungen, Kochrezepten und Gebrauchsanweisungen.

Abstrakt kann man sagen, dass die folgenden Bestandteile und Eigenschaften zu einem Algorithmus gehören:

- Eine Menge von **Objekten**, die durch den Algorithmus bearbeitet werden.
- Eine Menge von **Operationen**, die auf den Objekten ausgeführt werden.
- Ein definierter **Anfangszustand**, in dem sich die Objekte zu Beginn befinden.
- Ein gewünschter **Endzustand**, in dem sich die Objekte nach der Lösung des Problems befinden sollen.

Dies sei am Beispiel Kochrezept erläutert:

Objekte:	Zutaten, Geschirr, Herd, ...
Operationen:	Waschen, anbraten, schälen, passieren, ...
Anfangszustand:	Zutaten im „Rohzustand“, Teller leer, Herd kalt, ...
Endzustand:	Fantastische Mahlzeit auf dem Teller

Ergänzende Information Die elektronische Version dieses Kapitels enthält Zusatzmaterial, auf das über folgenden Link zugegriffen werden kann https://doi.org/10.1007/978-3-658-45209-4_3.

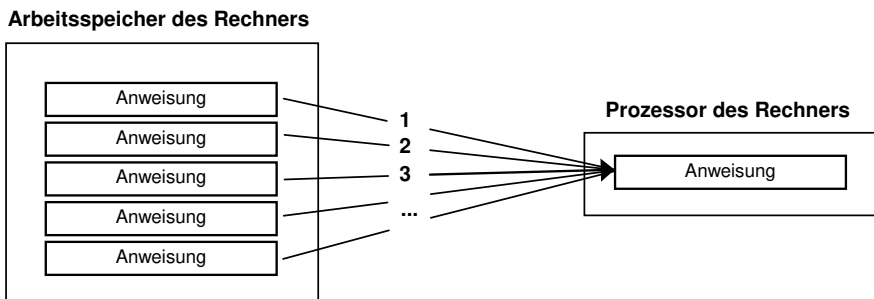
Was dann noch neben der Anleitung oder dem Rezept zur Lösung eines Problems gebraucht wird, ist jemand, der es macht, im angeführten Beispiel etwa ein Koch. Bei Programmiersprachen wird die „Anleitung“ von einem Prozessor umgesetzt.

Während bei einem Kochrezept viele Dinge gar nicht explizit gesagt werden müssen, sondern dem Koch aufgrund seiner Erfahrung implizit klar sind – beispielsweise das Rausholen des Kuchens aus dem Backofen, bevor er schwarz wird –, muss einem Prozessor alles explizit und eindeutig durch ein Programm, das aus Anweisungen einer Programmiersprache besteht, gesagt werden.

Ein Algorithmus in einer imperativen Programmiersprache sagt einem Prozessor präzise, was dieser tun soll. Ein **Programm** besteht aus **Anweisungen**, die von einem Prozessor ausgeführt werden können.



Das folgende Bild zeigt Anweisungen, die im Arbeitsspeicher des Rechners abgelegt sind und nacheinander durch den Prozessor des Rechners abgearbeitet werden:



3.1.1 Der euklidische Algorithmus als Beispiel

Als Beispiel wird der Algorithmus betrachtet, der von Euklid ca. 300 v. Chr. zur Bestimmung des größten gemeinsamen Teilers (ggT) zweier natürlicher Zahlen aufgestellt wurde. Der größte gemeinsame Teiler wird zum Kürzen von Brüchen benötigt:

$$\frac{x}{y} = \frac{x/\text{ggT}(x,y)}{y/\text{ggT}(x,y)} = \frac{a}{b}$$

Hierbei ist $\text{ggT}(x, y)$ der größte gemeinsame Teiler der beiden Zahlen x und y . Die Zahlen a und b stellen die gekürzten Zahlen dar. Beispielsweise:

$$\frac{24}{9} = \frac{24/\text{ggT}(24,9)}{9/\text{ggT}(24,9)} = \frac{24/3}{9/3} = \frac{8}{3}$$

Der euklidische Algorithmus zur Bestimmung des größten gemeinsamen Teilers zwischen zwei natürlichen Zahlen x und y definiert folgende Schritte:

Solange x ungleich y ist, wiederhole:

Wenn x größer als y ist, dann:

Ziehe y von x ab und weise das Ergebnis x zu.

Andernfalls:

Ziehe x von y ab und weise das Ergebnis y zu.

Wenn x gleich y ist, dann:

x (wie auch y) ist der gesuchte größte gemeinsame Teiler.

Es gibt in diesem Beispiel also die Objekte x und y , welche am Anfang beliebig vorgegebene Werte enthalten und sodann mit Vergleichen, Abziehen und Zuweisen so bearbeitet werden, dass sie am Schluss den größten gemeinsamen Teiler enthalten. Diese Objekte werden „Variablen“ genannt und im folgenden Kapitel [→ 3.1.2](#) genauer besprochen.

Des Weiteren ist ersichtlich, dass die Anweisungen nicht immer strikt der Reihe nach hintereinander ausgeführt werden, es sich somit nicht um eine rein sequenzielle Folge von Anweisungen handelt. Es gibt somit auch bestimmte Konstrukte, welche die einfache sequenzielle Folge gezielt verändern: Eine Auswahl zwischen Alternativen (Selektion) und eine Wiederholung von Anweisungen (Iteration). Diese Konstrukte werden „Kontrollstrukturen“ genannt und im Kapitel [→ 3.1.3](#) genauer behandelt.

3.1.2 Variablen und Zuweisungen

Die durch den Algorithmus von Euklid behandelten Objekte sind natürliche Zahlen. Diese Zahlen sollen jedoch nicht von vornherein festgelegt werden, sondern der Algorithmus soll für die Bestimmung des größten gemeinsamen Teilers beliebiger natürlicher Zahlen verwendbar sein.

Anstelle fest vorgegebener Zahlen werden im Programm Bezeichner verwendet, die einem Speicherplatz im Arbeitsspeicher oder einem Register zugeordnet sind und als variable Größen oder kurz „**Variablen**“ bezeichnet werden. Den Variablen werden im Verlauf des Algorithmus konkrete Werte zugewiesen.



Die Werteänderung von Variablen erfolgt durch sogenannte „**Zuweisungen**“. Als Zuweisungssymbol wird das Gleichheitszeichen = benutzt.



Für die Anweisung „Ziehe y von x ab und weise das Ergebnis x zu.“ wird in C folgende Anweisung geschrieben:

```
x = x - y;
```

Die Schreibweise $x = x - y$ ist zunächst etwas verwirrend. Diese Schreibweise ist nicht als mathematische Gleichung zu sehen, sondern meint etwas ganz anderes: Auf der rechten Seite des Gleichheitszeichens steht ein arithmetischer Ausdruck, dessen Wert zuerst berechnet werden soll. Dieser so berechnete Wert wird dann in einem zweiten Schritt der Variablen zugewiesen, deren Name auf der linken Seite steht. Im Beispiel also:

- Nimm den aktuellen Wert von x. Nimm den aktuellen Wert von y. Ziehe den Wert von y vom Wert von x ab.
- Der neue Wert von x sei die soeben ermittelte Differenz von x und y.

Bei einer Zuweisung wird zuerst der Ausdruck rechts vom Zuweisungs-Operator berechnet und der Wert dieses Ausdrucks dem Speicherplatz auf der linken Seite des Zuweisungs-Operators zugewiesen.



Eine Zuweisung verändert den Wert der Variablen auf der linken Seite, oder anders gesagt, deren Zustand. Daher werden solche Transformationen auch als „Zustandstransformationen“ bezeichnet. Eine solche Wertzuweisung ist eine

der grundlegenden Operationen, die ein Prozessor ausführen können muss. Auf Variablen wird noch ausführlicher in Kapitel → 7 eingegangen.

Der im obigen Beispiel beschriebene Algorithmus kann auch von einem menschlichen „Prozessor“ ausgeführt werden – andere Möglichkeiten hatten die Griechen in der damaligen Zeit nicht. Als Hilfsmittel braucht man dazu Papier und Bleistift, um die Zustände der Variablen – im obigen Beispiel die Zustände der Variablen x und y – zwischen den Verarbeitungsschritten festzuhalten. Man erhält dann eine Tabelle, die für die Zahlen x gleich 24 und y gleich 9 das folgende Aussehen hat:

Verarbeitungsschritt	Variable x	Variable y
Initialisierung	24	9
$x = x - y$	15	9
$x = x - y$	6	9
$y = y - x$	6	3
$x = x - y$	3	3

Diese Tabelle zeigt sehr deutlich die Funktion der Variablen auf:

Variablen speichern Werte. Sie repräsentieren über den Verlauf des Algorithmus hinweg ganz unterschiedliche Werte.



Zu Beginn eines Algorithmus werden den Variablen definierte Anfangs- oder Startwerte zugewiesen. Diesen Vorgang bezeichnet man als **Initialisierung** der Variablen.



Wird derselbe Algorithmus zweimal durchlaufen, wobei die Variablen am Anfang unterschiedlich initialisiert werden, dann erhält man in aller Regel auch unterschiedliche Abläufe. Sie folgen aber ein und demselben Verhaltensmuster, das durch den Algorithmus beschrieben ist.

Für eine andere Ausgangssituation sieht die Tabelle beispielsweise so aus:

Verarbeitungsschritt	Variable x	Variable y
Initialisierung	5	3
$x = x - y$	2	3
$y = y - x$	2	1
$x = x - y$	1	1

3.1.3 Kontrollstrukturen: Beschreibung sequenzieller Abläufe

Die Abarbeitungsreihenfolge von Anweisungen wird auch als „**Kontrollfluss**“ bezeichnet.



Den Prozessor stört es überhaupt nicht, wenn eine Anweisung einen Sprungbefehl zu einer anderen Anweisung enthält. Solche Sprungbefehle werden in manchen Programmiersprachen beispielsweise mit dem Befehl GOTO und Zeilenmarken wie beispielsweise 40 realisiert:

```
10 IF(a > b) GOTO 40
20 Anweisung Y
30 GOTO 50
40 Anweisung X
50 Anweisung Z
```

In Worten lauten diese Anweisungen an den Prozessor: „Vergleiche die Werte von a und b. Wenn a größer als b ist, springe in die Zeile mit der Marke 40 und führe die dort stehende Anweisung X aus. Fahre dann mit der Anweisung Z in der nächsten Zeile fort. Ist aber die Bedingung $a > b$ nicht erfüllt, so arbeite die Anweisung Y der nächsten Zeile ab. Springe dann zur Zeile mit der Marke 50 und fahre mit der Ausführung der Anweisung Z fort.“

Will man ein solches Programm jedoch lesen, so verliert man durch die Sprünge mit GOTO sehr leicht den Zusammenhang und damit das Verständnis.

Während in den sechziger Jahren somit typische Programme noch zahlreiche Sprünge enthielten, bemüht sich die Programmiergemeinschaft seit Dijkstras grundlegendem Artikel „Go To Statement Considered Harmful“ [Dij68], möglichst einen Kontrollfluss ohne Sprünge zu entwerfen.

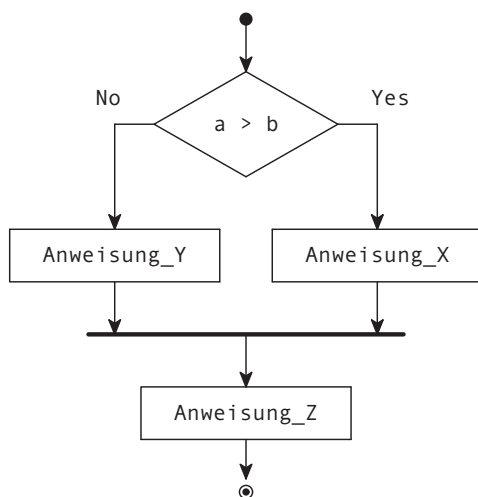
Die Ideen von Dijkstra und anderen fanden ihren Niederschlag in den Regeln für die „**Strukturierte Programmierung**“. Die Strukturierte Programmierung ist ein programmiersprachenunabhängiges Konzept. Es umfasst die Zerlegung eines Programms in Teilprogramme. Gestartet wird mit einem Hauptprogramm, welches dann Unterprogramme aufrufen kann, welche wiederum Unterprogramme aufrufen können.

Des Weiteren werden für sequenzielle Abläufe die drei grundlegenden Kontrollstrukturen Sequenz, Selektion und Iteration definiert. Eine reine Sprunganweisung wie GOTO ist in der Strukturierten Programmierung nicht zulässig.

Der oben mit GOTO beschriebene Ablauf kann mittels der Strukturierten Programmierung in C folgendermaßen realisiert werden:

```
if (a > b)
    Anweisung_X
else
    Anweisung_Y
Anweisung_Z
```

Hierbei ist `if (a > b)` die Abfrage, ob `a` größer als `b` ist. Ist dies der Fall, so wird die Anweisung `X` ausgeführt. Ist die Bedingung `a > b` nicht wahr, also nicht erfüllt, so wird die Anweisung `Y` des `else`-Zweiges durchgeführt. Damit ist die Fallunterscheidung zu Ende. Unabhängig davon, welcher der Zweige der Fallunterscheidung abgearbeitet wurde, wird nun die Anweisung `Z` ausgeführt. Dieser Ablauf ist im folgenden Bild grafisch veranschaulicht:



Für das menschliche Gehirn ist es am einfachsten, wenn ein Programm einen einfachen und überschaubaren Kontrollfluss hat.



Entsprechend wird im folgenden Kapitel das sogenannte „Flussdiagramm“ vorgestellt.

3.2 Entwurf mit Flussdiagrammen nach UML

Zur Visualisierung des Kontrollflusses von Programmen – das heißt, zur grafischen Veranschaulichung ihres Ablaufs – werden seit Jahren sogenannte **Flussdiagramme** (auf Englisch „**flow chart**“) verwendet. Sie werden in UML auch als „**Aktivitäts-Diagramme**“ bezeichnet.

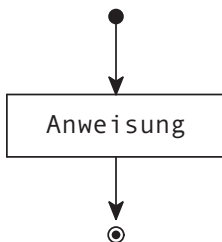
Grundsätzlich geht es bei Flussdiagrammen darum, den Kontrollfluss eines Programmes mittels Kästchen und Pfeilen so darzustellen, dass möglichst einfach ersichtlich wird, wann und in welcher Reihenfolge in einem Programm was passiert. Wie genau diese Diagramme aussehen, spielt normalerweise nicht so eine große Rolle. Manche Flussdiagramme nutzen abgerundete Kästchen, manche haben Titelleisten für bestimmte Kästchen, nutzen unterschiedliche Formen, sind farbig, manche schreiben Variablen hinzu, manche haben verschiedene Pfeilspitzen, etc.

Hier in diesem Buch wird versucht, ein sehr einfaches Set zur Verfügung zu stellen, welches durch die **UML**, die sogenannte „**Unified Modeling Language**“, motiviert ist.

Im Folgenden werden die Diagramme für die Sequenz, Selektion und Iteration in abstrakter Form, also ohne Notation in einer speziellen Programmiersprache, vorgestellt.

3.2.1 Start, Ende und einfache Anweisung

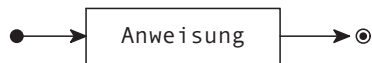
Folgendes Diagramm zeigt das fundamentale Prinzip eines Flussdiagramms:



Gestartet wird beim ausgefüllten Punkt. Dies ist der Startpunkt, von welchem aus das Programm sukzessive entlang der Pfeile abgearbeitet wird. Hier wird eine einfache Anweisung ausgeführt, was mit einem Kästchen angezeigt wird. Nach dieser Anweisung kommt das Flussdiagramm bereits zu einem Ende, markiert durch den umrandeten Punkt.

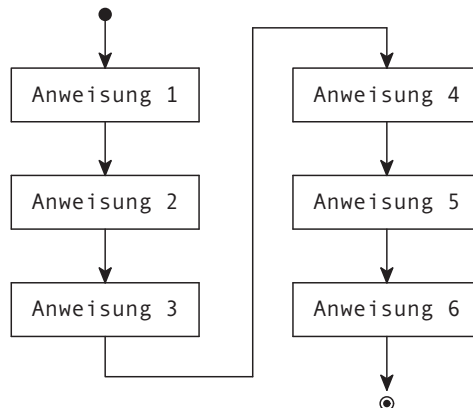
Der Begin und das Ende werden manchmal zusätzlich beschriftet. Beispielsweise werden die initialen Werte von Variablen angegeben.

Des Weiteren spielt es keine Rolle, ob ein Flussdiagramm vertikal ausgerichtet ist oder horizontal. Der Fluss folgt stets der Pfeilrichtung:



3.2.2 Anweisungen in Sequenz

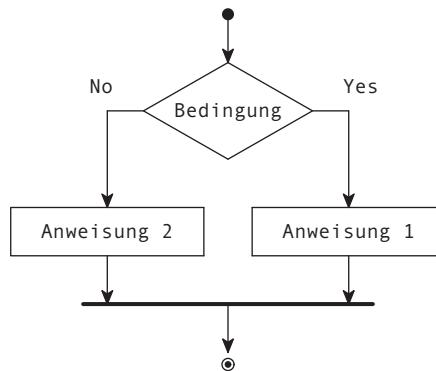
Wenn mehrere Anweisungen sequenziell hintereinander ausgeführt werden sollen, so werden sie auch im Flussdiagramm mittels Pfeile hintereinandergereiht:



Wie man sieht, kann dies auch in horizontal und vertikal gemischter Anordnung passieren. Hauptsache, die Pfeile zeigen genau an, wo der Fluss durchgeht.

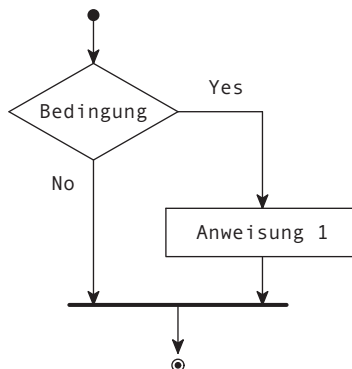
3.2.3 Einfache Selektion, einfache Alternative

Bei einer **einfachen Selektion** (auch „einfache Alternative“ genannt) wird aufgrund einer **Bedingung** der Kontrollfluss in zwei Pfade aufgeteilt. Die Bedingung wird in einen Rhombus gezeichnet, aus welchem die beiden Pfade als zwei Pfeile hervorkommen, beschriftet mit dem Ergebnis der Bedingung:



Aufgrund der Auswertung der Bedingung wird entweder Anweisung 1 oder 2 ausgeführt. Am Ende der Selektion kommen die beiden Pfade wieder zusammen, was durch eine etwas dickere Linie angedeutet wird. Danach wird wieder auf einem einzelnen Pfad fortgefahren.

Üblicherweise wird der Yes-Pfad auf der rechten Seite dargestellt, das ist aber nicht zwingend. Falls eine einfache Selektion keinen No-Pfad besitzt, so handelt es sich um eine sogenannte „**bedingte Anweisung**“:

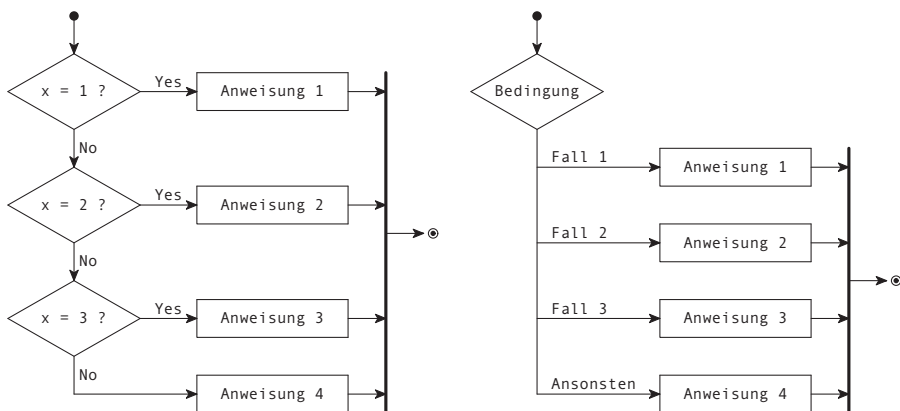


Die Anweisung wird nur ausgeführt, falls die gegebene Bedingung erfüllt ist.

3.2.4 Mehrfache Selektion, mehrfache Alternative

Eine **mehrfache Selektion** (auch „mehrfache Alternative“ genannt) tritt auf, wenn eine Bedingung geprüft wird, aufgrund derer mehr Möglichkeiten als nur Yes und No auftreten können. In C ist dies mittels der switch Kontrollstruktur möglich, welche in Kapitel [10.3](#) behandelt werden wird.

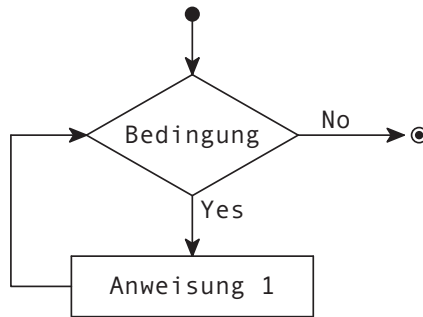
Für eine Darstellung einer Mehrfachentscheidung gibt es zwei Möglichkeiten:



Die linke Variante entspricht mehr den Prinzipien von UML, benötigt jedoch mehr Schreibarbeit. Entsprechend wird häufig die rechte Variante gewählt, auch deshalb, weil sie die Schreibweise der entsprechenden Kontrollstruktur mittels `switch` eins-zu-eins abbildet.

3.2.5 Schleifen, Iteration

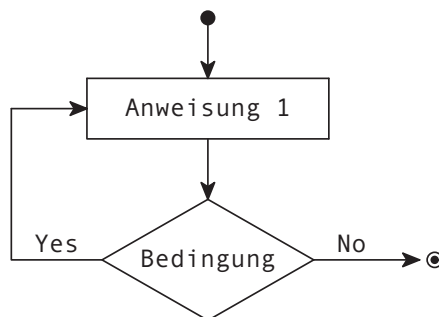
Um einen Pfad mehrere Male auszuführen, können die bisherigen Elemente einfach mittels einer Bedingung zu einem Kreis zusammengeschlossen werden:



Dies wird als eine **Schleife** bezeichnet. Diese einfache Schleife wird solange ausgeführt, wie die Bedingung erfüllt ist. Natürlich sollte die Anweisung irgendwann etwas am Zustand des Programmes verändern, ansonsten wird diese Schleife unendlich lange ausgeführt.

Diese Schleife wird als eine sogenannt „**abweisende Schleife**“ bezeichnet, da die Bedingung vor der Ausführung der Anweisung geprüft wird. Dadurch kann es sein, dass die Bedingung bereits vor dem ersten Durchlauf nicht erfüllt wird, und die Schleife somit komplett „abgewiesen“ wird.

Demgegenüber gibt es auch „**annehmende Schleifen**“. Diese sehen als Flussdiagramm folgendermaßen aus:



Hier wird die Anweisung auf jeden Fall mindestens einmal ausgeführt und erst dann mittels der Bedingung geprüft, ob sie wiederholt werden soll.

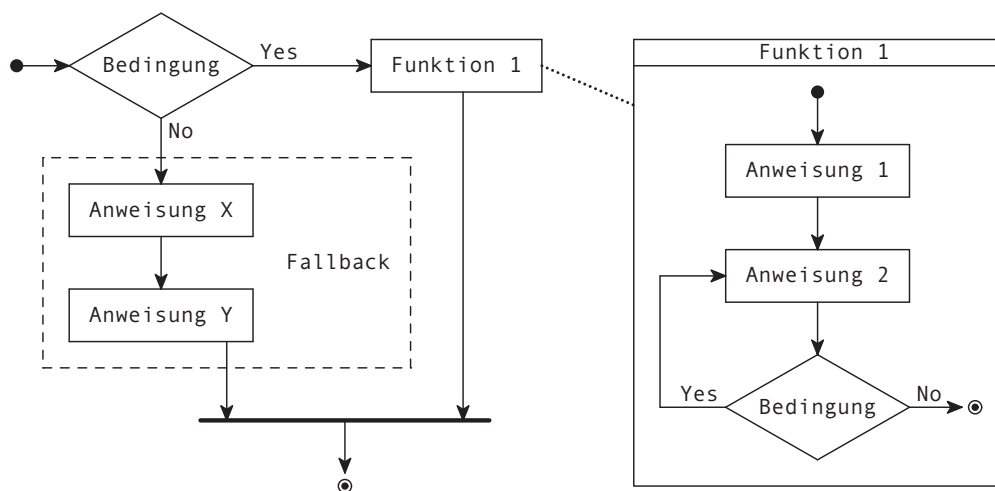
In der Programmiersprache C gibt es zwei abweisende Schleifen `for` und `while` und eine annehmende Schleife `do while`. Diese werden in Kapitel [→ 10](#) genauer behandelt.

Wird innerhalb einer Schleife eine Menge an Elementen durchschritten, beispielsweise alle Zahlen von 1 bis 100 oder alle Elemente eines Arrays, dann wird diese Schleife auch als eine „**Iteration**“ bezeichnet.

3.2.6 Gruppierungen, Blöcke, Funktionen

Häufig bilden mehrere Anweisungen und Selektionen zusammen eine neue Gesamt-Funktionalität, welche im tatsächlichen Code dann als **Anweisungsblock** oder **Funktion** ausprogrammiert wird. In einem Flussdiagramm ist man relativ frei, solche Gruppierungen vorzunehmen und sie zu benennen.

Beispielsweise könnte ein Programm so aussehen:

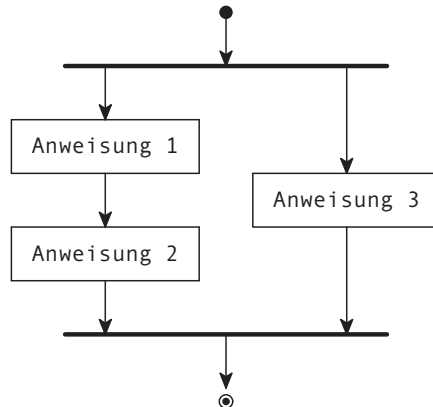


Hier wird ersichtlich, dass die beiden Anweisungen X und Y zusammen eine Gruppe namens „Fallback“ bilden. Zudem wird eine Funktion aufgerufen, deren Definition im rechten Kasten zu finden ist.

Die Verwendung der gestrichelten und gepunkteten Linien ist hier frei wählbar. Wichtig ist, dass es visuell klar strukturiert ist.

3.2.7 Parallele Anweisungen

Nebst rein sequenziellen Algorithmen gibt es auch Algorithmen zur Beschreibung von parallelen Aktivitäten, die zum gleichen Zeitpunkt nebeneinander ausgeführt werden. Diese Algorithmen werden unter anderem bei Betriebssystemen oder in der Prozessdatenverarbeitung benötigt. Ein entsprechendes Diagramm würde so aussehen:

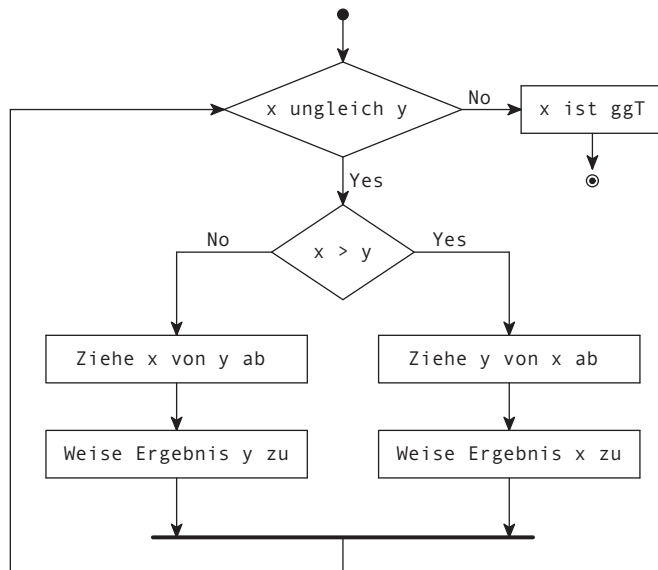


In diesem Beispiel werden zwei parallele Stränge erzeugt, wovon der eine die Anweisungen 1 und 2 abarbeitet und der andere die Anweisung 3. Die etwas dickeren Linien bezeichnen Synchronisationszeitpunkte, zu welchen die parallele Verarbeitung startet, beziehungsweise der Kontrollfluss abwartet, bis sämtliche parallelen Anweisungen abgearbeitet wurden, auf dass danach wieder rein sequenziell fortgefahren werden kann.

Parallele Abläufe werden erst in Kapitel [→ 23](#) behandelt. Dieses kleine Kapitel diene nur zur Veranschaulichung, wie mit Flussdiagrammen auch erweiterte Algorithmen dargestellt werden können. Fortan werden nur noch sequenzielle Abläufe behandelt, bei denen zum selben Zeitpunkt nur eine einzige Operation durchgeführt wird.

3.2.8 Der Euklidische Algorithmus als Flussdiagramm

Mit den Mitteln der Flussdiagramme kann nun der Algorithmus von Euklid, der in Kapitel [3.1.1](#) eingeführt wurde, in grafischer Form dargestellt werden:



Diese abstrakte Darstellung gilt es nun, in ein tatsächlich lauffähiges Programm umzusetzen. Personen, welche bereits viel Programmiererfahrung haben, können ein solch einfaches Beispielpogramm direkt umsetzen. Hier jedoch wird noch ein weiteres Hilfsmittel vorgestellt, um solch abstrakte Diagramme sukzessive in lauffähigen Code umzuwandeln: Mittels sogenanntem „Pseudocode“.

3.3 Pseudocode

Wenn ein Programm entworfen, also quasi „erfunden“ werden soll, kann man grafische Mittel wie Flussdiagramme nutzen oder man greift auf eine „halbformale“ Sprache, einen sogenannten freien Pseudocode, zurück, der es erlaubt, zuerst die Struktur eines Programms textuell festzulegen und die Details auf später aufzuschieben.

Ein **Pseudocode** ist eine Sprache, die dazu dient, Programme zu entwerfen. Pseudocode kann von einem freien Pseudocode bis zu einem formalen Pseudocode reichen.



Um zu verstehen, was freier und formaler Pseudocode ist, muss zunächst definiert werden, was eine sogenannte „formale“ Sprache ist.

Generell kann man bei Sprachen zwischen „natürlichen Sprachen“ wie der Umgangssprache oder den Fachsprachen einzelner Berufsgruppen und „formalen Sprachen“ unterscheiden.

Formale Sprachen sind beispielsweise die Notenschrift in der Musik, die Formelschrift in der Mathematik oder Programmiersprachen beim Computer. Nur das, was durch eine formale Sprache – hier die Programmiersprache – festgelegt ist, ist für den Übersetzer verständlich.

Hat man ein Problem zu lösen, dann ist es empfehlenswert, nicht gleich auf der Ebene einer formalen Programmiersprache zu beginnen. Würde man das tun, dann müsste man sich sofort mit den vielen formalen Details der Programmiersprache befassen, statt erst nachzudenken, wie der Lösungsweg denn aussehen soll.

Freie Pseudocodes sind für eine grobe Spezifikation vollkommen ausreichend.



Das Beispiel in Kapitel [→ 3.1.1](#) war in einem sogenannten „freien Pseudocode“ formuliert. Es wurden deutsche Schlüsselwörter wie „solange“, „wenn“ oder „andernfalls“ benutzt. Meist lehnt man sich aber bei einem Pseudocode an eine bekannte Programmiersprache an, sodass dann englische Begriffe dominierend sind.

Bei einem freien Pseudocode formuliert man Schlüsselwörter für die Iteration, Selektion und Blockbegrenzer und fügt in diesen Kontrollfluss Verarbeitungsschritte ein, die in der Umgangssprache beschrieben werden.



Demgegenüber gibt es jedoch auch sogenannten „formalen Pseudocode“, welcher durch vorgegebene Regeln definiert, wie ein Programm abgebildet werden soll. Entsprechend gibt es Programme, welche aus der Formulierung in einer solchen formalen Pseudo-Sprache automatisch Code generieren können.

Ein formaler Pseudocode, der alle Elemente enthält, die auch in einer Programmiersprache enthalten sind, ermöglicht eine automatische Codegenerierung für diese Zielsprache.



Grundsätzlich ist jedoch das eigentliche Ziel eines Pseudocodes, eine Spezifikation zu unterstützen.

Im Folgenden wird nun der Algorithmus von Euklid mithilfe eines Pseudocodes formuliert, welcher schon sehr ähnlich ist zum finalen Programm:

```
Euklidischer Algorithmus.  
{  
  Initialisiere x und y.  
  solange (x ungleich y)  
  {  
    falls (x kleiner als y)  
      y = y - x  
    ansonsten  
      x = x - y  
  }  
  x ist der groesste gemeinsame Teiler.  
}
```

Dieser Pseudocode und die grafische Darstellung als Flussdiagramm sind äquivalent.

3.4 Das finale Programm von Euklid in C

Der Schritt zu einem lauffähigen Programm ist nun überhaupt nicht mehr groß: Das, was soeben noch umgangssprachlich formuliert wurde, muss jetzt in C ausgedrückt werden. Die wesentlichen Punkte betreffen hier die Kommunikation des Programms mit der Person, welche es bedient: Welche Werte sollen x und y bekommen und in welcher Form soll das Ergebnis dargestellt werden? Da die Details der Ein- und Ausgabe von Daten in C-Programmen an dieser Stelle nicht behandelt werden sollen, arbeitet das folgende C-Programm für den euklidischen Algorithmus mit fest vorgegebenen Werten für x und y:

euklid.c

```
#include <stdio.h>

int main(void) {
    int x = 24;
    int y = 9;

    while (x != y) {
        if (x < y)
            y = y - x;
        else
            x = x - y;
    }

    printf("Der groesste gemeinsame Teiler ist: %d\n", x);
    return 0;
}
```

Die Ausgabe des Programmes ist:

```
Der groesste gemeinsame Teiler ist: 3
```

Viele Elemente dieses Programms sind bereits durch die vorherigen Kapitel bekannt. Es werden zwei Variablen x und y vereinbart, die Integers speichern können und jeweils mit einem vorgegebenen Wert initialisiert werden.

Die Abfragen, ob Werte ungleich sind beziehungsweise ob ein Wert kleiner als ein anderer ist, werden in C durch die Operatoren != beziehungsweise < ausgedrückt. Die Schleife wird mittels des Schlüsselwortes while gemacht und die einfache Bedingung mittels dem if-Schlüsselwort. Die Ausgabe des Resultates erfolgt mittels Aufruf der printf() Funktion.

3.5 Übungsaufgaben

Aufgabe 1: Quadratzahlen

Zeichnen Sie das Flussdiagramm für ein Programm, welches in einer (äußeren) Schleife bei jedem Durchgang einen Integer-Wert in eine Variable n einliest. Die Reaktion des Programms soll davon abhängen, ob der in die Variable eingelesene Wert positiv, negativ oder gleich null ist. Treffen Sie die folgende Fallunterscheidung:

- Ist die eingelesene Zahl n größer als null, so soll in einer inneren Schleife alle Quadratzahlen von 1 bis n ausgegeben werden, also beispielsweise bei der Eingabe 6:
1 4 9 16 25 36
- Ist die eingelesene Zahl n kleiner als null, so soll ausgegeben werden: „Negative Zahl“
- Ist die eingegebene Zahl n gleich null, so soll das Programm (die äußere Schleife) abbrechen.

Hinweise:

- Überlegen Sie sich, ob eine annehmende oder abweisende Schleife besser geeignet ist.
- Machen Sie sich keine Gedanken darüber, wie genau eine Zahl eingelesen oder ausgegeben wird. Nehmen Sie einfach an, dass es irgendwo eine Funktionalität gibt, die das kann.

4 Aufbau eines Programmes in C



Die Programmiersprache C gehört zu der Klasse der **prozeduralen Programmiersprachen**.

Ein prozedurales **Programm** besteht aus Funktionen, die auf Daten arbeiten.



In diesem Kapitel wird auf die Frage eingegangen, was genau Daten sind, von einzelnen Zeichen der Tastatur bis hin zu Werten mit unterschiedlichen Datentypen, und wie sie gespeichert werden in Variablen im Arbeitsspeicher.

Auch wird gezeigt, wie schlussendlich all diese Werte und Variablen in Funktionen übergeben, verarbeitet und wieder zurückgegeben werden, und wie ein Programm insgesamt aufgebaut sein muss, damit alle Teile zusammenpassen und korrekt miteinander interagieren können.

4.1 Zeichen und Codierungen

Wenn ein Programm mit Hilfe eines Texteditors geschrieben wird, werden Zeichen über die Tastatur eingegeben. Einzelne oder mehrere aneinandergereihte Zeichen haben im Programm eine spezielle Bedeutung. So repräsentierten beispielsweise die Zeichen x und y bei der Implementierung des euklidischen Algorithmus in Kapitel  3.4 die Namen von Variablen.

Ein „**Zeichen**“ ist ein von anderen Zeichen unterscheidbares Objekt, welches in einem bestimmten Zusammenhang eine definierte Bedeutung trägt.



Ergänzende Information Die elektronische Version dieses Kapitels enthält Zusatzmaterial, auf das über folgenden Link zugegriffen werden kann https://doi.org/10.1007/978-3-658-45209-4_4.

Zeichen können beliebige Symbole (beispielsweise Buchstaben), Bilder (beispielsweise Hieroglyphen), Töne (beispielsweise Morsezeichen) oder gar Objekte (beispielsweise Rauchzeichen) sein. Zeichen derselben Art sind Elemente eines Zeichenvorrats. So sind beispielsweise die Zeichen I, V, X, L, C, D und M Elemente des Zeichenvorrats der römischen Zahlen.

Eine arabische Ziffer ist ein Zeichen, das die Bedeutung einer Zahl hat. Diese Zeichen werden als Dezimalziffern 0, 1, ... 9 verwendet. Jedoch können durchaus auch Buchstaben die Rolle von Ziffern einnehmen, wie das Beispiel der römischen Zahlen zeigt.

Von einem „Alphabet“ spricht man, wenn der Zeichenvorrat eine strenge Ordnung aufweist.



So stellen beispielsweise folgende geordnete Folgen unterschiedliche Alphabete dar:

0, 1	das Binäralphabet,
a, b, c, ... , z	die Kleinbuchstaben ohne Umlaute und ohne ß,
0, 1, ... , 9	das Dezimalalphabet

4.1.1 Rechnerinterne Darstellung von Zeichen

Ein Computer funktioniert mit Strom und besitzt als Basiseinheit nur zwei Zustände: Strom aus oder Strom an. Diese zwei Zustände werden in einem sogenannten „**Bit**“ gespeichert, der kleinsten Speichereinheit eines Computers. Bit ist eine Abkürzung für den englischen Ausdruck „binary digit“.

Rechnerintern werden Zeichen durch Bits dargestellt. Ein Bit kann den Wert 0 oder 1 annehmen.



Mit einem Bit können also gerade mal zwei verschiedene Fälle dargestellt werden. Gruppiert man jedoch mehrer Bits zu einer Einheit, so entstehen schnell mehr Kombinationsmöglichkeiten. Mit einer Gruppe von 2 Bits sind $2 * 2 = 4$ Kombinationen möglich, mit einer Gruppe von 3 Bits können bereits $2 * 2 * 2 = 8$ verschiedene Fälle dargestellt werden. Die Anzahl an Möglichkeiten wächst exponentiell:

1 Bit	$2^1 = 2 = \mathbf{2}$ Möglichkeiten
2 Bits	$2^2 = 2 * 2 = \mathbf{4}$ Möglichkeiten
3 Bits	$2^3 = 2 * 2 * 2 = \mathbf{8}$ Möglichkeiten
4 Bits	$2^4 = 2 * 2 * 2 * 2 = \mathbf{16}$ Möglichkeiten
5 Bits	$2^5 = 2 * 2 * 2 * 2 * 2 = \mathbf{32}$ Möglichkeiten
6 Bits	$2^6 = 2 * 2 * 2 * 2 * 2 * 2 = \mathbf{64}$ Möglichkeiten
7 Bits	$2^7 = 2 * 2 * 2 * 2 * 2 * 2 * 2 = \mathbf{128}$ Möglichkeiten
8 Bits	$2^8 = 2 * 2 * 2 * 2 * 2 * 2 * 2 * 2 = \mathbf{256}$ Möglichkeiten

...

Folgende Kombinationen sind mit 3 Bits möglich:

000 001 010 011 100 101 110 111

Um aus diesen binären Zahlen ein Alphabet zu erstellen, ordnet ein Computer jeder dieser Bitgruppen ein Zeichen zu, das heißt, dass jede dieser Bitkombinationen ein Zeichen eines Alphabets repräsentieren kann. Im folgenden Beispiel wird jeder dieser Bitkombinationen eine Farbe zugeordnet:

000	Schwarz
001	Rot
010	Grün
011	Gelb
100	Blau
101	Magenta
110	Cyan
111	Weiß

Bei der rechnerinternen Darstellung von Zeichen durch Bits braucht man nur eine eindeutig umkehrbare Zuordnung (beispielsweise erzeugt durch eine Tabelle) und kann dann jedem Zeichen eine Bitkombination und jeder Bitkombination ein Zeichen zuordnen.



Mit anderen Worten, man bildet die Elemente eines Zeichenvorrats auf die Elemente eines anderen Zeichenvorrats ab.

Die Abbildung der Elemente eines Zeichenvorrats auf die Elemente eines anderen Zeichenvorrats bezeichnet man als „Codierung“.



Ein Beispiel einer häufig verwendeten Codierung ist dasjenige der positiven ganzen Zahlen. Folgende Tabelle weist jeder 8-Bit Kombination eine positive Dezimalzahl zu:

00000000	0
00000001	1
00000010	2
00000011	3
00000100	4
...	
11111101	253
11111110	254
11111111	255

Ein Computer speichert somit Zahlen intern nicht mit Dezimalziffern, sondern binär mit der eben gezeigten Codierung. Für negative Zahlen gibt es wiederum spezielle Codierungen, genauso wie auch für Gleitpunkt-Zahlen. Eine ausführliche Behandlung dieser Codierungen wird in Anhang [→ E.5](#) stattfinden.

Wichtig ist eine solche Codierung schlussendlich für die Person, welche ein Programm schreiben möchte. Während zu Anfangszeiten des digitalen Computerzeitalters tatsächlich einzelne Bits oder Bitfolgen (beispielsweise mittels Lochkarten) dem Computer eingespeist werden mussten, kann heutzutage ganz einfach eine Text-Datei geschrieben werden. Der Compiler wandelt sodann sämtliche Zeichen und Zahlen mittels der entsprechenden Codierung in Bitfolgen um.

4.1.2 Relevante Text-Codierungen für Rechner



Damit einem Computer mitgeteilt werden kann, was ein Programm ausführen soll, muss der Programmtext in einer **Codierung** geschrieben werden, welche der Computer versteht. Heutzutage sind insbesondere folgende Codierungen für die Sprache C relevant:

- **ISO 646:** Die Codierung ISO/IEC 646 wird als „Invariant Code Set“ bezeichnet und definiert 110 global festgelegte Zeichen sowie 18 länderspezifische Zeichen. Dieser Zeichensatz beinhaltet die lateinischen Buchstaben, die arabischen Ziffern, einige Interpunktionszeichen (Prozentzeichen, Punkt, Fragezeichen etc.) sowie einige Steuerzeichen. Dieser Zeichensatz war ursprünglich die minimale Anforderung, die ein C-Compiler verstehen muss. Heutige Compiler erwarten jedoch mindestens den ASCII-Zeichensatz.
- **ASCII:** Der ASCII-Zeichensatz definiert 128 Zeichen als 7-Bit-Code, wobei diese heutzutage in 8 Bit verpackt werden und das achte Bit auf 0 gesetzt wird. ASCII ist die Abkürzung für „American Standard Code for Information Interchange“. Der ASCII-Zeichensatz ist identisch mit der US-länderspezifischen Variante von ISO 646. ISO 646 ist zudem vorwärtskompatibel zu ASCII.
- **UTF-8:** Mit der Verbreitung von **Unicode** (auch bekannt als **UCS** = universal character set) wurde UTF-8 (UTC transformation format for 8 bit) immer populärer. Ab dem Standard C11 sind in C auch Unicode-Zeichen möglich. ASCII ist vorwärtskompatibel zu UTF-8, doch UTF-8 erlaubt es zusätzlich, sämtliche Zeichen des Unicodes (mit weit über einer Million Zeichen) abzubilden.
- **UTF-16:** UTF-16 ist ähnlich wie UTF-8 eine Codierung, welche Unicode abzubilden vermag. Leider aber benötigt ein Zeichen 16 und nicht 8 Bits. Deswegen ist ASCII nicht vorwärtskompatibel zu UTF-16, sondern erfordert eine Umkonvertierung.
- **UTF-32:** Diese Codierung ist genauso wie UTF-16 zu betrachten, jedoch benötigt ein Zeichen 32 Bits.

Die verschiedenen Codierungen, insbesondere die Umwandlungen von Unicode mittels UTF, werden in Anhang  detailliert beschrieben.

Im Rahmen dieses Buches kann vorerst davon ausgegangen werden, dass Zeichen in C normalerweise 8 Bits benötigen. Damit kann in der Regel dem ASCII-Standard gefolgt werden. Es sei jedoch angemerkt, dass manche Betriebssysteme intern UTF-16 oder gar UTF-32 als Codierung verwenden. Diese Zeichen müssen jedoch durch einen speziellen Typ und entsprechende Umwandlungsfunktionen angesprochen werden. In Kapitel [→ 6.1](#) wird genauer darauf eingegangen.

Zeichen sind zunächst Buchstaben, Ziffern oder Sonderzeichen. Zu diesen Zeichen können auch noch Steuerzeichen hinzukommen.



Ein Steuerzeichen ist beispielsweise `^C`, das durch das gleichzeitige Anschlagen der Taste `Strg` (Steuerung, oder `Ctrl` (Control) auf englischen Tastaturen) und der Taste `C` erzeugt wird. Die Eingabe von `^C` in einer Konsole kann dazu dienen, ein Programm abzubrechen.

4.2 Variablen und Werte

Bei imperativen Sprachen wie C besteht ein Programm aus einer Folge von Anweisungen, wie beispielsweise „Wenn `x` größer als `y` ist, dann ziehe `y` von `x` ab und weise das Ergebnis `x` zu“. Wesentlich an diesen Sprachen ist das Variablenkonzept.

Eine **Variable** ist im Gegensatz zu einer Konstanten eine veränderliche Größe. In der Programmierung werden Variablen benötigt, um in ihnen **Werte** abzulegen, die Werte wieder abzurufen, zu verarbeiten und erneut zu speichern.

Eine Variable hat in C vier Eigenschaften:

- Variablennamen
- Datentyp
- Wert
- Adresse



Eine Variable ist eine benannte Speicherstelle (Adresse) des Arbeitsspeichers. Über den Variablennamen kann auf die entsprechende Speicherstelle und somit auch auf den Wert, welcher dort gespeichert ist, zugegriffen werden.



Es ist auch möglich, dass Variablen in Registern des Prozessors liegen. Darauf wird in Kapitel [→ 15.6](#) eingegangen.

An so einer Speicherstelle steht eine Folge von Bits, welche mithilfe einer passenden Codierung einen gewünschten Wert speichern. Um dem Compiler klarzumachen, welche Codierung er für die gespeicherten Werte nutzen soll, wird jede Variable in C mit einem sogenannten „Typ“ deklariert.

4.2.1 Datentypen

Ein **Datentyp** – oder kurz **Typ** – ist der Bauplan für eine Variable. Ein Datentyp legt fest, welche **Operationen** auf einer Variablen möglich sind und wie die Codierung der Variablen im Speicher erfolgt.



Mit der Codierung wird festgelegt, wie viele Bytes die Variable im Speicher einnimmt und welche Bedeutung ein jedes Bit dieser Darstellung hat. In der Programmiersprache C gibt es viele vorgegebene Datentypen mit einer genau festgelegten Codierung. Beim Schreiben eines Programmes ist es zudem erlaubt, weitere Typen zu definieren. All diese Typen werden mittels eines Namens angesprochen. Nach einmaliger Definition kann ein solcher Typname in der vorgesehenen Bedeutung ohne weitere Maßnahmen verwendet werden.

Eine Variable wird vereinbart, indem mittels eines Typnamens die gewünschte Codierung festgelegt wird. Dadurch ist für den Compiler klar, wieviele Bytes eine Variable benötigt und wie sie zu interpretieren ist.

Der in einer Variablen gespeicherte Wert muss der Variablen explizit zugewiesen werden. Wenn einer Variablen nie ein Wert zugewiesen wird, sind die Bits an der Speicherstelle der Variablen mehr oder weniger zufällig angeordnet und ergeben sinnfreie Werte. Dies kann zu einer Fehlfunktion des Programms führen. Daher darf nicht versäumt werden, den Variablen die gewünschten Startwerte (Initialwerte) zuzuweisen, das heißt die Variablen zu **initialisieren**.

Beim Schreiben eines Programmes ist man selbst verantwortlich dafür, dass Variablen initialisiert werden.



Die Sprache C definiert mehrere Datentypen, welche unterteilt werden können in einfache und zusammengesetzte Datentypen.

Ein **einfacher Datentyp** ist nicht aus noch einfacheren Datentypen zusammengesetzt. Die Sprache C stellt standardmäßig einige einfachen Datentypen bereit wie `int` zur Darstellung von Integern oder `double` zur Darstellung von Gleitpunktzahlen. Beide Datentypen wurden bereits in Programmierbeispielen in Kapitel [→ 2](#) verwendet. Eine ausführliche Beschreibung dieser Typen findet sich in Kapitel [→ 7](#).

Ein Datentyp wie `int` oder `double` ist in C jedoch nicht nur definiert durch seine Codierung, sondern auch durch die zulässigen Operationen, welche mit diesem Datentyp ausgeführt werden können.

In der Programmierung bezeichnet eine Operation die Verknüpfung von **Operanden** mit einem **Operator**. Beispielsweise bezeichnet die Verknüpfung der Operanden 1 und 2 mit dem Operator `+` eine Operation, welche die beiden Zahlen addiert.

Man kann sich vorstellen, dass die Sprache C eine sehr große Tabelle angelegt hat, welche für alle möglichen Kombinationen aus Datentyp und Operation festlegt, was genau ausgeführt werden soll:

Operator	Operanden	Operation	Ergebnis
-	<code>int</code>	Negation	<code>int</code>
*	<code>(int, int)</code>	Multiplikation	<code>int</code>
<	<code>(float, float)</code>	Vergleich	<code>int</code> (Wahrheitswert)
=	<code>(double, int)</code>	Zuweisung	<code>double</code>

Diese Liste ist natürlich nicht vollständig, soll jedoch veranschaulichen, dass die Sprache C genau festlegt, wie Operatoren mit den zugrundeliegenden Werten verfahren sollen.

Beispielsweise ist der Multiplikations-Operator `*` folgendermaßen zu lesen: Der Multiplikations-Operator `*` verknüpft zwei `int`-Werte zu einem `int`-Wert als Ergebnis, indem er sie miteinander multipliziert.

Der Compiler nutzt diese Tabelle, um dann die genau zu dem Typ passenden Maschinenbefehle zu generieren.

Zusammengesetzte Datentypen erlauben es, Aufbau und Struktur von Variablen angepasst an eine gegebene Problemstellung zu definieren, indem man aus mehreren einfachen Datentypen einen neuen Datentyp konstruiert.

C bietet für zusammengesetzte Datentypen das Sprachkonstrukt der **Struktur** (struct).



Eine Struktur bildet ein Objekt der realen Welt in ein Schema ab, das der Compiler versteht, wobei ein Objekt beispielsweise ein Haus, ein Vertrag oder eine Firma sein kann – also prinzipiell jeder Gegenstand, der für einen Menschen eine Bedeutung hat und den er sprachlich beschreiben kann. Will man beispielsweise eine Software für das Personalwesen einer Firma schreiben, so ist es beispielsweise zweckmäßig, einen Datentyp mittels einer Struktur namens Mitarbeiter einzuführen.

Eine Struktur, die einen Mitarbeiter abbildet, könnte in C wie folgt deklariert werden:

```
struct Mitarbeiter {  
    char vorname[25];  
    char name[25];  
    int alter;  
    int gehalt;  
};
```

Auch Bibliotheken können solche Datentypen definieren, beispielsweise der Typ `struct tm` in der Bibliothek `<time.h>` (siehe Kapitel [→ 13.1.10](#)).

Auf zusammengesetzte Typen wird in Kapitel [→ 13.1](#) noch detailliert eingegangen.

Zusammengesetzte Typen sind sogenannten „selbst definierte“ Typen – sie werden nicht von der Sprache vorgegeben. Sie erlauben es, nützliche Typen bereitzustellen, welche genau auf das vorliegende Programm zugeschnitten sind.

Nebst den Strukturen zählen zudem die Enumerationen und Unions zu den selbst definierten Datentypen. Auf Enumerationen wird in Kapitel [→ 7.2.5](#) eingegangen und auf Unionen in Kapitel [→ 13.3](#).

4.3 Funktionen in prozeduralen Programmen

Ein Programm soll in kleinere Einheiten aufgeteilt werden können, die überschaubar sind, was ganz allgemein als **Modularisierung** bezeichnet wird. Riesengroße Programme, die sich über mehrere tausend Codezeilen erstrecken, sollen vermieden werden, da sie unübersichtlich und damit schwer wartbar sind.

Prozedurale Programme bestehen deswegen aus einem **Hauptprogramm** und mehreren **Unterprogrammen**.

Ein Programm soll verständlich, testbar und wartbar sein. Deswegen wird es in kleinere Unterprogramme unterteilt. Ein Unterprogramm umfasst eine Folge von Anweisungen und stellt eine Programmeinheit dar.



Diese Unterprogramme werden **Prozeduren** und **Funktionen** genannt. Jedes dieser Unterprogramme hat die Aufgabe, eine Teillösung in einen abgeschlossenen Rahmen zu verpacken.

Dadurch, dass ein Unterprogramm eine bestimmte Berechnung kapselt, kann eine Berechnung relativ leicht ausgetauscht und ein alternativer Algorithmus verwendet werden, ohne dass der Rest des Programms von dieser Änderung betroffen ist.



Prozeduren und Funktionen sind ein Mittel zur Strukturierung eines Programms.



Prozeduren werden traditionell eher als ausführende Programmteile betrachtet, wohingegen Funktionen eher den Charakter eines Unterprogramms haben, welches ein Resultat liefert. Die Programmiersprache C macht hierbei keine Unterscheidung und definiert jedes Unterprogramm als eine Funktion. Ein Programm besteht also in C stets aus Funktionen, wovon eine Funktion die Rolle des Hauptprogrammes übernimmt und alle anderen Funktionen Unterprogramme darstellen.

Das Hauptprogramm in C ist die Funktion mit dem Namen **main()**. Mit ihr beginnt ein Programm seine Ausführung.



Der Name `main()` ist exklusiv für das Hauptprogramm reserviert und darf in einem C-Programm nicht ein zweites Mal verwendet werden.



Auch andere Funktionen haben einen Namen. Somit können Funktionen per Name und mit wechselnden aktuellen Parametern aufgerufen werden. Dies kann den Programmtext erheblich verkürzen, indem an den entsprechenden Stellen anstelle des gesamten Codes der Funktion einfach nur noch der Aufruf der Funktion notiert werden muss.

Funktionen haben die Aufgabe, Teile eines Programms unter einem eigenen Namen zusammenzufassen.



Es ist somit möglich, die Funktionalität einer Funktion an beliebig vielen Stellen im Programm zu nutzen.

Funktionen sind wiederverwendbar. Kann ein und dasselbe Unterprogramm mehrfach in einem Programm aufgerufen werden, so wird das Programm insgesamt kürzer und es ist auch einfacher zu testen.



Funktionen können auch Ergebnisse zurückliefern. Die Programmausführung wird nach Abarbeitung einer Funktion an der Stelle des Aufrufs fortgesetzt.

4.3.1 Unterprogramme und Bibliotheken

Viele Unterprogramme sind nicht nur in einem einzigen Programm verwendbar, sondern lösen eine Aufgabe, die in vielen Programmen zu erledigen ist.

Unterprogramme aus zusammenhängenden Aufgabengebieten werden in sogenannten **Bibliotheken** (auf englisch „**libraries**“) zusammengefasst und anderen Codeteilen mittels Einbinden der passenden Datei zur Verfügung gestellt.



Bibliotheken werden vom Compilerhersteller zusammen mit dem Compiler ausgeliefert, können aber auch auf dem freien Markt bezogen oder selbst erstellt werden.

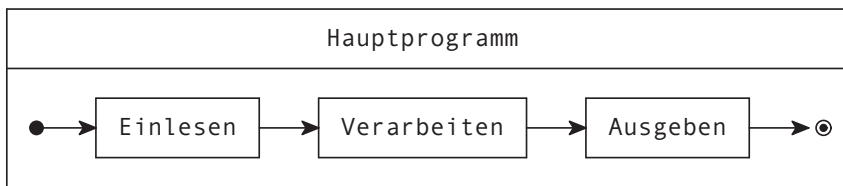
So wird beispielsweise in der Programmiersprache C das Programmieren durch eine ganze Reihe von Standardbibliotheken erleichtert, die jeder C-Compiler anbieten muss. Im „Hello World“-Programm in Kapitel [→ 1.1](#) wurde beispielsweise die Bibliothek `<stdio.h>` eingebunden, welche die Funktionalität der Standardein- und -ausgabe beinhaltet.

Die standardisierten Bibliotheken und deren verfügbare Funktionen sind im Anhang [→ A](#) aufgelistet.

4.3.2 Aufrufhierarchie von Unterprogrammen

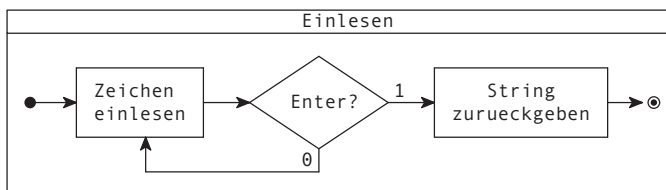
Genauso wie das Hauptprogramm kann jedes Unterprogramm wiederum Unterprogramme aufrufen. Durch diese Aufrufe entsteht eine Beziehung zwischen den verschiedenen Programmeinheiten, die als „**Aufrufhierarchie**“ bezeichnet wird.

Im Folgenden wird ein einfaches Beispiel für ein Programm betrachtet, welches aus einem Hauptprogramm und drei Unterprogrammen besteht. Welche Unterprogramme wann vom Hauptprogramm aufgerufen werden, kann wie in einem Flussdiagramm dargestellt werden:



Im Diagramm wird offensichtlich: Das gezeigte Hauptprogramm ruft die Unterprogramme Einlesen, Verarbeiten und Ausgeben hintereinander auf.

Was genau hinter einem dieser Unterprogramme steckt, ist jedoch in diesem ersten Diagramm noch nicht ersichtlich. Für ein Unterprogramm wird ein neues Diagramm erstellt. Beispielsweise könnte das erste Unterprogramm Einlesen folgendermaßen aussehen:



Hier zeigt sich, wie praktisch Flussdiagramme sind: Jedes Diagramm wird eindeutig mit seinem Anfang (ausgefüllter Punkt) und Ende (umrahmter Punkt) dargestellt, so dass jedes Sinnbild nach außen hin genau einen Eingang und einen Ausgang hat und somit als abgeschlossene Einheit betrachtet werden kann. Von außen betrachtet nennt sich dies eine sogenannte „Black Box“.

Innerhalb des Diagramms kann der **Programmfluss** erneut beliebig zerlegt werden. Jede Einheit (beispielsweise „Zeichen einlesen“) kann erneut ein vollständiges Flussdiagramm enthalten.

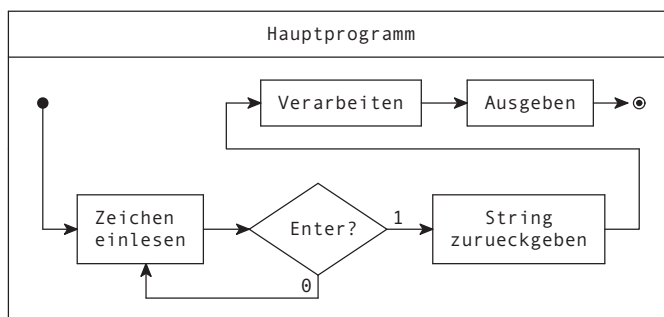
Ein Programm wird nicht durch ein einzelnes Diagramm beschrieben, sondern durch eine Menge von Diagrammen – für jedes Unterprogramm ein Diagramm.



Jedes Programm besitzt einen Zeiger (der sogenannte **Befehlszeiger**, oder auf Englisch „**instruction pointer**“), welcher dem Computer sagt, wo innerhalb des Codes gerade Anweisungen ausgeführt werden. Im Diagramm kann dies einfach nachvollzogen werden, indem ein Zeiger auf die verschiedenen Kästchen zeigt. Betrachtet man nun nur das Hauptprogramm, dann startet der Zeiger zunächst beim Startpunkt, dann zeigt er auf Einlesen, dann auf Verarbeiten, dann auf Ausgeben und dann geht der Zeiger ans Ende.

Gelangt ein Programm jedoch an ein Unterprogramm, so verschiebt sich der Befehlszeiger temporär an die Startstelle des Unterprogrammes. In der Programmiersprache C werden Unterprogramme mittels Funktionen realisiert. Bei einem sogenannten „Funktionsaufruf“ verschiebt sich der Befehlszeiger an den Startpunkt der Funktion. Man sagt auch, es wird in die Funktion „gesprungen“. Der Befehlszeiger arbeitet sodann die Anweisungen gemäß dem Diagramm des Unterprogramms ab und gelangt irgendwann zum Ende der Funktion. Dann wird zu dem Punkt im Hauptprogramm „zurückgesprungen“, an welchem das Unterprogramm aufgerufen wurde, und der Zeiger wird auf die nächste Anweisung dahinter gesetzt.

Würde man diesen Fluss in einem Diagramm darstellen, so sähe dies etwa so aus:



Dieses Flussdiagramm hat exakt die gleiche Funktionalität. Nur ist hier das Einlesen nicht mehr in eine Funktion verpackt. Es stellt sich nun natürlich die Frage, ab wann denn nun einzelne Anweisungen überhaupt in Funktionen verpackt werden sollen.

4.4 Schrittweise Verfeinerung beim Top-Down-Design

Wann führt man ein Unterprogramm ein und wann genügt es, die Verarbeitungsschritte direkt hinzuschreiben? Diese sehr wichtige Frage kann nicht einfach pauschal beantwortet werden. Viele Aspekte spielen hier eine Rolle, die zum Teil auch schon erwähnt wurden, wie etwa die Übersichtlichkeit oder die Wiederverwendbarkeit. In den folgenden Abschnitten wird eine Vorgehensweise gezeigt, welche die Aufteilung eines Programms in einzelne Unterprogrammeinheiten unterstützt.

Beim Entwurf eines neuen Programms geht man in der Regel **top-down** vor. Das bedeutet, dass man von groben Strukturen (top) ausgeht, die dann schrittweise in feinere Strukturen (bottom) zerlegt werden. Dies ist das Prinzip der **schrittweisen Verfeinerung**.

Bei der schrittweisen Verfeinerung wird das vorgegebene Problem in Teilprobleme und in die Beziehungen zwischen diesen Teilproblemen top-down so zerlegt, dass jede Teilaufgabe weitgehend unabhängig von den anderen Teilaufgaben gelöst werden kann.



Ist die Lösung eines Teilproblems nicht komplex, werden die notwendigen Schritte einfach hingeschrieben. Ist ein Teilproblem komplex, wird das Prinzip der schrittweisen Verfeinerung wiederum auf dieses Teilproblem angewandt. Man kann für jeden Teil entscheiden, ob es sich lohnt, diesen Teil als Unterprogramm zu realisieren.

Im Kontext einer prozeduralen Programmiersprache wie C werden zur Lösung von Teilproblemen Unterprogramme eingeführt.



4.4.1 Das EVA-Prinzip

Eine der wichtigsten Methoden, um Programmeinheiten (also Hauptprogramm und Unterprogramme) in weitere Programmeinheiten aufzuteilen, wurde bereits angesprochen:

Ein Datenverarbeitungsproblem kann sehr oft in die Teilprobleme **E**inlesen, **V**erarbeiten und **A**usgeben aufgespalten werden.



Dieses Prinzip der Zerlegung wird nach den Anfangsbuchstaben der Tätigkeiten auch als **EVA-Prinzip** bezeichnet. Daten müssen zuerst eingelesen werden. Dann werden diese Daten verarbeitet und am Ende müssen die Ergebnisse ausgegeben werden.



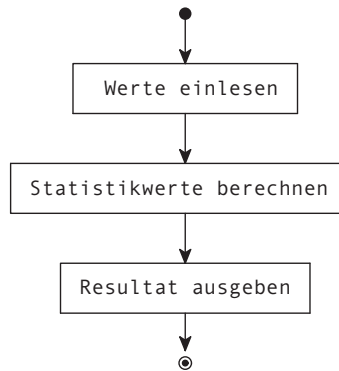
Nicht immer ist das EVA-Prinzip sinnvoll, jedoch ist es ein gutes Grundprinzip, wie das folgende Beispiel zeigt.

4.4.2 Exemplarische Durchführung eines Top-Down-Design

In diesem Kapitel wird ein Programm betrachtet, welches die Aufgabe hat, Durchschnittswerte zu berechnen. Das Programm wird top-down in seine Teilaufgaben aufgespalten:

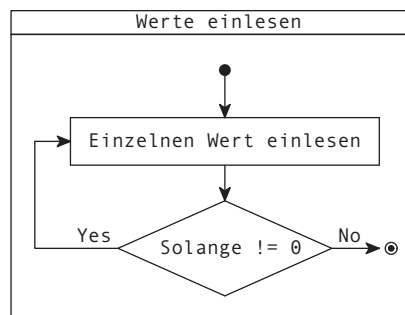
- Ein Programm liest Zahlen ein. Sobald eine Null gelesen wird, wird die Eingabe abgebrochen.
- Das Programm errechnet daraufhin den mathematischen Durchschnitt und die Standardabweichung der eingegebenen Werte.
- Das Programm gibt die errechneten Werte aus.

Alleine schon aus der Problemstellung lässt sich das EVA-Prinzip klar erkennen: Einlesen – Verarbeiten – Ausgeben. Das Diagramm sieht in diesem konkreten Beispiel somit folgendermaßen aus:

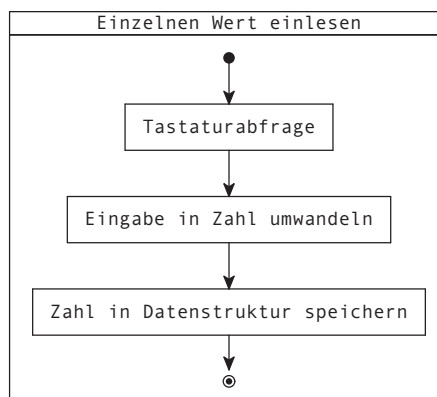


Bereits zu Beginn wird nun festgestellt, dass das Ausgeben der Resultate wohl keine Herausforderung darstellt, sodass die Ausgabe nicht weiter als Unterprogramm ausgeführt, sondern direkt als Anweisung geschrieben werden kann.

Für das Einlesen jedoch wird ein Unterprogramm top-down entworfen:



Die Anweisung „Einzelnen Wert einlesen“ wird nun wiederum mittels des EVA-Prinzips verfeinert:



Dieses Diagramm ist nun genügend fein ausgearbeitet, es werden somit keine weiteren Unterprogramme mehr definiert. Das Diagramm ist bereits auf der Beschreibungsebene von Anweisungen in der Programmiersprache angelangt. Mit anderen Worten: Die Verarbeitungsschritte entsprechen bereits einzelnen Anweisungen.

Es ist möglich, Flussdiagramme bis auf die Programmcode-Ebene zu verfeinern.

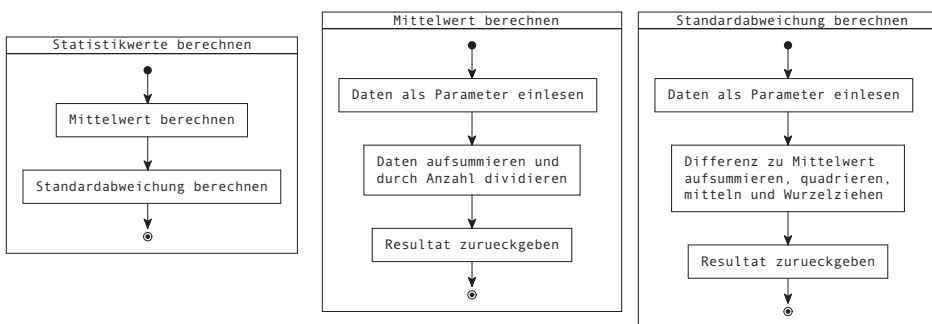


Dann entspricht jeder Verarbeitungsschritt einer Anweisung des Programms. Bei komplexen Programmen kommt man aber erst nach mehrfachen Verfeinerungen auf die Ebene einzelner Anweisungen.

Generell ist es aber nicht wünschenswert, den Entwurf bis auf die Ebene einzelner Anweisungen voranzutreiben, da dann identische Informationen in zweierlei Notation (Flussdiagramm, Programmcode) vorliegen würden. Änderungen an einer einzelnen Anweisung würden dann jedes Mal Änderungen in der Spezifikation nach sich ziehen.



Das zweiten Unterprogramm „Verarbeitung“ des Hauptprogrammes wird nun ebenfalls verfeinert. Hier werden nun gleich alle Verfeinerungsstufen gezeigt:



Für die Eingabe wurde hier die Parameterübergabe verwendet. Für die Ausgabe wurde die Rückgabe des Resultates eingesetzt. Nach diesem Prinzip können sämtliche Funktionen in C aufgebaut werden.

4.5 Funktionen in C

Im Programm „Hello world“ bestand das C-Programm nur aus einer einzigen Funktion, der `main()`-Funktion. Dies ist für kleine Programme auch ausreichend. Um ein Programm aber besser strukturieren zu können, erlaubt C die Definition eigener **Funktionen**.

Im folgenden Beispiel wird die Summe der ganzen Zahlen von 1 bis n gebildet – in mathematischer Schreibweise lautet die Formel dafür:

$$\text{summe} = \sum_{i=1}^n i$$

4.5.1 Definition von Funktionen

Bei der Definition einer Funktion unterscheidet man zwei Teile: den Funktionskopf und den Funktionsrumpf.

Der **Kopf einer Funktion** sieht generell folgendermaßen aus:

```
Rueckgabetyf Funktionsname (Parameterliste)
```

Der Funktionskopf enthält die Schnittstelle einer Funktion, wie sie sich nach außen, also gegenüber ihrem Aufrufer, verhält.

Wenn ein Programm aus mehreren Funktionen besteht, dann können diese Funktionen beim Aufruf Daten über Parameter sowie das Funktionsergebnis austauschen.



Der Funktionskopf enthält also den Namen der Funktion, die Liste der **Übergabeparameter** und den **Rückgabetyf**. Hier als Beispiel:

```
int summe(int n)
```

Die Funktion `summe()` erhält als Parameter die Obergrenze, bis zu der die Summe gebildet werden soll. Dieser Parameter n wird bei Aufruf des Unterprogramms vom Hauptprogramm passend gesetzt. In umgekehrter Richtung erwartet das Hauptprogramm nach Beendigung der Funktion ein Funktionsergebnis mit dem Typ `int`.

Dies stellt den Kopf der Funktion `summe()` dar. Der **Funktionsrumpf** ist der Teil mit den geschweiften Klammern. Er enthält die lokalen Definitionen der Variablen und die Anweisungen der Funktion, beispielsweise:

```
int summe(int n) {  
    int sum = 0;  
    for (int i = 1; i <= n; i = i + 1)  
        sum = sum + i;  
    return sum;  
}
```

Im Rumpf dieser Funktion wird zuerst die lokale Variable `sum` definiert. Dann folgen die Anweisungen, hier zuerst eine `for`-Schleife, in welcher in der Variablen `sum` die Summe berechnet wird. Wenn die Schleife beendet ist, enthält sie das gewünschte Ergebnis, das mittels der `return`-Anweisung an den Aufrufer zurückgegeben wird.

Vergleicht man den Aufbau der Funktion `main()` aus den vorherigen Kapiteln mit diesem allgemeinen Aufbau einer Funktion, so sieht man, dass die Funktion `main()` ganz genau diesem allgemeinen Aufbau einer Funktion folgt. Sie ist also auch einfach eine Funktion.

4.5.2 Aufruf von Funktionen

Eine Funktion kann ganz einfach nach folgendem Schema aufgerufen werden:

```
Funktionsname(Argumente)
```

So kann beispielsweise geschrieben werden:

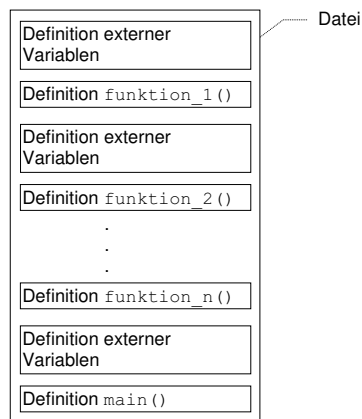
```
resultat = summe(10);
```

Dabei ist `10` das **Argument**, sprich der aktuelle Parameter der Funktion `summe()`. Das bedeutet, dass der formale Parameter `n` der Funktion `summe()` mit dem Wert `10` belegt wird und dass dann die Anweisungen des Funktionsrumpfes durchgeführt werden. Am Ende des **Funktionsaufrufes** kehrt das Programm an die Aufrufstelle zurück und das in der `return`-Anweisung zurückgegebene Funktionsergebnis kann wie jeder andere berechnete Wert verwendet werden – in diesem Falle wird das Funktionsergebnis in der Variablen `resultat` gespeichert.

4.6 Struktur einer Quelldatei in C

Bislang wurden nur kleine Programme betrachtet. Mit der Zeit wächst ein Programm jedoch in Funktionalität und Umfang, weswegen der Code dann auf mehrere Dateien verteilt wird. Dies erhöht die Übersichtlichkeit und erlaubt eine arbeitsteilige Entwicklung.

Eine Datei wird als „**Quelldatei**“ bezeichnet, auf Englisch „**source file**“. Eine einzelne Quelldatei in C besteht dabei hauptsächlich aus sogenannten „externen Variablen“ und Funktionsdefinitionen.



Externe Variablen werden auch „**globale Variablen**“ genannt. Extern bedeutet, dass sie außerhalb von Funktionen, also extern angelegt sind. Wird eine Variable außerhalb einer Funktion vereinbart, ist diese Variable innerhalb derselben Datei global sichtbar für all diejenigen Funktionen, die nach dieser Variablen definiert werden. Der Begriff „globale Variablen“ hat sich eingebürgert.

Da globale Variablen nach ihrer Definition überall sichtbar sind, können sie zur Kommunikation zwischen Funktionen eingesetzt werden. Nach allgemeinem Bewusstsein ist der Einsatz von globalen Variablen jedoch heutzutage verpönt, da sie die Prinzipien der geordneten Verkapselung von Daten durchbrechen. Sie bringen im Fehlerfall sehr viel Mühe mit sich, um die verursachende Funktion zu ermitteln, und daher ist die Kommunikation über Parameter und Funktionsergebnis vorzuziehen.



Globale (externe) Variablen werden ausführlich in Kapitel [→ 7.4.3](#) behandelt. Bis auf weiteres werden sie nicht weiter betrachtet.

Auch Funktionen sind extern, sprich, sie sind immer extern zu anderen Funktionen. Gewisse Compiler oder Erweiterungen erlauben zwar, Funktionen innerhalb von Funktionen zu definieren, was dann als eine sogenannte „nested function“ bezeichnet wird, der Standard definiert jedoch keine verschachtelten Funktionen.

Funktionsdefinitionen sind also stets extern. Eine Quelldatei besitzt üblicherweise mehrere davon.

Eine Funktion darf sich nicht über mehrere Dateien erstrecken, sondern muss komplett in einer einzigen Datei enthalten sein.



4.6.1 Prototypen und Reihenfolge der Namen

Da ein Compiler eine Quelldatei stets von oben nach unten durcharbeitet, ist die Reihenfolge der externen Variablen und Funktionen nicht unerheblich!

Alle Namen, die an einer Stelle im Programm benutzt werden, müssen zuvor dem Compiler bekanntgegeben worden sein, sonst kann der Compiler die entsprechenden Prüfungen nicht durchführen. Der Grund dafür ist, dass die Sprache C nur ein einziges Mal durch den zu compilierenden Quellcode durchläuft. Dies nennt sich „Single Pass Compiler“.



Bei einfachen Programmen ist dies nicht so ein Problem. Die Funktionsdefinitionen werden einfach in der Reihenfolge definiert, wie sie gebraucht werden.

Sehr schnell jedoch wird ein Programm komplexer, sodass die Einhaltung der Reihenfolge mühsam oder gar unmöglich wird. Wenn sich zwei Funktionen gegenseitig aufrufen, dann muss eine davon zuerst definiert werden. Das führt zu einem Problem.

Der Compiler muss die Schnittstelle einer Funktion kennen, damit er prüfen kann, ob sie korrekt aufgerufen wird.



In einem solchen Falle können sogenannte „**Funktions-Prototypen**“ eingesetzt werden. Genauer erläutert werden diese in Kapitel [→ 11.6](#), hier wird kurz das Prinzip erläutert:

Ein Funktions-Prototyp besteht aus dem Funktionskopf gefolgt von einem Strichpunkt. Mit einem solchen Prototyp wird dem Compiler die Aufrufschnittstelle einer Funktion bekannt gemacht, ohne sie explizit auszuprogrammieren.



Beispielsweise lautet der Prototyp der Funktion `summe()`:

```
int summe(int n);
```

Wenn diese Zeile irgendwo vor einem Aufruf steht, dann nimmt der Compiler an, dass diese Funktion irgendwo definiert werden wird und kennt gleichzeitig die Signatur derselben, sodass ein Aufruf der Funktion übersetzt werden kann.

Wenn eine Funktion keine Parameter erwartet, sollte die Parameterliste beim Prototyp mit dem Schlüsselwort `void` gekennzeichnet werden. Dies ist ein Überbleibsel aus alten Standards. Ab dem C23-Standard ist dieses zusätzliche `void` nicht mehr nötig. Der Compiler nahm früher an, dass bei leerer Parameterliste die Anzahl und der Typ der Parameter erst bei der tatsächlichen Definition der Funktion festgelegt wird.



4.6.2 Weitere Bestandteile eines Programms

Zu den Funktionen und externen Variablen eines Programms können noch Präprozessor-Anweisungen, globale Typ-Definitionen und Prototypen hinzukommen.



Prototypen werden üblicherweise im obersten Teil einer Datei hingeschrieben, sodass sie für sämtliche darauffolgenden Definitionen sichtbar sind. Da in einem umfangreicheren Programm die Anzahl an Prototypen sehr schnell anwächst, werden sie in sogenannten „Header-Dateien“ gesammelt und mittels der Präprozessor-Direktive `#include` eingebunden.

Header-Dateien bieten Schnittstellen für Code aus anderen Dateien an. Sie werden Header-Dateien genannt, da deren Inhalt normalerweise im Kopf einer Quelldatei, sprich vor den eigentlichen Definitionen, eingebunden wird. Header-Dateien können vom Standard gegeben (wie beispielsweise `<stdio.h>`), von Drittanbietern geliefert (wie beispielsweise bei der Grafikbibliothek OpenGL) oder selbst ausprogrammiert sein. Durch das Einbinden einer Header-Datei werden alle dort beschriebenen Prototypen, Typ-Deklarationen und mehr in das eigene Programm eingebunden.

Zu den Präprozessor-Anweisungen zählt auch die in Kapitel [→ 2.2.2](#) eingeführte `#define`-Anweisung, mit deren Hilfe symbolische Konstanten definiert werden können.

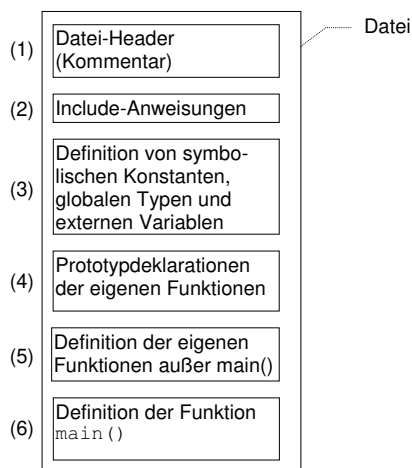
Das Typkonzept von C erlaubt die Vereinbarung eigener Typen wie etwa von Aufzählungs-Typen (`enum`) oder von Struktur-Typen (`struct`). Diese werden meist in mehreren Funktionen benutzt und müssen dann außerhalb dieser Funktionen – also extern – definiert werden. Oftmals werden auch sie in Header-Dateien verpackt.

4.6.3 Grober Aufbau einer Quelldatei

Im Laufe der Jahre hat sich folgendes, generelle Schema für eine **Quelldatei** in C rauskristallisiert:

- Oft beginnt eine Quelldatei mit einem aussagekräftigen **Kommentar**, um deutlich zu machen, was genau innerhalb der Datei ausprogrammiert wird. Auch werden häufig Anmerkungen zur rechtlichen Benutzung (Copyright) gegeben. Mittlerweile werden solche rechtlichen Angaben jedoch auch häufig erst am Ende der Datei hingeschrieben.
- Dann folgen die `#include`-Anweisungen, die vom Präprozessor bearbeitet werden. Es werden also gleich zu Beginn sämtliche benötigten Bibliotheken und anderweitige Deklarationen eingebunden.
- Es folgen symbolische Konstanten mit `#define`, globale Typvereinbarungen und (gegebenenfalls) globale Variablen sowie vereinzelte Prototypen von Funktionen. All das wird vor den eigenen Funktionen geschrieben.
- Nun folgen alle eigenen Funktionen. Durch die vorher bereits bekannt gemachten Prototypen können sich diese beliebig gegenseitig aufrufen.
- Die Funktion `main()` wird traditionell ans Ende der Datei geschrieben. Dadurch erspart man sich das Schreiben von Prototypen zu Beginn einer neuen Implementation.

Zusammenfassend ist eine C-Quelldatei dann wie im folgenden Bild aufgebaut:



4.6.4 Beispielprogramm für den Aufbau einer Quelldatei

Folgt man dieser vorgeschlagenen Struktur für das Beispiel der Summenberechnung, so sieht die Datei wie folgt aus:

summe.c

```
// Summenberechnung. Berechnet die Summe aller Zahlen
// von 1 bis MAX.

#include <stdio.h>

#define MAX 100

int addition(int a, int b);

int summe(int n) {
    int sum = 0;
    for (int i = 1; i <= n; i = i + 1)
        sum = addition(sum, i);
    return sum;
}

int addition(int a, int b) {
    return a + b;
}

int main(void) {
    int resultat = summe(MAX);
    printf("Die Summe von 1 bis %d ist %d\n", MAX, resultat);
    return 0;
}
```

Hier die Ausgabe des Programms:

```
Die Summe von 1 bis 100 ist 5050
```

Hier wurde zur Veranschaulichung des Prototyps eine Funktion `addition()` hinzugefügt.

5 Programmerzeugung und -ausführung



Der Quellcode eines Programms wird mit einem **Editor**, einem Werkzeug zur Erstellung von Texten, geschrieben und auf der Festplatte des Rechners unter einem Dateinamen als Datei abgespeichert.

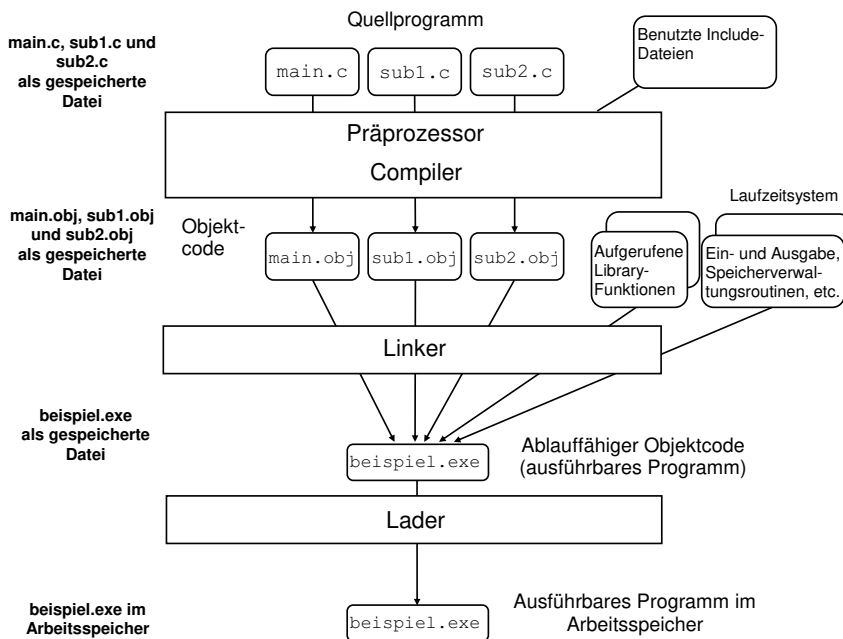


Da eine solche Datei Quellcode enthält, wird sie auch als **Quelldatei** (auf Englisch „**source file**“) bezeichnet.

Einfache Programme bestehen aus einer einzigen Quelldatei, komplexe aus mehreren Quelldateien.



Das folgende Bild zeigt einen Überblick über den Ablauf der Tätigkeiten, die durchzuführen sind, um ein C-Programm zu erzeugen und es auszuführen:



In diesem Kapitel werden alle Stationen dieses Diagramms ausführlich erläutert. Hier jedoch vorab die Kurzzusammenfassung:

Das gezeigte Beispielprogramm besteht aus drei separat compilierbaren Quelldatei `main.c`, `sub1.c` und `sub2.c`. Beim Compiler-Aufruf läuft in C vor dem eigentlichen Compiler als erstes der Präprozessor durch das Programm.

Zuerst muss der **Präprozessor** in den Quelldateien die Präprozessor-Anweisungen auflösen, bevor der Compiler die Dateien compilieren kann.



Durch Compilieren erhält man aus einer Quelldatei eine **Objektdatei**.



Das Auflösen der Präprozessor-Anweisungen geschieht bei den meisten Compilern automatisch, so dass kein Zwischenprodukt entsteht, sondern dass eine Objektdatei in einem einzigen Durchlauf aus einer Quelldatei resultiert.

Zu jeder Quelldatei wird durch den Compiler eine entsprechende Objektdatei erzeugt. Unter Windows wird normalerweise für Objekt-Dateien die Extension `.obj` verwendet, unter UNIX/Linux normalerweise die Extension `.o`.

Der Linker erzeugt aus den Objektdateien ein ausführbares **Programm**.



Ein ausführbares Programm hat unter Windows per Konvention die Extension `.exe`.

Ein ausführbares Programm kann mit Hilfe eines Laders beliebig oft zur Ausführung gebracht werden.



Im Folgenden wird die Funktionalität von Compilern, Linkern und Ladern genauer besprochen. Ferner werden Debugger und integrierte Entwicklungsumgebungen vorgestellt.

5.1 Compiler

Die Aufgabe eines **Compilers** (Übersetzers) für die Programmiersprache C ist, den Text (Quellcode) eines C-Programms in Maschinensprache zu wandeln.



Maschinensprache (auch „**Maschinencode**“ genannt) ist prozessorspezifischer Binärcode, welcher von einem speziellen Prozessor ohne weitere Übersetzung direkt verstanden und verarbeitet werden kann. Ein vollständig für den Prozessor übersetzter Programmablauf besteht somit grundsätzlich nur aus einer Folge von Nullen und Einsen und wird deswegen auch als das sogenannte „**binary**“ bezeichnet.



Da Maschinensprache die tiefstmögliche Art der Kommunikation mit dem Prozessor darstellt, wird eine Sprache wie C, die vom speziellen Prozessor abstrahiert, als „**höhere Programmiersprache**“ bezeichnet. Während die ersten Sprachen und Compiler in den 50er und 60er Jahren noch heuristisch entwickelt wurden, wurde die Spezifikation von Sprachen und der Bau von Compilern zunehmend formalisiert.

ALGOL 60 war die erste Sprache, deren Syntax formal definiert wurde. Als **Syntax** bezeichnet man die Schreibweise, also die Form und Struktur einer Sprache, im Deutschen etwa die Regeln für den Satzbau. Diese Definition der Syntax wurde mit Hilfe der sogenannten „Backus-Naur-Form“ (BNF) verfasst [NBB⁺63]. Die Backus-Naur-Form ist dabei eine sogenannte „Metasprache“, also eine Sprache, welche eine Sprache beschreibt.

Unabhängig von der Art der höheren Programmiersprache kann ein Compiler – bei Einhaltung bestimmter Regeln bei der Definition einer Sprache – grob in folgende 5 Bearbeitungsschritte gegliedert werden:

- Lexikalische Analyse
- Syntaktische Analyse
- Semantische Analyse
- Optimierungen
- Codeerzeugung

Die Zwischenergebnisse dieser Schritte werden während des Compilierens in Form von Zwischensprachen und ergänzenden Tabelleneinträgen intern gespeichert und an den nächsten Schritt weitergegeben.

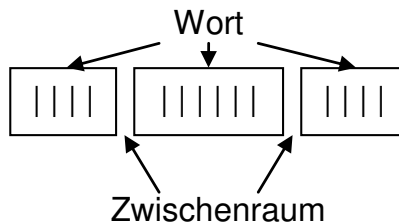
5.1.1 Lexikalische Analyse

Bei der lexikalischen Analyse wird versucht, in der Folge der Zeichen eines Quellcodes Einheiten der Sprache zu erkennen. Die lexikalische Analyse wird auf Englisch „Lexer“ genannt, auf Deutsch wird sie auch als „Scanner“ oder „Symbolentschlüsselung“ bezeichnet.



Dabei werden die kleinsten Einheiten einer Sprache ermittelt, die für sich selbst eine Bedeutung besitzen. Eine solche Einheit wird als ein „Wort“ bezeichnet und kann beispielsweise ein Name, ein Schlüsselwort, ein Operator, eine Zahl, etc. sein.

Folgendes Bild demonstriert das Erkennen von Wörtern. Die Striche sollen die einzelnen Zeichen einer Quelldatei darstellen.



Zwischenräume und Kommentare dienen dem Compiler dazu, zu erkennen, an welcher Stelle ein Wort zu Ende ist. Ansonsten haben sie keine Bedeutung für den Compiler und werden überlesen.

5.1.2 Syntaxanalyse

Für alle modernen Sprachen existiert ein Regelwerk, die sogenannte Grammatik. Die Grammatik einer Sprache legt formal die zulässigen Folgen von Symbolen (Wörtern) fest.

Im Rahmen der **Syntaxanalyse** wird geprüft, ob die bei der lexikalischen Analyse ermittelte Symbolfolge eines zu übersetzenden Programms zu der Menge der korrekten Symbolfolgen gehört.



Der Teil des Compilers, der die Syntaxanalyse durchführt, wird auch als „Parser“ bezeichnet.

5.1.3 Semantische Analyse

Die **semantische Analyse** versucht, die Bedeutung der durch die lexikalische Analyse erkannten Wörter herauszufinden und hält diese meist in Form eines Zwischencodes fest. Ein Zwischencode ist nicht mit der Maschinensprache einer realen Maschine vergleichbar, sondern auf einer relativ hohen Ebene angesiedelt. Er ist für eine hypothetische Maschine gedacht und dient einzig dazu, die gefundene Bedeutung eines Programms für die nachfolgenden Phasen eines Übersetzers festzuhalten.

Die Bedeutung in einem Programm bezieht sich im Wesentlichen auf dort vorkommende Namen. Also muss die semantische Analyse herausfinden, was ein Name, der im Programm vorkommt, bedeutet. Grundlage hierfür sind die Sichtbarkeits-, Gültigkeits- und Typregeln einer Sprache. Mehr dazu kann in Kapitel [→ 11.2](#) nachgelesen werden.

Jeder Name wird mit einer Bedeutung versehen, also an eine Deklaration des Namens im Programm gebunden. Eine Deklaration gibt im Programm den Typ eines Namens bekannt, beispielsweise ob es sich um eine Variable oder eine Funktion handelt, einen Integer- oder um einen Gleitpunkt-Typ.

Neben der Überprüfung, ob Namen im Rahmen ihrer Gültigkeitsbereiche verwendet werden, spielt die Überprüfung von Typverträglichkeiten bei Ausdrücken bei der semantischen Analyse eine Hauptrolle.



Ein wesentlicher Anteil der semantischen Analyse befasst sich also mit der Erkennung von Programmfehlern, die durch die Syntaxanalyse nicht erkannt werden konnten, wie beispielsweise die Addition von zwei Werten mit nicht verträglichem Typ. Nicht alle semantischen Regeln einer Programmiersprache können jedoch durch den Übersetzer geprüft werden.

Man unterscheidet zwischen der statischen Semantik (durch den Übersetzer prüfbar) und der dynamischen Semantik (erst zur Laufzeit eines Programms prüfbar). Die Prüfungen der dynamischen Semantik sind üblicherweise im sogenannten Laufzeitsystem realisiert (siehe Kapitel [→ 5.1.6](#)).



5.1.4 Optimierungen

In der **Optimierungsphase** wird versucht, den Code, so wie er jetzt vorliegt, zu verbessern, sowohl bezüglich des Speicherplatzverbrauchs als auch bezüglich der benötigten Rechenzeit.



Ein Compiler kann beispielsweise nicht benutzten Code ignorieren, sich repetierende Ausdrücke kombinieren oder Prüfungen eliminieren, wenn der zu testende Wert bereits zur Compilerzeit feststeht. Es gibt auch noch kompliziertere Optimierungen, die einen hohen Aufwand während der Übersetzung erfordern. Bei vielen Compilern können all diese Optimierungen beliebig konfiguriert werden.

5.1.5 Codeerzeugung

Während die lexikalische Analyse, Syntaxanalyse und semantische Analyse sich nur mit der Analyse des zu übersetzenden Quellcodes befassen, kommen bei der Codegenerierung die Rechnereigenschaften, nämlich die zur Verfügung stehende Maschinensprache und die Eigenschaften des Betriebssystems, ins Spiel.



Da bis zur semantischen Analyse die Rechnereigenschaften nicht berücksichtigt wurden, kann man die Ergebnisse dieses Schrittes auf verschiedenartige Rechner übertragen (portieren).

Im Rahmen der Codeerzeugung – auch Synthese genannt – wird der Zwischencode, der bei der semantischen Analyse erzeugt wurde, in Objektcode, also in die Maschinensprache des jeweiligen Zielrechners übersetzt.



Dabei müssen die Eigenheiten des jeweiligen Zielbetriebssystems beispielsweise für die Speicherverwaltung berücksichtigt werden. Soll der erzeugte Objektcode auf einem anderen Rechnertyp als der Compiler laufen, so wird der Compiler als „Cross-Compiler“ bezeichnet.

5.1.6 Laufzeitsystem

Ein **Laufzeitsystem** (auf Englisch „**runtime system**“) enthält alle Programmfragmente, die zur Ausführung irgendeines Programms einer Programmiersprache notwendig sind, für die aber gar nicht oder nur sehr schwer direkter Code durch den Compiler erzeugt werden kann, oder für die direkt erzeugter Code sehr ineffizient wäre.



Dazu gehören alle Interaktionen mit dem Betriebssystem, wie beispielsweise Ansteuerung und Abfrage von Hardware-Ports, Abfangen von Signalen und das Einbinden von dynamischen **Bibliotheken** zur Laufzeit. Auch gehören dazu Speicherverwaltungsroutinen für den Heap (siehe Kapitel [→ 18](#)).

In der Programmiersprache C werden die Ein-/Ausgabe-Operationen normalerweise über Bibliotheken angesprochen. Wie diese Bibliotheken implementiert sind, ist je nach System unterschiedlich, weswegen die Ein-/Ausgabe-Operationen in C nicht zum eigentlichen Laufzeitsystem gezählt werden können.

Zum Laufzeitsystem gehören im Allgemeinen aber alle Prüfungen der dynamischen Semantik, kurz eine ganze Reihe von Fehler Routinen mit der entsprechenden Anwenderschnittstelle. Dies beinhaltet beispielsweise den sogenannten „Speicherabzug“. Auf Englisch wird dies „**core dump**“ genannt im Andenken an die magnetischen Kern- (Core-) Speicher, die zu Beginn der Datenverarbeitung genutzt wurden.

Auch besondere Sprachkonzepte wie Threads (parallele Prozesse) oder Exceptions (Ausnahmen) werden in aller Regel ebenfalls im Laufzeitsystem realisiert. Threads sind eine optionale Erweiterung des C11-Standards, siehe Kapitel [→ 23](#). Exceptions gibt es erst in C++.

5.2 Linker

Wenn ein Programm in der Programmiersprache C aus mehreren Modulen (Dateien) besteht, dann entstehen durch das Compilieren auch mehrere Dateien in Maschinensprache. Diese Dateien werden „**Objektdateien**“ genannt. Die Extension dieser Dateien ist `.obj` unter Windows und `.o` unter Unix.

Der Maschinencode dieser Objektdateien ist noch nicht selbständig ablauffähig. Zum einen, weil gewisse Bibliotheksfunktionen noch fehlen, zum anderen, weil die Adressen von externen Funktionen und Variablen aus anderen Objektdateien (Querbezüge) noch nicht bekannt sind. Der Linker bindet diese Objektdateien, die aufgerufenen Bibliotheksfunktionen und das Laufzeitsystem zu einer ablauffähigen Einheit zusammen.

Ein „**Linker**“ (zu Deutsch „**Binder**“) hat die Aufgabe, den nach dem Compilieren vorliegenden Objektcode in ein auf dem Prozessor ausführbares **Programm** (auf Englisch „executable program“, oder kurz „**executable**“) zu überführen.



Ist beispielsweise ein Programm getrennt in einzelnen Dateien geschrieben und in einzelne Objektdateien übersetzt worden, so werden die Objektdateien vom Linker zusammengeführt. Durch den Linker werden alle benötigten Teile zu einem ablauffähigen Programm gebunden. Hierzu gehört auch das Laufzeitsystem, das durch den jeweiligen Compiler eingebaut wird.

Variablen und Funktionen, welche im Quellcode geschrieben stehen, werden als „Speicherobjekte“ bezeichnet. Hier handelt es sich nicht um Objekte im Sinne der Objektorientierung, sondern um zusammenhängende Speicherbereiche. Beim Compilieren werden diese Speicherobjekte an relativen Adressen innerhalb der jeweiligen Objektdatei abgelegt. Wenn mehrere Objektdateien zu einer einzigen Datei zusammengefügt werden, werden diese Adressen dann vom Linker jeweils von einer – immer gleichen – Anfangsadresse ausgehend angeordnet.

Werden in der übersetzten Programmeinheit externe Variablen oder Funktionen, beispielsweise aus anderen Programmeinheiten oder aus Bibliotheken, verwendet, so kann der Übersetzer für diese Objekte noch keine endgültigen Adressen einsetzen. Vielmehr vermerkt der Compiler im Objektcode, dass an bestimmten Stellen Querbezüge vorhanden sind, an denen noch die Adressen der externen Objekte eingefügt werden müssen. Das ist dann die Aufgabe des Linkers.

Hier zeigt sich der große Vorteil von Bibliotheken: Die Übersetzung von Quellcode benötigt viel Zeit. Objekt-Dateien enthalten den Code einer Bibliothek in einer bereits übersetzten Form, so dass sie nicht erneut compiliert werden müssen. Der Linker kann so eine vorcompilierte Bibliothek genau gleich wie jede andere Objektdatei verarbeiten und muss einfach nur noch die Querbezüge auflösen.

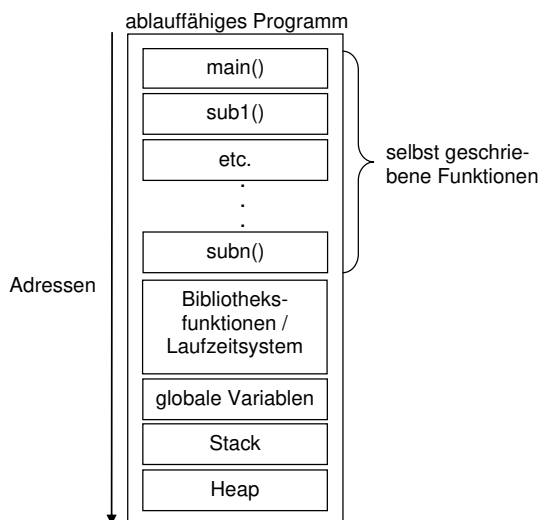
Ein Linker fügt die einzelnen Adressräume der Objektdateien, eingebundenen Standard-Bibliotheken und Funktionen des Laufzeitsystems so zusammen, dass sich die Adressen nicht überlappen, und löst die Querbezüge auf.



Hierzu stellt der Linker eine Tabelle her, welche alle Querbezüge (Adressen globaler Variablen, Einsprungadressen der Programmeinheiten) enthält. Damit können Referenzierungen von globalen Variablen oder von Funktionen aufgelöst werden. Durch den Linkvorgang wird ein einheitlicher Adressraum für das gesamte Programm hergestellt.

Das schlussendlich erstellte Programm hat unter Windows die Extension `.exe`, was für „executable program“ steht, auf Deutsch „ablauffähiges Programm“. Unter Unix gibt es keine Extension, jedoch wird auch dort das Programm als „executable“ bezeichnet. Eine ausführbare Datei wird unter Unix am sogenannten „executable-Flag“ erkannt.

Das folgende Bild zeigt als Beispiel den Adressraum eines ablauffähigen Programms:



5.3 Lader

Mit dem **Lader** (auf Englisch „**loader**“) wird das Programm in den Arbeitsspeicher des Computers geladen, wenn es gestartet wird.



Bei PCs mit modernem Speicherverwaltungssystem liegen die ablauffähigen Programme in Form eines verschiebbaren (auf englisch relocatable) Maschinencodes vor. Von einem verschiebbaren Maschinencode spricht man dann, wenn die Adressen des ablauffähigen Programms einfach von null an durchgezählt werden, ohne festzulegen, an welcher Stelle das Programm im Arbeitsspeicher liegen soll. Die Aufgabe des Laders besteht somit darin, den Code sowie die Daten an die richtigen Stellen im Arbeitsspeicher, die vom Betriebssystem bestimmt werden, abzulegen.

Der Lader lädt nicht nur den Code in den Arbeitsspeicher, sondern stellt auch sogenannte „Umgebungsvariablen“ dem Programm zur Verfügung und übernimmt Start-Argumente, welche an das Programm gesendet werden sollen. Mehr dazu kann in Kapitel [→ 17.1](#) nachgelesen werden.

Er setzt zudem das dynamische Laufzeitsystem auf, welches für die Speicherverwaltung, die Signalverarbeitung und das Laden dynamischer Bibliotheken zuständig ist.

Wenn das Programm vollständig geladen ist, wird es vom Lader gestartet, indem die `main()`-Funktion aufgerufen wird.



Die ursprünglichste Art, ein lauffähiges Programm zu starten, ist mithilfe einer **Konsole**. Unter Windows muss hierfür beispielsweise die „Eingabeaufforderung“ gestartet werden und dort der Programmname – gegebenenfalls mit einer Pfadangabe – eingegeben werden (Die Extension `.exe` kann bei der Eingabe des Dateinamens auch weggelassen werden). Unter UNIX wird das gleiche Kommando in einer Shell abgesetzt.

```
C:\MeineProgramme\hello.exe
```

Das Programm läuft sodann in der Umgebung (auf Englisch „environment“) dieser Konsole. Das bedeutet, dass Ausgaben, die das Programm bei der Ausführung erzeugt, ebenfalls in der Konsole ausgegeben werden.



```
C:\>C:\MeineProgramme\hello
Hello, world!
C:\>
```

Das Laden, Starten und Ausführen eines Programms kann jedoch in verschiedenen Umgebungen erfolgen:

- Ganz bequem über ein Tippen mit dem Finger auf den Handybildschirm oder mittels Doppelklick mit einer Maus auf ein Programmsymbol.
- Über ein Script, welches beispielsweise ein Systemadministrator bereitstellt. Damit werden häufig wiederkehrende Ablaufpläne wie beispielsweise Installationen vorgenommen.
- Über eine integrierte Entwicklungsumgebung, häufig sogar unter der Kontrolle eines Debuggers.

In Umgebungen, welche kein hochentwickeltes Betriebssystem besitzen, kann das Laden eines Programms eine durchaus nicht triviale Angelegenheit sein, beispielsweise wenn ein Programm von einem Entwicklungs-PC erst auf ein Steuergerät geladen werden muss, um dort ausgeführt zu werden.

5.4 Integrierte Entwicklungsumgebungen und Debugger

Um die Programmentwicklung komfortabler zu gestalten, wird ein Programm heutzutage üblicherweise auf einer sogenannten „**Integrierten Entwicklungsumgebung**“ (auf Englisch „integrated development environment“, kurz **IDE**) entwickelt.



In einer integrierten Entwicklungsumgebung werden alle Aspekte der Programmierung in einem einzigen (integrierten) Tool verpackt. Dazu gehört normalerweise ein komfortabler Editor zur Eingabe der Programmtexte, Zugriff auf ein Quellcode-Versionierungs-System, ein Debugger und verschiedene Analyseprogramme, um beispielsweise den Speicherplatzverbrauch oder die Performance eines Programmes zu messen.

Um das Programm zu erstellen, beinhaltet eine Entwicklungsumgebung zudem mindestens einen Präprozessor, Compiler und Linker. Zusätzlich gibt es jedoch heutzutage auch die Möglichkeit, beliebige Skripte vor oder nach den einzelnen Stationen auszuführen. Dabei wird beispielsweise die gesamte Projektstruktur automatisch aufgesetzt, es werden Dateien kopiert, Datenbanken aufgesetzt, Zugriffsrechte von Dateiordnern verändert, grafische Benutzeroberflächen automatisch erstellt usw. Der gesamte Prozess der Programmerstellung wird als „**build**“ bezeichnet.

Darüber hinaus ist in einer Integrierten Entwicklungsumgebung oftmals eine Projektverwaltung integriert. Dabei kann ein Projekt aus mehreren Programmmodulen (Dateien) bestehen, welche getrennt entwickelt werden. Um das ausführbare Programm zu erzeugen, müssen alle Module durch den Compiler getrennt übersetzt und durch den Linker gebunden werden. Die Projektverwaltung sorgt dafür, dass bei Änderungen eines einzelnen Moduls nur dieses Modul übersetzt und mit den anderen, nicht geänderten Modulen neu gebunden wird.

Nach dem Laden wird das erstellte Programm normalerweise direkt gestartet. Formal korrekte Programme können jedoch logische Fehler enthalten, die sich erst zur Laufzeit und leider oftmals erst unter bestimmten Umständen während des Betriebs eines Programms herausstellen. Um diese Fehler analysieren und beheben zu können, möchte man den Ablauf des Programms während der Fehleranalyse exakt beobachten. Dazu dienen Hilfsprogramme, die als „Debugger“ bezeichnet werden.

Mit Hilfe eines **Debuggers** kann man Programme laden und gezielt starten, an beliebigen Stellen anhalten (sogenannte „Haltepunkte“ setzen), Programme nach dem Anhalten fortsetzen, Programme Schritt für Schritt ausführen sowie Speicherinhalte anzeigen und gegebenenfalls verändern.



Der Name „Debugger“ kommt vom Englischen „to debug“, was auf Deutsch soviel heißt wie „entwanzen“. Angeblich kommt dieser Begriff aus der Zeit der Lochstreifen, als nach einem fehlerhaften Programmlauf festgestellt wurde, dass beim Durchlaufen der Lochstreifen durch die Lesewalze ein Käfer genau an der Stelle eines Loches eingeklemmt und zerdrückt wurde, womit das Loch verdeckt wurde und das Lesegerät anstelle einer 1 eine 0 las. So entstand der umgängliche Name „Bug“. Heutzutage wird der Begriff „Debugging“ ganz allgemein benutzt im Sinne von Fehler suchen und entfernen.

Debugger ersetzen nicht den systematischen Test von Programmen zum Detektieren von Fehlern. Sie helfen jedoch dabei, zu aufgetretenen Fehlern die Ursache in einem Programm zu lokalisieren.

6 Lexikalische Konventionen



„Lexikalisch“ bedeutet „ein Wort (eine Zeichengruppe) betreffend“, ohne den Textzusammenhang (Kontext), in dem dieses Wort steht, zu berücksichtigen. Eine lexikalische Einheit ist eine zusammengehörige Zeichengruppe beziehungsweise ein Wort eines Programmtextes.



Der **Quellcode** eines C-Programms besteht aus lexikalischen Einheiten und Trennern wie zum Beispiel Leerzeichen. Jede lexikalische Einheit darf nur Zeichen aus dem Zeichenvorrat (Zeichensatz) der Sprache umfassen.



Im Folgenden werden die lexikalischen Konventionen besprochen, also gewissermaßen die Rechtschreibregeln von C.

6.1 Zeichenvorrat von C

Um ein Programm in der Programmiersprache C zu verfassen, müssen die Anweisungen an den Computer in Quelldateien geschrieben werden. Diese Dateien müssen einen sogenannten „**Basiszeichensatz**“ unterstützen, sprich, ein minimales Set an Zeichen, welches von einem Compiler verarbeitet werden kann.

Der Basiszeichensatz für Quelldateien in C enthält die folgenden Zeichen: Die 26 Groß- und Kleinbuchstaben des lateinischen Alphabets, die 10 arabischen Dezimalziffern, sowie 29 Interpunktionszeichen:

```
A B C D E F G H I J K L M N O P Q R S T U V W X Y Z  
a b c d e f g h i j k l m n o p q r s t u v w x y z  
0 1 2 3 4 5 6 7 8 9  
! " # % & ' ( ) * + , - . / : ; < = > ? [ \ ] ^ _ { | } ~
```

Des Weiteren definiert der Basiszeichensatz das Leerzeichen, den Tabulator sowie ein paar wenige Steuerzeichen für Zeilenumbruch und Ende der Quelldatei. Zudem wird das sogenannte „**Nullzeichen**“ `'\0'` definiert, bei dem alle Bits null sind. Dieses wird beispielsweise verwendet, um ein Stringende zu markieren.

Ergänzende Information Die elektronische Version dieses Kapitels enthält Zusatzmaterial, auf das über folgenden Link zugegriffen werden kann https://doi.org/10.1007/978-3-658-45209-4_6.

Die Zeichen des Basiszeichensatzes werden durch ein einziges Byte dargestellt und reichen für die Programmierung in C vollkommen aus.



Während der Ausführung des Programmes werden jedoch noch mehr Zeichen benötigt. Insbesondere spezifiziert C zusätzliche Steuerzeichen wie Alarm, Backspace oder Zeilenvorschub. Diese haben in einer Quelldatei keine Bedeutung, jedoch während der Ausführung. Es wird somit zwischen „**Quellzeichensatz**“ und „**Ausführungszeichensatz**“ unterschieden.

C definiert sowohl einen Quell- als auch einen Ausführungszeichensatz. Beide definieren einen jeweiligen Basiszeichensatz, sprich ein minimales Set an Zeichen, welche entweder während der Compilierung oder während der Laufzeit verfügbar sein müssen.



Ursprünglich stellte der Zeichensatz **ISO 646** den Basiszeichensatz von C dar. Durch die fortlaufende Standardisierung hat sich daraus der **ASCII-Zeichensatz** gebildet, welcher heute als Basis für die meisten Programmiersprachen gilt. Mehr dazu kann im Anhang [→ D](#) nachgelesen werden.

Im Lauf der Jahre wurden die Ansprüche an Zeichensätze immer größer. Während für das Schreiben von Quellcode bis heute der Basiszeichensatz ausreichend ist, genügte selbiger während der Ausführung schon bald nicht den Anforderungen. Beispielsweise werden für Ausgaben auf dem Bildschirm zusätzliche Zeichen wie äöü benötigt oder gar chinesische Schriftzeichen.

Ein Zeichensatz, welcher solche erweiterten Zeichen enthält, wird als „erweiterter Zeichensatz“ bezeichnet.

Ein erweiterter Zeichensatz umfasst nebst dem Basiszeichensatz noch zusätzliche implementations- oder länderspezifische Zeichen, beispielsweise äöü.



Erst durch die Einführung dieser erweiterten Zeichensätze wurde es überhaupt möglich, Programme für den internationalen Markt zu erstellen. Um zu ermöglichen, dass solche internationale Zeichen überhaupt in den Programmcode geschrieben werden können, erlaubt C somit auch erweiterte Zeichensätze für den Quellcode.

Sowohl Quell- als auch Ausführungszeichensatz enthalten jeweils den zugehörigen Basiszeichensatz, können aber auch stets ein erweiterter Zeichensatz sein. Quell- und Ausführungszeichensatz müssen dabei nicht gleich sein.



Heutzutage hat sich sowohl für Quell- als auch für den Ausführungszeichensatz **Unicode** und insbesondere die **UTF-8-Codierung** durchgesetzt. Unicode beinhaltet den für C erforderlichen Basiszeichensatz für Quelldateien und Laufzeit, erweitert ihn jedoch um rund 4 Millionen Zeichen.

Wie diese Vielzahl an Zeichen während des Programmierens angesprochen werden kann, wird in Kapitel [→ 6.5.6](#) erläutert.

6.2 Lexikalische Einheiten

In der Sprache C haben viele Zeichen des Quellzeichensatzes je nach Situation eine andere Bedeutung. Insbesondere Interpunktionszeichen wie Komma, Minuszeichen oder Klammern kommen an den unterschiedlichsten Stellen vor. Die Aufgabe des Compilers ist es, diese Zeichen gemäß genau festgelegten syntaktischen Regeln zu lesen und ihnen eine Bedeutung (Semantik) zuzuordnen. Siehe auch Kapitel [→ 5.1](#).

Ein Programm besteht für einen Compiler zunächst nur aus einer Folge von Bytes. Die erste Phase des Compilierlaufs besteht aus einer lexikalischen Analyse (siehe Kapitel [→ 5.1.1](#)), die der sogenannte Scanner durchführt.

Der Scanner hat unter anderem die Aufgabe, Zeichengruppen, welche eine lexikalische Einheit bilden, zu finden. Eine solche Einheit wird auch „token“ genannt.

Eine lexikalische Einheit wird gefunden, indem man die Trenner, die sie begrenzen, findet. Trenner sind Zwischenraum (Whitespace-Zeichen), Operatoren und Interpunktionszeichen.



Zu den **Whitespace-Zeichen** gehören Leerzeichen, Tabulator, Zeilentrenner sowie Kommentare.



Kommentare zählen auch zu den Whitespace-Zeichen. Das ist zunächst etwas verwirrend, denn in einem Kommentar steht ja tatsächlich etwas. Nach dem Präprozessorlauf sind die Kommentare jedoch entfernt und an ihre Stellen wurden Leerzeichen eingesetzt. Wie Kommentare geschrieben werden, kann in Kapitel [→ 2.2.1](#) nachgelesen werden.

Zwischen zwei aufeinanderfolgenden lexikalischen Einheiten kann eine beliebige Anzahl an Whitespace-Zeichen eingefügt werden. Damit hat man die Möglichkeit, ein Programm optisch so zu gestalten, dass die Lesbarkeit verbessert wird.

Auch Operatoren und Interpunktionszeichen sind Trenner. Für den Compiler ist A&&B lesbar (&& ist der logische UND-Operator zwischen A und B). Um die Lesbarkeit von Code zu steigern, empfiehlt es sich, nicht alleine auf die Trenneigenschaft der Operatoren zu bauen, sondern nach jeder lexikalischen Einheit Leerzeichen einzugeben. Im genannten Beispiel also besser A && B schreiben!

Die lexikalischen Einheiten werden sodann vom Parser gemäß der Syntax der Sprache geprüft (siehe Kapitel [→ 5.1.2](#)) und grob in folgende Kategorien unterteilt:

- Reservierte Wörter (Schlüsselwörter)
- Bezeichner
- Literale Konstanten
- Operatoren und Interpunktionszeichen

Auf diese Kategorien wird in den folgenden Kapiteln im Einzelnen eingegangen.

6.3 Reservierte Wörter (Schlüsselwörter)

Die Namen von **Schlüsselwörtern** (auf Englisch „**keywords**“) sind reserviert. Die Bedeutung dieser Schlüsselwörter ist von der Programmiersprache festgelegt und kann nicht verändert werden.



Die Namen von Schlüsselwörtern dürfen nicht als Bezeichner von Objekten des Programms, also beispielsweise von Variablen und Funktionen oder selbst definierten Datentypen, verwendet werden.



Eine vollständige Erklärung dieser Schlüsselwörter kann erst in den späteren Kapiteln erfolgen. Im Folgenden werden alle Schlüsselwörter von C kurz aufgelistet:

asm	Einfügen von Assemblercode
auto	Speicherklassen-Bezeichner
break	Zum Herausspringen aus Schleifen oder der switch-Anweisung
case	Auswahl-Fall in switch-Anweisung
char	Typ-Bezeichner
const	Qualifikator für Typangabe
continue	Fortsetzungsanweisung
default	Standardeinsprungmarke in switch-Anweisung
do	Teil einer Schleifen-Anweisung
double	Typ-Bezeichner
else	Einleitung einer Alternative
enum	Aufzählungs-Typ-Bezeichner
extern	Speicherklassen-Bezeichner
float	Typ-Bezeichner
for	Schleifenanweisung
goto	Sprunganweisung
if	Beginn einer bedingten Anweisung
inline	Funktions-Spezifikator zur Spezifikation von Inline-Funktionen
int	Typ-Bezeichner
long	Typ-Modifikator beziehungsweise Typ-Bezeichner
register	Speicherklassen-Bezeichner

<code>restrict</code>	Qualifikator für Typangabe zur Einschränkung eines Zugangs zu einem Objekt über einen bestimmten Pointer
<code>return</code>	Rücksprung-Anweisung
<code>short</code>	Typ-Modifikator beziehungsweise Typ-Bezeichner
<code>signed</code>	Typ-Modifikator beziehungsweise Typ-Bezeichner
<code>sizeof</code>	Operator zur Bestimmung der Größe von Variablen, Typen und Konstanten
<code>static</code>	Speicherklassen-Bezeichner
<code>struct</code>	Strukturvereinbarung
<code>switch</code>	Auswahanweisung
<code>typedef</code>	Typnamenvereinbarung
<code>union</code>	Datenstruktur mit Alternativen
<code>unsigned</code>	Typ-Modifikator beziehungsweise Typ-Bezeichner
<code>void</code>	Typ-Bezeichner
<code>volatile</code>	Qualifikator für Typangabe
<code>wchar_t</code>	Typ-Bezeichner
<code>while</code>	Schleifenanweisung
<code>_Alignas</code>	Alignment-Spezifikator
<code>_Alignof</code>	Operator zur Bestimmung des Alignments von Typen
<code>_Atomic</code>	Zur Deklaration von atomaren Variablen
<code>_Bool</code>	Typ-Bezeichner
<code>_Complex</code>	Verwendet zur Spezifikation komplexer Datentypen
<code>_Generic</code>	Zur Simulation von Overloading
<code>_Imaginary</code>	Imaginäre Einheit
<code>_Noreturn</code>	Funktions-Spezifikator für eine Funktion, die nicht zum Aufrufer zurückkehrt
<code>_Static_assert</code>	Für statische Prüf-Ausdrücke
<code>_Thread_local</code>	Speicherklassen-Bezeichner für eine Variable pro Thread

Einige dieser Schlüsselwörter wurden erst in neueren Standards eingeführt. Des Weiteren fehlen in dieser Liste die Präprozessor-Direktiven sowie implementationsabhängige Schlüsselwörter, welche je nach Compiler zusätzlich definiert sein können.

6.4 Bezeichner

Ein **Bezeichner** (auf Englisch „**identifier**“) ist nichts anderes als ein Name, welcher im Quellcode einer bestimmten Einheit gegeben wird. Häufig wird auch der Begriff „**Symbol**“ verwendet.

Ein Bezeichner, besteht aus einer Zeichenfolge von Buchstaben und Ziffern, die mit einem Buchstaben beginnt. In C zählt auch der Unterstrich `_` zu den Buchstaben.



Bezeichner werden in C an verschiedenen Stellen verwendet:

- Variablennamen
- Funktionsnamen
- Etiketten (auf Englisch „tags“) von Strukturen, Unionen, Bitfeldern und Aufzählungs-Typen
- Komponenten von Strukturen
- Alternativen von Unionen
- Aufzählungs-Konstanten
- Typnamen (typedef-Namen)
- Marken
- Makronamen (symbolische Konstanten)
- Makroparameter

Hierbei ist zu beachten, dass ein Parser streng zwischen Groß- und Kleinbuchstaben unterscheidet. Der englische Ausdruck hierfür ist „**case sensitive**“. Bezeichner, die sich durch Groß- und Kleinschreibung unterscheiden, stellen verschiedene Namen dar. So ist beispielsweise der Name `alpha` ein anderer Name als `Alpha`.

Bezeichner können heutzutage beliebig lange sein. Zudem sind in modernen Compilern ab dem C99-Standard auch die Zeichen des erweiterten Zeichensatzes (beispielsweise ä, ö, ü, ß) erlaubt. In manchen Programmen finden sich gar Emojis als Bezeichner.

Ein Bezeichner muss zwingend mit einem Klein- oder Großbuchstaben beginnen!



Namen, die mit einem Unterstrich _ oder zwei Unterstrichen beginnen, sind zwar erlaubt, sollten gemäß Standard jedoch nicht verwendet werden, da sie für systemspezifische Bibliotheksfunktionen reserviert sind, was zu unerwarteten Konflikten führen kann.



Einige Beispiele für zulässige und unzulässige Namen:

querSumme	Zulässig
quer_summe	Zulässig, Unterstrich ist ein Buchstabe
quer-Summe	Unzulässig, Bindestrich ist kein Buchstabe
8Summe	Unzulässig, beginnt mit einer Ziffer
_summe	Zulässig aber nicht empfohlen, reserviert für Sprache und Compiler.
quärSumme	Zulässig, aber erweitertes Zeichen. Zudem falsch geschrieben.
Quer😊	Zulässig, aber erweitertes Zeichen.

6.5 Literale Konstanten

Es gibt verschiedene Arten von **literalen Konstanten**, wobei jede literale Konstante einen definierten Datentyp besitzt:

- Integer-Konstanten
- Gleitpunkt-Konstanten
- Zeichen-Konstanten
- Konstante Strings

6.5.1 Integer-Konstanten

Integer-Konstanten sind Zahlen wie 1234. Sie sind vom Typ `int`.



Integer-Konstanten lassen sich in verschiedenen Zahlensystemen darstellen.

Außer der gewöhnlichen Abbildung im Dezimalsystem (Basis 10) gibt es in C noch die Darstellungsform im Hexadezimalsystem (Basis 16) und im Oktalsystem (Basis 8). Ab dem Standard C23 ist zudem die Darstellung im Binärsystem (Basis 2) erlaubt.



Die genannten Zahlensysteme und die Vorgehensweise bei der Umrechnung von einer Darstellung in eine andere werden im Anhang  C ausführlich erklärt. Hier wird nur die Schreibweise und Interpretation behandelt.

Eine **Dezimalzahl** besteht aus den Dezimalziffern 0, 1, ... 9, also beispielsweise 72656. Die erste Ziffer darf dabei nicht 0 sein. Wenn die erste Ziffer 0 ist, wird die Konstante als eine Oktalzahl interpretiert, welche nur die Ziffern 0 bis 7 beinhalten darf. **Oktalzahlen** werden heutzutage nur noch selten benutzt.

In der Programmierung besonders wichtig sind **Hexadezimalzahlen**. Für die zusätzlichen Ziffernwerte 10 bis 15 verwendet man in C die Buchstaben a bis f oder A bis F. Dabei ist der Wert von a oder A gleich 10, der Wert von b oder B gleich 11, ... Um eine Zahl als Hexadezimalzahl zu kennzeichnen, muss sie mittels 0x oder 0X eingeleitet werden, beispielsweise 0x6aff33b1.

Binärzahlen sind erst ab dem Standard C23 erlaubt. Sie verwenden nur die beiden Ziffern 0 und 1 und müssen ähnlich wie Hexadezimalzahlen mittels 0b oder 0B eingeleitet werden. Beispielsweise 0b01100010.

Integer-Konstanten sind in C streng genommen stets positiv. Benötigt man einen negativen Wert, so schreibt man in C einfach den einstelligen Minus-Operator davor, wie beispielsweise -85.

Der Datentyp einer Integer-Konstante ist grundsätzlich int. Dieser Typ ist implementationsabhängig und kann somit je nach Umgebung unterschiedlich groß werden. Entsprechend müssen manchmal Konstanten mittels einem speziellen **Suffix** markiert werden, wenn sie sehr große Zahlen darstellen.

Sehr große Integer müssen manchmal mit dem Suffix ll oder LL markiert werden. Beispielsweise 1000000000000ll



Das Suffix ll steht für „long long“. Dies bezeichnet eine Typenerweiterung des int-Typs, welche heutzutage nicht mehr so verwendet werden sollte. Im Anhang [→ E](#) wird genauer darauf eingegangen.

Ein weiteres Suffix, welches manchmal für Integer-Konstanten verwendet wird ist u oder U, was für unsigned steht. Damit wird eine Konstante explizit als vorzeichenlos markiert.

Ab dem Standard C23 können Zahlen auch mit Trennzeichen versehen werden. Mittels des Apostrophs ' können so große Zahlen lesbarer gemacht werden. Es gibt keine Regeln, wie groß die Zifferngruppen sein sollten, die mittels der Trennzeichen entstehen – der Compiler überliest sie einfach. Für Dezimalzahlen empfiehlt es sich somit beispielsweise, jeweils nach drei Ziffern zu trennen, für Hexadezimalzahlen ist eine Trennung nach zwei, vier oder acht Ziffern möglicherweise sinnvoller.

Die verschiedenen int-Datentypen werden in Kapitel [→ 7.2](#) genauer behandelt.

Beispiele für Integer-Konstanten sind:

14	int-Konstante in Dezimaldarstellung mit dem Wert 14 dezimal
014	int-Konstante in Oktaldarstellung mit dem Wert 12 dezimal
0x14	int-Konstante in Hexadezimaldarstellung mit dem Wert 20 dezimal
14l	long long-Konstante in Dezimaldarstellung mit dem Wert 14
14u	unsigned-Konstante in Dezimaldarstellung mit dem Wert 14
0x14ull	unsigned long long-Konstante in Hexadezimaldarstellung
14'633'165	Große Zahl mit Tausendertrennzeichen (erst ab C23)

6.5.2 Gleitpunkt-Konstanten

Gleitpunkt-Konstanten (Fließkomma-Konstanten) haben einen **.** (**Dezimalpunkt**) oder ein E beziehungsweise e oder beides. Entweder der ganzzahlige Anteil vor dem Punkt oder der Dezimalbruch-Anteil nach dem Punkt darf fehlen, aber nicht beide zugleich.

Beispiele für Gleitpunkt-Konstanten sind:

300.0 300. .3 3e2 3.e2 .3e3 .3E3

Der Teil einer Gleitpunkt-Zahl vor dem E beziehungsweise e ist die sogenannte „**Signifikante**“ (früher „**Mantisse**“), der Teil dahinter der sogenannte „**Exponent**“. Wird ein Exponent angegeben, so ist die Signifikante mit 10^{Exponent} zu multiplizieren.

$$3e2 = 3 * 10^2 = 300$$

Eine Gleitpunkt-Konstante hat den Typ double. Durch die Angabe eines optionalen Typ-Suffixes f oder F wird der Typ der Konstanten zu float.



So ist 10.3 vom Typ double, 10.3f vom Typ float.

Weitere Informationen über die Suffixe können im Anhang → E nachgelesen werden.

6.5.3 Zeichen-Konstanten

Eine **Zeichen-Konstante** ist ein Zeichen eingeschlossen in einfachen Hochkommas, beispielsweise 'A'. Der Wert der Zeichen-Konstanten ist gegeben durch den numerischen Wert des Zeichens im aktuellen Ausführungszeichensatz.

Obwohl eine Zeichen-Konstante vom Compiler im Arbeitsspeicher als char-Typ, das heißt als ein Byte, abgelegt wird, ist der Typ einer Zeichen-Konstanten, auf die in einem Programm zugegriffen wird, der Typ int.



Mit Zeichen-Konstanten kann man rechnen wie mit Integer. So hat beispielsweise das Zeichen '0' im ASCII-Zeichensatz den Wert 48. Zeichen-Konstanten werden aber meist gebraucht, um Zeichen zu vergleichen. Schreibt man die Zeichen als Zeichen-Konstanten und nicht als Integer, so ist man bei den Vergleichen unabhängig vom verwendeten Zeichensatz des Rechners.

Es gibt auch Zeichen-Konstanten mit mehreren Zeichen innerhalb der einfachen Hochkommas, beispielsweise 'A2B2'. Der Standard spezifiziert, dass der Wert eines solchen Literals implementationsabhängig sei. Häufig werden ebensolche Konstanten mit zwei Zeichen als 16-Bit-Integer-Zahl und solche mit 4 Zeichen als 32-Bit-Integer-Zahl interpretiert. Dass dieses Verhalten jedoch plattformübergreifend funktioniert, ist nicht garantiert.

6.5.4 Konstante Strings

Konstante Strings sind Folgen von Zeichen, die in Anführungszeichen eingeschlossen sind. Die Anführungszeichen sind nicht Teil der Strings, sondern begrenzen sie nur.



Beispiele für konstante Strings sind etwa "Kernighan" oder "Ritchie". Ein konstanter String wird intern dargestellt als ein Array von Zeichen. Arrays werden in Kapitel [→ 8.3](#) eingeführt. Am Schluss des Arrays wird vom Compiler ein zusätzliches **Nullzeichen**, das Zeichen '\0', angehängt, um das Stringende zu charakterisieren. Das folgende Bild gibt ein Beispiel:

'R'	'i'	't'	'c'	'h'	'i'	'e'	'\0'
-----	-----	-----	-----	-----	-----	-----	------

Stringverarbeitungsfunktionen benötigen das Zeichen `'\0'`, damit sie das Stringende erkennen. Deshalb muss bei der Speicherung von Strings stets ein Speicherplatz für das Nullzeichen vorgesehen werden. So stehen in dem String `"Hallo Welt"` zwischen den Anführungszeichen 10 Zeichen (inklusive Leerzeichen). Für die Speicherung dieses Strings werden 11 Zeichen benötigt (10 Zeichen + Nullzeichen).

Eine Zeichen-Konstante `'a'` und ein String `"a"` mit einem einzelnen Zeichen sind zwei ganz verschiedene Dinge. `"a"` ist ein Array aus den Zeichen `'a'` und einem Nullzeichen `'\0'`.

Befindet sich das Zeichen `'\0'` innerhalb eines Strings, so wird von einer Stringverarbeitungsfunktion an dieser Stelle das Stringende erkannt und der Rest des Strings wird nicht gelesen.



Stehen in einem Quellprogramm mehrere Strings hintereinander, wie beispielsweise `"Hallo " "Welt"`, so erzeugt der Präprozessor daraus durch Verkettung einen einzigen String. Dabei ist dann nur am Ende das Zeichen `'\0'` angehängt.



6.5.5 Ersatzdarstellungen

Gewisse Zeichen können nicht so einfach in Zeichen-Konstanten oder Strings eingegeben werden. Beispielsweise ist das Zeichen `'` für Zeichen-Konstanten nicht verwendbar, da es das Ende der Zeichen-Konstante markiert. Genauso kann das Zeichen `"` für Strings nicht benutzt werden. Auch Kontrollzeichen wie das Nullzeichen oder ein Zeilenende sind nicht so einfach in den Quellcode zu schreiben.

Hierfür existieren sogenannte **Ersatzdarstellungen**, auf Englisch auch „**escape sequences**“ genannt. Ersatzdarstellungen werden stets mit Hilfe eines **Backslash** `\` (Gegenschrägstrich) konstruiert.

Mit Ersatzdarstellungen kann man Steuerzeichen oder Zeichen, die auf dem Eingabegerät nicht vorhanden oder nur umständlich zu erhalten sind, angeben.



Die Ersatzdarstellung für einen Zeilentrenner beispielsweise ist `\n`. Das `n` ist von Newline abgeleitet. `\n` ist ein Steuerzeichen, welches zur Laufzeit des Programmes den Text-Cursor an den Beginn der nächsten Zeile setzt.

Ersatzdarstellungen werden immer als mehrere Zeichen in den Quellcode geschrieben, werden aber vom Compiler in ein einziges Zeichen umgewandelt.

Das erste Zeichen ist immer ein Backslash. Die weiteren Zeichen legen die Bedeutung fest.



Die folgende Tabelle zeigt die Ersatzdarstellungen in C:

Ersatz	Bedeutung	Zeichen
<code>\0</code>	Nullzeichen	<code>0x0</code>
<code>\a</code>	Klingelzeichen	<code>0x7</code>
<code>\b</code>	Backspace	<code>0x8</code>
<code>\t</code>	Tabulatorzeichen	<code>0x9</code>
<code>\n</code>	Zeilenende-Zeichen	<code>0xa</code>
<code>\v</code>	Vertikal-Tabulator	<code>0xb</code>
<code>\f</code>	Seitenvorschub (Form Feed)	<code>0xc</code>
<code>\r</code>	Wagenrücklauf	<code>0xd</code>
<code>\\</code>	Gegenschrägstrich (Backslash)	<code>\</code>
<code>\?</code>	Fragezeichen	<code>?</code>
<code>\'</code>	Einfaches Hochkomma	<code>'</code>
<code>\"</code>	Anführungszeichen (doppeltes Hochkomma)	<code>"</code>
<code>\ooo</code>	oktaler Wert	
<code>\xhh</code>	hexadezimaler Wert	
<code>\uhhhh</code>	Unicode Zeichen mit 16 Bits	
<code>\Uhhhhhhh</code>	Unicode Zeichen mit 32 Bits	

Die Oktal- und Hexadezimalddarstellungen werden heutzutage kaum mehr genutzt und werden hier nicht weiter behandelt. Sie wurden größtenteils verdrängt durch die Ersatzdarstellungen `\uhhhh` und `\Uhhhhhhh`. Diese erlauben es, Zeichen explizit im Unicode-Format angeben zu können. Auf Unicode und diese Ersatzdarstellungen wird im nächsten Kapitel genauer eingegangen.

6.5.6 Der Unicode in C

In diesem Kapitel werden verschiedene Zeichencodierungen angesprochen. Da das Thema äußerst umfangreich ist, kann hier nur das wichtigste beschrieben werden. Es wird bereits jetzt auf Anhang  D verwiesen, wo eine Vertiefung zum **Unicode** und zu den Codierungen **UTF-8**, **UTF-16** und **UTF-32** gegeben ist.

Üblicherweise werden Zeichen-Konstanten mittels einfacher und Strings mittels doppelter Anführungszeichen geschrieben. Dies veranlasst den Compiler, die innerhalb der Anführungszeichen enthaltenen Zeichen in den Ausführungszeichensatz umzuwandeln, welcher durch den Compiler vorgegeben ist.

Je nach System definiert ein Compiler sogar zwei Ausführungszeichensätze: Einen, bei welchem die Zeichen den Typ **char** haben und einen, bei welchem die Zeichen den Typ **wchar_t** haben. Der Typ **wchar_t** steht für „**wide character**“ und hat die Aufgabe, erweiterte Zeichen zu speichern. Um anzugeben, dass ein Zeichen oder ein String den **wchar_t**-Typ speichern soll, wurde das **Präfix L** verwendet:

"hello"	char Zeichen gemäß Compilereinstellungen
L"hello"	wchar_t Zeichen gemäß Compilereinstellungen

In den gängigen Systemen wurde für den **char**-Typ die systeminterne Codierung verwendet. Auf Macintosh-Systemen war dies lange Zeit „MacRoman“ und später UTF-8, auf Windows war dies „Windows-1252“, auch umgangssprachlich bekannt als „ANSI“. Für den **wchar_t**-Typ wird oftmals die UTF-16 oder UTF-32 Codierung des Unicodes verwendet, doch standardisiert ist dies nicht. Jeder Compiler kann dies selbst festlegen.

Zu früheren Zeiten war die Steuerung des Compilers somit die einzige Möglichkeit, einen bestimmten Ausführungszeichensatz zu erzwingen. Wie genau dem Compiler mitgeteilt wird, welchen Ausführungszeichensatz er verwenden soll, ist jedoch implementationsspezifisch und somit nicht portabel. Um dem Abhilfe zu schaffen, wurde mit C11 die Initialisierung von Strings mit Unicode standardisiert.

Um einen String explizit mit Unicode zu codieren, verwendet man das Präfix `u8`, `u` oder `U`.



Bei Verwendung eines der Präfixe `u8`, `u` oder `U` kann beim Schreiben des Codes direkt angegeben werden, in welcher Unicode-Codierung der String schlussendlich zur Laufzeit verfügbar sein soll:

<code>u8"hello"</code>	UTF-8
<code>u"hello"</code>	UTF-16
<code>U"hello"</code>	UTF-32

Die Zeichen werden jeweils vom Compiler im Quellzeichensatz eingelesen. Alle normalen Zeichen werden in den gewünschten Laufzeitzeichensatz umkonvertiert. Falls Zeichen mittels Ersatzdarstellung enthalten sind, werden diese direkt in den gewünschten Laufzeitzeichensatz übertragen. So können mittels der Ersatzdarstellungen `\uhhhh` und `\Uhhhhhhh` direkt Unicode-Zeichen mittels ihres eindeutigen Hexadezimalcodes eingegeben werden.

Im Gegensatz zur Ersatzdarstellung für Oktal- und Hexadezimalwerte muss man sich bei der Ersatzdarstellung für Unicode-Zeichen nicht darum kümmern, in welcher Codierung der String intern tatsächlich gespeichert wird. Der Compiler erledigt dies automatisch.



Der konvertierte String wird vom Compiler schlussendlich direkt in das Maschinenprogramm hineinkodiert.

Für die Codierungen UTF-16 und UTF-32 existieren die beiden Typen `char16_t` und `char32_t`. Sie dürfen explizit nur für diese Codierungen verwendet werden. Für UTF-8-Zeichen gibt es vorerst keinen Datentyp `char8_t`, denn der Wert kann in einer Variablen vom Typ `char` gespeichert werden. Erst ab dem Standard C23 wird dieser Typ definiert sein. Er darf explizit nur für die UTF-8-Codierung verwendet werden.

Die Sprache C liefert jedoch nur ein Grundgerüst. Man muss sich bei Verwendung von Unicode darum kümmern, dass zum richtigen Zeitpunkt die richtige Codierung verfügbar ist. Um zwischen den Codierungen zu wechseln, werden Standardbibliotheksfunktionen bereitgestellt. Die neuen Typen sowie die Umwandlungsfunktionen werden in der Bibliothek `<uchar.h>` definiert.

6.6 Operatoren und Interpunktionszeichen

Mit **Interpunktionszeichen** wie Punkt, Komma oder Klammern wird die Sprache gegliedert. Interpunktionszeichen treten sowohl als einzelne Sprachelemente auf als auch in Kombination mit anderen Zeichen.

Einige Interpunktionszeichen werden sowohl als eigenständige Sprachelemente als auch für Operatoren verwendet. Der Strichpunkt beispielsweise tritt als Ende einer Anweisung oder in for-Schleifen als Trenner von Ausdrücken auf. Das Komma dient als Trenner von Listenelementen beispielsweise in der Parameterliste von Funktionen, ist jedoch auch als eigener Operator zulässig. Der Doppelpunkt wird bei der Definition von Bitfeldern benötigt wie auch als Teil des Bedingungs-Operators. Das Symbol `*` wird sowohl für die Definition von Pointern als auch als Multiplikations-Operator benötigt.

6.6.1 Operatoren

Operatoren sind die ausführenden Elemente der Sprache. Die meisten Operatoren bestehen aus einem oder mehreren zusammengesetzten Interpunktionszeichen. In C werden die folgenden Symbole für Operatoren verwendet:

<code>()</code>	<code>[]</code>	<code>-></code>	<code>.</code>	<code>!</code>	<code>~</code>	<code>++</code>	<code>--</code>	<code>+</code>	<code>-</code>	<code>*</code>	<code>&</code>	
<code>/</code>	<code>%</code>	<code><<</code>	<code>>></code>	<code><</code>	<code><=</code>	<code>></code>	<code>>=</code>	<code>==</code>	<code>!=</code>	<code>^</code>	<code> </code>	<code>&&</code>
<code> </code>	<code>?:</code>	<code>=</code>	<code>+=</code>	<code>-=</code>	<code>*=</code>	<code>/=</code>	<code>%=</code>	<code>&=</code>	<code>^=</code>	<code> =</code>	<code><<=</code>	<code>>>=</code>

Einige wenige Operatoren von C verwenden anstelle von Interpunktionszeichen ein Schlüsselwort und werden deswegen hier nicht aufgeführt. Sie werden jedoch alle in Kapitel [→ 9](#) und [→ 21](#) behandelt.

6.6.2 Interpunktionszeichen

Folgende Tabelle ist eine unvollständige Auflistung von Zeichen, wenn sie nicht als Operatoren verwendet werden:

" "	Kennzeichnung von Strings
' '	Kennzeichnung von einzelnen Zeichen
*	Pointer, Funktionspointer (also Pointer auf Funktionen), geklammerte Kommentare /* */
/	Kommentare // und /* */
%	printf()- und scanf()-Formatierung (als Teil des Strings)
()	Klammerung, treten stets paarweise auf
[]	Arrays, treten stets paarweise auf
{ }	Blöcke, treten stets paarweise auf
;	Begrenzer
:	Sprungmarken, Bitfelder
,	Variablendefinitionen, Argumente- und Parameterlisten, Initialisierungen, Enumerationen
.	Teil von Gleitpunkt-Werten (Literal), Teil von Ellipse . . .
\	Escape-Character
#	Präprozessor-Direktiven
_	Gilt als Buchstabe

Nicht in dieser Liste aufgeführt sind gewisse Ziffern und Buchstaben, welche manchmal als Präfix oder Suffix verwendet werden.

6.7 Übungsaufgaben

Aufgabe 1: Schreibweise für literale Konstanten

a) Welche der folgenden Konstanten sind syntaktisch richtig, was ist falsch?

```
#define ALPHA    -1e-0
#define BETA     -e12
#define GAMMA    .517
#define DELTA    3+
```

b) Welche dieser Konstanten sind vom Typ double und welche vom Typ int?

```
#define ZAHL1    55.5e5
#define ZAHL2    55.
#define ZAHL3    55e5f
#define ZAHL4    55
#define ZAHL5    55.5
```

c) Geben Sie obige Konstanten mit `printf()` aus. Wo wird `%d` benötigt, wo `%f`? Was passiert, wenn es verwechselt wird?

Aufgabe 2: Ausgabe von ASCII-Zeichen

Schreiben Sie ein Programm, das folgenden String ausgibt:

```
"Achtung,\ndas geheime Passwort lautet\t\x1b 1234\t\nDanke!"
```

Geben Sie den String Zeichen für Zeichen aus. Aber ersetzen Sie dabei:

- Jedes Zeilenende mit dem Text `"- STOP -"`
- Jeden Tabulator mit dem Text `"\n*****\n"`
- Jedes Zeichen `\x1b` (Escape-Taste) mit dem Text `"- VERBINDUNG UNTERBROCHEN -"`, woraufhin die Abarbeitung stoppt.

Benutzen Sie die Funktion `strlen()` der `<string.h>`-Bibliothek, um die Länge des Strings zu ermitteln. Um ein einzelnes Zeichen von Typ `char` auszugeben, kann das Formatelement `%c` für `printf()` verwendet werden.

7 Datentypen und Variablen in C



Die Programmiersprache C verwendet eine sogenannte strikte **Typisierung**, was bedeutet, dass alle Variablen mit einem genau definierten Typ festgelegt werden müssen.



Ein **Datentyp** oder kurz **Typ** definiert, wieviele Bits für eine Variable im Speicher benötigt werden, und wie diese interpretiert werden müssen. Dadurch bestimmt der Typ, welche Werte eine Variable annehmen kann und welche nicht. Dies wird als **Wertebereich** bezeichnet.



Einen Datentyp in einen anderen umzuwandeln ist grundsätzlich nicht trivial. Die Sprache C hat jedoch viele automatische Umwandlungen – gerade für arithmetische Typen – eingebaut.

Die Programmiersprache C verfolgt auch heute noch keine strenge **Typumwandlung**, sondern erlaubt es, Werte eines Typs mehr oder weniger beliebig in Variablen eines anderen Typs zu speichern.



Wie großzügig in C eine solche Typumwandlung erfolgt, zeigt der folgende Programmausschnitt:

```
float x = 3.9;  
int y = x;
```

Diese Zuweisung ist möglich und die Umwandlung von einem float-Typ in einen int-Typ erfolgt implizit durch festgelegte Regeln des Compilers.

Moderne Compiler haben jedoch häufig Warnungen eingebaut, um anzuzeigen, dass problematische Zuweisungen mit inkompatiblen Typen zu unerwünschten Effekten führen können. Obiges Beispiel würde ein Compiler mit eingeschalteten Warnungen in der zweiten Zeile anmerken mit dem Hinweis „Implicit conversion turns floating-point number into integer“. Solche Warnungen können mittels sogenannter „expliziter Casts“ entfernt werden. Siehe dazu Kapitel [→ 9.9.5](#).

Ergänzende Information Die elektronische Version dieses Kapitels enthält Zusatzmaterial, auf das über folgenden Link zugegriffen werden kann https://doi.org/10.1007/978-3-658-45209-4_7.

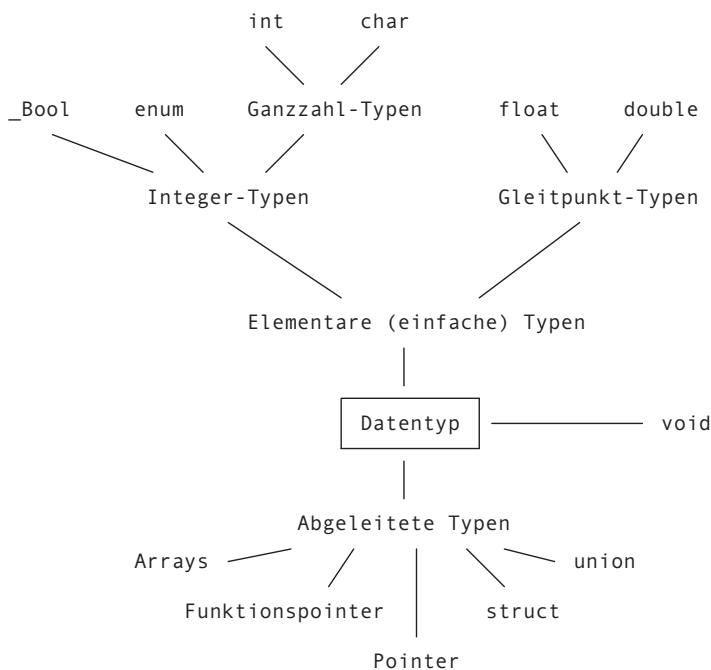
7.1 Typkategorien

In C kann grob zwischen elementaren und abgeleiteten Typen unterschieden werden.

Zu den **elementaren Typen** zählen die Integer-Typen, die Gleitpunkt-Typen sowie Typen, welche intern ebenfalls als Integer gespeichert werden wie Enumerationen und der Typ `_Bool`. Diese Typen werden in diesem Kapitel behandelt.

Abgeleitete Typen sind Typen, welche eine Referenz oder Zusammenstellung von elementaren oder abgeleiteten Typen darstellen. Dazu gehören Pointer (Zeiger), Funktionszeiger, Arrays, Strukturen und Unionen. Diese Typen werden in anderen Kapiteln besprochen.

Des Weiteren gibt es noch den speziellen Typ `void`, der in keine Kategorie passt. Er wird in Kapitel [→ 7.2.8](#) behandelt.



Ganzzahl- und Gleitpunkt-Typen werden auch „**arithmetische Typen**“ genannt. Arithmetische (elementare) Typen und Pointer-Typen werden auch als „**skalare Typen**“, Array-Typen und Struktur-Typen als „**zusammengesetzte Typen**“ (oder „Aggregat-Typen“) bezeichnet.



Es sei hier vermerkt, dass diese Auflistung bei weitem nicht vollständig ist. Einige Datentypen wurden erst durch neuere Standards eingeführt wie beispielsweise `_Complex`, andere wie beispielsweise `short` werden nur noch in Anhang [→ E](#) aufgeführt und nochmals andere sind umgebungsspezifisch wie beispielsweise der `wchar_t`-Typ, wie er in Kapitel [→ 6.5.6](#) besprochen wurde.

Die heutzutage wichtigen elementaren Datentypen für die Programmiersprache C werden im folgenden Unterkapitel genauer vorgestellt.

7.2 Elementare Datentypen in C

In diesem Kapitel werden die wichtigsten (elementaren) Datentypen in C genauer erläutert:

- `char`
- `int`
- `float`
- `double`
- Aufzählungs-Typen
- Boolescher Typ
- `void` Typ

7.2.1 Der Datentyp char

Die Größe einer Variablen vom Typ **char** ist 1 Byte.



Das Wort `char` ist die Abkürzung von „character“ (Schriftzeichen). Gewöhnlich wird der Datentyp `char` dafür verwendet, um einzelne Zeichen aus dem Zeichensatz zu verarbeiten, wie beispielsweise `'c'`, oder Steuerzeichen wie `'\n'`. Der Wert eines Zeichens ist ein Integer – entsprechend dem Ausführungszeichensatz auf der Maschine. Mehr darüber kann in Kapitel [→ 6.1](#) nachgelesen werden.

Ein gespeichertes Zeichen kann als Zeichen ausgegeben werden, sein Wert kann aber auch zu Berechnungen herangezogen werden. Genauso kann man einer `char`-Variablen ein Zeichen oder eine kleine Zahl zuweisen, wie beispielsweise `char var = 48`. Damit eignet sich der Datentyp `char` zur Darstellung beziehungsweise zur Verarbeitung von Integer mit einem kleinen Wertebereich.

Die Interpretation, ob Zahl oder Zeichen, hat durch den Anwender zu erfolgen. Will man beispielsweise den Wert 48 einer `char`-Variablen als Zahl in Dezimalnotation ausgeben, gibt man bei `printf()` das Formatelement `%d` an. Will man den Wert 48 als Zeichen ausgeben, so gibt man das Formatelement `%c` an.



Der vorzeichenlose Datentyp `unsigned char` wird durch 1 Byte ohne Vorzeichenbit dargestellt. Alle Bits stehen für die Darstellung des Wertes einer positiven ganzen Zahl zur Verfügung. Das folgende Bild zeigt ein Beispiel für eine Zahl vom Typ `unsigned char` in Form einer Stellenwerttabelle, die für jedes der 8 Bits den entsprechenden Stellenwert anzeigt:

Bit	7	6	5	4	3	2	1	0	
	1	0	1	1	0	1	0	0	Beispiel für eine Zahl vom Typ <code>unsigned char</code>
Stellenwert	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	

Der Wert der Zahl in diesem Beispiel berechnet sich zu:

$$1 \cdot 2^7 + 0 \cdot 2^6 + 1 \cdot 2^5 + 1 \cdot 2^4 + 0 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 0 \cdot 2^0 = 128 + 32 + 16 + 4 = 180$$

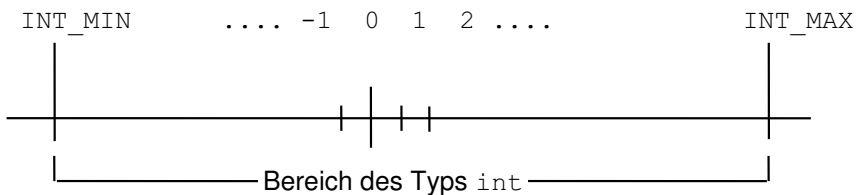
Es sei hier angemerkt, dass heutzutage angenommen werden kann, dass ein Byte 8 Bits besitzt. Der ursprüngliche Standard von C schrieb dies jedoch nicht vor. Somit hat ein `char` eigentlich keine genau vordefinierte Größe. Im Kapitel [→ 7.2.3](#) wird auf eine Lösung für dieses Versäumnis eingegangen.

7.2.2 Der Datentyp `int`

Der Datentyp `int` ist der Standard-Typ für die ganzen Zahlen (**Integer-Zahlen**).



Die `int`-Zahlen umfassen auf dem Computer einen endlichen Zahlenbereich, der nicht überschritten beziehungsweise unterschritten werden kann. Dieser Bereich ist in folgendem Bild dargestellt:



`INT_MIN` und `INT_MAX` stellen dabei die Grenzen der `int`-Werte auf einem Rechner dar. Somit gilt für jede beliebige Zahl x vom Typ `int`: $\text{INT_MIN} \leq x \leq \text{INT_MAX}$

Die Variablen vom Typ `int` haben als Werte ganze Zahlen im Bereich von `INT_MIN` bis `INT_MAX`.



Hierbei ist zu beachten, dass unterschiedliche Rechner den Typ `int` mit einer unterschiedlichen Anzahl Bits definieren. Entsprechend sind die Werte `INT_MIN` und `INT_MAX` auch unterschiedlich definiert.

Die Darstellung des Typs `int` ist implementierungsabhängig.



Früher entsprach die Größe eines `int` der prozessortypischen Repräsentation eines Integers. Dies bedeutet, dass je nach Prozessor ein `int` mit 16, 32 oder gar 64 Bits definiert sein konnte.

Heutzutage jedoch hat sich der Gebrauch des `int`-Typs geändert und wurde für moderne Architekturen mehr oder weniger fixiert: Selbst auf 64-Bit-Rechnern besteht ein `int` häufig nur aus 32 Bits.



Umfasst die interne Darstellung von `int`-Zahlen 32 Bit, so entspricht dies üblicherweise einem Zahlenbereich von -2^{31} bis $+2^{31} - 1$. Wird eine `int`-Zahl durch 64 Bit dargestellt, so wird ein Wertebereich von -2^{63} bis $+2^{63} - 1$ aufgespannt. Die Entscheidung, wie viele Bits letztendlich für die Darstellung von `int`-Zahlen genommen werden, hängt vom Compilerhersteller und vom Prozessor ab, auf dem das C-Programm ablaufen soll.

Der Einfachheit halber wird beim Programmieren häufig schlicht `int` geschrieben und soll auch hier bis auf weiteres der Standard-Typ für einen Integer darstellen.

Während des Programmierens sollte man sich jedoch Gedanken darüber machen, wie groß ein Integer tatsächlich sein kann und wo immer möglich anstelle des Typs `int` Integer-Typen mit genau definierter Bitanzahl verwenden.

7.2.3 Integer-Typen mit genau definierter Bitanzahl

Da unterschiedliche Prozessoren unterschiedliche Registergrößen besitzen, sind auch die Größen des `int`-Typs je nach Prozessor unterschiedlich. Die Verwendung des Typs `int` ist somit abhängig vom Prozessor.

Um vom Prozessor unabhängig programmieren zu können, wurden im Standard C99 neue Integer-Typen definiert, um die benötigte Anzahl Bits exakt festzulegen.

In der `<stdint.h>`-Bibliothek befinden sich Typ-Definitionen wie `int32_t`, welche einen Integer-Typ mit genau festgelegter Bitbreite definieren.



Es werden folgende Typen definiert:

Datentyp	Bits	Wertebereich
int8_t	8	-128 bis +127
int16_t	16	-32768 bis +32767
int32_t	32	-2147483648 bis +2147483647
int64_t	64	-9223372036854775808 bis +9223372036854775807
uint8_t	8	0 bis +255
uint16_t	16	0 bis +65535
uint32_t	32	0 bis +4294967295
uint64_t	64	0 bis +18446744073709551615

Die minimalen und maximalen Werte sind definiert durch folgende Makros:

```
int8_t:    INT8_MIN bis INT8_MAX
int16_t:   INT16_MIN bis INT16_MAX
int32_t:   INT32_MIN bis INT32_MAX
int64_t:   INT64_MIN bis INT64_MAX
uint8_t:   0 bis UINT8_MAX
uint16_t:  0 bis UINT16_MAX
uint32_t:  0 bis UINT32_MAX
uint64_t:  0 bis UINT64_MAX
```

Die <stdint.h>-Bibliothek definiert noch einige weitere spezielle Bitbreiten, worauf hier in diesem Buch jedoch nicht eingegangen wird.

Im Standard C11 wurden dann auch noch Typen definiert, die den Typ char ablösen sollen: char16_t und char32_t. Ab dem Standard C23 gibt es zusätzlich auch noch den Typ char8_t. Mehr dazu kann in Kapitel [→ 6.5.6](#) nachgelesen werden.

7.2.4 Die Datentypen float und double

Um Zahlen mit Nachkommastellen darzustellen, gibt es in C die sogenannten „**Gleitpunkt-Zahlen**“ in zwei Genauigkeiten: **float** und **double**.

float und double-Zahlen entsprechen den rationalen und reellen Zahlen der Mathematik.



Man verwendet in C nicht den mathematischen Begriff der reellen Zahlen, sondern spricht von Gleitpunkt-Zahlen. Sie werden anhand der Exponentialdarstellung codiert:

Zahl = Signifikante · Basis^{Exponent}

Der Typ float wird mit 32 Bits und der Typ double mit 64 Bits gespeichert. Der Typ double hat somit doppelt so viele Bits zur Verfügung, um Gleitpunkt-Zahlen zu speichern. Hierbei ist für beide Typen float und double genau festgelegt, wieviele Bits für die **Signifikante** und wieviele für den **Exponenten** verwendet werden. Folgende Tabelle zeigt, welche Wertebereiche möglich sind:

	float	double
Kleinstmögliche Zahl	1.175494e-38	2.225074e-308
Grösstmögliche Zahl	3.402823e+38	1.797693e+308
Anzahl genaue Dezimalstellen	6	15

Dies bedeutet beispielsweise, dass die Zahl π mit dem Typ float nur als die Zahl 3.141593 gespeichert werden kann, mit dem Typ double hingegen als die Zahl 3.141592653589793.

Die Codierung anhand der Exponentialdarstellung dient zur näherungsweisen Darstellung von reellen Zahlen auf Rechenanlagen, denn im Gegensatz zur Mathematik ist auf dem Rechner der Wertebereich nicht unendlich darstellbar und somit die Genauigkeit einer Zahl mit Nachkommastellen begrenzt.



Auf eine genaue Behandlung über die Codierung der Zahlen wird in Anhang → E.5 eingegangen.

Während in der Mathematik die reellen Zahlen unendlich dicht auf dem Zahlenstrahl liegen, haben Gleitpunkt-Zahlen tatsächlich diskrete Abstände zueinander. Es ist im Allgemeinen also nicht möglich, Brüche, Dezimalzahlen, transzendente Zahlen oder die übrigen nicht rationalen Zahlen wie beispielsweise die Quadratwurzel aus 2 exakt darzustellen.

Werden `float` oder `double`-Zahlen benutzt, so kommt es in der Regel zu **Rundungsfehlern**.



Wegen der Exponentialdarstellung werden die Rundungsfehler für große Zahlen größer, da die Abstände zwischen den im Rechner darstellbaren `float`-Zahlen zunehmen. Addiert man beispielsweise eine kleine Zahl y zu einer großen Zahl x und zieht anschließend die große Zahl x wieder ab, so erhält man meist nicht mehr den ursprünglichen Wert von y .

Auf Englisch heißen Gleitpunkt-Zahlen „floating point numbers“, woher auch das Schlüsselwort `float` kommt.

7.2.5 Aufzählungs-Typen

Aufzählungs-Typen definieren Integer, welche jedoch anhand eines Namens aufgezählt werden können. Hierfür wird das Schlüsselwort **enum** verwendet. Beispielsweise:

```
enum FileError {  
    NO_ERROR,           // Konstante mit Wert 0  
    FILE_NOT_FOUND,     // Konstante mit Wert 1  
    FILE_LOCKED         // Konstante mit Wert 2  
};
```

Definiert werden hier sowohl der neue Datentyp `enum FileError` als auch dessen Aufzählungs-Konstanten. Der Wertebereich des neuen Typs ist durch die Liste der Aufzählungs-Konstanten festgelegt.

Aufzählungs-Konstanten haben einen konstanten Integer-Wert. Der Typ einer Aufzählungs-Konstanten ist normalerweise `int`. Ab dem Standard C23 kann der Typ auch explizit festgelegt werden.



Der Typname des soeben definierten Typs ist `enum FileError`. Dabei ist „FileError“ das sogenannte „**Etikett**“ (auf Englisch „**enumeration tag**“), welches frei vergeben werden kann.

Zulässige Werte für Variablen eines Aufzählungs-Typs sind die Werte der Aufzählungs-Konstanten in der Liste der Definition des Aufzählungs-Typs. Es ist üblich, dass Aufzählungs-Konstanten groß geschrieben werden.



Auch wenn Aufzählungs-Konstanten groß geschrieben werden, sind sie doch nicht dasselbe wie symbolische Konstanten (siehe Kapitel [→ 21.2](#)).

Aufzählungs-Konstanten dürfen nicht in Präprozessor-Bedingungen verwendet werden.



Die erste Aufzählungs-Konstante in der Liste hat standardmäßig den Wert 0, die zweite den Wert 1 usw. Es ist aber auch möglich, für jede Aufzählungs-Konstante einen Wert explizit anzugeben. Werden einige Werte in der Liste nicht explizit belegt, so wird der Wert ausgehend vom letzten explizit belegten Wert jeweils um 1 bis zum nächsten explizit angegebenen Wert hochgezählt. Dies ist in den folgenden Beispielen zu sehen:

```
enum test {ALPHA, BETA, GAMMA};           // 0, 1, 2
enum test {ALPHA = 5, BETA = 3, GAMMA = 7}; // 5, 3, 7
enum test {ALPHA = 4, BETA, GAMMA = 3};    // 4, 5, 3
```

Es dürfen jedoch keine zwei Aufzählungs-Konstanten innerhalb eines Aufzählungs-Typs mit demselben Wert existieren!

Aufzählungs-Konstanten in verschiedenen Aufzählungs-Typen müssen voneinander verschiedene Namen haben, wenn sie im selben Gültigkeitsbereich verwendet werden.

Aufzählungs-Typen sind geeignet, um Konstanten zu definieren. Sie stellen damit in vielen Situationen eine Alternative zu der Definition von Konstanten mit Hilfe der Präprozessor-Anweisung `#define` dar.



Zu `#define` siehe Kapitel [→ 21.2](#). Für eine Beschreibung von Gültigkeitsbereichen, siehe Kapitel [→ 11.2](#).

Geschickt ist, dass bei der Definition von Aufzählungs-Konstanten Werte implizit generiert werden können, wie im folgenden Beispiel:

```
enum Monate {
    JAN = 1, FEB, MAR, APR, MAI, JUN, JUL, AUG, SEP, OKT, NOV, DEZ
};
```

Hier wird vollkommen automatisch der Februar (FEB) zum Monat 2, der März (MAR) zum Monat 3, usw.

Das Etikett kann auch weggelassen werden. Dann jedoch kann der Typ nicht mehr per Name angesprochen werden, nur die Aufzählungs-Konstanten sind noch ansprechbar. Um ihn dennoch verwenden zu können, kann er mittels typedef in einen selbst definierten Typ umgewandelt werden (siehe Kapitel [→ 14.2](#)):

```
typedef enum {NO_ERROR, FILE_NOT_FOUND, FILE_LOCKED} ErrorId;
```

Oder aber er kann direkt für die Definition einer Variablen verwendet werden, hier beispielsweise die Variable error:

```
enum {NO_ERROR, FILE_NOT_FOUND, FILE_LOCKED} error;
```

Man kann in C zwar Variablen eines Aufzählungs-Typs definieren. Dennoch wird von einem C-Compiler nicht verlangt, zu prüfen, ob einer Variablen eines Aufzählungs-Typs eine passende Aufzählungs-Konstante zugewiesen wird. Beispielsweise erzeugt folgender Code keinen Fehler:

```
error = 1234;
```

Die Zuweisung beliebiger Integer-Zahlen zu einer enum-Variablen wird durch den Compiler nicht verboten. Man ist selbst verantwortlich dafür, dass der Wertebereich eines Aufzählungs-Typs nicht verletzt wird.



7.2.6 Der Datentyp _Bool

Ursprünglich hatte C im Gegensatz zu anderen Programmiersprachen keinen booleschen Typ. Anstelle dessen wurden boolesche Werte mittels der Werte 0 und nicht-0 angegeben und als speichernde Typen werden üblicherweise Integer-Typen verwendet oder Pointer-Typen.

In C99 wurde die Standard-Bibliothek <stdbool.h> eingeführt, welche einen booleschen Typ für C deklariert namens _Bool mit den beiden **booleschen Werten** false und true als Aufzählungs-Konstanten. Häufig wird auch gleichzeitig der Typ bool definiert, welcher genau dasselbe darstellt wie _Bool. Da dieser Typ jedoch als Aufzählungs-Typ deklariert ist, ist er nach wie vor ein selbst definierter Typ und damit kein Standard-Datentyp des Compilers.

In C11 dann gilt der Typ `_Bool` als standardmäßig definiert, selbst wenn die `<stdbool.h>`-Bibliothek nicht eingebunden wird. Der Typ `bool` sowie die beiden Konstanten `false` und `true` hingegen sind nach wie vor nicht standardisiert und somit auch in C11 erst verfügbar, wenn die `<stdbool.h>`-Bibliothek eingebunden wird:

```
#include <stdbool.h>

int main(void) {
    bool a = true;
    bool b = false;
}
```

Ab dem kommenden Standard C23 dann werden `true` und `false` sowie der Typ `bool` endgültig in den Sprachumfang miteinbezogen.

7.2.7 Bit-Verarbeitung

In der Programmiersprache C gibt es keinen Typ für ein einzelnes Bit. Auch der boolesche Typ wird intern üblicherweise mit mehreren Bytes gespeichert.



Mit den in Kapitel [→ 9.8](#) beschriebenen Bit-Operatoren können jedoch Integer-Werte beliebig bitweise verknüpft werden. Dadurch ist es möglich, einzelne Bits manuell aus einem Integer auszulesen oder sie zu setzen.

Im aufkommenden Standard C23 werden zudem neue Funktionen in einer neuen Bibliothek `<stdbit.h>` eingeführt, welche das Bearbeiten von Bits vereinfacht. In diesem Buch wird nicht weiter darauf eingegangen.

7.2.8 Der Datentyp void

void ist ein Schlüsselwort und ein Datentyp. Dieser Typ bezeichnet eine leere Menge und wird beispielsweise verwendet, wenn eine Funktion keinen Rückgabewert oder keinen Übergabeparameter hat.

Pointer auf `void` werden in Kapitel [→ 8.2](#) beschrieben.

7.3 Modifikatoren und andere Besonderheiten

In diesem Kapitel wurden nur die wichtigsten elementaren Datentypen durchgenommen. Die Sprache C definiert in ihrem Kern jedoch insbesondere einige weitere arithmetische Typen, welche mittels sogenannter **Modifikatoren** angesprochen werden.

Zwei sehr bekannte Modifikatoren in der Programmiersprache C sind **short** und **long**, welche die Größe eines Integer-Typs modifizieren. Da solche Modifikatoren jedoch umgebungsabhängig, sprich von Compiler und System abhängig sind, wurden in neueren Standards präzisere, plattformunabhängige Typangaben eingeführt (siehe Kapitel [→ 7.2.3](#)). In diesem Kapitel wird somit auf eine Behandlung der lange Zeit in Verwendung gewesenen **short** und **long**-Modifikatoren verzichtet. Diese werden im Anhang [→ E](#) beschrieben.

Zudem sind heutzutage Speicherplatz und Rechengenauigkeit aufgrund der fortlaufenden Standardisierung und Verbesserung der Systeme nur noch in seltenen Fällen wirklich von großer Bedeutung, weswegen hier nur auf ein paar Besonderheiten aufmerksam gemacht werden soll.

7.3.1 Vorzeichenlose und vorzeichenbehaftete Datentypen

Für die elementaren Integer-Datentypen stehen die Typ-Modifikatoren **signed** und **unsigned** zur Verfügung. Durch diese beiden Modifikatoren wird festgelegt, ob ein Integer-Typ vorzeichenbehaftet ist oder nicht.

So können beispielsweise folgende Typen definiert werden:

```
signed char
unsigned char
signed int
unsigned int
```

Ob dabei der Modifikator vor oder nach dem Basis-Typ steht, spielt keine Rolle. Tatsächlich kann beim **int**-Typ der Basis-Typ gar weggelassen werden.

Werden nur die Schlüsselwörter `signed` oder `unsigned` ohne Angabe des Datentyps verwendet, so wird implizit der Datentyp `int` hinzugefügt, das heißt, es werden die Typen `signed int` beziehungsweise `unsigned int` benutzt.



Vorzeichenbehaftet, also von einem `signed`-Typ, sind standardmäßig alle Integer-Standard-Datentypen außer `char`. Der Typ `char` kann je nach Implementation oder Compiler-einstellung vorzeichenbehaftet oder vorzeichenlos sein.



Mit dem Voranstellen des Modifikators `signed` vor einen Integer-Standard-Datentyp, beispielsweise `signed char`, wird erreicht, dass eine vorzeichenbehaftete Darstellung erzwungen wird. So wird bei `signed char` der Wertebereich von -128 bis +127 dargestellt.

Der Modifikator `unsigned` (nicht vorzeichenbehaftet) wird vor den Integer-Typ gestellt, wenn man erzwingen will, dass alle Werte des entsprechenden Wertebereichs größer gleich 0 sind. So kann beispielsweise bei `unsigned char` ein Wertebereich von 0 bis 255 dargestellt werden.

7.3.2 Zahlenüberlauf

Wie auch immer eine Variable schlussendlich definiert ist, sie kann bei Berechnungen nur Werte aus ihrem Wertebereich annehmen. Speichert beispielsweise eine Variable `x` vom Typ `int` einen sehr großen Wert, so kann es sein, dass das Resultat von $2 * x$ größer als `INT_MAX` ist. Die Berechnung $2 * x$ führt so zu einem mathematischen Fehler, dem sogenannten **Zahlenüberlauf** (auf Englisch: „**overflow**“). Auch für negative Zahlen kann dies passieren, wenn das Resultat kleiner als `INT_MIN` ist. Dann nennt man dies einen **Unterlauf** (auf Englisch: „**underflow**“).

Auf solche Zahlenüberläufe (oder -unterläufe) muss man beim Schreiben eines Programmes selbst achten. Der Zahlenüberlauf wird nämlich in C nicht durch einen Fehler oder eine Warnung angezeigt.

Wenn ein Überlauf (oder ein Unterlauf) stattfindet, wird bei modernen Prozessoren normalerweise ein „wrap-around“ durchgeführt. Das bedeutet, dass von der größtmöglichen Zahl plötzlich auf die kleinstmögliche Zahl gesprungen wird und umgekehrt.



Addiert man beispielsweise zum größten positiven Integer mit vorzeichenbehafteten Typ eine 1 dazu, so befindet man sich bei der heute üblichen Darstellung im Zweierkomplement (siehe Anhang [→ E.4](#)) plötzlich im negativen Zahlenbereich.

Bei vorzeichenlosen Integer-Typen ist der Wertebereich stets eine Zahl modulo 2^n . Das bedeutet, dass von dem Resultat einer Berechnung der Rest bei der Integer-Division durch 2^n gebildet wird.

So ist beispielsweise bei einem Byte von 8 Bits die größte Zahl, die in den Typ `unsigned char` passt, die Zahl 255. Addiert man 2 dazu, so wird mit 1 weitergerechnet, da 257 modulo 256 den Wert 1 ergibt.

Zieht man eine große Zahl vom Typ `unsigned int` von einer kleinen `unsigned int`-Zahl ab, so muss das Ergebnis ebenfalls vom Typ `unsigned int` sein. Das Laufzeitsystem des Compilers hat damit kein Problem. Es rechnet einfach mit seiner Modulo-Arithmetik weiter.

Das Laufzeitsystem des Compilers muss auf Überläufe des Wertebereichs nicht reagieren.



Meist wird in der Praxis so verfahren, dass ein Datentyp gewählt wird, der offensichtlich für die Anwendung einen standardisierten und ausreichend großen, aber nicht übermäßig großen Wertebereich hat.

7.4 Variablen in C

In C gibt es zwei verschiedene Arten, wie **Variablen** vereinbart werden können: Definitionen und Deklarationen.

Der Begriff der **Vereinbarung** umfasst sowohl die Definition als auch die Deklaration.



Eine **Deklaration** legt die Art einer Variablen beziehungsweise die Schnittstelle einer Funktion fest.



Eine **Definition** von Variablen und Funktionen dient dazu, selbige am gewünschten Ort im Speicher anzulegen. Hierbei ist automatisch eine Deklaration mit eingeschlossen.



Kurz und bündig ausgedrückt bedeutet dies:

Definition = Deklaration + Reservierung des Speicherplatzes.



Reine Deklarationen dienen dazu, Datenobjekte beziehungsweise Funktionen dem Compiler bekanntzumachen, die in anderen Übersetzungseinheiten definiert werden oder in derselben Übersetzungseinheit erst nach ihrer Verwendung definiert werden.

Eine Deklaration umfasst stets den Namen eines Objektes und seinen Typ. Damit weiß der Compiler, mit welchem Typ er einen Namen verbinden muss.



Reine Deklarationen werden in den folgenden Kapiteln genauer behandelt:

- Deklaration von Variablen: [→ 15.3](#)
- Definition eines neuen Typnamens mit typedef: [→ 14.2](#)
- Vorwärtsdeklaration von Funktionen: [→ 11.6](#)

Bei einer Definition von Variablen werden nebst dem Typ die sogenannte „Speicherklasse“ sowie „Qualifikatoren“ festgelegt.

7.4.1 Definition einfacher Variablen

Eine einzige Variable wird definiert durch eine Vereinbarung der Form

```
datentyp name;
```

also beispielsweise durch

```
int x;
```

Vom selben Typ können mehrere Variablen in einer einzigen Vereinbarung definiert werden, indem man die Variablennamen durch Kommas trennt wie in folgendem Beispiel:

```
int x, y, z;
```

Auf diese Schreibweise sollte jedoch nur in Ausnahmefällen zurückgegriffen werden, da sie schwieriger zu lesen ist und bei Pointern und Arrays (siehe dazu Kapitel [→ 8.1.2](#)) zudem zu Fehlinterpretationen führen kann.

Die Namen der Variablen müssen den Namenskonventionen genügen (siehe Kapitel [→ 6.4](#)). Natürlich darf ein Variablenname nicht identisch mit einem Schlüsselwort sein. Jede Variable in einer Folge von Definitionen von Variablen muss selbstverständlich ihren eigenen, eindeutigen Namen erhalten.

7.4.2 Externe, interne, lokale und globale Variablen und deren Bindung

Im Folgenden werden folgende vier Begriffe beschrieben:

- Interne Bezeichner
- Externe Bezeichner
- Intern gebundene Bezeichner
- Extern gebundene Bezeichner

Ein C-Programm besteht aus Funktionen. Etwas, was sich innerhalb einer Funktion befindet, wird als „Interner Bezeichner“ bezeichnet. Etwas, was sich außerhalb befindet als „externer Bezeichner“.

Funktionen sind zueinander extern, da in C keine Funktion innerhalb einer Funktion definiert werden kann. Variablen können entweder innerhalb oder außerhalb von Funktionen definiert werden.

Interne Variablen werden umgangssprachlich als **lokale Variablen** bezeichnet. **Externe Variablen** als **globale Variablen**.



Die beiden Begriffe „lokal“ und „global“ sind umgangssprachliche Begriffe, welche im eigentlichen Standard nicht verwendet werden. Dennoch werden sie auch in diesem Buch gerne genutzt, um sie von den sogenannten „**Bindungen**“ (auf Englisch „**Linkage**“) zu unterscheiden:

Des Weiteren unterscheidet ein Compiler nämlich auch zwischen sogenannten „intern gebundenen“ und „extern gebundenen“ Bezeichnern.

Intern gebundene Bezeichner sind Namen, die innerhalb einer Übersetzungseinheit verwendet werden.



Extern gebundene Bezeichner sind Namen, die für mehrere Übersetzungseinheiten gültig sind. Sie haben also auch eine Bedeutung außerhalb der betrachteten Datei.



Extern gebundene Namen haben insbesondere eine Bedeutung für den Linker, der unter anderem die Bindungen zwischen Namen in separat bearbeiteten Übersetzungseinheiten herstellen muss. Siehe dazu auch Kapitel [→ 15.1.2](#).

Namen mit interner Bindung existieren eindeutig für jede Übersetzungseinheit. Namen mit externer Bindung existieren eindeutig für das ganze Programm.



7.4.3 Globale Variablen

Funktionsinterne Variablen sind lokal zu einer Funktion. Sie haben nur innerhalb ihrer Funktion eine Bedeutung. Sie werden üblicherweise als „lokale Variablen“ bezeichnet. Funktionsexterne Variablen haben die Bedeutung von globalen Variablen für diejenigen Funktionen, die in einer Datei nach ihnen definiert werden.



Globale Variablen können grundsätzlich allen Funktionen eines Programms zur Verfügung stehen.



Um zu verstehen, warum funktionsexterne Variablen globale Variablen sind, muss man zuerst den Begriff der **Sichtbarkeit** verstehen. Dieser sagt: Die Sichtbarkeit einer Variablen bedeutet, dass man von einer Programmstelle aus die Variable sieht, das heißt, dass man auf sie über ihren Namen zugreifen kann. In Kapitel [→ 11.2](#) wird genauer auf dieses Thema eingegangen.

Da globale Variablen für alle Funktionen, die nach ihnen in einer Datei definiert werden, sichtbar sind, können sie von all diesen Funktionen verwendet und gelesen oder beschrieben werden. Die externen Variablen stehen also diesen Funktionen gemeinsam, in anderen Worten global, zur Verfügung. Daher auch der Name globale Variablen.

Globale Variablen sollte man so wenig wie möglich verwenden, da sie leicht zu schwer erkennbaren Fehlern führen können.

Die Verwendung von globalen Variablen wird aus dem Blickwinkel des Software Engineering nicht gerne gesehen, da bei der Verwendung globaler Variablen leicht die Übersicht verloren geht und es unter Umständen zu schwer zu findenden Fehlern kommen kann. Arbeiten mehrere Funktionen auf einer globalen Variablen, so ist der Verursacher eines Fehlers schwer zu finden.



Dennoch kann es – vor allem bei der hardwarenahen Programmierung – zu Situationen kommen, wo man aus Performance-Gründen gezwungen ist, globale Variablen zu verwenden.

Deshalb sollten globale Variablen nur in zwingenden Fällen verwendet werden, beispielsweise wenn es aus Performance-Gründen nicht mehr ratsam ist, mit Übergabeparametern zu arbeiten.

Im folgenden Beispiel wird eine globale Variable alpha eingeführt:

globvar.c

```
#include <stdio.h>

int alpha = 3;

void f(void) {
    int a;
    a = alpha;
    printf("a hat den Wert %d\n", a);
}

int main(void) {
    int b = 4;
    printf("alpha hat den Wert %d\n", alpha);
    alpha = b;
    printf("alpha hat den Wert %d\n", alpha);
    f();

    return 0;
}
```

Die Ausgabe des Programms ist:

```
alpha hat den Wert 3  
alpha hat den Wert 4  
a hat den Wert 4
```

Mit der Vereinbarung `int alpha = 3;` wird die globale Variable `alpha` zu Programmbeginn angelegt. Damit ist die Variable `alpha` in den Funktionen `f()` und `main()` sichtbar. Man kann in diesen beiden Funktionen auf sie lesend oder schreibend zugreifen. Die Lebensdauer der externen Variablen `alpha` erstreckt sich bis zum Programmende. Erst nach Ablauf des Programms wird der Speicherplatz dieser Variablen freigegeben.

7.4.4 Initialisierung von Variablen

Jede einfache Variable kann bei ihrer Definition **initialisiert** werden, indem man einfach ein Gleichheitszeichen `=` gefolgt von einem Ausdruck des passenden Typs an den Namen der Variablen anhängt. Im folgenden Beispiel werden einige `int`- und `char`-Variablen bei ihrer Definition initialisiert:

```
int main (void) {  
    int a = 9;  
    int b = (4 + 5) * 6;  
    int c;                // c ist nicht initialisiert  
  
    char c1 = 'x';  
    char c2 = 'y';  
}
```

Da diese Initialisierung von Hand durch Zuweisung eines Wertes vorgenommen wird, wird sie auch „**manuelle Initialisierung**“ genannt.

Wenn eine lokale Variable nicht manuell initialisiert wird, so ist ihr Inhalt unbestimmt. Der Compiler nimmt für lokale Variablen keine automatische Initialisierung vor.



Globale Variablen werden zu Beginn eines Programms automatisch mit 0 initialisiert.



Mehr Informationen zu manuellen und automatischen Initialisierungen können in Kapitel [→ 15](#) nachgelesen werden.

Ab dem Standard C23 können Variablen mittels der aus C++ bekannten „Null-Initialisierung“ mittels öffnender und schließender geschweifter Klammer vollständig mit 0 gefüllt werden:

```
struct MyStruct s = {};
```

7.5 Qualifikatoren

Ein C-Compiler macht über die Verwendung von Variablen gewisse Annahmen. Beispielsweise, dass jede Variable innerhalb eines Codes grundsätzlich veränderbar ist oder dass die Einträge eines Arrays sich nicht mit einem zweiten Array überlappen.

Wenn bei der Erstellung eines Programmes jedoch klar ist, dass der Compiler diese Grundannahmen nicht machen soll, so kann mittels sogenannter „**Typ-Qualifikatoren**“ dem Compiler ein Hinweis gegeben werden, wie der Code zu behandeln ist.

Qualifikatoren sind an den Typ einer Variable gebunden. Sie verändern die Art und Weise, wie der Compiler auf die Speicherinhalte zugreift.



Dabei ist wichtig zu verstehen, dass Qualifikatoren an den Typ gebunden sind, und nicht an die Variable oder den Wert, der dahinter liegt. Auf Werte oder Speicheradressen wird oftmals im selben Programm an unterschiedlichen Stellen mittels unterschiedlich qualifizierter Typen zugegriffen. Die Angabe eines Qualifikators zum Typ ist für den Compiler somit ein Hinweis, wie das Ansprechen des Wertes während der Gültigkeit einer Variablen zu erfolgen hat.

Gerade bei Variablen mit einem Pointer- oder Array-Typ sind Hinweise für den Compiler wichtig, um falschen Zugriffen vorzubeugen und korrupte Daten zu vermeiden. Auf Pointer und Arrays wird in Kapitel [→ 8](#) eingegangen.

In C gibt es 4 Qualifikatoren: `const`, `restrict`, `volatile` und `_Atomic`. Der `restrict`-Qualifikator wird in Kapitel [→ 8.4](#) und der `_Atomic`-Qualifikator in Kapitel [→ 23.5.5](#) behandelt. Hier finden sich nur die Erläuterungen zu den Qualifikatoren `const` und `volatile`:

7.5.1 Der Qualifikator `const`

Mit dem Schlüsselwort **`const`** ist es möglich, eine Variable zu vereinbaren, welche nur gelesen werden kann. Der Compiler schützt diese Variable vor Schreibzugriffen.



Es ist somit beispielsweise möglich, anstatt mit `#define PI 3.1415927` eine klar typisierte, aber schreibgeschützte Variable zu definieren:

```
const double PI = 3.1415927;
```

Die sofortige Initialisierung der Variablen ist verbindlich, denn nach der Initialisierung ist die Variable für Schreibzugriffe gesperrt.

Sehr wichtig ist das `const`-Schlüsselwort insbesondere bei der Übergabe von Argumenten per Pointer. Wenn eine Funktion ein Argument mit dem Schlüsselwort `const` erwartet, so gibt dies den Hinweis, dass die Funktion dieses Argument nicht verändern wird. Mehr dazu kann in Kapitel [→ 11.5.2](#) nachgelesen werden.

Der Compiler unterstützt das Schreiben von sicherem Code, indem er Verletzungen von `const` als Fehler meldet.

Durchgängige Programmierung mit dem Schlüsselwort `const` wird als „**const-safe-Programmierung**“ bezeichnet. Gerade bei der Übergabe von Pointern als Parameter kann diese Sicherheitsvorkehrung einem Fehlverhalten von Code vorbeugen.



7.5.2 Der Qualifikator `volatile`



Der Qualifier **`volatile`** wird gebraucht, wenn eine Variable implementierungsabhängige Eigenschaften hat und der Compiler keine **Optimierung** durchführen soll. Implementierungsabhängig bedeutet hier, dass sich der Wert einer Variablen durch andere Prozesse wie beispielsweise durch Interrupts ändern kann.



Dies ist beispielsweise bei Memory-Mapped-Input/Output der Fall. Hier werden Register durch Memory-Adressen angesprochen und nicht über Ports (Kanäle). Hier müssen die entsprechenden Variablen stets an denselben Adressen stehen und der Compiler darf die Variablen nicht aus Optimierungsgründen woandershin wie beispielsweise in Register verschieben.

Weiteres Beispiel: Ein optimierender Compiler kann feststellen, ob sich der Wert einer Variablen in einem bestimmten Programmfluss ändert oder nicht. Ändert sich der Wert nicht, könnte er sich den zu Beginn berechneten Wert „merken“ und diesen zwischengespeicherten Wert im weiteren Verlauf des Programms verwenden, statt mehrfach auf den Speicher der Variablen zugreifen.

Daher darf ein Compiler bei der Verwendung des Schlüsselwortes `volatile` nicht optimieren, beispielsweise weder die Variable verschieben noch zwischengespeicherte Werte verwenden.

Entsprechende Variablen sind heutzutage kaum mehr anzutreffen, da moderne Betriebssysteme Zugriffe auf solche Speicherstellen durch gesicherte Bibliotheksaufrufe anbieten und sie so verstecken.

7.6 Übungsaufgaben

Aufgabe 1: Initialisierung von lokalen Variablen

Erstellen Sie ein Programm, in welchem jeweils eine lokale Variable der Typen `int`, `float`, `double` und `char` definiert, aber nicht initialisiert wird. Geben Sie den Inhalt der Variablen aus. Beobachten Sie das Ergebnis und beantworten Sie, weshalb Variablen immer initialisiert werden sollten.

Aufgabe 2: Initialisierung von globalen Variablen

Ändern Sie das vorherige Programm, indem Sie die lokalen Variablen zu globalen machen. Beobachten Sie erneut das Ergebnis. Was ist festzustellen?

8 Einführung in Pointer und Arrays



Pointer sind ein elementarer Bestandteil der Programmiersprache C. Um mit C praxistauglich programmieren zu können, sollte das Konzept der Pointer verstanden sein. Dazu dient diese Einführung in Pointer und Arrays. Eine Fortsetzung dieses Themas finden Sie in Kapitel → 12.

8.1 Pointer-Typen und Pointer-Variablen

Der Arbeitsspeicher eines Rechners ist in Speicherzellen eingeteilt. Jede Speicherzelle trägt eine Nummer.

Die Nummer einer Speicherzelle wird als **Adresse** bezeichnet.



Ist der Speicher eines Rechners byteweise (in der Regel 1 Byte = 8 Bits) aufgebaut, so sagt man, er sei byteweise adressierbar. Über Adressen sind dann einzelne Bytes ansprechbar.

Einzelne Bits können nicht adressiert werden.



Eine Variable belegt ein oder mehrere Bytes im Speicher. Je nach Typ der Variablen benötigt sie mehr oder weniger Bytes. Eine `float`-Variable beispielsweise benötigt 4 Bytes.

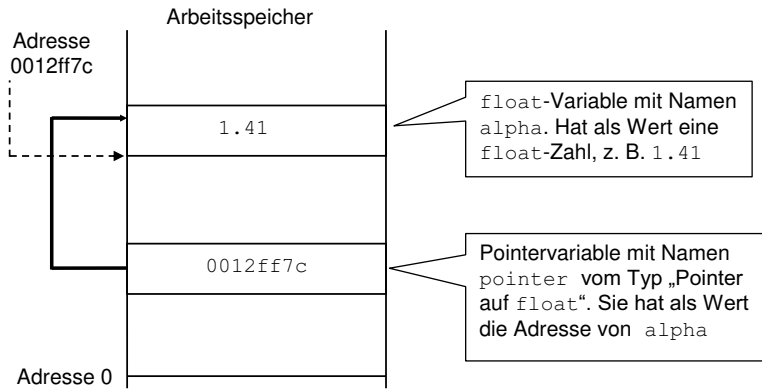
Im Arbeitsspeicher werden somit für eine `float`-Variable 4 Bytes reserviert. Das erste Byte dient dabei als Anfangspunkt. Die Adresse dieses ersten Bytes ist eine eindeutige Position, mit welcher die Variable lokalisiert werden kann. Mithilfe eines **Pointers** kann diese Adresse gespeichert werden.

Ein Pointer (**Zeiger**) ist eine Variable, welche die Adresse einer im Speicher sich befindlichen Variablen aufnehmen kann.



Ergänzende Information Die elektronische Version dieses Kapitels enthält Zusatzmaterial, auf das über folgenden Link zugegriffen werden kann https://doi.org/10.1007/978-3-658-45209-4_8.

Im folgenden Bild wird eine Pointer-Variable auf ein Speicherobjekt schematisch aufgezeigt:



Pointer und Speicherobjekte sind vom Typ her gekoppelt. Ist das Objekt vom Typ X, so braucht man einen Pointer vom Typ „Pointer auf X“, um auf dieses Objekt zeigen zu können.



Es ist also beispielsweise nicht erlaubt, einen Pointer auf int auf eine double-Variable zeigen zu lassen. Daher ist es nötig, den Datentyp anzugeben, den der Pointer referenzieren soll – das heißt auf den er zeigen beziehungsweise verweisen soll.

8.1.1 Definition von Pointer-Variablen

Ein Pointer wird formal wie eine Variable definiert, nur dass mittels eines Sternchens festgelegt wird, dass es sich um einen Pointer handelt. Die allgemeine Form der Definition eines Pointers ist:

```
Typname* Pointername;
```

Typname* ist der Datentyp des Pointers
Pointername ist der Name des Pointers.



Dabei ist Pointername ein Pointer auf den Datentyp Typname. Diese Definition wird von rechts nach links gelesen, wobei man den * als „ist Pointer auf“ liest: Pointername ist ein Pointer auf Typname.

Durch diese Definition wird eine Variable Pointername vom Typ „Pointer auf Typname“ definiert. Konkrete Beispiele für die Definition von Pointer-Variablen sind:

```
int*   pointer1;  
float* pointer2;
```

pointer1 ist ein Pointer auf int, pointer2 ist ein Pointer auf float. Der Datentyp dieser Pointer-Variablen ist int* beziehungsweise float*. Durch die Vereinbarung sind Pointer und zugeordneter Typ miteinander verbunden.

Es ist wichtig zu verstehen, dass durch die Definition einer Pointer-Variablen nur Speicherplatz für die Pointer-Variable selbst reserviert wird, jedoch nicht für das Objekt, auf welches der Pointer schlussendlich zeigen wird.



Da Variablen prinzipiell an jeder beliebigen Stelle des Adressraumes (mit Ausnahme der Adresse 0) liegen können, müssen mit einem Pointer letztendlich alle Adressen dargestellt werden können. Daher werden zur Speicherung eines Pointers genauso viele Bytes verwendet, wie es die interne Darstellung einer Adresse erfordert. Je nach Prozessorarchitektur kann eine Adresse somit unterschiedlich breit sein. Lange Zeit waren 32-Bit-Adressen üblich, mittlerweile setzen sich 64-Bit-Architekturen durch. Eine Adresse belegt auf modernen Maschinen somit normalerweise 4 oder 8 Bytes.

8.1.2 Wo gehört das Sternchen hin?

An dieser Stelle sei anzumerken, dass das Pointer-Zeichen (das Sternchen) formal zur Variablen und nicht zum Typ zugehörig ist.

```
int *pointer1;    // Wie es der Compiler liest.  
int* pointer1;   // Wie man es semantisch versteht.
```

Der Compiler liest somit „pointer1 ist eine Pointer-Variable mit dem zu referenzierenden Typ int“. Intuitiv liest sich der Code jedoch „pointer1 ist vom Typ int-Pointer“.

Auch gibt es Quellen, welche das Sternchen in die Mitte platzieren, um es optisch hervorzuheben:

```
int * pointer1;
```

Syntaktisch (sprich im Quellcode geschrieben) spielt es keine Rolle, wo das Sternchen steht. Die semantische Unterscheidung ist jedoch wichtig, wenn mehrere Pointer-Variablen in einer Zeile deklariert werden (siehe dazu Kapitel [→ 7.4.1](#)). Beachten Sie die folgenden Unterschiede bei der Definition von Pointer-Variablen:

```
int *pointer1, pointer2;  
int *pointer3, *pointer4;
```

In der ersten Zeile werden eine Pointer-Variable und eine int-Variable definiert, in der zweiten Zeile dagegen zwei Pointer-Variablen.

Um Fehldeklarationen zu vermeiden, sollte eine Pointer-Variable immer in einer separaten Zeile definiert werden.



Hier in diesem Buch wird das Sternchen in den Code-Beispielen grundsätzlich zum Typ geschrieben.

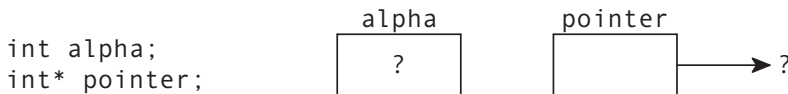
8.1.3 Adress-Operator

Die einfachste Möglichkeit, einen Pointer auf ein Objekt zeigen zu lassen, besteht darin, den **Adress-Operator** & auf eine Variable anzuwenden, denn es gilt:

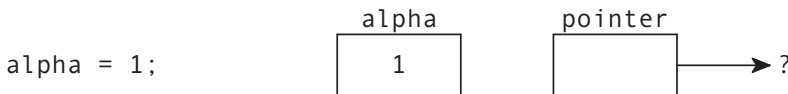
Ist alpha eine Variable vom Typ X, so liefert der Ausdruck &alpha einen Pointer auf das Objekt alpha vom Typ „Pointer auf X“.



Im folgenden Beispiel wird eine int-Variable alpha definiert und ein Pointer pointer auf eine int-Variable:



Durch die folgende Zuweisung `alpha = 1` wird der int-Variablen alpha der Wert 1 zugewiesen, das heißt an der Stelle des Adressraums des Rechners, an der alpha gespeichert wird, befindet sich nun der Wert 1. Zu diesem Zeitpunkt ist der Wert des Pointers noch undefiniert, denn ihm wurde noch nichts zugewiesen:



Erst durch die Zuweisung `pointer = &alpha` erhält der Pointer pointer einen definierten Wert, nämlich die Adresse der Variablen alpha. Dies zeigt das folgende Bild:



Der Adress-Operator liefert als Ergebnis die Adresse einer bereits im Speicher angelegten Variablen und diese Adresse kann in einer Pointer-Variablen (oder: in einem Pointer) gespeichert werden.

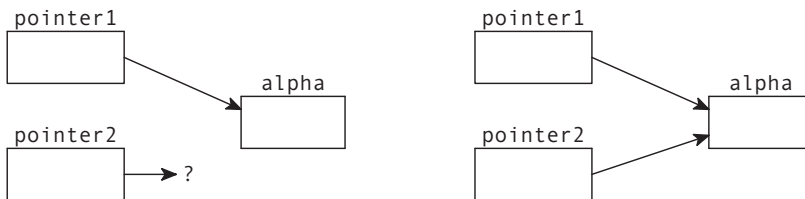


Eine Ausnahme bilden hierbei die register-Variablen (siehe Kapitel [15.6](#)). Da sie gegebenenfalls vom Computer in Registern des Prozessors abgelegt werden, kann der Adress-Operator hier nicht verwendet werden.

Mittels des Adress-Operators kann man einem Pointer also grundsätzlich jede beliebige Adresse zuweisen. Da Adressen nichts anderes als Integer sind, könnte man versucht sein, Adressen direkt als solche hinzuschreiben. Dies ist jedoch nicht erlaubt. Erlaubt sind jedoch Zuweisungen von anderen Pointern, sofern sie denselben Typ besitzen:

```
int* pointer1 = &alpha;  
int* pointer2;  
pointer2 = pointer1;
```

Dadurch wird dem Pointer `pointer2` der Wert des Pointers `pointer1` zugewiesen. Nach der Zuweisung haben beide Pointer denselben Inhalt und zeigen damit beide auf das Objekt, auf das zunächst von `pointer1` verwiesen wurde:



Um zu verifizieren, dass beide Pointer auf dieselbe Adresse zeigen, kann man sich die Adresse auch ausgeben lassen.

Pointer können mittels `printf()` mit dem Formatelement `%p` ausgegeben werden.



```
printf("alpha liegt an der Adresse %p\n", pointer1);  
printf("alpha liegt an der Adresse %p\n", pointer2);
```

Je nach Compiler und Laufzeitumgebung kann die Ausgabe unterschiedlich ausfallen. Eine mögliche Ausgabe wäre:

```
alpha liegt an der Adresse 020f35b0  
alpha liegt an der Adresse 020f35b0
```

8.1.4 NULL-Pointer

Wie im vorherigen Kapitel gezeigt, ist der Wert einer lokalen Pointer-Variablen nach der Variablendefinition zunächst unbestimmt, sprich undefiniert. Der Pointer referenziert also irgendeine Speicherstelle im Adressraum des Programms.

Ein Pointer als lokale Variable, der nicht initialisiert wurde, enthält einen zufälligen Wert und zeigt somit auf eine beliebige Speicherstelle. Er ist als solches nicht zu unterscheiden von einem Pointer mit einem gültigen Wert.



Um eine Pointer-Variable somit mit einem gültigen Wert zu initialisieren, muss ihr entweder direkt ein gültiger Wert zugewiesen werden, oder aber ihm wird der Wert NULL zugewiesen.

Ein **NULL**-Pointer hat den Wert 0.



Der Pointer NULL ist ein vordefinierter Pointer, dessen Wert sich von allen regulären Pointern unterscheidet. Im C-Standard ist festgelegt, dass Variablen und Funktionen immer an Speicherplätzen abgelegt werden, deren Adressen von 0 verschieden sind.

Die Konstante NULL ist in `<stddef.h>` somit als 0 definiert und zeigt damit auf kein gültiges Speicherobjekt. Die kann gleichbedeutend verwendet werden mit einem typfreien Pointer auf die Adresse 0, sprich `(void*) 0`. Siehe dazu auch Kapitel [8.2](#).

```
int* pointer = NULL;
```

Die Zuweisung des Wertes NULL zu einem Pointer wird beim Programmieren synonym verwendet zu Bedeutungen wie „leer“, „nicht gesetzt“, „ungültig“ oder „gelöscht“.

Der NULL-Pointer wird häufig dazu verwendet, Pointer-Variablen bei der Definition sofort zu initialisieren. Ein Pointer, der mit NULL initialisiert ist, zeigt an, dass er noch auf keine gültige Speicherstelle zeigt.



Funktionen, die einen Pointer als Funktionsergebnis liefern, können den NULL-Pointer verwenden, um eine erfolglose Aktion anzuzeigen. Liegt kein Fehler vor, so haben sie als Rückgabewert stets die Adresse eines Speicherobjektes, die von 0 verschieden ist.

Ab dem Standard C23 gibt es auch das vorgegebene Schlüsselwort `nullptr`. Damit gleicht sich der C-Standard der Sprache C++ an, wo dieses Schlüsselwort bereits existiert. Die Angabe `nullptr` ist grundsätzlich gleichzustellen mit NULL, ist aber zu bevorzugen, da damit durch einen speziell für Nullpointer eingeführten Typ besser auf Typsicherheit geprüft werden kann.

8.1.5 Zugriff auf ein Objekt über einen Pointer

Pointer-Variablen speichern nicht selbst Objekte, sondern zeigen nur darauf. Dies wird auch als „**Referenzieren**“ bezeichnet.

Referenzieren heißt, dass man mit einer Adresse auf ein Speicherobjekt zeigt.



Wurde einem Pointer ein Wert zugewiesen, so will man natürlich auch auf das referenzierte Objekt, also auf das Objekt, auf das der Pointer zeigt, zugreifen können. Dazu gibt es in C den **Dereferenzierungs-Operator** `*`.

Wird der Dereferenzierungs-Operator auf einen Pointer angewandt, so wird direkt auf das Objekt zugegriffen, auf das der Pointer verweist.

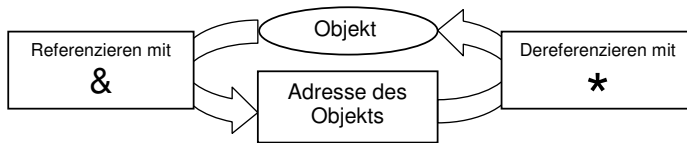


Ein Pointer referenziert ein Objekt – mit anderen Worten, er verweist auf das Objekt. Mit Hilfe des Dereferenzierungs-Operators erhält man aus einem Pointer, also einer Referenz auf ein Objekt, das Objekt selbst.

Beispielsweise wird im Folgenden der Variablen alpha der Wert 2 zugewiesen:

```
int alpha = 1;
int* pointer = &alpha;
*pointer = 2;
```

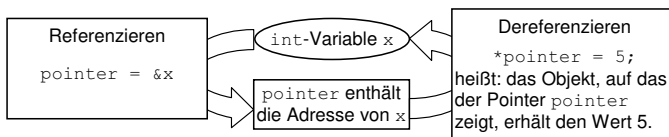
Es ist wichtig zu verstehen, dass der Dereferenzierungs-Operator komplementär zum Adress-Operator sich verhält. Das folgende Bild visualisiert die Verwendung des Adress- und Dereferenzierungs-Operators:



Diese Dualität ist das fundamentale Prinzip der Pointer-Programmierung:

- & wird als Adress-Operator bezeichnet. Mit ihm erhält man die Adresse eines Objekts.
- * wird als Dereferenzierungs-Operator bezeichnet. Mit ihm erhält man den Inhalt des Objekts, auf das der Pointer zeigt.

Folgendes ist ein konkretes Beispiel für die Verwendung des Adress- und Dereferenzierungs-Operators:



8.1.6 Beispiele für das Referenzieren und Dereferenzieren

Durch das komplementäre Verhalten des Adress-Operators und des Dereferenzierungs-Operators gilt folgendes:

`*&alpha` ist äquivalent zu `alpha`.



Folgendes lauffähiges Beispiel veranschaulicht dies:

pointer.c

```
#include <stdio.h>

int main(void) {
    float zahl = 3.5f;
    printf("Adresse von zahl: %p\n", &zahl);
    printf("Wert von zahl: %f\n", *&zahl);
    printf("Wert von zahl: %f\n", *&*&*&*&zahl);
    return 0;
}
```

Natürlich ist die wiederholte Anwendung der beiden Operatoren wenig sinnvoll und dient hier nur zur Veranschaulichung der Äquivalenz. Die Ausgabe des Programmes ist die Folgende:

```
Adresse von zahl: 0x16fdff248
Wert von zahl: 3.500000
Wert von zahl: 3.500000
```

Dabei ist zu beachten, dass die Adresse auf einem anderen Rechner natürlich anders ausfallen kann.

Das folgende Programm demonstriert erneut die beiden Operatoren:

ptrOp.c

```
#include <stdio.h>

int main(void) {
    int alpha;
    int* pointer1;
    int* pointer2;

    pointer1 = &alpha;
    *pointer1 = 5;
    printf("%d\n", *pointer1);

    *pointer1 = *pointer1 + 1;
    pointer2 = pointer1;
    printf("%d\n", *pointer2);

    return 0;
}
```

Zuerst wird `pointer1` mithilfe des Adress-Operators die Adresse von `alpha` zugewiesen. Dann wird mittels des Dereferenzierungs-Operators der Wert 5 an die Stelle im Speicher, wohin die Adresse zeigt, zugewiesen und dann ebenfalls mittels des Dereferenzierungs-Operators wiederum ausgelesen und ausgegeben.

Danach wird mittels des Dereferenzierungs-Operators der Wert an der Adresse ausgelesen, mit 1 aufsummiert und erneut zurückgeschrieben. Der zweite Pointer `pointer2` zeigt danach ebenfalls auf die selbe Adresse wie `pointer1` und der Inhalt der Speicherstelle an ebendieser Adresse wird erneut ausgegeben.

Hier die Ausgabe des Programmes:

```
5
6
```

8.1.7 Häufigste Fehlerquellen

Mit Pointern zu arbeiten ist nicht problemlos. Sehr schnell schleichen sich fehlerhafte Adressen in den Verlauf eines Programmes:

```
int* pointer1;  
*pointer1 = 6;
```

In diesem Beispiel wurde der Pointer `pointer1` nicht initialisiert, womit er auf eine beliebige Speicherstelle im gesamten Adressraum des Computers zeigt. Damit kann ungewollt eine beliebige Speicherstelle verändert werden. Liegt die Speicherstelle außerhalb des eigenen Adressbereichs, wird das Betriebssystem das Programm abbrechen, um einen Zugriff auf fremde Daten zu unterbinden.

Bestenfalls führt ein Zugriff auf eine ungültige Adresse zum Absturz des Programmes, schlimmstenfalls zu einer sogenannten „**heap corruption**“, was viele Stunden Fehlersuche nach sich ziehen kann.



Der Zugriff auf eine falsche Adresse ist ein häufiger und oftmals schwer aufzufindender Programmierfehler in C.



In neueren Programmiersprachen wie beispielsweise in Java hat man aus Gründen der Sicherheit keinen Zugriff mehr auf die Adresse einer Variablen im Arbeitsspeicher.

Eng mit dem obigen Beispiel verwandt ist auch der Zugriff auf einen **NULL**-Pointer:

```
int* pointer2 = NULL;  
*pointer2 = 6;
```

Hier wird versucht, einen Pointer zu dereferenzieren, welcher jedoch die ungültige Adresse `NULL` enthält. In diesem Falle wird das Betriebssystem das Programm abstürzen lassen. Heutzutage spricht man hierbei von einer sogenannten „**NULL-Pointer-Exception**“, abgeleitet von der Ausnahmebehandlung moderner Programmiersprachen.

Der Zugriff auf einen NULL-Pointer führt zu einem Absturz des Programmes.



Ein NULL-Pointer-Zugriff ist einer der häufigsten Absturz-Ursachen von Programmen. Demgegenüber ist jedoch die Fehlersuche bei NULL-Pointer-Zugriffen oftmals trivial, im Gegensatz zur obengenannten Heap-Corruption.

8.1.8 Fortgeschrittene Pointer-Definitionen

Bei aufmerksamer Lektüre ist bestimmt aufgefallen, dass eine Pointer-Variable selbst auch ein Speicherobjekt darstellen kann.

Da Pointer-Variablen ebenfalls Speicherobjekte sind, gibt es die Möglichkeit, dass Pointer-Variablen auf Pointer-Variablen zeigen.



Zudem sei hier angemerkt, dass auch Funktionen Speicherobjekte darstellen und es somit auch möglich ist, Pointer-Variablen auf Funktionen zeigen zu lassen.

Pointer auf Pointer und Pointer auf Funktionen (Funktionspointer) werden in Kapitel [→ 12](#) noch behandelt.

8.2 Pointer auf void

Wenn bei der Definition des Pointers der Typ der Variablen, auf die der Pointer zeigen soll, noch nicht feststeht, wird ein **Pointer auf den Typ void** vereinbart.



```
void* pointer;
```

Der Pointer auf den Typ void darf selbst nicht zum Zugriff auf Objekte verwendet werden, das heißt er darf nicht dereferenziert werden, da nicht bekannt ist, auf welche Objekte er verweist.



Später kann dann der Pointer in einen Pointer auf einen bestimmten Typ umgewandelt werden.

Der Pointer auf void ist ein untypisierter (typfreier, generischer) Pointer. Dieser ist zu allen anderen Pointer-Typen kompatibel und kann insbesondere in Zuweisungen mit typisierten Pointern gemischt werden.



Dies bedeutet, dass in der Programmiersprache C bei einer Zuweisung links des Zuweisungs-Operators ein typfreier Pointer stehen darf und rechts ein typisierter Pointer und auch umgekehrt! Ein Pointer auf void umgeht also bei einer Zuweisung die Typüberprüfungen des Compilers. Heutige Compiler geben jedoch auf Wunsch Warnungen aus.

Abgesehen von void* darf ohne explizite Typumwandlung kein Pointer auf einen Datentyp an einen Pointer auf einen anderen Datentyp zugewiesen werden. Jeder Pointer auf eine Variable kann durch eine Zuweisung in den Typ void* und zurück umgewandelt werden, ohne dass Information verloren geht.



void-Pointer können in jeden anderen Pointer-Typ gecastet werden, solange sie sich nicht in ihren Qualifikatoren (const, restrict, volatile) unterscheiden. Auch der umgekehrte Weg ist möglich: Jeder Pointer-Typ kann in einen entsprechenden void-Pointer gecastet werden.



Der Typumwandlungs-Operator – auch cast-Operator genannt – wird in Kapitel [→ 9.9.5](#) behandelt. Ein Beispiel zur Nutzung von Pointern auf void ist in Kapitel [→ 12.13](#) zu finden.

8.3 Arrays

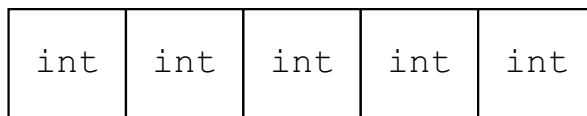
Unter einem **Array** versteht man die geordnete Aneinanderreihung von mehreren Variablen des gleichen Typs unter einem gemeinsamen Namen. Die allgemeine Form der Definition eines eindimensionalen Arrays ist:

```
Typname Arrayname[ANZAHL];
```

Dabei ist ANZAHL die Anzahl der **Array-Elemente**. Konkrete Beispiele hierfür sind:

```
int alpha[5];           // Array aus 5 Elementen vom Typ int
char beta[6];           // Array aus 6 Elementen vom Typ char
```

Das folgende Bild visualisiert die Speicherbelegung eines Arrays aus fünf int-Elementen:



Alle Elemente des Arrays werden vom Compiler direkt hintereinander im Arbeitsspeicher angelegt.

Ein C-Compiler erkennt ein Array an den eckigen Klammern, die bei der Definition die Anzahl der Elemente enthalten können. Die Anzahl der Elemente muss, wenn vorhanden, immer ein positiver Integer sein. Sie muss normalerweise gegeben sein durch eine Konstante oder einen konstanten Ausdruck. Erst ab dem C11-Standard werden sogenannte VLA (variable length arrays) als Erweiterung erlaubt, bei welchen die Anzahl nicht konstant sein muss.

Ist die Anzahl an Elementen erst zur Laufzeit bekannt, können Arrays mit Hilfe der Funktion `malloc()` dynamisch konstruiert werden. Darüber kann in Kapitel [→ 18.3](#) mehr erfahren werden.

8.3.1 Initialisierung eines Arrays

Wird ein Array definiert, so sind die Inhalte dessen zu Beginn unbestimmt. Um ein Array bei der Definition direkt mit Inhalten zu füllen, gibt es die sogenannte „**Aggregats-Initialisierung**“:

```
int alpha[5] = { 32, 62, 17, 105, 30 };
```

Die Zahlen innerhalb der geschweiften Klammern werden vom Compiler automatisch der Reihe nach in das Array geschrieben. Dies funktioniert aber nur bei Initialisierungen, sprich direkt bei der Definition der Array-Variablen.

Da der Compiler die Anzahl der Elemente in einer Aggregats-Liste ermitteln kann, ist es erlaubt, die Anzahl an Elementen bei der Definition wegzulassen:

```
int alpha[] = { 32, 62, 17, 105, 30 };
```

Auch wenn Aggregats-Initialisierungen einfach aussehen, benötigt der Computer Laufzeit, um alle Werte an die richtigen Stellen im Hauptspeicher zu schreiben.



8.3.2 Zugriff auf Elemente eines Arrays

Das folgende Beispiel zeigt ein Array aus fünf `int`-Elementen:

```
int alpha[5];
```

Um nun die einzelnen Elemente des Arrays anzusprechen, kann einfach der Name des Arrays zusammen mit dem **Array-Element-Operator** verwendet werden. Beispielsweise kann so das Element mit Index 3 angesprochen werden:

```
alpha[3];
```

Ob dieser Zugriff nun lesend oder schreibend ist, spielt keine Rolle. Folgender Code ist also möglich:

```
alpha[2] = 1234;  
int myValue = alpha[2];
```

Hier wurde in der ersten Zeile an die Stelle mit Index 2 der Wert 1234 zugewiesen. In der zweiten Zeile wurde ebendieser Wert wiederum gelesen und einer neuen Variable `myValue` zugewiesen.

Der Zugriff auf ein Element eines Arrays erfolgt über den **Array-Index**. Hat man ein Array mit `n` Elementen definiert, so ist darauf zu achten, dass in C die Indizierung der Array-Elemente mit 0 beginnt und bei `n - 1` endet.



Da der Index ein Integer sein muss, kann der Index auch aus einer Variablen kommen, wie beispielsweise:

```
int index = 2;  
int myValue = alpha[index];
```

C-Compiler erlauben hierbei jeden beliebigen Typ, der zu einem Integer ausgewertet. Korrekterweise sollte man jedoch für einen Array-Index den speziell hierfür definierten Typ **size_t** verwenden:

```
size_t index = 2;
int myValue = alpha[index];
```

Der Sinn eines speziellen Typs für einen Array-Index ist, dass der Typ `int` eine implementationsabhängige Größe hat, welche nicht zwingendermaßen mit der Größe übereinstimmen muss, mit welcher Adressen gebildet werden. In so einem Falle wäre es nicht möglich, mit einem `int` alle Elemente eines sehr großen Arrays zu adressieren. Der Typ `size_t` hingegen erlaubt stets die Adressierung jedes beliebigen Elements.

8.3.3 Arrays und Schleifen

Wir betrachten das folgende, einfache Array:

```
#define ANZAHL 5
int alpha[ANZAHL];
```

Um dieses Array mit Werten zu belegen, kann folgendes geschrieben werden:

```
alpha[0] = 1;
alpha[1] = 2;
alpha[2] = 3;
alpha[3] = 4;
alpha[4] = 5;
```

Dies ist jedoch sehr umständlich und birgt großes Fehlerpotential. Viel einfacher ist es, ein Array mittels einer Schleife zu bearbeiten.

Der Vorteil von Arrays gegenüber mehreren einfachen Variablen ist, dass Arrays sich leicht mit Schleifen bearbeiten lassen.



Diese for-Schleife tut dasselbe wie der Code vorher:

```
for (size_t index = 0; index < ANZAHL; index = index + 1) {  
    alpha[index] = index + 1;  
}
```

Mittels einer Schleife kann man die Werte der Elemente des oben eingeführten Arrays alpha auch ausgeben:

```
for (size_t index = 0; index < ANZAHL; index = index + 1) {  
    printf("%d, ", alpha[index]);  
}
```

Oder den Durchschnitt aller Werte des Arrays berechnen:

```
double summe = 0.f;  
for (size_t index = 0; index < ANZAHL; index = index + 1) {  
    summe = summe + alpha[index];  
}  
printf("Der Durchschnitt ist: %f", summe / ANZAHL);
```

Auf jeden Fall sollte man es sich bei Arrays zur Gewohnheit machen, immer mit symbolischen Konstanten wie beispielsweise `#define ANZAHL 5` und nicht direkt mit literalen Konstanten zu arbeiten. Soll nämlich das vorliegende Beispiel erweitert werden, sodass das Array 10 Elemente enthalten soll anstelle von 5, so müsste man ohne die Konstante doch an vielen Stellen Änderungen vornehmen, von denen leider gerne welche vergessen werden.



8.3.4 Strings und Arrays

Ein **String** wird vom Compiler intern als ein Array von Zeichen (char-Array) dargestellt. Dabei wird am Schluss ein zusätzliches Zeichen, das Zeichen `'\0'` (**Nullzeichen**) angehängt, um das Stringende zu markieren.



Stringverarbeitungsfunktionen benötigen unbedingt dieses Nullzeichen, damit sie das Stringende erkennen. Deshalb muss bei der Speicherung von Strings stets ein Speicherplatz für das Nullzeichen vorgesehen werden. So hat beispielsweise die folgende Definition nur Platz für 14 Buchstaben und das abschließende `'\0'`-Zeichen:

```
char vorname[15];
```

Werden Strings kopiert oder anderweitig verändert, so muss stets darauf geachtet werden, das `'\0'`-Zeichen ans Ende zu setzen. Das Vergessen dieses Zeichens ist ein häufiger Fehler.



Oftmals wird ein String direkt bei der Definition mit einer String-Konstante initialisiert. Damit man nicht mühsam die Anzahl Zeichen zählen muss, erledigt dies der Compiler mit der folgenden Schreibweise:

```
char stadtName[] = "New York";
```

Hierbei wird auch automatisch das abschließende `'\0'`-Zeichen mitgezählt.

8.3.5 Beispiel für Strings und Arrays

Das folgende Beispiel durchsucht ein char-Array nach dem ersten Zeichen 'a':

charArray.c

```
#include <stdio.h>
#include <string.h>

int main(void) {
    const char eingabe[] = "Dieses Programm findet Buchstaben.";
    size_t index = 0;

    while (eingabe[index] != '\0') {
        if (eingabe[index] == 'a') {
            break;
        }
        index = index + 1;
    }

    if (eingabe[index] == '\0') {
        printf("Ihr String enthaelt kein 'a'\n");
    } else {
        printf("Das erste a befand sich an Index %d.\n", (int)index);
    }

    return 0;
}
```

Die Programmausgabe ist:

```
Das erste a befand sich an Index 12.
```

Das Programm initialisiert das char-Array eingabe mit einem String. Dieser String wird dann in der while-Schleife Zeichen für Zeichen durchlaufen. Die Schleife bricht ab, wenn das '\0'-Zeichen erreicht wird. Jedes Zeichen wird innerhalb der Schleife mit dem gesuchten Zeichen 'a' verglichen. Wird ein 'a' gefunden, dann wird die Schleife mit break verlassen. Am Ende der Schleife wird das Zeichen an der Stelle eingabe[index] erneut mit dem Zeichen '\0' verglichen. Ist das Zeichen gleich '\0', dann wurde der String bis zum Ende durchsucht, ohne ein 'a' zu finden. Ansonsten wurde ein 'a' mit dem Index index gefunden.

Weitere Beispielprogramme mit Arrays und Strings befinden können in Kapitel → 12 nachgelesen werden.

8.4 Der Qualifikator restrict

Der **restrict**-Qualifikator wird verwendet, wenn zwei oder mehr Arrays oder Pointer lokal definiert sind. Wenn der restrict-Qualifikator angegeben wird, so kann der Compiler davon ausgehen, dass sich die beiden Arrays, beziehungsweise die Speicherbereiche, auf welche die Pointer zeigen, nicht überlappen.

Dadurch ist es dem Compiler möglich, bessere **Optimierungen** vorzunehmen. Denn wenn die Speicherbereiche nicht mit restrict deklariert werden, muss jede Auswertung einer Array-Variablen oder eines Pointers in der durch das Programm fest vorgegebenen Reihenfolge ausgewertet und zugewiesen werden. Es könnte ja sein, dass in eine Variable ein Wert geschrieben wird, welcher an anderer Stelle wieder gelesen werden muss. In diesem Falle darf der Compiler die beiden Anweisungen nicht austauschen, auch wenn sie rein logisch möglicherweise austauschbar und damit optimierbar wären.

Das Schlüsselwort restrict gibt es seit C99.

Die Verwendung des Schlüsselwortes restrict ist insbesondere für die Übergabe von Arrays als Funktionsparameter interessant. Bei der Übergabe eines Arrays wird nicht das gesamte Array, sondern nur ein Pointer auf das erste Element des Arrays übergeben. Mit dem Schlüsselwort restrict kann festgelegt werden, dass die Pointer der aktuellen Parameterliste keine Speicherbereiche bezeichnen, die sich überlappen.



Hier als Beispiel der Prototyp der String-Kopier-Funktion strcpy() aus der Standardbibliothek des C11-Standards mit Pointern als Übergabeparameter mit dem Qualifikator restrict:

```
char* strcpy(char* restrict s1, const char* restrict s2);
```

Hier bedeutet restrict somit konkret, dass s1 und s2 auf verschiedene disjunkte Objekte zeigen.

Hat man in einer Aufrufchnittstelle mehrere restrict-Pointer, so müssen sie auf verschiedene Objekte verweisen.



Definiert man einen Pointer `ptr` mit dem Qualifikator `restrict`, so schafft man ein besonderes Verhältnis zwischen diesem Pointer `ptr` und dem referenzierten Objekt: Ein `restrict`-Pointer `ptr` deutet an, dass während der Lebensdauer des entsprechenden Pointers nur mit diesem Pointer `ptr` oder mit einem von diesem Pointer abgeleiteten Pointer, wie beispielsweise `ptr + 1`, auf ein Speicherobjekt wie beispielsweise ein Array von Zeichen zugegriffen wird.



Arbeitet man mit einem `restrict`-Pointer, kann ein Compiler Optimierungen des Maschinencodes durchführen, er ist aber dazu nicht verpflichtet.

Bei der Angabe von `restrict` bei einem Pointer auf ein Objekt muss beim Schreiben des Programmes sichergestellt werden, dass der Zugriff auf dieses Speicherobjekt nur über diesen `restrict`-Pointer erfolgt. Die Angabe des Schlüsselworts `restrict` bei der Pointerdefinition ist letztendlich lediglich ein Versprechen für den Compiler, dass ein Zugriff auf das entsprechende Speicherobjekt nur von diesem `restrict`-Pointer aus erfolgt.

Ein Compiler prüft das aber nicht! Überlappen sich Speicherbereiche, welche mit `restrict` deklariert sind, dennoch, so ist das Verhalten des Programms gemäß Standard undefiniert.



Da einige Funktionen der Standardbibliothek wie zum Beispiel die `str`- oder `mem`-Funktionen in Kapitel [→ 12.9](#) und [→ 12.8](#) Gebrauch von `restrict`-Pointern machen, lohnt es sich bei der Verwendung dieser Funktionen, deren Deklaration genau zu betrachten, um einem „undefinierten Verhalten“ zu entgehen.

8.5 Übungsaufgaben

Aufgabe 1: Pointer und Adress-Operator

Schreiben Sie ein einfaches Programm, das die folgenden Definitionen von Variablen und die geforderten Anweisungen enthält:

- Definition einer Variablen `i` vom Typ `int`
- Definition eines Pointers `ptr` vom Typ `int*`
- Zuweisung der Adresse von `i` an den Pointer `ptr`
- Zuweisung des Wertes 1 an die Variable `i`
- Ausgabe des Wertes des Pointers `ptr`
- Ausgabe des Wertes von `i`
- Ausgabe des Wertes des Objekts, auf das der Pointer `ptr` zeigt, mit Hilfe des Dereferenzierungs-Operators
- Zuweisung des Wertes 2 an das Objekt, auf das der Pointer `ptr` zeigt, mit Hilfe des Dereferenzierungs-Operators
- Ausgabe des Wertes von `i`

Hinweis: Pointer werden bei `printf()` mit dem Formatelement `%p` ausgegeben.

Aufgabe 2: Array-Zugriffe

Überlegen Sie, was das folgende Programm ausgibt. Überzeugen Sie sich durch einen Programmlauf.

arrayExercise.c

```
#include <stdio.h>

int main(void) {
    size_t i;
    int ar[100];

    for (i = 0; i < 100; i = i + 1)
        ar[i] = 1;

    ar[11] = -5;
    ar[12] = ar[12] + 1;
    ar[13] = ar[0] + ar[11] + 4;

    for (i = 10; i <= 14; i = i + 1)
        printf("ar[%2d] = %4d\n", (int)i, ar[i]);

    return 0;
}
```

Aufgabe 3: Eigene Array-Programme

Nutzen Sie das Programm der Aufgabe 8.2 als Vorlage und erstellen Sie je ein eigenes Programm:

- Definieren Sie einem Array mit 128 Zeichen und weisen Sie diesem die Zeichen des ASCII-Zeichensatzes zu. Geben Sie die Zeichen mit dem ASCII-Code 48 bis 57 am Bildschirm aus.
- Erstellen Sie ein Array mit 10 `int`-Elementen und initialisieren Sie sie mit beliebigen Werten. Ermitteln Sie, welches Element den größten Wert hat und geben Sie die Nummer des Elements und seinen Wert am Bildschirm aus.
- Definieren sie zwei Arrays `a` und `b` aus je 3 `double`-Elementen. Bestimmen Sie das Skalarprodukt ($a_1 \cdot b_1 + a_2 \cdot b_2 + a_3 \cdot b_3$) mittels einer Schleife über alle drei Array-Elemente und geben Sie das Resultat am Bildschirm aus.

Aufgabe 4: Fehlende Überprüfung auf Überschreitung der Feldgrenzen bei Arrays

Führen Sie einen Programmlauf mit dem folgenden Programm durch. Analysieren Sie das Ergebnis!

arrayOutOfBounds.c

```
#include <stdio.h>

int main(void) {

    int i = 16;
    int k = 21;
    int l = 22;
    int p = 23;
    int q = 24;

    int ar[100];

    for (size_t i = 0; i < 100; i = i + 1)
        ar[i] = 27;

    printf("i ist %d\n", i);
    printf("ar[-1] ist %d\n", ar[-1]);
    printf("ar[0] ist %d\n", ar[0]);
    printf("ar[100] ist %d\n", ar[100]);
    printf("ar[101] ist %d\n", ar[101]);
    printf("ar[102] ist %d\n", ar[102]);
    printf("ar[103] ist %d\n", ar[103]);
    printf("ar[-2] ist %d\n", ar[-2]);
    printf("ar[-3] ist %d\n", ar[-3]);
    printf("k ist %d\n", k);
    printf("l ist %d\n", l);
    printf("p ist %d\n", p);
    printf("q ist %d\n", q);

    return 0;
}
```

Achtung, lassen Sie sich nicht von diesem Programmierbeispiel zu gewagten Indizierungs- oder Adressierungsmanövern verleiten. Überschreitungen von Feldgrenzen werden heutzutage allgemein als unzulässige Programmierung betrachtet. Das Beispiel dient lediglich der Illustration, was bei einer Überschreitung der Feldgrenzen passieren kann.

9 Ausdrücke, Anweisungen und Operatoren



In diesem Kapitel wird auf das Thema der Anweisungen und Ausdrücke sowie noch einmal detailliert auf die in C vorhandenen Operatoren eingegangen. Diese drei Themengebiete sind eng miteinander verknüpft.

In den ersten Unterkapiteln werden die in C gültigen Regeln für das Auswerten von Ausdrücken besprochen, sowie Möglichkeiten der Klassifizierung wie beispielsweise Rangordnung und Assoziativität. Außerdem wird auf Eigenheiten der Programmiersprache eingegangen wie Nebeneffekte, L-Werte und R-Werte.

Daraufhin werden die Operatoren einzeln mit Beispielen aufgelistet. Sie werden dabei nach ihrer Wirkungsweise klassifiziert, wie etwa arithmetische Operatoren, logische Operatoren, Zuweisungs-Operatoren oder Vergleichs-Operatoren.

9.1 Ausdrücke und Anweisungen

Im Quellcode einer C-Programmdatei werden grundsätzlich **Anweisungen** geschrieben. Sie werden vom Compiler in Prozessor-Befehle umgewandelt, die der Reihe nach vom Prozessor abgearbeitet werden.

Man unterscheidet zwischen zwei Arten von Anweisungen: Kontrollstrukturen und Ausdrucksanweisungen. Kontrollstrukturen werden in Kapitel → 10 behandelt.

Eine **Ausdrucksanweisung** besteht – wie der Name schon sagt – aus einem Ausdruck.

Ein **Ausdruck** ist eine Aneinanderreihung von Operanden und Operatoren. Jeder Ausdruck hat einen Rückgabewert und kann Teil eines größeren Ausdrucks sein.



Ergänzende Information Die elektronische Version dieses Kapitels enthält Zusatzmaterial, auf das über folgenden Link zugegriffen werden kann https://doi.org/10.1007/978-3-658-45209-4_9.

Folgendes sind Beispiele von Ausdrücken:

```
x
7
5 * 5
(a + b) * (c - d)
sin(PI / 2.)
i++
i = 0
printf("Hallo")
```

Ein Ausdruck ist in C im einfachsten Fall der Bezeichner (Name) einer Variablen oder einer Funktion, eine Konstante oder ein String. Meist interessiert der Wert eines Ausdrucks.



Werden alle Operanden eines Ausdrucks zu einer Konstanten aus, so handelt es sich um einen sogenannten „**konstanten Ausdruck**“ (auf Englisch „**const expression**“). Ein solcher Ausdruck kann vom Compiler bereits zur Compilezeit aufgelöst werden und benötigt somit keinerlei Laufzeit. Im Standard C23 wird zur Markierung eines konstanten Ausdrucks das Schlüsselwort `constexpr` eingeführt.



So hat eine Konstante einen Wert, eine Variable kann einen Wert liefern, ebenso wie der Aufruf einer Funktion.

Der Wert eines Ausdrucks wird auch als sein **Rückgabewert** bezeichnet. Jeder Rückgabewert hat einen Typ. Ausdrücke können Werte an Speicherstellen verändern, was als „Nebeneffekt“ bezeichnet wird.



Durch Verknüpfungen von Operanden durch Operatoren und gegebenenfalls auch runde Klammern entstehen komplexe Ausdrücke. Runde Klammern beeinflussen dabei die Auswertungsreihenfolge. Jeder Operand kann selbst auch ein Ausdruck sein.



Die Ziele der Verknüpfungen von Operatoren und Operanden zu Ausdrücken sind:

- Die Berechnung neuer Werte. Alles, was als Verknüpfung von Operatoren geschrieben werden kann, hat einen Wert und stellt einen Ausdruck dar.
- Das Erzeugen von gewollten Nebeneffekten.
- Ein Speicherobjekt, also eine Variable oder eine Funktion zu erhalten.

In C kann jeder Ausdruck eine Anweisung werden, indem ihr ein Semikolon angehängt wird.



Anweisungen haben keinen Rückgabewert. Sie stellen ein abgeschlossenes Element dar. Das bedeutet, dass der Rückgabewert des Ausdrucks der Anweisung nicht weiter behandelt wird. Folgendes sind Beispiele von Ausdrucksanweisungen:

```
5 * 5;  
i = 0;  
printf("Hallo");  
ang = sin(2 * PI) / 4.;
```

Dabei ist zu beachten, dass eine Ausdrucksanweisung wie `5 * 5` zwar erlaubt ist, jedoch keine Wirkung hat. Ausdrucksanweisungen sind lediglich dann sinnvoll, wenn der Ausdruck einen sogenannten „Nebeneffekt“ hat.

9.1.1 Nebeneffekte

Nebeneffekte werden auch als Seiteneffekte oder als Nebenwirkungen bezeichnet und bedeuten nichts anderes als: Speicherinhalte werden verändert.



Ein Ausdruck wie $5 * 5$ gibt zwar einen Wert zurück, verändert jedoch keine Speicherinhalte. Einen solchen Ausdruck in eine Anweisung umzuwandeln ist nutzlos, da er nichts bewirkt.

Diejenigen Ausdrücke, welche offensichtlich Nebeneffekte aufweisen, sind die Zuweisungen, welche mit dem Zuweisungs-Operator `=` geschrieben werden:

```
i = 1;
```

Hier ist eindeutig klar, dass der Inhalt (der Speicherinhalt) der Variablen `i` mit dem Wert 1 überschrieben wird. Dies ist der Nebeneffekt dieser Anweisung.

In der Programmiersprache C gibt es jedoch Operatoren, die nicht offensichtlich Nebeneffekte hervorrufen. Sie wurden definiert, um eine schnelle und kurze Programmierschreibweise zu erlauben. Es ist nämlich möglich, während der Auswertung eines Ausdrucks Programmvariablen nebenbei zu verändern. Ein Beispiel hierzu ist:

```
i++;
```

Der Operator `++` ist der sogenannte Inkrement-Operator. Er gibt hier grundsätzlich den Wert der Variablen `i` zurück, erhöht als Nebeneffekt jedoch zusätzlich die Variable `i` um 1.

Dann gibt es auch noch den Dekrement-Operator `--`, welcher die Variable um 1 verringert. Die beiden Operatoren `++` und `--` haben eine lange Tradition in den C-Sprachen und werden in diesem Kapitel noch an einigen Stellen erwähnt werden. Sie gelten jedoch aufgrund dessen, dass sie diese Nebeneffekte aufweisen, als nicht unproblematisch.

9.1.2 Sequenzpunkte bei Nebeneffekten



Nebeneffekte sind grundsätzlich notwendig, um überhaupt irgendetwas mit einem Programm zu bewirken, da ansonsten keinerlei Speicherinhalte verändert werden können. Wenn jedoch in einem Ausdruck Speicherinhalte gleichzeitig gelesen wie auch geschrieben werden, so kann dies unter gewissen Umständen zu Problemen führen, wie das folgende Beispiel zeigt:

```
n++ - n
```

Hier könnte der Compiler erst den linken Operanden auswerten und dessen Nebeneffekt durchführen. Dann ist der Wert des gesamten Ausdrucks -1 . Wird jedoch erst der rechte Operand ausgewertet, so ist der Wert des gesamten Ausdrucks gleich 0 .

Auch im folgenden Beispiel ist das Ergebnis nicht definiert, da beide Argumente einen Nebeneffekt haben.

```
a = f1(par++, par++);
```

Zwar wird gemäß Standard verlangt, dass alle Nebeneffekte vor dem eigentlichen Aufruf der Funktion abgearbeitet sein müssen, aber die Reihenfolge, in welcher die Argumente ausgewertet werden, ist Sache des Compilers. Es ist somit nicht klar, welche Werte tatsächlich an die Funktion übergeben werden.

Problematisch wird es auch, wenn zwei Funktionen $f1()$ und $f2()$ auf dieselben globalen Variablen zugreifen und diese verändern, also einen Nebeneffekt haben. Das Ergebnis des folgenden Ausdrucks ist dann ebenfalls nicht definiert, da der Standard nicht festlegt, welche der beiden Funktionen $f1()$ oder $f2()$ zuerst ausgewertet wird.

```
a = f1() * f2();
```


Um Nebeneffekte beim Programmieren unter Kontrolle zu halten, wurden im ISO-Standard sogenannte „**Sequenzpunkte**“ (auf Englisch „**sequence points**“) definiert.

An einem Sequenzpunkt müssen alle Nebeneffekte einer vorangegangenen Berechnungen durchgeführt worden sein und es dürfen noch keine Nebeneffekte von Ausdrücken stattgefunden haben, die im Programm nach dem Sequenzpunkt stehen.



Sequenzpunkte liegen an folgenden Stellen vor:

- Nach der Berechnung der Argumente und des Funktions-Bezeichners, bevor eine Funktion aufgerufen wird.
- Bei den folgenden Operatoren jeweils nach der Auswertung des ersten Operanden und vor der Auswertung des zweiten Operanden (gemäß der Abarbeitungsrichtung des Operators):
 - Logisches UND &&
 - Logisches ODER ||
 - Bedingungs-Operator, also der Bedingung $a \text{ in } a ? b : c$
 - Komma-Operator ,
- Am Ende der Auswertung der folgenden Ausdrücke:
 - Initialisierungsausdruck einer manuellen Initialisierung
 - Ausdruck in einer Ausdrucksanweisung
 - Bedingung in einer if-Anweisung
 - Selektionsausdruck in einer switch-Anweisung
 - Bedingung einer while- oder do while-Schleife
 - Alle drei Ausdrücke einer for-Anweisung
 - Ausdruck einer return-Anweisung

Aufgrund dieser Regeln ist undefiniertes Verhalten beim täglichen Programmieren äußerst selten anzutreffen. Dennoch gilt:

Wird ein Objekt zwischen zwei aufeinanderfolgenden Sequenzpunkten mehrmals gelesen, während es gleichzeitig verändert wird, so ist das Ergebnis nicht definiert.



Weitaus am verbreitetsten sind heutzutage versteckte Nebeneffekte bei der Parameterübergabe bei Funktionsaufrufen. Auf diese wird in Kapitel [→ 11.5.1](#) eingegangen.

9.1.3 L-Werte und R-Werte

Ausdrücke haben eine unterschiedliche Bedeutung, je nachdem, ob sie links oder rechts vom Zuweisungs-Operator stehen.

```
a = b
```

Hier beispielsweise steht der Ausdruck `b` auf der rechten Seite des Zuweisungs-Operators für einen Wert, während der Ausdruck `a` auf der linken Seite die Stelle angibt, an der dieser Wert zu speichern ist. Wenn wir dieses Beispiel noch etwas modifizieren, wird der Unterschied noch deutlicher:

```
a = a + 5
```

Der Variablenname `a`, der ja auch einen einfachen Ausdruck darstellt, wird hier in unterschiedlicher Bedeutung verwendet. Rechts vom Zuweisungs-Operator ist der Wert gemeint, der in der Speicherzelle `a` gespeichert ist, und links ist die Speicherzelle `a` gemeint, in der der Wert des Gesamtausdrucks der rechten Seite gespeichert werden soll.

Aus dieser Stellung links oder rechts des Zuweisungs-Operators wurden auch die Begriffe L-Wert und R-Wert abgeleitet.

Ein Ausdruck stellt einen **L-Wert** (**lvalue** oder **left value**) dar, wenn er sich auf ein Speicherobjekt bezieht. Steht ein Ausdruck links des Zuweisungs-Operators, so muss er ein L-Wert sein.



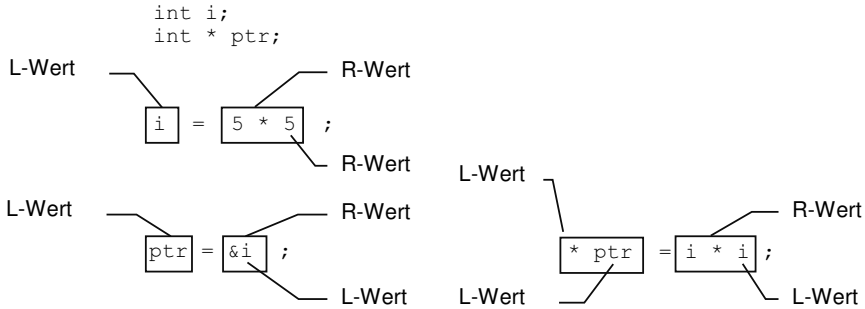
Ein Ausdruck, der keinen L-Wert darstellt, stellt einen **R-Wert** (**rvalue** oder **right value**) dar. Er bezieht sich nicht auf ein Speicherobjekt. Er darf nur rechts des Zuweisungs-Operators stehen.



Dabei ist zu beachten, dass ein Ausdruck, der einen L-Wert darstellt, auch rechts vom Zuweisungs-Operator stehen darf, er hat dann aber, wie oben erwähnt, eine andere Bedeutung. Steht ein L-Wert rechts vom Zuweisungs-Operator, so wird dessen Name beziehungsweise Adresse benötigt, um an der entsprechenden Speicherstelle den Wert der Variablen abzuholen. Links des Zuweisungs-Operators muss immer ein L-Wert stehen, da man den Namen beziehungsweise die Adresse einer Variablen braucht, um an der entsprechenden Speicherstelle den zuzuweisenden Wert abzulegen.

Damit einem L-Wert etwas zugewiesen werden kann, muss er jedoch zusätzlich noch modifizierbar sein. Ein nicht modifizierbarer L-Wert ist beispielsweise der Name eines Arrays. Ein Arrayname ist konstant und kann nicht modifiziert werden. Des Weiteren sind Variablen nicht modifizierbar, wenn sie einen unvollständigen Typen besitzen oder einen Typen mit dem Qualifikator `const`. Dies gilt insbesondere auch für Struktur- oder Union-Typen (siehe dazu Kapitel [→ 13.1](#) und [→ 13.3](#)), bei welchen eine seiner Komponenten – einschließlich aller rekursiv in einer Komponente enthaltenen Strukturen und Unionen – einen mit dem Qualifikator `const` versehenen Typ hat.

Auf der linken Seite einer Zuweisung darf also nur ein modifizierbarer L-Wert stehen, jedoch nicht ein R-Wert oder ein nicht modifizierbarer L-Wert. Das folgende Bild zeigt Beispiele für L- und R-Werte:



Bestimmte Operatoren können nur auf L-Werte angewandt werden. So kann man den Inkrement-Operator ++ oder den Adress-Operator & nur auf L-Werte anwenden.



5++ ist falsch, i++ korrekt, wobei i eine Variable darstellt.

Es ist zu beachten, dass ein Pointer zwar auf einen L-Wert zeigt, der Pointer selbst jedoch nicht zwingendermaßen als L-Wert verfügbar sein muss. Ein Pointer kann auch aus einem beliebigen Ausdruck entstehen, welcher zu einem R-Wert evaluiert.



```

int* pointer;
int alpha;
pointer = &alpha;
  
```

In diesem Beispiel ist &alpha ein R-Wert. (&alpha)++ ist nicht möglich, pointer++ hingegen schon. Demgegenüber ist jedoch sowohl *pointer = 2 als auch *&alpha = 2 zugelassen.

Folgende Operatoren erwarten stets einen L-Wert:

- Operand des Adress-Operators &
- Operand des Inkrement-Operators ++
- Operand des Dekrement-Operators --
- Der linke Operand des Punkt-Operators . bei Strukturen
- Der linke Operand des Zuweisungs-Operators =
- Der linke Operand des Funktionsaufruf-Operators ()

Folgende Operatoren geben einen L-Wert zurück:

- Rückgabewert des Punkt-Operators . bei Strukturen
- Rückgabewert des Pfeil-Operators -> bei Pointern
- Rückgabewert des Array-Element-Operators []
- Rückgabewert des Dereferenzierungs-Operators *

So wird im Ausdruck `&*pointer` beispielsweise der pointer zuerst dereferenziert, was einen L-Wert ergibt. Danach wird von diesem L-Wert gleich wieder die Adresse genommen.

9.2 Operatoren und Operanden

Die Sprache C kennt mehr als 30 **Operatoren**, von welchen die meisten in diesem Kapitel besprochen werden. Sie alle haben unterscheidbare Eigenschaften wie Priorität, die Anzahl an Operanden und Abarbeitungsrichtung (Assoziativität). Auf diese Eigenschaften wird in den folgenden Unterkapiteln eingegangen.

Folgende Tabelle gibt einen Überblick über die in C verfügbaren Operatoren:

Prio.	Operatoren		Operanden	Assoz.
1	() [] -> .	Funktionsaufruf Array-Index Komponentenzugriff	2	links
2	++ --	Postfix-Inkrement Postfix-Dekrement	1	links
3	! ~ ++ -- (Typname) sizeof + - * &	Negation logisch Negation bitweise Präfix-Inkrement Präfix-Dekrement Cast Byte-Größe Vorzeichen Dereferenzierung, Adresse	1	rechts
4	* / %	Multiplikation, Division Modulo	2	links
5	+ -	Summe, Differenz	2	links
6	<< >>	bitweises Schieben	2	links
7	< <= > >=	Kleiner, kleiner gleich Größer, größer gleich	2	links
8	== !=	Gleichheit, Ungleichheit	2	links
9	&	bitweises UND	2	links
10	^	bitweises Exklusiv-ODER	2	links
11		bitweises ODER	2	links
12	&&	logisches UND	2	links
13		logisches ODER	2	links
14	?:	bedingte Auswertung	3	rechts
15	= += -= *= /= %= &= ^= = <<= >>=	Wertzuweisungen	2	rechts
16	,	Komma-Operator	2	links

9.2.1 Anzahl Operanden

In C gibt es einstellige (unäre, monadische) Operatoren, zweistellige (binäre, dyadische) Operatoren und einen einzigen dreistelligen (ternären, triadischen) Operator.

Ein einstelliger (unärer) Operator hat einen einzigen Operanden.

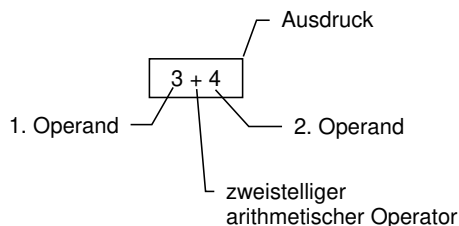


Ein Beispiel hierfür ist der Minus-Operator als Vorzeichen-Operator, der auf einen einzigen Operanden wirkt und das Vorzeichen dessen Wertes ändert. So ist in `-3` das `-` ein Vorzeichen-Operator, der auf die positive Konstante 3 angewandt wird.

Benötigt ein Operator zwei Operanden für die Verknüpfung, so spricht man von einem zweistelligen (binären) Operator.



Ein vertrautes Beispiel für einen binären Operator ist der Additions-Operator, der hier zur Addition der beiden Zahlen 3 und 4 verwendet werden soll:



Der einzige ternäre Operator ist der Bedingungs-Operator `?:` mit drei Operanden. Er wird in Kapitel [9.9.4](#) behandelt. Der erste Operand stellt eine Bedingung dar, gemäß welcher entweder der zweite oder dritte Operand ausgewertet wird:

```
(powerLevel > 9000) ? watchMeme() : doWork()
```

9.2.2 Postfix- und Präfix-Operatoren

Postfix-Operatoren sind unäre Operatoren, die hinter (post) ihrem Operanden stehen. **Präfix-Operatoren** sind unäre Operatoren, die vor (prä) ihrem Operanden stehen.



Der Ausdruck `i++` stellt die Anwendung des Postfix-Operators `++` auf seinen Operanden `i` dar.

Im Ausdruck `i++` bewirkt der Postfix-Operator `++`, dass die Variable `i` um 1 inkrementiert wird. Der Operator gibt jedoch den alten Wert von `i` zurück.



Im folgenden Beispiel wird durch den Postfix-Operator `++` als Nebeneffekt die Variable `i` um 1 erhöht, doch der Rückgabewert des Operators entspricht dem alten Wert von `i`. Erst ein erneutes Ansprechen der Variablen liefert den durch den Nebeneffekt errechneten und gespeicherten Wert:

```
int i = 3;
printf("%d", i++); // Ausgabe: 3
printf("%d", i);   // Ausgabe: 4
```

Genau andersrum funktioniert der Präfix-Operator `++`. Auch hier wird als Nebeneffekt die Variable `i` um 1 erhöht, doch wird nun der neue, sprich soeben berechnete Wert zurückgegeben:

```
int i = 3;
printf("%d", ++i); // Ausgabe: 4
printf("%d", i);   // Ausgabe: 4
```

Diese Unterscheidung ist jedoch nur bei den beiden Operatoren `++` und `--` von Wichtigkeit.

9.2.3 Auswertungsreihenfolge komplexer Ausdrücke

Hat man einen Ausdruck aus mehreren Operatoren und Operanden, so stellt sich die Frage, in welcher **Reihenfolge** die einzelnen Operatoren bei der Auswertung „dran kommen“.

Wie in der Mathematik spielt es bei C eine wichtige Rolle, in welcher Reihenfolge ein Ausdruck berechnet wird. So gilt beispielsweise auch in C die Regel „Punkt vor Strich“: Der Ausdruck $5 + 2 * 3$ ergibt somit 11 und nicht 21, da der Multiplikations-Operator vor dem Additions-Operator ausgeführt wird.

In C gibt es jedoch weitaus mehr Operatoren als nur die vier Grundoperationen. Daher muss genau festgelegt werden, welcher Operator im Zweifelsfall Priorität hat.

Es gelten die folgenden Regeln:

1. Wie in der Mathematik werden als erstes Teilausdrücke in **runden Klammern** ausgewertet.
2. Dann werden die Operatoren der Ausdrücke entsprechend festgelegter **Prioritäten** (auf Englisch „**operator precedence**“) abgearbeitet.
3. Wenn zwei Operatoren die gleiche Priorität besitzen, so wird mit der sogenannten „**Assoziativität**“ eine **Abarbeitungsrichtung** festgelegt, sprich, ob die Operatoren von rechts nach links oder von links nach rechts abgearbeitet werden.

Mithilfe der Tabelle zu Beginn dieses Kapitels ([→ 9.2](#)) kann man nun für jeden beliebigen Ausdruck bestimmen, in welcher Reihenfolge der Compiler die Operanden auswerten wird. In der Tabelle ist Priorität 1 die höchste Priorität. Beispielsweise ist in der Tabelle auch zu sehen, dass der Multiplikations-Operator wie in der Mathematik eine höhere Priorität als der Additions-Operator hat.

Als Beispiel zur Verdeutlichung der Vorgehensweise wird der folgende Ausdruck betrachtet.

```
*p++
```

Da dieser Ausdruck keine Klammern hat, wird bei der Auswertung dieses Ausdrucks nach Regel 2 verfahren: Der Inkrement-Operator `++` hat eine höhere Priorität als der Dereferenz-Operator `*`, also wird erst der Ausdruck `p++` ausgewertet und danach der Operator `*` auf den Rückgabewert des Ausdrucks `p++` angewandt.

Will man hingegen den Wert des Objektes `*p` durch den Postfix-Operator `++` erhöhen, so kann man mit Regel 1 diese Abarbeitungsreihenfolge erzwingen, indem man einfach Klammern um den gewünschten Ausdruck setzt: `(*p)++`.

`*p++` ist gleichbedeutend mit `*(p++)` und nicht mit `(*p)++`.



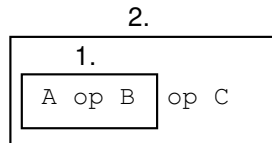
Haben zwei Operatoren dieselbe Priorität, so kommt die Regel 3 zum Zuge: Die Einhaltung der Assoziativität (Abarbeitungsrichtung).

Unter Assoziativität versteht man die Reihenfolge, wie Operatoren und Operanden verknüpft werden, wenn Operanden mit Operatoren gleicher Priorität (Vorrangstufe) miteinander verkettet sind.



Wiederum kann die Assoziativität von Operatoren in der Tabelle in Kapitel [→ 9.2](#) nachgelesen werden.

Ist ein Operator in C rechtsassoziativ, so wird eine Verkettung von Operatoren dieser Art von rechts nach links abgearbeitet, bei Linksassoziativität dementsprechend von links nach rechts. Das folgende Bild symbolisiert die Verknüpfungsreihenfolge bei einem linksassoziativen Operator op:



Es wird also zuerst der linke Operator op auf die Operanden A und B angewandt, und erst als zweites wird dann die Verknüpfung op mit C durchgeführt.

Als Beispiel dienen hier der Additions- und Subtraktions-Operator. Beide haben dieselbe Priorität:

$a - b + c$

Da beide Operatoren linksassoziativ sind, wird dieser Ausdruck wie $(a - b) + c$ verknüpft und nicht wie $a - (b + c)$.

Beachten Sie aber bitte, dass die Reihenfolge der Verknüpfung durch Operatoren nicht mit der Reihenfolge der Auswertung der Operanden übereinstimmen muss. Siehe dazu Kapitel [→ 9.1.2](#) .



Generell empfiehlt sich der Einsatz von Klammern, wenn man unsicher über die Priorität oder Assoziativität von Operatoren ist.

9.3 Einstellige arithmetische Operatoren

Im Folgenden werden die einstelligen (unären) Operatoren mit ihren Operanden anhand von Beispielen vorgestellt.

positiver Vorzeichen-Operator	<code>+x</code>
negativer Vorzeichen-Operator	<code>-x</code>
Postfix-Inkrement-Operator	<code>x++</code>
Präfix-Inkrement-Operator	<code>++x</code>
Postfix-Dekrement-Operator	<code>x--</code>
Präfix-Dekrement-Operator	<code>--x</code>

9.3.1 Positiver Vorzeichen-Operator: `+x`

Der positive Vorzeichen-Operator wird selten verwendet, da er lediglich den Wert seines Operanden wiedergibt. Es gibt keine Nebeneffekte.

`+a`

`+a` hat denselben Rückgabewert wie `a`.

9.3.2 Negativer Vorzeichen-Operator: `-x`

Will man den Wert eines Operanden mit umgekehrtem Vorzeichen erhalten, so ist der negative Vorzeichen-Operator von Bedeutung. Es gibt keine Nebeneffekte.

`-a`

`-a` hat denselben Betrag wie `a`, aber das umgekehrte Vorzeichen.

9.3.3 Postfix-Inkrement-Operator: x++

Der Rückgabewert ist der unveränderte Wert des Operanden. Als Nebeneffekt wird der Wert des Operanden um 1 inkrementiert. Bei Pointern wird um eine Objektgröße vom Typ, auf den der Pointer zeigt, inkrementiert. Der Inkrement-Operator kann nur auf modifizierbare L-Werte angewandt werden.

```
a = 1;  
b = a++;          // Ergebnis: b = 1, Nebeneffekt: a = 2  
ptr++;
```

9.3.4 Präfix-Inkrement-Operator: ++x

Der Rückgabewert ist der um 1 inkrementierte Wert des Operanden. Als Nebeneffekt wird der Wert des Operanden um 1 inkrementiert. Bei Pointern wird um eine Objektgröße vom Typ, auf den der Pointer zeigt, inkrementiert. Der Inkrement-Operator kann nur auf modifizierbare L-Werte angewandt werden.

```
a = 1;  
b = ++a;          // Ergebnis: b = 2, Nebeneffekt: a = 2  
++ptr;
```

9.3.5 Postfix-Dekrement-Operator: x--

Der Rückgabewert ist der unveränderte Wert des Operanden. Als Nebeneffekt wird der Wert des Operanden um 1 dekrementiert. Bei Pointern wird um eine Objektgröße des Typs, auf den der Pointer zeigt, dekrementiert. Der Dekrement-Operator kann nur auf modifizierbare L-Werte angewandt werden.

```
a = 1;  
b = a--;          // Ergebnis: b = 1, Nebeneffekt: a = 0  
ptr--;
```

9.3.6 Präfix-Dekrement-Operator: --x

Der Rückgabewert ist der um 1 dekrementierte Wert des Operanden. Als Nebeneffekt wird der Wert des Operanden um 1 dekrementiert. Bei Pointern wird um eine Objektgröße vom Typ, auf den der Pointer zeigt, dekrementiert. Der Dekrement-Operator kann nur auf modifizierbare L-Werte angewandt werden.

```
a = 1;  
b = --a;           // Ergebnis: b = 0, Nebeneffekt: a = 0  
--ptr;
```

9.4 Zweistellige arithmetische Operatoren

Im Folgenden werden die zweistelligen (binären) Operatoren mit ihren Operanden anhand von Beispielen vorgestellt. Zweistellige arithmetische Operatoren haben keine Nebeneffekte.

Es gelten die „üblichen“ Rechenregeln, also Klammerung vor Punkt und Punkt vor Strich. Der Multiplikations-, Divisions- und Modulo-Operator haben eine höhere Priorität wie der Additions- und Subtraktions-Operator.

Additions-Operator	$x + y$
Subtraktions-Operator	$x - y$
Multiplikations-Operator	$x * y$
Divisions-Operator	x / y
Restwert-Operator	$x \% y$

9.4.1 Additions-Operator: $x + y$

Wendet man den zweistelligen Additions-Operator auf seine Operanden an, so ist der Rückgabewert die Summe der Werte der beiden Operanden.

```
6 + (4 + 3)
a + 1.2e6
PI + 1
```

9.4.2 Subtraktions-Operator: $x - y$

Wendet man den zweistelligen Subtraktions-Operator auf die Operanden x und y an, so ist der Rückgabewert die Differenz der Werte der beiden Operanden.

```
6 - 4
7 - 2.3e4
PI - KONSTANTE_A
```

9.4.3 Multiplikations-Operator: $x * y$

Es wird die Multiplikation des Wertes von x mit dem Wert von y durchgeführt.

```
3 * 5 + 3      // Ergebnis: 18
3 * (5 + 3)    // Ergebnis: 24
```

9.4.4 Divisions-Operator: x / y

Der Divisions-Operator dividiert zwei Zahlen. Für Gleitpunkt-Zahlen entspricht dies der normalen mathematischen Division. Das Resultat einer solchen Operation ist wiederum eine Gleitpunkt-Zahl.

Bei der Verwendung des Divisions-Operator mit Integer-Operanden ist das Ergebnis wieder ein Integer. Der Nachkommateil des Ergebnisses wird abgeschnitten.

Eine Division durch 0 ist bei `int`-Zahlen nicht erlaubt.

Ist mindestens ein Operand eine double- oder float-Zahl, so ist das Ergebnis eine Gleitpunkt-Zahl.

```
5 / 5          // Ergebnis: 1
5 / 3          // Ergebnis: 1
5 / 0          // dieser Ausdruck ist nicht zulaessig
52.0 / 5.8     // Ergebnis: 8.9655...
11.0 / 5       // Ergebnis: 2.2
```

Beim Ausdruck `5 / 0` kann schon der Compiler den Fehler erkennen und das Programm zurückweisen. Lautet der Ausdruck jedoch etwa `5 / x`, wobei `x` während des Programmlaufs erst berechnet wird, ist für den Compiler kein Fehler ersichtlich. Hat jedoch dann `x` den Wert `0`, wird beim Versuch, diesen Ausdruck zu berechnen, das Programm mit einer Fehlermeldung abgebrochen.

9.4.5 Restwert-Operator: `x % y`

Der Restwert-Operator (auch als Modulo-Operator bezeichnet) gibt den Rest bei der Integer-Division des Operanden `x` durch den Operanden `y` an. Eine Division durch `0` ist nicht erlaubt. Gleitpunkt-Zahlen sind ebenfalls nicht erlaubt.

```
5 % 3          // Ergebnis: 2
10 % 5         // Ergebnis: 0
3 % 7          // Ergebnis: 3
2 % 0          // Dieser Ausdruck ist nicht zulaessig
3. % 2.        // Dieser Ausdruck ist nicht zulaessig
```


9.5 Zuweisungs-Operatoren

Zu den Zuweisungs-Operatoren gehören der einfach Zuweisungs-Operator (auf Englisch „simple assignment operator“) sowie die kombinierten Zuweisungs-Operatoren (auf Englisch „compound assignment operator“).

Zuweisungs-Operator	x = y
Additions-Zuweisungs-Operator	x += y
Subtraktions-Zuweisungs-Operator	x -= y
Multiplikations-Zuweisungs-Operator	x *= y
Divisions-Zuweisungs-Operator	x /= y
Restwert-Zuweisungs-Operator	x %= y
Bitweises-UND-Zuweisungs-Operator	x &= y
Bitweises-ODER-Zuweisungs-Operator	x = y
Bitweises-Exklusiv-ODER-Zuweisungs-Operator	x ^= y
Linksschiebe-Zuweisungs-Operator	x <<= y
Rechtsschiebe-Zuweisungs-Operator	x >>= y

Dabei darf zwischen den Zeichen eines kombinierten Zuweisungs-Operators kein Leerzeichen stehen.

9.5.1 Einfacher Zuweisungs-Operator x = y

Der Zuweisungs-Operator wird in C als binärer Operator betrachtet. Er speichert den Rückgabewert des rechten Operanden in dem linken Operanden. Diese Zuweisung wird als „Nebeneffekt“ bezeichnet, sprich es wird eine Speicherstelle verändert.

```
b = 1 + 3;
```

Es handelt sich bei einer Zuweisung um einen Ausdruck.



In C können Zuweisungen wiederum in größeren Ausdrücken weiterverwendet werden. Der Ausdruck liefert als Rückgabewert den Wert des rechten Operanden. Folgendes Beispiel ist also möglich:

```
a = b = c;
```

Dabei wird der Operator von rechts nach links abgearbeitet, sprich zuerst wird $b = c$ ausgeführt und ausgewertet und danach wird dieses Resultat a zugewiesen.

Zuweisungs-Operatoren haben eine geringe Priorität, so dass man beispielsweise bei einer Zuweisung $b = x + 3$ den Ausdruck $x + 3$ nicht in Klammern setzen muss. Erst erfolgt die Auswertung des arithmetischen Ausdrucks, dann erfolgt die Zuweisung.

9.5.2 Kombinierte Zuweisungs-Operatoren

Die kombinierten Zuweisungs-Operatoren sind – wie der Name schon verrät – aus mehreren Symbolen zusammengesetzte Operatoren. Sinngemäß führen sie auch mehrere Operationen auf einmal aus.

Beispielsweise wird beim Ausdruck $x += y$ zum einen die Addition $x + (y)$ durchgeführt. Der Rückgabewert dieser Addition ist $x + (y)$. Zum anderen erhält die Variable x als Nebeneffekt den Wert dieser Addition zugewiesen. Damit entspricht der Ausdruck $x += y$ semantisch genau dem Ausdruck $x = x + (y)$.

Die Klammer in dieser Erklärung sind nötig, da y selbst ein Ausdruck wie beispielsweise $b * 3$ sein kann. Es wird also zuerst der Ausdruck y ausgewertet, bevor $x + (y)$ berechnet wird.

Für die anderen kombinierten Zuweisungs-Operatoren gilt das Gleiche wie bei dem Additions-Zuweisungs-Operator. Hier sind alle kombinierten Zuweisungs-Operatoren aufgeführt mit der jeweiligen ausführlichen Schreibweise:

```
a -= 1           // a = a - 1
b *= 2           // b = b * 2
c /= 5           // c = c / 5
d %= 5           // d = d % 5
e &= 8           // e = e & 8
f |= 4           // f = f | 4
g ^= 3           // g = g ^ 3
h <<= 1          // h = h << 1
i >>= 1          // i = i >> 1
```

9.6 Relationale Operatoren

Im Folgenden werden die zweistelligen relationalen Operatoren vorgestellt:

Gleichheits-Operator	<code>x == y</code>
Ungleichheits-Operator	<code>x != y</code>
Größer-Operator	<code>x > y</code>
Kleiner-Operator	<code>x < y</code>
Größergleich-Operator	<code>x >= y</code>
Kleinergleich-Operator	<code>x <= y</code>

Relationale Operatoren werden auch als Vergleichs-Operatoren bezeichnet. Die Priorität der Operatoren `==` und `!=` (manchmal auch Äquivalenz-Operatoren genannt) ist kleiner als die der Operatoren `>`, `>=`, `<` und `<=`. Besitzen die Operanden unterschiedliche Datentypen, werden implizite Typumwandlungen durchgeführt, siehe Kapitel [→ 9.10](#). Nebeneffekte treten bei Vergleichsoperationen nicht auf.

9.6.1 Boolesche Werte

C kannte bis zum Standard C23 in seinem Sprachkern keine eigenen Datentypen für **boolesche Werte (Wahrheitswerte)**. Statt der booleschen Werte „wahr“ und „falsch“ oder `true` und `false` wurden stets einfach Zahlen verwendet.

Die `0` gilt als „falsch“, jede Zahl ungleich `0` wie beispielsweise `1`, `3`, `-17` oder `0.1` gilt als „wahr“.



Ab dem Standard C23 sind der Datentyp `bool`, sowie die beiden Werte `true` und `false` im Sprachkern eingebaut. Die alten Werte `0` und nicht-`0` sind jedoch nach wie vor genauso gültig.

Der Rückgabewert von Vergleichsoperationen ist immer vom Datentyp `int`. Wenn ein Vergleich falsch ist, ist der Rückgabewert `0`, wenn er wahr ist, ist der Rückgabewert `1`. Vergleichsoperationen für Pointer liefern dieselben Rückgabewerte. In Kapitel [→ 12.2.4](#) wird behandelt, welche Vergleiche für Pointer definiert sind.

9.6.2 Gleichheits-Operator: $x == y$

Mit dem Gleichheits-Operator wird überprüft, ob der Wert des linken Operanden mit dem Wert des rechten Operanden übereinstimmt. Ist das der Fall, hat der Rückgabewert den Wert 1. Andernfalls hat der Rückgabewert den Wert 0:

```
1 + 2 == 3          // Ergebnis: 1 (wahr)
2 - 2 == 1          // Ergebnis: 0 (falsch)
```

Ein folgenschwerer Fehler ist in C, statt des Gleichheits-Operators `==` versehentlich den Zuweisungs-Operator `=` zu schreiben. Ein solches Programm ist oft compiler- und lauffähig, erzeugt aber andere Ergebnisse als erwartet.



9.6.3 Ungleichheits-Operator: $x != y$

Mit dem Ungleichheits-Operator wird überprüft, ob der Wert des linken Operanden verschieden vom Wert des rechten Operanden ist. Bei Ungleichheit hat der Rückgabewert den Wert 1. Andernfalls hat der Rückgabewert den Wert 0.

```
5 != 5              // Ergebnis: 0 (falsch)
3 != 5              // Ergebnis: 1 (wahr)
```

9.6.4 Größer-Operator: $x > y$

Mit dem Größer-Operator wird überprüft, ob der Wert des linken Operanden größer als der Wert des rechten Operanden ist. Ist der Vergleich wahr, hat der Rückgabewert den Wert 1. Andernfalls hat der Rückgabewert den Wert 0.

```
5 > 3               // Ergebnis: 1 (wahr)
4 > 1 + 3           // Ergebnis: 0 (falsch)
```

Beim zweiten Beispiel ist zu beachten, dass der Größer-Operator weniger prioritär behandelt wird als der Additions-Operator, es wird also zuerst 1 und 3 zusammengezählt.

9.6.5 Kleiner-Operator: $x < y$

Mit dem Kleiner-Operator wird überprüft, ob der Wert des linken Operanden kleiner als der Wert des rechten Operanden ist. Ist der Vergleich wahr, hat der Rückgabewert den Wert 1. Andernfalls hat der Rückgabewert den Wert 0.

```
3 < 4           // Ergebnis: 1 (wahr)
1 == 1 < 3      // Ergebnis: 1 (wahr)
```

Beim zweiten Beispiel ist zu beachten, dass der Kleiner-Operator eine höhere Priorität hat als der Gleichheits-Operator, es wird also zuerst geprüft, ob 1 kleiner ist als 3.

9.6.6 Größergleich-Operator: $x >= y$

Der Größergleich-Operator liefert genau dann den Rückgabewert 1 (wahr), wenn entweder der Wert des linken Operanden größer als der Wert des rechten Operanden ist oder der Wert des linken Operanden dem Wert des rechten Operanden entspricht. Ansonsten ist der Rückgabewert 0 (falsch).

```
2 >= 1          // Ergebnis: 1 (wahr)
1 >= 1          // Ergebnis: 1 (wahr)
```

9.6.7 Kleingleich-Operator: $x <= y$

Der Kleingleich-Operator liefert genau dann den Rückgabewert 1 (wahr), wenn entweder der Wert des linken Operanden kleiner als der Wert des rechten Operanden ist oder der Wert des linken Operanden dem Wert des rechten Operanden entspricht. Ansonsten ist der Rückgabewert 0 (falsch).

```
10 <= 11        // Ergebnis: 1 (wahr)
11 <= 11        // Ergebnis: 1 (wahr)
```

9.7 Logische Operatoren

Im Folgenden werden die drei logischen Operatoren mit ihren Operanden anhand von Beispielen vorgestellt.

Operator für das logische UND	x && y
Operator für das logische ODER	x y
Logischer Negations-Operator	!x

Achten Sie bitte auf die Schreibweise für die logischen Operatoren && und ||. C kennt nämlich auch die Bit-Operatoren & und |. Daher meldet der Compiler keinen Fehler bei einem Ausdruck wie etwa `a & b`. Die Wirkung der Bit-Operatoren ist jedoch eine ganz andere als die der logischen Operatoren.



Die Operatoren für das logische UND/ODER sind zweistellig, wohingegen der logische Negations-Operator einstellig ist. Mit diesen Operatoren lassen sich logische Verknüpfungen von Ausdrücken durchführen. Von den logischen Operatoren hat der Negations-Operator die höchste Priorität, der ODER-Operator die geringste.

9.7.1 Operator für das logische UND: x && y

Der Operator für das logische UND liefert genau dann den Rückgabewert 1 (wahr), wenn der linke und der rechte Operand einen von 0 verschiedenen Wert – also beide den Wahrheitswert „wahr“ – haben. Ansonsten ist der Rückgabewert 0 (falsch). Die Operanden können verschiedene Typen besitzen, aber es muss ein skalarer Typ sein. Mit anderen Worten, die Operanden müssen einen arithmetischen Typ oder einen Pointer-Typ haben. Das folgende Bild zeigt die Wahrheitstabelle für das logische UND:

x	y	x && y
falsch	falsch	falsch
falsch	wahr	falsch
wahr	falsch	falsch
wahr	wahr	wahr

Die Wahrheitstabelle wird folgendermaßen interpretiert: Der logische Ausdruck `x && y` ist nur dann wahr, wenn der Ausdruck `x` und der Ausdruck `y` wahr sind.

Beispiele:

```
0 && 1           // Ergebnis: 0 (falsch)
5 && 6           // Ergebnis: 1 (wahr)
3 < 5 && 5 > 3   // Ergebnis: 1 (wahr)
```

9.7.2 Operator für das logische ODER: `x || y`

Der Operator für das logische ODER liefert genau dann den Rückgabewert 1 (wahr), wenn der linke oder der rechte Operand oder beide Operanden einen von 0 verschiedenen Wert, also den Wahrheitswert „wahr“, haben. Ansonsten ist der Rückgabewert 0 (falsch). Die Operanden können verschiedene Typen besitzen, aber es muss ein skalarer Typ sein. Mit anderen Worten, die Operanden müssen einen arithmetischen Typ oder einen Pointer-Typ haben. Das folgende Bild zeigt die Wahrheitstabelle für das logische ODER:

x	y	x y
falsch	falsch	falsch
falsch	wahr	wahr
wahr	falsch	wahr
wahr	wahr	wahr

Die Wahrheitstabelle wird folgendermaßen interpretiert: Der logische Ausdruck `x || y` ist dann wahr, wenn der Ausdruck `x` oder der Ausdruck `y` oder beide Ausdrücke wahr sind.

Beispiele:

```
0 || 1           // Ergebnis: 1 (wahr)
0 || (1 && 0)     // Ergebnis: 0 (falsch)
'b' == 'a' + 1 || 0 // Ergebnis: 1 (wahr)
```

Bei der Auswertung des letzten Beispiels ist die Priorität der Operatoren zu beachten. Es gilt die Reihenfolge: Additions-Operator, Vergleichs-Operator, dann der ODER-Operator.

9.7.3 Logischer Negations-Operator: !x

Mit dem einstelligen Negations-Operator werden Wahrheitswerte negiert, sprich aus „wahr“ wird „falsch“ (Rückgabewert 0) und aus „falsch“ wird „wahr“ (Rückgabewert 1). Wird der Negations-Operator zweimal auf seinen Operanden angewandt, bleibt der Wahrheitswert unverändert. Das folgende Bild zeigt die Wahrheitstabelle für die Negation:

x	!x
falsch	wahr
wahr	falsch

Die Wahrheitstabelle wird folgendermaßen interpretiert: Der logische Ausdruck !x ist nur dann wahr, wenn der Ausdruck x falsch ist.

Beispiele:

```
!0           // Ergebnis: 1 (wahr)
!!5          // Ergebnis: 1 (wahr)
!0 == 1      // Ergebnis: 1 (wahr)
```

9.7.4 Spezielles bei der Abarbeitung von logischen Operatoren

Die Operatoren für das logische UND/ODER haben eine sehr geringe Priorität. Die Vergleichs-Operatoren haben eine höhere Priorität als die logischen Operatoren. Deshalb sind Klammern für die Auswertung von Ausdrücken bei Kontrollstrukturen (siehe Kapitel [→ 10](#)) oft nicht notwendig. So sind die beiden folgenden Ausdrücke äquivalent. Die Klammern erhöhen lediglich die Übersichtlichkeit der Programme.

```
(a < b) && (c == d)
a < b  &&  c == d
```


Gemäß den Regeln der binären Logik hat der Operator `&&` eine höhere Priorität als der `||`-Operator. Dennoch werden zur besseren Lesbarkeit logische Ausdrücke oftmals zusätzlich geklammert. Manche Compiler geben sogar Warnungen aus, wenn ein logischer Ausdruck die beiden Operatoren mischt. Die folgenden beiden Ausdrücke sind äquivalent:

```
(a && b) || (c && d)
a && b || c && d
```

Logische Operatoren definieren Sequenzpunkte (siehe Kapitel [→ 9.1.2](#)). Damit ist eindeutig festgelegt, dass bei einem Ausdruck wie `x && y` zuerst der Ausdruck `x` ausgewertet werden muss.

Nebeneffekte des linken Operanden werden ausgeführt, bevor die weiteren Operanden ausgewertet werden. Die Auswertung wird abgebrochen, wenn das Ergebnis des Ausdrucks schon feststeht.



Damit ist gemeint:

- Ist bei einer logischen ODER-Verknüpfung bereits der erste Operand „wahr“, so ist der gesamte Ausdruck automatisch „wahr“ und der zweite Operand wird nicht ausgewertet.
- Ist bei einer logischen UND-Verknüpfung bereits der erste Operand „falsch“, so ist der gesamte Ausdruck automatisch „falsch“ und der zweite Operand wird nicht ausgewertet.

Das kann dazu führen, dass Nebeneffekte der weiter rechts stehenden Ausdrücke nicht mehr ausgeführt werden.



Hierfür ein Beispiel:

```
1 < 0 && 2 < a++    // a++ wird nie ausgeführt.
```

9.8 Bit-Operatoren

Bit-Operatoren erlauben es der Sprache C, Bit-Manipulationen auszuführen. Mit solchen Operatoren wird eine hardwarenahe Programmierung unterstützt.

Bits können bekanntermaßen zwei Zustände annehmen: 0 oder 1. Jeder Wert ist in einem Computer aus mehreren Bits zusammengesetzt.

Bit-Operationen finden auf allen Bits der Operanden statt. Bei den Bit-Operationen werden jeweils die Bits der entsprechenden Position miteinander verknüpft.



Die logischen Bit-Operatoren dürfen nur für Integer-Datentypen benutzt werden. Vorsicht ist geboten bei der Verwendung von vorzeichenbehafteten Datentypen, da hier implementierungsabhängige Aspekte auftreten. Bei vorzeichenlosen Datentypen ist die Verwendung der Bit-Operatoren problemlos.

Nebeneffekte treten bei den logischen Bit-Operatoren nicht auf. Im Folgenden werden die vier logischen Bit-Operatoren mit ihren Operanden sowie die beiden Shift-Operatoren für Bits vorgestellt:

UND-Operator für Bits	<code>x & y</code>
ODER-Operator für Bits	<code>x y</code>
Exklusiv-ODER-Operator für Bits	<code>x ^ y</code>
Negations-Operator für Bits	<code>~x</code>
Rechtsshift-Operator	<code>x >> y</code>
Linksshift-Operator	<code>x << y</code>

Achten Sie bitte auf die Schreibweise für die Bit-Operatoren `&` und `|`. C kennt nämlich auch die logischen Operatoren `&&` und `||`. Daher meldet der Compiler keinen Fehler bei einem Ausdruck wie etwa `a && b`. Die Wirkung der logischen Operatoren ist jedoch eine ganz andere als die der Bit-Operatoren.



9.8.1 UND-Operator für Bits: x & y

Die Operation bitweises UND findet auf allen Bits der Operanden statt. Dabei werden jeweils die Bits der entsprechenden Position miteinander verknüpft. Im Folgenden die Wahrheitstabelle für das bitweise UND:

Bit n von x	Bit n von y	Bit n von x & y
0	0	0
0	1	0
1	0	0
1	1	1

Die Wahrheitstabelle wird folgendermaßen interpretiert: Bei der Verknüpfung mit UND ist die 0 dominant, das heißt ist mindestens eines der Bits (Bit n von x oder Bit n von y) eine 0, so ist das Ergebnis 0 (falsch).

Mit dem UND-Operator für Bits kann man Bits in Bitmustern ausblenden. Damit ist gemeint, dass diejenigen Bits, welche mit 0 verknüpft werden, garantiert im Resultat auch 0 ergeben.



```
0 & 1           //      0 &    1 =    0
14 & 1          //  1110 & 0001 = 0000
14 & 7          //  1110 & 0111 = 0110
```

Der logische UND-Operator für Bits hat eine höhere Priorität als der logische ODER-Operator für Bits.

9.8.2 ODER-Operator für Bits: $x \mid y$

Die Operation bitweises ODER findet auf allen Bits der Operanden statt. Dabei werden jeweils die Bits der entsprechenden Position miteinander verknüpft. Hier die Wahrheitstabelle für das bitweise ODER:

Bit n von x	Bit n von y	Bit n von $x \mid y$
0	0	0
0	1	1
1	0	1
1	1	1

Die Wahrheitstabelle wird folgendermaßen interpretiert: Bei der Verknüpfung mit ODER ist die 1 dominant, das heißt ist mindestens eines der Bits (Bit n von x oder Bit n von y) eine 1, so ist das Ergebnis 1 (wahr).

Mit dem ODER-Operator für Bits kann man Bits in Bitmustern einblenden. Damit ist gemeint, dass diejenigen Bits, welche mit 1 verknüpft werden, garantiert im Resultat auch 1 ergeben.



0 1	//	0 1 = 1
8 1	//	1000 0001 = 1001
14 1	//	1110 0001 = 1111

9.8.3 Exklusiv-ODER-Operator für Bits: $x \wedge y$

Die Operation bitweises Exklusiv-ODER findet auf allen Bits der Operanden statt. Dabei werden jeweils die Bits der entsprechenden Position miteinander verknüpft. Es folgt die Wahrheitstabelle für das bitweise Exklusives-ODER:

Bit n von x	Bit n von y	Bit n von $x \wedge y$
0	0	0
0	1	1
1	0	1
1	1	0

Die Wahrheitstabelle wird folgendermaßen interpretiert: Bei der Verknüpfung mit Exklusiv-ODER ist das Ergebnis 1 (wahr), wenn entweder Bit n von Operand x oder Bit n von Operand y eine 1 ist, aber nicht beide gleichzeitig.

Mit dem Exklusiv-ODER-Operator für Bits kann man Bits invertieren.



```
0 ^ 1          // 0 ^ 1 = 1
14 ^ 1         // 1110 ^ 0001 = 1111
14 ^ 3         // 1110 ^ 0011 = 1101
```

9.8.4 Negations-Operator für Bits: ~x

Die Operation bitweise Negation findet auf allen Bits des Operanden statt. Hier die Wahrheitstabelle für die bitweise Negation:

Bit n von x	Bit n von ~x
0	1
1	0

Die Wahrheitstabelle wird folgendermaßen interpretiert: Bei der Negation für Bits wird einfach jedes Bit invertiert (Einer-Komplement). Aus der 0 wird durch Negation eine 1 und aus der 1 eine 0.

Der Negations-Operator für Bits invertiert jedes Bit.



```
unsigned char a, b;
a = 9;          // a = 00001001
b = ~a;         // b = 11110110
```

9.8.5 Rechtsshift-Operator: $x \gg y$

Mit dem Rechtsshift-Operator $\gg$ werden die Bits von x um y Bitstellen nach rechts geschoben. Es darf nur um ganzzahlige positive Stellen von Bits verschoben werden.

Dabei gehen die y niederwertigen Bits von x verloren. Wenn der Operand x von einem unsigned-Typ ist, werden die höherwertigen Bits mit Nullen aufgefüllt, was einer Integer-Division durch 2^y entspricht. Bei einem Operanden x eines signed-Typs mit einem negativen Wert ist das Ergebnis vom Compiler abhängig. Bei vielen Compilern bleibt das Vorzeichenbit erhalten.

```
unsigned char a;
a = 8;          /* 0000 1000 Bitmuster von 8 */
a = a >> 3;     /* 0000 0001 Bitmuster von 1 */
                aufgefüllt      verloren
```

Die Verschiebung um 3 Bits nach rechts entspricht einer Division durch 2^3 .

9.8.6 Linksshift-Operator: $x \ll y$

Bei dem Linksshift-Operator $\ll$ werden die Bits von x um y Bitstellen nach links geschoben. Es darf nur um ganzzahlige positive Stellen von Bits verschoben werden.

Dabei gehen die y höherwertigen Bits von x verloren. Die nachrückenden niederwertigen Bits werden mit Nullen aufgefüllt. Falls kein Überlauf eintritt, entspricht dies bei einem unsigned-Operanden einer Multiplikation mit 2^y . Im Falle eines Überlaufs ist das Resultat undefiniert.

```
unsigned char a = 128; /* 1000 0000 Bitmuster von 128 */
a = a << 1;           /* Overflow. Ergebnis: */
/* 0000 0000 */
a = 8;               /* 0000 1000 Bitmuster von 8 */
a = a << 3;          /* 0100 0000 Bitmuster von 64 */
                    verloren      aufgefüllt
```

Die Verschiebung um 3 Bits nach links entspricht einer Multiplikation mit 2^3 .

9.9 Sonstige Operatoren

Im diesem Kapitel werden die folgenden Operatoren vorgestellt:

sizeof-Operator	sizeof
_Alignof-Operator	_Alignof
Komma-Operator	,
Bedingungs-Operator	?:
Typumwandlungs-Operator	(Type)

9.9.1 Der sizeof-Operator

Der Operator **sizeof** dient zur Ermittlung der Größe in Bytes von Typen sowie von Datenobjekten im Hauptspeicher.



Der Operator `sizeof` darf dabei sowohl auf einen Ausdruck, also einen L-Wert wie einen Variablennamen oder einen R-Wert, als auch auf einen Typ-Bezeichner angewandt werden.

Die Syntax ist:

```
sizeof Ausdruck  
sizeof(Typname)
```

Der Rückgabewert ist dabei jeweils ein Integer, nämlich die Größe des angegebenen Operanden gemessen in Bytes. Der Typ des Rückgabewertes ist `size_t`.

Die Anzahl der Bytes wird angegeben in dem Typ `size_t`, der extra für den `sizeof`-Operator geschaffen wurde. Der Wertebereich von `size_t` ist ausreichend, um eine beliebige Speichergröße aufzunehmen. In der Regel entspricht `size_t` einem `unsigned int` mit 32 oder 64 Bit.

Im folgenden Beispielprogramm, welches auf einem spezifischen Computer läuft, werden einige Anwendungsfälle, auch für Arrays vorgestellt:

sizeof.c

```
#include <stdio.h>

int main(void) {
    int zahl1 = 1;
    int array[20] = {0};
    double zahl2 = 1.;

    printf("size of integer:   %2d Bytes\n", (int)sizeof zahl1);
    printf("size of float:     %2d Bytes\n", (int)sizeof(float));
    printf("size of double:      %2d Bytes\n", (int)sizeof zahl2);
    printf("size of array:        %2d Bytes\n", (int)sizeof array);
    printf("size of array[10]: %2d Bytes\n", (int)sizeof array[10]);
    return 0;
}
```

Hier das Protokoll des Programmlaufs:

```
size of integer:      4 Bytes
size of float:        4 Bytes
size of double:       8 Bytes
size of array:        80 Bytes
size of array[10]:    4 Bytes
```

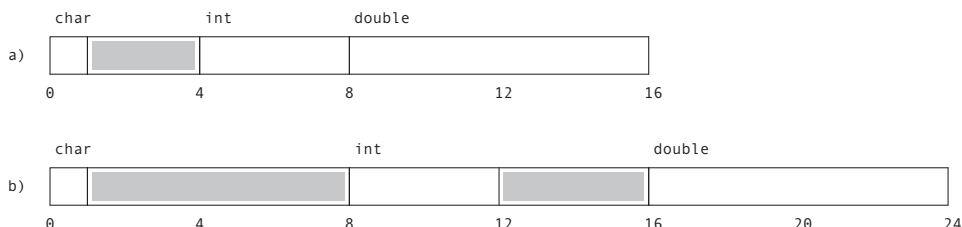
Wenn der sizeof-Operator auf einen Typ angewandt wird, müssen Klammern geschrieben werden. Bei Ausdrücken können die Klammern weggelassen werden. Wenn man sich nicht merken kann (oder will), ob man bei dem Operator sizeof Klammern braucht oder nicht, sollte man immer Klammern setzen. Denn ein geklammerter Ausdruck bleibt syntaktisch ein Ausdruck und wird somit beim sizeof-Operator akzeptiert.

Besonders oft wird der sizeof-Operator zur Berechnung der Größe von Strukturen verwendet. Strukturen sind zusammengesetzte Datentypen, deren Komponenten verschiedene Datentypen haben können. Da es dem Compiler freisteht, Komponenten der Struktur auf bestimmten Wortgrenzen beginnen zu lassen, kann die Struktur ungenutzten, namenlosen Platz enthalten. Damit lässt sich die Größe einer Struktur nicht durch Addition der Größen der Komponenten ermitteln.

Ein Compiler legt Objekte eines bestimmten Typs bei vielen Architekturen auf Speicheradressen, die ein gegebenes Vielfaches einer Byte-Adresse sind. Eine solche Anordnung von Objekten im Speicher wird als **Alignment** bezeichnet.



Das folgende Bild zeigt ein Speicherabbild einer Struktur für zwei verschiedene Compiler. Unbenutzte Bereiche sind grau hinterlegt:



Der Compiler legt im Beispiel a) int-Objekte auf 4-Bytegrenzen und double-Objekte auf 8-Bytegrenzen. Der Compiler im Beispiel b) legt sowohl int- als auch double-Variablen auf 8-Bytegrenzen. char-Variablen werden in beiden Fällen auf Byte-Adressen gelegt.

Der sizeof-Operator wird vor allem angewandt, um Programme portabler zu machen.



So ist es beispielsweise oft nötig, unter Verwendung der Bibliotheksfunktion `memcpy()` Objekte direkt im Speicher zu kopieren oder zu verschieben (genauer dazu kann in Kapitel [→ 12.8.1](#) nachgelesen werden). Dazu benötigt diese Funktion unter anderem die Größe des zu bearbeitenden Objektes. Statt diesen Wert nun als Konstante im Programm einzuführen, ist es bezüglich der Portierbarkeit auf andere Maschinen besser, dem Compiler die Ermittlung der Größe des Objektes zu überlassen. Sollte das Objekt auf einer anderen Maschine aufgrund der internen Darstellung eine andere Größe haben, so wird dieses Problem über den `sizeof`-Operator vom Compiler erledigt. Ein manueller Eingriff in das Programm ist also nicht erforderlich.

Auf einem Computer mit 32-Bit Integer-Zahlen ergeben sich folgende Beispiele:

```
int zahl = 1;
sizeof 534;           // Ergebnis: 4 (Bytes)
sizeof(int);          // Ergebnis: 4 (Bytes)
sizeof zahl++;        // Ergebnis: 4 (Bytes)
```

Der sizeof-Operator wertet einen Ausdruck, der ihm übergeben wurde, nicht aus. Er bestimmt lediglich den Typ des Ausdrucks und dann die Größe dieses Typs.



Der Wert von `zahl` wird also durch `sizeof zahl++` also nicht verändert, da der Ausdruck `zahl++` gar nicht ausgewertet wird, sondern nur sein Typ bestimmt wird.

Die Anzahl der Elemente eines Arrays lässt sich mit `sizeof` wie folgt portabel bestimmen:

```
int a[] = {4, 7, 11, 0, 8, 15};
size_t len = sizeof(a) / sizeof(a[0]);
```

Dieses Vorgehen funktioniert aber nur bei Arrays, deren Größe bei der Compilation des Programms bereits bekannt ist. Für ein Array `a`, das beispielsweise als Pointer einer Funktion übergeben wurde (siehe [→ 12.4](#)), ist das Vorgehen nicht erfolgreich.

9.9.2 Der `_Alignas`-, `_Alignof`- und `offsetof`-Operator



Der Compiler ordnet Daten im Speicher so an, dass sie vom Prozessor möglichst effizient gelesen und geschrieben werden können. Je nach Compiler und Prozessor wird die **Byte-Ausrichtung** (auf Englisch „**Alignment**“) der verschiedenen Datentypen im Speicher unterschiedlich ausfallen. Ein Objekt eines Typs, welcher 4 Bytes benötigt, wird beispielsweise von vielen Compilern an eine Adresse gesetzt, welche durch 4 teilbar ist. Dies wird als „Alignment an eine 4-Byte-Grenze“ bezeichnet.

Mittels des Operators `_Alignas` kann die Anordnung von Variablen explizit gesteuert werden. Ob und wie ein Compiler diese Angaben umsetzen kann, ist je nach Implementation unterschiedlich.



Mittels `_Alignof` kann die tatsächlich verwendete Anordnung eines Typs abgefragt werden.



```
int a;
_Alignas(16) int b;
_Alignas(double) int c;
printf("%d\n", (int)_Alignof(a)); // Ergebnis: 4
printf("%d\n", (int)_Alignof(b)); // Ergebnis: 16
printf("%d\n", (int)_Alignof(c)); // Ergebnis: 8
```

Der `_Alignas`-Operator erwartet eine Bytegröße oder einen bekannten Typ als Eingabe. Er kann gemäß Standard im Gegensatz zum `sizeof`-Operator nur auf Typen angewendet werden, nicht aber auf Ausdrücke. Manche Compiler erlauben dies dennoch.

Der `_Alignof`-Operator gibt genauso wie der `sizeof`-Operator einen (zur Compilierzeit bekannten) konstanten Wert des Typs `size_t` zurück. Dieser Wert entspricht dem Alignment des Typs, sprich der Bytegrenze, an welcher ein Typ ausgerichtet ist. Beispielsweise:

```
_Alignof(float); // Ergebnis 4 (Bytes)
_Alignof(double); // Ergebnis 8 (Bytes)
```

Im Folgenden ein Beispiel für Arrays:

```
sizeof (int[10]);      // Ergebnis 40 (Bytes)
_Alignof (int[10]);    // Ergebnis 4 (Bytes)
```

In diesem Beispiel werden auf einem System 4 Bytes für den Datentyp `int` benötigt. Dementsprechend werden 40 Bytes Speicher für ein Array mit 10 Komponenten vom Typ `int` benötigt. Die einzelnen Elemente und somit das gesamte Array jedoch werden vom Compiler auf eine (beliebige) 4-Byte-Grenze ausgerichtet.

Interessant wird der Rückgabewert bei Strukturen. Der Compiler hat die Aufgabe, alle Elemente einer Struktur so anzuordnen, dass jedes einzelne Element an seiner vom Compiler bestimmten Ausrichtungsgrenze zu liegen kommt. Mit Hilfe des `_Alignof`-Operators kann man herausfinden, wo der gesamte Speicherblock zu liegen kommt:

```
struct structtype{float x; double y;};
printf("%d\n", (int)sizeof(struct structtype)); // Ergebnis: 16
printf("%d\n", (int)_Alignof(struct structtype)); // Ergebnis: 8
```

Und mithilfe des `offsetof`-Operators erhält man die Startadresse innerhalb der Struktur für ein spezifisches Element:

```
printf("%d\n", (int)offsetof(struct structtype, x)); // Ergebnis: 0
printf("%d\n", (int)offsetof(struct structtype, y)); // Ergebnis: 8
```

Der `offsetof`-Operator existiert seit Beginn der Standardisierung von C in der `<stddef.h>`-Bibliothek. Die Operatoren `_Alignof()` und `_Alignas()` existieren jedoch erst seit dem C11-Standard. Der C11-Standard beschreibt zudem eine neue Bibliothek `<stdalign.h>`, welche die Makros `alignof` und `alignas` definiert, die zu `_Alignof` und `_Alignas` ausgewertet werden. Ab dem Standard C23 werden die neuen Schlüsselwörter `alignof` und `alignas` eingeführt.

Ab dem Standard C23 wird zudem die Funktion `memalignment()` eingeführt. Mit dieser Funktion kann die Byte-Ausrichtung eines beliebigen Pointers bestimmt werden, sprich an welcher Byte-Grenze Speicher reserviert werden soll. In diesem Buch wird auf diese Funktionalität nicht eingegangen.

9.9.3 Der Komma-Operator: x, y

Der Komma-Operator wird in der Praxis selten eingesetzt.

Ein Ausdruck x, y wird von links nach rechts abgearbeitet. Erst wird der Ausdruck x ausgewertet, dann der Ausdruck y . Nebeneffekte des linken Ausdrucks sind nach dessen Auswertung eingetreten.



Der Rückgabewert und -typ ist der Wert und der Typ von y , das heißt des rechten Operanden. Ein Komma-Operator liefert keinen L-Wert.

Die allermeisten Vorkommen des Komma-Zeichens im Quellcode sind jedoch nicht dem Komma-Operator zuzuschreiben, sondern beispielsweise Parameterlisten, Initialisierungen oder enum-Deklarationen.



Der Komma-Operator wird nur sehr selten verwendet, da er grundsätzlich nichts anderes macht, als zwei Ausdrücke sequenziell hintereinander auszuwerten. Im Normalfall wird man dies mittels zwei aufeinanderfolgenden Anweisungen bewerkstelligen.

Im Vergleich zu zwei aufeinanderfolgenden Anweisungen hat der Komma-Operator jedoch zwei Vorteile: Erstens gibt der gesamte Ausdruck einen Wert zurück und zweitens werden die verknüpften Ausdrücke als ein einziger Ausdruck angesehen.



Wozu kann der Komma-Operator gut sein? Beispielsweise kann man damit in einer for-Anweisung zwei Indizes gleichzeitig bearbeiten, um einen String mit einem herabzuzählenden Index und einem hochzählenden Index umzudrehen:

revers.c

```
#include <stdio.h>

int main(void) {
    char text1[] = "Hallo Welt";
    char text2[sizeof text1 / sizeof(char)];

    int i1, i2;
    for (i2 = 0, i1 = sizeof text1 - 2; i1 >= 0; ++i2, --i1) {
        text2[i2] = text1[i1];
    }
    text2[i2] = '\0';
    printf("%s\n%s\n", text1, text2);
    return 0;
}
```

Das Programm gibt aus:

```
Hallo Welt
tleW ollaH
```

9.9.4 Der Bedingungs-Operator: $x ? y : z$

Eine echte „Rarität“ in der Programmiersprache C ist der Bedingungs-Operator. Er ist nämlich der einzige Operator, der drei Operanden verarbeitet.

In einem bedingten Ausdruck $x ? y : z$ wird zuerst der Ausdruck x ausgewertet. Ist der Rückgabewert von Ausdruck x ungleich 0, also wahr, dann ist der Rückgabewert des gesamten bedingten Ausdrucks das Ergebnis von y . Ist jedoch der Ausdruck x gleich 0, also falsch, so wird der Ausdruck z ausgewertet und dessen Ergebnis ist der Rückgabewert des bedingten Ausdrucks.



```
5 < 10 ? 123 : 52          // Rueckgabewert: 123
0 ? 0 : 1                  // Rueckgabewert: 1
```

Ist die Bedingung x wahr, dann wird der Ausdruck z nicht ausgewertet und ist die Bedingung x falsch, dann wird der Ausdruck y nicht ausgewertet!



Der Typ des bedingten Ausdrucks $x ? y : z$ ist – unabhängig davon, ob der Rückgabewert dieses Ausdrucks y oder z ist – stets der mächtigere Typ (siehe Kapitel [→ 9.10](#)) der beiden Ausdrücke y und z . So ist beispielsweise der Rückgabotyp des folgenden Ausdrucks vom Typ `double` und der Rückgabewert ist `6.0`:

```
(3 > 4) ? 5.0 : 6
```

Zu beachten ist, dass beim Bedingungs-Operator zuerst die Bedingung ausgewertet wird. Nebeneffekte des linken Operanden werden damit ausgeführt, bevor die weiteren Operanden ausgewertet werden.



Der Bedingungs-Operator ist grundsätzlich eine vereinfachte Schreibweise einer Bedingung, welche gerne für Zuweisungen oder Funktionsrückgaben genutzt wird. Eine Funktion kann mit der `return`-Anweisung einen Wert an den Aufrufer zurückliefern, siehe dazu Kapitel [→ 11.4](#):

```
if (x)
    return y;
else
    return z;
```

Mit dem Bedingungs-Operator kann dies viel einfacher geschrieben werden:

```
return x ? y : z;
```

9.9.5 Der Typumwandlungs-Operator (Typname)

Eine **explizite Typumwandlung** eines beliebigen Ausdrucks kann man mit dem **cast-Operator** (Typumwandlungs-Operator) durchführen.



Das englische Wort „cast“ bedeutet hier „in eine Form gießen“.

Mit diesem Operator können impliziten Typumwandlungen (siehe dazu Kapitel [→ 9.10](#)), die der Compiler automatisch durchführen würde, vermieden werden. Da implizite Typumwandlungen komplex und fehlerträchtig sind, sollte man die notwendigen Typumwandlungen möglichst immer selbst durch Einsatz des cast-Operators festlegen.

Die Syntax des Typumwandlungs-Operators ist die folgende:

```
(Typname)Ausdruck
```

Damit wird der Wert des Ausdrucks in den Typ gewandelt, der in der Klammer angegeben ist. So erwartet beispielsweise die Bibliotheksfunktion `cos()`, die in der Bibliothek `<math.h>` deklariert ist, einen Ausdruck vom Typ `double`. Ist `n` ein Integer, dann kann mit `cos((double)n)` der Wert von `n` in `double` umgewandelt werden, bevor er als Parameter an `cos()` übergeben wird.

Es kann nicht jeder Typ in einen beliebigen anderen Typ gewandelt werden. Möglich sind insbesondere folgende Wandlungen:

- Wandlungen zwischen Integer-Typen
- Wandlungen zwischen Gleitpunkt-Typen
- Wandlungen zwischen Integer- und Gleitpunkt-Typen
- Wandlungen zwischen Pointern auf Variablen
- Wandlungen zwischen Pointern und Integer-Typen
- Wandlungen zwischen Pointern und dem Typ `void*`
- Wandlungen in den Typ `void`

Wandlungen zwischen arithmetischen Typen sind grundsätzlich erlaubt, verursachen jedoch möglicherweise eine Verfälschung des gespeicherten Wertes, wenn der Wertumfang des Ziel-Typs nicht mächtig genug ist, den ursprünglichen Wert abzubilden. Siehe hierzu die Konvertierungsregeln in Kapitel [9.11](#).

Zwischen Pointern und Integer-Typen zu wandeln gilt heutzutage als verpönt, da dadurch gefährliche Adress-Berechnungen durchgeführt werden können, welche nicht als sicher gelten.

Eine Umwandlung in den Typ `void` kann beispielsweise verwendet werden, um explizit darzustellen, dass der Rückgabewert eines Ausdrucks nicht verwendet wird, wie beispielsweise:

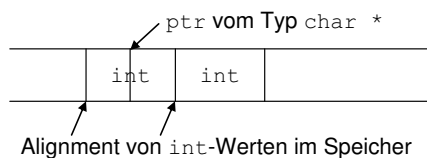
```
(void) printf("%d", x);
```

Bei der Umwandlung von Pointern sollte darauf geachtet werden, dass die zugrundeliegenden Typen zueinander kompatibel sind. Beispielsweise macht es wenig Sinn, einen `double*` in einen `char*` zu casten, verboten ist es jedoch nicht. Problematisch wird es jedoch spätestens, wenn das Alignment des zugrundeliegenden Typs nicht übereinstimmt.

Ein Pointer auf ein Objekt kann in einen Pointer auf ein anderes Objekt gewandelt werden. Der resultierende Pointer kann ungültig sein, wenn das Alignment für den Typ, auf den er zeigt, nicht stimmt.



Dies symbolisiert das folgende Beispiel:



Wird der Pointer `ptr` in diesem Beispiel in einen Pointer auf `int` gewandelt, so stimmt das Alignment nicht und der gewandelte Pointer zeigt auf einen unbrauchbaren Wert. Je nach Prozessor kann dies zu einem sofortigen Abbruch des Programmes führen.

9.10 Implizite Typumwandlung

In C ist es nicht notwendig, dass die Operanden eines Ausdrucks vom selben Typ sind. Genauso wenig muss bei einer Zuweisung der Typ der Operanden übereinstimmen. Auch bei der Übergabe von Werten an Funktionen und bei Rückgabewerten von Funktionen (siehe Kapitel [→ 11.3](#)) können übergebene Ausdrücke beziehungsweise der rückzugebende Ausdruck von den formalen Parametern beziehungsweise dem Rückgabebetyp verschieden sein. In solchen Fällen kann der Compiler selbsttätig **implizite Typumwandlungen** (also automatische Typumwandlungen) durchführen, die nach einem von der Sprache vorgeschriebenen Regelwerk ablaufen. Diese Regeln sollen in diesem Kapitel vorgestellt werden.

Implizite Typumwandlungen können gefährlich sein, da man sie oft nicht richtig einschätzt. Sorgen Sie am besten selbst dafür, dass die Typen jeweils übereinstimmen!



Man kann implizite Umwandlungen vermindern, indem bereits beim Schreiben der literalen Konstanten im Quellcode die Zahlen mit dem korrekten Suffix typisiert werden (siehe dazu Kapitel [→ 6.5](#)), oder aber man den expliziten Casting-Operator anwendet (siehe Kapitel [→ 9.9.5](#)). Wenn man selbst dafür sorgt, dass solche Typverschiedenheiten nicht vorkommen, braucht man sich um die implizite Typumwandlung nicht zu kümmern.

Implizite Typumwandlungen erfolgen in C prinzipiell nur zwischen verträglichen Datentypen. Zwischen unverträglichen Datentypen gibt es keine impliziten Umwandlungen. Hier wird der Compiler einen Fehler melden.

Implizite Typumwandlungen gibt es in folgenden Fällen:

- Bei einem Pointer auf void (siehe Kapitel [→ 8.2](#))
- Bei Operanden von arithmetischem Typ
- Bei Zuweisungen, Rückgabewerten und formalen Parametern von Funktionen (siehe Kapitel [→ 11.3](#))

Im Folgenden wird auf die Fälle der arithmetischen Typen eingegangen.

9.10.1 Arithmetische Typumwandlung

Wenn arithmetische Operanden als Teil einer Zuweisung auftreten, so wird stets der Operand auf der rechten Seite in den Typ auf der linken Seite umgewandelt.



Bei allen anderen binären Operatoren versucht der Compiler, eine möglichst optimale Umwandlung entweder des linken, des rechten oder von beiden Operanden vorzunehmen. Solche „gewöhnlichen“ **arithmetische Typumwandlungen** werden bei binären Operatoren mit Ausnahme der logischen Operatoren && und || durchgeführt. Sie werden auch beim ternären Bedingungs-Operator ?: durchgeführt. Das Ziel ist es, einen gemeinsamen Typ der Operanden des binären Operators zu erhalten, der dann auch der Typ des Ergebnisses ist.



Die allgemeinen Regeln, welche Typen in welche umgewandelt werden können, sind ganz genau spezifiziert im gültigen C-Standard. Dabei gibt es Unterschiede zwischen C90 und dem neueren Standard C11. Insbesondere werden in C11 Umwandlungen zwischen den Typmodifikatoren short, long und long long (siehe Anhang  E) für die jeweils vorzeichenbehaftete und vorzeichenlose Variante festgelegt.

Die genauen Regeln aufzulisten sprengt jedoch den Rahmen dieses Buches. Hier wird nur folgende, allgemeine Regel aufgeführt, welche für die alltägliche Programmierung mehr als ausreichend ist:

Die Sprache C weist jedem arithmetischen Typ eine sogenannte „Mächtigkeit“ beziehungsweise eine „Rangordnung“ zu. Ein Typ ist grundsätzlich mächtiger als ein anderer, wenn sein Wertebereich im positiven Bereich größer ist. Bei arithmetischen Operanden gilt generell, dass der „kleinere“ Datentyp in den „größeren“ Datentyp umgewandelt wird. Nur bei Zuweisungen kommen auch Wandlungen vom „größeren“ in den „kleineren“ Datentyp vor.



Wir betrachten als Beispiel den folgenden Code-Ausschnitt:

```
int fahr = 54;  
float celsius = (5.0 / 9) * (fahr - 32);
```

Die Temperatur in Fahrenheit ist hier in der Variablen `fahr` gespeichert, welche vom Typ `int` ist. Der Ausdruck `fahr - 32` in sich selbst benötigt keine implizite Konversion, da beide Operanden vom Typ `int` sind.

Der Ausdruck `(5.0 / 9)` beinhaltet die literale Konstante `5.0`, welche vom Typ `double` ist. Dadurch wird die `9` implizit ebenfalls in einen `double` umgewandelt.

Aufgrund dessen muss nun jedoch das Ergebnis des Ausdrucks `(fahr - 32)` als Ganzes in einen `double` umgewandelt werden, damit der Multiplikations-Operator ausgeführt werden kann. Das Resultat der gesamten rechten Seite der Anweisung ist somit vom Typ `double`.

Schlussendlich wird das Resultat der rechten Seite in die Variable auf der linken Seite gespeichert. Damit muss jedoch der `double`-Typ in einen `float`-Typ umgewandelt werden gemäß den Regeln der Zuweisung.

An dieser Stelle sei angemerkt, dass man sich diese letzte Umwandlung sparen kann, indem die Gleitpunkt-Zahl mit `5.0f` geschrieben wird. Dadurch wird die Zahl automatisch als `float` aufgefasst und sämtliche impliziten Konvertierungen werden danach ebenfalls vom Typ `float` sein.

In dem vorangegangenen Beispiel wurde der Divisions-Operator verwendet. Es muss darauf hingewiesen werden, dass der Divisions-Operator eine häufige Fehlerquelle darstellt, wenn es um implizite Typumwandlung geht: Der Ausdruck `(5.0 / 9)` dividiert eine Gleitpunkt-Zahl durch einen Integer. Würde anstelle der `5.0` nur der Integer `5` dastehen, so würde keine implizite Umwandlung stattfinden und es würde (fälschlicherweise) die Integer-Division `(5 / 9)` mit dem Ergebnis `0` ausgeführt werden.

Bei Divisionen wird der Compiler die Integer-Division durchführen, solange alle Operanden Integer sind. Eine implizite Umwandlung in eine Gleitpunkt-Zahl findet nicht statt.



Die obengenannte Regel bildet alle wichtigen Punkte über implizite arithmetische Umwandlungen ab. Ein Spezialfall sollte jedoch noch besprochen werden: Die Typ-Promotionen.

9.10.2 Die int- und double-Erweiterung (Promotion)

Die Typen `int` und `double` werden in C als Default-Typen angenommen. Für den Fall, dass beispielsweise Parameter bei Auslassungspunkten `...` übergeben werden oder bei binären Operatoren, welche die verwendeten Operanden-Typen nicht unterstützen, wird vom Compiler automatisch versucht, die Operanden zu dem Default-Typ zu erweitern, welcher den Wertebereich des Ursprungstyps abzubilden vermag. Dies wird als „**Typ-Erweiterung**“ (auf Englisch „**type promotion**“) bezeichnet und verläuft nach folgenden Regeln:

Wenn der vorzeichenbehaftete Typ `signed int` den Wertebereich des ursprünglichen Typs vollständig abbilden kann, wird der ursprüngliche Typ automatisch in den Typ `signed int` erweitert. Falls dies nicht der Fall ist, stattdessen jedoch der vorzeichenlose Typ `unsigned int` den Wertebereich des ursprünglichen Typs vollständig abbilden kann, wird der ursprüngliche Typ automatisch in den Typ `unsigned int` erweitert.

Wenn eine Gleitpunkt-Zahl als Parameter der Auslassungspunkte `...` einer Funktion übergeben wird, wird jedes Auftreten des Typs `float` automatisch zum Typ `double` erweitert.

9.11 Konvertierungsvorschriften

Im Folgenden werden die Wandlungsvorschriften zwischen verschiedenen Typen behandelt. Ein Compiler versucht grundsätzlich, die Konvertierung zwischen verschiedenen Typen – wenn immer möglich – werterhaltend zu gestalten. Sprich: Der umgewandelte Wert im Ziel-Typ entspricht – wenn immer möglich – dem ursprünglichen Wert im Ursprungstyp.

Wann immer der Wertebereich eines Ziel-Typs nicht ausreichend ist, um den zu konvertierenden Wert aufzunehmen, werden Fehler passieren, welche je nach Standard und Compiler implementationsabhängig sein können.

Hier ist zu beachten, dass diese Vorschriften sowohl bei impliziten, als auch expliziten Typumwandlungen mithilfe des Casting-Operators (siehe dazu Kapitel → 9.9.5) angewendet werden.

In den folgenden Unterkapiteln werden die verschiedenen Fälle behandelt und anschließend durch Beispiele veranschaulicht.

9.11.1 Umwandlungen zwischen Integer-Typen

Eine Konvertierung zwischen Integer-Typen ergibt genau dann den ursprünglichen arithmetischen Wert, wenn der Ziel-Typ den ursprünglichen Wert abbilden kann.

Andernfalls kann bei heutigen Compilern normalerweise davon ausgegangen werden, dass die höherwertigen Bits eines nicht abbildbaren Wertes einfach abgeschnitten werden.

In allen anderen Fällen ist das Resultat implementationsspezifisch.

9.11.2 Umwandlungen zwischen Integer- und Gleitpunkt-Typen

Wenn ein Wert aus einem Integer-Typ in einen Gleitpunkt-Typ umgewandelt wird, so werden im Prinzip als Nachkommastellen Nullen eingesetzt. Sehr große Zahlen können jedoch möglicherweise nicht exakt dargestellt werden. Das Resultat ist dann entweder der nächst höhere oder der nächst niedrigere darstellbare Wert.

Bei der Wandlung einer Gleitpunkt-Zahl in eine Integer-Zahl werden die Stellen hinter dem Komma abgeschnitten (Dies entspricht einer Rundung hin zu null). Ist die Zahl zu groß und kann nicht im Wertebereich des Integer-Typs dargestellt werden, so ist der Effekt der Umwandlung nicht definiert.

Sollen negative Gleitpunkt-Zahlen in unsigned Integer-Werte umgewandelt werden, so ist der Effekt nicht definiert.

9.11.3 Umwandlungen zwischen Gleitpunkt-Typen

Die Umwandlung zwischen verschiedenen Gleitpunkt-Typen wird den Wert erhalten, wann immer dies möglich ist. Ist es nicht exakt möglich, so wird der nächst kleinere oder größere Wert genommen. Gibt es keinen solchen Wert, so ist das Resultat nicht definiert.

9.11.4 Zwei Beispiele für Typumwandlungen

Dieses erste Beispiel demonstriert die Zuweisung eines float-Wertes an eine int-Variable:

truncate.c

```
#include <stdio.h>

int main(void) {
    double x = 4.2;
    int zahl;

    zahl = x;
    printf("zahl = %d\n", zahl);
    return 0;
}
```

Bei der Zuweisung `zahl = x` wird der Teil nach dem Dezimalpunkt abgeschnitten. Darauf weist eine Warnung des Compilers hin. Wenn das Programm dennoch ausgeführt wird, ist die Ausgabe 4.

Das zweite Beispiel zeigt implizite Typumwandlungen bei der Verwendung von `signed` und `unsigned int`:

typConvert.c

```
#include <stdio.h>

int main(void) {
    int i = -1;
    unsigned int u = 2;
    printf("%u\n", u * i);

    i = u * i;
    printf("%d\n", i);
    return 0;
}
```

Gemäß den Regeln des Standards wird vor der Produktbildung `u * i` der Operand `i` in `unsigned int` umgewandelt, da dieser als der mächtigere Typ gilt. Entsprechend ist die erste Ausgabe 4294967294.

Bei der Zuweisung `i = u * i` ist der Operand rechts somit ebenfalls vom Typ `unsigned int`. Er muss jedoch in den Typ links des Gleichheitszeichens, also in `int` gewandelt werden. Somit ist die Ausgabe hier -2.

9.12 Übungsaufgaben

Aufgabe 1: Arithmetische Operatoren

Schreiben Sie ein Programm, welches die Zahlen 1 bis 10 ausgibt und dabei zu jeder Zahl folgende Informationen mitschreibt:

- Die Summe mit der vorherigen Zahl.
- Die Differenz der Zahl zur Zahl 5.
- Das Quadrat der Zahl.
- Die Zahl geteilt durch 4.
- Ob die Zahl ohne Rest durch 3 teilbar ist.

Aufgabe 2: Vergleichs-Operatoren

Schreiben Sie ein Programm, welches die Zahlen 1 bis 10 ausgibt und dabei zu jeder Zahl ausgibt, ob sie kleiner, gleich oder größer als 5 sind. Die Zahl 7 soll jedoch nicht ausgegeben werden!

Aufgabe 3: Logische Operatoren

Schreiben Sie ein Login-Programm für eine Bank, welches eine gegebene Kontonummer (vom Typ `int`) und ein Passwort (ebenfalls vom Typ `int`) mit fest im Programmcode verankerten Werten gegenprüft. Nutzen Sie hierfür folgendes Gerüst:

kontoAbfrage.c

```
#include <stdio.h>

#define KONTONUMMER 6284
#define PIN 3141

int main(void) {
    int kontonummerEingabe = 1234;
    int pinEingabe = 777;

    // ...

    return 0;
}
```


Die beiden Variablen `kontonummerEingabe` und `pinEingabe` stellen die gegebenen Werte dar, die Konstanten `KONTONUMMER` und `PIN` sind die im Programmtext fest verankerten Werte.

- a) Verfassen Sie das Programm zuerst so, dass die beiden Eingabewerte gleichzeitig geprüft werden, sodass am Ende eine Nachricht über ein erfolgreiches Login oder Fehlschlag ausgegeben wird.
- b) Verändern Sie das Programm so, dass die beiden Eingabewerte einzeln nacheinander geprüft werden und je eine Fehlermeldung ausgegeben wird, wenn etwas davon nicht stimmt.
- c) Bauen Sie in das Programm zwei zusätzliche Kontonummern und Passwörter gemäß folgendem Code ein:

```
#define ANZAHL 3
int kontonummern[ANZAHL] = {6284, 7243, 1614};
int pins[ANZAHL] = {3141, 5974, 7777};
```

Das Programm muss für alle korrekten Kombinationen funktionieren und alle inkorrekten Kombinationen als Fehler anzeigen.

- d) Bankangestellte sind vergessliche Menschen und brauchen ein Hintertürchen. Bauen Sie die zusätzliche Kontonummer 1337 ein, welche jedes Passwort akzeptiert.
- e) Das Hintertürchen darf nicht von Hackern verwendet werden. Hacker zeichnen sich seltsamerweise dadurch aus, dass ihre Passwörter stets durch 7 teilbar sind. Verunmöglichen Sie es den Hackern, auf ein Bankkonto zuzugreifen!

Aufgabe 4: Bedingungs-Operator

Gegeben sei das folgende Programm:

condition.c

```
#include <stdio.h>

int main(void) {
    int i;
    int x = 1;
    int y = 2;
    x == y ? (i = 1) : (i = 0);
    printf("i = %d\n", i);
    return 0;
}
```

- Schreiben Sie die Zeile mit dem Bedingungs-Operator so um, dass sie direkt als Definition `int i = ...` funktioniert.
- Ersetzen Sie den Bedingungs-Operator durch eine `if`-Anweisung mit `else`-Teil.

Aufgabe 5: Der Subtraktions-Zuweisungs-Operator

Erläutern Sie, was das folgende Programm tut. Überzeugen Sie sich durch einen Programmlauf. Variieren Sie die beiden Variablen `a` und `b`.

subtraction.c

```
#include <stdio.h>

int main(void) {
    int a = 53;
    int b = 11;

    while (a >= b) {
        a -= b;
    }
    printf("??????? ist : %d\n", a);
    return 0;
}
```

Aufgabe 6: Notenrechner

- a) Analysieren Sie das folgende Programm. Variieren Sie den Wert der Variablen `note` und prüfen sie das ausgegebene Resultat.
- b) Ersetzen Sie die `else if`-Anweisungen durch den Operator `||` für das logische ODER, sodass nur ein `else` im Programm steht.
- c) Vereinfachen Sie das Programm weiter: Verwenden Sie den Bedingungs-Operator, um die Variable `bestanden`, sowie die beiden Definitionen `JA` und `NEIN` vollständig zu entfernen.

NotenRechner.c

```
#include <stdio.h>

#define SPITZE          1
#define GUT             2
#define BEFRIEDIGEND   3
#define AUSREICHEND     4
#define DURCHGEFALLEN  5
#define JA              1
#define NEIN            0

int main(void) {
    int note = 4;

    int bestanden = NEIN;
    if (note == SPITZE) bestanden = JA;
    else if (note == GUT) bestanden = JA;
    else if (note == BEFRIEDIGEND) bestanden = JA;
    else if (note == AUSREICHEND) bestanden = JA;
    else bestanden = NEIN;

    printf(bestanden ? "JA, " : "NICHT ");
    printf("WEITER SO !\n");

    return 0;
}
```

Aufgabe 7: Gebrauch verschiedener Operatoren

Gegeben sei:

```
int a = 2;  
int b = 1;  
int* ptr = &b;
```

Finden Sie ohne C-Compiler heraus, welcher Wert der Variablen `a` in den einzelnen Anweisungen zugewiesen wird. Beachten Sie dabei genau die Priorität der entsprechenden Operatoren (siehe Kapitel [→ 9.2](#)). Erläutern Sie, wie Sie auf das Ergebnis kommen. Verifizieren Sie ihr theoretisch ermitteltes Ergebnis gegebenenfalls durch einen Programmlauf.

```
a = b = 2;  
a = 5 * 3 + 2;  
a = 5 * (3 + 2);  
a *= 5 + 5;  
a %= 2 * 3;  
a = !(--b == 0);  
a = 0 && 0 + 2;  
a = b++ * 2;  
a = - 5 - 5;  
a = -(b++);  
a = 5 == 5 && 0 || 1;  
a = ((((((b + b) * 2) + b) && b || b)) == b);  
a = b ? 5 - 3 : b;  
a = 5 ** ptr;  
a = b + (++b);
```

10 Kontrollstrukturen



Die sequenzielle Programmausführung kann durch sogenannte „**Kontrollstrukturen**“ beeinflusst werden. Diese erlauben es, an bestimmten Stellen im Code an andere Stellen zu springen. Dadurch können in Abhängigkeit von der Bewertung von Ausdrücken Anweisungen übergangen oder ausgeführt werden.

10.1 Blöcke – Kontrollstruktur für die Sequenz

Der folgende Code zeigt einen **Block**:

```
{  
  Anweisung 1  
  Anweisung 2  
  Anweisung 3  
  ...  
  Anweisung n  
}
```

Die geschweiften Klammern { und } stellen die Blockbegrenzer dar. Die Anweisungen zwischen den Blockbegrenzern werden sequenziell abgearbeitet. Ein Block wird deshalb auch als Kontrollstruktur für die Sequenz bezeichnet.

Ein Block ist eine Sequenz von Anweisungen.



Ein Block zählt syntaktisch als eine einzige Anweisung.



Erfordert die Syntax eines Programms genau eine einzige Anweisung, wie beispielsweise bei der if-Struktur, so können dennoch mehrere Anweisungen geschrieben werden, wenn man sie in Form eines Blockes zusammenfasst.



Blöcke werden noch ausführlich in Kapitel [→ 11.1](#) behandelt.

Ergänzende Information Die elektronische Version dieses Kapitels enthält Zusatzmaterial, auf das über folgenden Link zugegriffen werden kann https://doi.org/10.1007/978-3-658-45209-4_10.

10.2 if und else – Einfache Selektion

Für die **einfache Selektion** werden die Schlüsselwörter **if** und **else** benötigt. Die Syntax ist die Folgende:

```
if (Bedingung)
    Anweisung 1
else
    Anweisung 2
```

Bei der einfachen Selektion wird die Bedingung in runden Klammern notiert. Ist die Bedingung wahr, so wird Anweisung 1 ausgeführt. Ist die Bedingung nicht wahr, so wird Anweisung 2 ausgeführt.

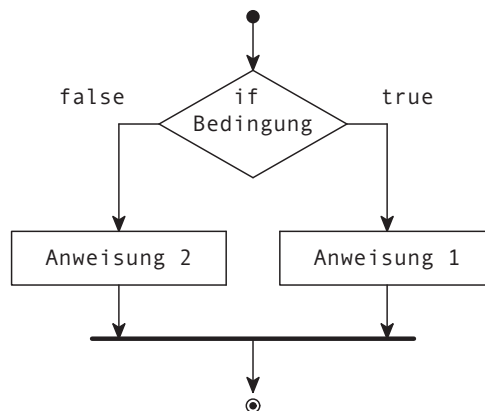


Eine **Bedingung** gilt als wahr, wenn der Ausdruck zu einer von 0 verschiedenen Zahl ausgewertet wird und als falsch, wenn der Ausdruck zu 0 ausgewertet wird.



Die beiden Anweisungen nach der Bedingung und nach dem else werden als die Zweige der Kontrollstruktur bezeichnet. Die if-Struktur „verzweigt“ somit zu einem der beiden Anweisungen. Im Englischen spricht man auch von „branching“.

Das folgende Bild zeigt das zugehörige Diagramm:



Soll mehr als eine einzige Anweisung ausgeführt werden, so ist ein Block mit geschweiften Klammern zu verwenden, der syntaktisch als eine einzige Anweisung zählt:

```
if (Bedingung) {  
    Anweisung 1.1  
    Anweisung 1.2  
} else {  
    Anweisung 2.1  
    Anweisung 2.2  
}
```

Heutzutage gilt es als unschön, eine if-Struktur ohne Block, sprich ohne geschweifte Klammer zu schreiben. Dennoch wird die Schreibweise ohne Klammern in seltenen Fällen auch heute noch verwendet, um kompakteren Code zu schreiben.

Der else-Zweig ist optional. Entfällt der else-Zweig, so spricht man von einer bedingten Anweisung:

```
if (Bedingung) {  
    Anweisung  
}
```

Bei einer bedingten Anweisung wird die Anweisung nur dann ausgeführt, wenn die Bedingung zutrifft. Trifft die Bedingung nicht zu, so wird sofort zu der Anweisung nach der bedingten Anweisung gesprungen.



Für die Bedingung in den runden Klammern werden am häufigsten Vergleichsoperatoren (→ 9.6) wie beispielsweise $a < 100$ verwendet. Die Bedingung kann jedoch ein beliebiger Ausdruck sein, der zu einem numerischen Wert ausgewertet wird. Es zählt einzig und alleine, ob dieser Wert 0 oder eine von 0 verschiedene Zahl ist. Somit können auch andere numerische Werte als Bedingung herhalten. Folgende beiden Zeilen sind somit äquivalent:

```
if (a != 0) ...  
if (a) ...
```

10.2.1 Geschachtelte if-Strukturen

Im if-Zweig darf eine beliebige Anweisung stehen. Da die if-Struktur selbst als Anweisung gilt, können somit auch geschachtelte if-Strukturen entstehen.



Ein einfaches Beispiel ist eine Struktur wie die folgende:

```
if (a > 1000)
    if (a > 1000000)
        printf("Sehr grosse Zahl");
    else
        printf("Grosse Zahl");
```

Hierbei ist zu beachten, dass in dieser komplexen Struktur nur ein else für die beiden if-Zweige existiert.

Der else-Zweig wird immer mit dem letzten if verbunden, für das noch kein else-Zweig existiert.



So gehört im obigen Beispiel der else-Zweig somit zur Bedingung $a > 1000000$. Im Code wird die Zuordnung der Fälle durch Einrücken der Codezeilen visuell kenntlich gemacht. Für den Compiler haben solche Einrückungen jedoch keinerlei Bedeutung. Bei komplexerem Code empfiehlt sich somit, mittels geschweifter Klammern Blöcke zu definieren:

```
if (a > 1000) {
    if (a > 1000000) {
        printf("Sehr grosse Zahl");
    } else {
        printf("Grosse Zahl");
    }
}
```

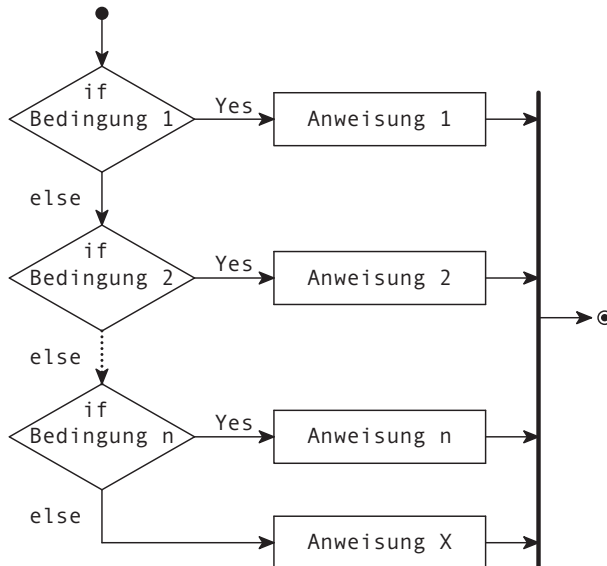

10.2.2 else if – Mehrfache Selektion

Im else-Zweig darf ebenfalls eine beliebige Anweisung stehen, also auch erneut eine if-Struktur. Dieses somit entstandene **else if** ist eine einfache Möglichkeit, eine Auswahl aus verschiedenen Fällen zu treffen.



Durch wiederholtes Einsetzen einer bedingten Anweisung in den jeweils letzten else-Zweig entsteht eine Mehrfach-Selektion:

```
if (Bedingung 1)
    Anweisung 1
else if (Bedingung 2)
    Anweisung 2
...
else if (Bedingung n)
    Anweisung n
else
    Anweisung X
```



In der angegebenen Reihenfolge wird ein Vergleich nach dem anderen durchgeführt. Bei der ersten Bedingung, die wahr ist, wird die zugehörige Anweisung abgearbeitet und danach die if-Struktur abgebrochen. Dabei kann wiederum statt einer einzelnen Anweisung stets auch ein Block von Anweisungen stehen.

Der letzte else-Zweig ist optional. Hier werden alle anderen Fälle behandelt, die in den vorherigen Bedingungen nicht explizit aufgeführt sind.



Dieser else-Zweig wird normalerweise entweder für die Behandlung von Daten verwendet, welche keine Spezialbehandlung benötigen, oder zum Abfangen von Fehlern oder noch nicht ausprogrammierten Programmteilen.

Mit `else if` können somit ganz einfach verschiedene Bedingungen geprüft werden. Die grundlegende Entscheidung, welcher Zweig denn nun ausgeführt werden soll, wird jedoch grundsätzlich immer nur zwischen zwei Möglichkeiten ($\emptyset$ oder nicht $\emptyset$) gefällt. Die `switch`-Struktur bietet eine andere Art der mehrfachen Selektion:

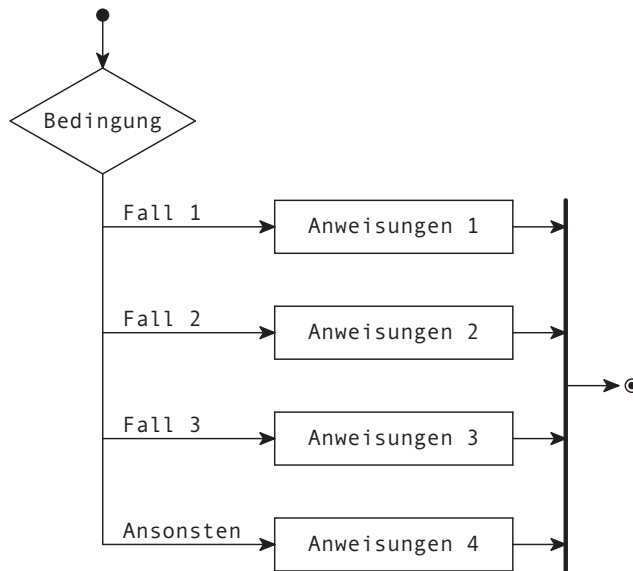
10.3 switch – Fallunterscheidung

Für eine **Mehrfach-Selektion** basierend auf einer Integer-Zahl kann die **switch**-Struktur verwendet werden. Die `switch`-Struktur verzweigt direkt zu dem gewünschten „Fall“, welcher dem gegebenen Integer-Wert entspricht.



```
switch (Bedingung) {  
    case 1:  
        Anweisungen 1  
        break;  
    case 2:  
        Anweisungen 2  
        break;  
    ...  
    case n:  
        Anweisungen n  
        break;  
    default:  
        Anweisungen X  
        break;  
}
```

Die vorangegangene switch-Struktur wird durch das folgende Diagramm visualisiert:



Jeder Fall wird mit dem Schlüsselwort **case**, der gewünschten Integer-Konstanten 1, ..., n und einem Doppelpunkt eingeleitet. Dies wird als case-Marke (auf Englisch „case label“) bezeichnet.

Die Integer-Konstanten können auch konstante Ausdrücke wie beispielsweise $(4 * 3)$ sein.

Hat der Ausdruck der Bedingung den gleichen Wert wie einer der konstanten Ausdrücke der case-Marken, so wird die Ausführung des Programms mit der Anweisung hinter dieser case-Marke weitergeführt. Stimmt keiner der konstanten Ausdrücke mit dem switch-Ausdruck überein, so wird zur Anweisung nach der **default**-Marke (auf Englisch „default label“) gesprungen.



Die default-Marke ist optional. Wenn keine default-Marke in der switch-Struktur hingeschrieben wird, wird das Programm bei Nichtzutreffen aller aufgeführten konstanten Ausdrücke mit der Anweisung nach der switch-Struktur fortgeführt.

Innerhalb einer switch-Struktur müssen alle case-Marken voneinander verschieden sein.



Wird bei der Auswertung der Bedingung eine passende case-Marke gefunden, werden die dort stehenden Anweisungen bis zum break ausgeführt. **break** springt dann zu der auf die switch-Struktur folgenden Anweisung. Die break-Anweisung wird ausführlich in Kapitel [→ 10.8](#) behandelt.

Die Reihenfolge der case-Marken ist beliebig. Auch die default-Marke muss nicht als letzte stehen. Dennoch hat es sich eingebürgert, dass die case-Marken normalerweise nach aufsteigenden Werten geordnet sind und die default-Marke am Schluss steht.

Fehlt die break-Anweisung, so werden die nach der nächsten case-Marke folgenden Anweisungen abgearbeitet. Dies geht so lange weiter, bis ein break gefunden wird oder bis das Ende der switch-Struktur erreicht ist.



Grundsätzlich sollte jeder Fall einer switch-Struktur mit einer break-Anweisung abgeschlossen werden. Wird eine break-Anweisung vergessen, führt das in aller Regel zu schwer zu entdeckenden Fehlern. Nur in ganz seltenen Fällen ist das Weglassen der break-Anweisung wirklich gewünscht.



10.3.1 enum-Aufzählungs-Konstanten als Bedingung

Die switch-Struktur eignet sich besonders gut für Fallunterscheidungen. Entsprechend wird diese Struktur häufig verwendet, um zwischen **Aufzählungs-Konstanten** zu unterscheiden, welche mittels des **enum**-Typs [→ 7.2.5](#) definiert wurden.

Der Code wird dadurch lesbarer, da anstelle von Zahlen symbolische Konstanten im Code stehen.

ampel.c

```
#include <stdio.h>

enum Color {RED, ORANGE, GREEN};

int main(void) {
    enum Color light = RED;

    switch (light) {
        case RED:
            printf("Stopp!");
            break;
        case ORANGE:
            printf("Warten...");
            break;
        case GREEN:
            printf("Los!");
            break;
        default:
            printf("Ampel defekt.");
    }
    return 0;
}
```

Die Ausgabe lautet:

Stopp!

Das Programm führt in der vorliegenden Form selbstverständlich stets zum selben Resultat, da als Wert von `light` im Programm `RED` fest vorgegeben wird. Nützlich wäre eine solche `switch`-Struktur beispielsweise in einer Funktion, welche die Variable `light` als Parameter definiert.

Die `break`-Anweisung am Ende des `default`-Falles kann ohne Probleme weggelassen werden, da die `switch`-Struktur an dieser Stelle schon fertig abgearbeitet ist. Würde man jedoch in diesem Beispiel die anderen `break`-Anweisungen vergessen, so wäre das Resultat `Stopp!Warten...Los!Ampel defekt.`

Es ist zu beachten, dass symbolische Konstanten nicht nur mittels `enum`, sondern auch mittels `#define` erstellt werden können → 21.2.1. Die Verwendung von `enum` ist jedoch klar bevorzugt.

10.3.2 Reihen von case-Marken

Die case-Marken einer switch-Struktur führen selbst keinen Code aus, sondern markieren nur den Ort, zu welchem gesprungen werden soll. Da jede case-Markierung auf die jeweils darauffolgende Anweisung zeigt, ist es auch möglich, mehrere case-Marken hintereinander zu schreiben, welche schlussendlich allesamt auf dieselbe Anweisung zeigen.

Im folgenden Beispiel wird ausgewertet, ob eine gewürfelte Zahl gerade oder ungerade ist:

wuerfel.c

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main(void) {
    srand((int)time(NULL));
    int zahl = rand() % 6 + 1;

    switch (zahl) {
        case 1:
        case 3:
        case 5:
            printf("Ungerade Zahl gewuerfelt: %d\n", zahl);
            break;
        case 2:
        case 4:
        case 6:
            printf("Gerade Zahl gewuerfelt: %d\n", zahl);
            break;
    }
    return 0;
}
```

10.3.3 Unterschiede zwischen switch und else if

Sowohl mit switch als auch mit else if kann **Mehrfach-Selektion** abgearbeitet werden. Dennoch werden die beiden Strukturen in unterschiedlichen Situationen genutzt.

Grundsätzlich gilt: Die if-Struktur eignet sich gut für logische Entscheidungen, wohingegen die switch-Struktur für Fallunterscheidungen gebraucht wird.

Die folgenden Unterschiede bestehen:

- switch prüft in der Bedingung auf die Gleichheit von Integer-Werten mit den vorhandenen case-Marken, wohingegen bei else if getestet wird, ob die Bedingung 0 oder nicht 0 ist.
- Die Bedingung der if-Struktur erlaubt beliebige Werte, solange sie zu einem Null-Wert ausgewertet werden. Dies beinhaltet Gleitpunkt-Zahlen sowie Pointer.
- Die Bedingung der switch-Struktur erlaubt explizit nur Integer oder Ausdrücke, welche zu solchen ausgewertet werden. Zeichen-Literale → 6.5.3 wie 'A' können beispielsweise als Integer-Zahlen interpretiert werden, ein Pointer oder eine Gleitpunkt-Zahl jedoch nicht.
- Die Effizienz der switch-Struktur ist gegenüber der else if-Anweisung in der Regel besser, da bedingt durch die konstanten case-Marken der Compiler bessere Optimierungsmöglichkeiten hat. Im Falle der else if-Bedingungen kann die Auswertung normalerweise erst zur Laufzeit erfolgen.
- Die Übersichtlichkeit beziehungsweise Erweiterbarkeit ist bei switch besser als bei else if.
- Moderne Compiler können bei switch auf die vollständige Auflistung aller Aufzählungs-Konstanten eines Aufzählungs-Typs enum → 7.2.5 prüfen. Dadurch werden keine Fälle vergessen.

Falls möglich, sollte statt umfangreicher else if-Konstruktionen bevorzugt switch benutzt werden.



10.3.4 Definition von Variablen innerhalb der switch-Struktur

Innerhalb der geschweiften Klammern einer switch-Struktur können Variablen definiert werden. Hierbei ist jedoch Vorsicht geboten:

Werden innerhalb einer switch-Struktur Variablen definiert, so sind sie grundsätzlich im gesamten switch-Block sichtbar. Sie werden jedoch nur in demjenigen case-Fall initialisiert, in welchem ihre Definition steht. Steht die Initialisierung vor sämtlichen case-Fällen, wird sie niemals initialisiert, was zu einem undefinierten Verhalten führt.



Variablen werden dementsprechend innerhalb einer switch-Struktur nur selten definiert. Um die damit entstehenden Probleme zu lösen, gibt es einige Möglichkeiten, welche jedoch meist den Code unleserlicher machen:

- Die Variable wird einfach außerhalb der Struktur definiert. Gewisse Fälle könnten diese Variable jedoch nie brauchen, weswegen die Initialisierung dann möglicherweise überflüssig war. Außerdem befindet sich die Variable dann in dem Block außerhalb, wo sie nicht hingehört.
- Umklammerung eines Falls mit geschweiften Klammern. Dadurch entsteht ein eigener Code-Block, was die Sichtbarkeit und Lebensdauer der Variablen klar definiert. Dies sieht aber nicht schön aus.
- Aufruf einer Funktion anstelle der Ausprogrammierung des Falls innerhalb der switch-Struktur. Wird eine Funktion aufgerufen, kann dort die Variable ohne Probleme definiert und initialisiert werden. Leider ist dafür jedoch ein kostspieliger Funktionsaufruf nötig.

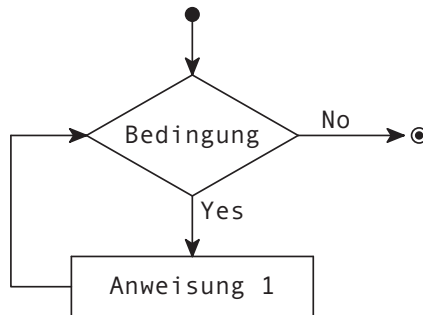
Es ist erwähnenswert, dass die Definition von Variablen innerhalb einer switch-Struktur vor dem Standard C99 nicht möglich war.

Auf Beispiele und weitere Erläuterungen wird hier verzichtet.

10.4 while – Bedingte Schleife

Die Syntax der **while**-Schleife lautet:

```
while (Bedingung)  
    Anweisung
```



In einer while-Schleife wird eine Anweisung in Abhängigkeit von einer Bedingung wiederholt ausgeführt.



Die Bedingung wird vor jedem Durchgang ausgewertet und die Anweisung nur dann ausgeführt, wenn das Resultat ungleich 0 ist. Dieser Vorgang wiederholt sich danach, indem der Ausdruck erneut ausgewertet und die Anweisung ausgeführt wird, solange, bis der Ausdruck der Bedingung 0 ergibt.

Sollen mehrere Anweisungen ausgeführt werden, so ist ein Block mit geschweiften Klammern zu verwenden.



Sobald die Bedingung nicht mehr erfüllt ist, fährt das Programm mit der ersten Anweisung nach der Schleife fort.

Da die Bedingung vor der Ausführung der Anweisung bewertet wird, kann es sein, dass die Anweisung niemals ausgeführt wird, da die Bedingung bereits zu Beginn nicht erfüllt ist. Es wird deswegen auch von einer „abweisenden“ Schleife gesprochen.

Folgendes einfaches Beispiel gibt von einem gegebenen String Zeichen für Zeichen aus, bis das erste Mal der Buchstabe 'e' auftaucht.

while.c

```
#include <stdio.h>

int main(void) {
    const char* text = "Hallo Welt";

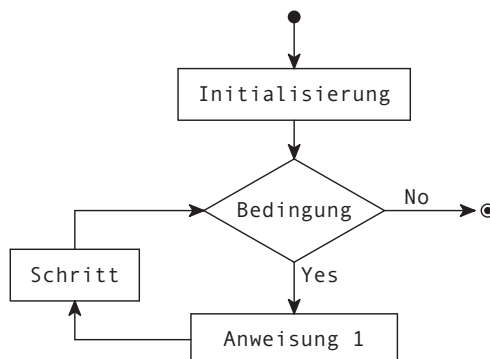
    int pos = 0;
    while (text[pos] && text[pos] != 'e') {
        printf("%c", text[pos]);
        ++pos;
    }
    return 0;
}
```

Für den Eingabe-String Hallo Welt wird somit Hallo W ausgegeben. Es ist zu beachten, dass die Bedingung zusätzlich zur Erkennung des Buchstabens 'e' auch noch überprüft, ob der Buchstabe an Position pos nicht der Null-Character '\0' ist. Damit wird das Ende des Strings erkannt. Ohne diese zusätzliche Prüfung würde das Programm bei einem Eingabe-String ohne den Buchstaben 'e' über das Stringende hinauslaufen und auf unzulässigen Speicher zugreifen.

10.5 for – Zählschleife

Die Syntax der **for**-Schleife lautet:

```
for (Initialisierung; Bedingung; Schritt)
    Anweisung
```



Die runden Klammern der for-Schleife enthalten drei durch Semikolons getrennte Ausdrücke:

- **Initialisierung:** Hier wird üblicherweise eine Variable definiert und initialisiert, welche im Verlauf der Schleife als **Laufvariable** dient. Andere gebräuchliche Namen sind Zählvariable oder Schleifenindex.
- **Bedingung:** Ein Ausdruck, der ähnlich wie bei einer while-Schleife vor jedem Durchlauf ausgewertet wird. Nur wenn dieser Ausdruck erfüllt ist, wird die Anweisung ausgeführt.
- **Schritt:** Wird am Ende jedes Durchlaufs automatisch ausgeführt.

Sollen mehrere Anweisungen ausgeführt werden, so ist ein Block mit geschweiften Klammern zu verwenden.



Die for-Schleife wird häufig als **Zählschleife** verwendet. Folgendes ist ein typisches Beispiel für eine for-Schleife:

increment.c

```
#include <stdio.h>
#define COUNT 10

int main(void) {
    for (int i = 0; i < COUNT; ++i) {
        printf("%d ", i);
    }
    return 0;
}
```

Die Ausgabe dieses Programmes lautet:

```
0 1 2 3 4 5 6 7 8 9
```

Die Laufvariable *i* wird in diesem Beispiel definiert und mit 0 initialisiert. Ein Durchlauf findet nur dann statt, wenn *i* < 10 gilt. Nach jedem Durchlauf wird die Laufvariable um 1 erhöht.

Die for-Schleife wird wie die while-Schleife eine „abweisende Schleife“ genannt, da vor Ausführung eines Durchlaufs die Bedingung geprüft wird und somit eine Schleife auch niemals durchlaufen werden kann.

10.5.1 Inhalt der Laufvariablen außerhalb der for-Schleife

Die Laufvariable wird üblicherweise direkt in den runden Klammern der for-Schleife definiert und initialisiert. Dadurch ist die Sichtbarkeit und Lebensdauer der Laufvariable klar auf die for-Schleife begrenzt.

Eine Variable zu definieren, ist jedoch nicht zwingend. Auch bereits außerhalb definierte Variablen können als Laufvariablen dienen. Diese Variablen haben selbstverständlich einen größeren Sichtbarkeitsbereich und eine längere Lebensdauer als die Schleife. Vor dem Standard C99 war dies gar die einzige Möglichkeit, eine Laufvariable zu definieren.

Nachdem die for-Schleife beendet wurde, bleibt in so einem Falle der aktuelle Wert der Laufvariablen erhalten, selbst wenn die Schleife mit der break-Anweisung [→ 10.8](#) verlassen wird.

Die Laufvariable einer for-Schleife, die bereits vor der Schleife definiert wird, enthält nach dem Verlassen der Schleife immer den zuletzt in der Schleife verwendeten Wert beziehungsweise den Wert, welcher zum Abbruch der Bedingung führte.



Die seit C99 gegebene Möglichkeit, die Variable innerhalb der runden Klammern zu definieren, wird in den meisten Fällen bevorzugt. Die Einschränkung der Sichtbarkeit der Laufvariablen ist sinnvoll, denn in den meisten Fällen ist der Zustand der Laufvariablen nach Abarbeitung der Schleife nicht mehr von Bedeutung. Wenn Variablen nicht sichtbar sind, werden unbeabsichtigte Fehler mit ihnen vermieden.

10.5.2 Dekrementierende for-Schleifen

Dieses Beispiel zeigt eine Schleife, bei welcher der Wert der Laufvariablen in jedem Durchgang verringert wird:

decrement.c

```
#include <stdio.h>
#define COUNT 10

int main(void) {
    for (int i = COUNT - 1; i >= 0; --i) {
        printf("%d ", i);
    }
    return 0;
}
```

Die Ausgabe dieses Programmes lautet:

```
9 8 7 6 5 4 3 2 1 0
```

Interessant hierbei ist, dass bei einer dekrementierenden Schleife üblicherweise mit dem Wert Startwert - 1 initialisiert wird und die Laufvariable danach auf größer gleich 0 geprüft wird. Dies deswegen, da Laufvariablen häufig für Indizes von Arrays verwendet werden, welche bekanntlich in C von 0 anfangen zu zählen

→ 8.3 .

10.5.3 Beispiel für Array-Indizes in einer for-Schleife

Häufig werden for-Schleifen für die Abarbeitung von ganzen Arrays verwendet. Das folgende Programm zeigt, wie ganz einfach die Fibonacci-Zahlen berechnet und in einem Array gespeichert werden können:

fibonacci.c

```
#include <stdio.h>

#define COUNT 10

int main(void) {
    int array[COUNT];
    array[0] = 1;
    array[1] = 1;
    printf("%d ", array[0]);
    printf("%d ", array[1]);

    for (int i = 2; i < COUNT; ++i) {
        array[i] = array[i - 1] + array[i - 2];
        printf("%d ", array[i]);
    }

    return 0;
}
```

Die Ausgabe dieses Programmes lautet:

```
1 1 2 3 5 8 13 21 34 55
```

Das Programm weist dem aktuellen Array-Element `array[i]` immer die Summe der beiden vorangegangenen Array-Einträge zu. Damit kein fehlerhafter Zugriff stattfindet, müssen die beiden ersten Array-Einträge manuell gefüllt und die Laufvariable muss mit dem Index 2 initialisiert werden.

10.5.4 Komplizierteres Beispiel

Alle drei Ausdrücke der for-Schleife können beliebig komplex werden. Folgendes (wenig sinnvolles) Beispiel hat folgende Anweisungen:

- **Initialisierung:** Variable i mit Startwert 5 und Variable k mit Startwert 1.
- **Bedingung:** Variable i muss kleiner als COUNT sein und außerdem muss Variable k kleiner als 10 sein.
- **Schritt:** Variable i wird um k erhöht und k um 1.

steps.c

```
#include <stdio.h>
#define COUNT 100

int main(void) {
    for (int i = 5, k = 1; i < COUNT && k < 10; i += k, ++k) {
        printf("%d ", i);
    }
    return 0;
}
```

Die Ausgabe dieses Programmes lautet:

```
5 6 8 11 15 20 26 33 41
```

Es ist zu beachten, dass bei der Initialisierung die beiden Variablen mithilfe des Kommas definiert wurden. So können zwei oder mehrere Variablen mit demselben Typ definiert werden (siehe Kapitel [→ 7.4.1](#)). Für die Schritt-Anweisung wurde das Komma des Sequenz-Operators [→ 9.9.3](#) verwendet. Dies ist ein kleiner Trick, welcher manchmal genutzt wird, um mehr als nur eine Variable im Schritt-Ausdruck zu verändern.

Auch wenn dieses Beispiel wenig sinnvoll war, zeigt es doch, wie kompliziert die Ausdrücke werden können und wie schwierig es werden kann, hierbei den Überblick zu behalten. Die Initialisierungs-, Bedingungs- und Schritt-Ausdrücke der for-Schleife sollten wann immer möglich sehr einfach gehalten werden.

10.5.5 Verschachtelte for-Schleifen

Zum Abschluss hier ein Beispiel mit zwei verschachtelten Schleifen. Es sollen die Primzahlen aufgezählt werden:

prim.c

```
#include <stdio.h>
#define MAX 30

int main(void) {
    for (int i = 2; i <= MAX; i = i + 1) {
        int istTeilbar = 0;

        for (int k = 2; k < i; ++k) {
            if (i % k == 0) {
                istTeilbar = 1;
            }
        }

        if (!istTeilbar) {
            printf("%d ", i);
        }
    }
    return 0;
}
```

Die Ausgabe dieses Programmes lautet:

```
2 3 5 7 11 13 17 19 23 29
```

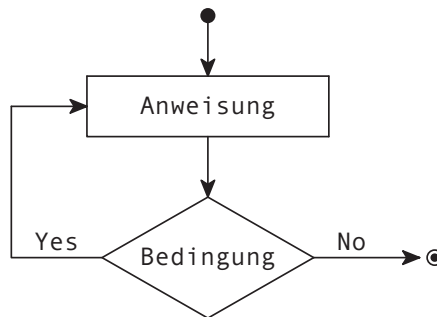
Das Programm berechnet die Primzahlen bis MAX. Dazu wird in einer äußeren for-Schleife die Laufvariable *i* von 2 bis MAX hochgezählt. Dann prüft die innere Schleife, die nach Teilern von *i* sucht, ob *i* modulo *k* den Wert 0 ergibt, also ob *i* durch *k* ohne Rest teilbar ist. Ist das der Fall, dann ist *i* keine Primzahl, da sie ja in zwei Faktoren ohne Rest zerlegt werden kann.

Diese Schleife läuft so lange, wie *k* kleiner als *i* ist. Falls *i* beim gesamten Durchlaufen der inneren Schleife nicht durch *k* ohne Rest teilbar war, liegt somit eine Primzahl vor und sie wird auf dem Bildschirm ausgegeben.

10.6 do while – Annehmende bedingte Schleife

Die Syntax der **do while**-Schleife ist:

```
do {  
    Anweisung  
} while (Bedingung);
```



Bei der **do while**-Schleife wird zuerst die Anweisung ausgeführt und erst danach wird die Bedingung zum ersten Mal ausgewertet. Genauso wie bei der **while**-Schleife wird danach die Anweisung solange ausgeführt, wie die Bedingung erfüllt ist. Sobald die Bedingung nicht mehr erfüllt ist, wird mit der ersten Anweisung nach der **do while**-Schleife fortgefahren.



Die Anweisung der **do while**-Schleife wird auf jeden Fall mindestens einmal durchlaufen, da die Bewertung des Ausdrucks erst am Ende der Schleife erfolgt. Die **do while**-Schleife wird somit als eine „annehmende Schleife“ bezeichnet. Im Gegensatz dazu können die **for**- und **while**-Schleife durchaus ihre Anweisung überhaupt nicht ausführen, nämlich dann, wenn die Schleifenbedingung von Anfang an nicht erfüllt ist.



Im Gegensatz zu allen anderen Schleifen wird bei der `do while`-Schleife ein Semikolon nach der Schleifenbedingung verlangt, weil hier das Ende der Anweisung erreicht ist.



In folgendem Beispiel wird eine Zahlenreihe von 0 bis 9 ausgegeben:

dowhile.c

```
#include <stdio.h>

int main(void) {
    int i = 0;
    do {
        printf("%d ", i);
        ++i;
    } while (i < 10);

    return 0;
}
```

Die Ausgabe dieses Programmes lautet:

```
0 1 2 3 4 5 6 7 8 9
```

Es ist zu beachten, dass die `do while`-Schleife äußerst selten benutzt wird. Zum einen ist der Fall, dass eine Schleife mindestens einmal durchlaufen werden muss, tendenziell rar. Zum anderen ist die Schleife weniger gut zu lesen, da sich die Bedingung am Ende befindet.

10.7 Endlosschleifen und leere Ausdrücke bei Schleifen

Eine **Endlosschleife** ist eine Schleife, bei welcher die Bedingung niemals fehlschlägt. Der Ausdruck der Schleife wird somit unendlich oft ausgeführt, sofern die Schleife nicht anderweitig unterbrochen wird.

Ein einfaches Beispiel einer Endlosschleife ist das Folgende:

```
while (1)
    Anweisung
```

Auch mit der for-Schleife kann entsprechend ganz einfach eine Endlosschleife erzeugt werden. Bei der for-Schleife gibt es jedoch noch eine andere Möglichkeit:

Jeder der drei Ausdrücke einer for-Schleife kann leer sein. Die Strichpunkte müssen aber trotz fehlendem Ausdruck stehen bleiben. Fehlt der Ausdruck der Initialisierung, wird nichts initialisiert. Fehlt der Ausdruck des Schritts, wird nach einem Durchgang nichts automatisch verändert. Fehlt der Ausdruck der Bedingung, so gilt die Bedingung immer als 1 und die Schleife wird somit eine Endlosschleife. Die geläufigste Form ist dabei, alle drei Ausdrücke wegzulassen, wie im folgenden Beispiel:

```
for ( ; ; )  
    Anweisung
```

Endlosschleifen können entweder gewollt oder ungewollt sein. Ungewollte Endlosschleifen führen zum „Aufhängen“ des Programmes, ein Zustand, in welchem das Programm nichts mehr tut und schlussendlich vom System gestoppt werden muss.

Solche ungewollte Endlosschleifen können passieren, wenn die Bedingung der Schleife nicht korrekt programmiert wurde oder wenn die Bedingung sich während der Abarbeitung der Anweisung niemals ändert.

Schleifen, bei denen sich die Bedingung nie ändert, können jedoch auch gewollt programmiert werden. Man nutzt dann häufig die **break**-Sprunganweisung (→ 10.8), um die Schleife zu einem beliebigen Zeitpunkt zu verlassen. Formal ist die Schleife immer noch eine Endlosschleife, aber das Abbruchkriterium wird nicht in der Bedingung, sondern innerhalb der Schleife geprüft. Ein einfaches Beispiel ist folgende Uhr:

busywait.c

```
// MSVC meldet fuer veraltete, unsichere C-Funktionen wie  
// localtime einen Fehler. So wird dieser Fehler ausgeschaltet:  
#define _CRT_SECURE_NO_WARNINGS  
  
#include <stdio.h>  
#include <time.h>  
#ifdef _WIN32  
    #include <windows.h>  
    #define SleepFunction Sleep  
#else  
    #include <unistd.h>  
    #define SleepFunction sleep  
#endif
```

```
int main(void) {
    while (1) {
        time_t t = time(0);
        struct tm* now = localtime(&t);

        printf("%d:%d:%d\n", now->tm_hour, now->tm_min, now->tm_sec);
        fflush(stdout);

        if (now->tm_hour == 12) {
            printf("Lunch!");
            break;
        }

        SleepFunction(1);
    }
    return 0;
}
```

Endlosschleifen ohne break-Anweisung sind in der Regel nicht sinnvoll.



Sinnvolle Anwendungen von Endlosschleifen ohne break-Anweisung gibt es jedoch: Beispielsweise arbeiten moderne GUI-Programme mittels sogenannter „Application-Loops“, welche einfach nur Eingaben abwarten und nach Bearbeitung einer Eingabe sofort wieder in Wartestellung gehen und dies unendlich lange wiederholen. Auch sogenannte „demons“, welche das Betriebssystem unterstützen, arbeiten häufig mit einer unendlichen Warteschleife. Auch beim Thema Multithreading [→ 23](#) werden häufig solche Loops verwendet.

10.8 break – Abbruch-Anweisung

Schleifen und switch-Strukturen können mittels der Anweisung break frühzeitig abgebrochen werden.

Mit der **break**-Anweisung kann eine do while-, while- und for-Schleife und eine switch-Struktur abgebrochen werden.



Abgebrochen wird immer nur die aktuelle Schleife beziehungsweise switch-Struktur. Sind mehrere Schleifen oder switch-Strukturen geschachtelt, wird lediglich die innerste verlassen.



Beispiele zur switch-Struktur können in Kapitel [→ 10.3](#) nachgelesen werden. Das folgende Beispiel zeigt das Verlassen einer for-Schleife mit break:

break.c

```
#include <stdio.h>
#include <string.h>

int main(void) {
    const char* text = "Hallo Welt";
    int pos;

    for (pos = 0; pos < strlen(text); ++pos) {
        if (text[pos] == 'e')
            break;
    }

    printf("Das erste e ist an Stelle %d.\n", pos);
    return 0;
}
```

Die Ausgabe lautet:

```
Das erste e ist an Stelle 7.
```

10.9 continue – Fortsetzungs-Anweisung

Die **continue**-Anweisung ist wie **break** eine Sprunganweisung. Im Gegensatz zu **break** wird aber eine Schleife nicht verlassen, sondern ein neuer Durchgang gestartet.



Die **continue**-Anweisung kann auf die **do while**-, die **while**- und die **for**-Schleife angewandt werden. Bei **do while** und **while** wird nach **continue** direkt zum Bedingungstest der Schleife gesprungen. Bei der **for**-Schleife wird zuerst noch der Schritt ausgeführt.

Angewandt wird die **continue**-Anweisung zum Beispiel, wenn an einer gewissen Stelle des Schleifenrumpfes mit einem Test festgestellt werden kann, ob der restliche Teil noch ausgeführt werden muss beziehungsweise darf.



Das folgende Beispiel zeigt die Verwendung von **continue** in einer **while**-Schleife. Die **continue**-Anweisung führt hier dazu, dass nur gerade Zahlen aufsummiert werden.

continue.c

```
#include <stdio.h>

int main(void) {
    int sum = 0;
    for (int i = 0; i < 20; ++i) {
        if (i % 2) {
            continue;
        }
        sum += i;
    }
    printf("Summe: %d\n", sum);

    return 0;
}
```

Die Ausgabe lautet:

```
Summe: 90
```

10.10 return – Rücksprung-Anweisung

Die return-Anweisung wird ausführlich in Kapitel → 11.4 behandelt.

10.11 goto – Sprunganweisung und Marken

Mit **goto** wird ohne Bedingung an eine gegebene Marke gesprungen. Eine **Marke** (auf Englisch „**label**“) wird durch Angabe eines Namens, gefolgt von einem Doppelpunkt : in den Code geschrieben.



Die Verwendung von Sprüngen und Marken ist in der Assembler-Programmierung eine Selbstverständlichkeit. In C wird diese Funktionalität mit der goto-Anweisung nachgebildet.

Eine Marke wird definiert durch einen Namen, der mit einem Doppelpunkt abgeschlossen ist. Stehen darf eine Marke vor jeder beliebigen Anweisung. Die Gültigkeit einer Marke erstreckt sich über die Funktion, in welcher sie definiert wurde, muss also nicht vorher deklariert werden. Gleichzeitig kann aber auch nicht aus einer Funktion herausgesprungen werden.

Um an eine bestimmte Marke zu springen, wird einfach die goto-Anweisung mit entsprechender Marke geschrieben:

```
Anweisungen  
goto HierGehtEsWeiter;  
Anweisungen  
HierGehtEsWeiter:  
Anweisungen
```

In der strukturierten Programmierung ist die goto-Anweisung nicht erlaubt.



Der Streit um die goto-Anweisung ist legendär: Viele Aufsätze und Gegen Aufsätze um dieses Thema wurden verfasst. Am berühmtesten ist wohl der Artikel „Go To Statement Considered Harmful“ [Dij68], was im Deutschen etwa „Goto ist gefährlich“ bedeutet.

Die in diesem Kapitel beschriebenen Kontrollstrukturen sind allesamt mächtig genug, um einen Einsatz von goto komplett zu vermeiden. Nur noch äußerst selten wird diese Sprunganweisung verwendet, beispielsweise um das Herausspringen aus mehrfach geschachtelten Schleifen im Fehlerfall einfacher zu gestalten. Aber diese Fälle sind nahezu ausgestorben.

10.12 Übungsaufgaben

Aufgabe 1: Maximum berechnen

- a) Schreiben Sie eine Funktion, welche 3 Integer-Zahlen erwartet und das Maximum der 3 Zahlen zurückgibt. Nutzen Sie hierfür eine if-Struktur.

```
int getMax3(int value1, int value2, int value3);
```

- b) Schreiben Sie eine Funktion, welche beliebig viele Zahlen als Array erwartet. Nutzen Sie hierfür eine for-Schleife.

```
int getMaxn(const int* values, int arrayCount);
```

- c) Schreiben Sie eine weitere Funktion, welche ein Array von Integer-Zahlen erwartet und prüft, ob sämtliche Zahlen kleiner sind als ein gegebenes Maximum. Falls dem so ist, wird 1 zurückgegeben, ansonsten 0. Nutzen Sie dabei die break- oder return-Anweisung, um frühzeitig abubrechen, wenn möglich.

```
int areAllLower(const int* values, int arrayCount, int max);
```

Aufgabe 2: Zwei Schleifen mit gleicher Funktionalität

Die Schleifen while und for verhalten sich unterschiedlich, doch können sie grundsätzlich immer durch die jeweils andere Schleife ersetzt werden.

Programmieren Sie folgende Aufgabe einmal mit einer for- und einmal mit einer while-Schleife. Schreiben Sie hierfür zwei Funktionen, einmal mit einer Index-Berechnung und einmal mit einer const char* Variable. Verwenden Sie zur Berechnung der Länge des gegebenen Strings die strlen()-Funktion der <string.h> Bibliothek.

Berechnen Sie die Quersumme einer Zahl. Die Zahl ist gegeben als ein String. Beispielsweise "623518894436". Die einfache Quersumme dieser Zahl ist die Summe aller Ziffern, in diesem Beispiel somit 59. Stellen Sie sicher, dass nur die Ziffern 0 - 9 gezählt werden!

Was sind die Vor- und Nachteile der jeweiligen Funktionen? Gibt es möglicherweise eine goldene Mitte?

Aufgabe 3: Römische Zahlen

- a) Schreiben Sie eine Funktion, welche eine einzelne römische Ziffer in die entsprechende Dezimalzahl umwandelt. Eine einzelne römische Ziffer soll dabei in einem char Typ verpackt sein. Verwenden Sie die folgende Umwandlungstabelle.

I = 1	V = 5	X = 10	L = 50	C = 100	D = 500	M = 1000
-------	-------	--------	--------	---------	---------	----------

- b) Erweitern Sie dieses Programm so, dass auch Kleinbuchstaben erkannt werden.
- c) Erweitern Sie das Programm so, dass ein ganzer String römischer Ziffern gelesen werden kann. Addieren Sie dabei die eingelesenen Ziffern und geben Sie das Resultat aus. Testen Sie Ihre Implementation mit der römischen Zahl MDCCCCLXXXIIII, welche 1984 ergeben sollte.
- d) Betrachten Sie die folgenden römischen Buchstabenkombinationen:

IV = 4	IX = 9	XL = 40	XC = 90	CD = 400	CM = 900
--------	--------	---------	---------	----------	----------

Erweitern Sie Ihr Programm, so dass diese Kombinationen erkannt werden können. Testen Sie Ihre Implementation mit der römischen Zahl MCMLXXXIV, welche ebenfalls 1984 ergeben sollte.

Achtung, diese letzte Aufgabe kann auf den ersten Blick schwierig sein. Tipp: Lesen Sie wie bisher die Ziffern einzeln ein. Speichern Sie dabei die im vorherigen Schleifendurchgang benutzte Ziffer in einer zusätzlichen Variablen und korrigieren Sie in jedem Schleifendurchlauf die Summe, falls eine der oben angegebenen Buchstabenkombinationen auftaucht.

11 Blöcke und Funktionen



Wer ein Programm in C schreibt, verpackt die auszuführenden Anweisungen in sogenannten „Funktionen“. Diese Funktionen besitzen einen Namen und können an beliebigen Stellen aufgerufen werden.

Eine **Funktion** ist eine Folge von Anweisungen mit einem Namen, die mittels eines Funktionsaufrufes ausgeführt werden.



Um dem Compiler klarzumachen, welche Anweisungen zu einer Funktion gehören, werden die Anweisungen in einen sogenannten „Block“ geschrieben. Eine Funktion besteht somit stets aus einem **Funktionskopf** (der sogenannten „**Signatur**“) und einem **Funktionsrumpf** (auch „Funktionskörper“ genannt, auf Englisch „body“).

```
int meineFunktion()      // Funktionskopf
{
    Anweisungen          // Funktionsrumpf
}
```

Ein **Block** stellt eine beliebige Folge von Anweisungen dar. Diese Folge von Anweisungen wird sequenziell im Programmcode ausgeführt.



Blöcke treten jedoch auch an anderen Stellen auf, insbesondere können sie ineinander verschachtelt werden. Diese Blöcke bezeichnen dann nicht Funktionskörper, sondern stehen einfach so für sich alleine ohne Namen und bilden eine abgrenzende Code-Umhüllung.

Innerhalb von Blöcken können Variablen definiert und Anweisungen ausgeführt werden. In den folgenden Unterkapiteln wird auf die verschiedenen Eigenschaften von Blöcken und Funktionen eingegangen.

Ergänzende Information Die elektronische Version dieses Kapitels enthält Zusatzmaterial, auf das über folgenden Link zugegriffen werden kann https://doi.org/10.1007/978-3-658-45209-4_11.

11.1 Struktur und Schachtelung von Blöcken

Die Anweisungen eines Blockes werden durch geschweifte Klammern als Blockbegrenzer zusammengefasst. Nach der schließenden geschweiften Klammer kommt kein Strichpunkt. Statt Block ist auch die Bezeichnung „zusammengesetzte Anweisung“ (auf Englisch „**compound statement**“) üblich.



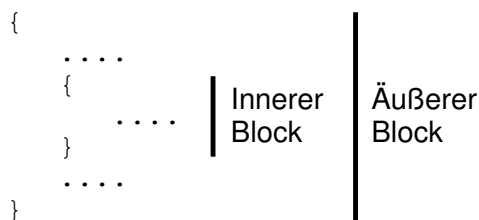
Beispielsweise:

```
{
  Anweisung1;
  Anweisung2;
}
```

Da eine Anweisung eines Blocks selbst wieder ein Block sein kann, können Blöcke geschachtelt werden.



Das folgende Bild symbolisiert die Schachtelung von Blöcken:



Blöcke treten im Programmcode an den folgenden Stellen auf:

- Der Rumpf einer Funktion ist ein Block.
- Da ein Block syntaktisch als eine einzige Anweisung gilt, kann er im Programmtext überall da stehen, wo von der Syntax her nur eine einzige Anweisung zugelassen ist, wie beispielsweise im if- oder else-Zweig einer if-Struktur. Darüber wurde bereits in Kapitel → 10.2 geschrieben.

- Ein Block kann auch einfach nur zum logischen Gliedern von Anweisungsfolgen dienen. Dies wird als „Sequenz“ bezeichnet. Diese Kontrollstruktur wurde bereits in Kapitel [→ 10.1](#) vorgestellt.
- Ein Block definiert einen Rahmen für die Gültigkeit, Sichtbarkeit und Lebensdauer von Bezeichnern. Er kann an bestimmten Stellen wie beispielsweise bei case-Marken dazu dienen, Variablendefinitionen zu verkapseln. Siehe dazu Kapitel [→ 10.3.4](#).

11.1.1 Vereinbarungen und Anweisungen innerhalb von Blöcken

Vereinbarungen umfassen Deklarationen und Definitionen von Variablen und Funktionen. Sie beschreiben die Bereitstellung von Variablen und Funktionen mittels eines Namens und unterscheiden sich somit grundsätzlich von tatsächlich ausführbaren Anweisungen. Siehe auch Kapitel [→ 7.4](#).

Bis zum Standard C90 hatten Blöcke den folgenden Aufbau:

```
{  
    Vereinbarung  
    Anweisungen  
}
```

Somit müssen in C90 die Vereinbarungen immer vor den Anweisungen stehen, sprich zuerst müssen alle Definitionen der Variablen aufgeführt werden, welche danach in Anweisungen benutzt werden. Seit dem C99-Standard existiert diese Einschränkung nicht mehr.

Nach C99 und C11 dürfen Vereinbarungen und Anweisungen gemischt werden. Eine Variable darf aber dennoch erst benutzt werden, nachdem sie vereinbart wurde.



Hierzu wurde von Stroustrup aus C++ das Konzept des „declaration statement“ (der Vereinbarungsanweisung) übernommen. Nach diesem Konzept werden Variablen erst angelegt, wenn man sie benötigt. Ein Vorteil dieses Konzepts ist, dass Variablen mit einer kürzeren Lebensdauer prinzipiell die Zahl der Fehler reduzieren. Es wird vermieden, dass Variablen am Anfang einer Funktion (eines Blockes) definiert werden müssen und anschließend immer wieder – auch für verschiedene Zwecke – gebraucht werden.

11.2 Lebensdauer, Gültigkeit und Sichtbarkeit von Variablen

In C können in jedem Block – auch in verschachtelten Blöcken – Vereinbarungen durchgeführt werden.



Generell können Definitionen und Deklarationen von Variablen an zwei Stellen vorgenommen werden:

- Außerhalb von Funktionen (extern)
- Innerhalb von Funktionen (intern) und Blöcken



Variablen, welche außerhalb von Funktionen definiert werden, heißen **externe Variablen**. Sie werden nach allgemeinem Bewusstsein auch „**globale Variablen**“ genannt. Variablen, die innerhalb einer Funktion definiert werden, heißen **interne Variablen** und werden auch als „**lokale Variablen**“ bezeichnet. Siehe dazu auch Kapitel [→ 7.4.2](#) und [→ 15.1.2](#).

Je nach Platzierung definiert eine Variable dabei eine unterschiedliche Lebensdauer, Gültigkeit und Sichtbarkeit:

Die **Lebensdauer** einer Variablen ist die Zeitspanne, in der das Laufzeitsystem des Compilers für die Variable einen Platz im Speicher reserviert hat. Mit anderen Worten, während ihrer Lebensdauer besitzt eine Variable einen Speicherplatz.



Der Speicherplatz für globale Variablen wird bereits vom Lader (siehe dazu Kapitel [→ 5.3](#)) zur Verfügung gestellt in dem sogenannten „Daten-Segment“ (siehe dazu Kapitel [→ 15.2](#)). Der Lader ermittelt, wieviele Bytes er für die Variablen benötigt und reserviert diesen Platz bis zum Ende des Programmes.

Der Speicherplatz für lokale Variablen wird auf dem Stack (siehe dazu auch Kapitel [→ 15.2](#)) angelegt. Der Speicherplatz von lokalen Variablen ist also verfügbar noch bevor sie innerhalb eines Blockes verwendet werden, sprich, bevor sie gültig werden.

Die **Gültigkeit** einer Variablen bedeutet, dass an einer Programmstelle der Name einer Variablen dem Compiler durch eine Vereinbarung bekannt ist.



Globale Variablen werden vor dem Aufruf der `main()`-Funktion initialisiert und sind ab der Stelle ihrer Definition gültig bis zum Ende des Programmes.

Lokale Variablen sind dem Compiler bekannt ab ihrer Definition, beziehungsweise ihrer Initialisierung innerhalb des zugehörigen Blockes. Ab diesem Punkt ist der Name dem entsprechenden Speicherplatz zugeordnet und bleibt bis zum Ende desselben Blockes verfügbar.

Beim Anlegen von Variablen sollte immer ein minimaler Gültigkeitsbereich angestrebt werden. Damit ist gemeint, dass der innerst-mögliche Block benutzt werden sollte.

Variablen sollten immer möglichst nahe an ihrer ersten Verwendung angelegt werden.



Ist eine Variable nur in einem Block von Nutzen, so sollte sie auch erst in diesem angelegt werden. So kann man sich beim Lesen des Programmtextes sicher sein, dass die Variable danach keine Rolle mehr spielt. Dies erhöht die Wart- und Lesbarkeit des Codes und macht die Programme weniger fehleranfällig.

Die **Sichtbarkeit** einer Variablen bedeutet, dass man von einer Programmstelle aus diese Variable sieht, das heißt, dass man auf sie über ihren Namen zugreifen kann.



Die in einem Block definierten Namen sind grundsätzlich innerhalb dieses Blockes und aller darin verschachtelter Blöcke sichtbar. In dem umfassenden Block sowie in allen Blöcken, welche nicht Teil der Verschachtelung sind, sind sie unsichtbar. Globale Variablen sind überall sichtbar.



Im folgenden Beispiel ist zu sehen, dass in den inneren Blöcken des „wahr“- und „falsch“-Zweiges der if-Anweisung lokale Variablen eingeführt werden:

sichtbar.c

```
#include <stdio.h>

int main(void) {
    int x;

    for (x = 0; x < 10; ++x) {
        if (x < 5) {
            int y = 2;
            printf("x * y hat den Wert %d\n", x * y);
        } else {
            int x = 1234;
            printf("Das innere x hat den Wert %d\n", x);
        }
    }

    printf("x hat den Wert %d\n", x);
    return 0;
}
```

Das Programm erzeugt folgenden Output:

```
x * y hat den Wert 0
x * y hat den Wert 2
x * y hat den Wert 4
x * y hat den Wert 6
x * y hat den Wert 8
Das innere x hat den Wert 1234
Das innere x hat den Wert 1234
Das innere x hat den Wert 1234
Das innere x hat den Wert 1234
Das innere x hat den Wert 1234
x hat den Wert 10
```

In diesem Beispiel sieht man zum einen, wie die Variable `y` lokal definiert wird. Man sieht aber auch, dass eine „neue“ Variable `x` in einem inneren Block definiert werden kann, obschon bereits eine Variable mit demselben Namen außerhalb definiert wird. In diesem Falle wird die äußere Variable von der inneren „verdeckt“. Diese Verdeckung des Namens wird im Englischen auch als „**shadowing**“ bezeichnet.

Eine Variable kann gültig sein und von einer Variablen des-
selben Namens verdeckt werden und deshalb nicht sichtbar
sein.



Auf eine verdeckte Variable kann nur über ihre Adresse zugegriffen werden,
nicht aber über ihren Namen.

Lokale Variablen können sogar nicht nur die Namen von Vari-
ablen, sondern auch die Namen von Funktionen verdecken.



Dies ist im folgenden Beispiel zu sehen:

quadrat.c

```
#include <stdio.h>

double quadrat(double n) {
    return n * n;
}

int main(void) {
    int quadrat;
    printf("%d", quadrat(5));
    return 0;
}
```

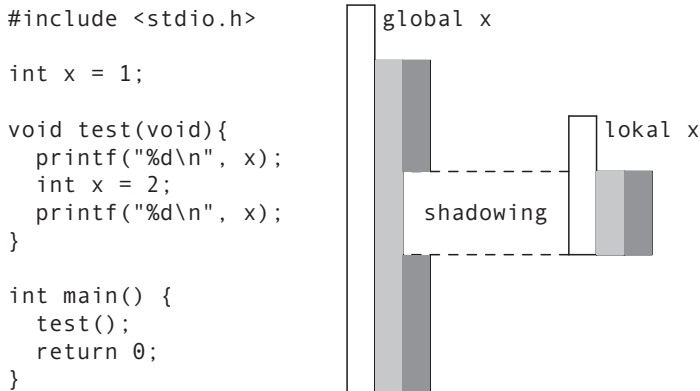
Hier wird durch die lokale Variable `quadrat` die Funktion `quadrat()` verborgen.
Folglich meldet der Compiler einen Fehler, dass `quadrat()` keine Funktion ist
und daher ein Funktionsaufruf im Hauptprogramm nicht zulässig ist.

Die folgende Tabelle fasst die Lebensdauer, die Sichtbarkeit und die Gültigkeits-
bereiche für lokale und globale Variablen zusammen:

	lokal (intern)	global (extern)
Lebensdauer	Block	Programm
Gültigkeitsbereich	ab Vereinbarung	ab Vereinbarung
Sichtbarkeit	Im Block, einschließlich der inneren Blöcke, ab Vereinbarung, solange nicht verdeckt.	Im Programm ab Verein- barung einschließlich der inneren Blöcke, solange nicht verdeckt.

Im folgenden Beispiel kann die Lebensdauer und Sichtbarkeit von Variablen durch die gezeichneten Balken beobachtet werden. Der weiße Balken zeigt die Lebensdauer, der hellgraue Balken die Gültigkeit und der dunkelgraue Balken die Sichtbarkeit.

Die linken Balken zeigen die Eigenschaften der globalen Variablen `x`. Die Variable lebt solange wie das Programm, ist gültig ab der Stelle ihrer Definition und grundsätzlich auch dann sichtbar, wenn sie nicht zwischenzeitlich durch die lokale Variable `x` verdeckt werden würde. Die Balken rechts zeigen die Eigenschaften der lokalen Variablen `x`. Sie lebt nur solange, wie die Funktion `test()` lebt, das heißt, während ihrer Abarbeitung. Gültig und sichtbar ist sie nur innerhalb der Funktion `test()` ab der Stelle ihrer Definition. Während der Gültigkeitsdauer der lokalen Variablen wird die globale Variable verdeckt.



Die Ausgabe dieses Programmes lautet:

```
1
2
```

11.3 Definition von Funktionen

Die Definition einer Funktion besteht in C aus dem Funktionskopf und dem Funktionsrumpf. Der Funktionskopf legt die Aufruf-Schnittstelle der Funktion fest. Der Funktionsrumpf enthält lokale Vereinbarungen und die Anweisungen der Funktion.



Die Hauptaufgabe einer Funktion ist es grundsätzlich, aus Eingabedaten Ausgabedaten zu erzeugen. Das bedeutet, dass eine Funktion Daten empfangen und am Ende einen Wert mittels **return** zurückgeben kann.

Während der Abarbeitung einer Funktion können jedoch auch beliebig andere Nebeneffekte auftreten wie beispielsweise:

- Änderungen an Variablen, deren Adressen an die Funktion über die Parameterliste übergeben wurden (siehe Kapitel [→ 11.5.1](#)).
- Änderungen an Werten globaler Variablen (siehe Kapitel [→ 7.4.3](#)).
- Systemaufrufe wie beispielsweise die Ausgabe auf dem Bildschirm oder das Öffnen und Schreiben einer Datei. Diese Nebeneffekte werden hier nicht weiter erläutert.

Wie eine Funktion genau definiert wird, wird in den folgenden Unterkapiteln erklärt.

11.3.1 Funktionskopf und Funktionsrumpf

Folgendes ist die allgemeine Syntax einer Funktionsdefinition:

```
Rueckgabetyf Funktionsname(Parameterliste) {  
    Anweisungen  
}
```

Eine Funktion hat einen Rückgabetyf, einen Namen und eine (möglicherweise leere) Parameterliste. Dies wird als der „**Funktionskopf**“ bezeichnet. Man spricht auch von der „**Signatur**“ oder der „**Aufrufsstelle**“ einer Funktion.

Innerhalb der geschweiften Klammern stehen die Anweisungen, welche ausgeführt werden sollen. Dies wird als der „**Funktionsrumpf**“ bezeichnet.

Der Funktionsname ist der Name, mit welchem die Funktion an anderer Stelle im Code aufgerufen werden kann. Für Funktionsnamen gelten dieselben Regeln für die Sichtbarkeit wie für Variablennamen: Sie sind innerhalb einer Datei verfügbar ab dem Punkt ihrer Definition, siehe dazu Kapitel [→ 11.2](#) .

Es ist möglich, Funktionen eines Programms auf verschiedene Dateien zu verteilen. Eine Funktion muss dabei jedoch stets am Stück in einer einzigen Datei enthalten sein.



Vorerst werden nur Programme betrachtet, die aus einer einzigen Datei bestehen. In Kapitel [→ 22](#) werden Programme, die aus mehreren Dateien bestehen, behandelt.

Der **Rückgabetyf** ist der Typ des Wertes, welcher von der Funktion schlussendlich mittels der return-Anweisung zurückgegeben wird. Siehe dazu auch Kapitel [→ 11.4](#) .

Wird der Typ **void** als Rückgabetyf angegeben, so kann zwar mit return die Funktion verlassen werden, die Rückgabe eines Wertes ist dabei aber nicht möglich. Für jeden anderen Typ muss immer ein Wert mit return zurückgegeben werden.



Funktionen mit Rückgabewert `void` werden auch als „**Prozedur**“ bezeichnet.

Diese `void`-Funktionen (Prozeduren also) werden eingesetzt, wenn eine Funktion nicht explizit einen Wert berechnet, sondern beispielsweise lediglich auf dem Bildschirm Ausgaben durchführt, oder aber Daten verändert, welche über Parameter übergeben wurden.

Der Umgang mit dem Rückgabetyt ist je nach Anwendung unterschiedlich. Während in manchen Funktionssammlungen sozusagen alle Funktionen den Rückgabetyt `void` besitzen, definieren andere Funktionssammlungen grundsätzlich zu jeder Funktion einen Rückgabewert, welcher oftmals zum Weiterreichen von Fehlercodes verwendet wird. Die Entscheidung, wie mit dem Rückgabewert umgegangen wird, ist jedem freigestellt.

Wird der Rückgabetyt weggelassen, so wird als Default-Wert vom Compiler der Rückgabetyt `int` angenommen. Dies war insbesondere im C90-Standard gebräuchlich, heutige Compiler geben eine Warnung aus.



Die **Parameterliste** schlussendlich beschreibt die Eingabedaten für eine Funktion. Die Funktion wird an der aufrufenden Stelle mit aktuellen Parametern gefüllt, welche sodann innerhalb der Funktion als formale Parameter verfügbar sind. Im Folgenden wird unterschieden zwischen parameterlosen Funktionen und Funktionen mit Parametern:

11.3.2 Parameterlose Funktionen

Hat eine Funktion keine Übergabeparameter, so wird an den Funktionsnamen bei der Definition ein Paar runder Klammern angehängt, beispielsweise:

```
void printHallo() {  
    printf("Hallo");  
}
```

Bei der Definition einer Funktion wird dem Compiler damit mitgeteilt, dass diese Funktion keine Parameter erwartet.

Der Aufruf der Funktion erfolgt durch Anschreiben des Funktionsnamens, gefolgt von einem Paar runder Klammern:

```
printHallo();
```

Da die Funktion keine Parameter erwartet, darf man beim Aufruf auch keine hinschreiben. Ansonsten meldet der Compiler einen Fehler.

Im Gegensatz zur Definition muss gemäß dem Standard bei der Deklaration (dem Prototyp, siehe Kapitel [→ 11.6](#)) einer parameterlosen Funktion mit dem Typ `void` beschrieben werden. Ansonsten nimmt der Compiler an, dass die Funktionsdeklaration unvollständig ist und die Anzahl an zu erwarteten Parametern noch unbestimmt ist. Heutige Compiler warnen vor diesem Versäumnis.



```
void printHallo(void);
```

11.3.3 Funktionen mit Parametern

Eine Funktion mit Parametern wird definiert, indem man in die Klammern eine durch Komma getrennte Auflistung von **formalen Parametern** schreibt:

```
Rueckgabe-Typ Funktionsname(Typ1 formalerParameter1,
                             Typ2 formalerParameter2,
                             ...,
                             Typn formalerParameterN)
{
    Anweisungen
}
```

Jede Parameterdefinition besteht aus einem Typ und einem Parameternamen. Innerhalb des Funktionsrumpfes sind diese Parameter als Variablen verfügbar.



Ein formaler Parameter ist eine spezielle lokale Variable. Deshalb darf eine lokale Variable nicht denselben Namen wie ein formaler Parameter tragen.



Als Beispiel für eine Funktion mit Parametern wird die Funktion `printPLZ()` betrachtet, welche eine gegebene Postleitzahl auf dem Bildschirm ausgibt:

```
void printPLZ(int plz) {
    printf("Die Postleitzahl ist: ");
    printf("%05d\n", plz);
}
```

Beim Aufruf der Funktion muss man genau so viele Parameter mit den passenden Typen übergeben, wie die Funktionsdefinition vorschreibt. Der Aufruf für dieses Beispiel kann somit erfolgen mit:

```
printPLZ(meinePLZ);
```

Hier ist `meinePLZ` der **aktuelle Parameter**. Er ist die Postleitzahl, die auf dem Bildschirm ausgegeben werden soll.

Erlaubt ist, dass der Typ eines aktuellen Parameters verschieden ist vom Typ des formalen Parameters, wenn zwischen diesen Typen implizite Typwandlungen möglich sind. Diese impliziten Typumwandlungen finden dann beim Aufruf statt. Da implizite Typwandlungen oft nicht auf Anhieb verständlich sind, ist es immer besser, wenn der Typ des aktuellen mit dem Typ des formalen Parameters übereinstimmt. Das Regelwerk für die implizite Typumwandlung von Parametern (siehe Kapitel [→ 9.10](#)) ist dasselbe wie bei einer Zuweisung, da beim Aufruf tatsächlich eine Zuweisung des Werts des aktuellen Parameters an den formalen Parameter stattfindet.

Ein formaler Parameter wird bei Aufruf der Funktion mit dem Wert des entsprechenden aktuellen Parameters initialisiert. Anders gesagt, der Wert des aktuellen Parameters wird dem formalen Parameter zugewiesen und damit kopiert.



Im Falle des oben aufgeführten **Funktionsaufrufs** `printPLZ(meinePLZ)` wird beim Aufruf der Funktion eine lokale Kopie von `meinePLZ` im formalen Parameter `plz` angelegt. Die Funktion arbeitet nur mit der lokalen Kopie `plz` und kann den aktuellen Parameter `meinePLZ` nicht beeinflussen. Werden Parameter als Kopie ihres Werts übergeben, so nennt man das „call by value“. Mehr dazu wird in Kapitel [→ 11.5](#) beschrieben werden.

Der aktuelle Parameter braucht keine Variable zu sein. Er kann ein beliebiger Ausdruck sein.



```
printPLZ(73730);
```

Anstelle der Begriffe „formaler Parameter“ und „aktueller Parameter“ wird oft auch das Begriffspaar **Parameter** und **Argument** verwendet.



11.4 Rücksprung aus einer Funktion – die return-Anweisung

Die **return**-Anweisung beendet einen Funktionsaufruf. Das Programm kehrt zu der Anweisung, in der die Funktion aufgerufen wurde, zurück und beendet diese Anweisung.



Gibt eine Funktion mit `return` einen Wert zurück, so kann dieser Rückgabewert in Ausdrücken weiterverwendet werden. So kann der Rückgabewert in die Berechnung komplexer Ausdrücke einfließen, er kann einer Variablen zugewiesen werden oder an eine andere Funktion übergeben werden. Anschließend wird die nächste Anweisung nach dem Funktionsaufruf abgearbeitet.

Im folgenden Beispiel wird die Funktion `sin()` aufgerufen (Sinus, siehe Anhang → A.2) und das Resultat sofort in einem Ausdruck weiterverwendet:

```
hangAbtrieb = gewicht * sin(alpha);
```

Es ist nicht zwingend notwendig, dass der Rückgabewert einer Funktion abgeholt wird.



So liefert beispielsweise die Funktion `printf()` als Rückgabewert die Anzahl der ausgegebenen Zeichen. Dieser Rückgabewert wird meist nicht abgeholt, wie in folgendem Beispiel:

```
printf("Der Rueckgabewert wird hier nicht abgeholt");
```

Durch das Anhängen eines Strichpunktes wird der Ausdruck hier zu einer Ausdrucksanweisung. Siehe dazu auch Kapitel → 9.1 .

Funktionen, die keinen Rückgabewert haben, können nicht in die Berechnung von Ausdrücken einfließen. Sie können auch nicht auf der rechten Seite einer Zuweisung – eine Zuweisung stellt auch einen Ausdruck dar – verwendet werden. Sie können nur als Ausdrucksanweisung angeschrieben werden.

Eine Funktion, welche keinen Resultatwert liefern soll, wird mit dem Rückgabebetyp `void` definiert. Wann immer aus einer solchen Funktion zurückgekehrt werden soll, kann folgende einfache Anweisung geschrieben werden:

```
return;
```

Enthält ein Funktionsrumpf einer Funktion mit dem Rückgabebetyp `void` keine `return`-Anweisung, so wird die Funktion beim Erreichen der den Funktionsrumpf abschließenden geschweiften Klammer beendet, wobei kein Ergebnis an den Aufrufer zurückgeliefert wird.



Hat eine Funktion einen von `void` verschiedenen Rückgabebetyp, so muss sie mit `return` einen Wert zurückgeben. Nach `return` kann ein beliebiger Ausdruck stehen:

```
return Ausdruck;
```

Mit Hilfe der `return`-Anweisung ist es möglich, den Wert eines Ausdrucks, der in der Funktion berechnet wird (das Funktionsergebnis), an den Aufrufer der Funktion zurückzugeben.



Wenn der Typ von dem Ausdruck nicht mit dem Resultat-Typ der Funktion übereinstimmt, versucht der Compiler, eine implizite Typumwandlung durchzuführen. Das Regelwerk für die implizite Typumwandlung (siehe Kapitel [→ 9.10](#)) ist dasselbe wie bei einer Zuweisung. Ist die implizite Typumwandlung nicht möglich, resultiert eine Fehlermeldung.

11.5 Konventionen bei der Parameterübergabe

Funktionen definieren durch ihren Funktionskopf eine Parameterliste. An der aufrufenden Stelle werden passende aktuelle Parameter (Argumente) an die Funktion übergeben.

In C gibt es grundsätzlich nur eine Art, wie Parameter übergeben werden, das sogenannte „call by value“:

Bei einem **call by value** wird der Wert eines aktuellen Parameters an eine Funktion als Kopie übergeben. Dabei kann man den aktuellen Parameter von der aufgerufenen Funktion aus nicht abändern, da die aufgerufene Funktion mit einer Kopie des aktuellen Parameters arbeitet.



Somit hat eine Funktion keine Möglichkeit, Werte außerhalb der Funktion zu verändern. Tatsächlich möchte man jedoch häufig auch Änderungen außerhalb der Funktion bewirken. Hierfür gibt es grundsätzlich drei Möglichkeiten:

- Die Verwendung von globalen Variablen
- call by reference
- call by pointer

Die Möglichkeiten, wie mit globalen Variablen Werte verändert werden können, wurden bereits in Kapitel [→ 7.4.3](#) behandelt.

call by reference ist eine Möglichkeit, welche in C nicht existiert, aber beispielsweise in C++. Mit call by reference ist es möglich, über Übergabeparameter nicht nur Werte in eine Funktion hinein, sondern auch aus ihr herauszubringen. Ein solcher Parameter wird dann „Referenzparameter“ genannt.

Ein Aliasname ist einfach ein zweiter Name für dieselbe Variable. Eine Variable kann sowohl über ihren eigentlichen als auch über ihren Aliasnamen angesprochen werden. Somit kann eine Funktion den Aliasnamen in ihrem lokalen Block verwenden und dennoch die tatsächliche Variable außerhalb der Funktion verändern.

Um in C ähnliche Mechanismen zu verwenden, muss auf Pointer zurückgegriffen werden.

11.5.1 call by pointer

In C ist eine call by reference-Schnittstelle als Sprachmittel nicht vorgesehen. Man kann das Verhalten dieser Schnittstelle jedoch auch mit der call by value-Schnittstelle erreichen, indem man einen Pointer auf den aktuellen Parameter mit call by value übergibt. Dies wird als **call by pointer** bezeichnet.



Dies zeigt das folgende Beispiel:

ref.c

```
#include <stdio.h>

void init(int* alpha) {
    *alpha = 10;
}

int main(void) {
    int a;
    init(&a);
    printf("Der Wert von a ist %d\n", a);
    return 0;
}
```

Die Ausgabe am Bildschirm ist:

```
Der Wert von a ist 10
```

Also wird der Variablen a der Wert 10 zugewiesen.

Im Detail wird hier die Funktion `init(int* alpha)` aufgerufen durch `init(&a)`. Somit wird die lokale Variable `alpha` beim Aufruf angelegt und mit dem Wert des aktuellen Parameters initialisiert, hier also mit der Adresse von `a`. Man kann sich das als Kopiervorgang vorstellen:

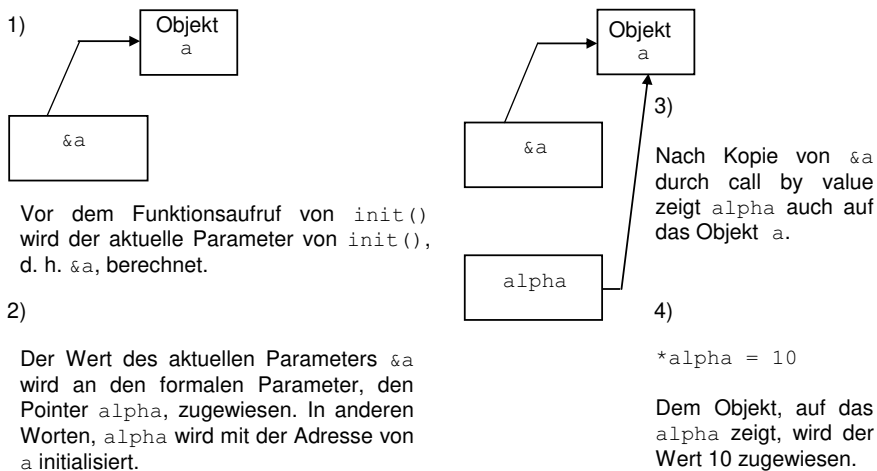
```
int* alpha = &a;
```

Damit steht im formalen Parameter `alpha` die Adresse der Variablen `a`. In der Anweisung innerhalb der Funktion wird sodann dem Objekt, auf das der Pointer `alpha` zeigt (`*alpha` eben), der Wert 10 zugewiesen.

```
*alpha = 10;
```

Da `alpha` schlussendlich jedoch auf dieselbe Speicherzelle zeigt wie `a`, ist somit der eigentliche Speicherinhalt der Variablen `a` verändert worden. Und so wurde innerhalb der Funktion eine Veränderung außerhalb der Funktion erwirkt.

Das folgende Bild symbolisiert die Zwischenschritte bei dem Funktionsaufruf nochmals:



11.5.2 const-safe-Programmierung

Wenn Funktionen auf Speicherbereiche außerhalb zugreifen dürfen, können viele Fehler passieren. Entsprechend hat man in der Programmiersprache C eine Schutzmaßnahme eingebaut:

Eine Funktion kann ein per Pointer übergebenes Objekt mittels des `const`-Schlüsselwortes vor Schreibzugriffen schützen. So kann im obigen Beispiel der Funktionskopf auch folgendermaßen definiert werden:

```
void init(const int* alpha)
```

Mit dieser Angabe kann man grundsätzlich davon ausgehen, dass die Variable `alpha` durch den Aufruf dieser Funktion nicht verändert werden wird. Die Anweisung `*alpha = 10;` innerhalb der Funktion wird bei diesem Funktionskopf vom Compiler als Fehler markiert.

Die Markierung eines Pointers mit `const` bewirkt, dass der Compiler den Inhalt des Objekts, auf das der Pointer zeigt, als unveränderbar annimmt. Der Pointer selbst kann verändert werden, der Wert des Objekts hingegen wird vom Compiler vor jeglichem Schreibzugriff geschützt.



Diese Art der Parameterübergabe wird als **const-safe-Programmierung** bezeichnet. Man geht dabei davon aus, dass innerhalb solcher Funktionen keine krummen Dinge mit den Pointern gedreht werden und somit die Daten vor Schreibzugriffen wirksam durch den Compiler geschützt sind.

const-safe-Programmierung ist beim erstmaligen Programmieren mit C oftmals sehr hinderlich, da der Compiler tatsächlich jegliche Schreibzugriffe verweigert. Für fortgeschrittene Programmierung bietet dies jedoch eine wertvolle Hilfe und führt zu einem guten Programmierstil. Es wird im Rahmen dieses Buches somit empfohlen, ein Programm erst allmählich const-safe zu gestalten und sich hierbei bei jedem Parameter zu überlegen: Braucht die Funktion auf das entsprechende Objekt einen Schreibzugriff?



Bei Funktionen, welche Pointer ohne `const` annehmen, muss beim Programmieren immer damit gerechnet werden, dass Nebeneffekte auftreten, welche möglicherweise nicht erwartet sind. Leider gibt es jedoch auch bei durchgängiger const-safe-Programmierung das Problem, dass in C mittels eines expliziten Casts (siehe Kapitel [→ 9.9.5](#)) ein Pointer beliebig in einen nicht-const-Pointer gewandelt werden kann. Deswegen gilt diese Art des Castens in der modernen Programmierung als verpönt.



Der const-Qualifikator wurde bereits in Kapitel [→ 7.5.1](#) besprochen als eine Möglichkeit, einen beliebigen Wert als „schreibgeschützt“ zu definieren.

Die übliche Schreibweise des const-Schlüsselwortes ist dabei auf der linken Seite des Typs:

```
const int;
```

Tatsächlich ist das const-Schlüsselwortes jedoch linksassoziativ, sprich, es wirkt auf das Element links davon. Korrekterweise müsste also Folgendes geschrieben werden:

```
int const;
```

Dass der Typ auch rechts von const stehen kann, ist eine Vereinfachung der Schreibweise (sogenannter „syntactic sugar“), welche durch den Compiler ermöglicht wird.

Bei einfachen Typen spielt dies keine Rolle, bei Pointer-Typen jedoch gibt es nun mehrere Möglichkeiten, das Schlüsselwort zu setzen:

```
const int *;  
int const *;  
int * const;
```

Die ersten beiden Zeilen sind äquivalent, denn const ist jeweils der Qualifikator für den int-Typ. Die dritte Zeile jedoch verändert die Bedeutung:

Während die Angabe `const int *` dasselbe bedeutet wie `int const *`, bedeutet `int * const` hingegen, dass der Pointer als const deklariert ist und nicht der int-Typ! Der Compiler verbietet somit zwar ein erneutes Zuweisen des Pointers, erlaubt jedoch, den Pointer zu dereferenzieren und den im referenzierten Objekt gespeicherten Wert zu überschreiben.



Auf weitere Eigenheiten der Platzierung des Schlüsselwortes const bei Pointern wird in Kapitel [→ 12.6](#) eingegangen.

11.6 Vorwärtsdeklaration von Funktionen

Im Rahmen der Standardisierung von C durch das ANSI-Komitee wurde festgelegt, dass die Konsistenz zwischen Funktionskopf und Funktionsaufrufen vom Compiler überprüft werden soll. Wenn der Compiler aber prüfen soll, ob eine Funktion richtig aufgerufen wird, dann muss ihm beim Aufruf der Funktion die Schnittstelle der Funktion, also der Funktionskopf, bereits bekannt sein.

In komplexeren Programmen wird man schnell in die Situation geraten, dass Funktionen sich gegenseitig aufrufen und somit keine Anordnung der Funktionen dem Compiler alle Schnittstellen vor den jeweiligen Aufrufen bereitstellen kann. Hierfür gibt es jedoch eine Lösung:

Steht die Definition einer Funktion im Programmcode erst nach ihrem Aufruf, so muss eine Vorwärtsdeklaration der Funktion erfolgen, indem vor dem Aufruf die Schnittstelle der Funktion deklariert wird.

Mit der **Vorwärtsdeklaration** (dem sogenannten **Funktions-Prototypen**) wird dem Compiler der Name der Funktion, der Typ ihres Rückgabewerts und der Aufbau ihrer Parameterliste bekannt gemacht. Stimmen die Vorwärtsdeklaration, der Aufruf der Funktion und die Definition der Funktion nicht überein, so resultiert eine Warnung des Compilers oder ein Compilerfehler.



Im folgenden Beispiel wird die Funktion `printQuadrat()` in der Funktion `main()` aufgerufen. Die Definition von `printQuadrat()` erfolgt jedoch im folgenden Programm erst nach dem Aufruf:

prototype.c

```
#include <stdio.h>

void printQuadrat(int x);           // Funktions-Prototyp

int main(void) {
    printQuadrat(5);               // Aufruf von printQuadrat()
    return 0;
}

void printQuadrat(int x) {         // Funktionsdefinition
    printf("%d\n", x * x);
}
```


Ein Funktions-Prototyp entspricht vom Aufbau her einem Funktionskopf. Dabei sind aber die folgenden Abweichungen zur Struktur des Funktionskopfes zugelassen:

- Der Name eines Parameters im Prototyp muss nicht mit dem Namen des entsprechenden formalen Parameters im Funktionskopf übereinstimmen.
- Der Name eines formalen Parameters kann im Prototyp auch weggelassen werden. Entscheidend aber ist, dass der Typ jedes formalen Parameters angegeben wird.
- Im Gegensatz zu einem Funktionskopf wird ein Funktions-Prototyp mit einem Semikolon abgeschlossen.

So könnte im obigen Beispiel `prototype.c` der Funktions-Prototyp auch wie folgt geschrieben werden:

```
void printQuadrat(int);  
void printQuadrat(int zahl);
```

Der Rückgabetyt sowie die Anzahl, die Datentypen und die Reihenfolge der formalen Parameter müssen identisch sein zwischen Prototyp und Funktionskopf.



Wird bei einem Prototyp oder bei einer Funktionsdefinition kein Rückgabetyt angegeben (also auch nicht `void`), dann setzt der Compiler in C90 automatisch den Rückgabetyt `int` für diese Funktion ein. In C99 und C11 ist diese implizite Typenerweiterung nicht gestattet. Viele Compiler tun es aus Kompatibilitätsgründen jedoch trotzdem, geben aber eine entsprechende Warnung aus.



11.6.1 Header-Dateien von Bibliotheken

Die Regeln von Funktions-Prototypen gelten auch für Bibliotheksfunktionen. Will man also Bibliotheksfunktionen aufrufen, so müssen ihre Prototypen bekannt sein. Diese befinden sich – neben Makros und Konstanten – in den Header-Dateien.

Durch das Einbinden der **Header-Datei** einer **Bibliothek** werden die Funktions-Prototypen der Bibliotheksfunktionen eingefügt. Dadurch werden die Parameterliste und der Rückgabebetyp jeder Bibliotheksfunktion dem Compiler bekannt gemacht.



So war bisher in den meisten Beispielen die Zeile `#include <stdio.h>` vorhanden. Der Präprozessor (siehe Kapitel [→ 21](#)), der diese Zeile bearbeitet, sorgt dafür, dass an dieser Stelle der Inhalt der Datei `stdio.h` in den Quellcode eingebunden und übersetzt wird, in der unter anderem die Prototypen der Funktionen für die Ein- und Ausgabe stehen. Dadurch kennt dann der Compiler die Prototypen der Bibliotheksfunktionen, während er den anschließenden Programmtext übersetzt. Anhang [→ A](#) stellt die verschiedenen Klassen von Bibliotheksfunktionen des ISO-Standards vor.

Genau genommen sollte man zwischen den Header-Dateien und den entsprechenden Bibliotheken unterscheiden. Im alltäglichen Sprachgebrauch wird jedoch häufig das Einbinden einer Header-Dateien mit der Verlinkung der tatsächlichen Bibliothek gleichgestellt.

11.6.2 Eigene Header-Dateien

Bei einem Programm, das in mehrere Quelldateien aufgeteilt ist, muss man zur Übersetzung in jeder Datei, in der man eine Funktion außerhalb der Datei aufruft, einen Prototypen zur Verfügung stellen. Fügt man in jeder Datei einen eigenen Prototypen ein, dann kann leicht ein Wildwuchs entstehen. Der Übersetzer kann dann nicht gänzlich prüfen, ob alle Prototypen zur Funktionsdefinition passen, ob alle Funktions-Prototypen für die gleiche Funktion untereinander verträglich sind, etc. Aus Gründen der Projektorganisation ist es günstig, nur einen einzigen Prototypen für eine Funktion zu haben, der überall da benutzt wird, wo die Funktion aufgerufen wird.

Die folgenden Regeln haben sich als zweckmäßig herausgestellt und ermöglichen größtmögliche Konsistenz über Dateigrenzen hinweg:

1. Jede Funktion, die außerhalb der Datei benutzt werden soll, in der sie definiert ist, erhält einen Prototypen in einer Header-Datei.
2. Jede Quellcodedatei, die einen Aufruf einer Funktion aus einer anderen Datei enthält, inkludiert die entsprechende Header-Datei.
3. Die Quelldatei, die die Definition einer Funktion enthält, soll die Header-Datei ebenfalls inkludieren.

Regel 1 gewährleistet, dass nur ein einziger Funktions-Prototyp für eine Funktion existiert. Regel 2 erlaubt es dem Übersetzer, die aufrufende Funktion zu überprüfen, ob sie verträglich mit dem Prototypen ist. Regel 3 schließlich stellt sicher, dass der Compiler prüfen kann, ob der Prototyp zur Definition passt. Auf die Verwendung eigener Header-Dateien wird in Kapitel [→ 22](#) noch ausführlich eingegangen.

Dadurch, dass Header-Dateien somit von verschiedensten Dateien eingebunden werden, kann es passieren, dass ein und dieselbe Header-Datei innerhalb einer Übersetzungseinheit mehrfach eingebunden wird. Damit dies nicht passiert, behilft man sich in C mit einer bedingten Compilierung, welche in Kapitel [→ 22.3.4](#) nachgelesen werden kann.

11.7 Die Ellipse ... – variable Parameteranzahl

Die Programmiersprache C bietet neben den Funktionen mit fester Parameteranzahl auch eine Möglichkeit, Funktionen so zu definieren, dass eine beliebige Anzahl von Parametern übergeben werden kann. Die Kennzeichnung einer solchen Funktion erfolgt mit drei Punkten ... nach dem letzten formalen Parameter in der Parameterliste. Dies wird als „**Ellipse**“ oder „Auslassung“ bezeichnet. Dabei muss die Funktion mindestens einen explizit angegebenen Parameter enthalten.



Beim Aufruf muss die Anzahl der aktuellen Parameter mindestens so groß sein wie die Anzahl der explizit angeschriebenen formalen Parameter. Folgendes Beispiel zeigt die Definition einer Funktion mit **variabler Parameterliste**:

```
int varFunc(int zahl1, double zahl2, ...);
```

Beispiele zum Aufruf der Funktion varFunc() sind:

```
int z1 = 3;
double z2 = 5.4;

varFunc(z1, z2, "String"); // 1 zusätzlicher String
varFunc(z1, z2, 19, 27);  // 2 zusätzliche Integer-Werte
varFunc(z1, z2);          // keine zusätzlichen Parameter
varFunc(z1);              // !! Fehler: nur 1 Parameter !!
```

Da die Parameter des variablen Anteils nicht als feste formale Parameter definiert werden können, kann der Compiler für den variablen Anteil natürlich keine Typüberprüfung der aktuellen Übergabeparameter gegen die formalen Parameter durchführen.



Eine Funktion mit einer variabel langen Parameterliste muss irgendwie Zugriff auf die aktuellen Parameterwerte bekommen und zudem erfahren, wie viele Werte und von welchem Typ an sie übergeben wurden. Bei einer Ellipse fehlen diese Angaben.

C stellt daher ein Hilfsmittel zur Verfügung, nämlich Typen und Makros, die in der Datei `<stdarg.h>` definiert sind und mit denen dieser Zugriff auf die einzelnen Parameter möglich wird. Das folgende Beispiel berechnet den prozentualen Ausschuss einer Menge von Prüflingen mit Hilfe eines Schwellwertes:

ellipse.c

```
#include <stdio.h>
#include <stdarg.h>

const double SCHWELLE = 3.0;
const double ENDE = -1;

double qualitaet(double, ...);

int main(void) {
    printf("Der Ausschuss betraegt %5.2f %%\n",
        100 * qualitaet(SCHWELLE, 2.5, 3.1, 2.9, 3.2, ENDE));
    printf("Der Ausschuss betraegt %5.2f %%\n",
        100 * qualitaet(SCHWELLE, 4.2, 3.8, 3.4, 2.9, 2.7, ENDE));
    return 0;
}

double qualitaet(double schwellwert, ...) {
    int anzahlSchlechterTeile = 0;
    double wert;
    int i = 0;
    va_list listenposition;
    va_start(listenposition, schwellwert);
    wert = va_arg(listenposition, double);

    while (wert != ENDE) {
        if (wert > schwellwert) ++anzahlSchlechterTeile;
        ++i;
        wert = va_arg(listenposition, double);
    }

    va_end(listenposition);
    return (double)anzahlSchlechterTeile / i;
}
```

Die Ausgabe des Programmes ist:

```
Der Ausschuss betraegt 50.00 %
Der Ausschuss betraegt 60.00 %
```

Die Funktion `qualitaet()` in diesem Beispiel erhält als ersten Parameter den gewünschten Schwellwert. Die nächsten Parameter stellen die aktuellen Messwerte der Prüflinge dar, die mit dem Schwellwert zu vergleichen sind. Der Wert `ENDE`, der mit keinem gültigen Messwert übereinstimmen darf, zeigt das Ende der Messreihe an.

Der Zugriff auf die aktuellen Parameter erfolgt folgendermaßen: Als erstes wird an der Stelle (1) eine Variable `listenposition` definiert mit dem Typ `va_list`. Dann wird mit Hilfe von `va_start()` an der Stelle (2) diese Variable so initialisiert, dass sie auf den ersten variablen Parameter zeigt. Dazu wird als zweiten Parameter der Funktion `va_start()` der letzte feste Parameter der Parameterliste übergeben (hier die Variable `schwellwert`). Ab dem C23-Standard wird die Angabe des letzten festen Parameters überflüssig.

Die Variable `listenposition` wird anschließend von `va_arg()` benutzt an den Stellen (3) und (4). Der Aufruf von `va_arg()` liefert als Ergebniswert den Wert des aktuellen Parameters, auf den `listenposition` aktuell zeigt. Der Typ dieses Parameters wird als zweiter Parameter an `va_arg()` übergeben. Mit jedem weiteren Aufruf von `va_arg()` zeigt `listenposition` auf den nächsten Parameter. Der Abschluss ist erreicht, wenn `listenposition` auf `ENDE` zeigt.

Es ist zu beachten, dass es sich bei `va_start()` und `va_arg()` nicht um Funktionen handelt, sondern um Makros mit Parametern (siehe Kapitel [→ 21.2](#)). Dementsprechend ist es `va_start()` möglich, die Variable zu initialisieren und `va_arg()` kann einen Typen als Parameter entgegennehmen. So etwas ist mit Funktionen nicht möglich.

Schlussendlich wird an der Stelle (5) noch `va_end()` aufgerufen. Dies dient zur Freigabe des Speichers, auf den `listenposition` zeigt.

Wie genau Parameter und deren Typinformation mit der Ellipse übergeben werden können, ist nicht in der Sprache definiert. Obiges Beispiel geht davon aus, dass alle Parameter vom selben Typ sind. So kann man als letzten aktuellen Parameter einen Wert übergeben, der als Endekennung dient. Diese Endekennung wird auch „Wächter“, oder auf Englisch „sentinel“ genannt. Auch möglich ist das Übergeben der Anzahl an Parameter als einer der fixen Parameter. Dann wird kein Wächter mehr benötigt.

Ein anderes Beispiel, wie Typen ermittelt werden können, zeigt die Funktion `printf()`. Hier wird als erster (fixer) Parameter ein String übergeben, in welchem codiert ist, welche Typen für die darauffolgenden Parameter erwartet werden. Beispielsweise `%d` für einen `int`-Typ oder `%f` für einen `double`-Typ.

11.8 Iteration und Rekursion

Ein Algorithmus heißt **iterativ**, wenn bestimmte Abschnitte des Algorithmus innerhalb einer einzigen Ausführung des Algorithmus mehrfach durchlaufen werden.



Diese Art eines Algorithmus läuft grundsätzlich auf die Benutzung einer Schleife hinaus. Innerhalb einer `while`- oder `for`-Schleife wird ein Wert immer mehr verfeinert, bis er am Ende der Schleife das finale Resultat darstellt. Insofern ist eine Iteration also nichts neues.

Ein Algorithmus heißt **rekursiv**, wenn er Bereiche enthält, die sich selbst wiederum aufrufen.



Dies läuft in der Programmiersprache C grundsätzlich auf die Verwendung einer Funktion hinaus, welche sich selbst im Verlauf ihrer Abarbeitung wiederum aufruft. Wenn eine Funktion sich selbst aufruft, wird dies als „direkte Rekursion“ bezeichnet.

Es gibt aber auch eine „indirekte Rekursion“. Eine indirekte Rekursion liegt beispielsweise vor, wenn zwei oder mehr Funktionen sich wechselseitig, beziehungsweise im Kreis aufrufen. Da indirekte Rekursionen schnell unübersichtlich werden können, wird in der Praxis versucht, eine indirekte Rekursion zu vermeiden beziehungsweise durch eine direkte Rekursion zu ersetzen. In der Praxis tritt die indirekte Rekursion eher als unbeabsichtigter Programmierfehler auf.



Die Rekursion eignet sich, wie man noch sehen wird, zunächst einmal gut zum Umkehren von Reihenfolgen wie beispielsweise von 1, 2, 3, 4 in 4, 3, 2, 1. Zum anderen wird die Rekursion jedoch meist angewandt, um Wachstumsvorgänge (beispielsweise Zellwachstum in der Biologie) einfach zu modellieren oder um Lösungsalgorithmen von „Rätseln“, zum Beispiel dem Finden eines Weges im Labyrinth mit Backtracking, zu programmieren. Auch beim Implementieren von „teile und herrsche“-Algorithmen (auf Englisch „divide and conquer“) leistet die Rekursion wertvolle Dienste. Beispiele für die beiden Ansätze werden beim Suchen und beim Backtracking in Kapitel [→ 20](#) erklärt.

Bei rekursiven Algorithmen, die „nach einer Problemlösung suchen“, ist es bei einer schrittweisen „Suche“ nach solch einer Lösung erforderlich, nicht erfolgreiche Ansätze zu verwerfen und an einer vorherigen Stelle der Lösungssuche erneut fortzufahren. Diese Vorgehensweise führt zum Begriff „Backtracking“.

Backtracking ist bei einer baumartigen Lösungsuche die Rückkehr aus einer Sackgasse zu einer vorhergehenden Stelle im Lösungsbaum, von der aus ein erneuter Lösungsversuch gestartet werden soll.



Iteration und Rekursion sind Prinzipien, die oft als Alternativen für die Programmkonstruktion erscheinen. Theoretisch sind Iteration und Rekursion äquivalent, weil man jede Iteration in eine Rekursion umformen kann und umgekehrt. In der Praxis gibt es allerdings oftmals den Fall, dass die iterative oder rekursive Lösung auf der Hand liegt, dass man aber auf die dazu alternative rekursive beziehungsweise iterative Lösung nicht so leicht kommt.

Im Folgenden wird auf die Rekursion und Iteration eingegangen. Es werden zwei Algorithmen erklärt und beispielhaft Code für eine iterative und eine rekursive Lösung gezeigt und erklärt. Um jedoch Rekursion besser zu begreifen, muss zunächst der Begriff des „Stacks“ erläutert werden.

11.8.1 Die LIFO-Struktur eines Stacks

Als **Stack** wird ein Speicherbereich bezeichnet, auf dem Informationen temporär abgelegt werden können. Ein Stack wird auch als „Stapel“ bezeichnet. Ganz allgemein ist das Typische an einem Stack, dass auf die Information, die zuletzt abgelegt worden ist, als erstes wieder zugegriffen werden kann. Denken Sie beispielsweise an einen Bücherstapel. Sie beginnen mit dem ersten Buch, legen darauf das zweite, dann das dritte und so fort. In diesem Beispiel soll beim fünften Buch Schluss sein. Beim Abräumen nehmen Sie zuerst das fünfte Buch weg, dann das vierte, dann das dritte und so weiter, bis kein Buch mehr da ist. Bei einem Stack ist es nicht erlaubt, Elemente von unten oder aus der Mitte des Stacks wegzunehmen. Die Datenstruktur eines Stacks wird als LIFO-Datenstruktur bezeichnet. LIFO ist die Abkürzung für „Last-In-First-Out“, sprich das, was als Letztes abgelegt wird, wird wieder als Erstes entnommen. Das Ablegen eines Elementes auf dem Stack wird als `push()`-Operation, das Wegnehmen eines Elementes als `pop()`-Operation bezeichnet.

In der Sprache C werden bei Funktionsaufrufen die übergebenen Parameter, die lokalen Variablen und der Pointer auf die aktuelle Anweisung einer unterbrochenen Funktion (Rücksprungadresse) auf einem Stack abgelegt. Aus diesem Grund wird der Stack auch „**Call-Stack**“ genannt. Dieser wird von der Speicherverwaltung in einem eigenen Bereich, dem **Stack-Segment**, abgelegt, siehe Kapitel [→ 15.2.4](#).

Man muss sich nicht darum kümmern, sondern das Laufzeitsystem des Compilers erledigt diese Arbeit: Wenn eine Funktion aufgerufen wird, werden die zuvor genannten Objekte einfach auf den Stack gelegt. Wenn aus einer Funktion zurückgesprungen wird, werden die auf dem Stack abgelegten Objekte wieder entfernt.

Mehr Informationen über die verschiedenen Speicherbereiche können in Kapitel [→ 15](#) nachgelesen werden.

11.8.2 Iterative und rekursive Berechnung der Fakultätsfunktion

Das Prinzip der Iteration und der Rekursion von Funktionen soll an dem folgenden Beispiel der Berechnung der Fakultätsfunktion veranschaulicht werden.

Iterativ ist die Fakultätsfunktion definiert durch:

$$n! = 1 \cdot 2 \cdot \dots \cdot n \quad \text{Oder umgekehrt:} \quad n! = n \cdot (n-1) \cdot (n-2) \cdot \dots \cdot 1$$

Dieser Algorithmus lässt sich leicht programmieren:

fakultaetIterativ.c

```
#include <stdio.h>

int main(void) {
    int n = 5;
    printf("Fakultaet(%d): ", n);

    int faku = 1;
    while (n > 1) {
        faku = faku * n;
        --n;
    }

    printf("%d\n", faku);
    return 0;
}
```

Folgendes gibt das Programm aus:

Fakultaet(5): 120

Der Algorithmus arbeitet wie folgt: Zuerst wird die Resultats-Variable faku mit 1 initialisiert. Man beachte: Eine Zahl multipliziert mit 1 ergibt die Zahl selbst. Sodann wird die while-Schleife solange durchlaufen, wie n größer ist als 1. Innerhalb der Schleife wird die Resultats-Variable mit dem aktuellen Wert von n multipliziert und der Wert von n danach um 1 verringert.

Um den Algorithmus **rekursiv** anzugehen, benötigt man eine alternative Beschreibung des Problems, bestehend aus einer Schritt-Anweisung und einer Stopp-Anweisung:

$$n! = n \cdot (n-1)!$$

$$1! = 1$$

Damit gilt:

$$4! = 4 \cdot 3!$$

$$3! = 3 \cdot 2!$$

$$2! = 2 \cdot 1!$$

$$1! = 1$$

Das bedeutet, man schaut jetzt auf eine Funktion $f(n)$ und versucht, diese Funktion durch sich selbst – aber mit anderen Aufrufparametern – darzustellen. Die mathematische Analyse ist im Beispiel der Fakultätsfunktion ziemlich leicht, denn man sieht sofort:

$$f(n) = n \cdot f(n-1)$$

Damit hat man das fundamentale Rekursionsprinzip für dieses Problem bereits gefunden. Dies ist die Schritt-Anweisung. Damit die Rekursion jedoch nicht ewig geht, braucht es eine Stopp-Anweisung. Dieses sogenannte „Abbruchkriterium“ wurde bereits oben erwähnt. Es heißt:

$$1! = 1$$

Der Algorithmus lässt sich nun leicht programmieren:

fakultaetRekursiv.c

```
#include <stdio.h>

int faku(int n) {
    printf("n: %d\n", n);
    if (n > 1) {
        return n * faku(n - 1);    // (1)
    } else {
        return 1;
    }
}

int main(void) {
    int n = 5;

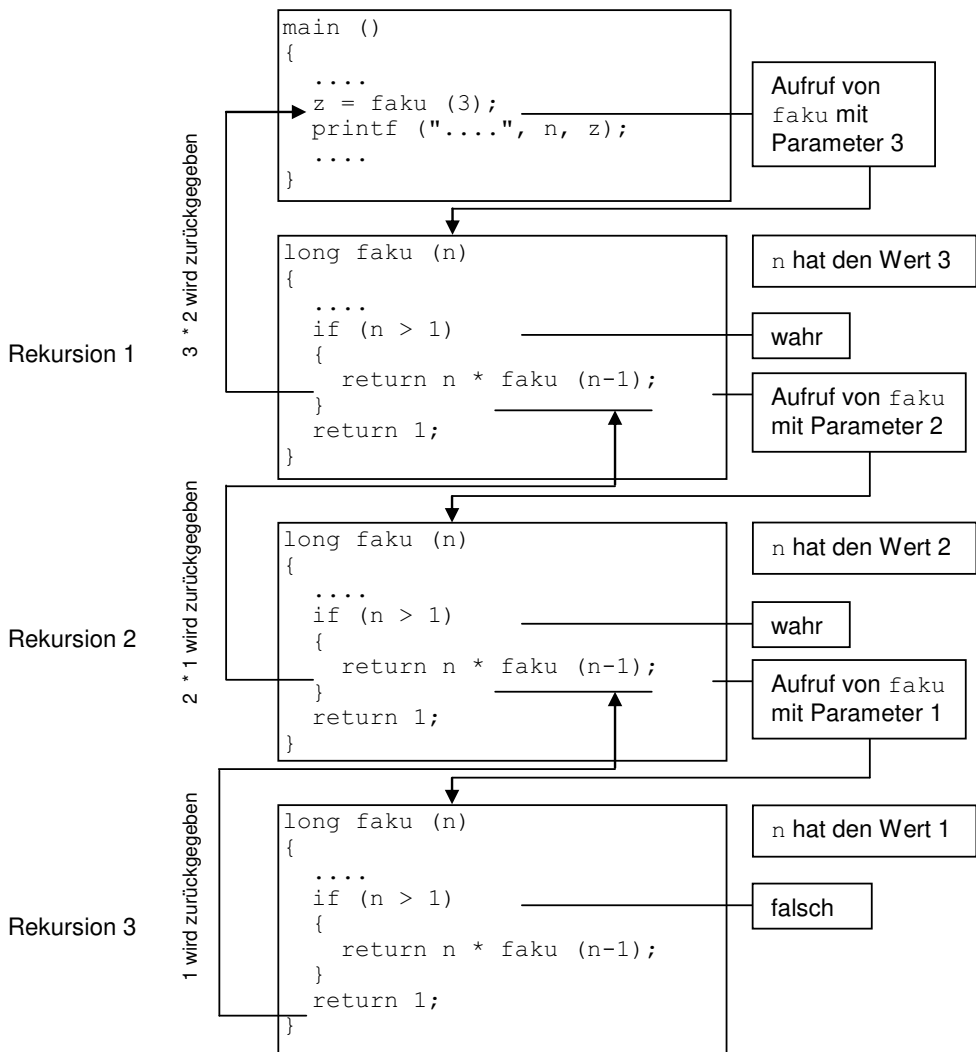
    printf("Fakultaet(%d) = %d\n", n, faku(n));
    return 0;
}
```

Das Programm gibt aus:

```
n: 5
n: 4
n: 3
n: 2
n: 1
Fakultaet(5) = 120
```

Die Funktion `faku()` enthält eine Abfrage, in welcher zwischen der Schritt- und der Stopp-Anweisung unterschieden wird. Ist der gegebene Parameter `n` größer als 1, so wird die Rekursion aufgerufen und das Resultat der Funktion zurückgegeben. Ist sie jedoch gleich 1, so wird einfach nur 1 zurückgegeben.

Die folgende Skizze veranschaulicht am Beispiel der Berechnung von `faku(3)` den rekursiven Aufruf von `faku()` bis zum Erreichen des Abbruchkriteriums und die Beendigung aller wartenden `faku()`-Funktionen nach Erreichen des Abbruchkriteriums:



Wird `faku()` von der `main()`-Funktion mit dem Wert 3 aufgerufen, so wird auf dem Stack die lokale Variable `n` mit dem Wert 3 angelegt und `faku(3)` wird abgearbeitet, bis zur Zeile (1). An dieser Stelle wird `faku()` mit dem neuen Wert `n - 1`, also 2, erneut aufgerufen. `faku(3)` wird aber noch nicht beendet, sondern wartet auf den Rückgabewert von `faku(2)`. Um bei dem erneuten Aufruf von `faku()` den lokalen Parameter `n` nicht zu überschreiben, wird somit eine neue lokale Variable `n` mit dem Wert 2 auf dem Stack abgelegt. Damit das Programm zudem nach Beendigung von `faku(2)` wieder an die richtige Stelle zurückspringen kann, wird die Rücksprungadresse ebenfalls auf dem Stack abgelegt.

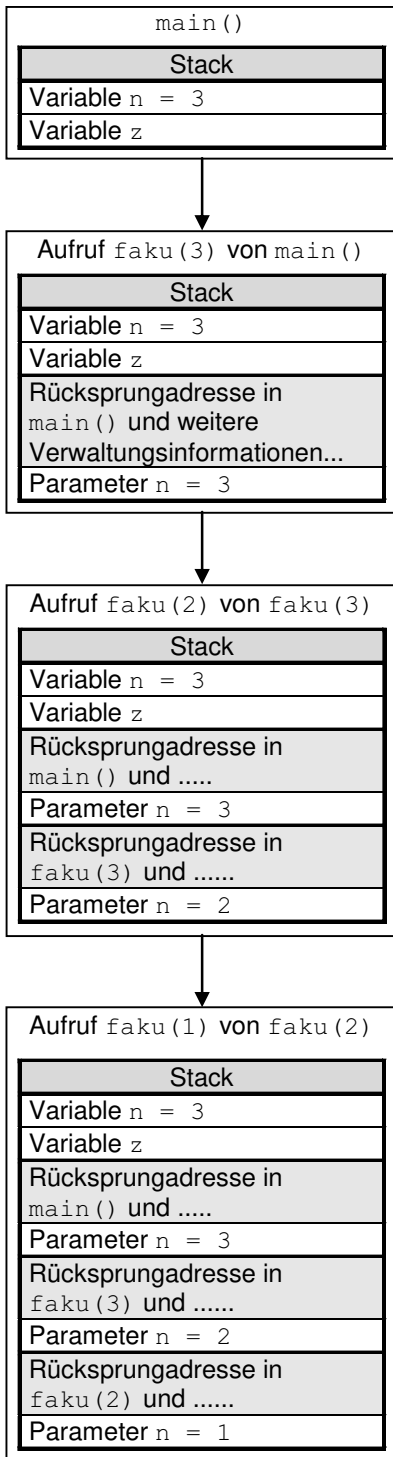
faku(2) wird sodann wieder bis zur Stelle (1) abgearbeitet, an der nun der Funktionsaufruf faku(1) stattfindet. Die lokale Variable *n* von faku(1) und die Rücksprungadresse werden erneut auf dem Stack abgelegt und faku(1) aufgerufen. faku(1) beendet nun die Rekursion, da hier die Bedingung falsch ist, und gibt somit direkt den Wert 1 an faku(2) zurück. Bei diesem Rücksprung an die auf dem Stack befindliche Rücksprungadresse wird der Stack abgebaut und somit auch die lokale Variable *n* von faku(1) entfernt. Damit bezeichnet *n* nun wieder die lokale Variable von faku(2). Nun kann faku(2) die Berechnung von $n * \text{faku}(1)$ durchführen und somit den Wert $2 * 1$ an faku(3) zurückgeben. Wieder werden die lokale Variable *n* von faku(2) und die Rücksprungadresse abgebaut. faku(3) kann jetzt die Berechnung $n * \text{faku}(2)$ durchführen und den Wert $3 * 2$ an *z* in der *main()*-Funktion zurückgeben. Damit sind alle von der Funktion faku() auf den Stack abgelegten Daten wieder abgeholt und alle Aufrufe beendet.

Das Ablegen auf dem Stack und das Aufräumen desselben wird auf den folgenden beiden Seiten in Diagrammen verdeutlicht.

Eine zu hohe Zahl von rekursiven Aufrufen führt zum Überlauf des Stacks.



Auch wenn es nicht zum Stacküberlauf kommen sollte, so ist dennoch zu berücksichtigen, dass die Rekursion mehr Speicherplatz und Rechenzeit erfordert als die entsprechende iterative Formulierung. Wenn man den zu einem rekursiven Algorithmus entsprechenden iterativen Algorithmus kennt, so ist dem iterativen Algorithmus üblicherweise der Vorzug zu geben.

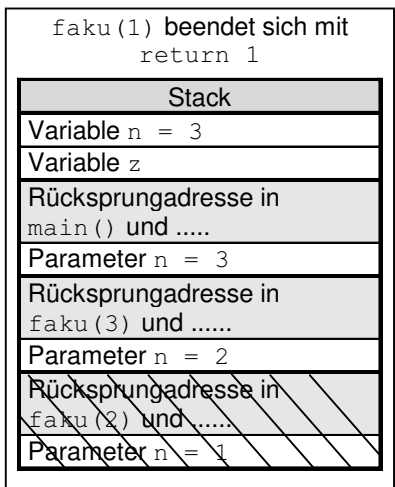


Aufbau des Stacks für `faku(3)`:

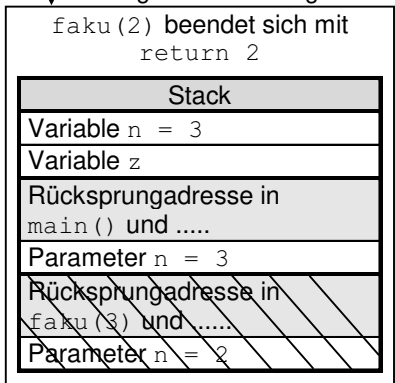
Bei jedem Aufruf von `faku()` werden die Rücksprungadresse und weitere Verwaltungsinformationen auf einem Stack abgelegt, der durch das Laufzeitsystem verwaltet wird. Auch die übergebenen Parameter (hier nur einer) werden auf diesem Stack abgelegt. Dabei wächst der Stack mit der Rekursionstiefe der Funktion.

Der letzte Aufruf von `faku()` mit dem Parameter `n` = 1 bewirkt keine weitere Rekursion, da ja die Abbruchbedingung erfüllt ist.

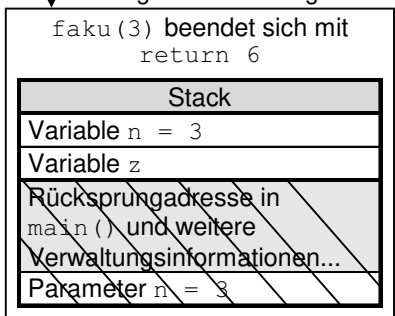
Der Abbau des Stacks geschieht in umgekehrter Reihenfolge, wie aus dem folgenden Bild ersichtlich wird.



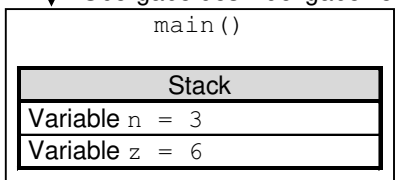
↓ Übergabe des Rückgabewertes über Register



↓ Übergabe des Rückgabewertes über Register



↓ Übergabe des Rückgabewertes über Register



Abbau des Stacks für faku(3):

Beim Beenden der aufgerufenen Funktion werden auf dem Stack die lokalen Variablen (inklusive der formalen Parameter) freigegeben und die Rücksprungadresse und sonstigen Verwaltungsinformationen abgeholt. Der Rückgabewert wird in diesem Beispiel über ein Register an die aufrufenden Funktionen zurückgegeben. Der Rückgabewert kann auf verschiedene Weise an die aufrufende Funktion zurückgegeben werden, beispielsweise auch über den Stack. Dies ist vom Compiler abhängig.

11.8.3 Beispiel für iterative und rekursive Berechnung der Binärdarstellung

Es soll die Binärdarstellung einer Zahl berechnet werden. Bei diesem Beispiel wird ebenfalls deutlich, welche Unterschiede zwischen einer iterativen und einer rekursiven Lösung bestehen und welche Vorteile die Rekursion bieten kann.

binaerIterativ.c

```
#include <stdio.h>

void binaerZahlIter(int zahl1) {
    int array[sizeof(int) * 8];
    int stelle;

    for (int i = 0; i < (sizeof(int) * 8); ++i) {
        array[i] = 0;
    }
    for (stelle = 0; zahl1 != 0; ++stelle) {
        array[stelle] = zahl1 % 2;
        zahl1 /= 2;
    }
    for (; stelle > 0; --stelle) {
        printf("%d ", array[stelle - 1]);
    }
}

int main(void) {
    int zahl = 35;
    binaerZahlIter(zahl);
    return 0;
}
```

Folgendes wird beim Programmablauf ausgegeben:

```
1 0 0 0 1 1
```

Das Programm `binaerit.c` berechnet aus einer gegebenen Dezimalzahl die zugehörige Binärzahl (siehe auch Anhang [→ C.1.2](#)). Dies geschieht durch Iteration. Wird eine Dezimalzahl durch 2 geteilt, so ist der Rest (also 0 oder 1) die letzte Stelle der zugehörigen Binärzahl. Teilt man nun das Ergebnis des Teilvorganges wieder durch 2, so ist der Rest die vorletzte Stelle der Binärzahl usw.

Die folgende Tabelle visualisiert das Umwandeln der Zahl 35 dezimal in eine Binärzahl mit Hilfe des Modulo-Operators:

Rechen- schritt	array						Ergebnis	Rest
	[5]	[4]	[3]	[2]	[1]	[0]		
35 / 2	0	0	0	0	0	1	17	1
17 / 2	0	0	0	0	1	1	8	1
8 / 2	0	0	0	0	1	1	4	0
4 / 2	0	0	0	0	1	1	2	0
2 / 2	0	0	0	0	1	1	1	0
1 / 2	1	0	0	0	1	1	0	1
Dualzahl 1 0 0 0 1 1								

Da nach Ablauf der ersten Schleife die letzte Stelle der Binärzahl an erster Stelle im Array array steht, muss dieses Array mit einer for-Schleife rückwärts ausgegeben werden. Deshalb wird der Schleifenzähler stelle erniedrigt, damit die erste Stelle der Binärzahl als letzte ausgegeben wird.

Da die Umkehr der Reihenfolge mit einer Rekursion einfacher auszuführen ist, jetzt zum Vergleich noch die rekursive Lösung:

binaerRekursiv.c

```
#include <stdio.h>

void binaerZahlReku(int zahl) {
    if (zahl > 0) {
        binaerZahlReku(zahl / 2);
        printf("%d ", zahl % 2);
    }
}

int main(void) {
    int zahl = 35;
    binaerZahlReku(zahl);
    return 0;
}
```

Und hier der Programmablauf:

```
1 0 0 0 1 1
```

Da die Rekursion hier vor der Ausgabe der Binärziffern durchgeführt wird, beginnt das Programm mit der Ausgabe einer Binärziffer erst, wenn die letzte Modulo-Operation durchgeführt wurde. Dies ist aber gerade die erste Stelle der Binärzahl. Dann werden rückwärts alle weiteren Stellen mit `zahl % 2` ausgegeben. Wie man sieht, ist die Realisierung einer „Reihenfolgeumkehr“ rekursiv mit weniger Aufwand zu realisieren als iterativ. Das liegt daran, dass der Programm-Stack als Zwischenspeicher zum Umkehren der Reihenfolge verwendet werden kann. Dies funktioniert im Gegensatz zur iterativen Lösung sogar ohne zu wissen, wie viele Binärziffern die Zahl letztendlich hat.

11.9 inline-Funktionen

Inline-Funktionen wurden mit C99 in C eingeführt. Einer Inline-Funktion muss das Schlüsselwort **inline** vorangestellt werden. Hier ein Beispiel:

```
inline double dot3(double* vec1, double* vec2) {  
    return vec1[0] * vec2[0]  
        + vec1[1] * vec2[1]  
        + vec1[2] * vec2[2];  
}
```

Inline-Funktionen definieren ganz normale Funktionen, können jedoch ähnlich wie Makros vom Compiler an der aufrufenden Stelle direkt in den Code hineinkopiert werden.



Inline-Funktionen haben gegenüber Makros den Vorteil, dass sie wie normale Funktionen mit aktuellen Parametern aufgerufen werden können. Der Compiler kann die Parameter wie auch den Rückgabewert prüfen.



Eine Inline-Funktion hat das Ziel, eine solche Funktion so schnell wie möglich zu machen. Es obliegt dem Compiler, ob er das Schlüsselwort `inline` akzeptiert. Ist die Inline-Funktion zu lang, so kann er einen normalen Unterprogrammaufruf daraus machen. In anderen Worten, Performance-Gewinn ist implementierungsabhängig.

Es ist sinnvoll, Inline-Funktionen anstelle von `define`-Makros einzusetzen. Der Programmcode einer Inline-Funktion wird vom Compiler an der Stelle des Aufrufs eingesetzt. Damit ist das Programm so schnell, als ob der Code direkt eingefügt worden wäre. Der aufwendige Aufruf eines Unterprogramms entfällt.



Da der Compiler den Code für das Kopieren und Einsetzen kennen muss, muss die Implementation einer Inline-Funktion mit interner Bindung definiert, sprich während der Compilation als Quellcode verfügbar sein. Siehe dazu auch Kapitel

→ 15.1.2

Es ist jedoch erlaubt, gleichzeitig zur internen `inline`-Funktion eine Definition mit externer Bindung bereitzustellen. Ist dies der Fall, so gilt die Implementation mit `inline` als Alternative zur externen Funktion. Falls die Implementation mit `inline` aus irgendwelchen Gründen nicht geeignet ist, wird der Compiler einen normalen Funktionsaufruf der externen Funktion ausführen. Wann genau welche Funktion aufgerufen wird, ist compilerspezifisch.

Da `inline`-Funktionen während der Compilation als Quellcode verfügbar sein müssen, werden sie häufig in Header-Dateien geschrieben. Da jedoch Header-Dateien oftmals in mehreren Übersetzungseinheiten gleichzeitig eingebunden werden, kann es passieren, dass am Ende in mehreren Übersetzungseinheiten ein und dieselbe `inline`-Funktion mehrfach definiert wird. Der Linker wird somit einen Fehler melden. Damit dies nicht passiert, behilft man sich bei `inline`-Funktionen in Header-Dateien mit dem Trick, die `inline`-Funktion zusätzlich als **`static`** zu vereinbaren (siehe Kapitel → 15.4). Dadurch wird gewährleistet, dass die Implementation der `inline`-Funktion nur innerhalb der aktuellen Übersetzungseinheit sichtbar sein wird. Der Nachteil hierbei ist, dass `inline`-Funktionen, welche aufgrund der Entscheidung des Compilers nicht in den Programmcode eingesetzt, sondern als Funktionsaufruf realisiert werden, in mehreren Objektdateien als exakte Kopie vorliegen. Da `inline`-Funktionen jedoch tendenziell eher klein sind, ist auch der dadurch entstehende Platzverbrauch vernachlässigbar und wird meistens in Kauf genommen.

11.10 Funktionen ohne Wiederkehr in C11



Es gibt einige wenige Funktionen, die nie zu ihrem Aufrufer zurückkehren, beispielsweise weil sie das Programm beenden. Dies lässt sich nun dem Compiler mit dem Schlüsselwort `_Noreturn` mitteilen, das mit C11 eingeführt wurde. Mit diesem Wissen ist es dem Compiler möglich, bessere Optimierungen beim Übersetzen durchzuführen.



Analysewerkzeuge können besser beim Debugging helfen, indem sie beispielsweise prüfen können, ob noch Code nach einer Funktion ohne Wiederkehr steht, der niemals erreicht werden kann.

Beispielsweise wird die `exit()`-Funktion (siehe Kapitel [→ 17.2.3](#)) folgendermaßen deklariert:

```
_Noreturn void exit(int status);
```

Alternativ lässt sich dafür auch die neue Bibliothek `<stdnoreturn.h>` verwenden. Die Deklaration sieht dann wie folgt aus:

```
#import <stdnoreturn.h>
noreturn void exit(int status);
```

Welche der beiden Notationen verwendet wird, ist Geschmackssache.

11.11 Übungsaufgaben

Aufgabe 1: Blöcke

Analysieren Sie das folgende Programm und sagen Sie voraus, welchen Wert x an den Stellen mit printf() haben wird. Starten Sie das Programm erst nach Ihrer Analyse.

blocks.c

```
#include <stdio.h>

int x = 5;

void function1(int* u) {
    int x = 4;
    *u = 6;
    printf("f1    - der Wert von x ist %d\n", x);
}

void function2(int x) {
    printf("f2    - der Wert von x ist %d\n", x);
}

int main(void) {
    printf("main - der Wert von x ist %d\n", x);
    function1(&x);
    function2(7);
    printf("main - der Wert von x ist %d\n", x);
    return 0;
}
```

Aufgabe 2: Funktionen

Schreiben Sie eine Funktion, welche den Ersatzwiderstand R einer Parallelschaltung aus zwei Widerständen R_1 und R_2 bestimmt. Die Funktion soll in der `main`-Funktion aufgerufen und dort das Resultat ausgegeben werden. Die Formel lautet:

$$1/R = 1/R_1 + 1/R_2$$

oder

$$R = (R_1 * R_2) / (R_1 + R_2)$$

Schreiben Sie die Funktion mehrmals mit unterschiedlicher Signatur:

- a) Die Funktion erwartet die Parameter R_1 und R_2 und gibt das Resultat als Rückgabewert zurück.
- b) Die Funktion erwartet die Parameter R_1 und R_2 und das Resultat mittels `call-by-pointer`.
- c) Die Funktion erwartet die Parameter R_1 und R_2 und nutzt eine globale Variable für das Resultat.
- d) Fortgeschritten:  Ermitteln Sie im Internet die korrekte Formel für den Ersatzwiderstand einer beliebigen Anzahl an Widerständen und schreiben Sie die Funktion mit einer Parameter-Ellipse, sodass bei Aufruf der Funktion die parallel geschalteten Widerstände einfach aufgelistet werden können.

Aufgabe 3: Rückgabe mit return und über die Parameterliste

Schreiben sie drei Funktionen, welche je ein Array von 10 Zahlen als Parameter annehmen und jeweils etwas unterschiedliches berechnen und als Rückgabewert zurückgeben: Die Funktion `summe()` berechnet die Summe der gegebenen Zahlen, die Funktion `maximum()` das Maximum der gegebenen Zahlen und die Funktion `durchschnitt()` den Durchschnitt der gegebenen Zahlen.

Schreiben Sie die Funktion für den Durchschnitt der Zahlen so, dass sie das Resultat der Summe nutzt.

Schreiben sie danach eine weitere Funktion `statistik()`, welche drei Pointer-Parameter `sum`, `max` und `avg` erwartet. Nach Aufruf dieser Funktion sollen alle passenden Werte in den entsprechenden Variablen vorzufinden sein.

Nutzen Sie folgendes Programmgerüst. Ergänzen Sie, wo immer nötig.

return.c

```
#include <stdio.h>

#define MAX 10

int summe(???) { ??? }

int maximum(???) { ??? }

double durchschnitt(???) { ??? }

void statistik(int* sum, int* max, double* avg, ???) { ??? }

int main(void) {
    int a[MAX] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

    int sum;
    int max;
    double avg;

    statistik(???);

    printf("Summe: %d\n", sum);
    printf("Maximum: %d\n", max);
    printf("Durchschnitt: %f\n", avg);

    return 0;
}
```

Aufgabe 4: Rekursion

Studieren Sie das folgende Programm:

recursion.c

```
#include <stdio.h>

void unbekannteFunktion(void) {
    int c = getchar();
    if (c != '\n') {
        unbekannteFunktion();
    }
    putchar(c);
}

int main(void) {
    unbekannteFunktion();
    return 0;
}
```

getchar und putchar werden in Kapitel [→ 16.8.1](#) und [→ 16.6.1](#) behandelt.

Welche Ausgabe erwarten Sie von dem Programm, wenn Sie 123 eintippen? Schreiben Sie das Ergebnis auf. Testen Sie anschließend das Programm.

Aufgabe 5: Potenzen iterativ und rekursiv berechnen

Die Potenz a^n soll für ein reellwertiges a und ein ganzzahliges positives n berechnet werden.

- Schreiben Sie eine Funktion, welche die Potenz mittels der `pow()`-Funktion berechnet. Siehe dazu Anhang [→ A.2](#).
- Berechnen Sie die Potenz mittels einer Schleife.
- Berechnen Sie die Potenz mittels einer Rekursion.

Alle drei Funktionen sollten dasselbe Resultat ergeben. Nehmen Sie die Funktion mit der `pow()`-Funktion als Referenz, sie gibt das korrekte Resultat zurück. Testen Sie Ihr Programm mit folgenden Eingabewerten:

5^2 2.5^8 1000^3 3^{1000} 7^1 1^7 1^{777777} 0^1 1^0 0^0

Wenn die Resultate der Funktionen nicht übereinstimmen, überlegen Sie sich wieso. Es könnte sogar sein, dass etwas gänzlich unerwartetes passiert.

12 Fortgeschrittene Programmierung mit Pointern



Pointer und **Arrays** wurden bereits in Kapitel → 8 beschrieben. Pointer sind jedoch in C ein so fundamentaler Bestandteil der Sprache, dass sie ein zusätzliches Kapitel erfordern.

Hier wird auf die fortgeschrittenen Eigenschaften von Pointern und Arrays eingegangen.

12.1 Gleichheit und Unterschiede von Arrays und Pointer

Wie aus Kapitel → 8 bereits bekannt ist, wird ein eindimensionales Array folgendermaßen definiert:

```
int alpha[5];    // Das Array alpha hat Platz fuer 5 int-Zahlen
```

Eine einfache Möglichkeit, einen Pointer auf ein Array-Element zeigen zu lassen, besteht darin, auf der rechten Seite des Zuweisungs-Operators den Adress-Operator & wie folgt auf ein Array-Element anzuwenden:

```
int* pointer = &alpha[i];
```

Hat *i* den Wert 1, so zeigt der Pointer *pointer* auf das Array-Element mit dem Index 1.

Dabei gibt es für das erste Element zwei gleichwertige Schreibweisen:

- `alpha[0]`
- `*alpha`

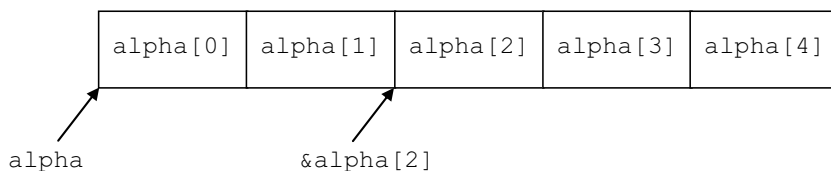
In der Sprache C wird ein **Arrayname** im Allgemeinen zu einem Pointer auf das erste Element des Arrays ausgewertet.



Ergänzende Information Die elektronische Version dieses Kapitels enthält Zusatzmaterial, auf das über folgenden Link zugegriffen werden kann https://doi.org/10.1007/978-3-658-45209-4_12.

© Der/die Autor(en), exklusiv lizenziert an
Springer Fachmedien Wiesbaden GmbH, ein Teil von Springer Nature 2024
J. Goll und T. Stamm, *C als erste Programmiersprache*,
https://doi.org/10.1007/978-3-658-45209-4_12

Das folgende Bild zeigt verschiedene Pointer auf ein eindimensionales Array:



Kapitel [→ 12.1.1](#) beschreibt die Ausnahmen, wann dies nicht der Fall ist.

Da ein Arrayname zu einem Pointer ausgewertet wird, ist es in C mit dem Vergleichs-Operator nicht möglich, zwei Arrays auf identischen Inhalt zu überprüfen, wie beispielsweise durch `arr1 == arr2`. Es wird nur verglichen, ob `arr1` und `arr2` auf dieselbe Adresse zeigen.

Für den Vergleich zweier eindimensionaler Arrays gibt es allerdings zwei Möglichkeiten. Die eine Möglichkeit ist die Überprüfung der einzelnen Array-Elemente in einer Schleife. Die andere, elegantere Möglichkeit wird mit der standardisierten Funktion `memcmp()` durchgeführt (siehe Kapitel [→ 12.8.3](#)). Die Bibliotheksfunktion `memcmp()` führt den byteweisen Vergleich einer Anzahl von Speicherstellen durch, die an über Parameter vorgegebenen Positionen im Adressraum des Speichers liegen.

12.1.1 Arrayname als nicht modifizierbarer L-Wert

Der Name eines Arrays bezeichnet grundsätzlich das (gesamte) Array im Speicher und somit einen L-Wert. Tatsächlich wird dieser L-Wert jedoch nur beim `sizeof`-Operator und dem Adress-Operator `&` sowie der Initialisierung mittels einer String-Konstanten so angesprochen. Bei allen anderen Operationen wird der Arrayname als die Adresse des ersten Elements ausgewertet, also einem R-Wert. Der Typ des Wertes entspricht einem Pointer auf den Komponenten-Typ des Arrays.



So ist es also möglich, einer entsprechenden Pointer-Variablen direkt die Adresse des ersten Elements zuzuweisen:

```
int alpha[5];  
int* pointer = alpha; // Pointer zeigt auf das erste Array-Element
```

Da es sich bei der Auswertung des Arraynamens um einen R-Wert handelt, sind Ausdrücke wie `alpha++` oder `alpha--` nicht erlaubt, da der Inkrement- und der Dekrement-Operator modifizierbare L-Werte voraussetzen. Ein Arrayname kann auch nicht auf der linken Seite einer Zuweisung stehen, da eine Zuweisung links vom Zuweisungs-Operator ebenfalls einen modifizierbaren L-Wert erfordert.

Einer Pointer-Variablen kann ein Wert zugewiesen werden, einem Arraynamen nicht.



Würde man den Adress-Operator `&` beim Arraynamen verwenden, so resultiert ein Pointer vom Typ `int (*) [5]`. Dies ist eine etwas verwirrende Typbeschreibung, liest sich jedoch wie ein geklammerter Ausdruck: Zuerst wird die Klammer ausgewertet, sprich, es ist ein Pointer. Dann wird der Rest des Typausdrucks ausgewertet, also ein Array mit 5 `int`-Werten. Zusammengesetzt entspricht dies einem „Pointer auf ein Array mit 5 `int` Elementen“.

Um eine Variable mit diesem Typ zu definieren, muss man also schreiben:

```
int (*pointerToArray)[5] = &alpha;
```

Wenn man nun diese Variable `pointerToArray` mit dem Dereferenzierungs-Operator auswertet, so ergibt sich wiederum das Array. Um nun zusätzlich noch den Pointer auf das erste Element zu erhalten, muss noch ein zusätzlicher Dereferenzierungs-Operator angefügt werden:

```
**pointerToArray = 5;
```

Diese Art der Adressierung und Dereferenzierung bietet insgesamt nicht viele Vorteile beim Programmieren. Sie dient hauptsächlich der Typ-Überprüfung bei der Parameterübergabe. Da das Schreiben der Typausdrücke verwirrend ist, wird entweder auf die Typ-Definition mittels `typedef` zurückgegriffen (siehe Kapitel [→ 14.2](#)), oder es wird auf die Typ-Überprüfung verzichtet, indem einfach nur der Pointer auf das erste Element übergeben wird.

12.2 Pointerarithmetik

Unter dem Begriff **Pointerarithmetik** fasst man die Menge der zulässigen Operationen mit Pointern zusammen. Folgende Operationen sind erlaubt:

- Zuweisungen mit Pointern
- Addition und Subtraktion bei Pointern
- Vergleiche von Pointern

Bevor die Möglichkeiten der Pointerarithmetik erläutert werden, lohnt sich ein erneutes Studium des in Kapitel [→ 8.1.4](#) vorgestellten NULL-Pointers.

12.2.1 Zuweisungen mit Pointern

Der Zuweisungs-Operator erlaubt es, Pointer-Variablen eine Adresse zuzuweisen. Die beiden Typen links und rechts des Zuweisungs-Operators sollten jedoch übereinstimmen.

Pointer unterschiedlicher Datentypen sollten einander nicht zugewiesen werden.



C erlaubt eine Zuweisung, allerdings geben moderne Compiler stets eine Warnung aus. C++ verbietet eine solche Zuweisung, es sei denn, es wird ein expliziter Cast verwendet.

Pointern vom Typ `void*` dürfen Pointer eines anderen Datentyps zugewiesen werden und Pointern eines beliebigen Datentyps dürfen Pointer vom Typ `void*` zugewiesen werden. Die Typüberprüfung des Compilers wird dabei aufgehoben.



C++ hingegen verbietet die Zuweisung eines **void-Pointers** an einen nicht-void-Pointer, es sei denn, es wird ein entsprechender Cast verwendet.

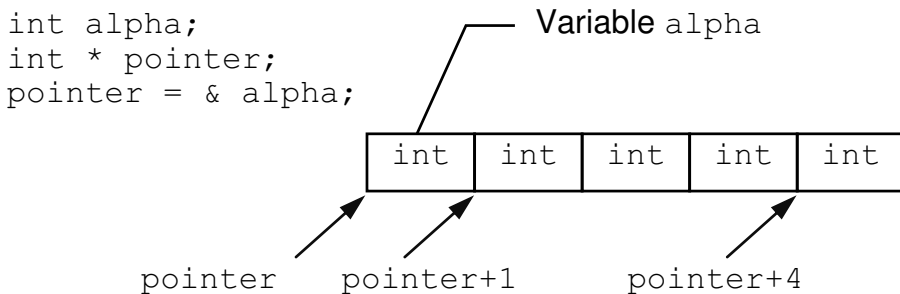
Der **NULL-Pointer** kann – da er gleichwertig zu `(void*)0` ist – ebenfalls jedem anderen Pointer zugewiesen werden.

12.2.2 Addition von Pointern mit einer Integer-Zahl

Zu einem Pointer kann ein Integer addiert oder von ihm abgezogen werden. Wird ein Pointer vom Typ `int*` um 1 erhöht, so zeigt er um ein `int`-Objekt weiter. Wird ein Pointer vom Typ `float*` um 1 erhöht, so zeigt er um ein `float`-Objekt weiter. Die Erhöhung um 1 bedeutet, dass der Pointer immer um ein Speicherobjekt vom Typ, auf den der Pointer zeigt, weiterläuft.



Das folgende Bild symbolisiert, dass die Pointerarithmetik in Längeneinheiten des Typs, auf den der Pointer zeigt, vorstatten geht:



Zeigt ein Pointer auf eine Variable des falschen Typs, so interpretiert der Dereferenzierungs-Operator den Inhalt der Speicherzelle, auf die der Pointer zeigt, gemäß dem Typ des Pointers und nicht gemäß dem Typ der Variablen, die an der Speicherstelle abgelegt wurde.



Die Anzahl Bytes, um welche ein Pointer bei der Erhöhung verschoben wird, wird vom Compiler mit dem `sizeof`-Operator berechnet. Der `sizeof`-Operator wird auf den referenzierten Typ angewandt. Im obigen Beispiel ist dies der Typ `int`. Ein Pointer kann aber auch auf einen zusammengesetzten Typ wie eine Struktur oder ein Array zeigen. Dementsprechend größer wird die Byte-Anzahl sein.

Genauso können Pointer erniedrigt werden. Generell gilt:

Ein Pointer, der auf ein Element in einem eindimensionalen Array zeigt, darf mit einem Integer (positiv wie negativ) addiert werden.



Zeigt der Pointer nach einer arithmetischen Operation nicht mehr in das Array, dann ist das Resultat eines Speicherzugriffs über diese Adresse undefiniert. Diese schwerwiegende Speicherverletzung wird als „Array-Out-of-Bounds“ bezeichnet.



Es ist durchaus möglich und manchmal auch beabsichtigt, dass ein Pointer auf eine Position außerhalb eines Arrays zeigt. Man darf nur mit dieser Adresse nicht auf den Speicher zugreifen. Das folgende Beispiel zeigt eine typische Art der Schleifenprogrammierung, bei der am Ende der Pointer auf das erste Element nach dem Array zeigt:

ptrAdd.c

```
#include <stdio.h>

int main(void) {
    int alpha[5] = {1, 2, 3, 4, 5};
    int* pointer = alpha;

    int i = 0;
    while (i < 5) {
        printf("%d\n", *pointer);
        ++pointer;
        ++i;
    }

    return 0;
}
```

Damit nimmt hier beispielsweise beim letzten Schleifendurchgang pointer den Wert `alpha + 5` an. Aber er wird nicht angesprochen, womit dieser Quellcode ungefährlich ist.

12.2.3 Subtraktion von zwei Pointern

Ist `pointer1` ein Pointer auf das Element mit Index `i` und `pointer2` ein Pointer auf das Element mit Index `j` eines eindimensionalen Arrays `eindimarray`, so gilt:

```
pointer1 == eindimarray + i
pointer2 == eindimarray + j.
```

Dies gilt, da der Name `eindimarray` zu einem Pointer ausgewertet wird, zu welchem sodann ein Integer hinzuaddiert wird.

Mit dieser Überlegung ist es auch möglich, zwei Pointer voneinander abzuziehen:

```
pointer2 - pointer1 == (eindimarray + j) - (eindimarray + i).
pointer2 - pointer1 == j - i.
```

Die zweite Zeile zeigt dabei, was passiert, wenn man den Ausdruck rechts kürzt. Übrig bleibt die Rechnung $j - i$, sprich die Anzahl an Elementen zwischen den Elementen mit Index `i` und `j`. Falls $j < i$, ist das Ergebnis negativ.

12.2.4 Vergleiche von Pointern

Zwei Pointer können auf Gleichheit beziehungsweise Ungleichheit verglichen werden, wenn beide Pointer denselben Typ haben oder einer der beiden der NULL-Pointer ist.

Wenn zwei Pointer auf dasselbe Speicherobjekt zeigen oder beide NULL sind, so ergibt der Test auf Gleichheit (`==`) den booleschen Wert „wahr“.



Für Pointer, die auf Elemente des gleichen eindimensionalen Arrays zeigen, kann aus dem Ergebnis der Vergleiche „größer“ (`>`) oder „kleiner“ (`<`) geschlossen werden, dass das eine Element „weiter vorne“ im Array liegt als das andere.



Das gleiche gilt für zwei Pointer, die auf Komponenten derselben Struktur zeigen. In allen anderen Fällen macht ein Vergleich keinen Sinn.

12.3 Initialisierung von Arrays

Die **Initialisierung** von Arrays kann automatisch oder manuell erfolgen.

Globale Arrays werden genauso wie einfache globale Variablen mit 0 initialisiert, das heißt alle Elemente eines globalen Arrays bekommen beim Start des Programmes automatisch den Wert 0 zugewiesen. Lokale Arrays werden nicht automatisch initialisiert.



Wenn der folgende Ausdruck somit im globalen Bereich steht, werden seine Werte automatisch auf 0 initialisiert. Steht er jedoch als eine lokale Definition innerhalb einer Funktion, so sind die Speicherinhalte unbestimmt.

```
int alpha[3];
```

Um ein Array manuell zu initialisieren, ist nach der eigentlichen Definition des Arrays ein Gleichheitszeichen gefolgt von einer Liste von Initialisierungswerten anzugeben.



Diese **Initialisierungsliste** enthält in geschweiften Klammern `{ }` die einzelnen Werte getrennt durch Kommas. Als Werte können Konstanten oder Ausdrücke aus Konstanten angegeben werden wie im folgenden Beispiel:

```
int alpha[3] = {1, 2 * 5, 3};
```

Diese Definition ist gleichwertig zu:

```
int alpha[3];  
alpha[0] = 1;  
alpha[1] = 2 * 5;  
alpha[2] = 3;
```


12.3.1 Unvollständige Initialisierung eines Arrays

Werden bei der Initialisierung von Arrays weniger Werte angegeben als das Array Elemente hat, so werden die restlichen, nicht initialisierten Elemente mit dem Wert 0 belegt.



So werden im Folgenden durch:

```
short alpha[200] = {3, 105, 17};
```

die ersten 3 Array-Elemente explizit mit Werten belegt und die restlichen 197 Elemente haben den Wert 0.

Generell ist es nicht möglich, ein Element in der Mitte eines Arrays zu initialisieren, ohne dass die vorangehenden Elemente auch initialisiert werden.

Enthält die Initialisierungsliste mehr Werte als das Array Elemente hat, so meldet der Compiler einen Fehler.

12.3.2 Initialisierung mit impliziter Längenbestimmung

Bei der Initialisierung mit impliziter Längenbestimmung wird die Größe des Feldes – also die Anzahl seiner Elemente – nicht bei der Definition angegeben, das heißt die eckigen Klammern bleiben leer. Die Größe wird vom Compiler durch Abzählen der Anzahl Elemente in der Initialisierungsliste festgelegt.



So enthält das folgende Array 4 Elemente:

```
int alpha[] = {1, 2, 3, 4};
```

Natürlich hätte man die Größe 4 auch in den eckigen Klammern explizit angeben können.

Das Array `int alpha[]` wird als Array ohne Längenangabe bezeichnet. Die Größe des Arrays ist unbestimmt und stellt somit einen sogenannten „unvollständigen Typ“ dar. Erst durch die Initialisierung wird die Größe des Arrays festgelegt, womit der Typ nicht mehr unvollständig ist.

Das Sprachmittel der Initialisierung mit impliziter Längenbestimmung wird vor allem bei Strings verwendet (siehe Kapitel [→ 12.3.6](#)).

12.3.3 Mehrdimensionale Arrays

In C ist es wie in anderen Programmiersprachen möglich, **mehrdimensionale Arrays** zu verwenden. Mehrdimensionale Arrays entstehen durch das Anhängen zusätzlicher eckiger Klammern wie beispielsweise `int alpha[3][4];`



Zweidimensionale Arrays lassen sich stets darstellen als ein eindimensionales Array, welches selbst wiederum Arrays in seinen Elementen speichert. Beispielsweise kann `alpha[3][4]` folgendermaßen interpretiert werden:

<code>alpha [0]</code>
<code>alpha [1]</code>
<code>alpha [2]</code>

Man beachte, dass Indizes bei Arrays stets bei 0 zu zählen beginnen. Jedes dieser Elemente des eindimensionalen Arrays ist selbst wiederum ein Array, bestehend aus 4 Elementen:

		Spaltenindex			
		↓			
Zeilenindex →	[0][0]	[0][1]	[0][2]	[0][3]	
	[1][0]	[1][1]	[1][2]	[1][3]	
	[2][0]	[2][1]	[2][2]	[2][3]	

Damit kann man `alpha` als ein zweidimensionales Array aus 3 Zeilen und 4 Spalten interpretieren. Um auf die einzelnen Elemente zuzugreifen, kann das zweidimensionale Array mit `arrayname[Zeilenindex][Spaltenindex]` angesprochen werden:

```
alpha[1][3]
```

Dieses Element kann auch beim Zuweisungs-Operator auf der linken Seite verwendet werden:

```
alpha[1][3] = 6;
```

Wie ein eindimensionales Array kann auch ein mehrdimensionales Array bereits bei seiner Definition initialisiert werden, beispielsweise durch:

```
int alpha[3][4] = {  
    { 1, 3, 5, 7 },  
    { 2, 4, 6, 8 },  
    { 3, 5, 7, 9 },  
};
```

Dabei wird durch 1, 3, 5, 7 die erste Zeile, durch 2, 4, 6, 8 die zweite Zeile, und 3, 5, 7, 9 die dritte Zeile initialisiert.

Es sind auch mehr als zwei Dimensionen möglich:

```
int beta[7][3][14][8];
```

In C kann ein n -dimensionales Array stets als ein eindimensionales Array interpretiert werden, dessen Komponenten $(n - 1)$ -dimensionale Arrays sind. Diese Interpretation ist rekursiv, da ein Array auf ein Array mit einer um 1 geringeren Dimension zurückgeführt wird.

Wenn ein Array Elemente hat, die selbst ebenfalls Arrays sind, so gelten die Initialisierungsregeln rekursiv. Das bedeutet, dass die Initialisierungsliste eines mehrdimensionalen Arrays geschachtelte Klammern enthalten.



12.3.4 Erweiterte Initialisierung bei mehrdimensionalen Arrays

Natürlich gelten bei geschachtelten Initialisierungslisten dieselben Regeln wie für eindimensionale Arrays. Es dürfen also nicht mehr Elemente initialisiert werden, als das das innere Array speichern kann. Es ist aber möglich, Elemente der inneren Initialisierung auszulassen, worauf diese mit 0 gefüllt werden.

Dabei dürfen sowohl Zeilen fehlen als auch Spalten innerhalb einer Zeile unvollständig initialisiert sein. Alle nicht initialisierten Elemente werden mit 0 initialisiert. Natürlich können Initialisierungen nur am Ende einer Zeile weggelassen werden beziehungsweise die letzten Zeilen können ganz fehlen, da ansonsten eine eindeutige Zuordnung der Werte zu den Elementen nicht möglich wäre.

```
int alpha[3][4] = {  
    {1},                // 1 0 0 0  
    {1,1}               // 1 1 0 0  
};                      // 0 0 0 0
```

Es ist zudem möglich, geschweifte Klammern von inneren Arrays bei der Initialisierung auch auszulassen. Beginnt die Initialisierung des inneren Arrays nicht mit einer linken geschweiften Klammer, dann werden nur genügend Initialisierungen für die Bestandteile des inneren Arrays aus der Liste entnommen. Sind noch Initialisierungswerte übrig, so werden sie für das nächste Element des äußeren Arrays herangezogen. Die folgende beiden Initialisierung sind somit äquivalent:

```
int alpha[3][4] = {  
    { 1, 3, 5, 7 },  
    { 2, 4, 6, 8 },  
    { 3, 5, 7, 9 },  
};  
  
float b[3][4] = {1, 3, 5, 7, 2, 4, 6, 8, 3, 5, 7, 9};
```

Bei der Initialisierung von `b` werden hier 4 Elemente für die Initialisierung von `b[0]` benutzt, die nächsten 4 für die Initialisierung von `b[1]` und die letzten 4 für die Initialisierung von `b[2]`. Damit ist das Array mit denselben Werten initialisiert worden wie `alpha`.

12.3.5 String-Konstanten

Eine **String-Konstante** (ein sogenanntes „**String-Literal**“) besteht aus einer Folge von Zeichen, die in Anführungszeichen eingeschlossen sind. Siehe auch Kapitel [→ 6.5.4](#).

```
"Hallo Welt!"
```

Eine String-Konstante wird vom Compiler intern als ein Array von Zeichen gespeichert. Dabei wird als letztes Element des Arrays automatisch ein zusätzliches Zeichen, das Zeichen `'\0'` (**Nullzeichen**), angehängt, um das Stringende anzuzeigen.



Die Speicherung eines Strings mit einem zusätzlichen Nullzeichen am Ende wird umgangssprachlich als „**C-String**“ bezeichnet.

Wie jede andere Konstante auch stellt eine String-Konstante einen Ausdruck dar, der nicht verändert, aber gelesen werden kann.

Der Rückgabewert einer String-Konstanten ist ein Pointer auf das erste Zeichen des Strings. Der Typ des Rückgabewertes ist `const char*`.



12.3.6 char-Arrays

char-Arrays werden verwendet, um Strings abzuspeichern:

```
char array[20];
```

Im Gegensatz zu String-Konstanten können char-Arrays auch verändert werden. In einem solchen Array können beispielsweise Zeichen einzeln mit Werten belegt werden:

```
array[0] = 'a';
```

Initialisiert man ein char-Array manuell sofort bei seiner Definition, so kann – wie bei jedem eindimensionalen Array – eine Initialisierungsliste in geschweiften Klammern verwendet werden. Die einzelnen Werte der Liste – hier die Zeichen – werden durch Kommas getrennt wie im folgenden Beispiel:

```
char array[20] = {'Z', 'e', 'i', 'c', 'h', 'e', 'n',  
                 'k', 'e', 't', 't', 'e', '\\0'};
```

Da das Anschreiben einer Initialisierungsliste als Array von Zeichen mit den vielen Hochkommas sehr mühsam ist, kann man ein char-Array auch mit einer String-Konstanten initialisieren.



Dies würde im Falle des obigen Beispiels dann folgendermaßen aussehen:

```
char array[20] = "Zeichenkette";
```

Beide manuellen Initialisierungen sind vom Speicherinhalt her äquivalent. Die zweite Formulierung stellt eine Abkürzung für die erste, längere Schreibweise dar. Die zweite Form der Initialisierung stellt allerdings einen Sonderfall dar, den es nur für eindimensionale Arrays von Zeichen gibt. Das Array wird mit den Zeichen der String-Konstante initialisiert, wobei ein zusätzliches, abschließendes Nullzeichen '\\0' automatisch angehängt wird.

Eine direkte Zuweisung eines Strings an ein eindimensionales Array kann nur bei der Initialisierung erfolgen. Im weiteren Programmablauf sind spezielle Bibliotheksfunktionen für diese Zwecke notwendig.



12.4 Übergabe von Arrays an Funktionen

Bei der Übergabe eines **Arrays als Parameter** an eine Funktion wird als aktueller Parameter der Arrayname angegeben. Dieser wird bei Aufruf der Funktion in einen Pointer auf das erste Element des Arrays ausgewertet.

Der formale Parameter für die Übergabe eines eindimensionalen Arrays kann ein Array ohne Längenangabe sein, also ein Array mit leeren eckigen Klammern. Da ein Arrayname jedoch bei einem Funktionsaufruf als Pointer ausgewertet wird, kann auch ein Pointer auf den Komponenten-Typ des Arrays als Typ des formalen Parameters verwendet werden.



Ein Array ohne Längenangabe hat jedoch – wie der Name schon sagt – keine explizite Länge.

Um die Länge eines Arrays automatisch zu ermitteln, gibt es die Möglichkeit, eine Endemarkierung als letztes Element des Arrays zu setzen, beispielsweise den Wert NULL. Dieser als „Wächter“ oder auf Englisch „sentinel“ bezeichnete Marker kann daraufhin im Code verwendet werden, um die tatsächliche Länge des Arrays bei Bedarf automatisch zu ermitteln. Dies wird beispielsweise bei der Verarbeitung von Strings (siehe Kapitel [→ 12.9.5](#)) häufig gemacht.

Eine weitaus einfachere Methode, um die Länge eines als Parameter übergebenen Arrays zu ermitteln, ist das Übergeben der Länge als zusätzlichen Parameter.

Folgendes Beispiel zeigt zwei Methoden, ein Array per Parameter zu übergeben:

arrayparameter.c

```
#include <stdio.h>
#define GROESSE 3

void init(int*, int);
void ausgabe(int[], int);

int main(void) {
    int i[GROESSE];
    init(i, GROESSE);
    ausgabe(i, GROESSE);
    return 0;
}
```

```
void init(int* alpha, int dim) {  
    for (int i = 0; i < dim; ++i) {  
        *alpha = i;  
        ++alpha;  
    }  
}  
  
void ausgabe(int alpha[], int dim) {  
    for (int i = 0; i < dim; ++i) {  
        printf("i[%d] hat den Wert: %d\n", i, alpha[i]);  
    }  
}
```

Das Programm erzeugt folgende Ausgabe:

```
i[0] hat den Wert: 0  
i[1] hat den Wert: 1  
i[2] hat den Wert: 2
```

Da es sich bei dem formalen Parameter `alpha` der Funktion `init()` um einen Pointer handelt, ist somit auch möglich, nur einen Teil des Arrays anzusprechen, indem einfach ein Pointer auf das erste Element des Teil-Arrays und die gewünschte Anzahl der Komponenten übergeben wird.

12.4.1 Übergabe von Strings

Da Strings vom Compiler intern als `char`-Arrays gespeichert werden, ist die Übergabe von **Strings als Parameter** identisch mit der Übergabe von `char`-Arrays. Der formale Parameter einer Funktion, der ein String übergeben bekommt, kann vom Typ `char*` oder `char[]` sein.



Dadurch, dass in einem String ein Nullzeichen das Ende definiert, benötigen Funktionen, die mit Strings arbeiten, keinen zusätzlichen Parameter mit einer Längenangabe. Falls nötig, kann die Länge des Strings über die Funktion `strlen()` (siehe Kapitel [→ 12.9](#)) berechnet werden.

Es ist jedoch zu beachten, dass String-Konstanten stets mit dem Typ `const char*` übergeben werden müssen.

12.4.2 Ausgabe von Strings und von char-Arrays

Der Rückgabewert eines Strings ist ein Pointer auf das erste Element des Strings. Damit ist klar, was bei der Übergabe eines Strings an die Funktion `printf()` wie im folgenden Beispiel passiert:

```
printf("Hello, world");
```

Die String-Konstante "Hello, world" wird beim Laden des Programmes in den Speicher geladen. An die Funktion `printf()` wird ein Pointer auf diesen String übergeben. So ist `printf()` in der Lage, den Inhalt des Arrays auszudrucken. Die Funktion `printf()` druckt beginnend vom ersten Zeichen alle Zeichen des Arrays aus, bis sie ein Nullzeichen '\0' findet.

Bekanntermaßen kann man mit der Funktion `printf()` formatierte Strings ausgeben (daher auch der Name: `printf` = `print formatted`). Nebst den bekannten Formatelementen `%d` und `%f` für Integer- und Gleitpunkt-Zahlen gibt es jedoch auch das Formatelement `%s`.

String-Variablen werden bei `printf()` mit Hilfe des Formatelements `%s` ausgegeben.



Das Umwandlungszeichen `s` steht für „String“ und die Funktion erwartet als Parameter einen String:

```
char adjektiv[] = "beautiful";  
printf("Hello, %s world", adjektiv);
```

Wie man sieht, können so String-Variablen ausgegeben werden.

12.5 Unterschiede zwischen char-Arrays und Pointern auf char

In den bisherigen Kapiteln wurden **char-Arrays** und **String-Konstanten** behandelt. Hier sollen nun die Unterschiede weiter verdeutlicht werden. Wir betrachten die folgenden beiden Variablen:

```
const char* string1  = "Hallo";  
char  string2[] = "Hallo";
```

Die Variable `string1` ist ein Pointer auf eine String-Konstante. Sie speichert selbst also nur eine Adresse, nicht den Inhalt selbst.

Die Variable `string2` ist ein char-Array. Es definiert mittels impliziter Längenbestimmung die Anzahl Elemente und speichert genau so viele char-Elemente.

Selbst wenn die Typen dieser beiden Variablen unterschiedlich sind, so können sie doch sehr häufig genau gleich als Argumente oder allgemein in Ausdrücken verwendet werden. Dies deswegen, da ein Arrayname dann zu einem Pointer auf das erste Element ausgewertet wird.

Die beiden folgenden Zeilen lassen somit keinen Unterschied erahnen:

```
printf("%s, Welt", string1);  
printf("%s, Welt", string2);
```

Trotzdem ist es wichtig, zu begreifen, was hinter diesen beiden Variablen steckt.

Der String "Hallo" wird direkt im Programmcode geschrieben, was im Englischen als „string literal“ bezeichnet wird. Dieser String wird bei Programmstart vom Lader in den Speicher geladen und ist dort verfügbar. Allerdings befindet sich dieser String in einem geschützten Bereich des Speichers, auf welchen nur lesend zugegriffen werden darf. Dies ist das sogenannte „Code-Segment“, siehe Kapitel [→ 15.2](#).

Der der Variablen `string1` zugewiesene Pointer zeigt nach der Initialisierung eben genau auf diesen Speicherbereich, auf welchen der Compiler keine Schreibzugriffe erlaubt. Dementsprechend muss der Typ der Variable `string1` mit `const` deklariert werden.

Unter C ist das Weglassen von `const` bei der Vereinbarung des Pointers erlaubt. Wenn jedoch auf den Inhalt dieses Pointers ein Schreibzugriff erfolgt, stürzt das Programm auf heutigen Betriebssystemen normalerweise ab. Unter C++ gibt der Compiler einen Fehler aus, wenn versucht wird, das `const` wegzulassen.



Es ist jedoch erlaubt, die Variable `string1` selbst zu verändern, sprich einen anderen Pointer zuzuweisen:

```
string1 = "Guten Morgen";
```

Zeigt ein Pointer auf eine String-Konstante, so ist eine Änderung des Speicherinhalts, auf welchen dieser Pointer zeigt, nicht erlaubt. Die Zuweisung eines neuen Pointers hingegen ist möglich.



Bei der Initialisierung des char-Arrays `string2` wird der String "Hallo" explizit in das Array hineinkopiert. Somit existieren nach einer Initialisierung zwei Strings im Speicher: Die String-Konstante, welche vom Lader in den Speicher geladen wurde, sowie der nicht konstante String in dem Array `string2`.

Die Elemente des Arrays `string2` sind jedoch nicht mit `const` deklariert und können somit beliebig angesprochen und verändert werden:

```
string2[1] = 'e';    \\ Hallo -> Hello
```

Jedoch ist es nicht möglich, die Variable selbst auf einen anderen Speicherort zeigen zu lassen. Folgende Anweisung erzeugt somit einen Fehler:

```
string2 = "C-Programming";    // Fehler!
```

Bei einem char-Array `string2` kann der in ihm gespeicherte String verändert werden. Die Variable `string2` selbst hingegen kann nicht verändert werden.



12.6 Das Schlüsselwort const bei Pointern und Arrays

Wie in Kapitel [→ 7.5.1](#) bereits angesprochen, können mithilfe des Schlüsselworts `const` schreibgeschützte Variablen vereinbart werden.

Die Deklaration mit **`const`** kann auch auf zusammengesetzte Datentypen wie Arrays angewendet werden:

```
const int feld[] = {1, 2, 3};
```

Hier bedeutet die `const`-Deklaration, dass alle Feldelemente `feld[0]`, `feld[1]` und `feld[2]` nicht überschrieben werden dürfen.

Aufpassen muss man bei der Anwendung des Schlüsselwortes `const` im Zusammenhang mit Pointern. Angenommen, ein `char`-Array sei folgendermaßen definiert:

```
char aussage[] = "Pointer sind wichtig.";
```

Dann bedeutet das Pointer-Sternchen `*` in der folgenden Zeile nicht, dass der Pointer `text` schreibgeschützt ist, sondern dass dieser Pointer auf einen schreibgeschützten String zeigt.

```
const char* text = "blick";
```

Demnach wird in der folgenden Zeile der Compiler einen Fehler ausgeben:

```
text[1] = 's';
```

Der Pointer jedoch kann auf einen anderen konstanten String zeigen, wie beispielsweise:

```
text = "Jetzt blicke auch ich durch.";
```

Soll tatsächlich der Pointer als schreibgeschützt deklariert werden, so muss `const` nach dem Pointer-Zeichen stehen wie im folgenden Beispiel:

```
char lili[] = "Ich liebe Lili";  
char* const hugo = lili;
```

Man kann sich diese Notation leicht merken, indem man den Typ-Ausdruck von rechts nach links liest mit den Worten „hugo ist ein schreibgeschützter (const) Pointer (*) auf den Typ char“.

In diesem Falle ist dann die folgende Zeile möglich:

```
hugo[13] = 'o';
```

Ob dies Lili gefällt, ist eine andere Frage. Wenigstens kann sie sichergehen, dass folgende Zeile vom Compiler mit einem Fehler geahndet wird:

```
hugo = "Ich liebe Susi";
```

Um hugo stets unzertrennlich mit lili zu verbinden, kann der folgende Typ vereinbart werden:

```
const char* const hugo = lili;
```

Hier sind sowohl der Pointer hugo, als auch der String schreibgeschützt.

Der Schutz eines const-Werts gilt auch für Übergabeparameter, beispielsweise

```
void f(const int* pointer) {  
    pointer[3] = 15;           // Fehler !  
}
```

Bei Übergabeparametern wird const hauptsächlich zum Schutz der übergebenen Speicherbereiche benutzt (siehe Kapitel [→ 11.5.2](#)). Eine so definierte Funktion kann auf diese Speicherbereiche nur lesend zugreifen.

Eine Funktion kann auch ein const Ergebnis liefern, wie beispielsweise einen Pointer auf eine String-Konstante. Hierzu muss beim Rückgabebetyp der Modifikator const angegeben werden.

```
const char* werLiebtSusi() {  
    return "Sag ich nicht.";  
}
```

12.7 Beispiel für Strings und Pointerarithmetik

Normalerweise verwendet man in der Praxis Standardfunktionen zur Stringverarbeitung (siehe Kapitel [→ 12.9](#)). Hier soll jedoch exemplarisch aufgezeigt werden, wie effizient die Programmiersprache C mit Strings umgehen kann.

Im folgenden Beispiel wird ein String „von Hand“ von einem Array alpha in ein Array beta kopiert:

manualcopy.c

```
#include <stdio.h>

int main(void) {
    char alpha[30] = "zu kopierender String";
    char beta[30]  = "";

    int i = 0;
    while (alpha[i] != '\0') {
        beta[i] = alpha[i];
        ++i;
    }
    beta[i] = '\0';

    printf("%s\n", alpha);
    printf("%s\n", beta);

    return 0;
}
```

Die Ausgabe dieses Beispiels lautet:

```
zu kopierender String
zu kopierender String
```

Dieses Beispiel ist sehr klar aufgebaut und sehr gut lesbar. Nun möchten wir dieses Beispiel sukzessive verfeinern. Zuallererst verkürzen wir die Anweisungen zum Kopieren wie folgt:

```
int i = 0;
while ((beta[i] = alpha[i]) != '\0') {
    ++i;
}
```

Hier wurde der Umstand genutzt, dass der Zuweisungs-Operator nicht nur den Wert von alpha nach beta kopiert, sondern dass der Rückgabewert von dem Zuweisungs-Operator zudem der Wert von beta[i] nach der Zuweisung ist. So werden automatisch sämtliche Zeichen kopiert. Erst bei Auftreten des abschließenden '\0' wird die Schleife abgebrochen, der Wert '\0' wurde jedoch ebenfalls bereits von alpha nach beta kopiert.

Da nun zudem der Wert von '\0' gleich 0 ist und so dem Wahrheitswert „falsch“ entspricht, kann man noch knapper schreiben:

```
int i = 0;
while (beta[i] = alpha[i]) {
    ++i;
}
```

Während bislang das Kopieren mit Hilfe von Array-Komponenten durchgeführt wurde, soll im Folgenden das Kopieren mit Hilfe von Pointern demonstriert werden. In der Pointerschreibweise muss man berücksichtigen, dass alpha und beta Array-Variablen und deshalb nicht veränderbar sind. Um somit veränderbare Pointer auf die Arrays zu erhalten, müssen die folgenden Pointer-Variablen vereinbart werden:

```
char* ptrAlpha = alpha;
char* ptrBeta = beta;
```

ptrAlpha zeigt auf alpha[0] und ptrBeta zeigt auf beta[0]. Damit kann man nun schreiben:

```
while (*ptrAlpha != '\0') {
    *ptrBeta = *ptrAlpha;
    ptrAlpha++;
    ptrBeta++;
}
*ptrBeta = '\0';
```

Eine knappere Formulierung ist:

```
while (*ptrAlpha != '\0') {
    *ptrBeta++ = *ptrAlpha++;
}
*ptrBeta = '\0';
```

Diese Vereinfachung funktioniert, da der Rückgabewert von `ptrAlpha++` dem Wert vor der Erhöhung um 1 entspricht (Postfix-Operator). Mit dem Operator `*` wird somit der Wert des „aktuellen“ `ptrAlpha` dereferenziert. Entsprechendes gilt für `ptrBeta++`. Nach der Ausdrucksanweisung muss der Nebeneffekt stattgefunden haben, das heißt vor dem nächsten Kopiervorgang zeigen `ptrAlpha` und `ptrBeta` jeweils um ein Zeichen weiter.

Noch kürzer wäre:

```
while (*ptrBeta++ = *ptrAlpha++);
```

Hierbei wird erstens ausgenutzt, dass eine Zuweisung auch einen Nebeneffekt hat, so dass das zugewiesene Zeichen `ptrAlpha` sich nach der Anweisung an der Stelle `ptrBeta` befindet. Zweitens wird die Schleife abgebrochen, wenn ein zu kopierendes Zeichen gleich dem Nullzeichen `'\0'` ist, da der zugewiesene Wert beim Zuweisungs-Operator gleichzeitig als Rückgabewert und somit als Ausdruck für die Bedingung der `while`-Schleife dient.

Das Nullzeichen `'\0'` ist das letzte Zeichen vor dem Abbruch der Iteration, das kopiert wurde. Man spart sich also sogar noch die Zuweisung des Nullzeichens nach der Schleife!

Wie diese Beispiele auch zeigen, benötigt man im Gegensatz zur Array-Schreibweise bei der Formulierung mit Pointern die Laufvariable `i` nicht mehr. Dafür aber die beiden Pointer `ptrAlpha` und `ptrBeta`.

12.8 Funktionen zur Speicherbearbeitung

In diesem Unterkapitel werden Funktionen zur Speicherbearbeitung betrachtet. Um sie zu verwenden, wird die Bibliothek `<string.h>` benötigt.

Speicherbearbeitungsfunktionen arbeiten auf sogenannten „**Puffern**“.

Als Puffer (auf Englisch „buffer“) wird ganz allgemein ein Speicher bezeichnet, der Daten zwischenspeichert.



Im Rahmen der Speicherbearbeitungsfunktionen entspricht ein Puffer einem Pointer auf einen Ort im Speicher, beispielsweise auf ein Array. Die formalen Parameter der Funktionen erwarten jeweils ein Objekt vom Typ `void*`. Zudem erwarten die Funktionen die Angabe einer entsprechenden Pufferlänge, gemessen in Anzahl Bytes. Die so definierten Puffer-Objekte werden von den Funktionen *byteweise* behandelt.

In der Standardbibliothek von C existieren die folgenden Funktionen:

<code>memcpy()</code>	Kopieren von Puffern
<code>memmove()</code>	Überlappendes Kopieren von Puffern
<code>memcmp()</code>	Vergleichen von Puffern
<code>memchr()</code>	Suchen nach einem Bytewert
<code>memset()</code>	Initialisieren eines Puffers

Zu jeder Funktion wird jeweils zuallererst der Prototyp hingeschrieben und danach die Funktionalität vorgestellt.

12.8.1 Die Funktion memcpy()

```
void* memcpy(void* dest, const void* src, size_t n);
```

Die Funktion memcpy() kopiert n Bytes aus dem Puffer, auf den der Pointer src zeigt, in den Puffer, auf den der Pointer dest zeigt.



Ist der zu kopierende Puffer größer als der Zielpuffer, dann werden nachfolgende Speicherobjekte überschrieben!



Der Rückgabewert der Funktion memcpy() ist der Pointer dest.

Ab dem Standard C99 werden die beiden Puffer-Parameter mit dem restrict-Schlüsselwort deklariert, siehe Kapitel [8.4](#). Überlappen sich die Puffer, auf die die Pointer src und dest zeigen, so ist das Ergebnis undefiniert.

Beispiel:

memcpy.c

```
#include <stdio.h>
#include <string.h>

int main(void) {
    int array1 [10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
    int array2 [3]  = {333, 444, 555};
    memcpy(array1 + 2, array2, sizeof(array2));

    for (size_t i = 0; i < 10; ++i) {
        printf("%d, ", array1[i]);
    }

    return 0;
}
```

Die Ausgabe ist:

```
Ergebnis: 1, 2, 333, 444, 555, 6, 7, 8, 9, 10
```

12.8.2 Die Funktion memmove()

```
void* memmove(void* dest, const void* src, size_t n);
```

Die Funktion memmove() kopiert n Bytes von einem Puffer in einen anderen – und zwar auch bei überlappenden Speicherbereichen korrekt.



Im Gegensatz zur Funktion memcpy() ist bei der Funktion memmove() sichergestellt, dass bei überlappenden Speicherbereichen das korrekte Ergebnis erzielt wird. Der Puffer, auf den der Pointer src zeigt, kann dabei überschrieben werden.

Ist der zu kopierende Puffer größer als der Zielpuffer, dann werden nachfolgende Speicherobjekte überschrieben!



Der Rückgabewert der Funktion memmove() ist der Pointer dest.

Beispiel:

memmove.c

```
#include <stdio.h>
#include <string.h>

int main(void) {
    char string[] = "12345678";
    memmove(string + 2, string, strlen(string) - 2);
    printf("Ergebnis: %s\n", string);
    return 0;
}
```

Ausgabe:

```
Ergebnis: 12123456
```

Würde dieses Beispiel mit `memcpy()` anstelle `memmove()` erfolgen, so ist das Ergebnis undefiniert. Es könnte sein, dass `memcpy()` eine einfache Schleife implementiert, welche die Inhalte Byte für Byte kopieren. Dann könnte das Ergebnis auch lauten:

Ergebnis: 12121212

Dies deswegen, da die Kopierfunktion bei dem Byte `string[4]` den Inhalt von `string[2]` kopiert, welcher bereits vorgängig von `string[0]` überschrieben wurde. Für diese Problematik gibt es eine Übungsaufgabe am Ende dieses Kapitels.

12.8.3 Die Funktion `memcmp()`

```
int memcmp(const void* s1, const void* s2, size_t n);
```

Die Funktion `memcmp()` vergleicht `n` Bytes aus zwei verschiedenen Puffern.



Die Funktion `memcmp()` führt einen byteweisen Vergleich der ersten `n` Bytes der an die Pointer `s1` und `s2` übergebenen Puffer durch. Die Puffer werden solange verglichen, bis entweder ein Byte unterschiedlich oder die Anzahl `n` Bytes erreicht ist.

Die Funktion `memcmp()` gibt folgende Rückgabewerte zurück:

- `< 0` wenn das erste Byte, das in beiden Puffern verschieden ist, im Puffer, auf den der Pointer `s1` zeigt, einen kleineren Wert hat.
- `== 0` wenn `n` Bytes der beiden Puffer gleich sind.
- `> 0` wenn das erste in beiden Puffern verschiedene Byte im Puffer, auf den der Pointer `s1` zeigt, einen größeren Wert hat.

Beispiel:

memcmp.c

```
#include <stdio.h>
#include <string.h>

int main(void) {
    char string1[] = {0x01, 0x02, 0x03, 0x04, 0x05, 0x06};
    char string2[] = {0x01, 0x02, 0x03, 0x14, 0x05, 0x06};
    int cmp = memcmp(string1, string2, sizeof(string1));
    printf("Vergleich String1 mit String2 ergibt: %d \n", cmp);
    return 0;
}
```

Die Ausgabe ist beispielsweise:

```
Vergleich String1 mit String2 ergibt: -1
```

Die Bildung des Rückgabewertes ist compilerabhängig. Manche Compiler geben den rechnerischen Unterschied des ersten unterschiedlichen Bytes aus, andere geben lediglich eine positive oder negative Zahl aus, was nach dem ISO-Standard genügt.

12.8.4 Die Funktion `memchr()`

```
void* memchr(const void* s, int c, size_t n);
```

Die Funktion `memchr()` durchsucht einen Puffer nach einem bestimmten Byte.



Die Funktion `memchr()` durchsucht die ersten `n` Bytes des Puffers, auf den der Pointer `s` zeigt, nach dem Wert von `c`. Dabei werden der Wert `c` wie auch alle `n` Bytes des Puffers als `unsigned char` interpretiert.

Wird der Wert von `c` gefunden, so gibt die Funktion `memchr()` einen Pointer auf das erste Vorkommen im Puffer zurück, auf den der Pointer `s` zeigt. Ist der Wert von `c` in den ersten `n` Bytes nicht enthalten, so wird ein `NULL`-Pointer zurückgegeben.

Beispiel:

memchr.c

```
#include <stdio.h>
#include <string.h>

int main(void) {
    char str1 [] = "Zeile1: Text";
    char* str2 = memchr(str1, ':', strlen(str1));
    if (str2 != NULL) {
        printf("%s\n", str2);
    }
    return 0;
}
```

Die Ausgabe ist:

: Text

Die Funktion `memchr()` erwartet den Puffer mit dem Typ `const void*`, gibt jedoch einen Wert vom Typ `void*` zurück. Ein Schreibzugriff auf den Inhalt des Rückgabewertes kann unerwünschte Nebeneffekte hervorrufen oder gar zum Absturz des Programmes führen.



12.8.5 Die Funktion `memset()`

```
void* memset(void* s, int c, size_t n);
```

Die Funktion `memset()` setzt die ersten `n` Bytes eines Puffers auf den Wert eines vorgegebenen Bytes `c`.



Dabei werden der Wert `c` wie auch alle `n` Bytes des Puffers als `unsigned char` interpretiert.

Der Rückgabewert der Funktion `memset()` ist der Pointer `s`.

Beispiel:

memset.c

```
#include <stdio.h>
#include <string.h>

int main(void) {
    char string[20] = "Hallo";
    printf("Ergebnis: %s\n", memset(string, '*', 5));
    return 0;
}
```

Die Ausgabe ist:

```
Ergebnis: *****
```

12.9 Standardfunktionen zur Stringverarbeitung

Der C-Standard definiert mehrere Funktionen, welche die Manipulation von Strings ermöglichen. Um sie zu verwenden, wird die Bibliothek `<string.h>` benötigt.

Stringverarbeitungsfunktionen funktionieren fast gleich wie die Funktionen zur Speicherbearbeitung. Im Gegensatz dazu berücksichtigen String-Funktionen hingegen zusätzlich noch das Nullzeichen `'\0'` am Ende eines Strings.

In diesem Kapitel werden nur die einfacheren Funktionen behandelt. In den Standardbibliotheken gibt es noch viele weitere Funktionen, welche beispielsweise mit `wide characters` umgehen oder einzelne Zeichen oder gar ganze Strings innerhalb eines Strings suchen können. Im C11-Standard wurden zudem für bestimmte Funktionen zusätzlich sogenannte „sichere“ Funktionen definiert, welche eine Prüfung der übergebenen Pufferlänge ermöglichen. Die Gesamtheit aller verfügbaren Funktionen kann an dieser Stelle nicht angesprochen werden.

Folgende Stringverarbeitungsfunktionen werden vorgestellt:

<code>strcpy()</code>	Kopieren von Strings
<code>strncpy()</code>	Kopieren von Strings mit Längenangabe
<code>strcat()</code>	Anhängen eines Strings an einen anderen
<code>strcmp()</code>	Vergleichen von Strings
<code>strlen()</code>	Ermitteln der Stringlänge

12.9.1 Die Funktion strcpy()

```
char* strcpy(char* dest, const char* src);
```

Die Funktion strcpy() kopiert den Inhalt des Strings, auf den der Pointer src zeigt, an die Adresse, auf die der Pointer dest zeigt.



Die Zieladresse muss hierbei an einen Ort im Speicher zeigen, an welchem genügend Platz reserviert ist, um den gesamten String inklusive das abschließende Stringende-Zeichen '\0' zu fassen.

Kopiert wird der gesamte Inhalt einschließlich des Stringende-Zeichens '\0'.



Die Funktion strcpy() überprüft dabei nicht, ob der Puffer, dessen Adresse übergeben wurde, genügend Platz zur Verfügung stellt. Hierfür muss man selbst Sorge tragen.

Ist der zu kopierende Puffer größer als der Zielpuffer, dann werden nachfolgende Speicherobjekte überschrieben!



Seit C99 werden die beiden Parameter mit dem restrict-Schlüsselwort deklariert, siehe Kapitel [→ 8.4](#). Dies bedeutet, dass wenn zwischen zwei sich überlappenden Objekten kopiert wird, das Verhalten undefiniert ist. Im obigen Prototyp wurde auf die Angabe des Schlüsselworts verzichtet.

Die Funktion strcpy() gibt als Rückgabewert den Pointer dest zurück.

Folgendes ist ein einfaches Programmbeispiel:

strcpy.c

```
#include <stdio.h>
#include <string.h>

int main(void) {
    char string1[25];
    char string2[] = "Zu kopierender String";
    strcpy(string1, string2);
    printf("Der kopierte String ist: %s\n", string1);
    return 0;
}
```

Das Beispiel erzeugt die folgende Ausgabe:

Der kopierte String ist: Zu kopierender String

12.9.2 Die Funktion strncpy()

```
char* strncpy(char* dest, const char* src, size_t n);
```

Die Funktion `strncpy()` kopiert genauso wie `strcpy()` einen String an eine andere Adresse, wird jedoch garantiert nicht mehr als eine vorgegebene Anzahl Zeichen kopieren.



Der Parameter `n` der Funktion `strncpy()` kann somit genutzt werden, um sicherzustellen, dass der Puffer `dest`, dessen Adresse übergeben wurde, nicht durch das Kopieren überquillt.

Dabei wird das Stringende-Zeichen `'\0'` bei den `n` Zeichen mitgezählt.

Hat der zu kopierende String (inklusive dem Stringende-Zeichen `'\0'`) mehr als `n` Zeichen, so wird die Kopie beim `n`-ten Zeichen abgebrochen. Der String an der Adresse `dest` ist in diesem Falle nicht mit einem Stringende-Zeichen `'\0'` abgeschlossen.



Wird als Parameter `n` eine Zahl angegeben, welche größer ist als der reservierte Speicherplatz an der Stelle des Puffers `dest`, werden die darauffolgenden Speicherbereiche überschrieben!



Hat der zu kopierende String weniger Zeichen als `n` Zeichen, so werden die überschüssigen Zeichen in `dest` mit dem Stringende-Zeichen `'\0'` aufgefüllt.



Wie schon bei `strcpy()` werden seit C99 die beiden Pointer-Parameter mit dem `restrict`-Schlüsselwort deklariert, siehe Kapitel [8.4](#). Dies bedeutet, dass wenn zwischen zwei sich überlappenden Objekten kopiert wird, das Verhalten undefiniert ist. Im obigen Prototyp wurde auf die Angabe des Schlüsselworts verzichtet.

Die Funktion `strncpy()` gibt als Rückgabewert den Pointer `dest` zurück.

Ein einfaches Beispiel:

`strncpy.c`

```
#include <stdio.h>
#include <string.h>

int main(void) {
    char string1[] = "XXXXXXXXXXXXXXXXXXXX";
    char string2[] = "Zu kopierender String";
    strncpy(string1, string2, 10);
    printf("Der kopierte String ist: %s\n", string1);
    return 0;
}
```

Folgendes ist die Ausgabe:

```
Der kopierte String ist: Zu kopiereXXXXXXXXXXXX
```

12.9.3 Die Funktion strcat()

```
char* strcat(char* dest, const char* src);
```

Die Funktion `strcat()` hängt einen String an einen anderen an.



Die Funktion `strcat()` hängt an den String, auf den der Pointer `dest` zeigt, den String an, auf den der Pointer `src` zeigt. Dabei wird das Stringende-Zeichen `'\0'` des Strings, auf den der Pointer `dest` zeigt, vom ersten Zeichen des Strings, auf den der Pointer `src` zeigt, überschrieben. Daraufhin wird der restliche String kopiert, auf den der Pointer `src` zeigt, einschließlich des Zeichens `'\0'`.

Der Name der Funktion kommt von dem englischen „concatenate“ = zusammenfügen.

Die Funktion `strcat()` prüft nicht, ob genügend Speicher im String, auf den der Pointer `dest` zeigt, vorhanden ist. Die Kontrolle des zur Verfügung stehenden Speichers muss man selbst verantworten. Reicht der Puffer nicht aus, werden nachfolgende Speicherobjekte überschrieben.



Auch hier gilt, dass seit C99 die beiden Pointer-Parameter mit dem `restrict`-Schlüsselwort deklariert werden, siehe Kapitel [→ 8.4](#). Dies bedeutet, dass wenn die beiden Speicherbereiche sich überlappenden, das Verhalten undefiniert ist. Im obigen Prototyp wurde auf die Angabe des Schlüsselworts verzichtet.

Der Rückgabewert der Funktion `strcat()` ist ein Pointer auf den zusammengeführten String, also der Pointer `dest`.

Beispiel:

strcat.c

```
#include <stdio.h>
#include <string.h>

int main(void) {
    char string[50] = "concatenate";
    strcat(string, " = zusammenfuegen");
    printf("%s\n", string);
    return 0;
}
```

Die Ausgabe ist:

```
concatenate = zusammenfuegen
```

12.9.4 Die Funktion strcmp()

```
int strcmp(const char* s1, const char* s2);
```

Die Funktion strcmp() vergleicht zwei Strings zeichenweise.



Die Funktion strcmp() führt einen zeichenweisen Vergleich der beiden Strings durch, auf die die Pointer s1 und s2 zeigen. Sie werden solange verglichen, bis ein Zeichen unterschiedlich oder bis ein Stringende-Zeichen '\0' erreicht ist.

Die Funktion strcmp() gibt einen int-Wert zurück mit folgender Bedeutung:

- < 0 wenn der String, auf den der Pointer s1 zeigt, lexikografisch kleiner ist als der String, auf den der Pointer s2 zeigt.
- $= 0$ wenn der String, auf den der Pointer s1 zeigt, lexikografisch gleich dem String ist, auf den der Pointer s2 zeigt.
- > 0 wenn der String, auf den der Pointer s1 zeigt, lexikografisch größer ist als der String, auf den der Pointer s2 zeigt.

Der Begriff „lexikographisch“ bedeutet hierbei, dass die Zeichen gemäß der Stellung im Alphabet bewertet werden. Anders gesagt, steht der Buchstabe A vor dem Buchstaben B, er ist also lexikographisch kleiner als B.

Die Werte < 0 beziehungsweise > 0 entstehen durch den Vergleich zweier unterschiedlicher unsigned char-Zeichen.

Der Vergleich von zwei Strings mit der Methode `strcmp()` erfolgt durch einen Vergleich der einzelnen Zeichen an den äquivalenten Positionen in den beiden Strings, und zwar von vorne nach hinten.



Beim Vergleich werden die entsprechenden Zeichen voneinander subtrahiert. Sobald das Ergebnis der Subtraktion zweier Zeichen eine Zahl ungleich 0 ist, wird der Vergleich abgebrochen. Sind die Strings ungleich lang, so wird bis zum Stringbegrenzungszeichen `'\0'` des kürzeren Strings verglichen.

Beispiel:

`strcmp.c`

```
#include <stdio.h>
#include <string.h>

int main(void) {
    printf("%d\n", strcmp("abcde", "abCde"));
    printf("%d\n", strcmp("abcde", "abcde"));
    printf("%d\n", strcmp("abcd", "abcde"));
    return 0;
}
```

Die Ausgabe lautet:

```
1
0
-1
```

12.9.5 Die Funktion strlen()

```
size_t strlen(const char* s);
```

Die Funktion `strlen()` bestimmt die Anzahl Zeichen eines Strings.



Die Funktion `strlen()` liefert als Rückgabewert die Anzahl der Zeichen des Strings, auf den der Pointer `s` zeigt. Das Stringende-Zeichen `'\0'` wird dabei nicht mitgezählt.

Beispiel:

strlen.c

```
#include <stdio.h>
#include <string.h>

int main(void) {
    char string[] = "Programm";
    printf("Das Array hat %d Elemente.\n", (int)sizeof(string));
    printf("Der String hat %d Zeichen.\n", (int)strlen(string));
    return 0;
}
```

Die Ausgabe ist:

```
Das Array hat 9 Elemente.
Der String hat 8 Zeichen.
```

Dieses Programm zeigt den Unterschied zwischen `sizeof` und `strlen()` auf. Mit `sizeof` wird die Größe des Arrays ermittelt und mit `strlen()`, wie weit es aktuell gefüllt ist.

Die Funktion `strlen()` liefert die Länge eines Strings zurück, ohne das Zeichen `'\0'` mitzuzählen. Will man beispielsweise prüfen, ob ein `char`-Array groß genug zur Aufnahme eines String ist, dann muss man noch +1 zur Länge des Strings dazu addieren.



12.10 Weitere Funktionen in <string.h>

Die verschiedenen Implementationen der Standardbibliothek (beispielsweise POSIX, GNU, BSD, ...) boten über die Jahre noch weitere Funktionen an. Diese als Extensions (Erweiterungen) bezeichneten Funktionen werden in diesem Buch nicht weiter beschrieben.

Nennenswert sind jedoch die Funktionen, welche in dem kommenden Standard C23 eingeführt werden. Sie werden hier kurz aufgelistet:

```
void* memset_explicit(void* dest, int ch, size_t count)
```

Stellt sicher, dass sämtliche Speicherinhalte mit einem gegebenen Zeichen überschrieben werden.

```
void* memccpy(void* restrict dest, const void* restrict src, int  
c, size_t count )
```

Kopieren von nicht-überlappendem Speicher bis zum Auftreten eines gegebenen Zeichens.

```
char* strdup(const char* str1)
```

Duplizieren eines Strings mit Speicherallokation.

```
char* strndup(const char *src, size_t size)
```

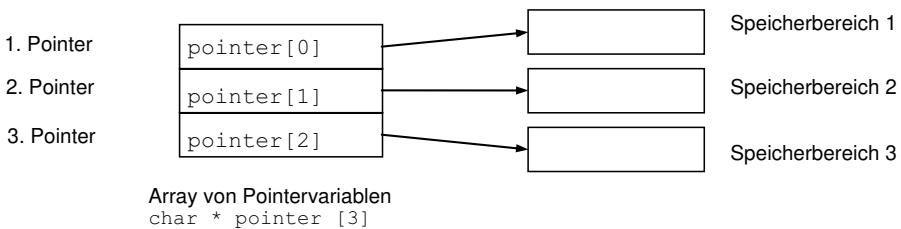
Duplizieren eines Strings mit einer gegebenen Länge mit Speicherallokation.

12.11 Beispiele für Arrays und Pointer

Um die Inhalte dieses Kapitels zu vertiefen, werden hier weitere Beispiele für Arrays und Pointer aufgeführt.

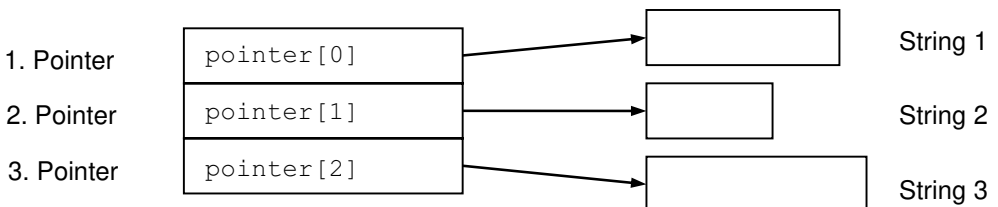
12.11.1 Beispiel für eindimensionale Arrays von Pointern

Ein Pointer ist eine Variable, in der die Adresse eines anderen Speicherobjektes (Variable, Funktion) gespeichert ist. Entsprechend einem eindimensionalen Array von gewöhnlichen Variablen kann natürlich auch ein eindimensionales Array von Pointer-Variablen gebildet werden, wie im folgenden Beispiel eines eindimensionalen Arrays aus 3 Pointern auf char zu sehen ist:



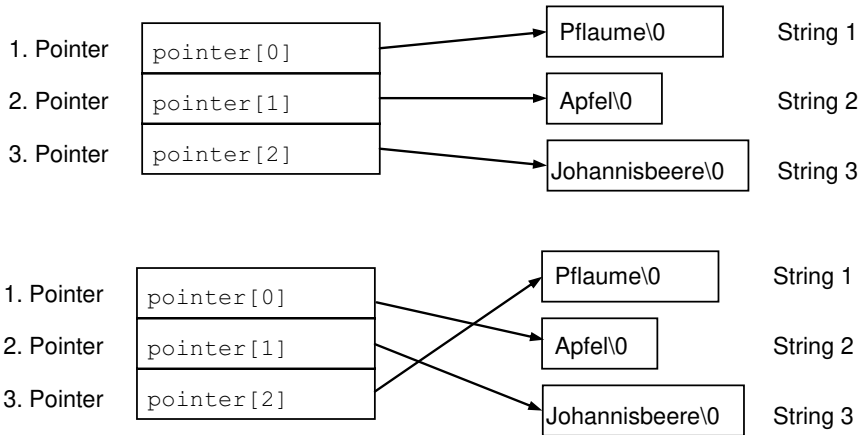
Arbeitet man mit einem String fester Länge, so legt man ein Zeichenarray einer festen Größe an, wie beispielsweise `char a[20]`. Ist die Länge eines Strings von vornherein jedoch nicht fest definiert, so verwendet man meist einen Pointer wie beispielsweise `char* pointer` und lässt den Pointer auf den String zeigen. Arbeitet man mit mehreren Strings, deren Länge nicht von vornherein bekannt ist, so verwendet man ein eindimensionales Array von Pointern auf char.

Im folgenden Beispiel stellt `char* pointer[3]` ein eindimensionales Array von drei Pointern auf char dar, die auf drei Strings zeigen:



Will man beispielsweise diese Strings sortieren, so muss dies nicht mit Hilfe von aufwendigen Kopieraktionen für die Strings durchgeführt werden. Es werden lediglich die Pointer so verändert, dass die geforderte Sortierung erreicht wird.

Die beiden folgenden Bilder zeigen die Pointer vor und nach dem Sortieren der Strings:



Ein weiteres Beispiel ist die folgende Funktion, welche zeilenweise einen Text auf dem Bildschirm ausgibt:

```
void textausgabe(char* textPointer[], int anzZeilen) {  
    for (size_t i = 0; i < anzZeilen; ++i)  
        printf("%s\n", textPointer[i]);  
}
```

In Kapitel [→ 12.3.2](#) wurde erläutert, dass der formale Parameter für die Übergabe eines Arrays in der Notation eines Arrays ohne Längenangaben geschrieben werden kann. Damit ist die Notation `char* textPointer[]` verständlich. Die Angabe `[]` ist jedoch äquivalent zu einem Pointer. Somit könnte man alternativ als aktuellen Parameter `char** textPointer` schreiben, also ein Pointer auf einen Pointer.

12.11.2 Beispiel für Pointer auf Pointer

Die Variable `pointer` aus Kapitel [→ 12.11.1](#) ist ein eindimensionales Array aus drei Elementen. Jedes Element ist ein Pointer auf einen `char`-Wert. Wird einer dieser Pointer dereferenziert, beispielsweise durch `*pointer[i]`, so erhält man das erste Zeichen dieses Strings.

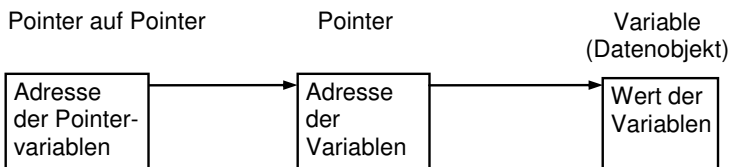
Im Folgenden soll nun das Beispiel etwas anders formuliert werden:

```
void textausgabe(char** textPointer, int anzahlZeilen) {  
    while (anzahlZeilen-- > 0)  
        printf("%s\n", *textPointer++);  
}
```

Die beiden Schreibweisen `char** textPointer` und `char* textPointer[]` sind bei formalen Parametern gleichwertig.



Das folgende Bild soll einen Pointer auf einen Pointer veranschaulichen:



Folgender Aufruf der oben gezeigten Funktion `textausgabe()` demonstriert die Funktionsweise von Pointern auf Pointer:

```
int main(void) {  
    char* zeilen[5] = {  
        "Mein",  
        "Computer",  
        "kennt",  
        "Else",  
        "nicht"  
    };  
    textausgabe(&zeilen[1], 3);  
    return 0;  
}
```

Der formale Parameter `char** textPointer` bekommt beim Aufruf der Funktion `textausgabe()` als aktuellen Parameter eine Adresse des ersten Elements eines Arrays übergeben. In diesem Beispiel ist dieses Array das Element `zeilen[1]`, wo ein Pointer auf den String "Computer" gespeichert ist. Der derererenzierte Pointer `*textPointer` wird der Funktion `printf()` übergeben, welche einen Pointer auf `char` erwartet. Gleichzeitig wird mit dem Inkrement-Operator die Variable `textPointer` um eine Einheit erhöht. Somit zeigt `textPointer` nach dem ersten Durchgang der Schleife auf das Element `zeilen[2]`, wo ein Pointer auf den String "kennt" gespeichert ist. Die Schleife wird solange fortgesetzt, bis 3 Strings ausgegeben sind.

Der vollständige Programmcode befindet sich in der Datei `elsescomputer.c`. Die Ausgabe des Programmes lautet:

```
Computer
kennt
Else
```

12.11.3 Beispiel für die Initialisierung von Arrays von Pointern

Das folgende Beispiel zeigt eine Funktion, die bei der Übergabe eines Fehlercodes einen Pointer auf den entsprechenden Fehlertext zurückliefert:

fehler.c

```
#include <stdio.h>

const char* fehlertext(int n) {
    static const char* errDesc[] = {
        "kein Fehler",
        "Fehlertext Eins",
        "Fehlertext Zwei",
        "Fehlertext Drei"};

    return (n < 0 || n > 3) ? "Fehlercode existiert nicht" :
        errDesc[n];
}

int main(void) {
    int fehlernummer = 2;
    printf("Text zu Fehler %d: %s\n",
        fehlernummer, fehlertext(fehlernummer));
    return 0;
}
```

Ein Beispiel für einen Programmlauf ist:

Text zu Fehler 2: Fehlertext Zwei

Die Funktion `main()` dient hier nur zu Testzwecken. In einem richtigen Programm wird die Funktion `fehlertext()` zur Laufzeit im Fehlerfall von anderen Funktionen aufgerufen, welche die Fehlernummer übergeben und den Fehlertext zurückerhalten, um ihn auszugeben.

Das Array von Pointern auf `const char* errDesc[]` muss als `static` angelegt werden, da hier eine lokale Pointer-Variable aus der Funktion zurückgegeben wird (siehe auch Kapitel [→ 15.4](#)). Ohne `static` wäre der Rückgabewert der Funktion nach dem Funktionsende undefiniert, da nach Ablauf einer Funktion ihre lokalen Variablen ungültig werden. Mit dem Schlüsselwort `static` bleiben die Fehlertexte als interne Variablen über die gesamte Laufzeit des Programms permanent im Speicher.

12.11.4 Vergleich zweidimensionales gegen eindimensionales Array von Pointern

Der Unterschied zwischen einem eindimensionalen Array von Pointern und einem zweidimensionalen Array ist, dass bei mehrdimensionalen Arrays die Anzahl der Elemente fest vorgegeben ist, bei eindimensionalen Arrays von Pointern hingegen nur die Anzahl an Pointern.



So definiert im folgenden Beispiel die erste Zeile ein Array mit insgesamt 50 `int`-Elementen und die zweite Zeile ein eindimensionales Array von 5 Pointern auf `int`:

```
int array2D      [5][10];  
int* pointerArray [5];
```

Der Vorteil des Arrays aus Pointern `pointerArray` besteht darin, dass die „2-te Dimension“ der einzelnen Elemente des Arrays unterschiedlich groß sein kann. Das heißt im Gegensatz zum `array2D` muss nicht jedes Element 10 `int`-Werte haben.

Die häufigste Anwendung besteht deshalb darin, ein Array unterschiedlich langer Strings zu bilden wie im Falle der Funktion `fehlertext()` in Kapitel [→ 12.11.3](#):

```
char* errDesc1[] = {  
    "Fehlercode existiert nicht",    // 27 Bytes  
    "Fehlertext 1",                // 13 Bytes  
    "Fehlertext 2" };              // 13 Bytes
```

Mit dieser Definition werden insgesamt 53 Bytes für die Zeichen des Strings benötigt. Zusätzlich benötigen die Pointer noch einige Bytes.

Zum Vergleich:

```
char errDesc2[][27] = {  
    "Fehlercode existiert nicht",    // 27 Bytes  
    "Fehlertext 1",                // 27 Bytes  
    "Fehlertext 2" };              // 27 Bytes
```

Hier muss mindestens die Anzahl an Elementen reserviert werden, die der längste String benötigt. Dies sind 27 Bytes für "Fehlercode existiert nicht". Die restlichen Strings benötigen zwar nur je 13 Zeichen, für sie sind aber ebenfalls 27 Zeichen reserviert. Somit benötigt `errDesc2` insgesamt 81 Bytes

Die Einsparung an Speicherplatz zeigt sich eindrucksvoll, wenn beispielsweise eine Maximallänge von 255 Zeichen festgelegt wird, die gespeicherten Strings jedoch durchschnittlich nur rund 10 Zeichen benötigen. Werden in einer solchen Struktur 1 Million Strings gespeichert, so benötigt das zweidimensionale Array 255 Millionen Bytes, das Pointer-Array jedoch gerade mal deren 10 Millionen.

12.12 Pointer auf Funktionen

Pointer auf Funktionen sind ein spezieller Pointer-Typ, welcher in einer Variablen die Adresse einer aufzurufenden Funktion speichern kann.

Über einen Pointer können Funktionen auch als Parameter an andere Funktionen übergeben werden.



Damit kann von Aufruf zu Aufruf eine unterschiedliche Funktion als Argument übergeben werden. Ein bekanntes Beispiel für die Anwendung von Pointern auf Funktionen ist die Übergabe unterschiedlicher Funktionen an eine Sortierfunktion, um beispielsweise aufsteigend oder absteigend zu sortieren.

Ein weiteres Einsatzgebiet von Pointern auf Funktionen ist die Programmierung von Interrupts. Eine Interrupt-Behandlung wird ermöglicht, indem man in die Interrupt-Tabelle des Betriebssystems den Pointer auf die Interrupt-Service-Routine (ISR) schreibt. Bei einem Interrupt wird dann die entsprechende Interrupt-Service-Routine aufgerufen.

Das folgende Bild zeigt eine Interrupt-Tabelle:

Pointer auf ISR n
...
...
Pointer auf ISR 2
Pointer auf ISR 1

12.12.1 Vereinbarung eines Pointers auf eine Funktion

Ein Funktionsname bezeichnet in C den Adressbereich, in welchem eine Funktion gespeichert ist. Durch Vereinbarung eines Pointers auf eine Funktion wird die Adresse der ersten Anweisung (genauer gesagt, des ersten Maschinenbefehls) einer Funktion gespeichert.



Die Vereinbarung eines Funktionspointers sieht auf den ersten Blick etwas kompliziert aus, wie im folgenden Beispiel:

```
int (*ptr)(char);
```

ptr ist hierbei ein Pointer auf eine Funktion mit einem Rückgabewert vom Typ int und einem Übergabeparameter vom Typ char. Das erste Klammernpaar ist unbedingt nötig, da es sich sonst an dieser Stelle um einen gewöhnlichen Funktions-Prototypen handeln würde. Infolge der Klammern muss man lesen „ptr ist ein Pointer auf“. Dann kommen entsprechend der Operatorpriorität die runden Klammern. Also ist „ptr ein Pointer auf eine Funktion mit einem Übergabeparameter vom Typ char“. Als letztes wird das int gelesen, das heißt die Funktion hat den Rückgabotyp int.

Wie funktioniert aber nun das Arbeiten mit einem Pointer auf eine Funktion? Dem noch nicht gesetzten Pointer auf eine Funktion muss hierzu eine Adresse einer bekannten Funktion desselben Typs zugewiesen werden, beispielsweise so:

```
ptr = &funktionsname;
```

Der Standard lässt bei der Ermittlung der Adresse einer Funktion auch zu, direkt den Funktionsnamen hinzuschreiben, ohne den Adress-Operator zu verwenden, da ein Funktionsname ähnlich wie eine Array-Variable bei Auftreten im Code automatisch in seine Adresse ausgewertet wird.

```
ptr = funktionsname;
```

12.12.2 Aufruf einer Funktion

Da nun `ptr` (Pointer auf eine Funktion) die Adresse der Funktion `funktionsname()` enthält, kann der Aufruf der Funktion auch durch die Dereferenzierung des Pointers erfolgen. Man beachte im folgenden Beispiel die Klammerung um den dereferenzierten Pointer:

```
int a;  
a = funktionsname('A');  
  
ptr = &funktionsname;  
a = (*ptr)('A');
```

Im diesem Beispiel sind beide Zuweisungen zur Variablen `a` somit equivalent.

C erlaubt es zudem auch, die Klammerung um die Dereferenzierung wegzulassen, sodass der Funktionspointer wie ein Funktionsname benutzt werden kann:

```
a = ptr('A');
```

12.12.3 Beispiel für Pointer auf eine Funktion

Pointer auf Funktionen werden normalerweise benutzt, um sie als Parameter an Funktionen zu übergeben, die dann über den Pointer auf eine gewünschte Funktionalität zugreifen, welche innerhalb der aufgerufenen Funktion sonst nicht verfügbar wäre. Manche Problemstellungen lassen sich durch den Einsatz von Pointern auf Funktionen elegant lösen.

Im folgenden Programm ist die Funktion `evalTime()` durch die Übergabe eines Pointers auf eine Funktion in der Lage, die Durchlaufzeit jeder übergebenen Funktion passenden Typs elegant zu berechnen, hier in diesem Beispiel die Zeit bis zum Drücken der <RETURN>-Taste:

functionPointer.c

```
#include <stdio.h>  
#include <time.h>  
  
void f1(void) {  
    printf("Druecken Sie bitte Enter:");  
    getchar();  
}
```



```
double evalTime(void (*ptr)(void)) {
    time_t begin, end;
    begin = time(NULL);
    ptr();
    end = time(NULL);
    return difftime(end, begin);
}

int main(void) {
    printf("Zeit bis Enter gedrueckt wurde: %1.0f sec\n",
        evalTime(f1));
    return 0;
}
```

Hier ein Beispiel für die Ausgabe des Programms:

```
Druecken Sie bitte Enter:
Zeit bis Enter gedrueckt wurde: 2 sec
```

Die Funktion `time()` liefert als Rückgabewert die aktuelle Kalenderzeit in einer Darstellung, die vom Compiler abhängig ist. Die Funktion `difftime()` berechnet die Zeit zwischen zwei Zeitangaben in Sekunden. Beide Funktionen sind in der Bibliothek `<time.h>` enthalten, vergleiche Kapitel [→ 13.1.10](#).

Es ist nun vorstellbar, dass an diese Funktion jeder beliebige Funktionspointer mit der richtigen Signatur übergeben werden kann. So ist es ganz einfach möglich, die Laufzeit beispielsweise einer komplizierten Berechnung zu messen.

Ein weiteres Beispiel für Pointer auf Funktionen im Zusammenspiel mit der Bibliotheksfunktion `qsort()` ist in Kapitel [→ 20.1.6](#) zu finden.

12.13 Pointer auf void für Strukturen



Für dieses Unterkapitel lohnt es sich, erst die Strukturen in Kapitel [→ 13](#) nachzulesen.

In diesem Kapitel sowie im Kapitel [→ 8.2](#) wurde bereits mehrfach auf void-Pointer hingewiesen. Es handelt sich um einen Pointer auf einen unbestimmten Typ. So wurde der Typ `void*` bereits als Parameter oder Rückgabewert einer Funktion verwendet, um auf Speicherbereiche wie Arrays oder Strings zuzugreifen.

Hier in diesem Unterkapitel soll nun ein fortgeschrittenes Beispiel gezeigt werden, bei welchem ein void-Pointer auf eine sogenannte „Struktur“ zeigt. Grob umrissen speichert eine Struktur mehrere Attribute eines Objektes in einem zusammengefassten Datentyp. Mittels des void-Pointer können solche Strukturen adressiert werden, ohne die genaue Zusammensetzung der Strukturen kennen zu müssen.

Es werden zwei Fahrzeugtypen deklariert: Personenkraftwagen (PKW) und Lastkraftwagen (LKW):

enums.h

```
enum Fahrzeugtyp {
    TYP_PKW,
    TYP_LKW,
};

struct Auto {
    enum Fahrzeugtyp fahrzeugtyp;
    char             marke[20];
    double           maxv;
};

struct Lastwagen {
    enum Fahrzeugtyp fahrzeugtyp;
    int              achsen;
    double           gewicht;
};
```

Beide Typen haben als erstes Feld eine Typkennung, unterscheiden sich jedoch in allen restlichen Feldern. Es wird nun eine Funktion definiert, die als Argument einen Pointer auf einen dieser beiden Fahrzeugtypen enthält:

printInfo.c

```
#include "enums.h"
#include <stdio.h>

void printInfo(const void* arg) {
    switch (*(enum Fahrzeugtyp*) arg) {        // (1)

    case TYP_PKW:
        printf("Automarke %s mit %f km/h\n",
            ((struct Auto*)arg)->marke,
            ((struct Auto*)arg)->maxv);
        break;

    case TYP_LKW:
        printf("Lastwagen %d-Achser mit %ft\n",
            ((struct Lastwagen*)arg)->achsen,
            ((struct Lastwagen*)arg)->gewicht);
        break;

    }
}
```

Damit beide Typen an die Funktion übergeben werden können, wird das Argument als void-Pointer vereinbart.

Um innerhalb der Funktion zu unterscheiden, welcher der beiden Struktur-Typen tatsächlich vorliegt, wird das erste Feld des übergebenen Parameters in der Zeile mit dem Kommentar (1) abgefragt. Da bei beiden Struktur-Typen Auto und Lastwagen das Feld `fahrzeugtyp` als das erste Feld der Struktur deklariert ist, zeigt der Pointer `arg` auf eben diesen Fahrzeugtyp, egal, um welchen Struktur-Typ es sich bei dem aktuellen Funktionsaufruf handelt. So kann der Parameter `arg` einfach in einen Pointer des Typs `enum Fahrzeugtyp*` gecastet und mittels des Dereferenzierungs-Operators angesprochen werden.

Aufgrund des im ersten Feld eingetragenen Fahrzeugtyps wird zum entsprechenden case-Fall gesprungen und die gewünschten Informationen zu der übergebenen Struktur ausgegeben.

In der Funktion `main()` können nun beliebige Struktur-Variablen vereinbart werden und mittels des Adress-Operators an die Funktion `printInfo()` übergeben werden:

main.c

```
#include "enums.h"

void printInfo(const void* arg);

int main(void) {
    struct Auto      meinAuto   = {TYP_PKW, "Trabbi", 180.};
    struct Lastwagen meinBrummi = {TYP_LKW, 10, 40.};
    printInfo(&meinAuto);
    printInfo(&meinBrummi);
    return 0;
}
```

Die Ausgabe dieses Programmes lautet

```
Automarke Trabbi mit 180.000000 km/h
Lastwagen 10-Achser mit 40.000000t
```

Bei der Initialisierung der Variablen muss unbedingt darauf geachtet werden, dass der Fahrzeugtyp (die Kennung) korrekt gesetzt wird.

Innerhalb der Funktion `printInfo()` könnten zu Beginn auch zwei Pointer vereinbart werden. Damit würde der formale `void`-Pointer `arg` zwei lokalen Variablen zugewiesen, welche den statischen Typ `struct Auto*` und `struct Lastwagen*` besitzen. Natürlich wäre nur jeweils eine Zuweisung auch tatsächlich sinnvoll. Aufgrund dieser lokalen Variablen könnten die Elemente der übergebenen Struktur im Folgenden ganz einfach ohne ständiges Casten angesprochen werden:

```
struct Auto*      pkw = (struct Auto*)arg;
struct Lastwagen* lkw = (struct Lastwagen*)arg;
...
case TYP_PKW:
    printf("Automarke %s mit %f km/h\n", pkw->marke, pkw->maxv);
    break;
case TYP_LKW:
    printf("Lastwagen %d-Achser mit %ft\n", lkw->achsen, lkw->gewicht);
    break;
```

Es ist hierbei zu beachten, dass die Variable `lkw` im Falle `TYP_PKW` und die Variable `pkw` im Falle `TYP_LKW` jeweils nicht verwendet wird.

Aufgrund der besseren Lesbarkeit wird diese Methode häufig angewendet.

12.14 Übungsaufgaben

Aufgabe 1: Arrays in C

Geben Sie an, was das folgende Programm auf dem Bildschirm ausgibt, ohne das Programm auszuführen.

array.c

```
#include <stdio.h>

int main(void) {
    char* text = "Dichter und Denker";
    char spruch[20] = "alles weise Lenker";
    printf("\n");
    for (int i = 1; i <= 3; ++i) printf("%c", text[i]);
    printf("%c", spruch[5]);
    spruch[9] = 's';
    spruch[10] = 's';
    spruch[11] = spruch[5];
    spruch[12] = '\0';
    printf("%s", spruch + 6);
    *(spruch + 5) = '\0';
    printf("%s\n", spruch);
    return 0;
}
```

Aufgabe 2: memcpy und memmove

Studieren Sie die Kommentare in Kapitel [→ 12.8.2](#) zum Unterschied zwischen memcpy und memmove.

- Schreiben sie Ihre eigene Version von memcpy mittels einer einfachen for-Schleife.
- Testen Sie Ihre Version mit folgendem Code:

```
char string1[] = "12345678";  
myMemcpy(string1 + 2, string1, strlen(string1) - 2);
```

Das Resultat sollte sein 12121212

- Schreiben Sie nun Ihre eigene Version von memmove. Testen Sie Ihre Version mit diesem Code:

```
char string2[] = "12345678";  
myMemMove(string2 + 2, string2, strlen(string2) - 2);
```

Das Resultat sollte nun lauten: 12123456

Aufgabe 3: Pointer und Arrays bei Strings

In Kapitel [→ 12.7](#) wurde die strcpy()-Funktion in eigenem Code verfasst. Nebst strcpy() wurde jedoch auch die Funktion strncpy() vorgestellt. Schreiben Sie jetzt Ihre eigene Version der Funktion strncpy().

Aufgabe 4: Übergabe von char-Arrays

Schreiben Sie eine C-Funktion stringIndex(), die ein bestimmtes Zeichen in einem String sucht und den Index des ersten Auftretens im String bestimmt.

Die Funktion stringIndex() soll zwei Übergabeparameter für den String und das Zeichen enthalten.

Wird das Zeichen nicht gefunden, soll die Funktion -1 zurückgeben.

Stellen Sie sicher, dass die Funktion const-safe ist.

Aufgabe 5: String-Verarbeitung

Implementieren Sie eine Funktion mit dem folgenden Funktionskopf:

```
void isHexadezimal(const char* string)
```

Die Funktion `isHexadezimal()` soll für jedes Zeichen eines gegebenen Strings (beispielsweise "aBr12") eine Ausgabe in der folgenden Form erstellen:

```
a: Ist eine Hex-Ziffer  
B: Ist eine Hex-Ziffer  
r: Ist keine Hex-Ziffer  
t: Ist keine Hex-Ziffer  
1: Ist eine Hex-Ziffer  
2: Ist eine Hex-Ziffer
```

Aufgabe 6: Suche nach Substring

Programmieren Sie eine Funktion mit dem folgenden Prototyp:

```
int check(const char* str1, const char* str2);
```

Die Funktion `check()` gibt die erste Position in `str1` an, an der der Teilstring `str2` im String `str1` beginnt. Tritt `str2` in `str1` nicht auf, so wird `-1` zurückgegeben.

Nutzen Sie im Rahmen dieser Übung die `<string.h>`-Bibliothek nicht!

Hinweis: Verpacken Sie zu Beginn nicht alles in eine einzige Funktion. Versuchen Sie, Teilprobleme wie beispielsweise die Ermittlung der Länge eines Strings in eigene Funktionen zu verpacken.

Hinweis: Sie können die Aufgabe mit Index-Berechnung oder mit Pointer-Arithmetik lösen.

13 Strukturen, Unionen und Bitfelder



Programme verarbeiten häufig **Datenstrukturen**, welche in sich eine Semantik enthalten. Beispielsweise könnte ein Programm für die Personalabteilung für jede Person einen Eintrag mit den folgenden Elementen speichern:

Personal-nummer	Nachname	Vorname	Straße	Haus-nummer	Postleit-zahl	Wohnort	Gehalt
-----------------	----------	---------	--------	-------------	---------------	---------	--------

Die einzelnen Elemente können beispielsweise die folgenden Typen haben:

int	char[20]	char[20]	char[20]	int	int	char[20]	float
-----	----------	----------	----------	-----	-----	----------	-------

Dies wird als eine zusammengesetzte Datenstruktur, oder in C schlicht als „**Struktur**“ bezeichnet. Eine solche Struktur enthält unterschiedliche Typen und fasst sie unter einem einzigen Typ zusammen, welcher eine durch die Teile festgelegte Anzahl an Bytes besitzt.

Eine Struktur enthält semantisch zusammengehörige Daten in einem zusammengesetzten Datentyp.



Diese Art der Datenstruktur konnte man bereits in COBOL definieren. Als ein eigenständiger Datentyp wurde sie dann von N. Wirth in Pascal eingeführt und dort „Record“ genannt. Der Name Record wurde nach dem Satz einer Datei auf der Festplatte gewählt, welcher im Englischen ebenfalls „record“ genannt wird. Ein solcher „record“ auf der Platte konnte Komponenten verschiedener Typen enthalten.

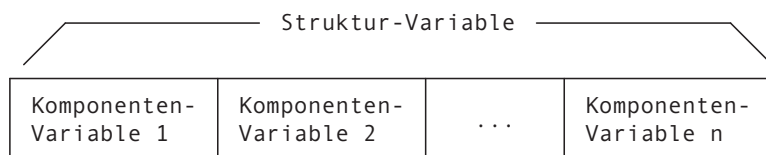
In Anlehnung an den Plattensatz werden die Komponenten eines Records oder einer Struktur oft als „**Feld**“ beziehungsweise „Datenfeld“ bezeichnet. Vor allem in der Java-Literatur wird der Begriff „Datenfeld“ bevorzugt, während in C jedoch auch die Begriffe „Komponente“ oder gerade in C++ auch „**Member**“ gängig sind.

Nebst Strukturen gibt es auch noch andere zusammengesetzte Typen wie Unionen und Bitfelder. Auf diese wird in den folgenden Unterkapiteln ebenfalls eingegangen, sie werden jedoch in der modernen Programmierung äußerst selten verwendet.

Ergänzende Information Die elektronische Version dieses Kapitels enthält Zusatzmaterial, auf das über folgenden Link zugegriffen werden kann https://doi.org/10.1007/978-3-658-45209-4_13.

13.1 Strukturen

In der Programmiersprache C werden zusammengehörige Variablen in einer sogenannten „**Struktur**“ zusammengefasst. Eine solche Struktur wird mittels eines sogenannten „Struktur-Typs“ definiert und die daraus resultierende Variable wird „Struktur-Variable“ genannt.



Ein Struktur-Typ ist ein selbst definierter zusammengesetzter Datentyp, welcher aus einer festen Anzahl von Komponenten besteht, die jeweils einen Namen haben. Um einen Struktur-Typ festzulegen, wird das Schlüsselwort **struct** verwendet. Die allgemeine Form für einen Struktur-Typ lautet:

```
struct Strukturname {  
    KomponenteTyp1 komponente1;  
    KomponenteTyp2 komponente2;  
    ...  
    KomponenteTypN komponenteN;  
};
```

Im Unterschied zu den Komponenten eines Arrays können die Komponenten einer Struktur verschiedene Typen haben.

In der Typ-Definition einer Struktur muss für jede Komponente deren Namen und Typ angegeben werden.



Der hier selbst definierte Typ-Bezeichner ist `struct Strukturname`, wobei `struct` ein Schlüsselwort ist und `Strukturname` als das sogenannte „**Etikett**“ der Struktur (auf englisch „**structure tag**“) bezeichnet wird. Dieser Datentyp ist definiert durch den Inhalt der geschweiften Klammern.

Hier ein einfaches Beispiel für eine Struktur:

```
struct Adresse {  
    char strasse[20];  
    int  hausnummer;  
    int  postleitzahl;  
    char stadt[20];  
};
```

Hier ist zu beachten, dass alle Namen der Komponenten einer Struktur verschieden sein müssen. In verschiedenen Strukturen dürfen jedoch Komponenten gleichen Namens auftreten, da jede Struktur einen eigenen Namensraum hat (siehe Kapitel [→ 14.3](#)). Im Beispiel ist auch zu sehen, dass Strukturen ganze Arrays beinhalten können.

Ein weiteres Beispiel:

```
struct Student {  
    int matrikelnummer;  
    char name[20];  
    char vorname[20];  
    struct Adresse wohnort;  
};
```

Hier wurde nun eine Struktur deklariert, bei welcher die Komponente wohnort wiederum vom Typ einer Struktur ist. Die Definition des Struktur-Typs struct Adresse muss dabei zwingend seiner Verwendung im Datentyp struct Student vorausgehen.

Um Variablen mit diesen Struktur-Typen zu definieren, schreibt man folgendes:

```
struct Adresse zuhause;  
struct Student meyer;  
struct Student studenten[50]; // ein Array aus 50 Studenten
```

Hierbei wird ersichtlich, dass das Schlüsselwort struct zum Typ dazugehört. Ohne das Schlüsselwort würde der Compiler das Etikett nicht finden.

Um die Komponenten einer Struktur-Variablen anzusprechen, werden sie nicht wie ein Array mittels eines Index, sondern mittels des **Punkt-Operators** und der Angabe des gewünschten Feldnamens dereferenziert:

```
meyer.matrikelnummer = 716347;
```

Dabei wird der Punkt-Operator sowohl für lesenden wie auch schreibenden Zugriff verwendet. Hier wurde die Komponente `matrikelnummer` mit dem Wert 716347 belegt.

Durch mehrfaches Verwenden des Punkt-Operators können direkt auch Komponenten von Komponenten angesprochen werden:

```
meyer.wohnort.postleitzahl = 73733;
```

Bei mehrfach zusammengesetzten Strukturen kann man über das mehrfache Anwenden des Punkt-Operators auf geschachtelte Komponenten zugreifen.



Einzelheiten zum Punkt-Operator folgen in Kapitel [→ 13.1.7](#).

13.1.1 Definition von Struktur-Variablen

Es ist möglich, gleichzeitig den Typ einer Struktur festzulegen sowie auch Variablen dieses neuen Typs zu definieren. Beispielsweise:

```
struct Student {  
    ...  
} tina;
```

Hier wird sowohl der Typ `struct Student` festgelegt als auch eine Variable `tina` vom Typ dieser Struktur definiert. Weitere Variablen dieses Typs können später über den Typnamen `struct Student` definiert werden.

Das Etikett der Struktur kann auch weggelassen werden. Dies ist jedoch nur dann sinnvoll, wenn sofort alle Variablen eines Typs definiert werden, da ohne ein Etikett später keine Variablen dieses Typs mehr vereinbart werden können.

```
struct {  
    ...  
} heinz, heinrich, anita;
```

Man beachte, dass der Name eines Etiketts, der Name einer Komponente und der Name einer Struktur-Variablen identisch sein dürfen, denn sie liegen in verschiedenen Namensräumen (siehe Kapitel [→ 14.3](#)). Dies macht dem Compiler keine Probleme, da er aus dem Kontext schließt, um welche Größe es sich handelt. Eine solche Namensgleichheit ist jedoch dennoch nicht zu empfehlen, da auch Menschen den Code lesen müssen, was leicht zu Missverständnissen führen kann.

13.1.2 Initialisierung einer Struktur mit einer Initialisierungsliste

Eine Initialisierung einer Struktur-Variablen kann direkt bei der Definition der Struktur-Variablen mit Hilfe einer **Initialisierungsliste** durchgeführt werden.



Dies zeigt das folgende Beispiel:

```
struct Student mueller =  
{  
    66202,  
    "Mueller",  
    "Herbert",  
    {  
        "Schillerplatz",  
        20,  
        73730,  
        "Esslingen"  
    }  
};
```

Die Initialisierungsliste enthält die Werte für die einzelnen Komponenten getrennt durch Kommas. Da die Komponenten-Variable wohnort selbst eine Struktur ist, erfolgt die Initialisierung der Komponenten-Variable wohnort wieder über eine Initialisierungsliste.

Array- und Struktur-Typen werden in C auch als Aggregat-Typen bezeichnet. Ein Aggregat-Typ ist ein anderes Wort für „zusammengesetzter Typ“. Wegen dieser Gemeinsamkeit erfolgt die Initialisierung von Strukturen und Arrays analog.



Struktur-Variablen können auch durch die Zuweisung einer Struktur-Variablen des gleichen Typs oder durch einen beliebigen Ausdruck, der zu einer Struktur-Variablen auswertet (beispielsweise ein Funktionsaufruf), initialisiert werden. In C11 kann eine Struktur-Variable auch mittels eines sogenannten compound literals (siehe Kapitel [→ 13.2.2](#)) initialisiert werden.

13.1.3 Initialisierung einzelner Komponenten einer Struktur

Seit C99 können einzelne Komponenten einer Struktur-Variablen initialisiert werden. Dies wird im Englischen als „**designated initializer**“ bezeichnet.



Beispielsweise kann eine Variable vom Typ struct Student wie im folgenden Beispiel initialisiert werden:

```
struct Student petra = {  
    .matrikelnummer = 4711,  
    .vorname = "Petra"  
};
```

Man muss dabei nicht auf die Reihenfolge der Komponenten achten. Man hätte auch schreiben können:

```
struct Student petra = {  
    .vorname = "Petra",  
    .matrikelnummer = 4711  
};
```

Werden die Elemente einer Struktur in der gleichen Reihenfolge initialisiert, wie sie bei der Definition des Typs angegeben wurden, so muss man den Elementnamen nicht angeben wie im folgenden Beispiel:

```
struct Student petra = {  
    4711,          // Initialisierung der Matrikelnummer  
    "Welsch",     // Initialisierung des Namens  
    "Petra"       // Initialisierung des Vornamens  
};
```

Unvollständige Initialisierungen und Initialisierungen ohne Angabe des Elementnamens sind eine große Fehlerquelle und führen zu Unleserlichkeit. Das zu initialisierende Element sollte stets angegeben werden. Unvollständige Initialisierungen sollten vermieden werden.



Will man auf eine bestimmte Komponente zugreifen und anschließend in derselben Reihenfolge weiterinitialisieren, so braucht man die Namen der folgenden Komponenten nicht anzugeben, wie im folgenden Beispiel:

```
struct Student petra = {  
    .name = "Welsch", // Initialisierung des Namens  
    "Petra"          // Initialisierung des Vornamens  
};
```

Werden nicht alle Komponenten initialisiert, so werden die nicht aufgeführten Elemente automatisch mit null initialisiert.



Hierfür eine detaillierte Aufstellung:

```
struct Student petra = {  
    // Initialisierung der Matrikelnummer mit 0  
    .name = "Welsch", // Initialisierung des Namens  
    .vorname = "Petra" // Initialisierung des Vornamens  
    // Initialisierung der Adresse: Der Compiler  
    // schreibt an die Speicherstelle der  
    // Struktur genau sizeof(struct Adresse) Nullen.  
};
```

13.1.4 Strukturen und Pointer auf Strukturen in Strukturen

Wie im Beispiel gezeigt, kann eine Struktur selbst wiederum Strukturen beinhalten: Ein Student besitzt eine Adresse. Damit die Struktur Student jedoch definiert werden kann, muss die Struktur Adresse bereits definiert sein. Es ist somit erforderlich, dass die Definition von Adresse vor Student kommt.

Wird hingegen anstelle der Struktur selbst nur ein **Pointer auf die Struktur** verwendet, so benötigt der Compiler nur den Namen der Struktur.

```
struct Adresse* wohnort;
```

Für die Definition eines Pointers auf eine Struktur müssen die Größe und der Aufbau der Struktur nicht bekannt sein, da alle Pointer-Typen dieselbe Anzahl Bytes beinhalten. Damit kann in der Definition einer Struktur eine Pointer-Variable als Komponente definiert werden, die auf eine Variable vom Typ der Struktur zeigt.



Die Verwendung von Pointern auf Strukturen wird insbesondere bei dynamischen Datenstrukturen verwendet. Siehe dazu Kapitel [→ 19](#).

Es ist zu beachten, dass, obschon die Struktur selbst nicht definiert sein muss, die **Vorwärtsdeklaration** des Namens der Struktur zwingend notwendig ist:

```
struct Adresse;  
  
struct Student {  
    ...  
    struct Adresse* wohnort;  
};  
  
struct Adresse {  
    ...  
};
```

Demzufolge darf eine Struktur somit auch nur einen Pointer auf sich selbst beinhalten, niemals sich selbst. Folgender Code erzeugt somit einen Fehler:

```
struct Test {  
    struct Test testStruktur; // Fehler  
};
```

Dies würde voraussetzen, dass im Definitionsteil von struct Test die Definition von struct Test bereits bekannt ist, was nicht der Fall ist, da die Definition noch nicht abgeschlossen ist. Außerdem würde dies zu einer Rekursion ohne Abbruchkriterium führen und somit eine Struktur unendlicher Größe entstehen.

13.1.5 String-Variablen in Strukturen

String-Variablen in Strukturen können char-Arrays oder Pointer-Variablen vom Typ char* sein.



Dies zeigt das folgende Beispiel:

```
struct Name {  
    char name[20];  
    char* vorname;  
};
```

Im Falle des char-Arrays „gehören“ alle Zeichen zu der entsprechenden Struktur-Variablen. Im Falle der Pointer-Variablen vom Typ char* steht in der entsprechenden Komponente nur ein Pointer auf einen String, das heißt auf ein char-Array, das sich nicht in der Struktur selbst befindet.

In beiden Fällen kann die Initialisierung mit String-Konstanten erfolgen:

```
struct Name maier = {"Maier", "Herbert"};
```

Bei Änderungen des Strings ist im Falle, dass die Komponente ein char-Array ist, die Funktion strcpy() (siehe Kapitel [→ 12.9.1](#)) zu verwenden. Im Falle der Pointer-Variablen kann einfach ein neuer Pointer zugewiesen werden. Über die Unterschiede zwischen char-Arrays und Pointer wurde bereits in Kapitel [→ 12.5](#) geschrieben.

13.1.6 Operationen auf Komponenten und ganzen Struktur-Variablen

Die Komponenten einer Struktur-Variablen können mittels des Punkt-Operators angesprochen werden. Für sie gelten dieselben Regeln, wie wenn sie eigenständige Variablen wären.

Auf Komponenten-Variablen sind diejenigen Operationen zugelassen, die für den entsprechenden Komponenten-Typ möglich sind.



Beispielsweise können die Adressen von Komponenten-Variablen mit Hilfe des Adress-Operators bestimmt werden wie im folgenden Beispiel:

```
char* namePtr = &meyer.name;
```

Doch auch auf ganzen Strukturen, sprich den Struktur-Variablen selbst, können Operationen durchgeführt werden.

So kann der Inhalt einer Struktur-Variablen einer anderen Struktur-Variablen mittels des Zuweisungs-Operators zugewiesen werden, sofern sie denn vom gleichen Struktur-Typ ist:

```
struct Adresse adresse1 = {"Ambrosiastrasse", 52, 68759, "Borrheim"};  
struct Adresse adresse2;  
adresse2 = adresse1;
```

Hierbei werden sämtliche Elemente der Struktur von `adresse1` nach `adresse2` kopiert.

Vorsicht ist geboten bei Komponenten, die Pointer sind: Bei einer Zuweisung wird nur der Pointer kopiert, nicht aber der Inhalt, auf den der Pointer zeigt.



Im Gegensatz zum Zuweisungs-Operator ist ein Vergleich von zwei Struktur-Variablen mit Vergleichs-Operatoren hingegen nicht möglich. Strukturen muss man immer komponentenweise vergleichen.

Die Größe eines Struktur-Typs oder einer Struktur-Variablen im Arbeitsspeicher kann nicht aus der Größe der einzelnen Komponenten berechnet werden, da Compiler die Komponenten oft auf bestimmte Wortgrenzen (Alignment) legen. Zur Ermittlung der Größe einer Struktur im Arbeitsspeicher muss der Operator `sizeof` (siehe Kapitel [→ 9.9.1](#)) verwendet werden, wie beispielsweise `sizeof(struct Student)`. Mit Hilfe des `_Alignof`-Operators kann seit dem C11-Standard das Alignment der ganzen Struktur ermittelt werden.

Die Adresse einer Struktur-Variablen `a` wird wie bei Variablen von einfachen Datentypen mit Hilfe des Adress-Operators ermittelt, das heißt durch `&a`. Der resultierende Pointer zeigt dabei auf das allererste Element der Struktur.

13.1.7 Selektion der Komponenten: Punkt- und Pfeil-Operator

Für den Zugriff auf Komponenten steht grundsätzlich der **Punkt-Operator** zur Verfügung. Sehr häufig jedoch sind Strukturen mittels Pointer verfügbar, beispielsweise, da sie als solche an Funktionen übergeben werden. Im folgenden Beispiel wird mittels des Adress-Operators ein solcher Pointer definiert:

```
struct Adresse* adressePtr = &adresse1;
```

Um eine solche Pointer-Struktur-Variable nun korrekt zu dereferenzieren, muss man schreiben:

```
(*adressePtr).hausnummer = 55;
```

Die Klammern sind notwendig, da der Punkt-Operator höher priorisiert wird, wie der Dereferenzierungs-Operator. Da diese Schreibweise sehr umständlich ist, wurde in C der sogenannte „**Pfeil-Operator**“ eingeführt, welcher genau dasselbe tut:

```
adressePtr->hausnummer = 55;
```

Der Pfeil-Operator `->` wird an der Tastatur durch ein Minuszeichen und ein Größerzeichen geschrieben.

Die Selektions-Operatoren (Auswahl-Operatoren) `.` und `->` haben die gleiche Vorrangstufe. Sie werden von links nach rechts abgearbeitet.



Beachten Sie, dass zwar der Operand des Punkt-Operators `.` ein L-Wert sein muss, nicht aber unbedingt der Operand des Pfeil-Operators `->`. Dieser benötigt nur einen Pointer, sprich eine Adresse, welche aus einem beliebigen Ausdruck entstehen kann. Dies wird insbesondere wichtig bei mehrfach zusammengesetzten Strukturen.

13.1.8 Mehrfach zusammengesetzte Strukturen

Im Folgenden werden Struktur-Variablen als Komponenten eines Datentyps betrachtet. Mit anderen Worten, es sollen mehrfach zusammengesetzte Strukturen eingeführt werden. Als Anwendungsbeispiel soll der Fitnessplan einer Person dienen. Wir definieren folgende beiden Datenstrukturen:

```
struct Lauf {  
    float km;           // Anzahl gerannte Kilometer  
    int   hoehenmeter;  // Zurueckgelegter Hoeehenunterschied  
    int   s;            // Benoetigte Sekunden fuer die Strecke  
};  
  
struct Trainingsplan {  
    struct Lauf morgenrunde;  
    struct Lauf mittagsrunde;  
    struct Lauf abendrunde;  
};
```

Nehmen wir an, die Person habe am Montag ihr Training absolviert. So sei `montag` eine Variable vom Typ `struct Trainingsplan`:

```
struct Trainingsplan montag;
```

Die Person absolviert ihr Training und läuft folgende Strecken:

```
montag.morgenrunde.km = 5;
montag.morgenrunde.hoehenmeter = 60;
montag.morgenrunde.s = 30 * 60;
montag.mittagsrunde.km = 3;
montag.mittagsrunde.hoehenmeter = 0;
montag.mittagsrunde.s = 15 * 60;
montag.abendrunde.km = 20;
montag.abendrunde.hoehenmeter = 200;
montag.abendrunde.s = 80 * 60;
```

In der Datenstruktur können somit die entsprechenden Felder direkt mittels des Punkt-Operators verschachtelt angesprochen werden.

Da die Person täglich trainiert und sie es leid ist, immer alle Daten einzutragen, fügt sie einen zusätzlichen Datentyp hinzu und schreibt den Datentyp Lauf um:

```
struct Strecke {
    float km;           // Anzahl gerannte Kilometer
    int    hoehenmeter; // Zurueckgelegter Hoehenunterschied
};

struct Lauf {
    struct Strecke* strecke; // Pointer auf die Strecke
    int              s;      // Benoetigte Sekunden fuer die Strecke
};
```

So kann die Person ihre Lieblingsstrecken folgendermaßen definieren:

```
struct Strecke finnenbahn = {5, 60};
struct Strecke seeweg    = {3, 0};
struct Strecke waldweg   = {20, 200};
```

Dadurch kann sie ihre Runden folgendermaßen aufschreiben:

```
montag.morgenrunde.strecke = &finnenbahn;
montag.morgenrunde.s = 30 * 60;
montag.mittagsrunde.strecke = &seeweg;
montag.mittagsrunde.s = 15 * 60;
montag.abendrunde.strecke = &waldweg;
montag.abendrunde.s = 80 * 60;
```

Und wenn sie am Dienstag dieselben Strecken nochmals abläuft, kann sie dieselben Pointer nochmals verwenden.

```
dienstag.morgenrunde.strecke = &finnenbahn;  
dienstag.morgenrunde.s = 25 * 60;  
...
```

Um nun beispielsweise die Statistik vom Training am Dienstagmorgen auszurechnen, kann folgendes geschrieben werden:

```
printf("%.0f Meter in %d:%02d Minuten = %.1f km/h\n",  
    dienstag.morgenrunde.strecke->km * 1000.f,  
    dienstag.morgenrunde.s / 60,  
    dienstag.morgenrunde.s % 60,  
    dienstag.morgenrunde.strecke->km / dienstag.morgenrunde.s * 3600);
```

Dieses gesamte Beispiel findet sich in der Datei `sportler.c` und erzeugt folgende Ausgabe:

```
5000 Meter in 30:00 Minuten = 10.0 km/h
```

13.1.9 Übergabe von Struktur-Variablen an Funktionen und Rückgabe

Strukturen werden als zusammengesetzte Variablen komplett an Funktionen übergeben. Es gibt hier keinen Unterschied zu Variablen von einfachen Datentypen wie beispielsweise `float` oder `int`.



Man muss nur einen formalen Parameter vom Typ der Struktur einführen und als aktuellen Parameter eine Struktur-Variable dieses Typs übergeben.

Auch die Rückgabe einer Struktur-Variablen unterscheidet sich nicht von der Rückgabe einer einfachen Variablen.



Der Rückgabetyt der Funktion muss selbstverständlich vom Typ der Struktur-Variablen sein, die zurückgegeben werden soll.

Bei der Übergabe von Strukturen werden sämtliche Komponenten genau gleich wie bei einer Zuweisung (siehe Kapitel [→ 13.1.6](#)) kopiert. Dies kann sehr ineffizient sein, da Strukturen häufig vergleichsweise große Datenmengen speichern. Somit ist die Übergabe von ganzen Strukturen eher selten anzutreffen.

Viel häufiger werden Strukturen per Pointer übergeben. Beispielsweise könnte die Statistik der Person aus dem vorherigen Unterkapitel mittels eines Funktionsaufrufs viel einfacher gestaltet werden:

```
void laufStatistik(struct Lauf* lauf) {  
    printf("%.03f Kilometer in %d:%02d Minuten = %.1f km/h\n",  
        lauf->strecke->km,  
        lauf->s / 60,  
        lauf->s % 60,  
        lauf->strecke->km / lauf->s * 3600);  
}
```

Dadurch kann diese Funktion ganz praktisch aufgerufen werden mit:

```
laufStatistik(&dienstag.morgenrunde);
```

Hierbei muss jedoch beachtet werden, dass in der aufgerufenen Funktion keine lokale Kopie, sondern nur der Pointer auf die Struktur der aufrufenden Funktion verfügbar ist. Jede Änderung innerhalb der aufgerufenen Funktion ändert somit die Struktur außerhalb.

Um Änderungen innerhalb der aufgerufenen Funktion zu verhindern, kann der Parameter mittels `const` deklariert werden (siehe Kapitel [→ 7.5.1](#)). Dadurch verhindert der Compiler jeglichen Schreibzugriff auf eine Komponente der Struktur.

```
void laufStatistik(const struct Lauf* lauf);
```

Die Angabe `const` schützt nur die übergebene Struktur-Variable und deren Komponenten, nicht aber Strukturen, welche als Pointer über eine Komponente dereferenziert werden!



So wäre es beispielsweise möglich, innerhalb der Funktion `laufStatistik()` die Kilometeranzahl einer Strecke zu manipulieren:

```
lauf->strecke->km = 100;
```

Um dies ebenfalls zu verhindern, müsste die Pointer-Komponente `strecke` der Struktur `Lauf` ebenfalls mit `const` deklariert werden:

```
const struct Strecke* strecke; // Pointer auf die Strecke
```

Diese durchgängige Verwendung des `const`-Schlüsselwortes wird **const-safe-Programmierung** genannt und kann im Detail in Kapitel [→ 11.5.2](#) nachgelesen werden.

13.1.10 Anwendungsmöglichkeiten von Strukturen

Die Verwendung von Strukturen empfiehlt sich immer bei semantisch zusammengehörenden strukturierten Daten. Einzelkomponenten mit unterschiedlichen Datentypen können als zusammengehörige Struktur wesentlich bequemer gehandhabt werden.



Hier ein einfaches Beispiel für eine Filmdatenbank:

moviedatabase.c

```
#include <stdio.h>

struct Movie {
    char        name[50];
    int         dauer;
    const char* tags[10];
};

void printMovie(const struct Movie* movie) {
    printf("\'%s\'", ", ", movie->name);
    printf("%d Minuten\\nTags:\\n", movie->dauer);

    int tagindex = 0;
    while (movie->tags[tagindex]) {
        printf("\'%s\'\\n", movie->tags[tagindex]);
        ++tagindex;
    }

    printf("\\n");
}

int main(void) {
    struct Movie film1 = {
        "Lion King",
        85,
        {"Simba", "Circle of Life"}
    };
    struct Movie film2 = {
        "Terminator 2",
        147,
        {"Bad to the bone", "T-1000", "Hasta la vista"}
    };

    printMovie(&film1);
    printMovie(&film2);
    return 0;
}
```


Die Ausgabe dieses Programmes lautet

```
Lion King", 85 Minuten
Tags:
"Simba"
"Circle of Life"

Terminator 2", 147 Minuten
Tags:
"Bad to the bone"
"T-1000"
"Hasta la vista"
```

Auch viele Systemfunktionen liefern logisch zusammengehörige Daten in Form von Strukturen oder Pointern auf Strukturen zurück. Die benötigten Daten erhält man dann durch Zugriff auf die einzelnen Komponenten der Struktur.

So enthält beispielsweise die im Folgenden gezeigte Struktur `tm` (siehe `<time.h>`) alle wichtigen Daten, die sich aus der Sekundenanzahl berechnen lassen:

```
struct tm {
    int tm_sec;      // Sekunden          [0,59]
    int tm_min;      // Minuten           [0,59]
    int tm_hour;     // Stunden            [0,23]
    int tm_mday;     // Tag                [1,31]
    int tm_mon;      // Monat                [0,11]
    int tm_year;     // Jahre seit 1900
    int tm_wday;     // Tage seit Sonntag   [0,6]
    int tm_yday;     // Tage seit 1. Januar [0,365]
    int tm_isdst;    // Daylight Saving Time
};
```

Dieser Datentyp wird von der Funktion `localtime()` genutzt, um die Sekundenanzahl, die von der Funktion `time()` berechnet wird, strukturiert zurückzugeben.

Die Systemzeit eines Computers wird in einem Unix-System als Anzahl der Sekunden seit Mitternacht des 1. Januars 1970 ausgegeben. Da sich aus der Anzahl der Sekunden sowohl die Uhrzeit, das Datum und weitere Zusatzinformationen ableiten lassen, müssten mehrere Funktionen geschrieben werden, die jeweils die gesuchte Information in einem bestimmten Format liefern. Um dies zu vermeiden, nutzt man Strukturen, um dort alle zusammengehörigen Informationen abzuspeichern.

13.1.11 Beispielprogramm Systemzeit

Das folgende Beispiel zeigt die Verwendung der Struktur `tm` in Verbindung mit der Funktion `localtime()`. Dieses Beispiel gibt für die Startzeit des Programms die Uhrzeit, das Datum und den Wochentag auf dem Bildschirm aus:

datum.c

```
#include <time.h>
#include <stdio.h>

int main(void) {
    time_t sekunden;
    time(&sekunden);
    struct tm* zeit = localtime(&sekunden);

    printf("Aktuelle Zeitangabe:\n\n");
    printf("%02d:%02d:%02d\n",
        zeit->tm_hour, zeit->tm_min, zeit->tm_sec);
    printf("%02d.%02d.%d \n",
        zeit->tm_mday, zeit->tm_mon + 1, 1900 + (zeit->tm_year));

    char* tag[] = {"Sonntag", "Montag", "Dienstag", "Mittwoch",
        "Donnerstag", "Freitag", "Samstag"};
    printf("Wochentag: %s\n", tag[zeit->tm_wday]);

    return 0;
}
```

Eine mögliche Ausgabe des Programms ist:

```
Aktuelle Zeitangabe:

21:11:47
30.03.2023
Wochentag: Donnerstag
```

Das Programm `datum.c` ruft mit der Funktion `time()` die aktuelle Systemzeit in Sekunden ab. Dann wird die Sekundenanzahl durch die Funktion `localtime()` in Uhrzeit, Datum und vergangene Tage umgerechnet und in der Struktur `zeit` vom Typ `tm` gespeichert. Die einzelnen Werte der Struktur werden dann als Uhrzeit, Datum und als Wochentag ausgegeben. Da die Struktur-Variable `tm_wday` die Anzahl der Tage seit Sonntag zurückliefert, kann man dies zur Ausgabe eines Wochentages nutzen, indem man `tm_wday` als Index eines `char`-Arrays benutzt.

13.1.12 Beispielprogramm Jahr-2038-Problem

Bei der Zeitermittlung mittels Sekunden seit 01.01.1970 ergibt sich im Jahr 2038 ein Problem, das Jahr-2038-Problem: Da `size_t` in der Regel eine 32-Bit Integer-Zahl ist, ergibt sich daraus ein maximaler Wert von 2147483647 für die Anzahl der Sekunden. Dies entspricht einem Datum vom 19.01.2038, wie das folgende Beispiel zeigt:

maxdatum.c

```
#include <limits.h>
#include <time.h>
#include <stdio.h>

int main(void) {
    time_t maxDatum = INT_MAX;
    struct tm* ptrMaxDatum;
    ptrMaxDatum = gmtime(&maxDatum);

    printf("Die 32-Bit-Zeit laeuft ab\n");
    printf("am %02d.%02d.%d\n",
        ptrMaxDatum->tm_mday,
        ptrMaxDatum->tm_mon + 1,
        1900+(ptrMaxDatum->tm_year));
    printf("um %02d:%02d:%02d Uhr\n",
        ptrMaxDatum->tm_hour,
        ptrMaxDatum->tm_min,
        ptrMaxDatum->tm_sec);
    return 0;
}
```

Als Sekundenanzahl wird hier einfach der größtmögliche Wert einer `int`-Zahl verwendet. Dann wird mit diesem Wert das Datum und die Uhrzeit berechnet.

Hier die Ausgabe des Programms:

```
Die 32-Bit-Zeit laeuft ab
am 19.01.2038
um 03:14:07 Uhr
```

Bis zum Jahr 2038 dauert es zwar noch ein paar Jahre, aber die Probleme, die dann kommen werden, dürften denen des Jahr-2000-Problems nur um wenig nachstehen.

13.2 Anonyme Strukturen

Normalerweise werden Felder von Strukturen genauso wie Variablen stets mit einem eindeutigen Namen versehen. Dadurch sind diese Komponenten klar ansprechbar. Nicht immer jedoch will man Variablen oder Felder explizit ansprechen, beispielsweise wenn Variablen nur einen einmaligen Übergabewert für eine Funktion darstellen oder die Felder eines Struktur-Typs direkt einer übergeordneten Struktur angehören sollen.

Im Standard C11 wurde genau aus diesem Grund festgelegt, dass Struktur-Variablen auch anonym – ohne Namen oder Etikett – vereinbart werden können.

13.2.1 Anonyme Struktur-Typen

Anonyme Strukturen können als Teil einer Definition von anderen Strukturen auftreten.



Im folgenden Beispiel hat die Struktur innerhalb der Struktur keinen Namen:

```
struct MeineStruktur {  
    int komponente1;  
    struct {  
        int komponente2;  
    };  
};
```

Da die Struktur keinen Variablennamen besitzt, lassen sich die Komponenten innerhalb der namenlosen Struktur ansprechen, als wären die Komponenten direkt Teil der übergeordneten benannten Struktur:

```
struct MeineStruktur strukturVariable;  
strukturVariable.komponente1;  
strukturVariable.komponente2;
```

Dabei ist zu beachten, dass die Namen der einzelnen Komponenten in der Struktur für eine eindeutige Zuordnung verschieden sein müssen. Der Aufruf einer Komponente innerhalb einer eingebetteten Struktur wird damit einfacher und kürzer, der Programmcode wird übersichtlicher.

13.2.2 Compound Literals

In C11 kann eine Struktur auch mittels eines sogenannten „compound literals“ direkt, also ohne Vereinbarung einer Variablen angegeben werden.

Ein **compound literal** ist ein Ausdruck, der ein anonymes Objekt (Objekt ohne Namen) erzeugt, dessen Wert durch eine Initialisierungsliste gegeben ist.



Im Folgenden ein erstes Beispiel für ein compound literal:

compoundliteral.c

```
#include <stdio.h>

void printArray(int array[4]) {
    for (int i = 0; i < 4; ++i) {
        printf("%d ", array[i]);
    }
}

int main(void) {
    printArray((int[4]) {1, 2, 3, 4});
    return 0;
}
```

Hier wird ein compound literal an die Funktion `printArray()` übergeben. Der Typ des Objektes ist `int[4]` und steht in runden Klammern vor der Initialisierungsliste. Der Typname kann ein vollständiger Datentyp oder ein Array von unbekannter Länge sei, nicht aber ein Array-Typ variabler Länge.

Beispiele:

```
(int){1}
(const int){2}
(struct Point2D){0, 0}
```

Compound literals erlauben eine Notation ähnlich wie literale Konstanten für Arrays, Strukturen und andere Typen (außer den Arrays variabler Länge). Ein compound literal kann an jeder Stelle benutzt werden, an der ein Objekt desselben Typs wie das compound literal benutzt werden kann. Beispielsweise:

```
int x = (int){1} + (int){4710};
```

Ein compound literal bezeichnet dabei stets einen L-Wert. Die Adresse eines compound literals ist die Adresse des anonymen Objekts, das durch das compound literal erklärt wird. Wenn das compound literal keinen mit `const` qualifizierten Typ hat, kann das compound literal über einen Pointer auf das compound literal abgeändert werden.

Beispiele:

```
int* p = (int[3]){1, 2, 3};  
*(p + 1) = 40;    // setzt die Komponente mit Index 1 auf 40
```

```
struct Point2D* p = &(struct Point2D){0, 0};  
p->x = 1;  
p->y = 1;  
// der Punkt hat jetzt die Koordinaten (1, 1)
```

Wird ein anonymes Objekt außerhalb einer Funktion angelegt, ist die Lebensdauer statisch während des Programmlaufs. Da die Initialisierung stattfindet, bevor ein Programm läuft, müssen die Initialisierer in der Initialisierungsliste konstante Ausdrücke sein. Innerhalb einer Funktion erfolgt die Initialisierung, wenn der entsprechende Block im Programmablauf erreicht wird. Die Ausdrücke in der Initialisierungsliste können in diesem Fall beliebige zur Laufzeit zu berechnende Ausdrücke sein.

Alle Einschränkungen für die „normale“ Initialisierungsliste gelten auch für compound literals. Wenn man in der Initialisierungsliste für Struktur-Typen und Arrays mit fester Elementanzahl somit nur einige Initialisierer liefert, werden die ersten Elemente initialisiert und die anderen mit null des entsprechenden Typs. Wie bei jeder anderen Initialisierungsliste mit geschweiften Klammern können „designated initializers“ verwendet werden, siehe Kapitel [→ 13.1.3](#). Bei einem anonymen Array ohne Längenangabe bestimmt sich die Länge des Arrays aus der Zahl der Elemente des Arrays.

13.3 Unionen

Eine **Union** besteht wie eine Struktur aus einer Reihe von Komponenten mit unterschiedlichen Datentypen. Im Gegensatz zur Struktur jedoch stellen die einzelnen Komponenten nicht Teile einer Gesamtstruktur dar, sondern nur Alternativen, welche unter dem Strukturnamen angesprochen werden können. Die Komponenten einer Union werden also nicht hintereinander im Speicher abgebildet, sondern beginnen alle an derselben Adresse. Der Speicherplatz wird vom Compiler so groß angelegt, dass der Speicherplatz auch für die größte Alternative reicht.

Bei einer Union ist zu einem bestimmten Zeitpunkt jeweils nur eine einzige Komponente einer Reihe von alternativen Komponenten gespeichert.



Eine Union wird mit dem Schlüsselwort **union** eingeleitet.



Als Beispiel wird hier eine Union vom Typ `union vario` eingeführt:

```
union vario {  
    int    intVar;  
    double doubleVar;  
    float  floatVar;  
} variant;
```

Hierbei ist `vario` das **Etikett** (auf Englisch das „**union tag**“) des Union-Typs. Ein Wert aus dem Wertebereich eines jeden der drei in der Union enthaltenen Datentypen kann an die Variable `variant` zugewiesen und in Ausdrücken benutzt werden. Man muss jedoch aufpassen, dass die Benutzung konsistent bleibt.

Beim Schreiben von Code mit Unionen muss verfolgt werden, welcher Typ jeweils in der Union gespeichert ist. Der Datentyp, der entnommen wird, muss derjenige sein, der zuletzt gespeichert wurde.



Die Auswahl von Komponenten erfolgt wie bei Strukturen über den Punkt- beziehungsweise den Pfeil-Operator:

```
variant.intVar = 123;  
int x = variant.intVar;
```

oder

```
union vario* ptr = &variant;  
double y = ptr->doubleVar;
```

Da weder beim Compilieren noch zur Laufzeit die Zugriffe auf die jeweiligen Komponenten überprüft werden, gelten Unionen heutzutage als verpönt. Unionen werden außer in der Systemprogrammierung kaum mehr verwendet.

13.3.1 Detaillierte Informationen zu Unionen

Obschon Unionen heute kaum mehr genutzt werden, werden hier dennoch einige fortgeschrittene Informationen und ein Beispiel gezeigt.

- Unionen können in Strukturen und Arrays auftreten und umgekehrt.
- Unions-Variablen können mittels des Zuweisungs-Operators gegenseitig sich zugewiesen werden.
- Die Ermittlung der Größe und Speicheranordnung einer Union erfolgt mit Hilfe des `sizeof`- und (seit C11) des `_Alignof`-Operators.
- Es ist möglich, die Adresse einer Unions-Variablen mittels des Adress-Operators zu ermitteln.

Wenn ein Pointer auf eine Union in den Typ eines Pointers auf eine Alternative umgewandelt wird, so verweist das Resultat auf diese Alternative. Voraussetzung ist natürlich, dass diese Alternative die gerade aktuell gespeicherte Alternative darstellt.



Dies ist unter anderem im folgenden Beispiel zu sehen:

union.c

```
#include <stdio.h>

int main(void) {
    union Zahl {
        int    intVar;
        double doubleVar;
        float  floatVar;
    };

    union Zahl feld[2];
    union Zahl* ptr;
    float* floatPtr;

    printf("Groesse der Union: %d\n", (int)sizeof(union Zahl));
    printf("Groesse der Komponenten: %d\n", (int)sizeof(feld[1]));
    printf("Groesse von int    : %d\n", (int)sizeof(int));
    printf("Groesse von double: %d\n", (int)sizeof(double));
    printf("Groesse von float : %d\n", (int)sizeof(float));

    printf("\n");

    feld[0].doubleVar = 5.;
    printf("Inhalt von feld[0]: %f\n", feld[0].doubleVar);
    feld[0].intVar = 10;
    printf("Inhalt von feld[0]: %d\n", feld[0].intVar);
    feld[0].floatVar = 100.0;
    printf("Inhalt von feld[0]: %6.2f\n", feld[0].floatVar);
    printf("-----\n");

    feld[1] = feld[0];
    printf("Inhalt von feld[1]: %6.2f\n", feld[1].floatVar);
    feld[1].floatVar += 25.;
    ptr = &feld[1];
    floatPtr = (float*)ptr;

    printf("Inhalt von feld[1]: %6.2f\n", ptr ->floatVar);
    printf("Inhalt von feld[1]: %6.2f\n", *floatPtr);

    return 0;
}
```

Hier die Ausgabe des Programms:

```
Groesse der Union: 8
Groesse der Komponenten: 8
Groesse von int   : 4
Groesse von double: 8
Groesse von float : 4

Inhalt von feld[0]: 5.000000
Inhalt von feld[0]: 10
Inhalt von feld[0]: 100.00
-----
Inhalt von feld[1]: 100.00
Inhalt von feld[1]: 125.00
Inhalt von feld[1]: 125.00
```

Bei einer Union kann nur eine Initialisierung der ersten Alternative erfolgen. Diese Initialisierung erfolgt durch einen in geschweiften Klammern stehenden konstanten Ausdruck.



Eine Variable vom Typ union `Zahl` aus obigem Beispiel kann also nur mit einem konstanten `int`-Ausdruck initialisiert werden.

13.4 Bitfelder

Bis zu dieser Stelle wurden lediglich die Bit-Operationen `|` (bitweises ODER), `&` (bitweises UND), `^` (Exklusives-ODER) und `~` (Einer-Komplement) als Möglichkeit zur Bit-Manipulation in der Programmiersprache C vorgestellt (siehe dazu Kapitel [→ 9.8](#)). Hierbei kann man durch gezieltes Verknüpfen der Operanden mit einem entsprechenden Bitmuster Bits setzen oder löschen. Eine weitere Möglichkeit, um mit Bits zu arbeiten, stellen die Bitfelder dar. Bitfelder ermöglichen es, Bits zu gruppieren.

Ein **Bitfeld** besteht aus einer angegebenen Zahl von Bits (einschließlich eines eventuellen Vorzeichenbits) und wird als Integer-Typ betrachtet. Die Länge des Bitfeldes wird vom Bitfeld-Namen durch einen Doppelpunkt getrennt.



```
struct {  
    ...  
    BitfeldTyp BitfeldName : BitAnzahl;  
    ...  
};
```

Dabei ist zu beachten, dass ein Bitfeld nur als Teil einer Struktur oder einer Union existieren kann.

Beispielsweise deklariert folgende Zeile ein Bitfeld `a` bestehend aus 4 Bits, welches als `unsigned int` interpretiert wird:

```
unsigned a : 4;
```

Bitfelder sind von der jeweiligen Implementierung des Compilers abhängig. So sind beispielsweise die zulässigen Typen für ein Bitfeld und die Anordnung der verschiedenen Bitfelder im Speicher je nach Compiler unterschiedlich.



Ein Bitfeld kann wie eine normale Variable mit dem definierten Typ gelesen und gesetzt werden, allerdings wird nur die angegebene Anzahl Bits wirklich gespeichert.

Ein Bitfeld wird wie eine normale Komponente einer Struktur mit dem Punkt-Operator `.` angesprochen. Der Pfeil-Operator `->` ist je nach Compilerhersteller zugelassen oder auch nicht, da ein Bitfeld nicht immer eine Adresse hat.



Werden einem Bitfeld Werte zugewiesen, die außerhalb des Wertebereichs des Bitfeldes liegen, so wird mit der Modulo-Arithmetik (siehe Kapitel [7.3.2](#)) ein Überlauf vermieden. Weist man somit der Variablen `a` den Wert 19 zu, so werden nur die untersten 4 Bits verwendet, was den gespeicherten Wert entsprechend verändert:

		Bitfeld der Größe 4
		unsigned a : 4; Wertebereich: 0 bis 15
a = 3;	0011	a ist 3
a = 19;	0011	a ist 3 (Bereichsüberschreitung)
Stellenwert:	$2^3 \ 2^2 \ 2^1 \ 2^0$	

Bei vorzeichenbehafteten Typen kann dies zu weiteren Problemen führen. Beim Typ `signed` wird normalerweise das Most Significant Bit (MSB) für die Darstellung des Vorzeichenbits in Zweierkomplement-Form benutzt. Ist das MSB gleich 1, so wird die Zahl als negative Zahl interpretiert.

signed b : 4;

Wenn wie im folgenden Beispiel der Variablen `b` der Wert 9 zugewiesen wird, so tritt eine Bereichsüberschreitung auf:

		Vorzeichenbit
		Bitfeld der Größe 4
		signed b : 4; Wertebereich: -8 bis +7
b = 3;	0011	b ist 3
b = 9;	1001	b ist -7 (Bereichsüberschreitung)
Stellenwert:	$-2^3 + 2^2 + 2^1 + 2^0$	

Das Bitfeld `b` hat eigentlich einen Zahlenbereich von -8 bis $+7$. Bei einer Zuweisung einer Zahl außerhalb des Zahlenbereichs werden die Bits den Stellen entsprechend hart zugewiesen, ohne dass vom Compiler auf einen Überlauf hingewiesen wird. Entsprechend der Darstellung im Zweierkomplement wird bei $7+1$ das höchste Bit gesetzt, die anderen Bits sind 0. Daraus ergibt sich der Wert -8 . Aus der Zahl 9 wird dann in dem Bitfeld entsprechend eine -7 , 10 entspricht -6 , und so fort.

Die Datentypen, die in einem Bitfeld verwendet werden dürfen, sind eingeschränkt. Nach dem Standard dürfen lediglich die Typen `int`, `signed int` oder `unsigned int` und unter C11 auch der Typ `_Bool` verwendet werden. Bei manchen Compilern wie beispielsweise beim Visual C++ Compiler sind auch die Typen `char`, `short` und `long` jeweils `signed` und `unsigned` erlaubt. Letztendlich ist der Datentyp eines Bitfeldes für die Interpretation der einzelnen Bits ausschlaggebend. Hierbei spielt auch eine entscheidende Rolle, ob das Bitfeld `signed` oder `unsigned` ist.

Bitfelder sind sehr implementierungsabhängig, was durch den ISO-Standard gewünscht wurde. Dies führt dazu, dass bei der Portierung von Programmen, die Bitfelder enthalten, große Vorsicht angeraten ist.



Aufgrund der enormen Fortschritte betreffend verfügbarem Speicherplatz werden Bitfelder heutzutage kaum mehr eingesetzt. Sie werden jedoch manchmal in der hardwarenahen Programmierung verwendet, wo Hardware-Bausteine über Bitmasken programmiert werden können.

13.5 Übungsaufgaben

Aufgabe 1: Strings

Studieren Sie das folgende Programm.

- Welche Codezeilen werden einen Fehler erzeugen? Überprüfen Sie Ihre Annahmen mit Hilfe des Compilers.
- Korrigieren Sie die Fehler, sodass das Programm lauffähig wird.
- Was wird nun ausgegeben werden? Überprüfen Sie Ihre Annahmen mit einem Programmdurchlauf.

structErrors.c

```
#include <stdio.h>

struct Person {
    char vorname[30];
    char* nachname;
};

int main(void) {
    struct Person person1 = {"Kathrin", "Knoll"};
    struct Person person2 = {"Thorsten", "Powlov"};

    person1.vorname = "Maria";
    person1.nachname = "Hanse";
    printf("%s %s\n", person1.vorname, person1.nachname);

    person1.vorname = person2.vorname;
    person1.nachname = person2.nachname;
    printf("%s %s\n", person1.vorname, person1.nachname);

    person1 = person2;
    printf("%s %s\n", person1.vorname, person1.nachname);

    person2.nachname = "Frick";
    printf("%s %s\n", person2.vorname, person2.nachname);

    return 0;
}
```

Aufgabe 2: Compound literal

1. Verändern und erweitern Sie das Programm aus Aufgabe 1 so, dass die Ausgabe des vollständigen Namens in einer Funktion mit folgendem Funktionskopf passiert:

```
void printPerson(struct Person myPerson);
```

2. Geben Sie mittels dieser Funktion mehrere neue Namen bestehend aus Vorname und Nachname aus, jedoch ohne Variablen zu verwenden. Nutzen Sie hierfür compound literals.
3. Verändern Sie die Funktion und die Aufrufe derselben so, dass ein Pointer als Argument erwartet wird:

```
void printPersonPointer(struct Person* myPerson);
```

Aufgabe 3: Strukturen. Pointer auf Strukturen.

- a) Schreiben Sie ein Programm, welches eine Münzsammlung speichern kann. Eine Münze soll Eigenschaften speichern wie Währung, Münzwert, Jahrgang, Tauschwert, Größe, Kaufdatum.
- b) Erweitern Sie das Programm mit einer Struktur „Münzkategorie“, welche die Währung und den Münzwert speichert. Münzen sollen sodann nur noch per Pointer auf diese Münzkategorien verweisen.
- c) Schreiben Sie eine Funktion, welche eine Münze als Pointer erwartet und deren Eigenschaften schön formatiert ausgibt.

14 Komplexere Vereinbarungen, eigene Typnamen und Namensräume



In Kapitel [→ 7](#) [→ 8.3](#) und [→ 13](#) wurden bereits einige Typen der Programmiersprache C vorgestellt. Diese Typen sind oft kombinier- und erweiterbar zu komplexen Datentypen.

In diesem Kapitel werden diese komplexen Datentypen, sowie eine Vereinfachung davon mittels der Benutzung des Schlüsselwortes `typedef` behandelt. Auch wird auf das Konzept der Namensräume in C eingegangen.

14.1 Komplexere Vereinbarungen

Vereinbarungen in C können im Allgemeinen nicht stur von links nach rechts gelesen werden. Die Typangaben **Pointer**, **Array** und **Pointer auf Funktion** haben genauso wie die entsprechenden Operatoren (siehe Kapitel [→ 9.2](#)) eine Vorrang-Reihenfolge. Diese muss bei der Typ-Definition beachtet werden. Genau so wie bei Operatoren können jedoch Typ-Ausdrücke auch geklammert und damit ihre Rangordnung verändert werden.

Bei komplexeren Vereinbarungen ist die **Vorrang-Reihenfolge** der Operatoren (**operator precedence**) zu beachten.



Im Folgenden werden einige komplexere Datentypen vorgestellt, bei denen die Anwendung der Vorrang-Reihenfolge sowie der Klammerung eine Rolle spielt.

14.1.1 Array von Pointern

Durch die folgende Programmzeile wird ein Array von Pointern vereinbart:

```
int* alpha[8];
```

Vereinbart wird eine Variable mit dem Namen alpha. Es ist zu beachten, dass der Array-Operator [] Vorrang vor dem *-Operator hat. Daher wird auf den Namen alpha als erstes der Operator [8] angewandt: alpha ist also ein eindimensionales Array mit 8 Komponenten. Der Typ einer Komponente ist int*, also Pointer auf int.

alpha im Typausdruck int* alpha[8] ist eindimensionales Array mit 8 Arraykomponenten, welche jeweils einen Pointer auf einen int speichern.



14.1.2 Pointer auf ein Array

Soll alpha kein Array, sondern ein Pointer sein, so kann dies durch die Benutzung von Klammern erzwungen werden:

```
(*alpha)
```

Damit ist alpha ein „Pointer auf“. Er soll jetzt auf ein Array aus 8 int-Komponenten zeigen. Damit ergibt sich die folgende Programmzeile:

```
int (*alpha)[8]
```

alpha im Typausdruck int (*alpha) [8] ist ein Pointer auf ein int-Array mit acht Komponenten.



14.1.3 Funktion mit Rückgabewert „Pointer auf“

Im Folgenden wird die Vereinbarung einer Funktion betrachtet. Die drei Punkte in Klammern symbolisieren die formalen Parameter:

```
int* func(...)
```

Die Funktionsaufrufs-Klammern haben eine höhere Priorität als das Pointer-Zeichen *. Damit ist func() eine Funktion mit dem Rückgabewert Pointer auf int.

int* func(...) ist eine Funktion mit einem Pointer auf int als Rückgabewert.



14.1.4 Pointer auf eine Funktion

Es wird nun der folgende Ausdruck analysiert:

```
int* (*func)(...)
```

Wegen der Klammern wird zuerst (*func) ausgewertet. func ist somit ein Pointer.

Da die Funktionsaufrufs-Klammern (...) Vorrang vor dem Operator * haben, ist (*func)(...) somit ein Pointer auf eine Funktion.

Der Rückgabebetyp dieser Funktion schlussendlich ist int*. Somit ergibt sich:

int* (*func)(...) ist ein Pointer auf eine Funktion mit einem Rückgabewert vom Typ Pointer auf int.



14.2 Vereinbarung eigener Typnamen mittels typedef

Die in Kapitel [→ 14.1](#) vorkommenden Beispiele zeigen, dass Typbezeichnungen sehr umfangreich und komplex sein können. Um diese Typbezeichnungen nicht immer so ausführlich hinschreiben zu müssen, erlaubt C die Vereinbarung eigener Typnamen für beliebige Typausdrücke.

Eigene Typnamen zu Typausdrücken können in C mit Hilfe von **typedef** vereinbart werden.



Die Syntax für die Verwendung von typedef ist die Folgende:

```
typedef Typausdruck Typname;
```

Ab dieser Zeile ist dem Compiler der neue Name Typname bekannt und wird äquivalent zum beschriebenen Typausdruck behandelt. Der neue Typname ist somit ein sogenannter „Alias“ zum entsprechenden Typausdruck.

Durch die typedef-Vereinbarung wird kein neuer Datentyp eingeführt. Es wird lediglich ein zusätzlicher Name für einen existenten Typ oder einen komplexen Typausdruck eingeführt. In den Typausdrücken können auch bereits vorher mittels typedef definierte Typnamen vorkommen.



Eigene Typnamen sind besonders bei zusammengesetzten Datentypen nützlich, um Schreibarbeit zu sparen. Im Folgenden werden nun einige Beispiele gegeben:

14.2.1 Verwendung von typedef bei einfachen Datentypen

Die Definition von Typen mittels typedef funktioniert für beliebige Typ-Ausdrücke, somit auch für einfache Datentypen. Beispielsweise wird durch folgende Vereinbarung ein neuer Typ Integer definiert:

```
typedef int Integer;
```

Der Typname Integer ist nun synonym zu int. Der Typ Integer kann sodann bei Vereinbarungen, Umwandlungsoperationen und ähnlichem genauso verwendet werden wie der Typ int selbst:

```
Integer len;  
Integer* array[8];
```

Sinnvoll ist die Definition solcher Aliasnamen beispielsweise beim Schreiben portabler Programme, was in Anhang [→ E.3.1](#) nachgelesen werden kann.

14.2.2 Verwendung von typedef bei komplexen Datentypen

Wie in Kapitel [→ 14.1](#) angesprochen, können Typen zu komplexen Typausdrücken zusammengesetzt werden. Um die verwirrende und lästige Wiederholung des Schreibens dieser Typausdrücke zu vermeiden, werden komplexe Typausdrücke häufig mittels typedef zu einem neuen Typ abgekürzt. Beispielsweise:

```
typedef int* IntPtr;  
typedef const struct Person* PersonPtr;  
typedef void** Handle;  
typedef double Vec3[3];  
typedef int (*FuncPtr)(char);
```

Hierbei ist zu beachten, dass auch Qualifikatoren wie const (siehe [→ 7.5.1](#)) zum Typausdruck hinzugehören.

Des Weiteren ist auch hier die Operatorenreihenfolge zu beachten, siehe Kapitel [→ 14.1](#). So bezeichnet Vec3 ein Array aus drei double-Werten und FuncPtr ist ein **Funktionspointer**.

14.2.3 typedef bei struct und enum

Grundsätzlich dürfen struct beziehungsweise enum bei einem Typausdruck für entsprechende Etiketten nicht weggelassen werden. Der Typ struct Punkt beispielsweise muss stets als struct Punkt hingeschrieben werden, einfach nur Punkt wird der Compiler als Fehler markieren.

Mittels typedef kann dies jedoch umgangen werden. Das folgende Beispiel führt einen neuen Typnamen für einen vorliegenden zusammengesetzten Datentyp ein:

```
struct Student {  
    int matrikelnummer;  
    char name[20];  
    char vorname[20];  
    struct adresse wohnort;  
};  
  
typedef struct Student Studi;  
Studi studenten[50];
```

Ohne typedef hätte man schreiben müssen:

```
struct Student studenten[50];
```

So spart man sich das lästige Schreiben von struct und verbessert die Lesbarkeit eines Programms.

In C++ ist im Gegensatz zu C die typedef-Zeile nicht nötig. Ein mit struct MyStruct vereinbarter Typ ist per sofort als der Typ MyStruct verfügbar.



Um eine ähnliche Vereinfachung auch in C zu erreichen, kann man folgende Zeilen schreiben:

```
typedef struct Student Student;  
Student studenten[50];
```

Dadurch wird durch Angabe des Typs `Student` automatisch der Typausdruck `struct Student` verwendet. Der Name des neuen Typs darf gleich sein wie der name des `struct`-Etiketts. Dies ist möglich, da die beiden Namen in unterschiedlichen „Namespaces“ (siehe Kapitel [→ 14.3](#)) liegen.

Dieser Trick wird sehr häufig angewendet, auch um sogenannte „opaque“ (nicht durchsichtige) Typen zu definieren:

14.2.4 Vorwärtsdeklaration und opaque Typen mit typedef

Häufig wird bei der Vereinbarung eines `struct`-Typs folgendes Schema angewendet:

```
typedef struct MyStruct MyStruct;  
struct MyStruct {....};
```

Wie bereits in Kapitel [→ 14.2.3](#) beschrieben, wird damit ein Typ mittels `typedef` vereinbart, der denselben Namen hat wie die Struktur mit dem gleichnamigen Etikett. Hier jedoch steht die Zeile mit dem `typedef` **vor** der eigentlichen Deklaration des `struct`-Typs.

Es ist zu beachten, dass das `typedef`-Schlüsselwort keinen vollständigen Typ verlangt. Der durch das `typedef` vereinbarte, unvollständige Typ bleibt solange unvollständig, bis er vervollständigt wird.



Der neue Typname ist somit eine **Vorwärtsdeklaration**. Die eigentliche Deklaration des `struct`-Typs kann irgendwann danach erfolgen.

Ein Vorteil hierbei ist, dass der mittels `typedef` definierte Typname selbst innerhalb der Strukturdefinition `{ . . . }` bereits als ein Pointer verwendet werden kann. Beispielsweise:

```
struct MyStruct {  
    MyStruct* next;  
    MyStruct* prev;  
};
```

Zudem ist eine solche Vorwärtsdeklaration nützlich für das sogenannte „**Information Hiding**“, wo mittels einer solchen typedef-Vereinbarung ein Typ zur Verfügung gestellt wird, die tatsächliche Definition der dahinterliegenden Struktur jedoch in einer nicht zugänglichen Quelldatei steht.

Sprich, die Zeile mit dem typedef befindet sich in einer .h-Datei, welche von allen mittels #include eingebunden werden kann. Die eigentliche Deklaration jedoch steht in einer .c-Datei. Diese Definition ist in anderen Dateien nicht sichtbar, der Typ ist somit „**opak**“, also nicht-transparent. Dadurch steht jedoch außerhalb des Definitionsbereiches des Typs auch dem Compiler nur der Name zur Verfügung, weswegen solche Typen dort nur mittels Pointer angesprochen werden können.

Versteckte, also opaque Typen können nur als Pointer verwendet werden.



14.2.5 Definition eines struct-Typs plus zusätzlichem typedef

Es ist möglich, auf einmal sowohl einen neuen struct-Typ als auch ein typedef einzuführen.



Dies wird im folgenden Beispiel gezeigt:

```
typedef struct Point {  
    int x;  
    int y;  
} Punkt;
```

Innerhalb der geschweiften Klammern werden die Komponenten des neuen Datentyps struct Point festgelegt. Der neue Typname ist Punkt.

Damit können Punkte über die folgenden beiden Zeilen äquivalent definiert werden:

```
struct Point p1;  
Punkt p2;
```

Ferner könnte das Etikett `Point` weggelassen werden:

```
typedef struct {  
    int x;  
    int y;  
} Punkt;
```

Dadurch wird die Typvereinbarung noch einfacher. Es gibt jedoch Compiler, die bei Fehlern mit solchen „unbenannten“ Strukturen sehr unleserliche Fehlermeldungen generieren.

14.3 Namensräume

Die Programmiersprache C verwendet sogenannte „**Namensräume**“, um die im Programmcode geschriebenen Namen zu speichern.

Namen, die zu verschiedenen Namensräumen gehören, dürfen auch innerhalb desselben Gültigkeitsbereichs gleich sein.



Nach dem ISO-Standard werden die folgenden Namensräume unterschieden:

- Ein Namensraum für Namen von Marken.
- Ein gemeinsamer Namensraum – nicht drei – für die Namen von Etiketten von Strukturen (`struct`), Unionen (`union`) und Aufzählungs-Typen (`enum`).
- Namensräume für die Komponentennamen von Strukturen und Unionen. Dabei hat jede Struktur oder Union ihren eigenen Namensraum für ihre Komponenten.
- Ein Namensraum für alle anderen Bezeichner von Variablen, Funktionen, typedef-Namen, Aufzählungs-Konstanten.

Mit anderen Worten ist demnach das Folgende durchaus möglich:

```
typedef struct Punkt { ... } Punkt;
```

Hier wird der Typ `struct Punkt` und sein Aliasname `Punkt` eingeführt. `Punkt` als Etikett-Name einer Struktur und `Punkt` als `typedef`-Name liegen in verschiedenen Namensräumen.

Hingegen meldet der Compiler einen Fehler bei folgenden Deklarationen, da es für alle Etiketten nur einen einzigen Namensraum gibt:

```
struct Farbe {int rot; int gruen; int blau;};  
enum Farbe {rot, gruen, blau};
```

Die Aufzählungs-Konstante `rot` und der Komponentename `rot` stören sich jedoch nicht, denn sie liegen wiederum in unterschiedlichen Namensräumen.

Generell gilt für die Sichtbarkeiten von Namen:

- Namen in inneren Blöcken sind nach außen nicht sichtbar.
- Externe Namen und Namen in äußeren Blöcken sind in inneren Blöcken sichtbar, wenn sie nicht verdeckt werden.
- Namen in inneren Blöcken, die identisch zu externen Namen oder Namen in einem umfassenden Block sind, verdecken im inneren Block die externen Namen beziehungsweise die Namen des umfassenden Blocks durch die Namensgleichheit, wenn sie im selben Namensraum sind.

15 Speicherklassen



Um ein Programm lauffähig zu machen, muss es verschiedene Stationen durchlaufen. Es geht darum, dass ein Compiler Quelldateien in Maschinensprache übersetzt und in sogenannten „Objektdateien“ speichert, welche sodann von einem Linker zu einem ganzen Programm zusammengesetzt werden, welches schlussendlich durch einen Lader in den Speicher geladen wird. Eine erste Übersicht zu diesem Thema wurde bereits in Kapitel → 5 gegeben.

Hier in diesem Kapitel geht es nun darum, wie mittels sogenannter „**Speicherklassen**“ die Definition von Variablen und Funktionen gesteuert werden kann, sodass Compiler, Linker und Lader die Daten an genau den richtigen Orten speichern und das Programm darauf zugreifen kann.

Um Speicherklassen zu verstehen, werden hier zunächst nochmals einige Grundprinzipien wiederholt:

Ein Compiler hat die Aufgabe, den erzeugten Maschinencode geeignet anzuordnen. Hierzu gehört zum einen der Programmcode selbst, als auch die Daten, welche mittels Variablen angesprochen werden können.

Die Namen von Variablen sind zur Laufzeit nicht mehr vorhanden. Der Compiler errechnet für Variablen, wie viel Speicherplatz sie benötigen und welche relative Adresse sie haben. Zur Laufzeit wird mit den Adressen der Variablen und nicht mit ihren Namen gearbeitet.



Typinformationen sind in C nur für den Compiler wichtig. Im ausführbaren Programm sind im Falle von C keine Informationen über die Typen enthalten.

Ein Compiler wird jedoch anhand der Typinformation für jede Variable ausreichend Speicher reservieren. An welchem Ort dieser Speicher reserviert wird, hängt von der Art einer Variablen ab.

Ergänzende Information Die elektronische Version dieses Kapitels enthält Zusatzmaterial, auf das über folgenden Link zugegriffen werden kann https://doi.org/10.1007/978-3-658-45209-4_15.

Es wird grundsätzlich zwischen **lokalen** (internen) und **globalen** (externen) Variablen unterschieden. Lokale Variablen werden zu Beginn eines Blockes angelegt und leben solange, bis dieser Block abgearbeitet ist. Sie sind nur innerhalb des Blockes sichtbar. Globale Variablen leben so lange wie das Programm.

Globale Variablen sind für alle Funktionen sichtbar, die nach ihrer Definition in derselben Datei definiert werden.



Sowohl lokale als auch globale Variablen können sogenannten Speicherklassen zugeordnet werden.



Was diese Speicherklassen bedeuten und welche Auswirkungen sie auf die Speicherung der in den Variablen befindlichen Daten hat, wird in den folgenden Kapiteln erklärt.

C11 kennt die Speicherklassen

- typedef
- extern
- static
- _Thread_local
- auto
- register



typedef zählt nur formal (syntaktisch) zu den Speicherklassen.



Die Speicherklasse `_Thread_local` wird in Kapitel [→ 23.5.3](#) bei den in C11 optionalen Threads behandelt.

Die Speicherklassen `extern`, `static`, `auto` und `register` werden später in diesem Kapitel besprochen.

15.1 Der Linker

C unterstützt eine getrennte Compilierung. Dies bedeutet, dass es möglich ist, ein großes Programm in Module zu zerlegen, die getrennt compiliert werden.

Der Begriff Modul ist vieldeutig. Hier wird unter Modul eine separat compilierfähige Einheit, mit anderen Worten eine Übersetzungseinheit, verstanden. Ein Modul entspricht einer Quelldatei. Verschiedene Quelldateien werden getrennt übersetzt.



Für jede Quelldatei wird eine **Objektdati** mit Maschinencode erzeugt. Dieser Maschinencode ist nicht ablauffähig, zum einen, weil die Bibliotheksfunktionen noch fehlen, zum anderen, weil die Adressen von Funktionen und Variablen anderer Dateien noch nicht bekannt sind. Die Aufgabe des **Linkers** ist es nun, die Querbezüge zwischen den Dateien herzustellen. Er bindet die erforderlichen Bibliotheksfunktionen und das Laufzeitsystem hinzu und bildet einen Adressraum für das Gesamtprogramm, sodass jede Funktion und globale Variable an einer eindeutigen Adresse liegt.

15.1.1 Virtuelle Adressen

Der Linker erzeugt das ausführbare Programm. Er baut einen virtuellen Adressraum des Programms auf, der aus **virtuellen Adressen** (auch logische Adressen genannt) besteht, die einfach der Reihe nach durchgezählt werden. In diesem virtuellen Adressraum hat jede Funktion und jede globale Variable ihre eigene Adresse.



Überschneidungen, dass mehrere Funktionen oder globale Variablen an derselben Adresse liegen, darf es nicht geben.

Der Linker fügt die einzelnen Adressräume der durch den Compiler erstellten Objektdateien sowie erforderlicher Bibliotheks-Dateien so zusammen, dass sich die Adressen nicht überlappen und dass die Querbezüge gegeben sind.



Hierzu stellt er eine Symbol-Tabelle (Linker Map) her, welche alle Querbezüge (Adressen globaler beziehungsweise externer Variablen, Einsprungsadressen der Funktionen) enthält. Der Linker bindet die compilierten Objektdateien, die aufgerufenen Bibliotheksfunktionen und das Laufzeitsystem (Fehlerfunktionen, Speicherverwaltungsroutinen, etc.) zu einem ablauffähigen Programm (**executable program**). Durch den Linkvorgang wird ein einheitlicher Adressraum für das gesamte Programm hergestellt.

Der Linker legt nur virtuelle Adressen in einem virtuellen Adressraum fest. Damit ist das Programm im Arbeitsspeicher verschiebbar. Die Zuordnung von virtuellen zu Adressen mit einer eindeutigen Position in den Memory-Chips führt dann die Komponente Memory Management des Betriebssystems (gegebenfalls unter Zuhilfenahme der Memory Management Unit der Hardware) durch.

15.1.2 Bindungen

Außer dem Gültigkeitsbereich und der Lebensdauer hat eine Variable und eine Funktion auch eine sogenannte „**Bindung**“.

Globale Variablen und Funktionen haben eine **externe Bindung**. Dies bedeutet, dass eine Funktion oder globale Variable aus allen Dateien eines Programms unter dem gleichen Namen ansprechbar ist und dass dieser Name eindeutig für das ganze Programm ist.



Globale Variablen und Funktionen, die als **static** vereinbart werden, haben eine **interne Bindung**. Ihr Name ist nur innerhalb ihrer eigenen Datei sichtbar und kollidiert nicht mit demselben Namen mit interner Bindung in anderen Dateien.



Dies wird in Kapitel [→ 15.4](#) genauer beschrieben.

Es ist zu unterscheiden zwischen externen (globalen) Variablen und externer Bindung. Der Begriff „externe Variable“ bedeutet lediglich, dass die Variable außerhalb (extern) von jeglichen Funktionen definiert ist. Eine „externe Bindung“ bedeutet jedoch, dass sie auch außerhalb der Übersetzungseinheit ansprechbar ist.



Bei der Übersetzung eines Programmes müssen die verwendeten Variablennamen und Funktionsnamen ihren jeweiligen Definitionen, sprich den Speicherorten von Variablen beziehungsweise Implementierungen der Funktionen zugeordnet werden.

Bei Bezeichnen mit interner Bindung müssen die jeweiligen Definitionen direkt vorliegen, das heißt sich in der zu übersetzenden Datei befinden.



Die Zuordnung erledigt dann der Compiler.

Bei externer Bindung muss sich die Definition von Variablen beziehungsweise Funktionen nicht in derselben Übersetzungseinheit befinden.



Die Zuordnung von Bezeichnern mit externer Bindung zu ihren entsprechenden Definitionen über mehrere Objektdateien hinweg wird vom Linker übernommen.

Objekte mit interner Bindung existieren eindeutig in einer Datei, Objekte mit externer Bindung eindeutig für das ganze Programm.



In einer Übersetzungseinheit können mehrere Deklarationen mit gleichem Namen existieren, solange sie in Typ und Bindung übereinstimmen. Es muss jedoch immer genau eine Definition dieses Namens existieren. Die Definition kann sich entweder in derselben Übersetzungseinheit oder aber in einer anderen Übersetzungseinheit befinden. Entsprechend werden schlussendlich alle Deklarationen eines Namens entweder durch den Compiler oder aber den Linker mit dieser einen Definition verbunden.



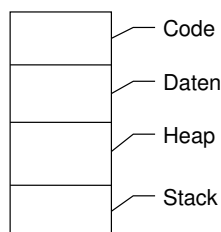
Mehrfache identische Deklarationen sind also möglich. Erscheint jedoch in derselben Übersetzungseinheit (Datei) derselbe Name mit externer und interner Bindung, dann ist das Verhalten undefiniert.

15.2 Der Lader und die Segmente des Adressraums eines Programms

Nachdem ein Linker ein Programm vollständig erstellt hat, kann es auf dem dafür vorgesehenen Prozessor ausgeführt werden. Dazu muss es jedoch in den Speicher geladen werden.

Bei hardwarenaher Programmierung wird das vollständige Programm entsprechend direkt in einen Speicherchip transferiert. Soll das Programm jedoch als Teil eines Betriebssystems laufen, so muss der sogenannte „**Lader**“ den zur Verfügung stehenden Speicher vorbereiten und aufteilen, sodass die Vorgaben des Betriebssystems erfüllt sind und das Programm sicher ausführbar wird.

Der schlussendlich einem Programm zur Verfügung stehenden Speicher wird als „Adressraum“ bezeichnet. Dieser Adressraum kann bei einem C-Programm folgendermaßen unterteilt werden:



Der Adressraum eines ablauffähigen C-Programms besteht grob aus den vier **Segmenten** (Regionen): Code, Daten, Heap und Stack.



Dabei kann das englische Wort „heap“ mit dem deutschen Wort „Halde“ und das Wort „stack“ mit „Stapel“ übersetzt werden.

15.2.1 Code-Segment

Im **Code-Segment** (auch als **Text-Segment** bekannt) liegt das Programm in Maschinensprache vor.



Häufig werden in diesem Segment auch Konstanten (insbesondere konstante Strings) abgelegt. Damit der Maschinencode oder die Konstanten nicht verändert werden können, kann das Code-Segment mit einem Schreibschutz versehen werden, wenn dies vom Betriebssystem und der Hardware unterstützt wird.

15.2.2 Daten-Segment

Globale Variablen werden im **Daten-Segment** abgelegt. Dieses Segment wird beim Programmstart vom Lader automatisch bereitgestellt und die einzelnen Variablen werden im Falle einer manuellen Initialisierung korrekt initialisiert. Nicht manuell initialisierte Variablen werden automatisch mit null initialisiert.



Im Gegensatz zum Code-Segment sind die Daten innerhalb des Daten-Segments nicht mit einem Schreibschutz versehen, sondern können verändert werden. Es ist jedoch nicht möglich, die Menge an hierfür benötigtem Speicherplatz während der Laufzeit des Programmes zu verändern. Es steht soviel Platz im Daten-Segment zur Verfügung, wie das Programm bei seinem Start belegt hat.

15.2.3 Heap-Segment

Im Heap können dynamische Speicherblöcke angelegt werden.

Dynamische Speicherblöcke werden im **Heap-Segment** (oder kurz: auf dem „**Heap**“) beispielsweise mit Hilfe der Bibliotheksfunktion `malloc()` angelegt. Sie können nicht über einen Namen angesprochen werden, sondern nur über den Pointer, den die Funktion `malloc()` liefert.



Entsprechende Funktionsaufrufe werden im Detail in Kapitel [→ 18](#) behandelt.

Lebensdauer und Gültigkeit von dynamischen Speicherblöcken unterliegen nicht den Blockgrenzen der Funktion, innerhalb der sie geschaffen wurden.



Dynamische Speicherblöcke sind gültig und sichtbar bis zu ihrer expliziten Vernichtung durch die Bibliotheksfunktion `free()` beziehungsweise bis zum Programmende.

Bei modernen Betriebssystemen bildet das Heap-Segment den weitaus größten Teil des Adressraumes ab.

15.2.4 Stack-Segment

Im **Stack-Segment** (oder kurz: auf dem „**Stack**“) werden lokale Variablen, die Parameter einer Funktion und die Rücksprungadresse eines Funktionsaufrufes abgelegt.



Auf die Funktionsweise eines Stacks wurde ausführlich in Kapitel [→ 11.8.1](#) eingegangen.

15.3 Die Speicherklasse extern bei Programmen aus mehreren Dateien

Im Folgenden soll als Beispiel ein Programm aus zwei Dateien betrachtet werden:

Datei ext1.c

```
int zahl = 6;  
  
main()
```

Datei ext2.c

```
f1()
```

Die Datei ext1.c enthält die Funktion main() und eine Variable zahl, die Datei ext2.c enthält die Funktion f1(). Die Funktion f1() möchte dabei auf die in der Datei ext1.c definierte Variable zahl zugreifen. Da diese Variable jedoch nicht in der eigenen Datei definiert ist, hat der Compiler keine Ahnung, dass eine solche Variable existiert.

Um einen Zugriff auf die Variable zahl innerhalb der Datei ext2.c dennoch zu ermöglichen, wird innerhalb der Datei ext2.c die Variable zahl vor ihrer Verwendung mit der Speicherklasse **extern** deklariert:

```
extern int zahl;
```

Formal sieht eine extern-Deklaration einer Variablen wie eine normale Vereinbarung aus, es wird nur zusätzlich das Schlüsselwort **extern** vorangestellt.



Somit ist die Variable zahl dem Compiler an der Stelle ihrer Verarbeitung in f1() bekannt. Diese Vereinbarung ist jedoch durch die Angabe des extern-Schlüsselworts explizit keine Definition, sondern nur eine Deklaration. Schlussendlich muss der Linker die Verbindung zwischen beiden Dateien herstellen, so dass der Zugriff auf die in der Datei ext1.c definierte Variable zahl erfolgt.

Eine mit extern deklarierte Variable darf nur in einer einzigen Datei definiert werden, in den anderen Dateien wird sie mit Hilfe der extern-Deklaration referenziert. Die Definition legt die Adresse einer globalen Variablen fest. Der Linker setzt in den anderen Dateien, die über die extern-Deklaration diese Variable referenzieren, die Adresse dieser Variablen ein.



Eine globale Variable kann an der Stelle einer extern-Deklaration manuell nicht initialisiert werden. Eine manuelle Initialisierung ist nur bei ihrer Definition möglich.

Globale Variablen werden vom Compiler an der Stelle ihrer Definition automatisch mit 0 initialisiert, falls sie nicht manuell initialisiert werden.



Um die Übersicht zu wahren, werden extern-Deklarationen meist außerhalb aller Funktionen zu Beginn einer Datei angeschrieben. Die so deklarierten globalen Variablen und Funktionen gelten dann in der ganzen Datei. Dabei werden sie oft in einer Header-Datei abgelegt, die zu Beginn der Datei mit #include eingefügt wird.



Auch Funktionen können mit dem extern-Schlüsselwort deklariert werden. Da Funktionen jedoch vom Compiler standardmäßig in die Speicherklasse extern gesetzt werden, kann das Schlüsselwort extern auch weggelassen werden.

In den folgenden beiden Unterkapiteln werden Beispielprogramme gezeigt für die Verwendung der extern-Deklaration. Auf das Schreiben eigener Header-Dateien wird ausführlich in Kapitel [→ 21](#) eingegangen. Weitere Beispielprogramme für die Nutzung von extern-Deklarationen und Header-Dateien sind in Kapitel [→ 22](#) zu finden.

15.3.1 Beispielprogramm

Um die Verwendung der Speicherklasse extern zu veranschaulichen, wird hier ein einfaches Beispiel gegeben:

ext1without.c

```
#include <stdio.h>

void f1(void);

int zahl = 6;

int main(void) {
    printf("Hier ist main, zahl = %d\n", zahl);
    f1();
    return 0;
}
```

ext2without.c

```
#include <stdio.h>

extern int zahl;

void f1(void) {
    printf("Hier ist f1, zahl = %d\n", zahl);
}
```

Die Ausgabe dieses Programmes ist:

```
Hier ist main, zahl = 6
Hier ist f1, zahl = 6
```

Es ist zu beachten, dass in der Datei `ext1without.c` die Funktion `f1()` als Prototyp angegeben werden muss, da ansonsten der Compiler die Funktion innerhalb der `main`-Funktion nicht kennt. Wie bereits erwähnt könnte dem Prototyp zur Verdeutlichung noch das Schlüsselwort `extern` vorangestellt werden.

In der Datei `ext2without.c` muss die Variable `zahl` mit Hilfe einer `extern`-Deklaration eingeführt werden, damit der Compiler die Nutzung dieser Variablen überprüfen kann, beispielsweise ob sie entsprechend ihres Typs verwendet wird.

15.3.2 Beispielprogramm extern-Deklaration mit Header-Datei

Im vorangegangenen Beispiel wurde deutlich, dass beide Dateien jeweils Namen von Variablen oder Funktionen der jeweils anderen Datei benötigen. Dies ist in der alltäglichen Programmierung sehr oft der Fall, weswegen solche Prototypen und extern-Deklarationen häufig in eine separate Header-Datei geschrieben werden. Hier findet sich dasselbe Beispiel nun nochmals, diesmal jedoch mit einer Header-Datei:

ext.h

```
void f1(void);  
extern int zahl;
```

ext1.c

```
#include <stdio.h>  
#include "ext.h"  
  
int zahl = 6;  
  
int main(void) {  
    printf("Hier ist main, zahl = %d\n", zahl);  
    f1();  
    return 0;  
}
```

ext2.c

```
#include <stdio.h>  
#include "ext.h"  
  
void f1(void) {  
    printf("Hier ist f1, zahl = %d\n", zahl);  
}
```

Dieses Beispiel verdeutlicht nochmals, dass es sich bei einer extern-Deklaration nicht um eine Definition handelt. Würde das extern-Schlüsselwort in der Datei ext.h fehlen, so wäre dies eine Definition und da diese von zwei Dateien mittels #include eingebunden wird, wird der Compiler einen Fehler melden, da die Variable zahl mehrfach definiert wird.

15.3.3 Die extern-Deklaration bei Arrays

Die Größe eines Arrays kann entweder bei der Definition explizit mit angegeben werden, oder aber man lässt die Größe automatisch vom Compiler anhand der Initialisierungsliste berechnen. Bei einer extern-Deklaration hingegen kann auch ein Array ohne Längenangabe und ohne Initialisierungsliste stehen, da an dieser Stelle die Größe keine Rolle spielt, sondern nur über den Namen ein Bezug zu der eigentlichen Definition hergestellt werden soll.

```
extern int array[];
```

Will man trotzdem die konkrete Größe mit angeben, so muss darauf geachtet werden, dass diese Angabe mit derjenigen der Definition übereinstimmt. Ansonsten wird der Compiler einen Fehler ausgeben.

15.4 Die Speicherklasse `static` bei Programmen aus mehreren Dateien

Generell kann man alle globalen Variablen und Funktionen aus einer anderen Datei benutzen, indem man in seiner Datei entsprechende extern-Deklarationen aufnimmt. Dies ist aber nicht immer gewünscht. Um Definitionen vor Zugriff zu schützen, ist es in C möglich, Variablen und Funktionen innerhalb einer Datei so zu kapseln, dass diese nur innerhalb der eigenen Datei verwendet werden können.

Hat man beispielsweise in einer Datei zwei Funktionen, die auf eine globale Variable dieser Datei zugreifen müssen, möchte man aber den Funktionen von anderen Dateien den Zugriff auf die globale Variable verwehren, so kann man dies dadurch erreichen, dass man diese Variable durch Angabe des Schlüsselwortes **static** als Teil der static-Speicherklasse definiert.

Das Schlüsselwort `static` bedeutet „interne Bindung“ (auf Englisch „internal linkage“). Damit ist gemeint, dass eben genau an dieser Stelle eine Definition stattfindet und keine Deklaration, welche eine andere Variable referenziert. Dann nützt auch eine extern-Deklaration dieser Variablen in einer anderen Datei nichts. Die statische (`static`) globale Variable ist außerhalb ihrer eigenen Datei unsichtbar. Auf die gleiche Weise kann man auch verhindern, dass Funktionen der eigenen Datei missbräuchlich von Funktionen in anderen Dateien aufgerufen werden können.

Funktionen und globale Variablen, die der Speicherklasse static angehören, sind nur in ihrer eigenen Datei sichtbar. Das Schlüsselwort static führt hier zu einer internen Bindung des entsprechenden Namens. Funktionen aus anderen Dateien können auf diese Funktionen und Variablen nicht zugreifen.



Damit kann mit Hilfe der Speicherklasse static das Prinzip des Information Hiding in C-Programmen realisiert werden. Eine ausführliche Behandlung dieses Aspektes erfolgt in Kapitel [→ 22](#) .

Das folgende Beispiel zeigt, dass es der Funktion main() in der Datei stat1.c nicht möglich ist, auf die statische Funktion f1 in der Datei stat2.c zuzugreifen, obschon sie als Prototyp deklariert wurde:

stat1.c

```
void f1(void);

int main(void) {
    f1();
    return 0;
}
```

stat2.c

```
#include <stdio.h>

static void f1(void) {
    printf("Diese Funktion hat interne Bindung.\n");
}
```

Der Compiler wird in diesem Programm keine Fehler finden, beim Linken jedoch erscheint eine Fehlermeldung wie „Verweis auf nicht aufgelöstes externes Symbol f1 in Funktion main“.

15.5 Die Speicherklasse auto bei lokalen Variablen

Mit der Speicherklasse **auto** werden sogenannte „automatische Variablen“ definiert.

Automatische Variablen werden so bezeichnet, weil sie automatisch angelegt werden und automatisch verschwinden. Sie sind nur innerhalb des Blockes sichtbar, in dem sie definiert sind, und leben vom Aufruf des Blockes bis zum Ende des Blockes.



Lokale Variablen gehören standardmäßig der Speicherklasse auto an. Die explizite Angabe auto kann somit auch weggelassen werden und ist dementsprechend in C-Code kaum anzutreffen.

Ab dem C23-Standard hat das Schlüsselwortes auto eine Zusatzfunktion: Ab dann kann bei Variablendefinitionen die Typangabe entfallen und mittels auto der Typ automatisch aufgrund der Initialisierung ermittelt werden. Dies funktioniert jedoch im Gegensatz zu C++ nur bei Definitionen!



Automatische Variablen sind dasselbe wie **lokale Variablen** ohne Angaben einer Speicherklasse.



Auch formale Parameter stellen automatische Variablen dar.



Wird ein Block betreten, so wird eine automatische Variable – sei es eine automatische Variable der Speicherklasse auto beziehungsweise eine automatische Variable ohne Angabe einer Speicherklasse – auf dem Stack dort angelegt, wo der Stackpointer hinzeigt.

Automatische Variablen werden auf dem Stack angelegt.



Nach dem Verlassen des Blockes wird der Speicherplatz der Variablen auf dem Stack durch Modifikation des Stackpointers zum Überschreiben frei gegeben.

Eine automatische Variable wird bei Beendigung eines Blockes ungültig. Werden Pointer auf lokale Variablen an die aufrufende Funktion zurückgegeben, so zeigen diese auf ungültige Speicherinhalte.



Eine automatische Variable wird bei jedem Blockeintritt erneut angelegt. Wenn sie nicht manuell initialisiert wird, so ist der Wert der Variablen undefiniert. Es ist nicht garantiert, dass diese Variable bei nochmaligem Aufruf der Funktion oder des Blockes den vorherigen Wert noch enthält.



15.6 Die Speicherklasser register bei lokalen Variablen



Mit Hilfe des Schlüsselwortes **register** kann man dem Compiler empfehlen, formale Parameter und lokale Variablen in Registern des Prozessors statt im Arbeitsspeicher abzulegen.



Zugriffe auf Register des Prozessors sind schneller als Zugriffe auf den Arbeitsspeicher. Deswegen kann man in C eine Variable oder einen Parameter mittels des `register`-Schlüsselwort markieren mit dem Ziel, dass der Compiler den Zugriff möglichst optimieren soll. Ob der Compiler dieser Aufforderung nachkommt, ist jedoch situationsabhängig.

Die Ablage in Registern ist nur für bestimmte Datentypen möglich. Der Compiler muss diesem Wunsch nicht nachkommen. Auf register-Variablen darf der Adress-Operator & nicht angewandt werden.



Wenn die register-Speicherklasse – aus welchen Gründen auch immer – nicht zur Anwendung kommt, so wird eine damit deklarierte Variable automatisch der auto Speicherklasse zugewiesen.

Hier ein Beispiel für die Verwendung des Schlüsselwortes register:

```
void beispiel(register int a) {  
    register int b;  
}
```

Die Speicherklasse register sollte nur für häufig benutzte Variablen eingesetzt werden, um die Zugriffszeit zu reduzieren. Gut optimierende Compiler erkennen jedoch solche Variablen durch geeignete Analyseverfahren und legen diese Variablen selbstständig – wenn immer möglich – in Registern ab.



Daher ist die Verwendung von register nur in speziellen Fällen vorteilhaft.

register ist die einzige Speicherklasse, die bei einem formalen Parameter angegeben werden kann.



15.7 Die Speicherklasse `static` bei lokalen Variablen

Wird eine **lokale Variable** mit dem Schlüsselwort **`static`** versehen, so dient sie als lokaler, aber permanenter Speicher innerhalb eines Blockes oder einer Funktion. Permanent bedeutet hier, dass die Variable ihren Wert zwischen zwei Funktionsaufrufen nicht verliert.

Statische lokale Variablen werden vom Compiler nicht auf dem Stack angelegt, sondern im Speicherbereich der globalen Variablen. Sie werden dadurch permanent und behalten ihren Wert auch zwischen zwei Funktionsaufrufen bei. Eine manuelle Initialisierung wird nur beim ersten Aufruf ausgeführt. Statische lokale Variablen haben also eine andere Lebensdauer als normale lokale Variablen. Bezüglich der Sichtbarkeit verhalten sich statische lokale Variablen aber nach wie vor wie normale lokale Variablen.



Dass eine manuelle Initialisierung von statischen lokalen Variablen nur beim ersten Aufruf durchgeführt wird, zeigt das folgende Beispiel:

`static.c`

```
#include <stdio.h>

void beispiel(void) {
    static int a = 1;
    printf("a = %d\n", a);
    ++a;
}

int main(void) {
    for (int i = 0; i <= 2; ++i) {
        beispiel();
    }
    return 0;
}
```

Hier die Ausgabe des Programms:

```
a = 1
a = 2
a = 3
```

Aus der Ausgabe wird ersichtlich, dass in der Variablen `a` die Anzahl der Aufrufe der Funktion mitgezählt wird. Ohne das Schlüsselwort `static` ginge das nicht.

Statische lokale Variablen werden wie globale Variablen automatisch mit 0 initialisiert, wenn sie nicht manuell initialisiert werden. Gewöhnen Sie sich aus Sicherheitsgründen bei statischen Variablen eine manuelle Initialisierung an!



Das folgende Beispiel zeigt, dass eine statische lokale Variable als permanenter Speicher während der gesamten Programmausführung dienen kann im Gegensatz zu einer normalen lokalen Variablen, deren Speicher nach jedem Funktionsaufruf zum Überschreiben freigegeben wird.

bank.c

```
#include <stdio.h>

void bilanz(float buchung) {
    static float summe = 0.f;
    printf("Zu Beginn ist der Kontenstand %.2f\n", summe);
    printf("Gebucht wurde: %.2f\n", buchung);
    summe += buchung;
    printf("Der neue Kontenstand ist: %.2f\n", summe);
}

int main(void) {
    bilanz(15.);
    bilanz(30.);
    return 0;
}
```

In diesem Falle ist die Variable `summe`, die den Kontostand enthält, permanent. Der Kontostand wird zwar durch eine Buchung in dem Funktionsaufruf geändert, bleibt dann aber bis zur nächsten Buchung erhalten.

Die Ausgabe des Programms ist:

```
Zu Beginn ist der Kontenstand 0.00
Gebucht wurde: 15.00
Der neue Kontenstand ist: 15.00
Zu Beginn ist der Kontenstand 15.00
Gebucht wurde: 30.00
Der neue Kontenstand ist: 45.00
```

15.8 Automatische und nicht automatische Initialisierung

In der Programmiersprache C werden globale und statische lokale Variablen bei der **Initialisierung** anders behandelt als automatische Variablen.

Globale Variablen und statische lokale Variablen werden **automatisch** mit 0 initialisiert. Automatische Variablen werden nicht automatisch initialisiert. Ihr Wert ist undefiniert und entspricht dem zufälligen Bitmuster an ihrer Speicherstelle – ein sinnfreier Wert.



Im Falle einer manuellen Initialisierung werden globale und statische Variablen nur ein einziges Mal initialisiert. Automatische Variablen, die manuell initialisiert werden, erhalten diesen Wert bei jedem Blockeintritt beziehungsweise Funktionsaufruf von neuem.



Globale und statische Variablen werden manuell initialisiert durch konstante Ausdrücke, automatische Variablen können durch Werte beliebiger Ausdrücke – auch Funktionsaufrufe – manuell initialisiert werden.



Bei automatischen Variablen stellt eine manuelle Initialisierung letztlich nichts anderes als eine Abkürzung für eine Zuweisung dar.

15.9 Überblick über die Speicherklassen

Die folgende Tabelle zeigt einige Eigenschaften der verschiedenen Speicherklassen im Überblick:

	Gültigkeit	Lebensdauer	automat. Initialisierung	Segment
register	Block	Block	nein	Stack oder in Register
auto	Block	Block	nein	Stack
static lokale Variable	Block	Programm	mit 0	Daten
static globale Variable	in Datei ab Definition, in anderen Dateien nicht	Programm	mit 0	Daten
extern	in Datei ab Definition, beziehungsweise ab extern-Deklaration	Programm	mit 0	Daten

15.10 Übungsaufgaben

Aufgabe 1: Speicherklassen. Programme aus mehreren Dateien

Betrachten Sie folgendes **globales** Array:

```
float arr[] = {62.7, 2.33, 75.27, 26.51, 11.94, 0.};
```

- Erstellen Sie eine Datei `main.c`, welche dieses Array als globale Definition beinhaltet. Schreiben sie eine `main()`-Funktion, die vorerst noch leer bleibt.
- Schreiben Sie eine Funktion `anzahl()`, welche die Länge des Arrays ermittelt. Nehmen Sie an, dass das letzte Element des Arrays stets 0 ist.
- Schreiben Sie die Funktionen `summe()` und `durchschnitt()`, welche auf die Funktionalität von `anzahl()` zugreifen. Die Funktionen sollen dabei **keine** Parameter nutzen.
- Schreiben Sie die drei Funktionen `anzahl()`, `summe()` und `durchschnitt()` je in eine eigene Datei und rufen sie in der Funktion `main()` den Durchschnitt auf. Schreiben Sie dazu eine Header-Datei.

Aufgabe 2: Speicherklasse extern

In dieser Aufgabe schreiben Sie 3 Dateien: `error.h`, `error.c` sowie die Datei `main.c`, welche die `main()`-Funktion enthalten soll. Der untenstehende Code befindet sich in der Header-Datei. Er stellt mehrere Konstanten zur Verfügung sowie den Namen einer globalen Variablen. Schreiben Sie ein Programm, welches in der `main()`-Funktion für eine gegebene Fehlermeldungs-Id einen entsprechenden String ausgibt, beispielsweise „Kein Speicher mehr verfügbar“ für die Konstante `ERROR_OUT_OF_MEMORY`. Definieren Sie hierfür die Strings in der Datei `error.c` und greifen Sie direkt in der Datei `main.c` darauf zu.

```
typedef enum {
    NO_ERROR,
    ERROR_DIVISION_BY_ZERO,
    ERROR_OUT_OF_MEMORY,
    ERROR_NULL_POINTER_EXCEPTION,
    ERROR_COUNT
} ErrorId;

extern const char* errorMessages[ERROR_COUNT];
```

Aufgabe 3: Speicherklasse static

Studieren Sie das folgende Programm. Angenommen, die Funktion `getPrime()` würde sehr häufig aufgerufen werden. Welche Probleme treten auf? Wie könnte man dies verbessern?

static.c

```
#include <stdio.h>

int getPrime(int i) {
    int primes[] = {
        2, 3, 5, 7, 11, 13, 17, 19, 23,
        31, 37, 41, 43, 47, 59, 61, 67,
        71, 73, 79, 83, 89, 97};
    return (i > 0 && i < 23) ? primes[i] : 0;
}

int main(void) {
    printf("Primzahl mit Index 5: %d\n", getPrime(5));
    printf("Primzahl mit Index 15: %d\n", getPrime(15));
    printf("Primzahl mit Index 22: %d\n", getPrime(22));
    return 0;
}
```


16 Ein- und Ausgabe



Programme laufen fast immer nach dem Schema „Eingabe/Verarbeitung/Ausgabe“ (EVA-Prinzip) ab. Schon daran ist die Wichtigkeit der **Ein- und Ausgabe** von Programmen zu erkennen.

Ein gutes Ein-/Ausgabe-System ist für die Akzeptanz einer Programmiersprache von großer Bedeutung.



Die Ein- und Ausgabe in einem Programm kann unterschiedlichen Zwecken dienen. Die wichtigsten Fälle sind: Kommunikation mit einem Anwender über die Tastatur, den Bildschirm etc. und Zugriff auf die Daten, die in einem Dateisystem auf einem Massenspeicher langfristig gespeichert sind.

Für die Ein- und Ausgabe stellt C eine Standardbibliothek zur Verfügung. Durch Einbinden der Standard-Bibliothek `<stdio.h>` kann auf eine große Anzahl nützlicher Prototypen von Ein- und Ausgabefunktionen zugegriffen werden, die auf dem Konzept von Datenströmen (auf Englisch „streams“) basieren.



Ein **Datenstrom** kann eine Datenquelle wie eine Tastatur beziehungsweise ein Datenziel wie einen Bildschirm mit dem Programm verbinden. Mit Hilfe eines Datenstroms kann ein Programm aber auch mit einer Datei verbunden werden. Dateien können dabei sowohl Datenquelle als auch Datenziel sein.

Das Architekturprinzip von UNIX, nämlich Dateien und Peripheriegeräte über Datenströme in gleicher Weise zu behandeln, ist ebenso auch in C zu finden.



Es muss hier noch erwähnt werden, dass die Sprache C keinen Standard für grafische Oberflächen definiert und dass solche Programme nur über zusätzliche, meist betriebssystem- und/oder compilerabhängige Bibliotheken für grafische Oberflächen erstellt werden können. Darauf kann im Rahmen dieses Buches nicht eingegangen werden.

Ergänzende Information Die elektronische Version dieses Kapitels enthält Zusatzmaterial, auf das über folgenden Link zugegriffen werden kann https://doi.org/10.1007/978-3-658-45209-4_16.

16.1 Speicherung von Daten in Dateisystemen

Bei der Datenverarbeitung ist die automatische Verarbeitung von Daten mit Hilfe von Programmen das Hauptziel. Programme, die vom Prozessor bearbeitet werden sollen, müssen für die Dauer ihrer Abarbeitung in den Hauptspeicher geladen werden. Die dauerhafte Speicherung der Programme und ihrer Daten erfolgt auf Massenspeichern.

Massenspeicher wie zum Beispiel eine Festplatte werden mit Hilfe des Betriebssystems angesprochen und verwaltet. Die Komponente des Betriebssystems, die dafür verantwortlich ist, ist das **Dateisystem** (auf Englisch „**file system**“).

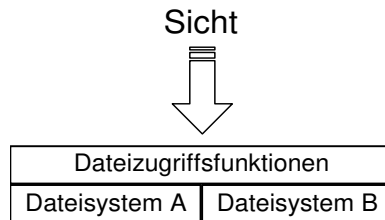


Nicht jedes Betriebssystem muss ein Dateisystem haben. In der Prozessdatenverarbeitung werden auch Betriebssysteme verwendet, die keine Massenspeicher verwalten. Bei Betriebssystemen, welche auf Massenspeicher zugreifen, konnte aufgrund der Vielfalt der Geräte, ihrer unterschiedlichsten Hardware-Eigenschaften und Verwaltungs-Mechanismen der Zugriff auf Dateien der peripheren Speicher früher im Allgemeinen nicht direkt durch die Programmiersprache erfolgen, sondern wurde durch Funktionen des Betriebssystems ausgeführt, die für die entsprechende Programmiersprache zur Verfügung gestellt wurden. Für die verschiedenartigen Massenspeicher wie Festplatten, Disketten, CDs, Memorysticks etc. wurden im Laufe der Zeit durch die Betriebssysteme zunehmend komfortablere Dateisysteme bereitgestellt.

Eine **Datei** (auf Englisch „**file**“) ist eine Zusammenstellung von logisch zusammengehörigen Daten, die als eine Einheit behandelt werden und unter einem dem Betriebssystem bekannten Dateinamen (file name) abgespeichert werden.



Erst durch UNIX, das Geräte als Dateien behandelte, und durch das Konzept der Datenströme wurde es möglich, von der Programmiersprache aus mit Dateizugriffsfunktionen direkt auf Peripheriegeräte zuzugreifen. Das spezielle Dateisystem bleibt dabei verborgen. Dies symbolisiert das folgende Bild:



Über einen Datenstrom kann man direkt auf die Festplatte schreiben oder von ihr lesen (siehe Kapitel [→ 16.6](#) und [→ 16.8](#)). Programme können damit leichter portiert werden, da sie nicht von den Interna des Dateisystems eines bestimmten Betriebssystems abhängen, sondern eine wohldefinierte Programmschnittstelle benutzen.

16.1.1 Dateien unter UNIX – das Streamkonzept

Die Programmiersprache C wurde geschrieben, um mit ihrer Hilfe das Betriebssystem UNIX zu implementieren. Die Architekturvorstellungen beim Entwurf von UNIX spiegeln sich deshalb auch in C wider.

Eine Datei unter UNIX ist ein einziger mit einem Namen versehener Datensatz beliebiger Länge, der aus einer Folge von Bytes besteht. Ein solcher Datensatz aus beliebig vielen Bytes wird im Englischen als Datenstrom (auf Englisch „stream“) bezeichnet.



Ein **Stream** ist eine Folge von Bytes. Die Bedeutung dieser Bytes (Buchstaben, Zahlen, Bitmuster) spielt dabei für die Verarbeitung durch das Dateisystem keine Rolle.



Da das Dateisystem als Struktureinheit nur Bytes kennt, bietet es keine Unterstützung für das Lesen und das Schreiben von strukturierten Informationen. Hierfür werden jedoch in C Bibliotheksfunktionen bereitgestellt.

Die Dateien des Dateisystems, die angesprochen werden, sind in UNIX also Folgen von Bytes oder Byte-Arrays.

Das Interpretieren eines Stroms von Bytes beim Lesen oder Schreiben hat nicht durch das Dateisystem, sondern durch den Anwender zu erfolgen.



Der Anwender nimmt die Interpretation des Bytestroms meist nicht selbst vor, sondern bedient sich hierzu der Bibliotheksfunktionen, die es beispielsweise erlauben, von einer Textdatei zeilenweise zu lesen oder in eine tabellarische Datei Datensätze fester Länge zu schreiben. Unter einer tabellarischen Datei wird hierbei eine Datei verstanden, bei welcher jeder Datensatz dieselbe Struktur aufweist, beispielsweise die Auflistung von Ein- und Ausgaben bei einer Buchhaltungs-Software.

Programme, die über Dateien kommunizieren, müssen sich über das Format, wie die entsprechende Datei beschrieben und gelesen wird, verständigen.



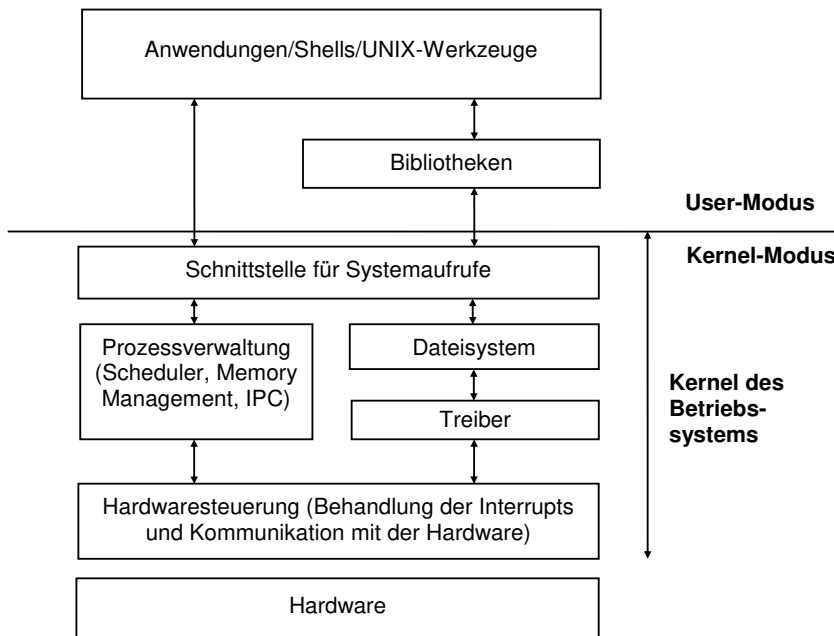
Die Umsetzung auf das byteweise arbeitende Dateisystem nehmen die Bibliotheksfunktionen vor. Die Umsetzung vom Dateisystem auf die eigentliche Geräthardware übernimmt dann ein Gerätetreiber (auf Englisch device driver).

16.1.2 Schichtenmodell von UNIX für die Ein- und Ausgabe

Ein von einem Anwender geschriebenes Programm greift nicht direkt auf die Ein-/Ausgabegeräte zu, sondern auf C-Bibliotheksfunktionen, die für die Ein-/Ausgabe vorgesehen sind. Diese wiederum rufen Systemfunktionen des Betriebssystems auf.



Das Besondere an UNIX ist, dass sämtliche Ein- und Ausgaben über ein Dateisystem im Kernel des Betriebssystems gehen – nicht nur die Ein-/Ausgaben von und zu der Festplatte, sondern generell für alle Peripheriegeräte wie Bildschirm, Tastatur und so fort. Im Folgenden ein Bild der Architektur von UNIX (gemäß [Bac91]):



Alle Ein-/Ausgaben eines Programms beziehen sich in UNIX auf Dateien im Dateisystem.



Die Geräteabhängigkeit ist bei UNIX durch das Dateisystem verborgen.



Die Geräte selbst werden durch Treiberprogramme (auf Englisch device driver) angesteuert. Der Kernel ist derjenige Teil des Betriebssystems, der in einem besonders geschützten Modus (Kernel-Modus) läuft, damit beispielsweise Programmierfehler eines Anwenders keinen Schaden anrichten können. Der Kommandointerpreter – unter UNIX Shell genannt – und UNIX-Werkzeuge sind auch Teile des Betriebssystems. Sie werden jedoch nicht im Kernel-Modus, sondern im User-Modus ausgeführt und haben damit denselben Schutz wie normale Programme.

In dieser Architektur lassen sich sowohl blockorientierte Geräte wie Festplatten als auch zeichenorientierte Schnittstellen wie Tastaturen oder Netzwerkschnittstellen in einfacher Weise einbinden. Jedes Gerät wird durch einen Namen, der wie ein Dateiname aussieht, angesprochen und die Ein- und Ausgabe erfolgt durch das Lesen von Dateien beziehungsweise Schreiben in Dateien, als ob Geräte gewöhnliche Dateien auf einer Festplatte wären.



Es ist sogar möglich und üblich, dass ein Gerät wie eine Festplatte oder eine SD-Karte zwei Gerätetreiber hat, eine Block- und eine Zeichenschnittstelle. Aber so tief – bis zu den Gerätetreibern – muss man nicht sehen. Man ruft einfach die C-Bibliotheksfunktionen auf, die auf das Dateisystem zugreifen. Was darunter liegt, ist verborgen.

16.2 Das Ein-/Ausgabe-Konzept von C

C bietet selbst keine Sprachmittel für die Ein- und Ausgabe. Für die Ein- und Ausgabe werden stattdessen Bibliotheksfunktionen verwendet, die jedoch wie die Sprache selbst standardisiert sind. Ihre Schnittstellen sind in der Bibliothek `<stdio.h>` aufgeführt.



Auf eine Datei wird in einem C-Programm über eine **Dateivariablen** zugegriffen. Diese ist entweder ein sogenannter **File-Pointer** (siehe Kapitel [→ 16.5](#)) oder eine systemspezifische Datei-Identifikation (siehe Anhang [→ B](#)).

Eine Datei liegt dabei normalerweise im Dateisystem und hat einen Namen, beispielsweise den Namen `test.dat`. Zur Laufzeit wird dann die Verknüpfung zwischen der Dateivariablen und der Datei auf der Festplatte hergestellt.

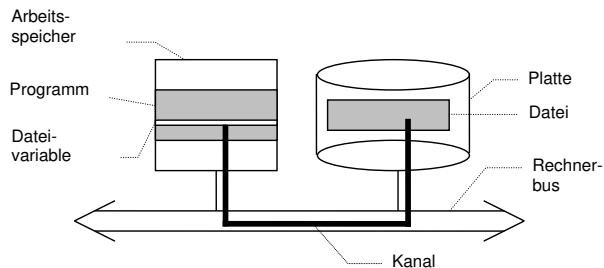
Beim Öffnen einer Datei muss dem Programm mitgeteilt werden, welche Dateivariablen des Programms mit welcher Datei des Dateisystems verknüpft werden soll.



Dies erfolgt beispielsweise durch folgende Anweisung:

```
filePointer = fopen("test.dat", "w");
```

Hier wird mit der Standardfunktion `fopen()` eine Datei `test.dat` zum Schreiben (w für write) geöffnet und mit der Dateivariablen `filePointer` verknüpft. `filePointer` stellt bildlich gesprochen den Kanal vom Programm zur Datei auf der Festplatte dar:



Über diesen Kanal `filePointer` kann in dem oben skizzierten Fall ("w") nur geschrieben werden. Prinzipiell kann jedoch über einen Kanal gelesen oder geschrieben werden.

16.2.1 Physische und logische Ebene

Beim Programmieren sieht man nur die logische Ebene einer Datei, in anderen Worten die Struktur ihrer Informationen und nicht die Blöcke auf der Festplatte. Die Informationen einer Datei, die man beim Programmieren als eine logische Einheit sieht, können physisch auf zahlreiche Plattenblöcke aufgeteilt sein. Diese Plattenblöcke einer Datei müssen nicht unbedingt sequenziell hintereinander auf der Festplatte liegen, sondern können über die Festplatte verstreut sein. Um die physischen Blöcke einer Festplatte braucht man sich nicht zu kümmern.

Die logische Struktur einer Datei wird auf die physischen Blöcke einer Festplatte durch das Dateisystem umgesetzt.



Die einzige Stelle, an der man im Programm noch das Dateisystem sieht, ist die Verknüpfung einer Dateivariablen des Programms, die den (logischen) Kanal zum Peripheriegerät repräsentiert, mit dem Namen einer Datei, der dem Dateisystem bekannt ist. Der Kanal zur Datei wird vom Betriebssystem zur Verfügung gestellt.

16.3 Standardeingabe und Standardausgabe in C

Vom Laufzeitsystem des C-Compilers werden drei Standardkanäle für die Standardeingabe, die Standardausgabe und die Standardfehlerausgabe in Form der globalen File-Pointer **stdin**, **stdout** und **stderr** zur Verfügung gestellt.



Das Laufzeitsystem initialisiert diese File-Pointer vor dem Programmstart. Normalerweise stellen diese Standardkanäle Kanäle zum Bildschirm beziehungsweise zur Tastatur dar. Sie können aber auch beim Programmstart mit Hilfe von Kommandos an das Betriebssystem auf Dateien oder andere Geräte umgelenkt werden. Diese File-Pointer stehen jedem C-Programm zur Verfügung und stellen ein wichtiges Hilfsmittel für die Ein- und Ausgabe dar.

Der Erfolg von UNIX hängt unter anderem auch mit seiner großen Flexibilität im Umgang mit Dateien zusammen. Hierbei haben sich zwei Konzepte als sehr nützlich erwiesen – weshalb sie auch von Windows übernommen wurden – nämlich die Umlenkung (Umleitung) der Ein- und Ausgabe und das Pipelining. Diese beiden Konzepte werden in den folgenden beiden Unterkapiteln genauer erläutert.

16.3.1 Umlenkung der Ein- und Ausgabe

Das **Umlenken** der Standardeingabe und Standardausgabe erfolgt auf der Kommandoebene des Betriebssystems unter Windows gleich wie unter UNIX, nämlich durch die Umlenk-Operatoren **>** und **<**.



Die Umlenkung der **Standardausgabe** erfolgt wie im folgenden Beispiel:

```
myprog > test.out
```

Die vom Programm `myprog` an die Standardausgabe geleiteten Zeichen werden in die Datei `test.out` geschrieben. Zum Beispiel werden alle Ausgaben von `printf()` jetzt in die Datei `test.out` geschrieben.

Von einer Umlenkung merkt ein Programm nichts.



Unter UNIX wie auch unter Windows wird durch `myprog > test.out` automatisch die gewünschte Datei erzeugt. Danach steht die Datei `test.out` im aktuellen Verzeichnis.

Die Umlenkung der **Standardeingabe** erfolgt wie im folgenden Beispiel:

```
myprog < test.inp
```

Damit kommen die Eingaben für das Programm `myprog` nicht mehr von der Tastatur, sondern aus der Datei `test.inp`. Auch hier merkt das Programm nichts davon. Es behandelt jedes Byte so, als wäre es von der Tastatur gekommen.

Beide Umlenkungsarten können auch kombiniert werden, das heißt, man kann Standardeingabe und Standardausgabe gleichzeitig umlenken:

```
myprog < test.inp > test.out
```

Die Eingabe erfolgt dann nicht mehr von der Tastatur und die Ausgabe geht nicht mehr an den Bildschirm.

Die Angaben zur Umlenkung der Standardeingabe und der Standardausgabe werden dabei nicht an das Programm weitergeleitet, sondern die Umlenkung wird vom Kommandointerpreter des Betriebssystems durchgeführt.

Damit Fehlermeldungen nach wie vor trotz erfolgter Umlenkung der Standardausgabe am Bildschirm erscheinen können, gibt es einen dritten, unabhängigen Standardkanal, die **Standardfehlerausgabe**. Auch diese kann mit Mitteln des Betriebssystems umgelenkt werden. Beispielsweise durch folgende Zeile:

```
myprog 2> errors.log
```

Die 2 entspricht dabei der internen Nummer des Kanals `stderr`. Der Kanal `stdin` hat die Nummer 0 und `stdout` hat die Nummer 1. Die Nummern 0, 1 und 2 haben bei allen Programmen genau diese Bedeutung.

16.3.2 Pipelining

Durch **Pipes** (Röhren) werden mehrere Programme in der Weise verbunden, dass die Standardausgabe des einen Programms als Standardeingabe des anderen Programms verwendet wird.



Das Pipelining erfolgt auf der Kommandoebene des Betriebssystems unter Windows gleich wie unter UNIX in der Form:

```
Programm1 | Programm2
```

Der senkrechte Strich bedeutet, dass die Ausgabe von Programm1 umgeleitet und zur Eingabe von Programm2 wird. Diese Syntax soll durch ein bekanntes Beispiel aus der UNIX-Welt illustriert werden. Die mittels einer Pipe verbundenen Kommandos

```
ls | sort
```

geben eine sortierte Liste der Dateien des aktuellen Verzeichnisses aus. Hierbei wird die Standardausgabe des Programms `ls` als Standardeingabe an das Programm `sort` gegeben.

Die Standardausgabe eines Programms wird durch den Pipe-Operator `|` zur Standardeingabe für das nächste Programm.



Der dabei verwendete Puffer, in den das eine Programm hineinschreibt und aus dem das andere Programm herausliest, wird als Pipe bezeichnet.



Der Mechanismus wird als Fließbandverarbeitung oder Pipelining bezeichnet. Der Vorteil ist zum einen, dass hierbei kein Zugriff zur Festplatte erforderlich ist, sondern nur zum Arbeitsspeicher. Im Arbeitsspeicher wird quasi eine temporäre Datei angelegt, die wieder automatisch gelöscht wird. Durch das Hintereinanderreihen von Programmen mit Hilfe von Pipes kann man mächtige Funktionen erzeugen. Die einzelnen Programme können dabei ziemlich klein sein.

Ein Programm, welches nur Daten von der Standardeingabe liest, diese verarbeitet und sein Resultat nur auf die Standardausgabe schreibt, wird als Filter bezeichnet. Verschiedene Filter können durch Pipes verbunden werden.



```
prog1 | filter1 | filter2 | prog2
```

16.3.3 Funktionen für die Standardeingabe und Standardausgabe

Wie man sieht, eröffnet die Programmierung mit der Standardeingabe und Standardausgabe sehr viele Möglichkeiten: Diese Programme können sowohl mit Bildschirm und Tastatur arbeiten, bei Bedarf – beispielsweise wenn sie im Hintergrund arbeiten sollen – ihre Daten aber auch aus Dateien beziehen oder ihre Ergebnisse in Dateien ablegen. Sie sind nicht von speziellen Dateinamen abhängig, sondern bei jedem Programmstart kann eine andere Datei in einer Umlenkung berücksichtigt werden. Einfache Programme können über Pipes zu komplexeren Operationen verknüpft werden, ohne dass in eines der Programme eingegriffen werden muss.

Wegen der herausragenden Bedeutung der Standardeingabe und Standardausgabe stellt die C-Bibliothek eigene Funktionen für die Standardeingabe und die Standardausgabe zur Verfügung.



Häufig sind diese nur als Makros implementiert, die vom Präprozessor auf Aufrufe von Funktionen des Betriebssystems umgesetzt werden. Sie stellen also nur Abkürzungen dar, da sie häufig benutzt werden. Ihre Existenz bedeutet nicht, dass Standardeingabe, Standardausgabe und „normale“ Dateien unterschiedlich behandelt werden.

16.4 High- und Low-Level-Bibliotheksfunktionen zur Ein- und Ausgabe

Die C-Bibliothek implementiert für die Ein- und Ausgabe das Stream-Konzept, welches von UNIX her bekannt ist. Eine Datei wird als ein Array von Zeichen betrachtet.



Arbeitet das Betriebssystem anders, so wird dies durch die Bibliotheksfunktionen von C verborgen.

Die Bibliotheksfunktionen zur Ein- und Ausgabe in C lassen sich unterteilen in High-Level-Funktionen und Low-Level-Funktionen.

16.4.1 High-Level-Funktionen

Die **High-Level-Funktionen** bieten eine einfache und portable Schnittstelle für den Umgang mit Dateien. Wie oben bereits erwähnt, gibt es spezielle Funktionen, die auf die Standardein- beziehungsweise -ausgabe zugreifen. Eine der bekanntesten Funktionen ist `printf()`.

`printf()` ist eine Abkürzung für „print formatted“ und dient zur formatierten Ausgabe auf die Standardausgabe.



Mit `printf()` können beispielsweise `int`-Zahlen formatiert – also in einer lesbaren Form – als Folge von Dezimalzeichen ausgegeben werden.

In der Standardbibliothek ist jedoch die Funktionalität von `printf()` auch für das Schreiben in Dateien verfügbar. Man fügt dem Namen der Funktion einfach ein `f` am Anfang hinzu.

Das erste `f` von `fprintf()` kommt von „file“. `fprintf()` dient also allgemein zur formatierten Ausgabe in irgend eine Datei, beziehungsweise irgend einen Stream.



Hierauf wird in den folgenden Kapiteln noch genauer eingegangen.

16.4.2 Low-Level-Funktionen

Low-Level-Funktionen werden auch elementare Ein-/Ausgabefunktionen genannt.

Die Low-Level-Funktionen für den Dateizugriff sind Bibliotheksfunktionen, die Aufrufe von Betriebssystemfunktionen bewirken.



Die High-Level-Dateizugriffsfunktionen der Standardbibliothek sind – zumindest unter UNIX – mit Hilfe von Aufrufen der Low-Level-Funktionen für den Dateizugriff implementiert.



Die Low-Level-Funktionen werden in Anhang → B beschrieben.

Low-Level-Funktionen bieten auch Möglichkeiten für die Ein- und Ausgabe, die es bei den High-Level-Funktionen nicht gibt.

Die Low-Level-Funktionen haben jedoch den Nachteil, dass sie nicht unabhängig vom Betriebssystem standardisiert sind und deshalb vom Betriebssystem abhängig sind.



Für UNIX-Betriebssysteme sind sie jedoch durch den POSIX-Standard standardisiert. Die Low-Level-Funktionen haben eine große Bedeutung für die Systemprogrammierung unter UNIX, beispielsweise für die Programmierung der Interprozesskommunikation zwischen Betriebssystemprozessen. Die High-Level-Dateizugriffsfunktionen hingegen verstecken das Betriebssystem. Damit sind Programme, welche nur die High-Level-Dateizugriffsfunktionen verwenden, portabel, sprich sie können auf andere Rechner beziehungsweise Betriebssysteme übertragen werden und laufen auch dort.

16.4.3 Weiterführende Dokumentation der Funktionen

Sowohl Low-Level- wie auch High-Level-Funktionen bieten eine unglaubliche Vielzahl an Optionen, welche den Rahmen dieses Buches sprengen würden. In den folgenden Kapiteln werden nur die wichtigsten Vertreter der High-Level-Funktionen mit den geläufigsten Parametern beschrieben.

Für eine umfassende Dokumentation dieser Funktionen wird auf die in jeder heutigen IDE eingebaute Hilfe verwiesen. Unter Unix oder Systemen mit Unix-ähnlichen Konsolen können alle benötigten Informationen beispielsweise auch mittels folgender Konsolenanweisung abgefragt werden:

```
man 3 printf
```

Diese sogenannten man pages (Manual-Seiten) sind in sämtlichen Unix-ähnlichen Systemen eingebaut. Auch im Web können mittels einfacher Suche nach „man printf“ oder einfach „printf“ eben diese man pages abgerufen werden.

16.5 Der File-Pointer bei High-Level-Funktionen

Dieses Unterkapitel behandelt den allgemeinen Umgang mit File-Pointern. Aufgrund der Vielzahl an Möglichkeiten, welche die Standardbibliothek `stdio.h` bietet, können hier nur die wichtigsten Funktionen besprochen werden. Allesamt werden sie am Ende in Kapitel [→ 16.5.6](#) in einem Beispiel angewendet.

Der Zugriff auf eine Datei über die High-Level Dateizugriffsfunktionen wird über einen File-Pointer durchgeführt. Der File-Pointer wird beim Öffnen einer Datei von der Funktion `fopen()` zurückgegeben und zeigt auf eine Struktur vom Typ **FILE**. Je nach Betriebssystem und Compiler sind die Inhalte der **FILE**-Struktur unterschiedlich.



Variablen vom Typ **FILE*** bezeichnet man im Allgemeinen als **Stream**. Andere Bezeichnungen sind **File-Pointer** oder **Dateivariablen**. Die Struktur **FILE** ist in der Bibliothek `<stdio.h>` definiert.



Die genaue Struktur von FILE muss nicht bekannt sein, da der Inhalt der Struktur nur für die Implementierung der High-Level-Dateifunktionen relevant ist.

Die Struktur FILE verfügt über einen **Dateipuffer**. Dies bedeutet, dass die Funktionen des High-Level-Dateizugriffs zum Lesen und Schreiben grundsätzlich nur auf einen Dateipuffer zugreifen, der sich im Hauptspeicher befindet. Dies beschleunigt Lese- und Schreibprozesse erheblich, da Zugriffe auf den Hauptspeicher um ein Vielfaches schneller sind als auf eine Datei. Das tatsächliche Schreiben und Lesen des Puffers wird nur dann durchgeführt, wenn dies erforderlich ist. Bei Ausgabedateien kann man mittels der `fflush()`-Funktion den Puffer manuell leeren (siehe Kapitel [→ 16.5.3](#)).

Die Struktur FILE speichert zudem Flags, die den Status einer Datei darstellen. Unter anderem wird der Dateiende-Status (End-Of-File, EOF) und der Fehlerstatus des Streams vermerkt. Diese beiden Status können mit den Funktionen `feof()` und `ferror()` abgefragt werden (siehe Kapitel [→ 16.5.5](#)).

16.5.1 Standard-File-Pointer

Es gibt bereits vordefinierte Standard-File-Pointer, die das Ansprechen der Ein- und Ausgabegeräte gestatten. Die folgenden File-Pointer sind in `<stdio.h>` als symbolische Konstanten definiert:

File-Pointer	Ansprechbare Ein-/Ausgabegeräte
<code>stdin</code>	Standardeingabe (Tastatur)
<code>stdout</code>	Standardausgabe (Bildschirm)
<code>stderr</code>	Standardfehlerausgabe (Bildschirm)

Diese File-Pointer entsprechen konstanten Pointern auf den Typ `FILE*`.

Unter dem Betriebssystem UNIX gibt es die drei reservierten Namen für die Standardkanäle **`stdin`**, **`stdout`** und **`stderr`**. Beim Start eines Programms werden die File-Pointer automatisch vom Laufzeitsystem mit den entsprechenden Standardkanälen und damit mit den in der obigen Tabelle genannten Ein-/Ausgabegeräten verknüpft. Die File-Pointer sind damit initialisiert – ein Aufruf von `fopen()` ist nicht nötig und auch nicht möglich.

Auf der Ebene des Betriebssystems kann die Zuordnung der Standardkanäle zu Geräten beziehungsweise Dateien aber auch geändert werden – etwa durch Umlenkung oder Pipelining (siehe dazu Kapitel [→ 16.3](#)). Die Standardfehlerausgabe ist eine zusätzliche Einheit zu stdout. Schreibt man Fehlermeldungen nicht nach stdout, sondern nach stderr, so kann verhindert werden, dass die Fehlerausgaben beim Umlenken der Standardausgabe nicht mehr am Bildschirm sichtbar sind.

Man sollte es sich zur Angewohnheit machen, Fehler und Warnungen nie mit `printf(...)` oder mit `fprintf(stdout, ...)` nach stdout auszugeben. Man sollte sie immer nach stderr mit `fprintf(stderr, ...)` ausgeben.



Damit erreicht man nämlich, dass die Fehler- und Warnungsausgaben am Bildschirm sichtbar bleiben und man sehr leicht feststellen kann, ob und wie oft es Fehler und Warnungen gab.

16.5.2 Öffnen und Schließen von Dateien

Wenn mit Dateien gearbeitet wird, müssen sie normalerweise zuerst geöffnet werden. Dies geschieht beispielsweise, indem angegeben wird, unter welchem Dateinamen eine Datei im Betriebssystem aufzufinden ist. Wenn die Datei erfolgreich geöffnet wurde, kann man andere Funktionen aufrufen, um die Datei zu manipulieren oder um Daten aus ihr herauszulesen. Am Ende der Verarbeitung wird die Datei wieder geschlossen.

Die Funktion `fopen()` öffnet eine Datei mit bestimmten Zugriffsrechten wie beispielsweise zum Lesen oder Schreiben. Die Funktion `fclose()` schließt alle Schreibvorgänge ab und schließt die Datei.



Die beiden Funktionen `fopen()` und `fclose()` haben die folgenden Prototypen:

```
FILE* fopen (const char* filename, const char* mode);  
int    fclose(FILE* stream);
```


In C11 sind die beiden Parameter von `fopen()` zusätzlich mit dem `restrict`-Schlüsselwort deklariert.

Die Funktion `fopen()` öffnet die durch `filename` definierte Datei mit dem im Parameter `mode` angegebenen Schalter für das Zugriffsrecht. Wenn die Datei erfolgreich geöffnet werden konnte, gibt die Funktion einen Pointer auf `FILE` zurück. Im Fehlerfall wird `NULL` zurückgegeben.

Wenn die Datei im Programm nicht mehr benötigt wird, kann einfach die Funktion `fclose()` mit dem entsprechenden File-Pointer aufgerufen werden. Dadurch werden sämtliche Schreibvorgänge abgeschlossen, wenn noch welche abzuarbeiten sind. Sprich, der Puffer wird in die Datei geschrieben und der Datei-Stream wird vom Programm getrennt.

Grundsätzlich sollte eine Datei immer sofort nach Abschluss der Dateibearbeitung geschlossen werden. Dies verhindert einen eventuellen Datenverlust bei einem späteren Programmabsturz. Außerdem ist die Anzahl der gleichzeitig geöffneten Dateien durch das Betriebssystem begrenzt.



Ein einfaches Beispiel:

```
FILE* fp = fopen ("antwort.txt", "w");
if (fp != NULL) {
    fprintf(fp, "Zweiundvierzig\n");
    fclose(fp);
}
```

Hier wird eine Datei zum Schreiben im Textmodus geöffnet. Der Parameter `mode` wird hier mittels `"w"` angegeben. Das `w` ist dabei ein sogenannter „Schalter“. Folgendes sind die wichtigsten Schalter:

Schalter	Bedeutung
r	Öffnen einer Datei ausschließlich zum Lesen.
w	Datei zum Schreiben erzeugen. Existiert die Datei bereits, so wird sie überschrieben.
a	Öffnen einer Datei zum Anfügen. Existiert die Datei bereits, so werden neue Daten an das Dateieinde angefügt, ansonsten wird die Datei neu erstellt.
b	Zusatz-Schalter für das Öffnen einer Datei im Binärmodus.

Die Schalter können teilweise auch kombiniert werden. So ist es mit dem Modus "rw" beispielsweise möglich, eine Datei sowohl zu lesen als auch zu schreiben. Auch kann man sich ab dem C11-Standard gar vom Betriebssystem Exklusiv-Rechte erbitten. Hier wird auf die weiterführende Dokumentation verwiesen.

Zum Öffnen einer Datei für eine binäre Ein-/Ausgabe muss zusätzlich der Buchstabe b als Schalter angegeben werden, also "rb" oder "wb". Wenn dieser Schalter nicht angegeben wird, so wird eine Datei im Textmodus geöffnet.

Der Unterschied zwischen einer Datei im **Text-** und im **Binärmodus** ist, dass im Textmodus – im Falle von Windows-Compilern – beim Schreiben eines Zeilenendes ein LF durch ein CR LF (Carriage Return, Line Feed) ersetzt wird, zum anderen wird beim Lesen ein CR LF durch ein LF ersetzt. Wenn die Datei im Binärmodus geöffnet wurde, wird dieser Ersetzungsvorgang nicht durchgeführt.



Bei UNIX-Compilern wird der Schalter b zwar akzeptiert, es gibt jedoch keinen Unterschied zwischen Text- und Binärmodus. Es wird auf den Ersetzungsvorgang am Zeilenende komplett verzichtet.

Zusätzlich zu `fopen()` und `fclose()` gibt es die Funktion `freopen()`, welche versucht, einen bereits geöffneten File-Pointer zu schließen und danach mittels eines neuen Modus erneut zu öffnen. Hier wird auf eine genauere Behandlung verzichtet.

16.5.3 Puffersteuerung

Wie bereits besprochen, hat ein File-Pointer einen **Dateipuffer**, welcher einzelne Lese- und Schreib-Operationen enorm beschleunigen kann. Wünscht man an einem Punkt im Programm explizit, dass der Puffer geleert werden soll, so kann dies mit einem Aufruf der Funktion `fflush()` erreicht werden. Mit den Funktionen `setbuf()` und `setvbuf()` kann zudem für jeden File-Pointer einzeln gesteuert werden, was genau gepuffert werden soll.

Die Prototypen für die Funktionen sind die folgenden:

```
int fflush (FILE* stream);  
int setvbuf(FILE* stream, char* buf, int mode, size_t size);  
void setbuf (FILE* stream, char* buf);
```

In C11 sind die ersten beiden Parameter von `setvbuf()` und `setbuf()` zusätzlich mit dem `restrict`-Schlüsselwort deklariert.

Die Funktion `fflush()` sorgt bei einem Ausgabestrom dafür, dass alle in den Dateipuffern existierenden und noch nicht geschriebenen Daten auf die Festplatte geschrieben werden. Die Wirkungsweise von `fflush()` auf einen Eingabestrom ist laut Standard undefiniert.



Mittels der Funktion `setvbuf()` kann gesteuert werden, wie groß der Puffer eines File-Pointers sein soll, wie er genutzt werden soll, und es kann sogar ein eigener Puffer angegeben werden. Die Funktion `setbuf()` ist nur eine vereinfachte Version von `setvbuf()` und ruft selbige Funktion mit vorgegebenen Standardwerten auf. Hier wird auf die weiterführende Dokumentation verwiesen.

Ein Aufruf der Funktion `fclose()` führt automatisch auch einen Aufruf von `fflush()` aus. So werden alle noch nicht geschriebenen Daten in die Datei gespeichert.

16.5.4 Navigieren innerhalb von Dateien

Ein File-Pointer speichert intern, wo genau sich das Programm momentan in der Datei befindet. Wird eine Datei beispielsweise im Lesemodus "r" geöffnet, so zeigt der interne **Dateipositionszeiger** zu Beginn auf den Anfang der Datei. Beim Lesen einer Datei rückt der Dateipositionszeiger allmählich vor, bis schlussendlich das Ende der Datei erreicht wird. Wenn gewünscht, kann jedoch der Dateipositionszeiger auch beliebig verschoben werden.

Mittels der Funktion `ftell()` kann ermittelt werden, an welcher Position sich der Dateipositionszeiger gerade befindet. Mittels `fseek()` kann der Zeiger auf eine beliebige Position innerhalb der Datei gesetzt werden. Die Funktion `rewind()` setzt den Zeiger zurück zum Anfang der Datei.



Folgendes sind die Prototypen der Funktionen:

```
long ftell (FILE* stream);  
int  fseek (FILE* stream, long offset, int whence);  
void rewind(FILE* stream);
```

Diese Funktionen erlauben es, den Dateipositionszeiger innerhalb von binären Dateien beliebig zu setzen. Für Textdateien jedoch funktionieren diese Funktionen nur unzuverlässig, da die automatische Umwandlung von CR in CR LF unter Windows ein einwandfreies Arbeiten bei Dateien im Textmodus verhindert. Für Textdateien, welche Zeichen des Typs `wchar_t` speichern, gibt es seit C11 zusätzlich die Funktionen `fgetpos()` und `fsetpos()`, welche eine exakte Positionierung erlauben. Hier in diesem Buch werden sie nicht weiter angesprochen.

Die Funktion `fseek()` setzt den Dateipositions-Zeiger der durch `stream` definierten Datei auf die Position, die `offset` Bytes von `whence` entfernt ist. Für den Parameter `whence` sind in `<stdio.h>` 3 Konstanten definiert:

<code>SEEK_SET</code>	offset ist relativ zum Dateianfang
<code>SEEK_CUR</code>	offset ist relativ zur aktuellen Position
<code>SEEK_END</code>	offset ist relativ zum Dateende

Die Funktion `rewind()` positioniert den Dateipositions-Zeiger der durch `stream` definierten Datei an den Dateianfang und setzt zusätzlich sämtliche Fehler-Flags zurück.

16.5.5 Fehlerbehandlung

Ein File-Pointer speichert nebst dem obengenannten Dateipositionszeiger auch **Status-** und **Fehler-Flags**. Beispielsweise ist es beim Lesen einer Datei wichtig zu wissen, ob der Dateipositionszeiger am Ende der Datei angelangt ist.

In einigen Funktionen zur Ein-/Ausgabe in Dateien kann nicht zwischen einem eigentlichen Fehler bei der Dateiverarbeitung und dem Erreichen des Dateiendes unterschieden werden. Um den Stream in einem solchen Fall genauer zu analysieren, werden folgende Funktionen bereitgestellt:

```
int feof (FILE* stream);  
int ferror (FILE* stream);  
void clearerr(FILE* stream);
```

Die Funktion `feof()` prüft, ob das Dateiende erreicht ist. Die Funktion `ferror()` prüft das Fehlerflag einer Datei. Die Funktion `clearerr()` setzt das Dateiende- und das Fehlerflag einer Datei zurück.



Die Funktionen `feof()` und `ferror()` geben eine Zahl ungleich null zurück, wenn das entsprechende Flag gesetzt ist. Was genau die zurückgegebene Zahl aussagt, ist jedoch implementationsspezifisch.

Bei jeder Funktion der Standardbibliotheken (nicht nur bei Verwendung von `<stdio.h>`), bei welcher ein Fehler passiert, wird die globale Fehler-Variable `errno` mit einem fest vorgegebenen Wert gesetzt. Alle möglichen Fehlernummern können in der Standard-Headerdatei `<errno.h>` nachgelesen werden. Zudem kann man mittels der Funktion `perror()` sich eine lesbare Fehlermeldung ausgeben lassen.

```
int perror(const char* s);
```

Der übergebene String wird mit einem Doppelpunkt ausgegeben, gefolgt von der entsprechenden Fehlermeldung.

16.5.6 Beispiel für den Umgang mit File-Pointern

Hier wird in einem etwas ausführlicheren Beispiel gezeigt, wie mit den in diesem Kapitel vorgestellten Funktionen umgegangen wird. Es wird eine Datei eingelesen mit einzelnen Zahlen in mehreren Zeilen. Aus dieser Datei wird eine Ausgabedatei generiert, in welcher die Zahlen als auf 1 normierte, kumulative Verteilfunktion aufgelistet sind:

verteilung.c

```
#include "stdlib.h"
#include "stdio.h"

int main(void) {
    char number[10];
    int sum = 0;
    int cumulation = 0;

    // Oeffnen der Input-Datei
    FILE* infile = fopen("input.txt", "r");
    if (infile == NULL) {
        perror("Inputdatei"); exit(1);
    }

    // Oeffnen der Output-Datei
    FILE* outfile = fopen("result.txt", "w");
    if (outfile == NULL) {
        perror("Outputdatei"); exit(1);
    }

    // Lesen der Zahlen:
    while (!feof(infile)) {
        fgets(number, 10, infile);    // Einlesen einer Zeile
        sum += atoi(number);          // Umwandeln in int
    }

    // Zuruecksetzen der Input-Datei. Auch moeglich: rewind(infile);
    fseek(infile, 0, SEEK_SET);

    // Berechnen der kumulativen Verteilfunktion
    while (!feof(infile)) {
        fgets(number, 10, infile);
        fprintf(outfile, "%f\n", (double)cumulation / sum);
        cumulation += atoi(number);
    }
    fprintf(outfile, "%f\n", 1.);
}
```

```
fflush(outfile); // Nicht noetig. Einfach zur Demo.
printf("Datei result.txt erfolgreich beschrieben.\n");

// Schliessen der Dateien
fclose(outfile);
fclose(infile);

return 0;
}
```

Die Datei `input.txt` auf der linken Seite ergibt die Datei `result.txt` auf der rechten Seite

input.txt	result.txt
54	0.000000
63	0.210117
88	0.455253
52	0.797665
	1.000000

Die Ausgabe dieses Beispiels könnte nun beispielsweise als Eingabe für ein Programm dienen, welches aus diesen Zahlen eine grafische Darstellung dieser kumulativen Verteilfunktion erstellt.

16.5.7 Operationen auf Dateien

Der C-Standard sieht ein paar wenige Funktionen vor, mit denen man mit dem Betriebssystem kommunizieren kann, um neue Dateien anzulegen, Dateien umzubenennen oder gar Dateien zu löschen.

Dies sind die Prototypen der Funktionen:

```
char* tmpnam (char* s);
FILE* tmpfile(void);
int rename (const char* old, const char* new);
int remove (const char* filename);
```

Die Funktion `tmpnam()` erstellt einen eindeutigen Dateinamen, welcher keinem anderen Namen einer existierenden Datei entspricht. Mit der Funktion `tmpfile()` erstellt und öffnet das Betriebssystem eine neue, binäre Datei mit Schreibzugriff, welche einen eindeutigen Namen besitzt, aber nur temporär existiert. Die Datei wird am Ende des Programmes oder beim Schließen der Datei automatisch wieder gelöscht.



Mit der Funktion `rename()` kann eine Datei umbenannt werden. Die Funktion `remove()` löscht eine Datei.



16.6 Einfaches Schreiben in Dateien mit High-Level-Funktionen

Wenn eine Datei erfolgreich im Schreib-Modus geöffnet wurde, kann sie mit Hilfe verschiedener Funktionsaufrufe mit Daten gefüllt werden. Das Schreiben von Daten geschieht grundsätzlich zeichenweise, das heißt, jedes Byte wird einzeln vom Hauptspeicher in die Datei geschrieben.

Die Funktion `fputc()` ist die grundlegendste Funktion, um Daten in eine Datei zu schreiben. Unter dem Namen `putc()` ohne `f` existiert zudem ein Makro, welches exakt dasselbe tut wie `fputc()`, jedoch als Makro ausprogrammiert ist. Das Makro sollte somit nicht verwendet werden, wenn dessen Parameter Ausdrücke mit Nebeneffekten darstellen. Die Prototypen von `fputc()` beziehungsweise `putc()` sind:

```
int fputc(int c, FILE* stream);  
int putc (int c, FILE* stream);
```

Die Funktion `fputc()` schreibt das (von `int` in `unsigned char` umgewandelte) Zeichen `c` in die durch `stream` definierte Datei. Die Funktion `fputc()` liefert bei fehlerfreier Abarbeitung das geschriebene Zeichen `c` zurück. Im Fehlerfall wird das Fehlerflag für den Stream gesetzt und `EOF` zurückgegeben.

Die Funktion `fputc()` schreibt ein einzelnes Zeichen (ein Byte) in einen gegebenen File-Pointer. Wenn die Datei als Binärdatei geöffnet wurde, so wird das Byte ohne weitere Konvertierung direkt in die Datei geschrieben. Wenn die Datei als Textdatei geöffnet wurde, so werden gegebenenfalls Zeilenende-Zeichen je nach Betriebssystem umkonvertiert.



Alle anderen Schreib-Funktionen bauen auf `fputc()` auf. Man muss sich also beim Öffnen der Datei genau überlegen, ob die Datei als Binärdatei oder als Textdatei behandelt werden soll.

Im Folgenden werden die erweiterten Schreib-Funktionen der Standardbibliothek `<stdio.h>` vorgestellt. Hier wird grob unterschieden zwischen Objekten und formatierten Strings.

16.6.1 Schreiben von Objekten und Strings

Unter Objekten werden hier beliebig große Strukturen verstanden, welche als ein Array von Bytes angesprochen werden. Die folgenden Funktionen erlauben es, ganze Arrays von solchen Objekten und Strings mittels eines einzigen Funktionsaufrufes in eine Datei zu speichern:

```
size_t fwrite(  
    const void* ptr,  
    size_t size,  
    size_t nmemb,  
    FILE* stream);  
  
int fputs(  
    const char* s,  
    FILE* stream);
```

Die Funktion `fwrite()` schreibt ganze Arrays von Objekten in eine Datei. Die Funktion `fputs()` schreibt einen String in eine Datei.



Die Funktion `fwrite()` schreibt `nmemb` Objekte der Größe `size` aus dem Array, auf das der Pointer `ptr` zeigt, in die durch `stream` definierte Datei. Insgesamt werden maximal `nmemb * size` Bytes geschrieben.

Die Funktion `fputs()` schreibt den String, auf den der Pointer `s` zeigt, in die durch `stream` definierte Datei. Beim Schreiben wird das Stringende-Zeichen `'\0'` nicht in die Datei geschrieben.

Der Datentyp `size_t` ist in `<stddef.h>` definiert und wurde bereits im Zusammenhang mit dem `sizeof`-Operator in Kapitel [9.9.1](#) erläutert.

16.6.2 Schreiben von ganzen Strings auf die Standardausgabe

Für die Ausgabe von ganzen Strings ohne Formatierung auf dem Standard-Ausgabegerät gibt es folgende Funktions-Prototypen:

```
int puts (const char* s);  
int putchar(const char* s);
```

Hierbei entspricht `puts()` der Funktion `fputs()` und `putchar()` entspricht dem Makro `putc()`. Bei beiden Funktionen wird als `stream` die Standardausgabe `stdout` verwendet.

Bei der Funktion `puts()` wird die Ausgabe mit einem automatisch angehängten Zeilenende ergänzt. Das abschließende Nullzeichen `'\0'` wird jedoch nicht geschrieben.

16.7 Schreiben von formatierten Strings

Die Funktion **printf()** („print formatted“) dient zur formatierten Ausgabe von Werten von Ausdrücken auf der Standardausgabe.



Der Funktions-Prototyp von `printf()` lautet:

```
int printf(const char* format, ...);
```

In C11 ist der Parameter `format` zusätzlich mit dem `restrict`-Schlüsselwort deklariert. Der Prototyp steht in der Bibliothek `<stdio.h>`.

16.7.1 Varianten von `printf()`

Mit `printf()` werden Werte formatiert ausgegeben. Ein Ausdruck mit einem Wert kann all das sein, was einen Wert hat, beispielsweise eine Konstante, eine Variable, ein Funktionsaufruf, der einen Wert zurückgibt, oder die Verknüpfung eines Ausdrucks mit einem anderen Ausdruck durch Operatoren.

Bevor jedoch erklärt wird, wie genau Ausdrücke formatiert werden können, werden die verschiedenen Varianten von `printf()` vorgestellt.

Von der Funktion `printf()` gibt es verschiedene Implementierungen, welche allesamt dieselben Formatierungen vornehmen, jedoch unterschiedliche Ein- und Ausgänge besitzen. Die entsprechenden Funktionen werden durch unterschiedliche Präfixe vor dem `printf` ausgedrückt:

<code>printf</code>	Ausgabe nach <code>stdout</code>
<code>fprintf</code>	Ausgabe in Datei (<code>f</code> = File)
<code>sprintf</code>	Ausgabe in String
<code>snprintf</code>	Ausgabe in String mit Maximallänge
<code>vprintf</code>	Ausgabe nach <code>stdout</code> , Parameter aus variabler Parameterliste
<code>vfprintf</code>	Ausgabe in Datei (<code>f</code> = File), Parameter aus variabler Parameterliste
<code>vsprintf</code>	Ausgabe in String, Parameter aus variabler Parameterliste
<code>vsnprintf</code>	Ausgabe in String mit Maximallänge, Parameter aus variabler Parameterliste

Die Funktion `printf()` muss in der Lage sein, beliebig viele Argumente zu formatieren. Daher enthält der Prototyp von `printf()` auch die Auslassung `(...)`. Die Auslassung wird meist als Ellipse bezeichnet. Wenn die Funktion jedoch das `v`-Präfix besitzt, so werden die Eingabewerte anstelle der Auslassung mittels einer variablen Parameterliste mit dem Typ `va_list` angegeben. Diese Funktionen werden hier nicht weiter behandelt. Weitere Informationen zu variablen Parameterlisten und den Ellipsen können in Kapitel [→ 11.7](#) nachgelesen werden.

Das Präfix `f` sorgt dafür, dass die Ausgabe nicht auf die Standardausgabe `stdout` erfolgt, sondern in einen beliebigen File-Pointer. Der Prototyp der Funktion `fprintf()` lautet:

```
int fprintf(FILE* stream, const char* format, ...);
```

In C11 sind die beiden Parameter zusätzlich mit dem `restrict`-Schlüsselwort deklariert.

Die Präfixe `s` und `sn` sorgen dafür, dass die Ausgabe nicht auf die Standardausgabe `stdout` erfolgt, sondern in einen String. Folgendes sind die Prototypen:

```
int sprintf (char* s, const char* format, ...);  
int snprintf(char* s, size_t n, const char* format, ...);
```

In C11 sind die beiden Pointer-Parameter wiederum zusätzlich mit dem `restrict`-Schlüsselwort deklariert.

Bei beiden Funktionen wird die formatierte Ausgabe in den durch `s` gegebenen String gespeichert. Beide Funktionen hängen automatisch an das Ende des formatierten Strings ein Null-Byte an. Man muss selbst dafür sorgen, dass der in `s` gegebene Puffer groß genug ist für alle Bytes inklusive des abschließenden Nullzeichens.

Bei der `sn`-Variante kann man jedoch explizit angeben, wie groß der String maximal sein darf. Die Funktion wird sodann die Ausgabe rechtzeitig abbrechen, damit der Puffer nicht überlaufen kann.

16.7.2 Formatieren von Werten

Der Prototyp der Funktion `printf()` deklariert vor der Ellipse nur einen einzigen formalen Parameter, den Parameter `format`. Dieser Parameter ist vom Typ Pointer auf `char` und stellt den sogenannten **Format-String** dar. Mit diesem String wird gesteuert, wie die restlichen Parameter formatiert werden sollen.

Der Format-String enthält sogenannte „**Formatelemente**“, welche allesamt mit dem Prozent-Zeichen `%` eingeleitet werden. Für jedes Formatelement in dem String des ersten Argumentes muss ein weiteres Argument als Teil der Ellipse übergeben werden. An derjenigen Stelle des Strings, an der das Formatelement steht, erfolgt die Ausgabe des entsprechenden Argumentes.

```
printf("Einnahmen an Tag %d: %d Euro", tagNummer, einnahmen);
```

Ergibt beispielsweise:

```
Einnahmen an Tag 7: 3558 Euro
```

Die Reihenfolge der weiteren Argumente der Funktion `printf()` und deren Typ muss mit der Reihenfolge und Art der Formatelemente im Format-String übereinstimmen. Der Format-String wird von links nach rechts abgearbeitet. Gewöhnliche Zeichen in diesem String werden auf die Standardausgabe geschrieben. Die Formatelemente bestimmen das Format der auszugebenden weiteren Argumente.



Formatelemente beginnen mit einem %-Zeichen. Jedes Formatelement wird durch Umwandlungszeichen beendet.



Ein Umwandlungszeichen definiert, was der übergebene Parameter für einen Typ besitzt und wie er dargestellt werden soll. Folgende Tabelle definiert die wichtigsten Umwandlungszeichen:

Ausgabe von Integer		
d, i	int	dezimal, gegebenenfalls mit Vorzeichen.
o	unsigned int	oktal ohne Vorzeichen (ohne führende null).
x, X	unsigned int	hexadezimal ohne Vorzeichen in Klein- beziehungsweise Großbuchstaben (ohne führendes 0x beziehungsweise 0X).
u	unsigned int	ohne Vorzeichen in dezimaler Form.
c	int	als Zeichen. Dabei wird das Argument in den Typ unsigned char gewandelt.
b	int	binär, existiert erst ab dem Standard C23.
Ausgabe von Strings		
s	char*	als String. Zeichen des Arrays werden bis zum Nullzeichen (jedoch nicht einschließlich) geschrieben.
Ausgabe von Gleitpunkt-Zahlen		
f	double	als dezimale Zahl.
e, E	double	als Exponentialzahl, wobei das den Exponenten anzeigende e klein beziehungsweise groß geschrieben ist.
g, G	double	als Exponentialzahl beziehungsweise als Dezimalzahl in Abhängigkeit vom Wert. Nullen am Schluss sowie ein Dezimalpunkt am Schluss werden nicht ausgegeben.
Weiteres		
p	Pointer-Typ	als Adresse.
%		als %-Zeichen.
n	int*	In das übergebene Argument wird die Zahl der von printf() geschriebenen Zeichen abgelegt.

Wenn die Formatelemente nicht zum Typ der Argumente passen oder die Zahl der Argumente nicht stimmt, wird die Ausgabe von `printf()` falsch oder kann gar zum Absturz des Programmes führen.



Enthält der Format-String keine Formatelemente, so stellt dieser String einfach einen String dar, der ausgegeben werden soll, wie im Beispiel:

```
printf("Hier gibt es keine Formatelemente.");
```

Je nach Formatelement können zwischen dem einleitenden %-Zeichen und dem abschließenden Umwandlungszeichen weitere Formatierungs-Zeichen stehen. Diese werden in den folgenden Unterkapiteln genauer erläutert.

16.7.3 Wichtige Formate für die Ausgabe von Integer

Bei Integer wird die Feldbreite der Ausgabe angegeben durch `%nd`. Die Zahl `n` legt dabei die Feldbreite, also die Anzahl der Stellen, fest.



Ist die Zahl schmaler als die angegebene Feldbreite, so wird mit Leerzeichen bis zur angegebenen Feldbreite aufgefüllt. Da die angegebene Feldbreite vom Compiler ignoriert wird, wenn sie nicht ausreicht, spricht man oft von der sogenannten minimalen Feldbreite. Es ist zu beachten, dass ein eventuelles Vorzeichen ebenfalls in diese Feldbreite miteinbezogen wird. Bei negativen Zahlen hat also stets eine Ziffer weniger Platz.

Wenn vor der Feldbreite eine `0` (null) geschrieben wird, so wird die Zahl von links her mit Nullen aufgefüllt, sodass sie die Feldbreite voll ausnutzt. Wird vor die Feldbreite ein Minus-Zeichen geschrieben, so wird die Zahl nicht rechtsbündig, sondern linksbündig ausgegeben. Wenn zusätzlich ein Plus-Zeichen geschrieben wird, wird das Vorzeichen der Zahl auch für positive Werte ausgegeben.

Im folgenden Beispiel sind die wichtigsten Formatelemente zu beobachten:

```
int x = 99;
printf("%d Dalmatiner\n", x);
printf("%5d Dalmatiner\n", x);
printf("%05d Dalmatiner\n", x);
printf("%-5d Dalmatiner\n", x);
printf("%+5d Dalmatiner\n", x);
printf("%+05d Dalmatiner\n", x);
printf("%-+5d Dalmatiner\n", x);
```

Die Ausgabe ist:

```
99 Dalmatiner
   99 Dalmatiner
00099 Dalmatiner
99  Dalmatiner
+99 Dalmatiner
+0099 Dalmatiner
+99  Dalmatiner
```

16.7.4 Wichtige Formate für die Ausgabe von Gleitpunkt-Zahlen

Im Folgenden werden die verschiedenen Formate von Gleitpunkt-Zahlen betrachtet:

```
double a = 0.000006;
double b = 123.4;
printf("%e", a);
printf("%E", a);
printf("%f", a);
printf("%g", a);
printf("%G", a);
printf("%g", b);
printf("%G", b);
```


Die Ausgabe ist:

```
a:  
6.000000e-06  
6.000000E-06  
0.000006  
6e-06  
6E-06  
b:  
123.4  
123.4
```

Die Formatelemente %e und %E dienen zur Darstellung einer Gleitpunkt-Zahl als Exponentialzahl mit Signifikante und Exponent.



Dabei wird das e, das den Exponenten charakterisiert, bei Angabe von %e als kleines e ausgegeben, bei Angabe von %E als großes E. Das Formatelement %f dient zur Ausgabe als Dezimalzahl. %g und %G geben je nach Größe der Zahl diese als Exponentialzahl oder als Dezimalzahl aus.

Mit Hilfe der Formatelemente von printf() zur Ausgabe von Gleitpunkt-Zahlen als Dezimalzahlen kann die Gesamtbreite der Ausgabe (Feldbreite) und die Zahl der Stellen hinter dem Dezimalpunkt beeinflusst werden, wie aus den folgenden Beispielen ersichtlich ist:

%f	Gibt eine Gleitpunkt-Zahl im Default-Format des Compilers aus.
%5.2f	Gibt eine Gleitpunkt-Zahl mit insgesamt 5 Stellen aus, 2 hinter dem Punkt, 1 für den Punkt und 2 vor dem Punkt.
%5.0f	Die Gleitpunkt-Zahl wird mit 5 Stellen ausgegeben ohne Punkt und ohne Stelle nach dem Punkt.
%.3f	Es sollen 3 Stellen hinter dem Punkt und der Punkt ausgegeben werden. Für die Zahl der Stellen vor dem Punkt erfolgt keine Anweisung.

Für die Formatangaben vor dem Dezimalpunkt können dieselben Formatierungsmöglichkeiten benutzt werden wie bei Integer.

Die folgenden Zeilen verwenden diese Formatelemente:

```
float zahl = 12.3456f;  
printf("zahl = %f\n", zahl);  
printf("zahl = %5.2f\n", zahl);  
printf("zahl = %5.0f\n", zahl);  
printf("zahl = %.3f\n", zahl);
```

Dies erzeugt folgende Ausgabe:

```
zahl = 12.345600  
zahl = 12.35  
zahl = 12  
zahl = 12.346
```

Es ist dabei zu beachten, dass die Zahl gegebenenfalls automatisch auf die entsprechende Anzahl Stellen gerundet wird.

16.7.5 Wichtige Formate für die Ausgabe von Strings

Strings werden ausgegeben mit dem Formatelement %s.



Dies zeigen folgende Beispiele:

```
const char* s1 = "ist";  
printf("Programmieren %s einfach\n", s1);  
printf("Programmieren %6s einfach\n", s1);  
printf("Programmieren %-6s einfach\n", s1);
```

Die Ausgabe ist:

```
Programmieren ist einfach  
Programmieren  ist einfach  
Programmieren ist  einfach
```

Bei der Ausgabe von char-Arrays erhält die Funktion `printf()` einen Pointer auf das erste Zeichen eines char-Arrays. Die Funktion `printf()` gibt die Zeichen des Arrays aus, bis ein Nullzeichen `'\0'` gefunden wird. Das Nullzeichen wird nicht ausgegeben.



Strings mit Angabe einer positiven Feldbreite werden rechtsbündig ausgegeben. Wird die Feldbreite negativ angegeben, so wird der String linksbündig ausgegeben.



16.8 Lesen von Dateien mit High-Level-Funktionen

Für das Lesen von Dateien existieren grundsätzlich äquivalente Funktionen wie für das Schreiben von Dateien (gemäß Kapitel [→ 16.6](#)). Beim Lesen werden die Funktionen jedoch mit `fget()` anstatt mit `fput()`, mit `fread()` anstatt mit `fwrite()` beziehungsweise mit `fscan()` anstatt mit `fprint()` bezeichnet.

Grundsätzlich können die obigen Erklärungen somit analog für das Lesen von Dateien angewendet werden. Dennoch gibt es bei den Lese-Funktionen ein paar nennenswerte Ausnahmen. Im Folgenden werden die Lese-Funktionen der Standardbibliothek mit demselben Aufbau wie bei den Schreib-Funktionen durchgearbeitet.

Wenn eine Datei erfolgreich im Lese-Modus geöffnet wurde, können die in der Datei enthaltenen Daten mit Hilfe verschiedener Funktionsaufrufe in Variablen des Programmes eingelesen werden. Genauso wie beim Schreiben von Dateien findet das Lesen grundsätzlich zeichenweise statt. Das heißt, dass jedes Byte einzeln von der Datei gelesen und in den Hauptspeicher übertragen wird.

Die Funktion `fgetc()` ist die grundlegendste Funktion, um Daten von einer Datei einzulesen. Unter dem Namen `getc()` ohne `f` existiert wiederum ein Makro, welches exakt dasselbe tut wie `fgetc()`, jedoch als Makro ausprogrammiert ist. Das Makro sollte nicht verwendet werden, wenn die Parameter Ausdrücke mit Nebeneffekten darstellen. Nebst den folgenden beiden Prototypen gibt es noch die zusätzliche Funktion `ungetc()`, die weiter unten behandelt wird. Hier die Prototypen:

```
int fgetc (FILE* stream);  
int getc (FILE* stream);  
int ungetc(int c, FILE* stream);
```

Die Funktion `fgetc()` liest – wenn vorhanden – das nächste Zeichen als unsigned char aus der durch `stream` definierten Datei und konvertiert es nach `int`. Im Fehlerfall wird das Fehlerflag für den Stream gesetzt und EOF zurückgegeben.

Die Funktion `fgetc()` liest ein Zeichen aus einer Datei. Wenn die Datei als Binärdatei geöffnet wurde, so wird das Byte ohne weitere Konvertierung direkt übertragen. Wenn die Datei als Textdatei geöffnet wurde, so werden gegebenenfalls Zeilenende-Zeichen je nach Betriebssystem umkonvertiert.



Alle anderen Lese-Funktionen bauen auf `fgetc()` auf. Man muss sich also beim Öffnen der Datei genau überlegen, ob die Datei als Binärdatei oder als Textdatei behandelt werden soll.

Mit der Funktion `ungetc()` kann man ein Zeichen `c` für eine erneute Leseoperation in die durch `stream` definierte Datei zurückstellen. Dabei verhält sich die Funktion `ungetc()` ähnlich wie die Funktion `fputc()`, wird jedoch durch die Standardbibliotheken dahingehend unterstützt, dass diese Funktion auch auf nur lesbaren Dateien angewendet und mindestens ein Funktionsaufruf garantiert ausgeführt werden kann.

Damit ist es möglich, ganz einfach Parser zu schreiben, welche jeweils ein einzelnes Zeichen lesen und daraufhin entscheiden, ob das Zeichen zusammen mit den nachfolgenden Zeichen als zusammengehöriges Token gelesen werden soll. Das Programm stellt in diesem Falle das Zeichen einfach wieder in die Datei zurück und liest daraufhin das Token vollständig ein.

Die Funktion `ungetc()` stellt ein Zeichen in eine Datei zurück. Direkt nach dem Zurückstellen kann das Zeichen wieder beispielsweise mit der Funktion `fgetc()` gelesen werden.



Im Folgenden werden die erweiterten Lese-Funktionen der Standardbibliothek `<stdio.h>` vorgestellt. Genauso wie bei den Schreib-Funktionen wird grob unterschieden zwischen Objekten und formatierten Strings.

16.8.1 Lesen von Objekten und Strings

Die folgenden Funktionen erlauben es, Objekte und Strings mittels eines einzigen Funktionsaufrufes aus einer Datei einzulesen:

```
size_t fread(void* ptr,
             size_t size,
             size_t nmemb,
             FILE* stream);

char* fgets (char* s,
             int n, FILE* stream);
```

Die Funktion `fread()` liest ganze Arrays von Objekten aus einer Datei. Die Funktion `fgets()` liest einen String aus einer Datei.



Die Funktion `fgets()` liest aus der durch `stream` definierten Datei und schreibt das Ergebnis in den Puffer, auf den der Pointer `s` zeigt. Das Lesen wird abgebrochen, wenn entweder das Zeilenende-Zeichen `'\n'` oder das Dateiende erreicht ist oder `n - 1` Zeichen gelesen wurden. Das Zeilenende-Zeichen wird in den Puffer kopiert, ein Dateiende-Zeichen nicht. An die in den Puffer geschriebenen Zeichen wird als Stringende-Zeichen das Zeichen `'\0'` angehängt.

Die Funktion `fread()` liest `nmemb` Objekte der Größe `size` aus der durch `stream` definierten Datei aus und schreibt diese in das Array, auf das der Pointer `ptr` zeigt. Insgesamt werden maximal `nmemb * size` Bytes gelesen.

Für die Eingabe vom Standard-Eingabegerät gibt es zudem folgende Funktions-Prototypen:

```
int    getchar(void);  
char* gets_s (char* s, rsize_t n); // erst seit C11
```

Hierbei entspricht `getchar()` dem Makro `getc()` und `gets_s()` entspricht der Funktion `fgets()`. Beide Funktionen nutzen die Standardeingabe `stdin`.

Bei beiden Funktionen ist zu beachten, dass die Standardeingabe zeilengepuffert ist. Das bedeutet, dass die Funktionen `getchar()` und `gets_s()` solange warten, bis eine Eingabezeile mit <RETURN> abgeschlossen wurde. Erst dann wird der Tastaturpuffer geleert und als Dateipuffer von `stdin` an das Programm übergeben.

Mit der Funktion `getchar()` wird ein einzelnes Zeichen vom Eingabestrom `stdin` gelesen.



Das Zeichen wird als `unsigned char` gelesen und in `int` umgewandelt.

`gets_s()` liest einen String von der Standardeingabe `stdin` in ein Array von `n` Zeichen ein, auf das der Pointer `s` zeigt.



Die Funktion garantiert, dass nicht mehr als `n - 1` Zeichen gelesen werden und der Puffer somit nicht überlaufen kann. Es werden solange Zeichen von `stdin` eingelesen, bis ein Zeilenende-Zeichen oder das Zeichen EOF auftritt. An das letzte Zeichen wird ein Stringende-Zeichen `'\0'` angehängt.

Die Funktion `gets_s()` existiert erst seit dem C11-Standard. Vor dem C11-Standard wurde von den Standardbibliotheken eine Funktion `gets()` unterstützt, welche jedoch keinen Parameter `n` aufwies.

Bei der veralteten Funktion `gets()` konnte ein eingelesener String einen Überlauf des Puffers verursachen und den Speicher beliebig überschreiben.



Sichere Ein- und Ausgabefunktionen werden in Kapitel [→ 16.10](#) behandelt.

16.9 Lesen von formatierten Strings

Die Funktion `scanf()` („scan formatted“) dient zum Einlesen von formatierten Werten von der Standardeingabe.



Es sei angemerkt, dass die Standardeingabe zeilengepuffert ist. Das bedeutet, dass Eingaben grundsätzlich durch `<RETURN>` abgeschlossen werden müssen. Der Funktions-Prototyp von `scanf()` lautet:

```
int scanf(const char* format, ...);
```

In C11 ist der Parameter `format` zusätzlich mit dem `restrict`-Schlüsselwort deklariert.

Für `float`-Typen wird bei `scanf()` das Formatelement `%f` verwendet und für `double`-Typen das Formatelement `%lf`. Bei `printf()` wird jeder `float`-Wert implizit in einen `double`-Wert konvertiert, somit spielt diese Unterscheidung bei `printf()` keine Rolle.



Die Funktion `scanf()` gilt heutzutage als unsicher. Aufgrund des Formatstrings und der Übergabe von Speicheradressen kann es zu unerlaubten Speicherzugriffen kommen, wenn falsche Eingaben geliefert werden. Mehr Informationen zu sicheren Ein- und Ausgabefunktionen können in Kapitel [16.10](#) nachgelesen werden.

16.9.1 Varianten von `scanf()`

Mit `scanf()` werden Werte formatiert eingelesen. Die Werte werden schlussendlich in Variablen gespeichert.

Bevor jedoch erklärt wird, wie Ausdrücke formatiert eingelesen werden können, werden die verschiedenen Varianten von `scanf()` vorgestellt:

Von dieser Funktion gibt es verschiedene Implementierungen, welche allesamt dieselben Formatierungen einlesen können, jedoch unterschiedliche Ein- und Ausgänge besitzen. Die entsprechenden Funktionen werden durch unterschiedliche Präfixe vor dem `scanf()` ausgedrückt:

<code>scanf</code>	Eingabe aus <code>stdin</code>
<code>fscanf</code>	Eingabe aus Datei (<code>f</code> = File)
<code>sscanf</code>	Eingabe aus String
<code>vscanf</code>	Eingabe aus <code>stdin</code> , Parameter aus variabler Parameterliste
<code>vfscanf</code>	Eingabe aus Datei (<code>f</code> = File), Parameter aus variabler Parameterliste
<code>vsscanf</code>	Eingabe aus String, Parameter aus variabler Parameterliste

Die Funktion `scanf()` muss in der Lage sein, beliebig viele Werte einzulesen. Daher enthält der Prototyp von `scanf()` auch die Auslassung `(...)`. Genauso wie bei `printf()` können mit dem `v`-Präfix die aufzufüllenden Variablen anstelle der Auslassung mittels einer variablen Parameterliste mit dem Typ `va_list` angegeben werden. Hier werden diese Funktionen nicht weiter behandelt. Weitere Informationen zu variablen Parameterlisten und den Ellipsen können in Kapitel [→ 11.7](#) nachgelesen werden.

Das Präfix `f` sorgt dafür, dass die Eingabe nicht von der Standardeingabe `stdin` erfolgt, sondern von einem beliebigen File-Pointer. Folgendes ist der Prototyp der Funktion:

```
int fscanf(FILE* stream, const char* format, ...);
```

In C11 sind die beiden Parameter zusätzlich mit dem `restrict`-Schlüsselwort deklariert.

Das Präfix `s` sorgt dafür, dass die Eingabe von einem String erfolgt. Folgendes ist der Prototyp:

```
int sscanf(const char* s, const char* format, ...);
```


In C11 sind die beiden Pointer-Parameter wiederum zusätzlich mit dem `restrict`-Schlüsselwort deklariert.

Bei dieser Funktion werden die Werte für die aufzufüllenden Variablen aus dem in `s` gegebenen String extrahiert.

Es ist anzumerken, dass im Gegensatz zu den Schreib-Funktionen hier keine `sn`-Variante existiert. Während beim Schreiben in einen String der gegebene Puffer `s` ohne Angabe der Größe des Puffers überlaufen könnte, kann beim Lesen von einem `'\0'`-terminierten String ein ebensolcher Überlauf nicht stattfinden, da die Funktion nicht über das `'\0'`-Zeichen hinaus liest.

Die meisten Lesefunktionen erwarten explizit einen `'\0'`-terminierten String als Quelle. Handelt es sich bei der Quelle nicht um einen `'\0'`-terminierten String, so ist das Verhalten unbestimmt.



16.9.2 Formatiertes Einlesen von Werten

Genauso wie bei den Schreib-Funktionen hat die Funktion `scanf()` den Parameter `format`. Dieser Format-String enthält grundsätzlich genau dieselben Formatelemente mit dem Prozent-Zeichen `%` wie die Funktion `printf()`. Für genaue Erklärungen der Formatelemente wird somit auf die Beschreibung der Schreib-Funktionen in Kapitel [16.7.2](#) verwiesen.

Der wichtigste Unterschied zwischen `printf()` und `scanf()` zeigt sich bei den Parametern nach dem Format-String:

Während die Funktion `printf()` als Argumente bis auf den Format-String Ausdrücke mit Werten erwartet, erwartet die Funktion `scanf()` Adressen von Variablen.



```
int a;  
scanf("%d", &a);
```

Dies deswegen, da die Funktion `scanf()` den gelesenen Wert irgendwo speichern, also einem L-Wert zuweisen muss. Dies kann nur mittels Angabe einer Speicheradresse erreicht werden.

Beim Lesen von formatierten Werten muss man normalerweise nur angeben, von welchem Typ die Variable ist, in welcher der Wert gespeichert werden soll.

Stimmt ein Formatelement nicht mit dem Pointer-Typ des entsprechenden Argumentes überein, so kann das Programm abstürzen.



Des Weiteren hat `scanf()` ein spezielles Verhalten bei fehlerhaftem Lesen. Wird von der Eingabe ein nicht kompatibler Wert gelesen (beispielsweise ein String anstatt eines Integer), so resultiert ein Fehler. Wird bei einem Formatelement ein solch inkompatibler Wert gelesen, so beendet `scanf()` bei genau diesem Formatelement seine Tätigkeit und stellt das unverdauliche Zeichen zurück in die Standardeingabe.

Sollen also beispielsweise ein Integer `i` und eine Gleitpunkt-Zahl `x` über die Tastatur eingelesen werden, so wird folgende Zeile programmiert:

```
scanf("%d %e", &i, &x);
```

Erfolgt nun fälschlicherweise die Eingabe `"13 abc"`, so entspricht `"abc"` nicht dem erlaubten Format einer `float`-Zahl. In diesem Fall wird zwar in der Variablen `i` der erwartete Wert 13 stehen, der Inhalt von `x` ist jedoch undefiniert. Um solche Fälle aufzudecken, gibt `scanf()` als Rückgabewert die Anzahl der erfolgreich gelesenen Werte zurück. Der Rückgabewert von `scanf()` muss daher immer überprüft werden, wenn nicht ausgeschlossen werden kann, dass unpassende Eingaben kommen. Bei einer fehlerhaften Eingabe ist allerdings auch noch darauf zu achten, dass der Eingabepuffer mit den fehlerhaften Zeichen gelöscht wird, da sie ja nicht gelesen werden konnten und somit noch nicht verarbeitet wurden.

Als Rückgabewert gibt die Funktion `scanf()` die Anzahl der erfolgreich eingelesenen Eingabefelder zurück. Geht das Einlesen beim ersten Formatelement schief, so ist 0 der Rückgabewert. Der Rückgabewert von `scanf()` muss stets überprüft werden, um nicht verträgliche Eingaben erkennen und behandeln zu können.



Steht im Format-String ein Whitespace-Zeichen wie beispielsweise ein Leerzeichen oder ein Newline-Zeichen '\n', so bedeutet es für `scanf()`, alle Whitespace-Zeichen, die von der Standardeingabe kommen, zu überlesen. `scanf()` liest diese Zeichen aus der Standardeingabe und wirft sie weg. Steht kein Leerzeichen da, so wird auch keines gelesen (oder überlesen).

Ein unvorsichtigerweise im Format-String an ungünstiger Stelle angegebenes Leerzeichen oder Newline-Zeichen kann zu unerwarteten Effekten führen.



Die Funktion `scanf()` ist nützlich für einfache Eingaben, kann jedoch sehr kompliziert werden, wenn komplexe Daten damit gelesen werden sollen. Die Vielzahl an Möglichkeiten der Funktion `scanf()` kann in diesem Buch nicht behandelt werden. Abschließend wird im Folgenden ein einfaches Programmbeispiel vorgestellt, welches einen einfachen Einstieg in das Einlesen von Parametern über die Standardeingabe ermöglicht:

scanf1.c

```
// MSVC meldet fuer veraltete, unsichere C-Funktionen wie
// scanf einen Fehler.
// Mittels folgender Zeile koennen diese Fehler ausgeschaltet
// werden:
#define _CRT_SECURE_NO_WARNINGS

#include <stdio.h>

int main(void) {
    printf("Eingabe: ");

    int zahl;
    int anzahl = scanf("%d", &zahl);

    printf("%d Wert erfolgreich eingelesen.\n", anzahl);
    printf("Der Wert war %d.\n", zahl);
    return 0;
}
```

Der folgende Dialog wurde geführt:

```
Eingabe: 3
1 Wert erfolgreich eingelesen,
der Wert war 3.
```

16.10 Alternative Funktionen für mehr Sicherheit nach C11

„Der Programmierer weiß, was er tut“ – dieser Ansatz ist früher wie heute ein Kernpunkt der Philosophie der Programmiersprache C. Das Schreiben eines Programmes soll demnach durch die Sprache nicht eingeschränkt werden. Diese Tatsache eröffnet sehr viele Möglichkeiten, die in anderen Sprachen – wie beispielsweise in Java – nicht verfügbar sind. Allerdings stellt dieser Ansatz auch ein recht großes Sicherheitsproblem dar. Aus Unachtsamkeit können sehr leicht unbeabsichtigte Pufferüberläufe auftreten. Sie sind häufig das Ergebnis, wenn wichtige Abfragen vergessen gehen, etwas übersehen wird oder schlicht und einfach jemand eben nicht ganz so genau wusste, was zu tun war. Die Auswirkungen können ganz harmlos oder aber sehr schwerwiegend sein, wie beispielsweise der Absturz des Systems oder das Löschen wichtiger Daten. Sie lassen sich von vornherein nicht vorhersagen. Pufferüberläufe können außerdem von außen ausgenutzt werden, um Angriffe auf das System zu starten, was in Zeiten des Internets keine Seltenheit mehr ist.

Viele Funktionen zur Datei- und String-Manipulation der Standardbibliothek in C vertrauten früher darauf, dass man beim Einlesen Arrays bereitstellte, die groß genug für die Anzahl der einzulesenden Zeichen waren. Die Standardbibliothek stellte zudem wie beispielsweise bei der Funktion `gets()` keine Möglichkeiten bereit, diesen Umstand zu überprüfen. Die Überprüfung musste selbst vorgenommen werden.

Ein guter Programmierstil war es, wenn die Größe der `char`-Arrays so gewählt wurde, dass dieser Puffer für den normalen Gebrauch ausreichte. Waren diese Arrays allerdings doch nicht groß genug, dann wurden Daten über das Array hinaus in den Speicher geschrieben und damit eventuell andere Daten oder gar Programmanweisungen überschrieben. Das Programm erfuhr nie etwas davon, wenn es tatsächlich zum Pufferüberlauf kam und hatte daher keine Möglichkeit, den Anwender zu warnen oder diesen Fehler irgendwie zu behandeln. Ein solch nachlässiger Programmierstil wurde früher von der Standardbibliothek indirekt gefördert, da viele Funktionen keine Überprüfungsmöglichkeiten bereitstellten, um solche Pufferüberläufe zu verhindern.

Viele alte Funktionen der C-Standardbibliothek konnten zu Sicherheitsproblemen führen. Beispielsweise verhinderte es die Funktion `gets()` in keinsten Weise, dass es zu Pufferüberläufen kommen konnte.



Der Microsoft Visual C Compiler meldet deswegen für unsichere Funktionen mittlerweile einen Fehler. Um diese Fehler zu umgehen und die ursprünglich für C standardisierten Funktionen nutzen zu können, kann in der ersten Zeile einer Implementationsdatei das Makro `_CRT_SECURE_NO_WARNINGS` definiert werden. Dann wird der Compiler keine Fehler generieren.



Die ursprünglich definierten Funktionen sind grundsätzlich immer noch verfügbar, existieren aber nur noch zur Kompatibilität mit früherem Code. Heutige C-Standards definieren neue, sichere Funktionen.

Um beispielsweise Pufferüberläufe zu verhindern, führt C11 die sogenannten „**bounds checking interfaces**“ ein. Es handelt sich hierbei um eine optionale Erweiterung, die eine Sammlung von alternativen **sicheren Funktionen** zu bereits existierenden unsicheren Funktionen der Standardbibliothek bereitstellt. Diese neuen Funktionen haben alle ein abschließendes `_s` am Ende ihres Namens, um sie von den normalen Funktionen unterscheiden zu können.



Diese neuen Funktionen sollen ein sicheres Programmieren fördern, indem sie beispielsweise die Puffergröße prüfen und Fehlermeldungen ausgeben, wenn der Puffer nicht groß genug ist. Damit können Pufferüberläufe einfacher vermieden werden. Diese und weitere Vorteile bieten die rund 60 neuen Funktionen beispielsweise für die Bereiche Ein- und Ausgabe, das Arbeiten mit Streams oder Zeitfunktionen. Hier in diesem Buch werden sie nur konzeptionell angesprochen.

Da die bounds checking interfaces eine optionale Erweiterung in C11 sind, existiert ein Makro, über das sich prüfen lässt, ob die neuen sicheren Funktionen überhaupt verfügbar sind. Ist das Makro `__STDC_WANT_LIB_EXT1__` als 1 definiert, dann sind diese Funktionen enthalten, ist es aber als 0 definiert, dann sind sie nicht in der Bibliothek enthalten. Ist dieses Makro dagegen gar nicht definiert, dann ist es implementierungsabhängig, ob diese neuen Sicherheitsfunktionen in der Standardbibliothek enthalten sind oder nicht.

Das Benutzen einer alternativen sicheren `_s`-Funktion anstatt ihres unsicheren Gegenparts aus der Standardbibliothek bringt verschiedene Vorteile mit sich. Die wichtigsten werden hier aufgeführt:

- Pufferüberläufe werden verhindert.
- Ergebnisstrings erhalten immer ein abschließendes `'\0'`-Zeichen.
- Zugriffsrechte können eingeschränkt werden.
- Rückgabewerte sind einheitlich vom Typ `errno_t` und geben Aufschluss über den Erfolg der durchgeführten Operation.

Jede Funktion mit `_s` am Ende kann ihr jeweiliges Gegenstück (ohne `_s`) ersetzen.



Hierbei müssen nur ein bis zwei Zeilen Code geändert werden.

16.10.1 Die Funktionen `gets()` und `gets_s()`

Die Standardbibliotheksfunktion `gets()` ist nach C90 folgendermaßen deklariert:

```
char* gets(char* s);
```

Sie liest eine Zeile aus der Standard-Eingabe (`stdin`) ein, bis sie auf ein Newline-Zeichen (`'\n'`) oder ein End of File stößt (EOF), und schreibt sie in einen Puffer, der vom Aufrufer zur Verfügung gestellt wird. Diese Funktion kennt allerdings ihre maximal verfügbare Puffergröße nicht. Dies stellt ein Sicherheitsproblem dar, da es sehr leicht zu Pufferüberläufen kommen kann. Aus diesem Grund gilt `gets()` als unsichere Funktion, wird seit C99 als veraltet eingestuft und ist mit C11 schließlich aus dem Standard entfernt worden. Ersetzt wird sie durch die Funktion `gets_s()`:

```
char* gets_s(char* s, rsize_t n);
```

Die `gets_s()`-Funktion arbeitet im Prinzip fast genauso wie ihr Vorgänger `gets()`. Sie liest allerdings nur maximal $n - 1$ Zeichen ein, da ihr Puffer $n - 1$ Zeichen plus das anzuhängende Zeichen `'\0'` aufnehmen kann. Ist die maximale Anzahl einzulesender Zeichen erreicht, dann wird der Lesevorgang beendet, auch wenn das Zeilenende noch nicht erreicht ist.



Wird das Zeilenende erreicht, dann wird das Newline-Zeichen nicht mit in den Puffer geschrieben. Nachdem das letzte Zeichen eingelesen wurde, wird ein abschließendes `'\0'`-Zeichen ans Ende des Puffers eingefügt.

Die Funktion `gets_s()` ist allerdings Teil der optionalen Erweiterung von C11. Als Alternative wird im Standard auf die `fgets()`-Funktion verwiesen:

```
char* fgets(char* s, int n, FILE* stream);
```

Diese Funktion verhält sich ganz ähnlich wie `gets_s()`. Sie liest statt von der Standard-Eingabe von einer Datei ein und schreibt – im Gegensatz zu `gets_s()` – dabei auch das Newline-Zeichen in den Puffer, falls es vorkommt. Will man von der Tastatur einlesen, so muss man einfach `stdin` als File-Pointer angeben.

16.10.2 Die Funktion `fopen_s()`

Eine weitere Funktion aus den bounds checking interfaces ist die Funktion `fopen_s()`:

```
errno_t fopen_s(  
    FILE* restrict * restrict streamptr,  
    const char* restrict filename,  
    const char* restrict mode);
```

Im Prinzip macht die Funktion `fopen_s()` genau dasselbe wie die Funktion `fopen()`, sie öffnet eine Datei. Der Pointer `streamptr` auf den Stream wird der Funktion als Parameter übergeben. Konnte die Datei erfolgreich geöffnet werden, dann zeigt `streamptr` auf die neu geöffnete Datei, ansonsten ist dieser Pointer NULL. Gibt die Funktion eine 0 zurück, dann war das Öffnen der Funktion erfolgreich. Für den Parameter `mode` sind alle Modi zulässig, die auch für die `fopen()`-Funktion gelten.

16.11 Übungsaufgaben

Hinweis: Wird bei der Funktion `fopen()` als Dateiname ein String wie `"test.dat"` angegeben, so wird die Datei im aktuellen Arbeitsverzeichnis geöffnet. Das aktuelle Arbeitsverzeichnis wird normalerweise von der Entwicklungsumgebung festgelegt oder durch die Programmierumgebung. Da diese je nach Situation unterschiedlich sein kann, empfiehlt sich daher, für die Übungen absolute Pfade anzugeben, beispielsweise `"C:/cbuch/test.dat"`.

Aufgabe 1: Formatiert auf Festplatte schreiben und von der Festplatte lesen

- a) Schreiben Sie ein Programm, das `n` `int`-Zahlen als String in eine Datei schreibt. Schreiben Sie jeweils eine Zahl pro Zeile.
- b) Schreiben Sie ein zweites Programm, das diese Datei wieder einliest und den Durchschnitt der Zahlen berechnet.

Aufgabe 2: Zeilenweise lesen.

Schreiben Sie ein Programm, welches eine beliebige Textdatei öffnet und die Anzahl darin enthaltener Zeilen zählt. Nutzen Sie hierfür die Funktionen `fgets()` und `feof()`.

Beachten Sie beim Testen, dass `fgets()` auch leere Zeilen (beispielsweise am Ende der Datei) erfasst.

Aufgabe 3: Blockweise Ein- und Ausgabe.

Lassen Sie folgendes Programm laufen. Es erzeugt eine Binärdatei, in welche 7 int-Zahlen und 3 float-Zahlen geschrieben werden.

binaerDatei.c

```
// MSVC meldet fuer veraltete, unsichere C-Funktionen wie
// fopen einen Fehler.
// Mittels folgender Zeile koennen diese Fehler ausgeschaltet
// werden:
#define _CRT_SECURE_NO_WARNINGS

#include <stdio.h>
#define iN 7
#define fN 3

int iArr[iN] = { 1, 2, 3, 4, 5, 6, 7 };
float fArr[fN] = { 1.1f, 2.2f, 3.3f };

int main(void) {
    FILE* fp = fopen("test.dat", "wb");
    if (fp == NULL) {
        printf("Kann Datei nicht oeffnen.\n");
    } else {
        fwrite(iArr, sizeof(int), iN, fp);
        fwrite(fArr, sizeof(float), fN, fp);
        fclose(fp);
    }
    return 0;
}
```

Schreiben Sie ein Programm, welches diese Datei öffnet, alle Zahlen liest und sie auf dem Bildschirm ausgibt.

Aufgabe 4: Dateien, Strukturen und formatierte Ausgabe

Gegeben seien Daten, die der Verwaltung von Lagerbeständen dienen. Eine Liste solcher Daten sieht typischerweise folgendermaßen aus:

Artikelnummer	Lagerbestand	Gewicht je Stück	Preis
9169	500	6.334	19.47
4464	962	29.358	15.48
9961	827	23.281	28.15
5436	827	11.942	4.00

- Schreiben Sie eine Struktur, welche diese Datenstruktur aufnimmt und füllen Sie die oben angegebenen Daten (die 4 Datenreihen) in ein Array ein.
- Geben Sie den Inhalt dieses Arrays auf dem Bildschirm aus mit folgendem Format: Artikelnummer linksbündig, maximal 8 Zeichen, Lagerbestand rechtsbündig, Gewicht rechtsbündig mit stets drei Nachkommastellen, Preis rechtsbündig mit stets zwei Nachkommastellen und dem Zusatz „EUR“.
- Schreiben Sie den Inhalt dieses Arrays in eine Datei. Nutzen Sie hierfür keine String-Funktionen!
- Schreiben Sie dann eine Funktion zu folgendem Prototyp:

```
void transferInDollar(  
    const char* dateiNameIn,  
    const char* dateiNameOut);
```

Diese Funktion öffnet die gegebene Eingabedatei und konvertiert alle Preise von Euro in Dollar und schreibt die Daten in die gegebene Ausgabedatei.

Gehen Sie davon aus, dass es stets 4 Datenreihen sind.

- Überlegen Sie sich, wie man Daten lesen könnte, wenn nicht von vornherein klar ist, wieviele Daten sich in der Datei befinden.
- Könnte man diese Daten auch mittels Stringfunktionen speichern? Was für Probleme treten möglicherweise auf?

17 Programmaufruf und -beendigung



Ein Programm, welches in C geschrieben wird, muss zu einem bestimmten Zeitpunkt gestartet werden. Im Bereich des embedded programming wird hierfür schlicht das Programm in den Chip geladen und bei der ersten Anweisung mit der Ausführung begonnen.

Läuft ein Programm jedoch in einem Betriebssystem, so können beim Start zusätzliche **Argumente** übergeben werden, welche innerhalb des Programmes ausgelesen werden können und so die Ausführung des Programmes beeinflussen.

Bei Beendigung eines Programmes kann zudem ein Wert zurückgegeben werden. Dieser Wert kann vom Betriebssystem (oder von irgend einem aufrufenden Programm) ausgewertet werden.

17.1 Übergabe von Argumenten beim Programmaufruf

Informationen an ein Programm können beim Start des Programms in der Konsole als Argumente übergeben werden.



Nicht nur die Konsole erlaubt das Übergeben von Argumenten. Auch Programmierumgebungen (Visual Studio, Eclipse etc.) können Programme starten. Diese Programmierumgebungen bieten Möglichkeiten, wie Programme, die Parameter erwarten, mit Argumenten versorgt werden können. Dies ist insbesondere nötig, wenn man solche Programme entwickeln und debuggen/testen will.

Beim Start-up des Programms wird die Funktion `main()` aufgerufen. Es gibt grundsätzlich zwei Möglichkeiten für die Definition von `main()`:

```
int main(void) { }
```

oder

```
int main(int argc, char* argv[]) { }
```

Ergänzende Information Die elektronische Version dieses Kapitels enthält Zusatzmaterial, auf das über folgenden Link zugegriffen werden kann https://doi.org/10.1007/978-3-658-45209-4_17.

Will man Argumente an das Programm übergeben, so ist die zweite Variante von `main()` zu wählen.

Für die Übergabe von Argumenten an die Funktion `main()` sind zwei spezielle formale Parameter vorgesehen: **`argc`** (**argument count**) und **`argv`** (**argument vector**).



Die Funktion `main()` darf normalerweise nur diese beiden formalen Parameter aufweisen. In manchen Umgebungen sind weitere Argumente erlaubt, wie beispielsweise die Umgebungsvariablen einer Konsole oder plattformspezifische Werte. Diese sind jedoch nicht standardisiert.

Es ist üblich, diese beiden formalen Parameter als `argc` und `argv` zu bezeichnen. Der Parameter `argc` enthält die Anzahl der übergebenen Argumente und ist vom Typ `int`. Der Parameter `argv` ist ein eindimensionales Array von Pointern auf `char`. Da ein eindimensionales Array äquivalent zu einem Pointer auf Pointer ist, kann der Typ dieses Parameters sowohl `char* argv []` als auch `char** argv` sein.

Der String, auf den der erste Pointer zeigt, ist das erste Argument der Konsole, also der Programmname.

Die Anzahl der übergebenen Argumente beträgt in der Regel mindestens 1, da der Programmname als erstes Argument zählt. Ausnahme: Programme, welche ohne Unterstützung eines Betriebssystems laufen, besitzen möglicherweise keinen Namen. Der Wert von `argc` wird in diesem Falle 0 statt 1 sein.



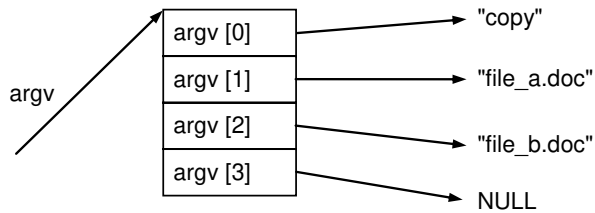
Da `char* argv[]` ein Array ohne Längenangabe ist, können theoretisch beliebig viele Argumente übergeben werden. Im Standard wird verlangt, dass am Ende des Arrays – sprich an der Stelle `argv[argc]` – ein NULL-Pointer gespeichert ist.

Im Folgenden ein Beispiel zur Veranschaulichung des Argumentvektors:

```
copy fileA.doc fileB.doc
```

In diesem Fall ist `copy` das ausführbare Programm und es erhält zwei Argumente (`fileA.doc` und `fileB.doc`). Hierbei soll das Programm `copy` von der Datei `fileA.doc` eine Kopie unter dem Namen `fileB.doc` anlegen.

Beim Aufruf von `copy` erhält `argc` die Anzahl der übergebenen Argumente, sprich die Zahl 3, und die Komponenten von `argv` erhalten Pointer auf die Argumente und den abschließenden `NULL`-Pointer:



17.1.1 Beispielprogramm:

Als Beispiel soll ein ausführbares Programm addieren geschrieben werden, bei dem die zwei übergebenen Zahlen addiert werden sollen.

Wird bei Programmaufruf eine Zahl übergeben, so ist zu beachten, dass diese als String übergeben wird und vor ihrer Verwendung erst in einen `int`-Wert umgewandelt werden muss.



Diese Umsetzung kann beispielsweise mit der Standardfunktion `atoi()` (ascii to integer) erfolgen (siehe auch Anhang [→ A](#)).

Hier das Programm im Quellcode:

addiere.c

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char * argv[]) {
    if (argc == 3) {
        int resultat = atoi(argv[1]) + atoi(argv[2]);
        printf("%s + %s = %d\n", argv[1], argv[2], resultat);
    } else {
        printf("Bitte 2 Zahlen eingeben.\n");
    }
    return 0;
}
```

Der Dialog mit dem Programm soll beispielsweise sein:

addiere 3 8	Aufruf des Programms
3 + 8 = 11	Ausgabe des Programms

Um Fehlfunktionen zu vermeiden, sollte immer überprüft werden, ob auch die korrekte Anzahl an Argumenten übergeben wurde, wie im Beispiel zu sehen ist.

17.2 Beendigung von Programmen

Nach Erledigung seiner Aufgabe beendet ein Programm normalerweise seine Tätigkeit. Läuft es auf einem Betriebssystem, so kann eine Beendigung auch erzwungen werden.

In den folgenden Unterkapiteln wird auf die verschiedenen Möglichkeiten eingegangen, Programme zu beenden. Zuerst jedoch wird erläutert, was genau bei Beendigung eines Programmes passiert.

17.2.1 Geordnete Beendigung eines Programmes

Wenn ein Programm beendet wird, so gibt es oftmals noch Dinge, welche erledigt werden müssen. Bei einem sogenannten „**geordneten**“ Programmabbruch werden folgende Schritte durchlaufen:

- Zuerst werden alle Funktionen, die bei der `atexit()`-Funktion (siehe Kapitel [→ 17.2.4](#)) registriert worden sind, in der umgekehrten Reihenfolge ihrer Registrierung aufgerufen.
- Alle noch nicht geschriebenen Ausgabepuffer werden in ihre zugeordnete Datei geschrieben und alle offenen Dateien geschlossen.
- Temporäre Dateien, welche mit der `tmpfile()`-Funktion (siehe Kapitel [→ 16.5.7](#)) erzeugt wurden, werden gelöscht.
- Dann wird die Kontrolle an die Umgebung, von der aus das Programm gestartet wurde, zurückgegeben. Hierbei wird der Abbruchstatus des Programmes in Form eines Integer an die Umgebung zurückgegeben.

Das übergeordnete Programm kann den Rückgabewert ermitteln und so auf die Beendigung reagieren. Dieser Rückgabewert wird dann entsprechend auch „**Exit-Status**“, oder schlicht „**Status**“ genannt. Ein Beispiel hierfür bei Aufruf eines Programmes von der Konsole wird in Kapitel [→ 17.2.9](#) gezeigt. Dort speichert ein Batch-Script den Rückgabewert automatisch in einer vorgegebenen `ERRORLEVEL`-Variablen.

Nicht nur das Betriebssystem jedoch kann diese Werte auslesen, sondern jeder Prozess, welcher einen anderen aufgerufen hat. Gerade in der Parallelverarbeitung werden Prozesse anhand der jeweils an den Steuerprozess zurückgegebenen Werte gesteuert. Unter Unix gibt es beispielsweise die Funktion `waitpid()`, welche auf die Beendigung eines beliebigen Prozesses wartet und den Status in einer Variablen speichert. Unter WINAPI gibt es eine ähnliche Funktion namens `GetExitCodeProcess()`.

Da die Art, wie der Status eines Programmes abgefragt werden kann, implementationsspezifisch ist, kann im Rahmen dieses Buches nicht weiter darauf eingegangen werden.

Normalerweise wird der Wert 0 zurückgegeben, wenn das Programm ohne Fehler beendet werden kann, ansonsten wird ein Wert ungleich 0 zurückgegeben. Hierfür existieren zwei Makros in der `stdlib` Bibliothek: `EXIT_SUCCESS` und `EXIT_FAILURE`. Es ist jedoch erlaubt, auch andere Werte zu verwenden.

17.2.2 return aus dem Hauptprogramm

In der `main()`-Funktion kann ein Programm mit der **return**-Anweisung verlassen werden. Hierbei gibt es die Möglichkeit, den Wert eines Ausdrucks zurückzugeben.



Die Beendigung mit `return` bewirkt einen geordneten Programmabbruch. Diese Art der Beendigung eines Programmes wird normalerweise für eher kleinere Programme verwendet.

17.2.3 Die Funktion `exit()`

Mit der `exit()`-Funktion kann ein Programm an beliebiger Stelle verlassen werden. Der Parameter der Funktion wird als Rückgabewert verwendet.



Genauso wie ein `return` aus der `main()`-Funktion bewirkt auch der Aufruf der `exit()`-Funktion einen geordneten Programmabbruch.

Syntax:

```
#include <stdlib.h>
void exit(int status);
```

Es ist zu beachten, dass ab dem Standard C11 der Rückgabewert mit dem Typ `_Noreturn void` angegeben wird. Damit wird dem Compiler der Hinweis gegeben, dass die Funktion `exit()` nicht zu ihrem Aufrufer zurückkehren kann, da das Programm abgebrochen wird.

17.2.4 Die Funktion `atexit()`

Die Funktion `atexit()` registriert eine Funktion, damit diese bei einem geordneten Programmabbruch ausgeführt wird.



Syntax:

```
#include <stdlib.h>
int atexit(void (*func)(void))
```

Die Funktion `atexit()` registriert eine Funktion, auf die der Funktionszeiger `func` verweist. Eine registrierte Funktion wird bei einem geordneten Programmabbruch (siehe Kapitel [→ 17.2.1](#)) ohne Argumente aufgerufen. Die Funktionen dürfen also keinen Parameter erwarten.

Beispiel:

atexit.c

```
#include <stdio.h>
#include <stdlib.h>

void final_1(void) {
    printf("final_1 ist dran\n");
}

void final_2(void) {
    printf("final_2 ist dran\n");
}

void final_3(void) {
    printf("final_3 ist dran\n");
}

int main(void) {
    atexit(final_1);
    atexit(final_2);
    atexit(final_3);
    printf("main ist dran\n");
    exit(EXIT_SUCCESS);
}
```

Das Ergebnis eines Programmlaufs ist:

```
main ist dran
final_3 ist dran
final_2 ist dran
final_1 ist dran
```

Nach dem Standard müssen mindestens 32 Funktionen registriert werden können. Diese werden alle aufgerufen und zwar in der umgekehrten Reihenfolge der Registrierung. Wird eine Funktion mehrmals registriert, wird sie auch mehrmals aufgerufen.

17.2.5 Die Funktion `abort()`

Der Aufruf der Funktion **`abort()`** führt zu einem **abnormalen** Abbruch des Programms.



Syntax:

```
#include <stdlib.h>
void abort(void);
```

Ab dem C11-Standard wird als Rückgabetyt `_Noreturn void` angegeben, sprich die Funktion kann nicht zu ihrem Aufrufer zurückkehren, da das Programm abgebrochen wird.

Ein abnormaler Abbruch eines Programmes entspricht einer sofortigen Beendigung mit möglichst wenigen Nebeneffekten. Explizit bedeutet dies, dass alle mittels `atexit()` registrierten Funktionen übergangen werden und nur die nötigsten Aufräumarbeiten für geöffnete Dateien geleistet werden.

Bei modernen Betriebssystemen werden zumindest offene Dateien üblicherweise geschlossen, um die Ressource anderen Prozessen wieder freigeben zu können. Es ist jedoch implementierungsabhängig, ob Daten, die sich noch im Puffer befanden, in ihre zugeordnete Datei geschrieben wurden.

Ein Aufruf der Funktion `abort()` bewirkt das sogenannte „Senden des Signals `SIGABRT`“. Ein solches Signal kann mittels eines geeigneten Signalhandlers vom Programm abgefangen werden, um beispielsweise doch noch Daten zu sichern. Signale und Signalhandler können im Rahmen dieses Buches jedoch nicht weiter behandelt werden.

Der Rückgabewert an das aufrufende Programm ist üblicherweise die Nummer des (auf Unix-Systemen standardisierten) Signals: 6.

17.2.6 Die Funktion `quick_exit()`

Die Funktion `quick_exit()` ist erst seit C11 definiert.

Die Funktion `quick_exit()` sorgt für einen beschleunigten Programmabbruch. Es werden keine noch nicht abgearbeiteten Ressourcen, wie beispielsweise Dateipuffer geschrieben. Vor dem Abbruch werden jedoch die mithilfe der Funktion `at_quick_exit()` registrierten Funktionen ausgeführt.



Syntax:

```
#include <stdlib.h>
_Noreturn void quick_exit(int status);
```

Im Gegensatz zu `exit()` führt `quick_exit()` nur die notwendigsten Dinge aus. Auch Funktionen, welche mit `atexit()` registriert wurden, werden ausgelassen. Dafür werden Funktionen aufgerufen, welche mit `at_quick_exit()` registriert wurden.

Im Gegensatz zu `abort()` handelt es sich bei `quick_exit()` jedoch um einen normalen Programmabbruch, bei welchem auch ein beliebiger Status zurückgegeben werden kann.

17.2.7 Die Funktion `at_quick_exit()`

Die Funktion `at_quick_exit()` ist erst seit C11 definiert.

Die Funktion `at_quick_exit()` registriert eine Funktion, damit diese bei einem Programmabbruch anhand der Funktion `quick_exit()` ausgeführt wird.



Syntax:

```
#include <stdlib.h>
int at_quick_exit(void (*func)(void));
```

Analog zur `atexit()`-Funktion lassen sich eigene Funktionen mit der Funktion `at_quick_exit()` registrieren. Diese werden bei Aufruf von `quick_exit()` ohne Parameter aufgerufen. Die Funktionen dürfen also keinen Parameter erwarten. Auch hier können nach Standard mindestens 32 Funktionen registriert werden. Die Funktionen werden ohne Argumente aufgerufen.

17.2.8 Die Funktion `_Exit()`

Die Funktion `_Exit()` führt eine sofortige Beendigung durch. Im Gegensatz zur Funktion `exit()` wird von der Funktion `_Exit()` jedoch keine Funktionen ausgeführt, welche mittels `atexit()` oder `at_quick_exit()` registriert wurden. Ob weitere automatische Aufräumarbeiten durchgeführt werden, ist implementationsspezifisch.



Syntax:

```
#include <stdlib.h>
void _Exit(int status);
```

Ab dem C11-Standard wird als Rückgabetyt `_Noreturn void` angegeben, sprich die Funktion kann nicht zu ihrem Aufrufer zurückkehren, da das Programm abgebrochen wird.

Auch wenn keine weiteren Funktionen ausgeführt oder andere automatische Aufräumarbeiten durchgeführt werden, wird dennoch der angegebene Status an die Umgebung zurückgeliefert.

17.2.9 Beispiel für eine Abfrage des Rückgabewertes

Das folgende Beispiel ruft unter Windows in einer Kommandoprozedur `open.bat` das Programm `open.exe` auf. Der Exit-Status von `open.exe` wird an eine Variable namens `ERRORLEVEL` übergeben und von der Batch-Datei `open.bat` ausgewertet.

Anmerkung: Um diese `bat`-Datei korrekt auszuführen, muss sie in einer passenden Konsole (etwa die Eingabeaufforderung) aufgerufen werden. Nicht jede Konsole kennt beispielsweise den `GOTO` Befehl.

open.bat

```
ECHO OFF
open
IF ERRORLEVEL 1 GOTO FATAL

IF ERRORLEVEL 0 ECHO Open erfolgreich gelaufen
GOTO ENDE

:FATAL
ECHO Fehler bei Open, Returnstatus = 1
:ENDE
```

Und hier der Quellcode des Programms open.exe:

open.c

```
#include <stdio.h>

int main(void) {
    FILE * fp;

    if ((fp = fopen("aufgabe.c", "r")) == NULL) {
        printf("aufgabe.c nicht vorhanden\n");
        return 1;
    }
    fclose(fp);

    printf("aufgabe.c vorhanden\n");
    return 0;
}
```

Je nachdem, ob die Datei `aufgabe.c` im aktuellen Pfad vorhanden ist oder nicht, wird ausgegeben:

```
aufgabe.c vorhanden
Open erfolgreich gelaufen
```

beziehungsweise

```
aufgabe.c nicht vorhanden
Fehler bei Open, Returnstatus = 1
```

18 Dynamische Speicherzuweisung



Bislang wurde stets mit Variablen gearbeitet, deren Struktur und Größe bereits während des Compilierens bekannt waren. Es gibt jedoch viele Situationen, bei denen nicht vorausgesagt werden kann, wie viel Speicherplatz gebraucht werden wird, so zum Beispiel, wenn eine (beliebig große) Datei eingelesen werden soll.

Die Sprache C bietet Mechanismen an, welche es erlauben, zur Laufzeit des Programmes dynamisch eine variable Menge an Bytes zu reservieren und wieder freizugeben. Diese sogenannten **dynamischen Speicherblöcke** oder Speicherbereiche befinden sich nicht wie die üblichen Variablen im Daten- oder Stack-Segment, sondern im sogenannten Heap-Segment eines Programmes (siehe auch Kapitel [→ 15.2.3](#)).

Der **Heap** ist ein Segment eines Programms neben den Segmenten Code, Daten und Stack. Aufgabe des Heaps ist es, Speicherplatz für die Schaffung dynamischer Speicherblöcke bereitzuhalten.



Dynamische Speicherblöcke werden in C bei Bedarf zur Laufzeit des Programms durch Aufruf der entsprechenden Bibliotheksfunktionen mit Hilfe des Laufzeitsystems auf dem Heap angelegt.



Die Verwaltung des Heaps erfolgt durch eine Komponente im Laufzeitsystem, die auch als Heap-Verwaltung bezeichnet wird. Die Heap-Verwaltung entscheidet beispielsweise, an welcher Stelle des Heaps Speicherplatz reserviert wird. Die Funktionen, die vom Programm aufgerufen werden, um dynamische Speicherblöcke anzulegen, geben dem Programm einen Pointer auf den im Heap reservierten Speicherbereich zurück.

Dieser Pointer kann in einer Variablen mit dem passenden Pointer-Typ gespeichert werden. Über diesen Pointer kann man dann auf die reservierte Speicherstelle im Heap zugreifen. In C kann durch Aufruf einer entsprechenden Bibliotheksfunktion ein Speicherblock auch wieder freigegeben werden. Dynamische Speicherblöcke stehen somit dem Programm von ihrer Erzeugung bis zu ihrer Freigabe zur Verfügung.

Ergänzende Information Die elektronische Version dieses Kapitels enthält Zusatzmaterial, auf das über folgenden Link zugegriffen werden kann https://doi.org/10.1007/978-3-658-45209-4_18.

Die **Gültigkeit** und **Lebensdauer** eines dynamischen Speicherblockes wird nicht durch Blockgrenzen bestimmt, das heißt nicht durch die statische Struktur des Programms.



In C erfolgt die Reservierung von Speicherblöcken auf dem Heap beispielsweise mit den Bibliotheksfunktionen `malloc()` beziehungsweise `calloc()`, die Freigabe erfolgt mit der Bibliotheksfunktion `free()`.

Je nach System ist die Größe des Heaps stark begrenzt. Moderne Computer besitzen mehr als ausreichend Speicherplatz, dennoch sollte er nicht verschwendet werden. Wenn ständig nur Speicher angefordert und kein Speicher zurückgegeben wird, kann es zu einem Überlauf des Heaps kommen. Hinzu kommt das Problem, dass mit zunehmendem Gebrauch der Heap mehr und mehr zerstückelt wird, so dass es sein kann, dass keine größeren Speicherobjekte mehr angelegt werden können, obwohl in der Summe genügend freier Speicher vorhanden ist, aber eben nicht am Stück.

In Sprachen wie Java oder Smalltalk werden Speicherobjekte im Heap nicht explizit freigegeben. Ein Garbage Collector gibt hier Speicherplatz, der nicht mehr referenziert wird, wieder frei. Der Garbage Collector wird in unregelmäßigen Abständen durch das Laufzeitsystem aufgerufen. Er ordnet ferner den Speicher neu, so dass größere homogene unbenutzte Speicherbereiche auf dem Heap wiederhergestellt werden. In C gibt es keinen Garbage Collector.

18.1 Reservierung von Speicher

Dieses Kapitel beschreibt die verschiedenen Funktionen, um Speicher zu reservieren und wieder freizugeben.

Alle Funktionen befinden sich in der Bibliothek `<stdlib.h>`.

18.1.1 Die Funktion `malloc()`

Die Funktion **`malloc()`** dient zum dynamischen Reservieren eines Speicherobjekts im Heap.



Syntax:

```
void* malloc(size_t size);
```

In C kann die Reservierung von Speicher im Heap mit der Funktion `malloc()` erfolgen. Der Name `malloc()` kommt von „memory allocation“. Ein solchermaßen erzeugtes Speicherobjekt ist bis zum Programmende oder bis zur Rückgabe des Speichers mit `free()` gültig.

Im Gegensatz zu Java und C++, wo es jeweils eine Funktion `new()` gibt, welche Speicherobjekte des gewünschten Datentyps anlegt, ist die Sichtweise in C nicht so abstrakt. Die Funktion `malloc()` muss wissen, wie viele Bytes benötigt werden. Die Zahl der Bytes wird üblicherweise mit Hilfe des `sizeof`-Operators bestimmt. Beispielsweise:

```
MyObject* obj = malloc(sizeof(MyObject));
```

Wenn die Speicherreservierung durchgeführt werden konnte, gibt die Funktion `malloc()` einen Pointer auf den reservierten Speicherbereich zurück, ansonsten wird `NULL` zurückgegeben.

Der zurückgegebene Pointer ist vom Typ `void*` und wird bei einer Zuweisung implizit in den Typ des Pointers auf der linken Seite der Zuweisung gewandelt. Es sei daran erinnert, dass abgesehen von `void*` kein Pointer auf einen Datentyp an einen Pointer auf einen anderen Datentyp ohne explizite Umwandlung zugewiesen werden kann (siehe Kapitel [→ 8.2](#)).

Es empfiehlt sich, bei Zuweisungen mit `malloc()` stets einen expliziten Cast zu verwenden, damit der Compiler die Zuweisung auf Typsicherheit prüfen kann.

Hier noch ein Beispiel:

`malloc.c`

```
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int* pointer = (int*)malloc(sizeof(int));
    if (pointer!= NULL) {
        *pointer = 3;
        printf("Wert an Pointer-Adresse: %d\n", *pointer);
        free(pointer);
    } else {
        printf("Nicht genugend Speicher verfuegbar.\n");
    }
    return 0;
}
```

Hier die Ausgabe des Programms:

Inhalt des Pointers: 3

18.1.2 Die Funktion `aligned_alloc()`

Die Funktion `aligned_alloc()` existiert seit dem C11-Standard und erlaubt im Gegensatz zur Funktion `malloc()` nicht nur die gewünschte Größe des Speicherblocks anzugeben, sondern zusätzlich dessen Alignment (Anordnung) im Speicher. Der Prototyp für die Funktion `aligned_alloc()` ist der Folgende:

```
void* aligned_alloc(size_t alignment, size_t size);
```

Weitere Informationen zum Alignieren (Anordnen) von Speicherbereichen können im Kapitel [→ 9.9.1](#) nachgelesen werden.

18.1.3 Die Funktion `calloc()`

Die Funktion `calloc()` reserviert ebenfalls Speicher im Heap, allerdings eine bestimmte Anzahl von dynamischen Speicherblöcken. Außerdem führt sie eine Initialisierung durch.



Syntax:

```
void calloc(size_t num, size_t size);
```

Als Übergabeparameter erwartet die Funktion `calloc()` zwei Parameter. Als erster Parameter wird die Anzahl der benötigten Variablen erwartet. Der zweite Parameter entspricht der Größe einer einzelnen Variablen in Bytes. Damit eignet sich `calloc()` sehr gut zur Reservierung von Speicher für Arrays.

Im Gegensatz zu `malloc()` initialisiert `calloc()` den bereitgestellten Speicher und setzt dabei alle Bits auf 0. Daher rührt auch der Name: `calloc()` bedeutet „clear allocation“.



Hiebei ist wichtig zu wissen, dass die Funktion `free()` Speicherplatz zwar freigibt, ihn jedoch nicht initialisiert. Mittels `calloc()` wird somit eine irrtümliche Weiterverwendung von zuvor mit `free()` freigegebenen Daten vermieden. Siehe dazu Kapitel [→ 18.2](#).

Die Funktion `calloc()` gibt genauso wie `malloc()` einen Pointer auf den reservierten Speicherbereich zurück, wenn die Speicherreservierung durchgeführt werden konnte, ansonsten wird `NULL` zurückgegeben. Der Pointer ist vom Typ `void*` und wird bei einer Zuweisung implizit in den Typ des Pointers auf der linken Seite der Zuweisung gewandelt.

Hier wieder ein Beispiel:

calloc.c

```
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int* a = calloc(3, sizeof(int));
    if (a != NULL) {
        printf("Inhalt des Arrays: %d, %d, %d\n", a[0], a[1], a[2]);
        a[0] = 1;
        a[1] = 2;
        a[2] = 3;
        printf("Inhalt des Arrays: %d, %d, %d\n", a[0], a[1], a[2]);
        free(a);
    } else {
        printf("Nicht genug Speicher verfuegbar.\n");
    }
    return 0;
}
```

Hier die Ausgabe des Programms:

```
Inhalt des Arrays: 0, 0, 0
Inhalt des Arrays: 1, 2, 3
```

18.1.4 Die Funktion `realloc()`

Mit `realloc()` kann man die Größe eines dynamischen Speicherbereichs ändern.



Syntax:

```
void* realloc(void* memblock, size_t size);
```

Um einen mit `malloc()` oder `calloc()` erzeugten dynamischen Speicherbereich auch nachträglich noch in seiner Größe ändern zu können, existiert die Funktion `realloc()`. Diese Funktion bekommt einen Pointer auf einen bereits existierenden dynamischen Speicherbereich übergeben und zusätzlich noch die Größe des gewünschten neuen Speicherbereiches.

Ist die angeforderte neue Anzahl Bytes größer wie bisher, so probiert die Funktion, den angeforderten Speicher noch an die bisherige Adresse anzuhängen. Ist dies nicht möglich, dann verschiebt `realloc()` den vorhandenen Speicherbereich an eine Stelle im Speicher, an der noch genügend Speicher frei ist.

Die Funktion `realloc()` kann aufgrund der Verschiebung von Speicher sehr aufwändig sein.



Ist die angeforderte neue Anzahl Bytes kleiner wie bisher, so wird der Speicherbereich nach der gewünschten Anzahl freigegeben. Dieser Speicherbereich darf danach ausgehend von der Adresse in `memblock` nicht mehr angesprochen werden.

Speicher, welcher aufgrund von `realloc()` freigegeben wird, wird genauso wie bei `free()` nicht initialisiert (siehe Kapitel [→ 18.2](#)). Der Inhalt bleibt bestehen.



Die Funktion `realloc()` gibt einen Pointer auf den reservierten Speicherbereich zurück, wenn die Speicherreservierung durchgeführt werden konnte, ansonsten wird `NULL` zurückgegeben. Der Pointer ist vom Typ `void*` und wird bei einer Zuweisung implizit in den Typ des Pointers auf der linken Seite gewandelt.

Übergibt man an `realloc()` den `NULL`-Pointer als Adresse auf einen vorhandenen dynamischen Speicherbereich, dann arbeitet `realloc()` wie `malloc()` und gibt einen Pointer auf einen neu erstellten dynamischen Speicherbereich zurück.

Hier wieder ein Beispiel:

realloc.c

```
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int* a = malloc(sizeof(int));
    *a = 3;
    printf("Inhalt nach malloc(): %d\n", *a);

    a = realloc(a, 2 * (sizeof(int)));
    a[1] = 6;
    printf("Inhalt nach realloc(): %d, %d\n", a[0], a[1]);

    free(a);
    return 0;
}
```

Hier die Ausgabe des Programms:

```
Inhalt nach malloc(): 3
Inhalt nach realloc(): 3, 6
```

Man beachte, dass in diesem Beispiel der Übersichtlichkeit wegen auf eine Prüfung des zurückgegebenen Pointers auf `NULL` weggelassen wurde.

18.2 Freigabe von Speicher mittels free()

Mit **free()** kann man einen dynamischen Speicherbereich freigeben.



Syntax:

```
void free(void* pointer);
```

In C muss man den nicht mehr benötigten Speicher, der mit den Funktionen `malloc()`, `calloc()` oder `realloc()` beschafft wurde, mit Hilfe der Funktion `free()` wieder freigeben. Der entsprechende Speicherbereich kann dann von der Heapverwaltung erneut vergeben werden.

Wenn der Funktion `free()` ein NULL-Pointer übergeben wird, passiert nichts.



Der formale Parameter von `free()` ist vom Typ `void*`. Damit kann ein Pointer auf einen beliebigen Datentyp übergeben werden. Die Funktion `free()` ist mit dem Pointer, den `malloc()`, `calloc()` oder `realloc()` geliefert hat, aufzurufen.

Wird die Funktion `free()` mit einem Pointer aufgerufen, der nicht zuvor mittels `malloc()` oder einer ähnlichen Funktion reserviert wurde, so ist das Ergebnis undefiniert.



Nach dem Aufruf von `free()` darf über den Pointer nicht mehr auf das Speicherobjekt zugegriffen werden, da er auf kein gültiges Speicherobjekt mehr zeigt.



Es ist aber selbstverständlich möglich, erneut `malloc()`, `calloc()` oder `realloc()` aufzurufen und den Rückgabewert von `malloc()`, `calloc()` oder `realloc()` dem vorher schon benutzten Pointer wieder zuzuweisen.

Das Löschen eines Speicherbereiches mittels free() bewirkt nicht das Löschen der Inhalte desselben. Sollte beispielsweise ein späterer Aufruf von malloc() wieder auf dieselbe Adresse zeigen, so sind die vorherigen Inhalte immer noch abrufbar.



Wenn reservierter Speicher nicht mehr benötigt wird, es jedoch vergessen geht, ihn mittels free() wieder freizugeben, so bleibt dieser Speicherbereich bis zum Programmende unbrauchbar. Man spricht dann von einem sogenannten „memory leak“, was im nächsten Unterkapitel noch genauer erläutert wird.

Beispiel:

free.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(void) {
    int* intPointer = malloc(sizeof(int));

    *intPointer = 42;
    printf("Inhalt der Variablen: %d\n", *intPointer);

    // Memory Leak:
    intPointer = malloc(sizeof(int));
    // Nicht initialisierter Wert:
    printf("Inhalt der Variablen: %d\n", *intPointer);

    *intPointer = 101;
    printf("Inhalt der Variablen: %d\n", *intPointer);

    free(intPointer);
    return 0;
}
```

Hier die Ausgabe des Programms:

```
Inhalt der Variablen: 42
Inhalt der Variablen: 0
Inhalt der Variablen: 101
```


Hier wird der Pointer vor Ausgabe der zweiten Zeile überschrieben, ohne dass der alte Pointer freigegeben wird. Der alte Wert ist somit verloren. Gleichzeitig wird der neu zugewiesene Speicherplatz nicht initialisiert. Das Resultat der zweiten Zeile könnte somit auch komplett anders aussehen, denn der Inhalt der Variablen ist zu diesem Zeitpunkt unbestimmt, sprich es könnten irgendwelche Werte ausgegeben werden.

18.2.1 Speicherlecks

Wenn nicht daran gedacht wird, beanspruchten Speicher wieder freizugeben, entstehen sogenannte **Speicherlecks**, auf Englisch „**memory leaks**“ genannt.

Während der Laufzeit eines Programmes bleiben diese Speicherbereiche unerreichbar. Erst beim Programmende erfolgt automatisch die Freigabe von jeglichem noch reserviertem Speicher durch das Betriebssystem.

Bei kleineren Systeme, die ja gerade oft in C implementiert werden, wie zum Beispiel Robotersteuerungen, müssen Speicherlecks zwingend vermieden werden. Dort können aufgrund der knappen Ressourcen mit der Zeit sich die unbrauchbaren Speicherbereiche aufkumulieren und das System zum Erliegen bringen. Der Speicher kann nur durch einen sogenannten „Kaltstart“ des Gerätes wieder vollständig zurückgesetzt werden. Auch Programme auf großen Betriebssystemen können sich zu wahren Speicherfressern entwickeln, wenn nicht daran gedacht wird, angeforderten, aber mittlerweile nicht mehr benötigten Speicher wieder freizugeben.

Memory leaks treten normalerweise auf, wenn der Aufruf der Funktion free() schlicht vergessen geht. Sie können aber beispielsweise auch passieren, wenn Pointer-Variablen wiederverwendet werden, sprich der letzte Pointer, welcher die Adresse des reservierten Speicherbereiches speichert, überschrieben wird, ohne vorher den Speicher freizugeben.

In der markierten Zeile des Beispiels in Kapitel → 18.2 passiert genau das: Die Variable `intPointer` speichert eine noch gültige Adresse, wird jedoch durch die erneute Zuweisung mittels `malloc()` überschrieben. Das bisherige Speichersegment ist somit nicht mehr adressierbar und bleibt bis zum Programmende ungenutzt.

Bei Sprachen, die einen Garbage Collector vorsehen, werden solche nicht referenzierten Speicherobjekte vom Garbage Collector ermittelt und dem zur Verfügung stehenden freien Speicher zugeordnet.

Speicheranforderung und Speicherfreigabe verhalten sich wie eine öffnende und eine schließende Klammer, die stets paarig sein müssen. Während Compiler unpaarige Klammern entdecken können, hängen unpaarige Speicheranforderungen und Speicherfreigaben vom jeweiligen Programmablauf ab und können daher nicht von einem Compiler gefunden werden.

Man sollte es sich zur Angewohnheit machen, bei jedem `malloc()` beziehungsweise bei jedem `calloc()` sofort zu planen, wann und wo der angeforderte Speicher wieder freigegeben wird. Oft ist es nicht möglich, dies sofort zu codieren. Dann muss man gewissenhaft eine ToDo-Liste führen und ToDo-Kommentare in den Code einfügen, damit nicht mit der Zeit immer mehr potentielle Speicherlöcher entstehen, die äußerst schwer im Nachhinein zu entdecken und zu beseitigen sind.

18.3 Dynamisch erzeugte Arrays

In der Praxis ist zur Compilierzeit in den meisten Fällen nicht bekannt, wie viele Objekte einer Struktur gebraucht werden. Auch die Länge von Strings ist von vornherein meist schwer zu schätzen. Manchmal wird der Ansatz genommen, einfach zur Compile-Zeit ein sehr großes Array mit beispielsweise 10000 Komponenten anzulegen und darauf zu hoffen, dass dies für alle Zeiten genug ist.

Dies führt jedoch zu ineffizienten Programmen, deren Laufzeitverhalten und Speicherbedarf zu wünschen übrig lässt. In diesem Abschnitt soll daher gezeigt werden, wie man zur Laufzeit ein Array dynamisch auf dem Heap erzeugt.

Mit `malloc()` oder `calloc()` kann zur Laufzeit ein **dynamisches Array** einer aktuell berechneten Größe angelegt werden.



An dieser Stelle ein einfaches Beispiel:

einArray.c

```
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int anzahl = 3;
    int* ptrArray;

    ptrArray = malloc(anzahl * sizeof(int));
    for (size_t index = 0; index < anzahl; ++index) {
        ptrArray[index] = (int)index;
        printf("malloc Array[%d]: %d\n", (int)index, ptrArray[index]);
    }
    free(ptrArray);

    ptrArray = calloc(anzahl, sizeof(int));
    for (size_t index = 0; index < anzahl; ++index) {
        ptrArray[index] = (int)index;
        printf("calloc Array[%d]: %d\n", (int)index, ptrArray[index]);
    }
    free(ptrArray);

    return 0;
}
```

Hier eine mögliche Ausgabe des Programms:

```
malloc Array[0]: 0
malloc Array[1]: 1
malloc Array[2]: 2
calloc Array[0]: 0
calloc Array[1]: 1
calloc Array[2]: 2
```

In diesem Beispiel wurde erneut auf die Prüfung des Rückgabewertes von `malloc()` beziehungsweise `calloc()` verzichtet. Heutige Betriebssysteme ermöglichen Applikationen ohne große Mühen die Nutzung von Megabytes und Gigabytes an Speicher, weswegen ein Test auf fehlenden Speicher häufig vergessen wird. Wenn heutzutage die Funktion `malloc()` jemals fehlschlägt, dann hat das Programm sehr wahrscheinlich ein anderes, ernsthafteres Problem, als dass angeblich zu wenig Speicherplatz zur Verfügung stünde. Auf älteren oder kleineren Geräten jedoch sollte dieser Test nicht fehlen.

18.4 Übungsaufgaben

Aufgabe 1: Speichern eines Datenobjektes

Betrachten Sie folgende Definition:

```
typedef struct Person Person;  
struct Person {  
    char name[20];  
    int alter;  
};
```

- a) Schreiben Sie eine Funktion, welche den Inhalt einer solchen Struktur auf dem Bildschirm ausgeben kann. Die Funktion soll als einzigen Parameter einen Pointer auf die Struktur erhalten.

```
void printPerson(const Person* p);
```

- b) Schreiben Sie eine Funktion mit den Parametern `name` und `alter`, welche genügend Speicher für eine solche Struktur reserviert, sie mit den Werten korrekt befüllt und die Adresse zurückgibt. Nutzen Sie die Funktion `strncpy()`, um den Namen zu kopieren.

```
Person* allocatePerson(const char* name, int alter);
```

- c) Erstellen Sie ein Programm, welches mehrere dieser Strukturen anhand der gegebenen Funktionen erstellt und sie auf dem Bildschirm ausgibt. Danach soll der Speicher wieder gelöscht werden.
- d) Ändern Sie die Deklaration des Feldes `name` von `char[20]` auf den Typ `char*`. Stellen Sie sicher, dass beim Erstellen der Struktur genügend Speicherplatz reserviert wird, um den gesamten Namen zu speichern. Wie löschen Sie diesen Speicher wieder?
- e) Erstellen Sie eine Funktion, die alleinig dafür zuständig ist, eine als Pointer gegebene Struktur `Person` wieder geordnet aus dem Speicher zu löschen.

```
void deallocatePerson(Person* p);
```

- f) Schreiben sie zusätzlich eine Funktion, welche einen Pointer auf eine solche Struktur erwartet und sie mit den zusätzlichen Parametern name und alter befüllt. Rufen Sie diese Funktion von allocatePerson() aus auf.

```
void initPerson(Person* p, const char* name, int alter);
```

- g) Erstellen Sie ein ganzes Array von solchen Strukturen als Pointer:

```
Person* persons[COUNT];
```

Ändern Sie alle Funktionen so ab, dass mittels for-Schleifen die Allokation, das Ausdrucken und die Deallokation stattfindet.

Aufgabe 2: Eigener String mit realloc()

Betrachten Sie folgende Struktur:

```
typedef struct MyString MyString;
struct MyString {
    char* ptr;
    size_t byteSize;
};
```

- a) Schreiben Sie eine Funktion, welche eine solche Struktur zurückgibt, bei welcher der Pointer ptr auf einen reservierten Speicherblock zeigt mit der gewünschten Byteanzahl und dem gegebenen String.

```
MyString makeString(const char* str);
```

Vergessen Sie nicht das abschließende Null-Byte! Beachten Sie auch, dass hier nicht ein Pointer zurückgegeben wird.

Geben Sie den Inhalt des Pointers zur Kontrolle auf dem Bildschirm aus.

- b) Schreiben Sie eine weitere Funktion, welche eine solche Struktur als Pointer erwartet und sie so verändert, dass sie nach der Funktion die gewünschte neue Anzahl an Bytes speichern kann. Der Inhalt des Pointers ptr soll dabei so gut es geht erhalten bleiben. Nutzen Sie dafür die Funktion realloc().

```
void enlargeString(MyString* string, size_t byteSize);
```

- c) Schreiben genau dieselbe Funktion nochmals, aber ohne die Funktion realloc().

19 Dynamische Datenstrukturen



Um Daten geordnet im Speicher zu halten, werden in der Programmiersprache C die Strukturen (Kapitel → 13) verwendet. Kombiniert man diese mit der dynamischen Speicherzuweisung (Kapitel → 18), so ist es damit möglich, beliebig viel Speicher für beliebige Datenblöcke zu reservieren.

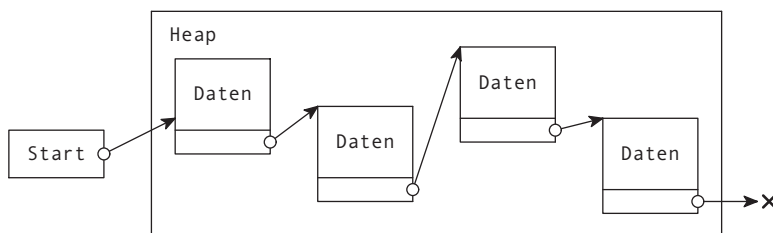
Das Ansprechen solcher Speicherblöcke erfolgt über eine Adresse. Um somit Strukturen mittels dynamischer Speicherzuweisung zu verwalten, müssen insbesondere die Pointer zu diesen Blöcken gespeichert und verwaltet werden.

Bei sehr vielen Datenblöcken kann es dabei schwierig werden, den Überblick über alle gespeicherten Datenblöcke zu bewahren. Aus diesem Grund gibt es die sogenannten „**dynamischen Datenstrukturen**“, welche die Aufgabe haben, die dynamischen Speicherblöcke in Relation zueinander geordnet abzuliegen.

In diesem Kapitel werden insbesondere zwei dieser dynamischen Datenstrukturen vorgestellt: die verkettete Liste und die Baumstruktur. Beide Datenstrukturen erlauben es, je nach Bedarf Datenblöcke geordnet hinzuzufügen, sie zu durchsuchen, oder wieder wegzunehmen.

19.1 Verkettete Listen

Bei einer verketteten **Liste** stehen die **Elemente** nicht wie bei einem Array sequenziell hintereinander, sondern können beliebig im Heap verstreut sein. Da somit nicht sichergestellt ist, dass die Daten, welche auf dem Heap abgelegt werden, immer hintereinander stehen, muss man zum Referenzieren der Elemente entweder einen Pointer auf jedes Element anlegen (beispielsweise mittels eines Pointer-Arrays) oder aber die einzelnen Elemente so miteinander über Pointer verknüpfen, dass jeweils ein Element auf das nächste zeigt und es möglich ist, über einen Pointer auf ein Ausgangselement und über die Verknüpfungen zwischen den Elementen jedes einzelne Element wiederzufinden. Bei dieser zweiten Art der Ablage spricht man von einer verketteten Liste.



Ergänzende Information Die elektronische Version dieses Kapitels enthält Zusatzmaterial, auf das über folgenden Link zugegriffen werden kann https://doi.org/10.1007/978-3-658-45209-4_19.

Eine verkettete Liste besteht aus gleichartigen Datenelementen. Um von einem Listenelement zum nächsten zu finden, muss ein Listenelement einen Pointer auf das nächste Listenelement besitzen. Ein Listenelement besteht aus einem Datenteil mit Nutzdaten und einem Pointer, der zur Verkettung der Listenelemente dient.



Die Listenelemente werden im Heap angelegt. Sie können an beliebiger Stelle im Heap liegen. Wird ein Element in der Liste gesucht, so muss von einem Start-Pointer, der auf das erste Element der Liste zeigt, solange von einem Element zum nächsten gegangen werden, bis entweder das letzte Element der Liste erreicht oder das gesuchte Element gefunden ist.

19.1.1 Datentyp eines Listenelements

Ein Listenelement wird als eine Struktur definiert, wie beispielsweise

```
struct Element {  
    float f;  
    struct Element* next;  
};
```

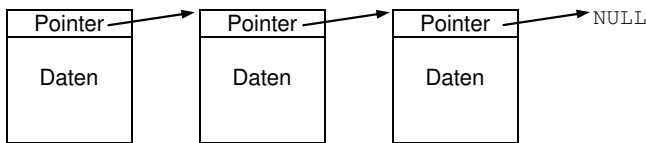
Man beachte, dass, obwohl der Typ `struct Element` noch nicht definiert ist, er innerhalb der Struktur bereits benutzt werden kann (`struct Element* next`). Dies ist möglich, da – ähnlich zur Vorwärtsdeklaration von Funktionen – der Compiler den Namen der Struktur bereits mit der Deklaration `struct Element` registriert. Siehe dazu Kapitel [→ 13.1.4](#)

19.1.2 Einfach verkettete Liste

Die einfachste Form einer verketteten Liste ist die Einfachverkettung. Eine einfache Verkettung besteht darin, dass jedes Element der Liste immer einen Nachfolger hat und auf diesen mit einem Pointer verweist.



Im Folgenden werden Elemente einer verketteten Liste symbolisiert:

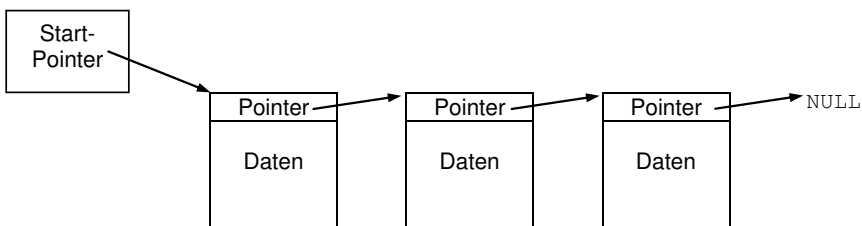


Jedes dieser Kästchen bezeichnet einen einzelnen Datenblock, welcher beispielsweise mittels der Funktion `malloc()` (siehe Kapitel [→ 18.1.1](#)) irgendwo im Heap reserviert wurde. Wo genau die einzelnen Datenblöcke stehen, ist irrelevant, da jedes Kästchen über seinen Pointer-Eintrag festhält, wo sich der nächste Datenblock befindet.

Das letzte Element hat keinen Nachfolger und bezeichnet somit das Ende der Liste. Um das Ende der Liste zu definieren, wird häufig der Pointer des letzten Elements auf NULL gesetzt.



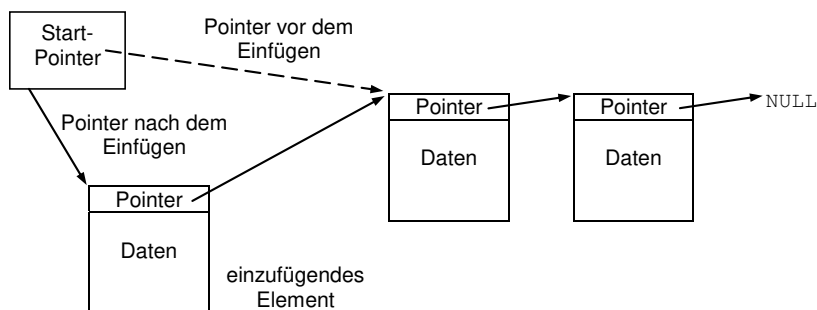
Um die gesamte Liste im Heap zu finden, muss jetzt lediglich ein Pointer auf das erste Element der Liste angelegt werden.



Um mit der Liste arbeiten zu können, werden insbesondere zwei Aktionen benötigt: Das Einfügen eines neuen Elements und das Entfernen eines Elements.

19.1.3 Neues Listenelement einfügen

Das folgende Bild zeigt das Einfügen eines Elements in eine einfach verkettete Liste:



Hier wird das neue Element am Anfang der Liste eingefügt: Dem Pointer des neu einzutragenden Listenelements wird die Adresse des bisher ersten Listenelements zugewiesen und dem Startpointer wird danach die Adresse des neuen Listenelements zugewiesen.

Ein neues Listenelement kann grundsätzlich an einer beliebigen Stelle in die bestehende Liste eingefügt werden. Ist jedoch die Adresse des Listenelements, an welchem das Einfügen passieren soll, nicht bekannt, so muss es zuerst innerhalb der Liste gefunden werden.

Das Suchen nach einem bestimmten Element innerhalb der Liste funktioniert so, dass man mit dem ersten Element der Liste beginnt und daraufhin jedes Element einzeln abarbeitet (möglicherweise bis zum letzten Element), bis das gewünschte Element gefunden wurde. Eine Suche auf einer einfach verketteten Liste hat somit kein gutes Laufzeitverhalten.

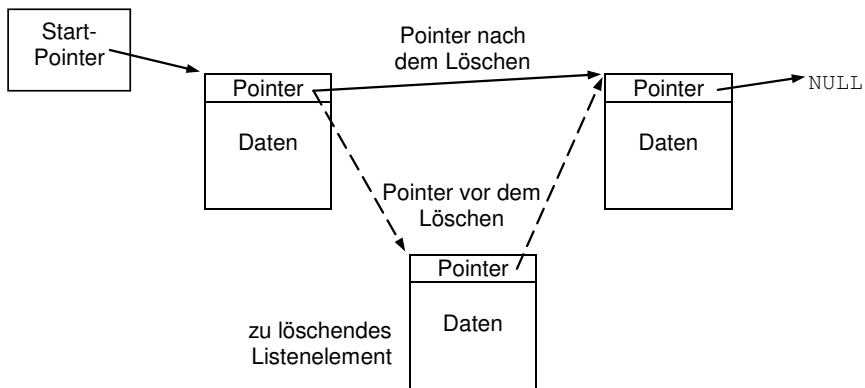
Operationen auf Elementen von einfach verketteten Listen, zu welchen die Adresse nicht bereits bekannt ist, sind sehr zeitaufwendig.



Siehe dazu auch Kapitel [→ 20](#) über Suchen und Sortieren.

19.1.4 Listenelement entfernen

Um ein Element aus einer einfach verketteten Liste zu entfernen, muss im ersten Schritt das entsprechende Element gesucht werden. Dann wird der Pointer des Vorgängers des zu entfernenden Elementes auf den Nachfolger des zu entfernenden Elementes gesetzt. Falls zudem das erste Element der Liste entfernt wird, muss der Start-Pointer berichtigt werden.



Es ist zu beachten, dass nach diesem Vorgang das Element zwar aus der Liste entfernt wurde, es jedoch dennoch im Heap Speicher belegt. Soll das Element nicht nur entfernt, sondern auch gelöscht werden, so muss das Element nach diesem Vorgang somit manuell gelöscht werden, beispielsweise mittels `free()`.

19.1.5 Programmbeispiel für eine einfach verkettete Liste

Hier ein Programmbeispiel für eine einfach verkettete Liste. Jedes Listenelement speichert einen Messwert und den Namen des entsprechenden Sensors.

liste.c

```
#include <stdlib.h>
#include <stdio.h>
#include <string.h>

struct Messdaten {
    struct Messdaten* next;
    char    sensorname[30];
    double messwert;
};

struct Messdaten* startPointer = NULL;

struct Messdaten* allocMessdaten(const char* name, double messwert) {
    struct Messdaten* ptr = malloc(sizeof(struct Messdaten));
    strcpy(ptr->sensorname, name);

    ptr->messwert = messwert;
    ptr->next = NULL;

    return ptr;
}

void freeMessdaten(struct Messdaten* messdaten) {
    free(messdaten);
}

void elementHinzufuegen(struct Messdaten* messdaten) {
    messdaten->next = startPointer;
    startPointer = messdaten;
}

struct Messdaten* elementSuchen(const char* name) {
    struct Messdaten* ptr = startPointer;

    while (ptr != NULL && strcmp(ptr->sensorname, name)) {
        ptr = ptr->next;
    }

    return ptr;
}
```

```
struct Messdaten* elementEntfernen(struct Messdaten* messdaten) {
    struct Messdaten* ptr = startPointer;
    struct Messdaten* vorgaenger = NULL;

    while (ptr != messdaten) {
        vorgaenger = ptr;
        ptr = ptr->next;
    }
    if (ptr == NULL)
        return NULL;
    if (ptr == startPointer)
        startPointer = ptr->next;
    else
        vorgaenger->next = ptr->next;

    return ptr;
}

void elementeAusgeben(void) {
    struct Messdaten* ptr = startPointer;
    while (ptr != NULL) {
        printf("%s: %f\n", ptr->sensorname, ptr->messwert);
        ptr = ptr->next;
    }
    printf("\n");
}

int main(void) {
    elementHinzufuegen(allocMessdaten("Temperatur", 23.));
    elementHinzufuegen(allocMessdaten("Luftdruck", 1023.));
    elementHinzufuegen(allocMessdaten("Wasserpegel", 37.));
    elementeAusgeben();

    struct Messdaten* luftdruck = elementSuchen("Luftdruck");
    if (luftdruck)
        printf("Aktueller Luftdruck: %f\n\n", luftdruck->messwert);

    freeMessdaten(elementEntfernen(luftdruck));
    elementeAusgeben();

    while (startPointer) {
        freeMessdaten(elementEntfernen(startPointer));
    }

    return 0;
}
```

Die Funktion `main()` gibt als erstes die gesamte Liste aus, sucht danach nach dem Eintrag des Luftdrucks, gibt ihn als einzelnen Wert auf dem Bildschirm aus, löscht ihn aus der Liste und gibt die übrigbleibende Liste wieder aus.

Hier die Ausgabe des Programms:

```
Wasserpegel: 37.000000
Luftdruck: 1023.000000
Temperatur: 23.000000

Aktueller Luftdruck: 1023.000000

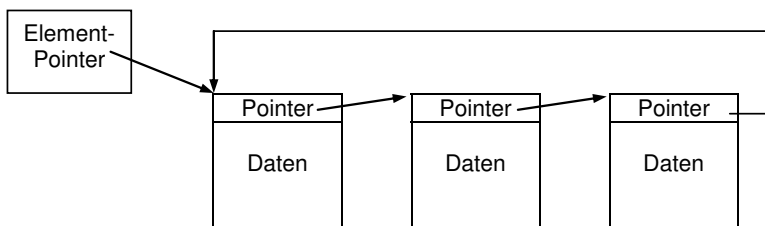
Wasserpegel: 37.000000
Temperatur: 23.000000
```

Beachten Sie, dass die eingegebenen Daten in umgekehrter Reihe ausgegeben werden, da die Elemente jeweils zu Beginn der Liste eingefügt werden.

19.1.6 Andere Listenarten

Neben einer einfach verketteten Liste gibt es noch verschiedene andere Listenarten wie beispielsweise Ringstrukturen und doppelt verkettete Listen.

Belegt man das letzte Element einer einfach verketteten Liste nicht mit dem Wert NULL, sondern verbindet dieses Element wieder mit dem Anfang der Liste, so ergibt sich eine Ringstruktur.



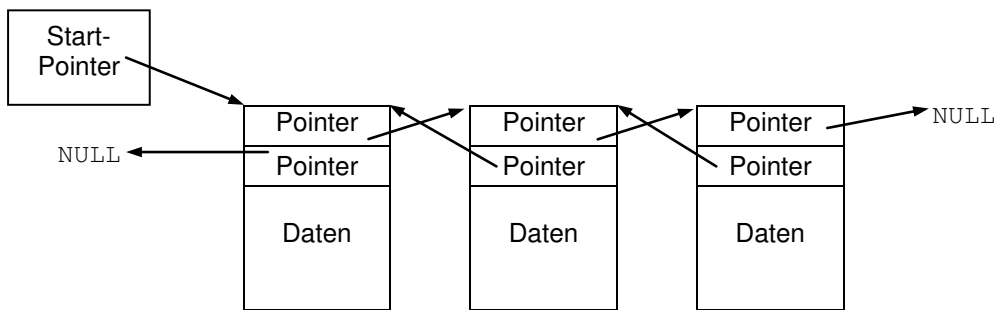
Um auf die Elemente zuzugreifen, muss ein Pointer auf ein beliebiges Element der Liste vorhanden sein. Egal welches Element verwendet wird, von ihm aus werden alle anderen Elemente des Rings angesprochen.

Da kein „einfaches“ Abbruchkriterium wie der NULL-Pointer bei einer linearen verketteten Liste vorhanden ist, muss beim Durchsuchen einer Ringstruktur insbesondere darauf geachtet werden, dass keine Endlosschleife programmiert wird.



Eine weitere Art von Listen sind solche, durch welche sowohl vorwärts, als auch rückwärts gegangen werden kann.

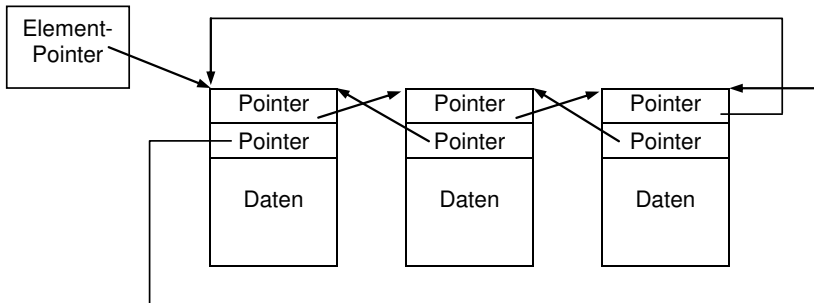
Speichert man in einem Listenelement außer der Adresse des Nachfolgers auch noch die Adresse des Vorgängers, so spricht man von einer doppelt verketteten Liste.



Dies kann für einige Anwendungen günstig sein. Insbesondere ist das Entfernen einfacher, da ein zu entfernendes Listenelement bereits den Pointer auf seinen Vorgänger und seinen Nachfolger enthält. Im Beispielprogramm `liste.c` wird damit in der Funktion `elementLoeschen()` die Suche nach dem Vorgänger überflüssig.

Um eine doppelt verkettete Liste rückwärts vom letzten Element aus zu durchlaufen, kann zusätzlich das letzte Element gespeichert werden, was dann allgemein als End-Pointer bezeichnet wird. Start- und End-Pointer werden häufig in einer Struktur, dem sogenannten „Listenkopf“, gespeichert.

Eine doppelt verkettete Liste kann aber auch zu einer Ringstruktur angeordnet werden:



Der Vorteil einer solchen Ringstruktur ist, dass man mittels des Ansprechens des Rückwärtspointers des ersten Elements mit nur einem Schritt auf das letzte Element zugreifen kann. So kann man selbst ohne Listenkopf vor- als auch rückwärts in der Liste suchen.

Auf ein Beispiel zu einer doppelt verketteten Liste wird an dieser Stelle verzichtet. Es wird jedoch in einer Übungsaufgabe darauf eingegangen. Es sei darauf hingewiesen, dass die Aktionen Einfügen und Entfernen mit Sorgfalt zu programmieren sind. Eine Handskizze der einzelnen Listenelemente hilft bei der Pointervielfalt oft weiter.

Als letzte Anmerkung soll noch die Idee des sogenannten „**Sentinels**“ angesprochen werden. Bei einer doppelt verketteten Liste, welche zu einer Ringstruktur verknüpft ist, kann man ein spezielles „nulltes“ Element definieren, das Sentinel. Dieses speichert als Vorwärts- und Rückwärts-Pointer einen Pointer auf das erste, beziehungsweise das letzte Element der Liste. Als Daten wird ein NULL-Pointer gespeichert. Anders gesagt, speichert man im „nullten“ Element den Listenkopf der Liste.

Mittels eines solchen Sentinels ist es möglich, eine Implementation zu generieren, welche fast gänzlich ohne Spezialfälle (sprich `if`-Strukturen) auskommt. Auch hierfür wird auf eine Übungsaufgabe am Ende dieses Kapitels verwiesen.

19.2 Baumstrukturen

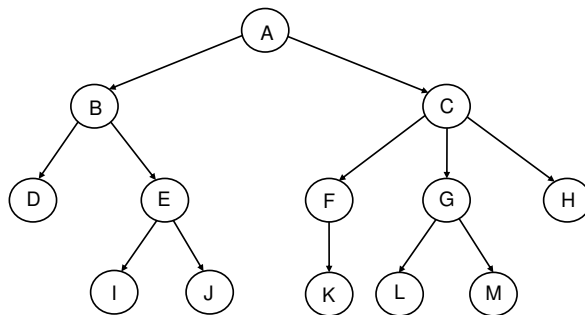
Ein bekanntes Beispiel für **Bäume** aus dem Alltag ist die Evolutionstheorie nach Darwin, wo jede Tiergattung evolutionsgeschichtlich aus einer anderen Gattung hervorging und gleichzeitig mehrere Untergattungen hervorbringen kann. Ein anderes Beispiel ist das Organigramm einer großen Firma, wo zuoberst der Chef als Wurzelknoten steht, darunter die Filialen, darunter die Abteilungen, darunter die Gruppen, und ganz zuunterst die Mitarbeiter.

Bäume als Datenstrukturen können sehr gut zum effizienten Abspeichern und Suchen mit Hilfe eines Schlüsselwertes eingesetzt werden.



19.2.1 Allgemeine Darstellung von Baumstrukturen

Zur Darstellung von Baumstrukturen gibt es verschiedene grafische und alphanumerische Möglichkeiten. In folgenden Bild wird ein Baum durch einen Graphen aus **Knoten** und **Kanten** dargestellt:



Die Darstellung der Knoten ist nicht genormt. Üblicherweise werden sie kreisförmig oder rechteckig dargestellt.

Wie man leicht in diesem Bild sieht, stehen in der Informatik die Bäume auf dem Kopf.

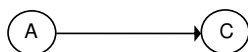
Der oberste Knoten heißt „**Wurzel**“. Die Knoten ohne Nachfolger bilden die „Blätter“ (**Blattknoten**) des Baumes. Alle anderen Knoten werden als „**innere Knoten**“ bezeichnet.



Im obigen Beispiel ist A die Wurzel, Knoten B, E, F, C und G sind innere Knoten und Knoten D, I, J, K, L, M und H sind die Blätter.

19.2.2 Formale Definition eines Baumes

Ein Baum ist ein sogenannt gerichteter, azyklischer Graph, auf Englisch auch DAG genannt (directed acyclic graph). Gerichtet bedeutet, dass die Verbindung zwischen Knoten stets von einem sogenannten „Elternknoten“ (näher an der Wurzel) zu einem „Kinds-knoten“ (näher bei den Blättern) zeigt. Im Beispiel ist A der Elternknoten vom Kind C.



In den üblichen Baumdarstellungen steht ein Elternknoten stets höher als das Kind. So wird damit implizit die Richtung der Kanten ausgedrückt und die Pfeilspitze an den Kanten kann dann der Einfachheit halber auch weggelassen werden.

Formal kann ein Baum folgendermaßen definiert werden:

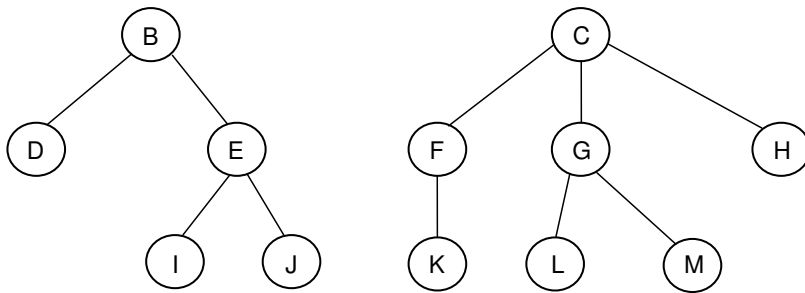
- Ein Baum besteht aus einer endlichen Anzahl Knoten, die durch gerichtete Kanten verbunden sind.
- Es gibt nur einen einzigen Knoten, der keine Beziehung zu einem Elternknoten hat. Dieser Knoten stellt die Wurzel dar.
- Alle übrigen Knoten haben genau einen Elternknoten.
- Ein Knoten kann eine beliebige endliche Zahl an Kinds-knoten haben.



Aus diesen Regeln folgt zudem: Es darf keine disjunkten Teile geben, das heißt alle Knoten müssen untereinander verknüpft sein. Es darf also kein loses Teil dabei sein.

Knoten, die denselben Elternknoten haben, werden als Geschwister bezeichnet.

Entfernt man die Wurzel eines Baumes, beispielsweise die Wurzel im vorangehenden Bild, so werden die Nachfolgerknoten zu den Wurzeln der entsprechenden Teilbäume (Unterbäume):



Diese Teilbäume sind disjunkt, bilden also insgesamt keinen gemeinsamen Baum, sind jedoch für sich selbst genommen jeweils wiederum eigenständige Bäume.

Ein Baum wird als „**geordneter Baum**“ bezeichnet, wenn die Reihenfolge der Knoten einer vorgeschriebenen Ordnung unterliegt.



In diesem Beispiel sind alle Knoten in alphabetischer Reihenfolge von oben nach unten und dann von links nach rechts sortiert.

Bei geordneten Bäumen spielt die Reihenfolge der Teilbäume eine wichtige Rolle. Würde man die Positionen der Teilbäume vertauschen, so würde man den Informationsgehalt des Baumes unzulässigerweise manipulieren.

Man kann den Begriff eines Baumes erweitern, wenn man auch leere Knoten im Baum zulässt. Leere Knoten sind Platzhalter, die bei Bedarf echte Knoten aufnehmen können.

Als spezieller Baum ergibt sich der leere Baum, der aus einer leeren Wurzel besteht und keinen einzigen echten Knoten hat.



19.2.3 Binäre Bäume

Binäre Bäume sind Bäume, für die gilt:

- Alle Knoten, die nicht leer sind, besitzen zwei Kinder.
- Maximal ein Kind eines Elternknotens kann leer sein.

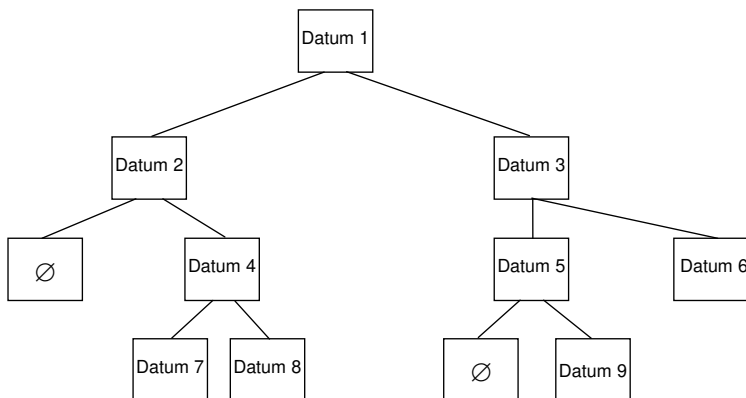
Rekursiv kann ein binärer Baum folgendermaßen definiert werden:

Ein binärer Baum

- ist entweder leer (leerer Baum)
- oder besteht aus einem einzigen Knoten
- oder besteht aus einer Wurzel, die einen linken und einen rechten binären Unterbaum hat, die beide nicht zugleich leere Bäume sein dürfen.



Das folgende Bild gibt ein Beispiel für einen binären Baum:



Die im Bild dargestellten Knoten mit dem Symbol $\emptyset$ stellen leere Knoten dar.

Binäre Bäume werden sehr häufig als geordnete Bäume programmiert. Dadurch ist – wie oben bereits angesprochen – die Reihenfolge der Teilbäume wichtig. Man kann in der Zeichnung die leeren Knoten weglassen, muss dann aber sicherstellen, dass aus der Anordnung der Knoten eindeutig hervorgeht, ob der rechte oder der linke Knoten leer ist.

Geordnete binäre Bäume sind in der Informatik allgegenwärtig. Beispielsweise eignen sie sich sehr gut zur Speicherung von Ausdrücken mit zweistelligen Operatoren, wobei jeder Operator einen Knoten mit den zugehörigen Operanden als Teilbäume darstellt. Weiterhin werden sie bei lexikalischen Sortierverfahren eingesetzt. In diesem Falle spricht man auch von binären Suchbäumen.

19.2.4 Binäre Suchbäume

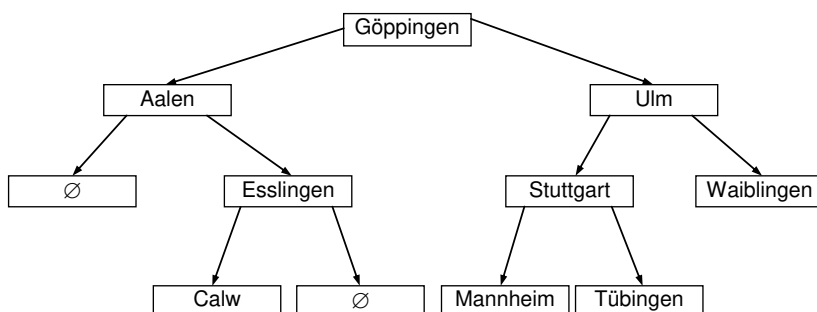
Binäre Suchbäume speichern eine Menge von Daten mit einem sogenannten „Schlüsselwert“, auf Englisch auch „key value“ genannt. Die Knoten werden so im Baum abgelegt, dass die Schlüsselwerte einer gegebenen Sortierung folgen. Dies wird beispielsweise folgendermaßen definiert:

Der Schlüsselwert des linken Teilbaums ist stets kleiner als der Schlüsselwert der Wurzel und der Schlüsselwert des rechten Teilbaumes ist stets größer als der Schlüsselwert der Wurzel.



Dabei bezeichnet der Schlüsselwert eines Teilbaumes den Schlüsselwert des Wurzelknotens des Teilbaumes.

Folgendes Bild zeigt ein Beispiel für einen solchen binären **Suchbaum**:



In diesem Baum werden Schlüsselwerte in Form eines Names von Ortschaften abgelegt. Wie man leicht nachprüfen kann, wurde der Baum geordnet aufgebaut. Die linken Teilbäume enthalten dabei Informationen, deren Schlüsselwerte kleiner als die Schlüsselwerte der Informationen der rechten Teilbäume sind. Kleiner bezieht sich hierbei auf die alphabetische Ordnung.

Der Vorteil einer solchen Ordnung zeigt sich beim Suchen und Sortieren.

19.2.5 Suchen in binären Suchbäumen

Um ein Element in einem binären Suchbaum zu finden, wird nach dem gewünschten Schlüsselwert gesucht.

Man vergleicht als erstes den Suchbegriff mit dem Schlüsselwert der Wurzel. Ist der Suchbegriff kleiner, so sucht man im linken Teilbaum weiter, ist er gleich, so hat man den gesuchten Knoten gefunden, ist er größer, so sucht man im rechten Teilbaum weiter.

Nach dieser Vorschrift sucht man rekursiv weiter, bis der Knoten gefunden ist. Wird er nicht gefunden, so ist der Suchbegriff im Baum nicht enthalten.

Die Höhe des Baumes (die maximale Anzahl der Ebenen) entspricht dabei der maximalen Anzahl von Vergleichen. In dem Beispiel mit den Ortsnamen werden maximal 4 Zugriffe benötigt, obschon 9 Ortschaften eingetragen sind. Bei größeren Datenmengen ist die Zeitersparnis noch bedeutend größer. Knoten können somit in binären Bäumen sehr schnell gesucht werden (daher der Name Suchbaum).

19.2.6 Einfügen und Entfernen von Elementen in binären Suchbäumen



Um Elemente in einen binären Suchbaum einzufügen wird zuerst nach dem Schlüsselwert gesucht. Sobald ein Knoten erreicht wird, bei welchem es keinen Nachfolger mehr gibt, so wird ein neuer Knoten rechts oder links von diesem Knoten erzeugt.

Um einen Knoten zu entfernen, wird zuerst ganz einfach nach dem Schlüsselwert des Knotens gesucht. Sofern es sich beim gefundenen Knoten um ein Blatt handelt (Knoten ohne Kinder), so wird einfach dieser Knoten entfernt.

Falls der gefundene Knoten jedoch noch ein oder zwei Teilbäume hat, so muss in dem zu entfernenden Knoten der Inhalt eines anderen, geeigneten Knotens gespeichert werden. Dies wird beispielsweise so gelöst, indem der Inhalt des links-äußersten Blatt-Knotens des rechten Teilbaumes oder der Inhalt des rechts-äußersten Blatt-Knotens des linken Teilbaumes an die Stelle des zu entfernenden Knotens gesetzt wird. So kann sichergestellt werden, dass sämtliche Knoten der beiden Teilbäume erneut garantiert entweder alle kleiner oder alle größer als der neue Wurzelknoten sind.

19.2.7 Durchlaufen von Bäumen

Zum Durchlaufen allgemeiner Bäume gibt es zwei verschiedene Durchlaufalgorithmen: Die Präfix-Ordnung und die Postfix-Ordnung. Bei binären Bäumen kommt noch die Infix-Ordnung dazu.

Eine Durchlaufordnung bezeichnet die Reihenfolge, in welcher die Eltern- und Kindsknoten abgearbeitet werden.



Bei der Präfix-Ordnung lautet die Reihenfolge:

1. Zuerst die Wurzel,
2. Dann alle Unterbäume in Präfix-Ordnung

Bei der Postfix-Ordnung lautet die Reihenfolge:

1. Zuerst alle Unterbäume in Postfix-Ordnung,
2. Dann die Wurzel.

Präfix- und Postfix-Ordnung sind bei der Implementierung von Sprachumwandlern von Bedeutung. Diese beiden Ordnungen erlauben die Umwandlung arithmetischer Ausdrücke, die als binäre Bäume dargestellt sind, in klammerlose Darstellungen – nämlich in die sogenannte Polnische beziehungsweise in die Umgekehrte Polnische Notation.



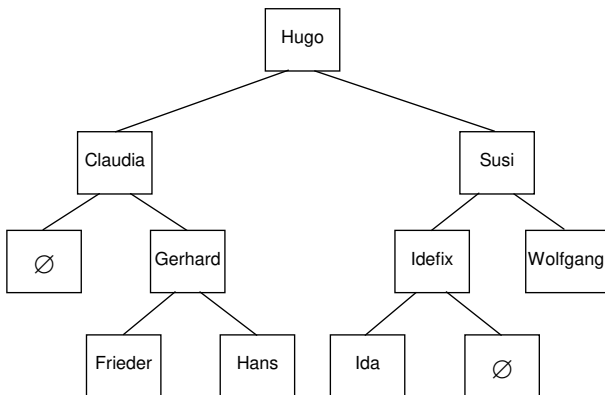
Bei binären Bäumen ist die Infix-Ordnung wichtig. Da alle Knoten eines binären Baumes zwei Kindsknoten besitzen, lautet die Reihenfolge:

1. Zuerst der linke Teilbaum in Infix-Ordnung,
2. Dann die Wurzel.
3. Dann der rechte Teilbaum in Infix-Ordnung.

Die Infix-Ordnung wird auch symmetrischer Durchlauf genannt.



Der folgende Baum soll in der durch diese drei verschiedenen Ordnungen festgelegten Reihenfolge durchlaufen werden.



Zur besseren Unterscheidung wird im Folgenden ein Knoten nur durch seinen Knotennamen und ein Unterbaum, der in einem Knoten beginnt, durch {Unterbaum Knotenname} dargestellt. Damit ergeben sich die folgenden Reihenfolgen für die Durchläufe durch die einzelnen Knoten:

Durchlauf in Präfix-Ordnung:

Hugo, {Unterbaum Claudia}, {Unterbaum Susi}
 = Hugo, Claudia, {Unterbaum Gerhard}, Susi, {Unterbaum Idefix}, Wolfgang
 = Hugo, Claudia, Gerhard, Frieder, Hans, Susi, Idefix, Ida, Wolfgang

Durchlauf in Postfix-Ordnung:

{Unterbaum Claudia}, {Unterbaum Susi}, Hugo
 = {Unterbaum Gerhard}, Claudia, {Unterbaum Idefix}, Wolfgang, Susi, Hugo
 = Frieder, Hans, Gerhard, Claudia, Ida, Idefix, Wolfgang, Susi, Hugo

Durchlauf in Infix-Ordnung:

{Unterbaum Claudia}, Hugo, {Unterbaum Susi}
 = Claudia, {Unterbaum Gerhard}, Hugo, {Unterbaum Idefix}, Susi, Wolfgang
 = Claudia, Frieder, Gerhard, Hans, Hugo, Ida, Idefix, Susi, Wolfgang

Druckt man also beispielsweise die Namen aller Knoten des Baumes aus, so kommt man je nach Durchlauf-Ordnung zu der soeben ermittelten Reihenfolge der Namen. Dabei ist zu beachten, dass die Infix-Ordnung aufgrund der Sortieranweisung des binären Suchbaumes automatisch zu einer alphabetischen Sortierung führt.

Würde man die Infix-Durchlaufordnung so ändern, dass zuerst der rechte und danach der linke Teilbaum durchlaufen wird, so erhält man die umgekehrte symmetrische Ordnung, sprich das Resultat entspricht einer umgekehrten alphabetischen Sortierung:

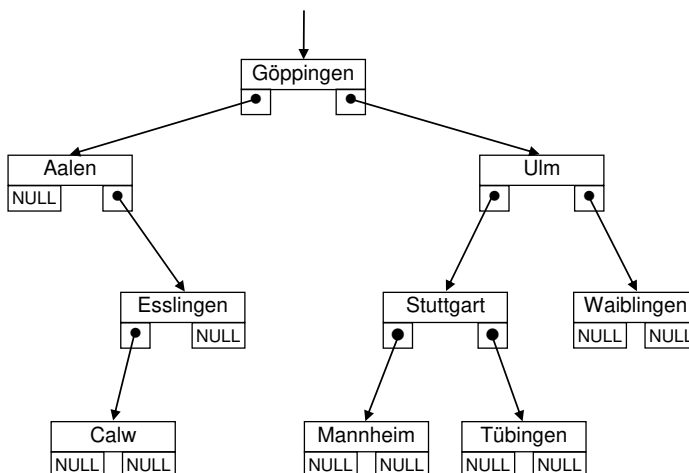
{Unterbaum Susi}, Hugo, {Unterbaum Claudia}
 = Wolfgang, Susi, {Unterbaum Idefix}, Hugo, {Unterbaum Gerhard}, Claudia
 = Wolfgang, Susi, Idefix, Ida, Hugo, Hans, Gerhard, Frieder, Claudia

Bei den hier vorgestellten Durchlaufalgorithmen läuft man zuerst tief die Äste des Baumes hinunter bis zu den Blättern. Man spricht deswegen auch von Tiefensuchverfahren (depth first search = DFS). Im Gegensatz dazu bearbeitet man bei dem Breitensuchverfahren (breadth first search = BFS) zuerst alle Knoten auf einer Ebene. Das bedeutet, man bearbeitet zuerst alle Kinder der Wurzel, dann alle Kinder dieser Kinder, usw.

Auf Algorithmen für Breitensuchverfahren und weitere Algorithmen für Tiefensuchverfahren kann an dieser Stelle nicht eingegangen werden.

19.2.8 Beispielprogramm für binäre Bäume

Im Folgenden wird ein binärer Baum mit Hilfe von Strukturen und Pointern auf Strukturen realisiert. Eine Struktur repräsentiert einen Knoten. Jede Struktur enthält zwei Pointer. Diese Pointer referenzieren dabei die Wurzel des linken beziehungsweise die Wurzel des rechten Teilbaums. Ist ein Teilbaum leer, so wird der NULL-Pointer als Pointer abgelegt. In einem Blatt wird folglich zweimal der NULL-Pointer abgespeichert.



Das folgende Programm baut einen binären Suchbaum auf, erlaubt es, einen Knoten im Baum zu suchen und druckt einen gesamten Teilbaum aus.

baum.c

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

struct Knoten {
    char str[30];
    int plz;
    struct Knoten* pointerLinks;
    struct Knoten* pointerRechts;
};

struct Knoten* wurzel = NULL;

struct Knoten* knotenEinfuegen(
    struct Knoten* ptr,
    const char* string,
    int plz)
{
    struct Knoten* neuPtr = NULL;

    if (ptr == NULL) {
        neuPtr = malloc(sizeof(struct Knoten));
        if (neuPtr != NULL) {
            strcpy(neuPtr->str, string) ;
            neuPtr->plz = plz;
            neuPtr->pointerLinks = NULL;
            neuPtr->pointerRechts = NULL;
        }
    } else if (strcmp(string, ptr->str) < 0) {
        neuPtr = knotenEinfuegen(ptr->pointerLinks, string, plz);
        if ((neuPtr != NULL) && (ptr->pointerLinks == NULL)) {
            ptr->pointerLinks = neuPtr;
        }
    } else if (strcmp(string, ptr->str) > 0) {
        neuPtr = knotenEinfuegen (ptr->pointerRechts, string, plz);
        if ((neuPtr != NULL) && (ptr->pointerRechts == NULL)) {
            ptr->pointerRechts = neuPtr;
        }
    } else {
        printf("Eintrag in Baum bereits vorhanden\n");
    }
}
```

```
    if (wurzel == NULL)
        wurzel = neuPtr;

    return neuPtr;
}

struct Knoten* suche(struct Knoten* ptr, const char* string) {
    if (ptr != NULL) {
        int comp = strcmp(string, ptr->str);
        if (comp < 0)
            return suche(ptr->pointerLinks, string);
        else if (comp > 0)
            return suche(ptr->pointerRechts, string);
        else
            return ptr;
    }
    return NULL;
}

void drucke(const struct Knoten* ptr) {
    if (ptr != NULL) {
        drucke(ptr->pointerLinks);
        printf("%s: %d\n", ptr->str, ptr->plz);
        drucke(ptr->pointerRechts);
    }
}

int main(void) {
    knotenEinfuegen(wurzel, "Goeppingen", 73033);
    knotenEinfuegen(wurzel, "Ulm", 89073);
    knotenEinfuegen(wurzel, "Aalen", 73430);
    knotenEinfuegen(wurzel, "Weiblingen", 71336);
    knotenEinfuegen(wurzel, "Stuttgart", 70173);
    knotenEinfuegen(wurzel, "Esslingen", 70771);
    knotenEinfuegen(wurzel, "Calw", 75365);
    knotenEinfuegen(wurzel, "Tuebingen", 72070);
    knotenEinfuegen(wurzel, "Mannheim", 68159);

    drucke(wurzel);
    printf("\n");

    struct Knoten* stuttgart = suche(wurzel, "Stuttgart");
    drucke(stuttgart);
    printf("\n");
    return 0;
}
```

Es ist zu beachten, dass das Löschen eines Elementes in diesem Beispielprogramm explizit weggelassen wurde. Es gibt hierfür eine Übungsaufgabe.

In der Funktion `main()` wird zuerst der gesamte Baum ausgegeben und danach der Teilbaum ab dem Ort „Stuttgart“. Die Ausgabe des Programmes lautet:

```
Aalen: 73430
Calw: 75365
Esslingen: 70771
Goepingen: 73033
Mannheim: 68159
Stuttgart: 70173
Tuebingen: 72070
Ulm: 89073
Weiblingen: 71336

Mannheim: 68159
Stuttgart: 70173
Tuebingen: 72070
```

Die Funktion `suche()` beginnt die Suche eines Knotens in der Wurzel und vergleicht den Suchstring mit dem in der Wurzel gespeicherten String. Ist der Suchstring größer als der dort gespeicherte String, so wird zum rechten Kind gegangen und der dort gespeicherte String wird mit dem Suchstring verglichen. Dieses Verfahren wird fortgesetzt, bis der Knoten gefunden ist, beziehungsweise bis der ganze Baum durchlaufen ist. Wird der ganze Baum durchlaufen und der Knoten ist nicht gefunden, so ist er auch nicht im Baum enthalten.

Der Funktion `knotenEinfuegen()` wird ein Pointer `ptr` auf einen Knoten und die in dem anzulegenden Knoten zu speichernden Daten übergeben. Ist der übergebene Pointer der NULL-Pointer, so wird auf ein leeres Blatt gezeigt. Deshalb kann an dieser Stelle der neue Knoten erzeugt und in den Baum eingefügt werden.

Ist der übergebene Pointer von NULL verschieden, so ist der Platz, auf den der Pointer zeigt, durch einen Knoten besetzt. Der einzufügende String wird mit dem String des vorhandenen Knotens verglichen. Ist der einzufügende String kleiner, so wird durch Aufruf von `knotenEinfuegen()` zum linken Kind des vorhandenen Knotens gegangen, wobei der Pointer `ptr->pointerLinks`, an `knotenEinfuegen()` übergeben wird. Ist der einzufügende String größer, so wird in entsprechender Weise zum rechten Kind gegangen. Die Funktion `knotenEinfuegen()` wird also rekursiv aufgerufen.

Ist nun das Kind ein leeres Blatt, so wird das rekursive Aufrufen beendet. Es wird nun ein Knoten angelegt und in ihm die übergebenen Daten abgespeichert. Durch die Beendigung der Rekursion wird rückwärts durch die unterbrochenen Funktionen gegangen. Die einzige Funktion, die dabei noch Anweisungen abarbeitet, ist die Funktion, die `knotenEinfuegen()` mit dem NULL-Pointer aufgerufen hat. Sie erhält als Rückgabewert dieser Funktion den Pointer auf den neuen Knoten und kann damit den neu geschaffenen Knoten mit dem bestehenden Baum verketten. Alle früher aufgerufenen Funktionen `knotenEinfuegen()` tun nichts weiteres, als den Pointer auf den neuen Knoten als Return-Wert an ihren Elternknoten durchzureichen.

19.3 Übungsaufgaben

Aufgabe 1: Einfach verkettete Liste

Betrachten Sie folgende Datenstruktur:

```
typedef struct Element Element;  
struct Element {  
    int x;  
};
```

- a) Schreiben Sie eine Funktion, welche ein Element im Speicher dynamisch reserviert und einen gegebenen int-Wert speichert.

```
Element* erstelleElement(int x);
```

- b) Erweitern Sie das Programm so, dass aus dieser Struktur eine einfach verkettete Liste aus Elementen erstellt werden kann. Speichern sie einen Pointer auf das Start-Element in einer globalen Variable. Schreiben Sie eine Funktion, um ein Element an den Anfang der Liste hinzuzufügen:

```
void hinzufuegenElement(Element* element);
```

- c) Schreiben Sie eine Ausgabe-Funktion, welche die Werte x einer Liste der Reihe nach ausgibt. Verwenden Sie dazu eine while-Schleife.

```
void ausgabeListe()
```

- d) Schreiben Sie eine Funktion, um ein Element ans Ende der Liste hinzuzufügen:

```
void hinzufuegenElementEnde(Element* element);
```

- e) Schreiben Sie eine zweite Ausgabe-Funktion, welche die Liste der Reihe nach ausgibt. Verwenden Sie diesmal dazu eine Rekursion.

```
void ausgabeListe2(Element* element)
```

- f) Schreiben Sie eine dritte Ausgabe-Funktion. Diesmal sollen die Werte jedoch in umgekehrter Reihenfolge ausgegeben werden. Nutzen sie dazu die Rekursion.

```
void ausgabeListe3(Element* element)
```

Aufgabe 2: Doppelt verkettete Liste

- a) Erweitern Sie die Struktur aus Aufgabe 1 so, dass sie nebst einem Pointer auf das nächste Element auch einen Pointer auf das vorherige Element besitzt.
- b) Schreiben Sie `hinzufuegenElement()` und `hinzufuegenElementEnde()` neu.
- c) Schreiben Sie eine neue Funktion, welche ein Element an einer beliebigen Stelle in der Liste **nach** einem gegebenen Element hinzufügt.

```
void hinzufuegenElementNach(  
    Element* element,  
    Element* vorheriges);
```

- d) Schreiben Sie eine weitere Ausgabe-Funktion, welche die Liste ebenfalls in umgekehrter Reihenfolge ausgibt, diesmal jedoch mit einer `while`-Schleife.

```
void ausgabeListe4(Element* element)
```

- e) **Schwierig:** Lesen Sie nochmals das Kapitel [→ 19.1.6](#) und bauen Sie ein Sentinel in die Liste ein. Das Sentinel soll bei einer Liste stets vorhanden sein, sprich auch bei einer leeren Liste. Die Vorwärts- und Rückwärts-Pointer zeigen bei einer leeren Liste auf das Sentinel selbst. Versuchen Sie, die Implementation aller Funktionen zu vereinfachen.

Aufgabe 3: Löschen von Baum-Knoten

Im Kapitel [→ 19.2.8](#) wird ein Beispielprogramm für einen Binärbaum gegeben. Dort wurde explizit auf das Löschen der Knoten verzichtet.

- a) Vereinfachen Sie die Programmierung mittels folgender Zeile:

```
typedef struct Knoten Knoten;
```

- b) Schreiben Sie eine Funktion, welche den Eltern-Knoten eines beliebigen Knotens findet, gegeben den Wurzelknoten eines Teilbaumes.

```
Knoten* sucheParent(Knoten* teilbaum, Knoten* ptr);
```

- c) Schreiben Sie eine Funktion, welche einen beliebigen Teilbaum, definiert durch Angabe seines Wurzelknotens, löscht.

```
void teilbaumLoeschen(Knoten* teilbaum);
```

- d) **Schwierig:** Schreiben Sie eine Funktion, welche einen beliebigen Knoten aus dem Baum entfernt, ohne dabei Teilbäume, welche unterhalb des Knotens liegen, zu zerstören. Studieren Sie hierfür nochmals das Kapitel [→ 19.2.6](#). Beachten Sie, dass der zu entfernende Knoten nicht gelöscht, sondern nur entfernt werden soll!

```
void knotenEntfernen(Knoten* knoten);
```


20 Sortieren und Suchen



Sortieren und **Suchen** sind wohl die häufigsten Tätigkeiten in der Datenverarbeitung. Das kann man auch an der historischen Entwicklung sehen: die Vorläufer der heutigen Computer waren Sortiermaschinen für Lochkarten.

Die beiden Tätigkeiten Sortieren und Suchen hängen miteinander zusammen. In einem Sprichwort heißt es: „Wer Ordnung hält, ist nur zu faul zu suchen“. Frei übersetzt bedeutet es, dass die meisten Suchverfahren am besten dann funktionieren, wenn die Daten in sortierter Form vorliegen.

Unter Sortieren versteht man die Realisierung einer Ordnungsrelation innerhalb einer gegebenen Menge von Objekten.



Die Objekte können dabei in einer Liste, einer Tabelle oder in einem Array stehen. Im allgemeinen Fall können die Elemente auch unterschiedliche Längen haben.

Objekte werden sortiert, damit man später den Suchvorgang vereinfachen kann. Dabei kann das Sortierkriterium – die Ordnungsrelation – aus einem oder mehreren Schlüsseln bestehen.



So kann man beispielsweise eine Studentenliste zunächst nach Studiengängen sortieren und dann innerhalb der Studiengänge nach den Namen der Studenten suchen. Für das Sortieren existieren verschiedene Verfahren, die sich nach Aufwand, Speicherbedarf etc. unterscheiden.

Unter internen Sortierv Verfahren versteht man Verfahren, bei denen der Sortiervorgang im Arbeitsspeicher durchgeführt wird. Externe Sortierv Verfahren arbeiten auf Daten, die in Dateien auf einem externen Speicher gespeichert sind.



Ergänzende Information Die elektronische Version dieses Kapitels enthält Zusatzmaterial, auf das über folgenden Link zugegriffen werden kann https://doi.org/10.1007/978-3-658-45209-4_20.

Im Folgenden werden nur interne Sortier- und Suchverfahren betrachtet, also solche Verfahren, bei denen die Elemente in Arrays oder verketteten Listen (siehe Kapitel [→ 19](#)) im Arbeitsspeicher stehen. Wenn die Objekte sich nicht im Arbeitsspeicher, sondern in Dateien befinden, können im Prinzip die gleichen Algorithmen angewandt werden, sofern die technischen Voraussetzungen – beispielsweise wahlfreier Zugriff zu allen Objekten – für das jeweilige Verfahren gegeben sind.

In diesem Kapitel sind verschiedene Einsatzmöglichkeiten von Arrays (siehe Kapitel [→ 8.3](#)) sowie Anwendungen des **Rekursionsprinzips** (siehe Kapitel [→ 11.8](#)) zu finden.

Einige effiziente Such- und Sortierverfahren basieren auf Baumstrukturen und wurden bereits in Kapitel [→ 19.2.4](#) behandelt.

20.1 Interne Sortierverfahren

Im Folgenden sollen zwei grundsätzliche Algorithmen interner Sortierverfahren vorgestellt werden:

- Sortieren durch direktes Auswählen
- Sortieren mit dem Quicksort-Verfahren

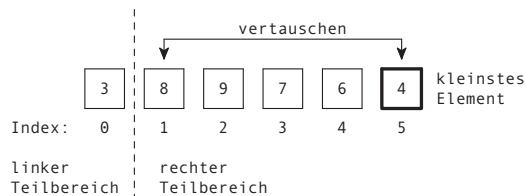
Um die Einführung in das Sortieren einfach zu gestalten, werden hier nur eindimensionale Arrays mit Integer-Werten nach aufsteigender Reihenfolge sortiert. Die Verschiebe- und Austauschoperationen werden direkt mit den Array-Elementen durchgeführt. Für den Vergleich der Werte genügen die Vergleichsoperatoren des C-Sprachumfanges. Die Verfahren können auf beliebige andere Schlüssel übertragen werden, wobei nur die entsprechenden Vergleichsrelationen vorhanden sein müssen.

20.1.1 Konzept für das iterative Sortieren durch direktes Auswählen

Sortieren durch direktes Auswählen wird auch als „**selection sort**“ oder – je nach Sortierrichtung – als „MinSort“ beziehungsweise „MaxSort“ bezeichnet und arbeitet nach folgendem Prinzip:

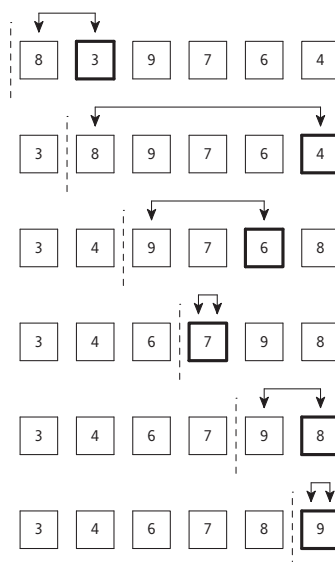
1. Teile das Array in zwei Bereiche: Ein linker, sortierter Teilbereich, der zu Beginn leer ist und ein rechter, unsortierter Teilbereich.
2. Suche im rechten unsortierten Teilbereich das Element mit dem kleinsten Wert.
3. Tausche das erste Element des rechten Teilbereichs mit dem gefundenen Element, das den kleinsten Wert hat.
4. Vergrößere den linken Bereich rechts um 1 Element und verkürze den rechten Bereich links um ein Element. Weiter bei Schritt 2.

Dieses Verfahren wird durch folgendes Bild veranschaulicht:



Das genannte Verfahren beginnt mit einem leeren linken Teilbereich, während der rechte Teilbereich zunächst das ganze Array enthält. Das Verfahren endet, wenn der rechte Teilbereich nur noch 1 Element enthält.

Das folgende Bild zeigt den gesamten Ablauf des Verfahrens:



20.1.2 Beispielprogramm für das Sortieren durch Auswählen

Im Folgenden wird als Beispiel ein C-Programm für das Sortieren durch Auswählen angegeben:

auswahl.c

```
#include <stdio.h>

void printArray(int* array, int anzahl, int schritt) {
    printf("Arbeitsschritt: %i  Array: ", schritt);
    for (int i = 0; i < anzahl; ++i)
        printf("%i ", array[i]);
    printf("\n");
}

int main(void) {

    int array[] = {8, 3, 9, 7, 6, 4};
    const int anzahl = sizeof(array) / sizeof(array[0]);

    printArray(array, anzahl, 0);

    // Schrittweise durch das gesamte Array
    for (int schritt = 0; schritt < anzahl - 1; ++schritt) {
        int minPos = schritt;

        // Suche nach kleinstem Element
        for (int pos = schritt + 1; pos < anzahl; ++pos) {
            if (array[pos] < array[minPos]) {
                minPos = pos;
            }
        }

        int swap = array[minPos];
        array[minPos] = array[schritt];
        array[schritt] = swap;

        printArray(array, anzahl, schritt + 1);
    }
    return 0;
}
```

Beispiel für den Programmablauf:

```
Arbeitsschritt: 0  Array: 8 3 9 7 6 4  
Arbeitsschritt: 1  Array: 3 8 9 7 6 4  
Arbeitsschritt: 2  Array: 3 4 9 7 6 8  
Arbeitsschritt: 3  Array: 3 4 6 7 9 8  
Arbeitsschritt: 4  Array: 3 4 6 7 9 8  
Arbeitsschritt: 5  Array: 3 4 6 7 8 9
```

20.1.3 Konzept für das rekursive Sortieren mit dem Quicksort-Verfahren

Das **Quicksort**-Verfahren wurde von Hoare [Hoa62] entwickelt. Es ist ein effizientes Sortierverfahren und ein gutes Beispiel für die Anwendung eines rekursiven Algorithmus. Der Algorithmus wird hier nur in der einfachsten Form dargestellt.

Das Verfahren arbeitet nach dem Prinzip „**teile und herrsche**“ (auf Englisch „**divide and conquer**“): Man teilt das Array in zwei Teile – oder genauer gesagt – in zwei Teilarrays auf.

Für das rekursive Sortieren mit dem Quicksort-Verfahren wird ein beliebiges Vergleichselement aus dem Array benötigt. Es bietet sich an, das mittlere Element des Arrays als Vergleichselement zu wählen. Die Bedingung an die Teilarrays ist dann, dass das linke Teilarray nur solche Elemente hat, die kleiner oder gleich wie das Vergleichselement sind und das rechte Teilarray hat nur solche Elemente, die größer oder gleich sind wie das Vergleichselement. Diese Teilarrays sind relativ einfach durch Vergleich und Austausch herzustellen. Danach wird das gleiche Teilungsverfahren auf das jeweilige linke und rechte Teilarray rekursiv angewandt, bis jedes Teilarray weniger als 2 Elemente hat.



Das Verfahren hat also folgende Schritte:

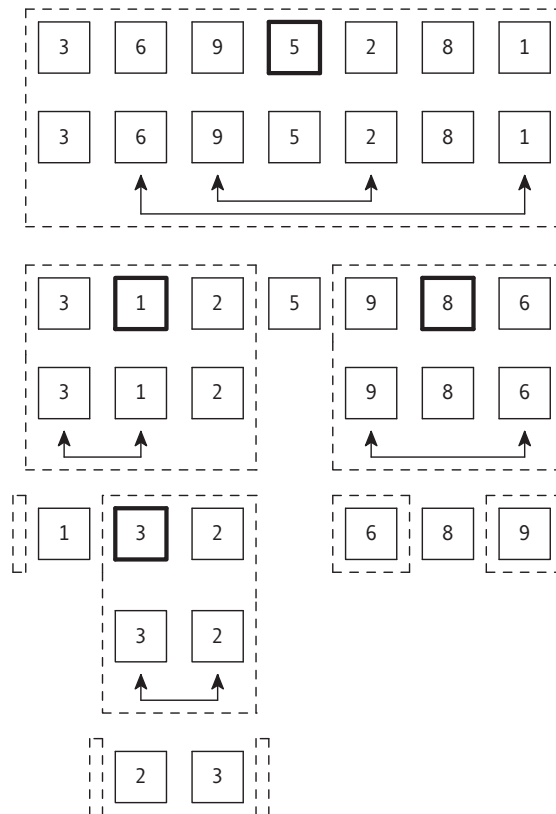
1. Auswahl eines beliebigen Vergleichswertes, beispielsweise derjenige des mittleren Elements des Arrays:

```
vergleichsWert = array[(startLinks + startRechts) / 2];
```

Dabei stehen `startLinks` und `startRechts` für den äußerst linken und äußerst rechten Index des Arrays, am Anfang also für 0 und $n - 1$.

2. Linkes Teilarray mit kleineren (oder gleichen) und rechtes Teilarray mit größeren (oder gleichen) Elementen erzeugen.
 - a) Absuchen des linken Arrays von links her mittels einer Laufvariablen `laufLinks`, bis ein Element mit größerem Wert als der Vergleichswert gefunden wird. `laufLinks` ist zu Beginn gleich `startLinks` und wird in jedem Schritt inkrementiert.
 - b) Absuchen des rechten Arrays von rechts her mittels einer Laufvariable `laufRechts`, bis ein Element mit kleinerem Wert als der Vergleichswert gefunden wird. `laufRechts` ist zu Beginn gleich `startRechts` und wird in jedem Schritt dekrementiert.
 - c) Vertauschen des gefundenen größeren Elements an `laufLinks` mit dem gefundenen kleineren Element an `laufRechts`.
 - d) Inkrementieren von `laufLinks` und dekrementieren von `laufRechts` um jeweils 1.
 - e) Schritte 2.a bis 2.d wiederholen, bis sich die beiden Suchindizes überkreuzen. Die Kreuzung definiert die Stelle, an der das Array in Schritt 3 in Teilarrays zerteilt wird.
3. Rekursive Zerlegung des linken und rechten Teilarrays. Das neue linke Teilarray besteht aus den Elementen `[startLinks, laufRechts]` und das neue rechte Teilarray aus den Elementen `[laufLinks, startRechts]`. Die Schritte 1 und 2 werden nun mit diesen beiden Teilarrays erneut durchgeführt.
4. Die Rekursion erfolgt solange, bis die Teilarrays weniger als 2 Elemente haben. Teilarrays mit einem einzigen Element sind trivialerweise sortiert.

Nach Ende von Schritt 2 stehen im linken Teilarray nur Elemente mit Werten, die kleiner als der Wert des Vergleichselements sind, und im rechten Teilarray nur Elemente, deren Werte größer als der Wert des Vergleichselements sind. Das folgende Bild symbolisiert den Ablauf des Verfahrens anhand eines Beispiels:



Die dick umrandeten Felder im obigen Bild stellen die ausgewählten Vergleichselemente dar. Die gestrichelten Felder zeigen die Teilarrays, welche noch sortiert werden müssen.

Das Vergleichselement bei den gegebenen Beispielzahlen {3, 6, 9, 5, 2, 8, 1} ist wegen der in Verfahrensschritt (1) beschriebenen Vorschrift die Zahl 5. Das Array wird nun in zwei Teilarrays {3, 1, 2} und {9, 8, 6} geteilt, wobei im linken Teilarray alle Zahlen kleiner als 5, im rechten alle größer als 5 sind. Das Vergleichselement mit dem Wert 5 ist tatsächlich nicht in den Teilarrays enthalten, da durch die Schritt-Variablen über Elemente hinwegwandern, welche gleich sind wie das Vergleichselement. Das Vergleichselement befindet sich nach diesem Schritt somit bereits an seiner endgültigen Position.

Als nächstes wird dieser Zerlegungsvorgang rekursiv auf das linke Teilarray {3, 1, 2} angewandt. Dieses wird mit der Vergleichszahl 1 durch Vertauschen in ein leeres Teilarray {} und das Teilarray {3, 2} geteilt. Das leere Teilarray entsteht, da die Laufvariable beim Vergleich über das Vergleichselement hinwegwandert. Dieser besondere Umstand muss unbedingt in der Implementation abgefangen werden. Das Teilarray {3, 2} schlussendlich wird nochmals rekursiv weiterbehandelt. Nach nur einem weiteren Schritt bleiben nur leere Teilarrays übrig. Somit ist dieses Teilarray vollständig sortiert.

Das rechte Teilarray {9, 8, 6} wird mit dem Vergleichselement 8 entsprechend so sortiert, dass links das Teilarray {6} und rechts das Teilarray {9} übrigbleibt. Da Arrays mit nur einem Element automatisch sortiert sind, ist damit der Sortiervorgang abgeschlossen.

20.1.4 Beispielprogramm für das Sortieren mit dem Quicksort-Algorithmus

Im Folgenden wird als Beispiel eine C-Funktion für das Sortieren mit dem Quicksort-Algorithmus angegeben:

quicksort.c

```
#include <stdio.h>

void printArray(int* array, int start, int ende) {
    for (int i = 0; i < start; ++i)
        printf(" ");

    for (int i = start; i <= ende; ++i)
        printf("%i ", array[i]);

    printf("\n");
}

void zerlege(int* array, int startLinks, int startRechts) {
    int laufLinks = startLinks;
    int laufRechts = startRechts;
    int vergleichsWert = array[(startLinks + startRechts) / 2];

    do {
        while (array[laufLinks] < vergleichsWert)
            ++laufLinks;

        while (array[laufRechts] > vergleichsWert)
            --laufRechts;
```



```

        if (laufLinks <= laufRechts) {
            int swap = array[laufLinks];
            array[laufLinks] = array[laufRechts];
            array[laufRechts] = swap;
            ++laufLinks;
            --laufRechts;
        }
    } while (laufLinks <= laufRechts);

    printArray(array, startLinks, startRechts);

    // Teilarrays rekursiv sortieren
    if (startLinks < laufRechts)
        zerlege(array, startLinks, laufRechts);

    if (laufLinks < startRechts)
        zerlege(array, laufLinks, startRechts);
}

void quickSort(int* array, int n) {
    zerlege(array, 0, n - 1);
}

int main(void) {
    int array[] = {7, 3, 8, 6, 9, 2, 1, 4};
    int numInArray = sizeof(array) / sizeof(int);

    printArray(array, 0, numInArray - 1);
    quickSort (array, numInArray);
    printArray(array, 0, numInArray - 1);

    return 0;
}

```

Beispiel für den Programmablauf:

```

7 3 8 6 9 2 1 4
4 3 1 2 9 6 8 7
2 1 3 4
1 2
    3 4
        6 9 8 7
            7 8 9
1 2 3 4 6 7 8 9

```

20.1.5 Sortierverfahren im Vergleich



Wenn es mehrere Verfahren zur Lösung des gleichen Problems gibt, stellt sich automatisch die Frage, welches das „bessere“ Verfahren ist. Genauso verhält es sich mit den Sortierverfahren. In der Informatik werden diese Fragestellungen unter dem Stichwort „**Komplexitätsbetrachtungen**“ untersucht. Dabei wird ein Verfahren beziehungsweise ein Algorithmus im Hinblick auf seinen Speicherbedarf oder die benötigte Rechenzeit analysiert. Die Ergebnisse erlauben dann einen Vergleich der Verfahren.

Die hier vorgestellten Sortierverfahren sollen bezüglich der benötigten Rechenzeit miteinander verglichen werden. Dazu dienen die folgenden Vorüberlegungen, die Einfluss auf die Rechenzeit haben:

Grundsätzlich werden für jedes Sortierverfahren Vergleichs- und Austauschoperationen benötigt. Der Aufwand wird zunächst immer für die Anzahl der Vergleiche (V) sowie die Anzahl der Bewegungen (B) von Elementen berechnet, wobei von einer Anzahl von n unsortierten Elementen ausgegangen wird.



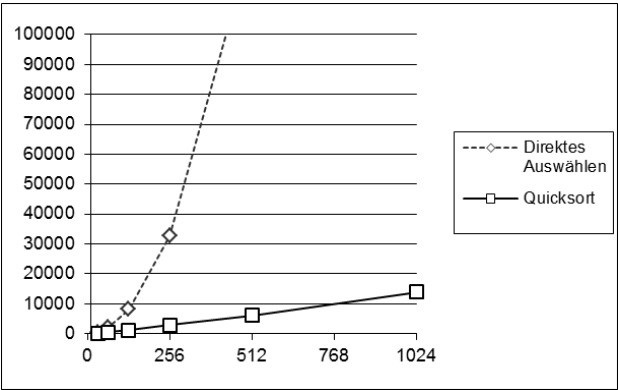
Komplexitätsbetrachtungen sind vor allem bei einer größeren Anzahl von Elementen wichtig. Dabei geht es insbesondere darum, wie sich der Aufwand des Verfahrens in Abhängigkeit zu der Anzahl der Elemente n verhält. In der Informatik wird hierfür die sogenannte **O-Notation** verwendet. Man sagt beispielsweise: Ein Suchverfahren hat eine Laufzeit von $O(n^2)$. Das bedeutet, dass die Laufzeit mit wachsendem n quadratisch ansteigt. Für 1000 Elemente benötigt ein solches Verfahren somit 1 Million Schritte. Ein Verfahren mit $O(n)$ hingegen benötigt für 1000 Elemente nur 1000 Schritte. Somit ergibt sich eine klare Aussage: $O(n^2)$ ist langsamer als $O(n)$.

Die beiden dargestellten Sortierverfahren „Sortieren durch direktes Auswählen“ und „Quicksort“ werden hier bezüglich des Aufwandes für Vergleiche und für Bewegungen einander gegenübergestellt. Anzumerken ist, dass es weitere Sortierverfahren wie beispielsweise „Bubblesort“, „Sortieren durch direktes Einfügen“ und „Shellsort“ gibt, die bezüglich des Aufwandes schlechter sind als „Quicksort“. Dabei wird als Ausgangssituation für die Gegenüberstellung ein unsortiertes, mit zufällig erzeugten Zahlen belegtes Array gewählt. Als Ergebnis wird die Anzahl der Vergleiche und die Anzahl der Bewegungen für verschieden große Arrays dargestellt.

Die folgende Tabelle vergleicht den Aufwand für das Sortieren für verschieden große Arrays. Es werden dabei einfach alle Operationen gezählt, welche entweder einen Vergleich ausführen, oder aber eine Bewegung.

Anzahl der Arrayelemente		32	64	128	256	512	1024
Direktes Auswählen	Vergleiche	496	2016	8128	32640	130816	523776
	Bewegungen	163	366	792	1781	4077	8700
Quicksort	Vergleiche	195	499	1198	2882	6120	13885
	Bewegungen	163	342	824	1910	4140	8809

Das folgende Bild veranschaulicht diese Resultate:



Es ist deutlich zu sehen, dass das Quicksort-Verfahren dem anderen weit überlegen ist. Selbst für große Arrays steigt die Laufzeit nur langsam an.

Tatsächlich hat das gesamte Sortierverfahrenverfahren eine gemittelte Laufzeit von $O(n \log_2(n))$.

Im Gegensatz dazu steigt der mittlere Aufwand des einfachen Sortierverfahrens (direktes Auswählen) quadratisch mit der Anzahl der Elemente an: $O(n^2)$

Für eine große Anzahl von Elementen sollte daher immer eines der besseren Sortierverfahren wie Quicksort verwendet werden.

Es ist zu beachten, dass Quicksort unter bestimmten Bedingungen ebenfalls eine schlechte Laufzeit von $O(n^2)$ aufweisen kann. Man geht jedoch von der optimistischen Annahme aus, dass das gewählte Element das Array in zwei möglichst gleich große Hälften teilt. Geht man von einer zufälligen Anordnung der Elemente aus, so steigt bei zufälliger Wahl des Vergleichselements der durchschnittliche Aufwand an Tauschoperationen gegenüber dem günstigsten Fall tatsächlich nur um den Faktor $2 \ln 2 = 1.39$ an (siehe [Hoa62]). Dies ist keine wesentliche Verschlechterung des optimistischen Erwartungswertes. Deswegen wird Quicksort im Allgemeinen stets mit der Laufzeit $O(n \log_2(n))$ angegeben.

Wenn allerdings zufälligerweise immer das größte oder das kleinste Element als Teilungselement gewählt wird, so wird Quicksort zu einem extrem langsamen Verfahren.



Bei der Zerlegung eines Arrays mit n Elementen würden in diesem Fall jeweils ein Teilarray mit $n - 1$ Elementen und ein Teilarray mit nur einem einzigen Element entstehen. Dadurch wird der Aufwand an Zerlegungen und die Rekursionstiefe extrem groß, da jeweils Teilarrays der Länge 1 entstehen.

Um das schlechte Verhalten im Extremfall zu vermeiden, sollte das Quicksort-Verfahren bezüglich der Wahl des Vergleichselements optimiert werden. Hierzu gibt es Vorschläge für eine relativ gute Wahl des Vergleichselements.



Ein Vorschlag ist beispielsweise, dass man 3 Elemente zufällig aussucht und das mittlere der 3 Elemente als Vergleichselement nimmt.

20.1.6 Die Quicksort-Funktion der Standardbibliothek

Die Funktion `qsort()` aus der Standard-Bibliothek von C (Prototyp in `<stdlib.h>`) enthält eine optimierte Form des Quicksort-Algorithmus mit einer beliebigen Vergleichsfunktion, die mit Hilfe eines Funktionszeigers übergeben wird.



Der Prototyp ist wie folgt aufgebaut:

```
void qsort(  
    void* base,  
    size_t nmemb,  
    size_t size,  
    int (*compar)(const void*, const void*));
```

Die 4 Parameter der Funktion `qsort()` haben die folgende Bedeutung:

- **base:** Pointer auf das erste Element des Arrays, also in der Regel der Arrayname.
- **nmemb:** Anzahl der Elemente im Array.
- **size:** Größe eines Array-Elements in Bytes (beispielsweise mittels `sizeof`).
- **compar:** Funktionsname einer existierenden, selbst geschriebenen Vergleichsfunktion, die als Parameter zwei Pointer auf `void` übergeben bekommt, die auf zwei zu vergleichende Elemente aus dem Array zeigen. Die Funktion muss diese beiden Elemente vergleichen und das Ergebnis des Vergleichs als `int`-Zahl wie folgt zurückgeben: Falls beide Elemente gleich sind, so wird eine 0 zurückgegeben. Ist das erste Element größer als das zweite Element, wird eine Zahl größer 0 zurückgegeben und im umgekehrten Fall eine Zahl kleiner 0.

Die `qsort()`-Funktion ruft im Zuge des intern ablaufenden Sortiervorganges immer wieder die Vergleichsfunktion auf und übergibt ihr als Parameter zwei Pointer auf zwei zu vergleichende Elemente.



Dadurch ist es möglich, die `qsort()`-Funktion sehr flexibel einzusetzen und beliebige Daten zu sortieren, da die Funktion sich nicht um den Vergleich an sich kümmern muss. Beim Programmieren der Vergleichsfunktion wird ja vorgegeben, was größer, kleiner oder gleich ist. Dies muss dann für unterschiedliche Datentypen eben mit verschiedenen Vergleichsfunktionen erfolgen.

Die Pointer auf `void` können in der Vergleichsfunktion in einen entsprechenden Pointer-Typ gewandelt werden (siehe Kapitel [8.2](#)), damit der Vergleich durchgeführt werden kann. Dieser Pointer-Typ muss so gewählt werden, dass mittels einer Dereferenzierung der Zugriff auf die zu vergleichenden Objekte möglich wird.

Folgendes ist ein Beispiel:

`qsort.c`

```
#include <stdlib.h>
#include <stdio.h>

int aufsteigend(const void* a, const void* b) {
    return (*(int*)a - *(int*)b);
}

int absteigend(const void* a, const void* b) {
    return (*(int*)b - *(int*)a);
}

void printArray(int* array, int anzahl) {
    for (int i = 0; i < anzahl; ++i)
        printf("%i ", array[i]);
    printf("\n");
}

int main(void) {
    int array[] = {7, 3, 8, 6, 9, 2, 1, 4};
    int anzahl = sizeof(array) / sizeof(int);

    printArray(array, anzahl);
    qsort(array, anzahl, sizeof(int), aufsteigend);
    printArray(array, anzahl);
    qsort(array, anzahl, sizeof(int), absteigend);
    printArray(array, anzahl);
}
```

Ausgabe:

7 3 8 6 9 2 1 4

1 2 3 4 6 7 8 9

9 8 7 6 4 3 2 1

Um die `qsort()`-Funktion aus der Standardbibliothek zu nutzen, muss nur eine solche Vergleichsfunktion für die Elemente des Arrays implementiert werden. Die Funktionalität des Quicksort-Verfahrens steht also über die Standardbibliothek ohne großen Entwicklungsaufwand zur Verfügung.

20.2 Einfache Suchverfahren

Eine häufige Anwendung in der Informationsverarbeitung ist das **Suchen** eines Objektes in einer Menge von Objekten gleichen Typs, wie zum Beispiel das Suchen der Daten eines bestimmten Mitarbeiters in den Personaldaten. Die folgende Tabelle symbolisiert irgendwelche Personaldaten:

Personalnummer	Name	Vorname	Abteilung
125	Müller	Hans	EK1
015	Sommer	Elke	FE
093	Braun	Charly	FE
007	Bond	Johannes	SEC
...			

Ein Element umfasst die Datenfelder Personalnummer, Name, Vorname und Abteilung. Eine derartige Tabelle kann man im Arbeitsspeicher in Form eines Arrays aus Strukturen implementieren.

Die Tabelle soll dabei beliebig durchsucht werden können, beispielsweise nach dem Vornamen „Charly“ oder der Personalnummer 007. Das Suchen wird beendet, wenn das gesuchte Element gefunden oder das Ende der Tabelle erreicht wurde.

Der Suchbegriff beziehungsweise **Schlüssel** ist Teil eines Objektes.



Die Objekte (Elemente) können dabei in Listen, Arrays oder Dateien stehen, wobei im allgemeinen Fall die Objekte unterschiedliche Größen haben dürfen.

Beim Suchen kommt es dem Anwender darauf an, dass die gesuchten Elemente schnell gefunden werden.



Hierbei unterscheidet man grundsätzlich zwischen dem Suchen in unsortierten Mengen und dem Suchen in sortierten Mengen.

20.2.1 Unsortierte Menge: Sequenzielles Suchen

Falls die Elemente einer Menge nicht sortiert sind, kann nur sequenziell gesucht werden, sprich jedes Element muss einzeln geprüft werden.

Der Algorithmus „**Sequenzielles Suchen**“ besteht aus dem Vergleich des Suchbegriffes mit den Schlüsselwerten der Elemente einer ungeordneten Menge, die beispielsweise in Form eines Arrays oder einer verketteten Liste vorliegen kann.



Sequenzielles Suchen ist eine ineffiziente Suchmethode, da im ungünstigsten Falle alle n Elemente der Menge mit dem Suchbegriff verglichen werden müssen.



Im Mittel sind bei diesem Algorithmus $n/2$ Suchschritte erforderlich.



Die Effizienz eines solchen Verfahrens kann gesteigert werden, wenn die am häufigsten gesuchten Elemente an den Anfang des Arrays geschrieben werden. In diesem Fall muss allerdings die Suchwahrscheinlichkeit bekannt sein und die Menge vorher entsprechend aufbereitet werden.

Bei einer relativ kleinen Menge kann je nach Situation der Aufwand von sequenziellem Suchen noch akzeptabel sein. Ab einer gewissen Größe jedoch wird man auf effizientere Algorithmen zählen müssen.

20.2.2 Sortierte Menge: Halbierungssuchen

Voraussetzung für das Halbierungssuchen, welches auch unter dem Begriff „**Binäres Suchen**“, oder auf Englisch „**binary search**“ bekannt ist, ist eine sortierte Menge von Elementen.



Um eine Menge zu sortieren, können verschiedene Sortierverfahren angewendet werden. Zwei Beispiele wurden in Kapitel [→ 20.1](#) gegeben.

Im Folgenden wird zur Vereinfachung ein Array bestehend aus `int`-Werten betrachtet.

Beim Halbierungssuchen wird wie folgt vorgegangen:

1. Zu Beginn ist das gesamte Array das Suchintervall.
2. Das Element in der Mitte des Intervalls wird geprüft.
3. Entspricht das Element in der Mitte dem Suchkriterium, so ist das Element gefunden und der Algorithmus ist beendet.
4. Ist der Schlüssel des mittleren Elements größer als der gesuchte Schlüssel, so muss sich das Element im unteren Teilintervall befinden. Ist er kleiner, so muss es sich im oberen Teilintervall befinden. Die Intervallgrenzen werden entsprechend angepasst und bei Schritt 2 fortgefahren.
5. Die Halbierungen der Teilintervalle werden solange fortgesetzt, bis das Element gefunden wurde oder das Teilintervall leer ist. Im letzteren Falle ist das gesuchte Element nicht in der Liste.

Das Verfahren des Halbierungssuchens arbeitet korrekt für beliebige n , der Algorithmus hat die Komplexität $O(\log_2(n))$.



Bei 1000 Elementen wird somit mit diesem Verfahren das gesuchte Element nach höchstens rund 10 Suchschritten ($\log_2(1000) \approx 10$) gefunden, während bei sequenziellem Suchen im Mittel 500 Suchschritte notwendig gewesen wären.

20.2.3 Beispiel für das Halbierungssuchen

Im folgenden Beispiel für das Halbierungssuchen (binäres Suchen) liegt ein geordnetes Array mit int-Elementen vor.

binaerSuchen.c

```
#include <stdio.h>

int binaer_suchen(int suchbegriff, const int* array, int anzahl) {
    int low = 0;
    int high = anzahl - 1;
    int mid;

    while (low <= high) {
        mid = (low + high) / 2;
        printf("Suche in [%d, %d] an Index %d: %d\n",
            low, high, mid, array[mid]);

        if (suchbegriff < array[mid])
            high = mid - 1;
        else if (suchbegriff > array[mid])
            low = mid + 1;
        else
            return mid; // gefunden
    }
    return -1; // nicht gefunden
}

int main(void) {
    int array[] = {2, 5, 11, 17, 23, 31, 33, 37, 44, 50, 59, 62};
    int anzahl = sizeof(array) / sizeof(int);

    int suchbegriff = 23;
    int index = binaer_suchen(suchbegriff, array, anzahl);
    if (index == -1)
        printf("Nicht gefunden \n");
    else
        printf("Element mit Index %d = %d\n", index, suchbegriff);

    return 0;
}
```

Hier die Ausgabe des Programms:

```
Suche in [0, 11] an Index 5: 31
Suche in [0, 4] an Index 2: 11
Suche in [3, 4] an Index 3: 17
Suche in [4, 4] an Index 4: 23
Element mit Index 4 = 23
```

Im ersten Schritt wird der Index 5 als Mitte bestimmt. Das gefundene Element 31 ist größer als 23, deswegen wird das tiefere Intervall $[0, 4]$ durchsucht. Im nächsten Schritt befindet sich die Mitte an Index 2, das Element ist jedoch kleiner als 23, also wird das obere Teilintervall $[3, 4]$ genommen. Im dritten Schritt ist das mittlere Element ebenfalls kleiner als 23, weswegen schlussendlich das Intervall $[4, 4]$ durchsucht wird. Wenn das zu durchsuchende Intervall nur aus einem Element besteht, dann muss dieses entweder das gesuchte Element sein oder aber das Element ist nicht in der Menge. Hier entspricht das Element tatsächlich der Zahl 23 und der Algorithmus gibt den gefundenen Index 4 zurück.

20.2.4 Beispielprogramm mit `bsearch()`

Genauso wie es für das Quicksort-Verfahren eine Funktion in der Standardbibliothek `stdlib.h` gibt (vergleiche `qsort()` in Kapitel [→ 20.1.6](#)), bietet diese Bibliothek auch eine Implementation der binären Suche an unter dem Namen `bsearch()`:

```
void* bsearch(
    const void* key,
    const void* base,
    size_t nmemb,
    size_t size,
    int (*compar)(const void*, const void*));
```

Ähnlich wie bei der Funktion `qsort()` wird beim Parameter `base` der Pointer auf das zu durchsuchende Array angegeben, der Parameter `nmemb` definiert, wie viele Elemente sich in dem Array befinden und mit `size` wird die Größe eines einzelnen Elements angegeben. Die Vergleichsfunktion `compar()` funktioniert genau gleich wie bei `qsort()`. Als zusätzlichen ersten Parameter erwartet die Funktion `bsearch()` einen Pointer `key` auf den Schlüssel, nach dem gesucht werden soll.

Wenn der Schlüssel in dem Array nicht gefunden wird, so wird ein NULL-Pointer zurückgegeben. Wenn er jedoch gefunden wird, so wird ein Pointer auf den Eintrag im Array zurückgegeben, innerhalb welchem der Schlüssel zu finden ist. Wenn derselbe Schlüssel an mehreren Stellen im Array zu finden ist, ist undefiniert, welche der möglichen Stellen zurückgegeben wird.

Folgendes Beispiel verdeutlicht die Verwendung der Funktion `bsearch()`:

`bsearch.c`

```
#include <stdlib.h>
#include <stdio.h>

int ordnung(const void* a, const void* b) {
    return (*(int*)a - *(int*)b);
}

int main(void) {
    int array[] = {2, 5, 11, 17, 23, 31, 33, 37, 44, 50, 59, 62};
    int anzahl = sizeof(array) / sizeof(int);

    int key = 23;
    int* keyptr = bsearch(&key, array, anzahl, sizeof(int), ordnung);

    if (keyptr)
        printf("Gefunden an Index %d\n", (int)((int*)keyptr - array));
    else
        printf("Nicht gefunden\n");
}
```

Die Ausgabe lautet:

```
Gefunden an Index 4
```

20.3 Suchen nach dem Hashverfahren

Beim binären Suchen wurde der Aufwand gegenüber dem sequenziellen Suchen von $O(n)$ auf $O(\log_2(n))$ reduziert. Bei 1000 Elementen werden somit bei binärem Suchen immer noch 10 Schritte benötigt. Wünschenswert wäre es, mit ein bis zwei Schritten direkt ein gesuchtes Element zu finden. Man spricht dann von einem $O(1)$ -Algorithmus.

Mit einem bis zwei Schritten direkt ein gesuchtes Element zu finden, ist näherungsweise erreichbar, wenn aus dem Suchbegriff direkt durch Berechnung auf die Position im Array geschlossen werden kann.



Solche Verfahren werden „Hashverfahren“ genannt, aus Gründen, die noch erklärt werden.

20.3.1 Einführendes konzeptionelles Beispiel

Es seien die vier Zahlen $\{2, 4, 8, 9\}$ gegeben. Diese Zahlen werden in einem Array mit 10 Elementen so gespeichert, dass die jeweilige Zahl i an der Position i in dem Array gespeichert wird. Die anderen, unbenutzten Positionen des Arrays werden durch eine „Frei“-Markierung -1 als unbesetzt gekennzeichnet.

	-1	-1	2	-1	4	-1	-1	-1	8	9
Index:	0	1	2	3	4	5	6	7	8	9

Wenn nun ein Suchbegriff x vorgelegt wird (Zahl zwischen 0 und 9), so kann man durch Vergleich der Zahl x mit dem Element an Position x des Arrays feststellen, ob die Zahl sich im Array befindet oder dort nicht enthalten ist.

20.3.2 Einfache Schlüsseltransformation und Konflikte

Die in dem einführenden Beispiel dargestellte Methode lässt sich für beliebige Schlüssel in der im Folgenden beschriebenen Form sinnvoll erweitern.

Es sollen n Elemente mit beliebigen **Schlüsseln** (hier der Einfachheit halber vom Typ `int`) in einer Tabelle gespeichert werden. Die Schlüssel seien beliebig verteilt über $[0 \dots K]$. Zur Speicherung der Elemente soll eine Tabelle mit M Elementen verwendet werden, wobei M größer ist als n . Aus dem Schlüssel wird nun die Position in der Tabelle berechnet. An einer Position der Tabelle kann in dem betrachteten Fall jeweils ein einziger Schlüsselwert gespeichert werden.

Zunächst sei angenommen, die Tabellenlänge M sei eine Primzahl. Die Position eines Elementes mit dem Schlüssel x in der Tabelle kann dann durch folgende Transformation ermittelt werden:

$$\text{Position} = x \bmod M$$

Damit können auch Schlüssel, die größer als M sind, in der Tabelle untergebracht werden.

Durch die Modulo-Funktion wird – bildlich gesprochen – ein Teil des Schlüssels abgehackt (auf Englisch „hashed“). Daher wird dieses Verfahren im Englischen als „**hash**“-Verfahren bezeichnet.



Angenommen, M sei deutlich größer als n . Wenn nun n Elemente in eine Tabelle mit M Plätzen an die nach obigem Algorithmus berechneten Positionen eingetragen werden, so werden die Elemente in der Tabelle mit Lücken gespeichert sein. Daher stammt auch die deutsche Bezeichnung „Streuspeicherung“ für das Verfahren.

Im folgenden Beispiel wird eine Tabelle mit $M = 7$ Plätzen und $n = 3$ Elementen gezeigt, deren Positionen über $x \bmod M$ berechnet wurden:

Position	Inhalt	Berechnung
0	77	$77 \bmod 7 = 0$
1		
2	16	$16 \bmod 7 = 2$
3		
4		
5	40	$40 \bmod 7 = 5$
6		

Wie man sieht, sind die 3 Werte über die Tabelle gestreut, 4 Plätze der Tabelle sind noch frei. Zunächst wird angenommen, dass alle Schlüssel zu verschiedenen Positionen in der Tabelle führen. In einer derart organisierten Tabelle lässt sich nun ein Element sehr leicht finden, indem man mit dem Schlüssel des gesuchten Elementes die gleiche Transformation durchführt wie beim Eintragen eines Elementes in die Tabelle. Ein Vergleich mit dem Inhalt an der berechneten Tabellenposition zeigt nun, ob das gesuchte Element in der Tabelle enthalten ist oder nicht.

Leider stimmt die Annahme, dass alle Schlüssel zu verschiedenen Positionen führen, häufig nicht. Es tritt ein sogenannter Konflikt auf.



Fügt man obigem Beispiel noch die Zahl 23 hinzu, so ergibt $23 \bmod 7$ den Index 2.

Somit gibt es zwei Zahlen (16 und 23), welche beide am Index 2 gespeichert werden sollten. Diese Situation wird als „**Konflikt**“ oder „**Kollision**“ bezeichnet. Dieser Konflikt muss beseitigt werden. Dazu existieren verschiedene Strategien, die im Weiteren noch erläutert werden.

20.3.3 Konfliktlösung mit linearer Sondierung

Zunächst wird eine einfache Konfliktstrategie erläutert, die auf einer linearen Sondierung besteht. Es wird dabei davon ausgegangen, dass das Array im Verhältnis zu der Anzahl an zu speichernden Werte sehr groß ist.

Bei der Konfliktlösung mit linearer Sondierung wird, wenn eine Position in der Tabelle schon besetzt ist und ein neuer Schlüssel zur gleichen Position in der Tabelle führt, ab der berechneten Position linear aufwärts nach der nächsten freien Position gesucht und das Element dort gespeichert. Bei dieser neuen Position muss natürlich ebenfalls mit modulo M sichergestellt werden, dass sie nicht die Größe der Tabelle sprengt.



Im obigen Beispiel werden die Konflikte dann in folgender Weise gelöst:

Position	Inhalt	Berechnung
0	77	$77 \bmod 7 = 0$
1		
2	16	$16 \bmod 7 = 2$
3	23	$23 \bmod 7 = 2 \dots +1$
4		
5	40	$40 \bmod 7 = 5$
6		

Beim Suchen wird dann das gleiche Verfahren angewandt. Zunächst wird aus dem Suchschlüssel die Position berechnet und das gesuchte Element mit dem dort gespeicherten Element verglichen.

Beim Suchen ist bei Übereinstimmung der Position das Element gefunden, bei Nichtübereinstimmung wird ab der berechneten Position linear aufwärts das Element in der Liste gesucht (mit modulo M), bis es gefunden wurde oder ein Platz frei ist. Wenn ein freier Platz gefunden wurde, ist das Element nicht in der Liste und die Suche ist beendet.



Es ist offensichtlich, dass dieses Verfahren bei häufigen Kollisionen sehr ineffizient wird. Die Kollisionswahrscheinlichkeit steigt mit dem Füllgrad der Liste n/M deutlich an.

Die Tabellenlänge muss vorher bekannt sein. Die Anzahl n darf M nie übertreffen. Generell sollte die Tabelle einen Füllgrad n/M von 80% bis 90% nicht übersteigen.



In diesem Verfahren wurden bei Kollisionen die neuen Positionen jeweils mit einer Erhöhung um die Schrittweite 1 berechnet. Man kann auch andere Schrittweiten, die größer als 1 und kleiner als die Tabellenlänge M sind, wählen, erhält aber dadurch keine Verbesserung des Verfahrens.

Das Löschen von Elementen ist ebenfalls möglich. Ein Element wird gelöscht, indem eine Löschmarkierung als Schlüssel in die Tabelle eingetragen wird. Dies muss ein Wert sein, der sonst nicht als Schlüssel vorkommen darf und sich auch von der „Frei“-Markierung unterscheiden muss.

Beim Einfügen von Elementen in die Tabelle können gelöschte Plätze wieder benutzt werden, das heißt gelöschte Plätze sind beim Einfügen wie leere Plätze zu behandeln. Beim Suchen müssen aber gelöschte Elemente aufgrund eventuell aufgetretener Kollisionsfälle fast wie normale Elemente behandelt werden, da ja bei noch nicht erfolgter Konfliktlösung „hinter“ ihnen noch Elemente stehen können, die zuvor mit den jetzt gelöschten Elementen kollidierten. Würde man einfach eine „Frei“-Markierung beim Löschen einsetzen, so würde die Suche an dieser „Frei“-Markierung stoppen, obwohl nach dem gelöschten Element noch weitere Kollisionsfälle liegen. Natürlich kommen die gelöschten Elemente selbst als Ergebnis des Suchlaufs nicht mehr in Betracht.

Falls die zu speichernden Elemente sehr umfangreich sind oder eine variable Länge haben, kann man in der Hashtabelle statt der Elemente auch nur Schlüssel und Pointer auf das jeweilige Element oder sogar nur Pointer auf Elemente speichern und die Elemente durch dynamische Speicherzuweisung (mit `malloc()`) erzeugen. Wenn nur Pointer verwendet werden, so könnte die „Frei“-Markierung beispielsweise der NULL-Pointer sein, während die Löschmarkierung ein Pointer auf ein einziges und eindeutiges „Löschelement“ sein könnte.

Es gibt auch andere Kollisionsbehandlungen als die lineare Sondierung. Beispielsweise muss das sondierte Element nicht zwingendermaßen das nächste im Array sein, sondern der Abstand zwischen den sondierten Elementen kann genauso wie der Schlüssel selbst abhängig vom tatsächlich zu speichernden Wert sein. Je nach Typ des Schlüssels und der erwarteten Verteilung können verschiedene Verfahren zu einer guten Verteilung innerhalb des Arrays führen.

Die Vielzahl der Möglichkeiten, Kollisionen innerhalb des Arrays aufzulösen, sprengt den Rahmen dieses Buches bei weitem, weswegen nicht weiter darauf eingegangen wird. Im Kapitel [→ 20.3.6](#) wird jedoch noch auf die Möglichkeit eingegangen, Konflikte mittels Elemente zu lösen, welche sich nicht mehr innerhalb des Arrays befinden.

20.3.4 Programmbeispiel für die Programmierung einer Hashtabelle mit linearer Sondierung

Das folgende Beispiel errechnet den Hash-Schlüssel eines Strings gemäß der folgenden Formel:

$$\left(\sum_{i=0}^{i < k} \text{asciiCode}(c_i) \right) \bmod M$$

Hierbei werden die einzelnen ASCII-Codes der Zeichen aufaddiert und dann diese Summe modulo der Tabellengröße geteilt. Jetzt das Programm:

hashtab.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define FREI      "0"      // Festlegung eines FREI-Zeichens
#define GELOESCHT "1"      // Festlegung eines GELOESCHT-Zeichens

typedef struct Messwert {
    char name[40];
    double wert;
} Messwert;

#define ANZAHL 8
Messwert tabelle[ANZAHL];
```

```
int hash(const char* name) {
    int summeASCII = 0;
    const char* ptrName = name;
    while (*ptrName != '\0') {
        summeASCII = (int)*ptrName + summeASCII;
        ++ptrName;
    }
    return (summeASCII % ANZAHL);
}

int rehash(int hash) {
    return (hash + 1) % ANZAHL;
}

void einfuegen(const char* name, double wert) {
    int z = 0;
    int index = hash(name);
    while ((strcmp(tabelle[index].name, FREI)) &&
        (strcmp(tabelle[index].name, GELOESCHT)))
    {
        index = rehash(index);
        ++z;
        if (z == ANZAHL) {
            printf("Tabelle voll\n");
            exit(1);
        }
    }

    // Freier Eintrag gefunden
    strcpy(tabelle[index].name, name);
    tabelle[index].wert = wert;
}

int sucheSchluessel(const char* name) {
    int z = 0;
    int index = hash(name);
    while (strcmp(tabelle[index].name, name)) {
        index = rehash(index);
        ++z;
        if (!strcmp(tabelle[index].name, FREI) || z == ANZAHL) {
            return -1;
        }
    }
    return index;
}
```

```
void loescheSchluessel(const char* name) {
    int index = sucheSchluessel(name);
    if (index != -1) {
        strcpy(tabelle[index].name, GELOESCHT);
        tabelle[index].wert = 0.;
    }else{
        printf("Schluessel %s nicht in Tabelle\n", name);
    }
}

static void initTabelle() {
    for (int i = 0; i < ANZAHL; ++i) {
        strcpy(tabelle[i].name, FREI);
        tabelle[i].wert = 0.;
    }
}

void ausgebenTabelle(void) {
    printf("\n");
    for (int i = 0; i < ANZAHL; ++i) {
        if (!strcmp(tabelle[i].name, FREI)) {
            printf("Index %d: FREI\n", i);
        } else if (!strcmp(tabelle[i].name, GELOESCHT)) {
            printf("Index %d: GELOESCHT\n", i);
        } else {
            printf("Index %d: %s (%d) = %1.02f\n",
                i,
                tabelle[i].name,
                hash(tabelle[i].name),
                tabelle[i].wert);
        }
    }
    printf("\n");
}

int main(void) {
    initTabelle();

    einfuegen("Temperatur", 23.);
    einfuegen("Luftfeuchtigkeit", 64.);
    einfuegen("Luftdruck", 1022.);
    einfuegen("Gewicht", 483.);
    einfuegen("Pegelstand", 644.);

    ausgebenTabelle();
}
```

```
int gewichtIndex = sucheSchluessel("Gewicht");
printf("Gewicht gefunden an Index %d\n", gewichtIndex);
int feuchtIndex = sucheSchluessel("Luftfeuchtigkeit");
printf("Luftfeuchtigkeit gefunden an Index %d\n", feuchtIndex);

loescheSchluessel("Luftfeuchtigkeit");
printf("Luftfeuchtigkeit geloescht.\n");

feuchtIndex = sucheSchluessel("Luftfeuchtigkeit");
printf("Luftfeuchtigkeit gefunden an Index %d\n", feuchtIndex);

ausgebenTabelle();

return 0;
}
```

Folgendes ist der Programmablauf:

```
Index 0: Pegelstand (7) = 644.00
Index 1: Temperatur (1) = 23.00
Index 2: FREI
Index 3: Gewicht (3) = 483.00
Index 4: Luftdruck (4) = 1022.00
Index 5: FREI
Index 6: FREI
Index 7: Luftfeuchtigkeit (7) = 64.00

Gewicht gefunden an Index 3
Luftfeuchtigkeit gefunden an Index 7
Luftfeuchtigkeit geloescht.
Luftfeuchtigkeit gefunden an Index -1

Index 0: Pegelstand (7) = 644.00
Index 1: Temperatur (1) = 23.00
Index 2: FREI
Index 3: Gewicht (3) = 483.00
Index 4: Luftdruck (4) = 1022.00
Index 5: FREI
Index 6: FREI
Index 7: GELOESCHT
```

20.3.5 Weitere Schlüsseltransformationen

Geeignete Schlüsseltransformationen zur Berechnung der ersten Position innerhalb der Tabelle mit Hilfe einer Hash-Funktion hängen stark von der Beschaffenheit des Schlüssels ab.



Es gibt viele Verfahren von Schlüsseltransformationen. Das in diesem Kapitel dargestellte einfache Verfahren (Index = x modulo M) arbeitet – bei einigermaßen gleichförmig verteilten Schlüsseln und wenn M eine Primzahl ist – recht gut. Man erreicht dadurch meist eine gute Verteilung über die ganze Tabelle. M muss aber nicht eine Primzahl sein [Wet80].

Andere Verfahren selektieren einige Bits aus dem Schlüssel zur Berechnung des Hash-Index. Häufig bestehen die Schlüssel aus Strings, beispielsweise bei Symboltabellen in Compilern oder bei Namenslisten. So kann für Namen die erste Tabellenposition beispielsweise folgendermaßen berechnet werden:

Der Namensstring bestehe der Einfachheit halber nur aus Großbuchstaben und aus maximal n Zeichen.

Name = $z_0 z_1 z_2 z_3 \dots z_{n-1}$

Den Namen kann man sich als variabel lange n -stellige Zahl zur Basis 27 (26 Buchstaben des Alphabets, 0 für Leerzeichen) vorstellen und daraus je unterschiedlichem Namen eine eindeutige Zahl erzeugen. Wenn man dem Buchstaben „A“ den Wert 1, dem Buchstaben „B“ den Wert 2, ... und „Z“ den Wert 26 zuordnet, so kann man daraus eine eindeutige Schlüsselzahl berechnen:

$$x = z_0 \cdot 27^{n-1} + z_1 \cdot 27^{n-2} + z_2 \cdot 27^{n-3} + z_3 \cdot 27^{n-4} + \dots + z_{n-1} \cdot 27^0$$

So liefert beispielsweise der Name „ABC“ die Zahl $x = 1 \cdot 27^2 + 2 \cdot 27^1 + 3 \cdot 27^0 = 786$

Bei beliebig großen Namenslängen könnten sehr große Zahlen entstehen. Um die zugeordneten Schlüsselzahlen dann auf einen bestimmten Wertebereich (beispielsweise 32-Bit Zahlen) einzuschränken, kann man die Umrechnung auf die ersten 6 Buchstaben des Namens einschränken und zusätzlich die Namenslänge n mit in die Umrechnung einbeziehen. Die Zuordnung ist dann nicht mehr eindeutig, das heißt ein Kollisionsverfahren ist erforderlich.

Damit können den Namen beispielsweise folgende Schlüsselzahlen zugeordnet werden:

$$\text{Schlüsselzahl } x = z_0 \cdot 27^5 + z_1 \cdot 27^4 + z_2 \cdot 27^3 + z_3 \cdot 27^2 + z_4 \cdot 27^1 + z_5 \cdot 27^0 + n;$$

Mit dieser Berechnung ergeben sich Schlüsselzahlen, die kleiner sind als $27^6 - 1 + n$. Diese Werte sind selbst bei großen Namenslängen n deutlich kleiner als 2^{32} .

Folgendes ist ein Beispiel, wie so eine Hash-Funktion ausprogrammiert sein könnte:

hashName.c

```
#include <string.h>
#include <stdio.h>
#include <ctype.h>

int hashName(const char* name) {
    int zahl = toupper(name[0]) - 'A' + 1;
    int len = (int)strlen(name);
    if (len > 6) len = 6;
    for (int i = 1; i < len; ++i) {
        if (isalpha(name[i]))
            zahl = zahl * 27 + toupper(name[i]) - 'A' + 1;
    }

    return zahl + (int)strlen(name);
}

int main(void) {
    printf("Hash: %d\n", hashName("C Programmieren"));
    printf("Hash: %d\n", hashName("D Programmieren"));
    return 0;
}
```

Das Ergebnis dieses Programmes lautet:

```
Hash: 1922800
Hash: 2454241
```

Es gibt noch viel mehr Möglichkeiten, Hash-Schlüssel zu definieren. Hier wird nicht weiter darauf eingegangen.

20.3.6 Direkte Verkettung bei Kollisionen

Bei den bisher dargestellten Verfahren der Konfliktlösung mussten die Tabellen ausreichend groß sein, um Platz für alle Elemente zu haben. Die Konflikte wurden innerhalb der Tabelle gelöst.

Hashverfahren, bei denen die Konflikte innerhalb der zur Verfügung gestellten Tabelle aufgelöst werden, werden auch als geschlossene Hashverfahren bezeichnet. Hashverfahren, bei denen im Kollisionsfall neue Elemente in einem zur Hashtabelle externen Bereich untergebracht werden, heißen offene Hashverfahren.



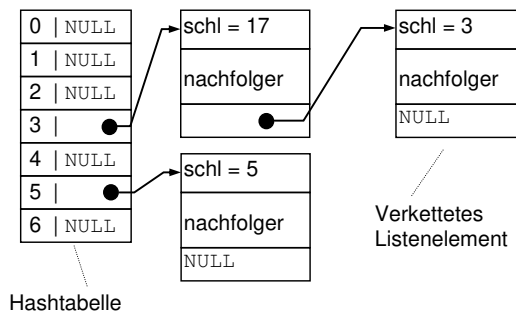
Ein Beispiel eines offenen Verfahrens ist die Verwendung einer einfach verketteten Liste, genannt „Direkte Verkettung“.

Bei der direkten Verkettung wird bei Kollisionen für die Elemente mit gleichem Tabellenindex eine verkettete Liste außerhalb der Tabelle angelegt, die dann linear verlängert und beim Suchen linear durchsucht wird.



Bei dieser Organisation der Hashtabelle reicht es aus, dass die Hashtabelle nur Pointer auf die Elemente enthält. Die Elemente selbst enthalten den Schlüssel, die restlichen Daten des Elementes sowie einen Verkettungs-Pointer zum Aufbau einer verketteten Liste im Kollisionsfall.

Das folgende Beispiel zeigt eine Hashtabelle mit verketteter Liste als Konfliktlösungsstrategie:



Die Position ergebe sich zu $\text{Position} = x \text{ modulo } M$, die Tabellenlänge sei $M = 7$. Es sollen drei Elemente mit den Schlüsseln 17, 3 und 5 eingetragen werden. Der Schlüssel 17 ergibt eine Position von 3, der nachträglich einzutragende Schlüssel 3 ergibt ebenfalls eine Position von 3. Dieser Konflikt wird durch Aufbau einer verketteten Liste für alle Elemente mit der gleichen Position (hier 3) gelöst. Bei diesem Verfahren müssen alle Array-Elemente der Hashtabelle zu Anfang mit dem NULL-Pointer belegt werden. Wird für eine Position ein Element eingetragen, so muss der Pointer an der Position der Hashtabelle auf dieses Element zeigen. Das Ende der verketteten Liste für Elemente mit gleichem Schlüssel wird durch den NULL-Pointer in diesem Element angezeigt. Wird für diese Position der Hashtabelle ein weiteres Element eingetragen, wird es an diese Liste angehängt.

Das Suchen geschieht in der Weise, dass zunächst aus dem Suchbegriff die Tabellenposition berechnet wird. Anschließend müssen im ungünstigsten Fall alle an der Position vorhandenen Listenelemente untersucht werden, bis das Element durch einen direkten Schlüsselvergleich gefunden wird. Beim Löschen muss man zunächst das Element suchen und anschließend aus der verketteten Liste entfernen.

Das folgende Programm zeigt, wie eine solche Hashtabelle erstellt werden kann. Die Löschoption wird jedoch ausgelassen.

hash.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct Element {
    int schluessel;
    struct Element* nachfolger;
} Element;

#define ANZAHL 11
Element* tabelle[ANZAHL];

void initTabelle(void) {
    for (int i = 0; i < ANZAHL; ++i)
        tabelle[i] = NULL;
}

int hash(int schluessel) {
    return (schluessel % ANZAHL);
}
```

```
void einfuegen(int schluessel) {
    int index = hash(schluessel);

    Element* neuesElement = malloc(sizeof(Element));
    neuesElement->schluessel = schluessel;
    neuesElement->nachfolger = NULL;

    if (tabelle[index] == NULL) {
        // Neues Element in die Tabelle einfuegen.
        tabelle[index] = neuesElement;
    } else {
        Element* element = tabelle[index];

        // Suche nach dem letzten Element
        while (element->nachfolger)
            element = element->nachfolger;

        // Neues Element in die Liste einfuegen.
        element->nachfolger = neuesElement;
    }
}

Element* sucheElement(int schluessel) {
    int index = hash(schluessel);
    Element* element = tabelle[index];

    while (element) {
        if (element->schluessel == schluessel)
            return element;
        element = element->nachfolger;
    }
    return element;
}

void ausgebenTabelle(void) {
    printf("Index | Schluessel\n");
    for (int i = 0; i < ANZAHL; ++i) {
        const Element* ptr = tabelle[i];
        while (ptr != NULL) {
            printf(" %2d | %5d\n", i, ptr->schluessel);
            ptr = ptr->nachfolger;
        }
    }
}
```

```
int main(void) {
    initTabelle();
    einfuegen(25);
    einfuegen(16);
    einfuegen(85);
    einfuegen(3);
    ausgebenTabelle();

    Element* element = sucheElement(3);
    if (element) {
        printf("Element gefunden: %d\n", element->schluessel);
    } else {
        printf("Element nicht gefunden.\n");
    }

    return 0;
}
```

Die Ausgabe des Programmes lautet:

Index		Schluessel
3		25
3		3
5		16
8		85

Element gefunden: 3

20.3.7 Vor- und Nachteile der Hashverfahren

Je höher der Füllgrad (n/M) einer Hashtabelle ist, umso höher ist die Wahrscheinlichkeit einer Kollision.

Grundsätzlich ist zu vermeiden, dass die Hashtabellen zu voll werden, da dann bei allen Kollisionsbehandlungen der Tabellenzugriff sehr ineffizient wird.



Bei guten Verfahren – also guter Streuung der ersten berechneten Position und verschiedenen Indexfolgen bei Kollisionen – können Hashtabellen bis zu einer Auslastung von 80% bis 90% noch effizient arbeiten.

Wichtig bei allen Hashverfahren ist eine gute erste Schlüsseltransformation, welche die verschiedenen vorkommenden Schlüssel möglichst gleichförmig über die Hashtabelle verteilt.



Bei extrem schlechter Schlüsseltransformation – wenn beispielsweise alle Transformationen zur gleichen Position führen – nähert sich das Hashverfahren im Aufwand dem ineffizienten sequenziellen Suchen.



Löschoperationen können dazu führen, dass die Speicherstruktur nicht mehr ganz so effizient arbeitet. Einem zu löschenden Element ist grundsätzlich nicht anzumerken, ob andere Elemente mit diesem Element in Konflikt stehen und welche somit mittels Sondierung „nach“ dem zu löschenden Element eingefügt wurden. Aus diesem Grund kann bei einer Löschoperation das zu löschende Element nicht einfach entfernt werden, sondern muss durch eine Löschmarke ersetzt werden. Nur so kann gewährleistet werden, dass nachfolgende Suchoperationen andere Elemente, welche mit dem gelöschten Element in Konflikt standen, noch auffinden können. Bei Kollisionsauflösung mit direkter Verkettung ist dies kein Problem. Hier können gelöschte Elemente durch korrektes Setzen der Pointer einfach entfernt werden, ohne die Effizienz beim Suchen zu beeinträchtigen.

Ein weiterer Nachteil der Hashverfahren gegenüber anderen Suchmethoden ist der höhere Speicherbedarf, da Hashverfahren nur bei nicht voll besetzten Tabellen noch effizient arbeiten.

Man sieht an dieser Stelle wieder den sogenannten Time-Space-Tradeoff, einen der häufigsten Zielkonflikte beim Programmieren. Der Konflikt besteht darin, dass man eine Berechnung meist nur auf Kosten des Speicherverbrauchs beschleunigen kann, beziehungsweise dass man eingesparten Speicherplatz mit einer erhöhten Rechenzeit erkauft.

Bei den geschlossenen Verfahren muss die Größe der Tabelle vorher bekannt sein. Dies ist der Hauptnachteil dieser Verfahren. Wenn hingegen ein offenes Hashverfahren – beispielsweise mit direkter Verkettung – verwendet wird, ist eine variable Anzahl an Elementen möglich. Der Nachteil dabei ist aber, dass bei Häufungen der primären Positionen (verschiedene Schlüssel führen zu gleicher Position) die Effizienz sehr stark zurückgeht.

20.4 Suchen mit Hilfe von Backtracking

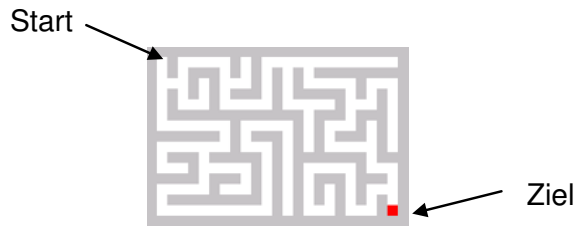
Nachdem am Beispiel des Sortierens zu sehen war, dass sich die Rekursion gut zum Erstellen von Algorithmen nach dem Prinzip „**teile und herrsche**“ eignet und auf diese Art leistungsfähige Algorithmen gebaut werden können, soll hier ein weiteres Problemlösungsprinzip beschrieben werden, das auf Rekursion beruht: das sogenannte „Backtracking“.

Backtracking beruht auf dem Prinzip „Versuch und Irrtum“: Es wird eine Möglichkeit ausgewählt und geprüft, ob der durch diese Auswahl eingeschlagene Weg zur Lösung führt. Wenn nicht, wenn also der Weg in einer Sackgasse endet, muss bis zu einer Stelle zurückgekehrt werden, bei der eine andere Möglichkeit ausgewählt und geprüft werden kann. Das englische Wort „to backtrack“ bedeutet auf Deutsch „zurückverfolgen“.



Viele wirtschaftliche und wissenschaftliche Probleme werden durch Backtracking-Algorithmen bearbeitet. Das Suchen optimaler Leiterplatten-Layouts und die optimale Routen- und Tourenplanung in einem Fuhrunternehmen seien hier als Beispiele genannt. Auch bei Strategiespielen wie beispielsweise Schach setzen Computer meist Backtracking-Algorithmen ein.

Als anschauliches Beispiel für eine Suche mittels Backtracking soll dabei die Suche des Ausganges in einem Labyrinth wie etwa in folgendem Bild dienen:



Eine einfache Möglichkeit, ein solches Labyrinth in C zu modellieren, ist, ein zwei-dimensionales char-Array anzulegen, in dem die Wände durch das #-Zeichen dargestellt werden. Das Ziel wird durch ein 'Z' markiert.

```
char zellen[ZEILEN][SPALTEN] = {
    "#####",
    "# #      # #      #",
    "# # ### # # # # #####",
    "# # # # # # # # #",
    "# # # # # # # # #",
    "# # # # # # # # #",
    "# ##### ### ## #",
    "# #      # # # # #",
    "##### # ### # # # # #",
    "# #      # # # # #",
    "# ### ##### # ##### #",
    "# #      # # # # #",
    "##### # # # # # # #",
    "# #      # # # # #",
    "# ##### # # # # #",
    "# #      # # # # #",
    "# ##### # # # # #",
    "# #      # # # #Z#",
    "#####",
};
```

Eine Grundidee des Backtrackings ist es, einen Pfad von Start bis Ziel als eine Folge von Schritten zu unterteilen, wobei jeder Schritt einen neuen potentiellen Startpunkt darstellt, von wo aus das Ziel erreicht werden kann. Somit muss an jedem Punkt auf dem Pfad sich nur noch die Frage gestellt werden: Wohin führt mich der nächste Schritt.

Wenn man bei der Suche an einem beliebigen Punkt steht, so hat man theoretisch 4 Möglichkeiten weiterzulaufen: nach Oben, Unten, Rechts oder Links. Entscheidet man sich für eine dieser 4 Möglichkeiten, so markiert man die derzeitige Zelle, die man soeben besucht hat, mit einem Zeichen, beispielsweise dem '0', um zu vermeiden, dass man später im Labyrinth wieder darüber stolpert und somit nicht im Kreis sucht.

Hat man den gewählten Schritt dann getätigt, nimmt man die damit erreichte Position als neue Startposition und setzt von da aus die Suche rekursiv fort. Man entscheidet sich also von dort aus wieder für eine der 4 Richtungen und macht den nächsten Schritt. Dies kann mittels eines rekursiven Aufrufs ein und derselben Funktion sehr einfach programmiert werden.

Nun kommt die zweite Grundidee des Backtrackings ins Spiel: Sobald ein Weg als nicht frei erkannt wird, wird die Rekursion gestoppt. Man muss wieder einen Schritt zurücktreten, sprich, an die aufrufende Funktion zurückkehren mit der Information, dass diese Richtung unbrauchbar war. Nachdem man den letzten Schritt rückgängig gemacht hat, also bei dem vorhergehenden Punkt steht, kann von dort aus eine der verbleibenden Möglichkeiten ausprobiert werden.

In dem hier vorgestellten Beispiel erfolgt das Zurückkehren zu einer vorherigen Situation – also das Backtracking – nicht explizit, sondern implizit beim Beenden eines rekursiven Aufrufs der Funktion.

Es gibt mehrere Gründe, warum eine Richtung unbrauchbar sein kann: Erstens kann in diese Richtung eine Wand (also ein #-Zeichen) sein. Zweitens kann man mit dem neuen Schritt eine Zelle besuchen, die man schon auf einem zuvor verfolgten Weg betreten hat, die also schon mit dem 0-Zeichen markiert wurde.

Die letzte Möglichkeit ist, dass sämtliche Richtungsmöglichkeiten (Oben, Unten, Rechts und Links) ausprobiert wurden und keine der Richtungen erfolgreich war. Auch dann wird die Rekursion gestoppt und an die aufrufende Funktion zurückgekehrt. Um den Fortschritt im folgenden Beispielprogramm sichtbar zu machen, wird die soeben besuchte Zelle jedoch noch mit 'X' markiert.

Die Suche hört erst auf, wenn man entweder einen Weg gefunden hat, der zum Ziel führt, oder aber keine der 4 Richtungen vom Startpunkt aus zum Ziel führen und somit der Algorithmus erfolglos zur `main()`-Funktion zurückkehrt.

Hier das Programmbeispiel:

laby.c

```
#include <stdio.h>

typedef enum Richtungen {
    RECHTS,
    OBEN,
    LINKS,
    UNTEN
} Richtung;

typedef enum Markierungen {
    WAND    = '#',
    FREI     = ' ',
    WEG      = 'O',
    IRRWEG   = 'X',
    ZIEL     = 'Z'
} Markierungen;

typedef enum SuchErgebnis {
    MISSERFOLG,
    ERFOLG
} SuchErgebnis;

#define ZEILEN 17
#define SPALTEN 25
char zellen[ZEILEN][SPALTEN] = {
    "#####",
    "# #      # #          #",
    "# # ### # # # # #####",
    "# # # # # # # # # #",
    "# # # # ### # ##### # # #",
    "# # # # # # # # # #",
    "# ##### ### ## #",
    "# # # # # # # # # #",
    "##### # ### # # ### # #",
    "# # # # # # # # # #",
    "# ### ##### # ##### ##",
    "# # # # # # # # # #",
    "##### ### # # # ### ## #",
    "# # # # # # # # # #",
    "# ##### # # # # # # #",
    "# # # # # # # #Z#",
    "#####",
};
```



```
void printLabyrinth(void) {
    for (int z = 0; z < ZEILEN; ++z) {
        for (int s = 0; s < SPALTEN; ++s) {
            printf("%c", zellen[z][s]);
        }
        printf("\n");
    }
    printf("\n");
}

SuchErgebnis suche(int z, int s) {
    if (zellen[z][s] == ZIEL) {
        return ERFOLG;
    }
    if (zellen[z][s] == FREI) {
        zellen[z][s] = WEG; // Weg markieren

        for (Richtung richtung = 0; richtung < 4; ++richtung) {
            SuchErgebnis ergebnis;
            switch (richtung) {
                case RECHTS: ergebnis = suche(z, s + 1); break;
                case OBEN:   ergebnis = suche(z - 1, s); break;
                case LINKS:  ergebnis = suche(z, s - 1); break;
                case UNTEN:  ergebnis = suche(z + 1, s); break;
            }
            if (ergebnis == ERFOLG)
                return ERFOLG;
        }

        zellen[z][s] = IRRWEG; // Misserfolg markieren
    }
    return MISSERFOLG;
}

int main(void) {
    SuchErgebnis ergebnis = suche(1, 1);
    printf(ergebnis == ERFOLG ? "Gefunden!\n" : "Meh.\n");
    printLabyrinth();
    return 0;
}
```

Hier die Ausgabe des Programms:

```
Gefunden!
#####
#0#00000#X#00000XXXXXXXXX#
#0#0###0#X#0# #0#####
#000# #0XX#0# #00000#XXX#
# # # #0###0# #####0#X#X#
# # #00000# #0XX#X#
# ##### #0###X#
# # # #000#X#X#
##### # ### # #0###X#X#
# # # # #00000#X#
# ### ##### # #####0###
# # # # #XXXXX#000#
##### ### # # #X###X###0#
# # # # #X#X#X#X#X#0#
# ##### # #X#X#X#X#0#
# # #XXX#XXX#Z#
#####
```

Von der Ausgangszelle `zellen[1][1]` ausgehend wird in diesem Beispiel in der `for`-Schleife zunächst in RECHTS-Richtung gesucht, also `suche(1,2)` aufgerufen. Da die Zelle `[1][2]` jedoch nicht frei, sondern mit dem Zeichen '#' belegt ist, wird sofort mit der Rückgabe `MISSERFOLG` zurückgewandert (Backtracking). Sodann wird in OBEN- und LINKS-Richtung gesucht, was ebenfalls in einer Wand endet.

Nach diesen drei Misserfolgen wird in die UNTEN-Richtung gewandert. Da die Zelle `[2][1]` frei ist, wird nun die Suche rekursiv fortgesetzt. Es wird erneut wieder in alle Richtungen gesucht, worauf man schlussendlich bei Zelle `[3][1]` landet. Dies wird solange fortgesetzt, bis das Ziel erreicht ist.

Etwas untypisch an diesem Beispiel für Backtracking ist dessen gutmütiges Laufzeitverhalten: Es ist proportional zur Anzahl der Zellen. Im Allgemeinen ist es jedoch so, dass Backtracking-Algorithmen exponentielles Laufzeitverhalten haben, da es nicht gelingt, hinreichend oft zurückzuwandern, also ganze Teile des zu durchsuchenden Raumes auszugrenzen. Ein Beispiel ist hier das Schachspiel, bei dem – wenn Rechner gegen Menschen antreten – enorme Rechnerleistungen eingesetzt werden, um dem exponentiellen Laufzeitverhalten Herr zu werden. Mehr zum Backtracking und dessen Analyse findet sich in [Sed92].

20.5 Übungsaufgaben

Aufgabe 1: Bubblesort

Ein beliebtes (aber sehr ineffizientes) Suchverfahren ist das sogenannte „**Bubblesort**“. Bei diesem Verfahren steigen quasi die großen Werte ganz allmählich wie Luftblasen im Array nach oben, während die kleinen Werte langsam „auf den Grund sinken“. Das Verfahren führt folgende Schritte aus:

1. Starte beim Array-Index 0.
2. Vergleiche den Wert des aktuellen Index i mit dem Wert des Index $i + 1$.
3. Wenn der erste Wert größer ist als der zweite, vertausche die beiden Werte. Ansonsten muss nichts vertauscht werden.
4. Wiederhole Schritte 2 und 3 an der jeweils nächsten Position $i + 1$, solange, bis der Index $n - 1$ erreicht wurde.
5. Starte daraufhin den gesamten Algorithmus erneut ab Schritt 1. Führe diese Wiederholung solange aus, bis in einem Durchlauf keine Vertauschungen mehr durchgeführt werden mussten.

Implementieren Sie diesen Algorithmus. Was sind die Vorteile und was sind die Nachteile gegenüber dem Sortieren mittels Selektion (siehe Kapitel [→ 20.1.1](#))?

Aufgabe 2: Analyse der Vergleiche und Bewegungen

Betrachten Sie die Programme für das Sortieren mittels Selektion aus Kapitel [→ 20.1.1](#), das Programm für Quicksort aus Kapitel [→ 20.1.3](#) und das Bubblesort-Programm aus der Übungsaufgabe 20.1.

- Erweitern Sie die Programme so, dass zu Beginn des Programmes das Array mit einer beliebig großen Anzahl n Elementen angelegt und automatisch mit zufälligen Werten gefüllt wird. Nutzen Sie hierfür die `malloc()`- und `rand()`-Funktionen der `<stdlib.h>`-Bibliothek.
- Erweitern Sie die Programme, sodass die Anzahl Vergleiche und Bewegungen in einer globalen Variable gezählt, und am Ende des Algorithmus ausgegeben werden.
- Testen Sie die Algorithmen mit verschiedenen Array-Größen und machen Sie sich einen Überblick, wie sich die verschiedenen Algorithmen verhalten.

Aufgabe 3: Löschen bei Hashtabellen mit Liste

Im Kapitel [→ 20.3.6](#) wurde das Hashverfahren mithilfe einer linearen Liste vorgestellt. Erweitern Sie die Programmierung, sodass Elemente auch wieder gelöscht werden können.

Aufgabe 4: Lösen von Sudokus mit Backtracking

Wie zum Finden von Wegen im Labyrinth eignet sich Backtracking auch sehr gut, um Sudoku-Rätsel zu lösen. Ein Sudoku sieht zum Beispiel so aus:

			5	4	2			
9	6	7						
						3	1	8
				7		8	6	4
	2		6		4		9	
6	4	5		1				
8	9	1						
						5	4	7
			3	2	6			

Die Regeln für die korrekte Lösung sind einfach: Die leeren Felder in der 9×9-Matrix sollen so mit den Ziffern 1...9 gefüllt werden, dass ...

- ... in keiner Zeile eine Ziffer doppelt vorkommt.
- ... in keiner Spalte eine Ziffer doppelt vorkommt.
- ... in keinem der fett gedruckten 3×3 Blöcke eine Ziffer doppelt vorkommt.

Ziel der Aufgabe ist es, mittels eines Backtracking-Algorithmus die Lösung für ein 9×9-Sudoku zu berechnen.

Der Backtracking-Algorithmus soll versuchen, in jeder noch nicht gefüllten Zelle die Werte 1...9 durchzuprobieren. Die Prüfung, ob es sich dabei um einen gültigen Wert handelt, ist etwas komplizierter als beim Labyrinth, bei dem nur geprüft werden musste, ob die Zelle frei ist. Hier bietet es sich an, zunächst Funktionen auszuprogrammieren, welche prüfen, ob der einzutragende Wert in einer Zeile, einer Spalte und in einem Block überhaupt erlaubt ist:

```
int istZeileOK (int zeile, char wert);
int istSpalteOK(int spalte, char wert);
int istBlockOK (int zeile, int spalte, char wert);
```

Diese drei Funktionen müssen alle ihr OK geben, bevor der Algorithmus rekursiv versucht, einen Wert 1 . . . 9 einzutragen. Wenn alle Ziffern 1 . . . 9 erfolglos ausprobiert wurden wird die Suche abgebrochen und in der aufrufenden Suche die nächste mögliche Zahl ausprobiert.

Das Sudoku und auch dessen Lösung sollen dabei wie das Labyrinth in einem zweidimensionalen char-Array abgelegt werden:

```
char zellen[9][9] = {
    "  542  ",
    "967    ",
    "      318",
    "    7 864",
    " 2 6 4 9 ",
    "645 1    ",
    "891     ",
    "      547",
    "   326   ",
};
```

Die Lösung sollte folgendermaßen aussehen:

```
+ + + + +
| 1 8 3 | 5 4 2 | 6 7 9 |
| 9 6 7 | 1 8 3 | 4 5 2 |
| 4 5 2 | 7 6 9 | 3 1 8 |
+ + + + +
| 3 1 9 | 2 7 5 | 8 6 4 |
| 7 2 8 | 6 3 4 | 1 9 5 |
| 6 4 5 | 9 1 8 | 7 2 3 |
+ + + + +
| 8 9 1 | 4 5 7 | 2 3 6 |
| 2 3 6 | 8 9 1 | 5 4 7 |
| 5 7 4 | 3 2 6 | 9 8 1 |
+ + + + +
```

Zusatzaufgabe: Wenn Sie die drei Ziffern in der ersten Zeile weglassen, gibt es insgesamt 3 mögliche Lösungen. Bringen Sie ihr Programm dazu, alle drei Lösungen auszugeben.

Beim Aufruf eines C-Compilers wird vor den eigentlichen Compilerläufen wie Syntax- und Semantik-Prüfung sowie Code-Generierung der **Präprozessor** (auf Englisch „**preprocessor**“) gestartet.



Die wichtigsten Aufgaben des Präprozessors sind:

- Einfügen von Dateien
- Ersetzen von Text
- Bedingte Compilierung

Daneben stellt der Präprozessor Informationen über den Übersetzungskontext bereit, die beispielsweise zur Ausgabe von Fehlermeldungen sehr nützlich sein können.

Die allgemeine Syntax für eine **Präprozessor-Direktive** lautet:

```
#Direktive Text
```

Eine Präprozessor-Direktive wird auch Präprozessor-Befehl oder Präprozessor-Anweisung genannt. Sie beginnt grundsätzlich mit einem Nummernzeichen #. Vor dem Nummernzeichen können beliebige Whitespace-Zeichen stehen. Zwischen dem Nummernzeichen und der Direktive und ebenso zwischen der Direktive und dem Text können Leerzeichen, Tabulatorzeichen oder Kommentare stehen. Es hat sich jedoch durchgesetzt, dass der Befehl direkt auf das Nummernzeichen folgt.

Eine Präprozessor-Direktive kann an einer beliebigen Stelle im Source-Code eingeschoben sein, muss jedoch in einer eigenen Zeile stehen.



Im normalen C-Quellcode kann aus Gründen der Lesbarkeit eine Anweisung über mehrere Zeilen verteilt werden. Der Präprozessor jedoch arbeitet zeilenorientiert.

Ergänzende Information Die elektronische Version dieses Kapitels enthält Zusatzmaterial, auf das über folgenden Link zugegriffen werden kann https://doi.org/10.1007/978-3-658-45209-4_21.

Eine Präprozessor-Direktive endet mit dem Zeilenende-Zeichen, während eine normale C-Anweisung durch einen Strichpunkt abgeschlossen wird.

Ist eine Zeile für eine Präprozessor-Direktive nicht ausreichend oder soll aus optischen Gründen die Zeile anders formatiert werden, so kann durch Abschluss der Zeile mit einem Backslash \ die Präprozessor-Direktive in der nächsten Zeile weitergeführt werden.

Bevor die einzelnen Präprozessor-Direktiven erklärt werden, ist es wichtig, die Arbeitsweise des Präprozessors und vor allem die Reihenfolge, in der der Präprozessor seine Aufgaben durchführt, zu verstehen.

Entscheidend ist vor allem, dass der Präprozessor nichts mit der eigentlichen Compilierung des Source-Codes zu tun hat, sondern nur Vorbereitungen hierzu durchführt.



Der Präprozessor kann im Prinzip mit einem Textverarbeitungsprogramm verglichen werden, welches Quellcode an gewissen Stellen hinzufügt oder ersetzt.

Die folgenden Aufgaben werden durch den Präprozessor der Reihe nach vor Beginn des Compilierlaufs ausgeführt:

- Falls erforderlich, Ersetzung von Zeilenende-Steuerzeichen in einer Datei durch das Zeichen Newline. Entfernen des Fortsetzungszeichens \ und Zusammenfügen der Zeilen der Präprozessor-Anweisungen.
- Zerlegung eines Programms in Präprozessor-Token, die durch Whitespace-Zeichen getrennt sind. Kommentare werden durch Leerzeichen ersetzt.
- Bearbeiten der Präprozessor-Direktiven (Einfügen von Dateien, Ersetzen von Text).
- Wird mit `#include` eine Datei eingefügt, so werden in ihr ebenfalls alle hier aufgelisteten Aufgaben von vorne ausgeführt.
- Ersetzen von Ersatzdarstellungen in Zeichen-Konstanten und String-Konstanten durch die entsprechenden Zeichen.
- Zusammenfügen von benachbarten Strings.

Nach diesen Schritten liegen die lexikalischen Einheiten (Token) eines Programms vor.

21.1 Einfügen von Dateien

Das Einfügen einer beliebigen Datei erfolgt mit Hilfe der **#include**-Direktive:

```
#include "filename"  
#include <filename>
```

Ist filename innerhalb der #include-Direktive in spitzen Klammern <> eingeschlossen, so sucht der Compiler in den Standard-Include-Verzeichnissen.



Unter UNIX ist das Standard-Include-Verzeichnis meist /usr/include und unter dem Visual C++ Compiler \vc\include. Die Compiler erlauben auch eine Erweiterung der Standard-Include-Verzeichnisse mit Hilfe von Compileroptionen.

Setzt man in der #include-Direktive den Dateinamen in Anführungszeichen, so wird zuerst in dem aktuellen Arbeitsverzeichnis beziehungsweise in dem in der #include-Direktive aufgeführten Verzeichnis gesucht. Wird die entsprechende Datei dort nicht gefunden, so wird die Suche in den Standard-Include-Verzeichnissen fortgesetzt.



Beim Einfügen einer Datei wird die Zeile mit der Präprozessor-Direktive entfernt und stattdessen der Inhalt der Datei eingefügt.

Die #include-Direktive wird hauptsächlich für Header-Dateien verwendet, welche vom Präprozessor daraufhin ebenfalls bearbeitet werden.

Befindet sich innerhalb der eingefügten Datei erneut eine #include-Direktive, so wird diese ebenfalls ausgeführt.



Das Einfügen wird dabei in einer temporären, nur für den Compilierlauf angelegten Datei durchgeführt. Die ursprüngliche Datei bleibt erhalten.

Auch wenn mit `#include` beliebige Dateien eingefügt werden können, muss darauf geachtet werden, dass am Ende des Einfüge-Prozesses eine übersetzbare Einheit in der erzeugten temporären Datei vorhanden sein muss.



Das Einfügen von Dateien wird meistens verwendet, um Konstanten (`#define`-Direktive), Deklarationen von Typen und Funktionen, Makros und gegebenenfalls weitere `#include`-Direktiven in einen Quellcode einzubinden. Als Beispiel kann hier die Bibliothek `<stdio.h>` genommen werden. Unter anderem sind in `<stdio.h>` folgende Sprachelemente enthalten:

- Definitionen von Typnamen wie `size_t`, `FILE`, ...,
- Konstantendefinitionen für die Konstanten `EOF`, `stdin`, `stdout`, ...,
- Prototypen für die Funktionen `printf()`, `scanf()`, `fopen()`, ...
- und Makros wie `getc()`, `putc()`, ...

Nach dem gleichen Prinzip können aber auch eigene Header-Dateien geschrieben werden, die über eine `#include`-Direktive in den Quellcode eingebunden werden können. Dazu mehr in Kapitel [→ 22](#).

21.1.1 Einfügen von Binär-Dateien

Ab dem Standard C23 gibt es die neue Direktive `#embed`, welche genauso wie die `#include`-Direktive eine Datei direkt in den Quellcode einbindet. Hierbei wird die Datei jedoch binär eingebunden, was bedeutet, dass beispielsweise keine automatische Konvertierung von Zeilenenden stattfindet und die Datei auch nicht vom Präprozessor weiter verarbeitet wird. Nützlich ist dies beispielsweise, wenn ein Array mit Daten gefüllt werden soll.

Des Weiteren wird in C23 der Operator `__has_embed` eingeführt, welcher es erlaubt, während der Compilierzeit zu ermitteln, ob eine Datei verfügbar ist.

21.2 Symbolische Konstanten und Makros mit Parametern

Mit der Präprozessor-Direktive **#define** wird das Ersetzen von Text festgelegt:

```
#define Bezeichner Ersatztext
```

Der Bezeichner wird infolge dieser Direktive durch die beliebige Folge von Zeichen in Ersatztext ersetzt.

Symbolische Konstanten haben einen Namen, der den einzusetzenden Quellcode repräsentiert. Sie werden in C allgemein als „**Makros**“ bezeichnet.



Der Ersatztext ist der Text bis zum Zeilenende. Soll ein Ersatztext aus mehreren Zeilen definiert werden, so kann dies mit dem Backslash \ am Zeilenende erfolgen. Das Ersetzen von Text findet während des Präprozessor-Laufs ab der Stelle der #define-Direktive bis zum Dateiende statt. Ersetzt werden nur komplette Namen. Nicht ersetzt wird:

- Text, der als Substring in einem Namen enthalten ist.
- Text innerhalb von String-Konstanten.

Gegeben sei folgendes Makro:

```
#define HUND DOG
```

Diese Direktive bewirkt, dass jeder deutsche HUND in einen englischen DOG umgewandelt wird. Der Bezeichner HUNDERT hingegen bleibt unverändert, da HUND zum Ersetzen als eigenes Wort stehen muss. Genauso bleibt beispielsweise der String "Wo ist der HUND" unangetastet.

Durch die Präprozessor-Direktive **#undef** kann eine zuvor durch #define eingeführte Makrodefinition wieder aufgehoben werden.

```
#undef Bezeichner
```

Da ohne den Einsatz von `#undef` ein Makro immer ab der Stelle, an der es definiert wurde, bis zum Dateiende gilt, hat man mit einer Kombination von `#define` und `#undef` die Möglichkeit, die Gültigkeit nur auf einen bestimmten Abschnitt der Datei zu begrenzen.

Die erlaubten Zeichen eines Präprozessor-Bezeichners sind dieselben wie die für einen Variablennamen (siehe Kapitel [→ 6.4](#)).

In diesem Kapitel werden zwei Arten von Makros unterschieden: Einfache symbolische Konstanten und Makros mit Parametern.

21.2.1 Symbolische Konstanten

Generell wird das, was durch `#define` definiert wird, als Makro bezeichnet. Ein Makro ohne Parameter wird auch als „**Symbolische Konstante**“ bezeichnet. Folgendes ist ein typisches Beispiel:

```
#define PI 3.1416
```

PI ist eine symbolische Konstante, die Zahl 3.1416 eine literale Konstante. Es hat sich eingebürgert, die Namen von symbolischen Konstanten stets groß zu schreiben.

Mit der `#define`-Direktive wird also eine symbolische Konstante mit dem Namen PI eingeführt, die als Wert die literale Konstante 3.1416 hat. Das bedeutet, dass überall nach dieser Direktive im Code, wo der Bezeichner PI steht, die literale Konstante 3.1416 zu stehen kommt.

Symbolische Konstanten sind wichtig, wenn in einem Programm wiederkehrende Werte verwendet werden sollen. Der Vorteil von Makros ist, dass sie im Gegensatz zu den in Kapitel [→ 6.5](#) vorgestellten literalen Konstanten einen Namen besitzen. Wäre die Zahl 3.1416 überall im Code als literale Konstante geschrieben, wäre es schwierig, ein solches Programm in korrekter Weise abzuändern, wenn sich der Wert ändern sollte, beispielsweise weil eine weitere Kommastelle für verbesserte Genauigkeit hinzugefügt werden würde: 3.14159. Sehr leicht könnte eine der zu ändernden Stellen vergessen werden. Wird eine symbolische Konstante verwendet, so erfolgt eine Änderung eines wiederkehrenden Wertes an einer einzigen zentralen Stelle.

Überall da, wo von der Syntax her Konstanten erlaubt sind, können auch konstante Ausdrücke stehen.



Ein konstanter Ausdruck ist ein Ausdruck, an dem nur Konstanten beteiligt sind. Solche Ausdrücke werden vom Compiler zur Compilierzeit und nicht zur Laufzeit bewertet.

Überall da, wo von der Syntax Konstanten oder konstante Ausdrücke erlaubt sind, kann man literale Konstanten oder symbolische Konstanten einsetzen.



21.2.2 Makros mit Parametern

Um einem Makro einen oder auch mehrere **Parameter** übergeben zu können, muss eine Parameterliste nach dem Bezeichner in folgender Form aufgeführt werden:

```
#define Bezeichner(Parameterliste) Ersatztext
```

Wichtig ist dabei, dass zwischen Bezeichner und der öffnenden Klammer kein Whitespace-Zeichen stehen darf, da sonst diese Klammer mitsamt der Parameterliste als Ersatztext interpretiert wird.

Die Parameter in der Parameterliste werden durch Kommas getrennt. Es handelt sich hierbei jedoch nicht um Variablen mit Typen, wie man es von den Übergabeparametern von Funktionen her kennt, sondern um Parameternamen, welche die übergebenen Textfragmente bezeichnen. Einem parametrisierten Makro kann somit auch einen Ausdruck wie beispielsweise $(a+1)$ übergeben werden. Ein solcher Ausdruck wird durch den Präprozessor nicht ausgewertet, sondern mittels Copy und Paste an die gewünschte Stelle im Ersatztext eingefügt.

Das Ersetzen eines Makros in einer Datei erfolgt somit, indem die Parameter des Makros durch die aktuellen Parameter ersetzt werden und der dadurch entstandene Ersatztext wiederum an die Stelle kopiert wird, wo das Makro auftritt.

Gegeben sei beispielsweise die folgende Makro-Definition:

```
#define reihenSumme(z) z * (z + 1) / 2
```

Wenn im Code nun folgendes steht:

```
int a = 5;  
int b = reihenSumme(a);
```

So wandelt der Präprozessor dies wie folgt um:

```
int a = 5;  
int b = a * (a + 1) / 2;
```

Der Text des Makros wird an der passenden Stelle eingesetzt und der Parameter z wird durch den aktuellen Parameter a ersetzt.

21.2.3 Komplikationen bei der Textersetzung und ihre Beseitigung

Bei einem rein textuellen Ersetzen von Parametern können einige Komplikationen auftreten, hierzu ein paar Beispiele:

makro.c

```
#include <stdio.h>  
#define square(x) x * x  
#define twotimes(x) x + x  
  
int main(void) {  
    int zahl = 4;  
    printf("zahl = %d\n", zahl);  
    printf("1) square(5) = %d\n", square(5));  
    printf("2) square(5+1) = %d\n", square(5+1));  
    printf("3) square(zahl) = %d\n", square(zahl));  
    printf("4) square(zahl++) = %d\n", square(zahl++));  
    printf("5) zahl = %d\n", zahl);  
    printf("6) twotimes(5) = %d\n", twotimes(5));  
    printf("7) twotimes(5) * twotimes(6) = %d\n",  
           twotimes(5) * twotimes(6));  
    return 0;  
}
```

Die Ausgabe des Programms ist:

```
zahl = 4
1) square(5) = 25
2) square(5+1) = 11
3) square(zahl) = 16
4) square(zahl++) = 20
5) zahl = 6
6) twotimes(5) = 10
7) twotimes(5) * twotimes(6) = 41
```

Die Berechnungen bei Punkt 2, 4, 5 und 7 entsprechen somit nicht dem intuitiv erwarteten Resultat. Hierbei muss man sich in Erinnerung rufen, dass Makros keine Funktionsaufrufe, sondern reine Textersetzungen sind:

1. $5 * 5 = 25$
2. $5 + 1 * 5 + 1 = 11$
3. $zahl * zahl = 16$
4. $zahl++ * zahl++ = 20$
5. $zahl = 6$
6. $5 + 5 = 10$
7. $5 + 5 * 6 + 6 = 41$

Somit wird klar, dass bei Punkt 2 die Punkt-vor-Strich-Regel zu dem unerwarteten Ergebnis führt. Bei den Punkten 4 und 5 wird ersichtlich, dass die Zahl zweimal hintereinander inkrementiert wird. Bei Punkt 7 führt wiederum die Punkt-vor-Strich-Regel zu einem unerwarteten Ergebnis.

Das Problem bei Punkt 2 kann durch den Einsatz von Klammern um die Makroparameter verhindert werden:

```
#define square(x) (x) * (x)
```

Die Ausgabe des Programms ist dann:

```
2) square(5+1) = 36
```

Das Problem bei den Punkten 4 und 5 ist dadurch jedoch nicht gelöst. Die unerwarteten Ausgaben folgen daher, dass bei Punkt 4 der Inkrement-Operator durch die Textersetzung zweimal in den Code geschrieben und somit auch zweimal ausgeführt wird. Es ist nicht definiert, in welcher Reihenfolge der Compiler die Berechnung und die Auswertung der Nebeneffekte durchführt. Ein anderer Compiler oder bereits eine andere Compiler-Version kann somit andere Ergebnisse liefern. Es sollte deshalb – wenn immer möglich – vermieden werden, Ausdrücke mit Nebeneffekten bei Makros zu verwenden. Genauere Informationen zu Nebeneffekten können in Kapitel [→ 9.1.1](#) nachgelesen werden.

Das Problem bei Punkt 7 kann ebenfalls durch Klammern gelöst werden. Es werden hier jedoch Klammern um den gesamten Ausdruck verwendet:

```
#define twotimes(x) ((x) + (x))
```

Die Ausgabe des Programms ist dann:

```
7) twotimes(5) * twotimes(6) = 120
```

21.3 Bedingte Compilierung

Bedingte Compilierung bedeutet, dass zur Compilierzeit anhand von Ausdrücken und Symbolen entschieden wird, welcher Teil des Source-Codes compiliert werden soll. Diese Aufgabe übernimmt der Präprozessor, indem er Source-Code entfernt, welcher die gegebenen Bedingungen nicht erfüllt.



Zur bedingten Compilierung stehen folgende Präprozessor-Direktiven zur Verfügung:

```
#if konstanterAusdruck
#elif konstanterAusdruck
#else
#endif

defined

#ifdef Symbol
#endif
#ifdef Symbol
#elifdef Symbol (ab C23)
#elifndef Symbol (ab C23)
```

Die bedingte Compilierung wird beispielsweise für Programme benutzt, die auf unterschiedlichen Betriebssystemen oder Prozessoren laufen sollen. Benutzt ein Programm außer der Standardbibliothek von C noch andere Bibliotheken wie beispielsweise die der Bedienoberfläche, so können diese je nach Betriebssystem verschieden sein. Ebenso kann mit der bedingten Compilierung eine Testversion eines Programms mit besonderen Trace-Ausgaben generiert oder maschinenabhängige Programmteile (Programmteile, die speziell auf die jeweiligen Prozessoren zugeschnitten sind) gesondert programmiert werden. Es ist also möglich, mit Hilfe der bedingten Compilierung mehrere Versionen eines Programms aus einem einzigen Source-Code zu generieren. Dies erleichtert die Pflege und Wartung der Software erheblich.

In Kapitel [→ 22.3.4](#) kann zudem eine sehr gebräuchliche Methode nachgelesen werden, welche bedingte Compilierung verwendet, um sicherzustellen, dass eine Header-Datei nur ein einziges Mal in den Programmcode eingebunden wird.

Die Präprozessor-Direktiven **#if**, **#elif**, **#else**, **#endif** entsprechen einer **#if**-Struktur in C. **#elif** und **#else** sind dabei optional. **#if** und **#elif** müssen einen konstanten Ausdruck als Bedingung erhalten. Der Wert des konstanten Ausdrucks wird wie in der **#if**-Anweisung in C als wahr oder falsch interpretiert (falsch ist gleich 0, wahr ist ungleich 0). Der Präprozessor entfernt alle Programmenteile, die in einem Zweig enthalten sind, der als falsch interpretiert wird.

Das Schlüsselwort **defined** ist ein Operator und keine Direktive, dementsprechend gibt es kein **#**. Es ermittelt für einen gegebenen Namen, ob ein entsprechendes Makro definiert ist. Beispielsweise wird mit folgendem Ausdruck ermittelt, ob ein Programm für die Windows-Plattform compiliert wird:

```
#if defined _WIN32
```

Die Direktiven **#ifdef** und **#ifndef** sind Abkürzungen für die Ausdrücke **#if defined** und **#if !defined**. Ab dem Standard C23 gibt es auch die Direktiven **#elifdef** und **#elifndef**, welches Abkürzungen sind für die Ausdrücke **#elif defined** und **#elif !defined**.

Hier ein Beispiel zum Erzeugen einer Programmversion für ein 16-Bit oder ein 32-Bit Betriebssystem:

systemComparison.c

```
#include <stdio.h>
#include <limits.h>

#define TESTVERSION 1

int main(void) {
    #if UINT_MAX > 65535
        int i;
    #else
        long i;
    #endif

    #if TESTVERSION
        printf("Testausgabe: sizeof (int) = %d\n", (int)sizeof(int));
        printf("Testausgabe: sizeof (long) = %d\n", (int)sizeof(long));
    #endif

    i = 300;
    printf("%ld * %ld = %ld\n", (long)i, (long)i, (long)(i*i));
    return 0;
}
```

Die Ausgabe des Programms ist auf einem 16-Bit Betriebssystem beispielsweise:

```
Testausgabe: sizeof(int)  = 2
Testausgabe: sizeof(long) = 4
300 * 300 = 90000
```

Und auf einem 32-Bit Betriebssystem beispielsweise:

```
Testausgabe: sizeof(int)  = 4
Testausgabe: sizeof(long) = 8
300 * 300 = 90000
```

Es werden in diesem Beispiel zwei bedingte Compilierungen verwendet. Zum einen wird durch die symbolische Konstante `TESTVERSION` definiert, ob eine Debugging-Ausgabe auf den Bildschirm ausgegeben werden soll.

Anstatt das Makro `TESTVERSION` auf einen bestimmten Wert zu prüfen, kann mit dem Präprozessor-Operator `defined` geprüft werden, ob `TESTVERSION` überhaupt definiert ist:

```
#define TESTVERSION
...
#if defined TESTVERSION
    printf("Testausgabe: ...
#endif
```

Der Ausdruck `#if defined TESTVERSION` kann zudem auch folgendermaßen geschrieben werden:

```
#ifdef TESTVERSION
```

Dies ist eine einfachere Schreibweise und wird gerne verwendet. Mit den Direktiven `#ifdef` und `#ifndef` kann also festgestellt werden, ob ein bestimmtes Makro definiert wurde oder nicht. `#ifdef` steht für `#if defined` und `#ifndef` steht für `#if !defined`.

21.4 Informationen über den Übersetzungskontext

Um Informationen über den Übersetzungskontext während der Übersetzung selbst in das Programm übernehmen zu können, gibt es vordefinierte Makros.



Folgende vordefinierten Makros existieren bei jedem standardisierten Compiler:

<code>__DATE__</code>	Datum der Compilierung des Programms (String-Konstante in der Form "Mmm tt jjjj")
<code>__TIME__</code>	Uhrzeit der Compilierung des Programms (String-Konstante in der Form "hh:mm:ss")
<code>__STDC__</code>	= 1, wenn Programm in Übereinstimmung mit dem Standard compiliert wurde
<code>__STDC_VERSION__</code>	Ist definiert als eine Zahl, welche den Standard beschreibt, mit welchem compiliert wurde. Folgende Zahlen sind in Verwendung: 199409L für C95, 199901L für C99 und 201112L für C11
<code>__FILE__</code>	Name der Quelldatei (String-Konstante)
<code>__LINE__</code>	Nummer der Zeile im Quellcode (Integer-Konstante)

Die Makros `__FILE__` und `__LINE__` sind insbesondere zur Ausgabe von Fehlermeldungen sehr nützlich.

21.4.1 Die Präprozessor-Direktive `#line`

Die `#line`-Direktive hat folgende Syntax:

```
#line Konstante [Dateiname]
```

Damit können die Inhalte der vordefinierten Makros `__LINE__` und `__FILE__` gezielt verändert werden.

Beispielsweise kann man mit der folgenden Direktive bewirken, dass ab dieser Stelle der Dateiname `hallo.c` lautet und die Zeilenzahl wieder bei 0 beginnt.

```
#line 0 "hallo.c"
```

21.4.2 Die Präprozessor-Direktiven `#error` und `#warning`

```
#error Fehlertext
```

Die Präprozessor-Direktive `#error` erzeugt einen Fehler mit entsprechendem Fehlertext beim Compilieren des Programms. Diese Direktive kann beispielsweise bei bedingter Compilierung sinnvoll eingesetzt werden, wenn verhindert werden soll, dass das Programm auf einer Hardware compiliert wird, für die es nicht vorgesehen ist.

Genauso erzeugt die Direktive `#warning` eine Warnung, bei welcher zwar ein Fehlertext ausgegeben wird, die Compilierung jedoch nicht wie bei einem Fehler abbricht. Diese Direktive ist offiziell erst ab dem Standard C23 verfügbar, viele Compiler unterstützen sie jedoch bereits heute.

Das folgende Beispiel zeigt eine Anwendung der `#error`-Direktive. Zuerst wird geprüft, ob der benutzte Compiler sich nach dem C-Standard verhält (Makro `__STDC__`). Ist dies nicht der Fall, wird eine Fehlermeldung ausgegeben und die Übersetzung abgebrochen:

```
#if (__STDC__ != 1)
    #error "Compiler ist kein Standardcompiler!"
#endif
```

21.5 String-Umwandlungs- und Verknüpfungs-Operatoren

Die Verwendung von parametrisierten Makros kann helfen, Schreibarbeit zu sparen. Im Gegensatz zu Funktionen haben sie den Vorteil, dass sie tatsächlich Textersetzungen sind, und somit während des Compilierens neuen Code generieren.

Um Makros flexibler gestalten zu können, gibt es hierfür zusätzliche Präprozessor-Operatoren, um aus den Parametern eines parametrisierten Makros korrekten Quellcode zu erzeugen.

21.5.1 Parameter mit # in Strings umwandeln

Parameter, vor denen der Präprozessor-Operator # steht, werden wie normale Parameter durch die aktuellen Parameter ersetzt. Dabei wird aber das Ergebnis anschließend als String-Konstante behandelt. Hierfür ein Beispiel:

```
#define Varausgabe(a) printf(#a " = %d\n", a)
```

Die Programmzeilen

```
long z = 123456;  
Varausgabe(z);
```

werden mit Hilfe des Präprozessors ersetzt durch

```
long z = 123456;  
printf("z" " = %d\n", z);
```

Das z wurde also in einen String "z" umgewandelt. Der Präprozessor wird zu dem Strings, welche direkt hintereinander stehen, ohne Abstand miteinander verknüpfen. So entsteht schlussendlich folgender Code:

```
long z = 123456;  
printf("z = %d\n", z);
```

21.5.2 Parameter mit ## verknüpfen

Mit Hilfe des Präprozessor-Operators ## können dynamisch Bezeichner gebildet werden, wobei die Argumente zu einem einzigen Bezeichner verkettet werden.



Die Argumente dürfen jedoch selbst keine Makros sein. Bei dem Ersetzungsvorgang während des Präprozessor-Laufs werden der Operator ## und die umgebenden Whitespace-Zeichen entfernt. Das resultierende Token wird erneut geprüft, ob es nochmals einer Ersetzung unterzogen werden kann.

Folgendes ist ein einfaches Beispiel zum Präprozessor-Operator ##:

dynamicSymbol.c

```
#include <stdio.h>

#define DYN_SYMB(a, b) a ## b

int main (void) {
    int i1 = 1, i2 = 2;
    printf("i1 = %d\n", DYN_SYMB (i, 1));    // Verkettung zu i1
    printf("i2 = %d\n", DYN_SYMB (i, 2));    // Verkettung zu i2
    return 0;
}
```

Die Ausgabe des Programms ist:

```
i1 = 1
i2 = 2
```

21.5.3 Makros mit beliebigen Parametern ...

Es gibt die Möglichkeit, ein Makro zu definieren, ohne die genaue Anzahl an Parametern festzulegen. Diese sogenannten „variadischen“ Makros nutzen das Makro __VA_ARGS__ und ab dem Standard C23 auch __VA_OPT__, um auf die ab den Auslassungspunkten ... angegebenen Parameter zuzugreifen. Beispielsweise:

```
#define PRINT(string, ...) printf(string, __VA_ARGS__)
```

21.6 Präprozessor-Erweiterungen unter C99 und C11

Auch wenn die grundlegenden Funktionen des Präprozessors in allen C-Standards gleich geblieben sind, haben die Standards C99 und C11 jeweils den Funktionsumfang ein wenig erweitert. So wurde es mit der Zeit einfacher, auf Umgebungsinformationen zuzugreifen und entsprechend die Compilierung des Codes besser zu steuern.

21.6.1 Der Pragma-Operator

Ein Pragma dient dazu, um in einer Programmiersprache Informationen an den Compiler zu geben. Hierfür gibt es sowohl die Präprozessor-Direktive **#pragma** als auch seit C99 den Operator `_Pragma`.



In der Programmiersprache C konnten ursprünglich nur mit der `#pragma`-Direktive zusätzliche Informationen an den Compiler gegeben werden. Da die Direktive `#pragma` jedoch als einzelne Zeile geschrieben werden muss und nicht als Resultat einer Makro-Expansion generiert werden kann, führte bereits der C99-Standard den Operator `_Pragma` für den Präprozessor ein. Der `_Pragma`-Operator kann in einem Makro verwendet werden.

Die Verwendung der `#pragma`-Direktive sowie des `_Pragma`-Operators ist jedoch nicht standardisiert. Beides sind Compiler-spezifische Angaben.

21.6.2 Generische Auswahl

In Programmiersprachen wie C++ oder Java hat man die Möglichkeit, Funktionen zu überladen. Erlaubt eine Sprache das Überladen von Methoden (auf Englisch „overloading“), so können Methoden mit verschiedenen Parameterlisten denselben Namen tragen. Der Aufruf der richtigen Methode ist Aufgabe des Compilers. Anhand des Datentyps des Übergabeparameters erkennt der Compiler, welche der Methoden gemeint ist. Der Nutzen des Überladens von Methoden ist, dass man gleichartige Methoden mit dem gleichen Namen ansprechen kann. Die Verständlichkeit der Programme kann dadurch erhöht werden.

Der Standard C11 führt die sogenannten Generic Selections ein, die eine ganz ähnliche Funktionalität wie das Überladen beispielsweise in C++ bieten.



Ein echtes Überladen von Funktionen findet damit zwar nicht statt, es wird allerdings eine passende Funktion aufgrund des übergebenen Typs ausgewählt. Mit dem Schlüsselwort `_Generic` lassen sich Makros definieren, über die eine Fallunterscheidung aufgrund des Typs des Arguments einer Funktion ermöglicht wird.

Das folgende Beispiel zeigt die Verwendung des Makros `_Generic`:

generic.c

```
#include <math.h>
#include <stdio.h>

#define sinus(X) _Generic ((X),
                           default: sin,
                           long double: sinl,
                           float: sinf
                           )(X)

int main(void) {
    int i = 4;
    float f = 1.2f;
    double d = 3.2;

    printf("%f\n", sinus(d)); // ruft Defaultwert sin(d) auf
    printf("%f\n", sinus(f)); // ruft sinf(f) auf
    printf("%f\n", sinus(i)); // ruft Defaultwert sin(i) auf

    return 0;
}
```


Vor der `main()`-Funktion ist die Definition des Makros `sinus(X)` zu sehen. Das `X` dient dabei wie gewohnt als Parameter, welcher an die Funktion weitergereicht wird. Hinter dem Schlüsselwort `_Generic` hingegen dient das `X` als Platzhalter für den übergebenen Datentyp. Hier finden die jeweiligen Zuweisungen von Datentyp und Funktion statt. Innerhalb der `main()`-Methode sind beispielhafte Aufrufe zu sehen. Wird `sinus(X)` beispielsweise ein Argument vom Typ `float` übergeben, dann wird aufgrund des oben definierten Makros automatisch die Funktion `sinf()` ausgewählt und aufgerufen. Passt kein Typ zum übergebenen Argument, dann wird die Funktion aufgerufen, die unter `default` hinterlegt ist.

Äußerst praktisch ist diese Funktionalität, wenn man viele mathematische Funktionen verwendet. Damit muss man sich nur einmal zu Beginn des Programms Gedanken darüber machen, bei welchem Datentyp welche Funktion verwendet werden muss. Für den weiteren Verlauf lassen sich dann alle diese Funktionen über das stellvertretend definierte Makro aufrufen.



21.6.3 Statische Zusicherungen



Mit Hilfe von Zusicherungen (auf Englisch „assertions“) können zur Laufzeit Ausdrücke auf logische Fehler getestet werden. In ANSI C wird dafür das Makro `#assert` definiert. Es wertet einen Ausdruck aus, der darüber entscheidet, ob ein Programm normal weiter läuft oder abgebrochen wird. Ist der Ausdruck 0, dann wird eine Fehlermeldung in `stderr` geschrieben und das Programm wird mit der `abort()`-Funktion abgebrochen (siehe auch Kapitel [→ 17.2.5](#)).

Zum Einsatz kommen solche Zusicherungen in der Entwicklungsphase. Ist diese abgeschlossen, lassen sich alle Zusicherungen mit der Definition von `NDEBUG` „entfernen“. Alle `#assert`-Makros werden hierbei als Makros ohne ausführenden Code definiert und dadurch vom Compiler ignoriert.

`NDEBUG` meint wörtlich „no debug“, was soviel bedeutet wie: Wenn dieses Makro definiert ist, soll der Compiler keinen Debug-Code umsetzen. Entwicklungsumgebungen definieren dieses Makro häufig automatisch, sobald ein sogenanntes Release kompiliert wird. Dieses Makro kann auch für interne Tests benutzt werden, indem es mittels `#ifndef NDEBUG` abgefragt und innerhalb der bedingten Compilierung Debug-Code ausführt wird.

Zusätzlich zu den Zusicherungen, welche zur Laufzeit geprüft werden, kommen in C11 sogenannte „statische Zusicherungen“ (auf Englisch „static assertions“) dazu. Bei statischen Zusicherungen handelt es sich um Zusicherungen zur Compilierzeit.



Diese Zusicherungen werden erst dann ausgewertet, wenn der Präprozessor durchgelaufen ist und alle Argument-Typen bekannt sind. Statische Zusicherungen sind wie folgt definiert:

```
_Static_assert(constant-expression, string-literal);
```

Wie beim Makro `#assert` wertet auch die statische Variante `_Static_assert` einen Ausdruck aus. Ist der Ausdruck `constant-expression` gleich 0, dann wird eine Fehlermeldung ausgegeben, bestehend aus dem String `string-literal`, dem Dateinamen, der Zeilennummer und gegebenenfalls der Funktion, in welcher diese Zusicherung steht.

Statische Zusicherungen werden bereits zur Compilierzeit überprüft und können daher den Compiliervorgang abbrechen. Besonders nützlich sind solche Zusicherungen beim Schreiben von portablem Code. So kann verhindert werden, dass Code für eine Plattform kompiliert wird, die nicht unterstützt wird.

22 Modulares Design in C



Jede Programmiersprache unterstützt bestimmte Vorgehensweisen beim Programmentwurf. Neue Programmiersprachen werden geschaffen als Antwort auf neue Konzepte, die sich als nützlich und effizient erwiesen haben. So wurde Pascal entworfen mit dem Ziel, das Konzept des strukturierten Programmierens zu unterstützen. Modula-2 wurde entworfen, um das Konzept des Programmierens mit sogenannten Modulen zu unterstützen.

In diesem Kapitel sollen diese Konzepte genauer betrachtet werden. Als Basis dient hierbei das sogenannte „Strukturierte Design“, bei welchem das einzige Strukturierungsmittel Funktionen sind.

Auch wenn die Programmiersprache C für das Strukturierte Design besonders geeignet ist, so wie die Programmiersprache Modula-2 für das Modulare Design, so kann man dennoch mit den Sprachmitteln von C das Konzept des Modularen Designs unterstützen.



22.1 Konzept des Strukturierten Designs

Beim **Strukturiertes Design** (auf Englisch „structured design“) wird ein Programm in Unterprogramme zerlegt, die zu anderen Unterprogrammen möglichst wenige Querbeziehungen haben.

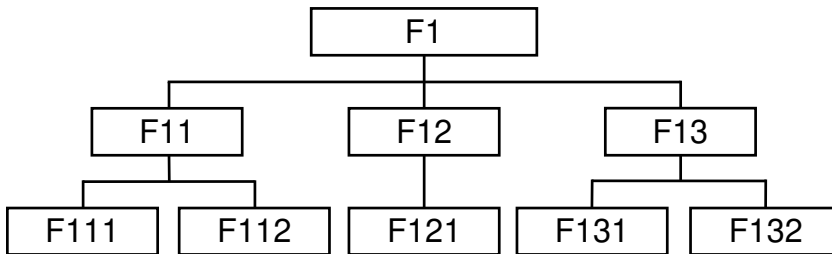


In C bedeutet die Umsetzung des Strukturierten Designs, dass man ein Programm in Funktionen strukturiert, die möglichst wenig Schnittstellen zu ihrer Umgebung haben, also wenig Übergabeparameter benötigen und möglichst ohne globale Variablen auskommen.



Ergänzende Information Die elektronische Version dieses Kapitels enthält Zusatzmaterial, auf das über folgenden Link zugegriffen werden kann https://doi.org/10.1007/978-3-658-45209-4_22.

Dadurch, dass die Funktionen einander aufrufen, entsteht eine Beziehung zwischen ihnen, die man als Aufrufhierarchie der Funktionen des Programms bezeichnet. Eine solche Aufrufhierarchie ist exemplarisch im folgenden Bild dargestellt:



Im Mittelpunkt des Strukturierten Designs steht also der Funktionsbegriff aus den anfangs der siebziger Jahre vorherrschenden Programmiersprachen. Die Funktionen dienten vornehmlich dem „Programmieren im Kleinen“.

22.2 Konzept des Modularen Designs

Basierend auf den Erkenntnissen von Parnas [Par72] und den Entwicklungen im Programmiersprachenbereich – wie beispielsweise das Entstehen von Modula-2 – war es notwendig, den Begriff des Moduls, der zunächst eine compilierfähige Einheit – also eine Datei – bezeichnete, zu erweitern.

Beim **Modularen Design** (auf Englisch „modular design“) stehen nicht nur einzelne Funktionen im Mittelpunkt, sondern die Zusammenfassung von Funktionen und Daten zu größeren Einheiten, den Modulen.



Man sprach vom „Programmieren im Großen“ und die dafür entwickelte Methode erhielt den Namen „modular design“. Der Begriff des Moduls wurde dabei nun für die größeren Einheiten verwendet. Auch heute noch wird der Begriff „Modul“ in zweierlei Hinsicht verwendet: Modul als compilierfähige Einheit und Modul im Sinne des Modularen Designs.

22.2.1 Kapselung und Information Hiding

Ein Modul besteht aus Funktionen sowie Daten, auf denen die Funktionen arbeiten. Im Sinne des Modularen Designs sollen diese verborgen bleiben. Entsprechend werden Implementationen der Funktionen und Definitionen der Daten in eine eigene Übersetzungseinheit verpackt, welche nicht ohne weiteres von außen angesprochen werden können. Dieser sogenannte „Modul-Rumpf“ stellt die eigentliche Implementierung eines Moduls dar.

Zentraler Gedanke beim Modularen Design ist einerseits die Kapselung der Daten und Funktionen als Einheit und andererseits die Definition der **Schnittstellen** zwischen Modulen.



Über eine **Export-Schnittstelle** stellt ein Modul Ressourcen für andere Module bereit. Nur die Funktionen und Daten, welche in einer Export-Schnittstelle deklariert sind, stehen anderen Modulen zur Verfügung. Alle anderen Interna sind verborgen, was als „**Information Hiding**“ bezeichnet wird. Die Export-Schnittstellen stellen eine Abstraktion der nach außen angebotenen Funktionalität dar.



Um innerhalb eines Moduls auf die Funktionalität anderer Module zuzugreifen, listet man die benötigte Funktionalität in der **Import-Schnittstelle** auf.



Mit dem neuen Schnittstellenkonzept hat man das Folgende erreicht:

- Ein Modul ist ersetzbar durch ein Modul mit gleicher Export- und Import-Schnittstelle.
- Durch Prüfung der Export- und der Import-Schnittstellen der verschiedenen Module durch den Compiler lässt sich feststellen, ob die wechselseitigen Aufrufe der Module fehlerfrei funktionieren.
- Beim Entwickeln der Module hat man die Kontrolle darüber, was von einem Modul benutzt werden kann (Export-Schnittstelle) und was verborgen bleibt (Information Hiding).

Durch die konsequente Anwendung von Information Hiding wird ein Modul zur Black-Box: Man kennt nur die Export- und Import-Schnittstellen, aber nicht die genaue Funktionsweise.

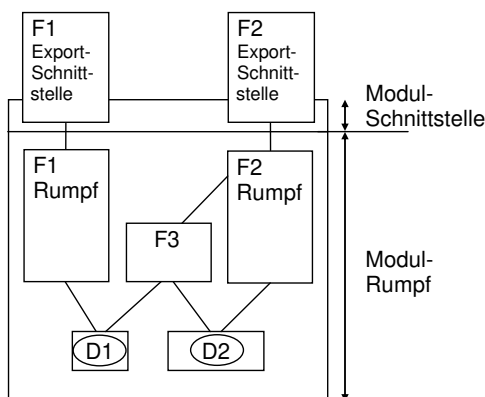


Im Gegensatz zum Strukturierten Design, bei dem einzelne Funktionen im Mittelpunkt stehen, werden nun Module aus mehreren Funktionen betrachtet.

In einer Modul-Schnittstelle wird spezifiziert, welche Funktionen ein Modul zur Verfügung stellt (Export) und welche Funktionen ein Modul benötigt, um korrekt arbeiten zu können (Import).

22.2.2 Export-Schnittstellen

Folgendes Bild veranschaulicht, wie die Rümpfe von F1 und F2, die Modul-interne Funktion F3 und die Daten D1 und D2 nach außen nicht sichtbar sind:



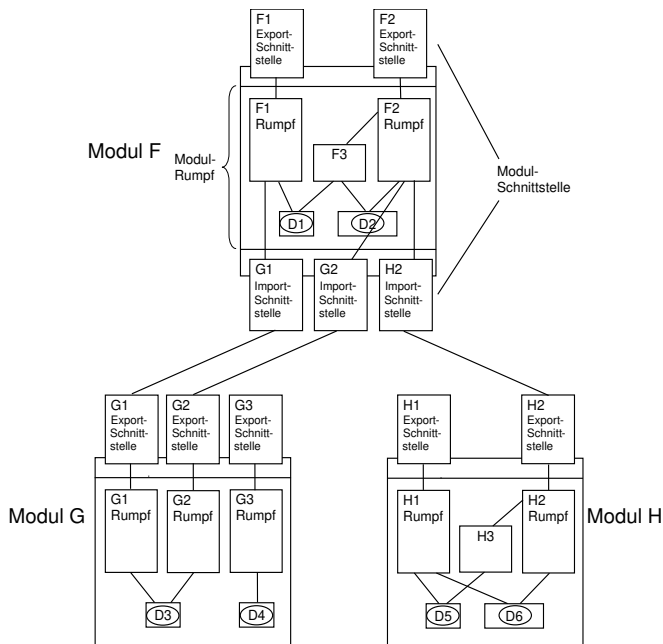
Daten können von außen nur über Aufrufe der Export-Funktionen F1 und F2 abgeholt oder geändert werden. Die Zerlegung in Schnittstelle und Rumpf eines Moduls bringt den Vorteil, dass Änderungen im Rumpf, wie beispielsweise zur Verbesserung der Performance durch einen anderen Algorithmus, nach außen nicht sichtbar werden, wenn die Schnittstelle durch die Änderungen nicht beeinflusst wird. Dies trägt zur Stabilität in einem Projekt bei.

22.2.3 Import-Schnittstellen

Ein Modul kann ein anderes Modul benutzen, indem eine Funktion des einen Moduls eine Funktion eines anderen Moduls als Import-Funktion aufruft. Im Rahmen des Modularen Designs müssen Module bekanntgeben, dass sie Funktionen anderer Module benutzen wollen. Dies wird als Import-Schnittstelle eines Moduls bezeichnet.

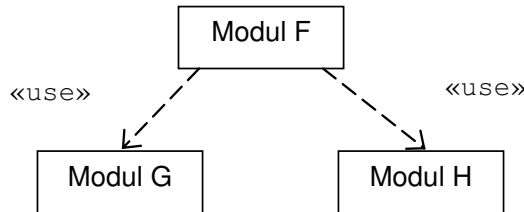


Das folgende Bild zeigt nun zusätzlich die Import-Schnittstellen des Moduls:



Diese Import-Schnittstellen sind so zu interpretieren, dass das Modul F die Funktionen G1, G2 und H2 benötigt, um korrekt arbeiten zu können. Es muss daher Module geben, die diese Funktionen über ihre Export-Schnittstelle zur Verfügung stellen, damit das gesamte System funktioniert. Durch Benutzung entsteht zwischen den Modulen eine Beziehung, die sogenannte Import-Relation. In obigem Bild benutzt Modul F die Module G und H.

Diese Import-Relation kann grafisch in einem Modul-Diagramm dargestellt werden:



Bei dem Entwurf von Modulen sollen gegenseitige Abhängigkeiten – sowohl direkt zwischen zwei Modulen als auch indirekt über mehrere Module hinweg – vermieden werden.



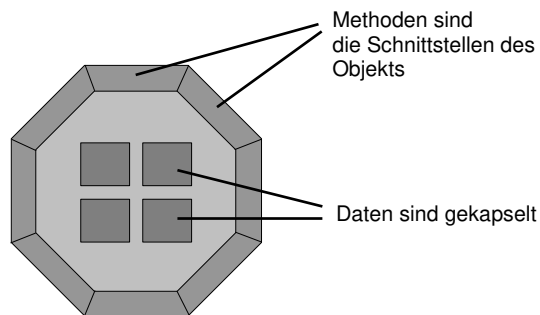
Solche unerwünschten Abhängigkeiten sind in einem Modul-Diagramm als Zyklus sichtbar.

22.2.4 Das Modulkonzept – eine Vorstufe der Objektorientierung

Die Kapselung von Daten, Funktionsrümpfen und internen Funktionen in einem Modul als Einheit entspricht bereits dem Ansatz der **Objektorientierung**. Das bedeutet, dass die Funktionen bei den Daten stehen müssen, die sie tatsächlich bearbeiten und dass nur die sogenannten „öffentlichen“ Funktionen die Schnittstelle zwischen diesen Daten und der Außenwelt darstellen.



Dies symbolisiert das folgende Bild der Objektorientierung:



Dass in der Objektorientierung Funktionen als Methoden bezeichnet werden, tut hierbei nichts zur Sache.

Auch der Ansatz der Trennung von Schnittstelle und Implementierung eines Moduls entspricht bereits der Objektorientierung: ein Objekt kann mit einem anderen Objekt nur über die Schnittstellen der Methoden dieses Objektes reden. Die Implementierung der Methoden eines Objektes ist nach außen nicht sichtbar.

Im Gegensatz zum Modularen Design bietet eine Klasse der Objektorientierung jedoch nach außen nur Schnittstellen von Export-Methoden an. Sie hat keine Schnittstellen für Import-Methoden. Des Weiteren entspricht bei der Objektorientierung eine Klasse einem Datentyp. Ein Modul ist aber kein Datentyp, sondern stellt lediglich eine prozedurale Strukturierungseinheit dar.

22.3 Umsetzung des Modularen Designs in C

Ein Modul im Sinne des Modularen Designs wird in C in eine übersetzbare Einheit umgesetzt, also eine Quelldatei, landläufig auch als C-Datei bezeichnet.

22.3.1 Information Hiding mit `static`

Die globalen Variablen und Funktionen eines Moduls, die im Modul verborgen gehalten werden sollen, werden mit dem Schlüsselwort **`static`** versehen (siehe Kapitel [→ 15.4](#)). Dadurch können sie von Funktionen aus anderen Dateien nicht verwendet werden – auch nicht unter Verwendung der extern-Deklaration. Damit ist das Geheimnisprinzip (**Information Hiding**) gewahrt.

Ein Schlüssel zur Modularisierung in der Programmiersprache C ist das Schlüsselwort `static`. Da in der Programmiersprache C immer von einer anderen Datei auf Funktionen und globale Variablen zugegriffen werden kann, muss man, um eine globale Variable oder eine Funktion nach außen zu verbergen, das Schlüsselwort `static` verwenden.



22.3.2 Trennung von Schnittstelle und Implementierung mit Header-Datei

Um in C zwischen Schnittstelle und Implementierung eines Moduls zu trennen, wird die Technik der **Header-Dateien** eingesetzt.

Eine Header-Datei enthält insbesondere Funktions-Prototypen für alle Funktionen, die ein Modul nach außen zur Verfügung stellt. Dazu können Typvereinbarungen (meist Strukturen) und Konstanten kommen, die für den Umgang mit den Funktionen wichtig sind wie beispielsweise Fehlercodes.



Somit entspricht die Header-Datei der Schnittstelle des Moduls.

Die C-Datei implementiert die Funktionen der zugehörigen Header-Datei. Da die Header-Datei extern-Deklarationen enthält, wird diese Prototypen-Header-Datei auch von denjenigen Modulen inkludiert, die diese Funktionen aufrufen wollen.



Jede C-Datei wird dadurch entkoppelt von anderen C-Dateien. So ist es möglich, C-Dateien unabhängig voneinander zu bearbeiten, also eine **separate Compilierung** durchzuführen.

Das folgende Bild zeigt die Implementierung einer Header-Datei und der zugeordneten C-Datei:



Eine C- und die dazugehörige Header-Datei bilden eine logische Einheit. Dies soll sich auch im Dateinamen ausdrücken: Beide Dateien werden mit einem Namen benannt, welcher die in den Dateien enthaltenen Deklarationen und Definitionen als zusammengehörige, modulare Einheit kennzeichnet. Sie unterscheiden sich nur durch unterschiedliche Dateinamenserweiterungen.

Da die Header-Datei die Funktions-Prototypen der C-Datei enthält, wird diese üblicherweise in der C-Datei inkludiert. Der Compiler hat so die Möglichkeit, einen Funktions-Prototyp und seine Implementierung auf Konsistenz zu prüfen.



Funktions-Prototypen und Typ-Definitionen sind Informationen für den Compiler und erzeugen noch keinen Speicherplatz im Programm. Ebenso sind alle Anweisungen an den Präprozessor wie etwa eine symbolische Konstantendefinition mit `#define` unkritisch. Wenn globale Variablen vom Modul exportiert werden sollen, dann darf in der Header-Datei nur eine entsprechende extern-Deklaration dieser Objekte stehen. Eine extern-Deklaration reserviert nämlich keinen Speicher.

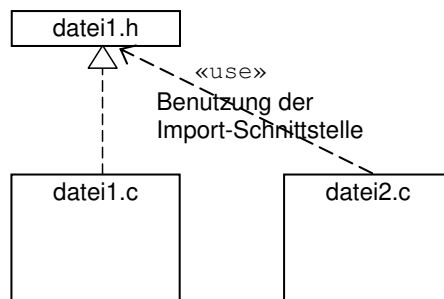
Erst die Definition dieser Objekte im Rumpf des Moduls in der entsprechenden C-Datei legt den benötigten Speicherplatz an.

Eine Header-Datei sollte keine nicht statische Definition enthalten, die in irgendeiner Weise zu einer Speicherallokation führt. Eine Header-Datei sollte, wenn immer möglich, nur Deklarationen enthalten.



22.3.3 Benutzung anderer Module durch Inkludieren der Header-Datei

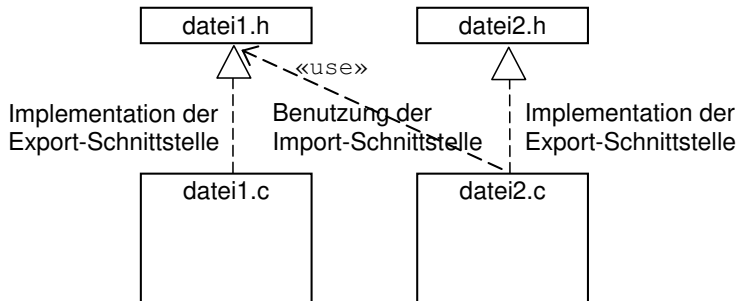
Wenn in der C-Datei eines Moduls `datei2.c` ein anderes Modul `datei1` benutzt werden soll, dann muss die C-Datei `datei2.c` die entsprechende Header-Datei `datei1.h` inkludieren. Das folgende Bild zeigt die Benutzung der Header-Datei als Schnittstelle:



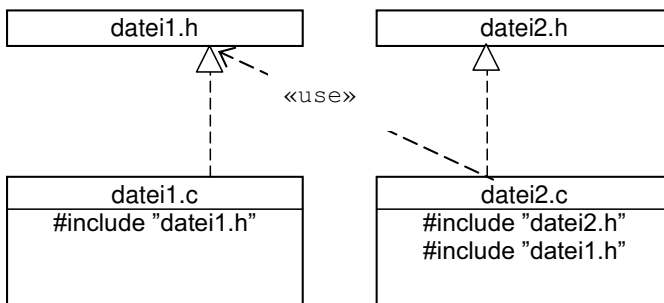
Inkludiert eine Datei `datei2.c` die Header-Datei `datei1.h` einer Datei `datei1.c`, so stellt die Header-Datei `datei1.h` für die Datei `datei2.c` die Import-Schnittstelle dar.



Eine Header-Datei in C dient also sowohl als Export-Schnittstelle für die eine Datei, als auch als Import-Schnittstelle für die andere Datei:



Das Ansprechen der Import-Schnittstelle findet in C mittels der `#include`-Direktive statt (siehe Kapitel [→ 21.1](#)).



Die vorgeschlagene Nutzung der `#include`-Direktive, wie sie hier schematisch dargestellt ist, ist typisch für C-Programme. Wie bereits angemerkt, wird durch das Einfügen der „eigenen“ Header-Datei die Implementierung auf Konsistenz geprüft.

22.3.4 Verhindern von mehrfachem Inkludieren der gleichen Header-Datei

Wenn die Abhängigkeiten zwischen den Dateien komplizierter werden, kann es leicht passieren, dass eine Header-Datei **mehrfach inkludiert** wird. Das führt ohne weitere Vorkehrungen jedoch zu Fehlermeldungen, da zum Beispiel eine Struktur oder eine symbolische Konstante dann mehrfach definiert wird, was nicht erlaubt ist.

Dieses Problem kann mit Hilfe der `#ifndef`-Präprozessor-Direktive (siehe auch Kapitel [→ 21.3](#)) gelöst werden, wie im folgenden Beispiel gezeigt:

person.h

```
#ifndef PERSON_H_INCLUDED
#define PERSON_H_INCLUDED

typedef struct person {
    char vorname [80];
    char nachname [80];
    short geburtsjahr;
} person;

void printPerson(const person* const p);

// Weitere exportierte Funktions-Prototypen ...

#endif
```

Wird in einem Quellcode das erste Mal die Header-Datei `person.h` inkludiert, so ist die symbolische Konstante `PERSON_H_INCLUDED` nicht definiert. Entsprechend wird der Code zwischen `#ifndef` und `#endif` nicht vom Präprozessor gelöscht. Somit wird die symbolische Konstante `PERSON_H_INCLUDED` sofort in der nächsten Zeile definiert.

Wird in demselben Quellcode die Header-Datei `person.h` ein weiteres Mal inkludiert, so ist die symbolische Konstante `PERSON_H_INCLUDED` diesmal definiert, weswegen alle Zeilen bis zur abschließenden `#endif`-Direktive gelöscht werden. Die Struktur `struct person` wird also nicht noch ein weiteres Mal definiert – der Fehler beim Übersetzen ist vermieden.

22.4 Realisierung eines Stacks mit Modularem Design in C

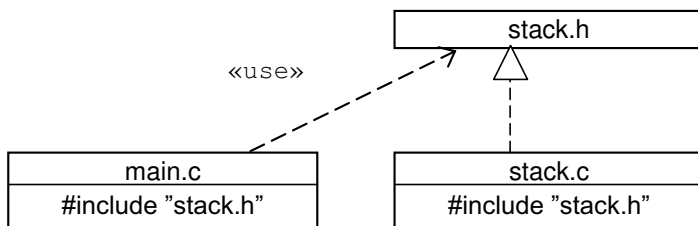
In diesem Kapitel sollen die Konzepte des Modularen Designs in C an einem einfachen Beispiel vorgestellt werden. Ziel ist es, einen **Stack** zu implementieren, in dem nach dem Prinzip „Last In – First Out“ (LIFO-Prinzip) `int`-Zahlen gespeichert werden können.

Der Stack ist eine wichtige Datenstruktur in der Informatik. Er wurde bereits in Kapitel [→ 11.8.1](#) im Rahmen des Call-Stacks vorgestellt, welcher zur Abwicklung rekursiver Funktionsaufrufe eingesetzt wird.

Bei einem Call-Stack kann ein C-Compiler dabei heutzutage meist auf die Hardware vertrauen: Die meisten gängigen Prozessoren unterstützen das Stack-Prinzip durch entsprechende Befehle und Register. Hier in diesem Kapitel wird jedoch nicht der Call-Stack implementiert werden, sondern ein „selbst gebauter“ Stack für beliebige Inhalte mit einer klar definierten Export-Schnittstelle.

Durch die feste Export-Schnittstelle des Stack-Moduls ist es dabei sogar möglich, flexibel auf unterschiedliche Anwendungen und auch auf sehr hohe Anforderungen an die Stackgröße zu reagieren. So ist es zum Beispiel möglich, dass diese Implementierung des Stacks für Dateien oder Datenbanken eingesetzt werden kann. Weitere Anwendungsgebiete sind beispielsweise die Überprüfung der korrekten Schachtelung (Klammerung) in einer Eingabe oder die Umkehrung einer Reihenfolge durch das LIFO-Prinzip etwa bei der Berechnung der Binärdarstellung einer Zahl (siehe dazu auch die entsprechende Übungsaufgabe).

Das folgende Programm besteht aus drei Dateien `main.c` (Testrahmen), `stack.h` (die Export-Schnittstelle des Moduls Stack) und `stack.c` (die Implementierung des Moduls Stack). Das folgende Bild zeigt die Abhängigkeiten über die `#include`-Direktiven zwischen diesen Dateien:



22.4.1 Die Header-Datei stack.h

Hier zunächst die Datei stack.h:

stack.h

```
#ifndef STACK_H_INCLUDED
#define STACK_H_INCLUDED

enum stackError {
    STACK_OK,
    STACK_VOLL,
    STACK_LEER
};

// Eine int-Zahl auf den Stack legen.
// Gibt einen Fehlercode zurueck.
extern enum stackError stackPush(int wert);

// Eine int-Zahl vom Stack abholen.
// Parameter: Pointer auf int-Zahl, die mit dem obersten
//           Stack-Wert gefuehlt wird.
// Gibt einen Fehlercode zurueck.
extern enum stackError stackPop(int* wertPtr);

#endif
```

Zunächst wird wieder durch den Einsatz der `#ifndef`-Präprozessor-Direktive sichergestellt, dass die Header-Datei nicht mehrfach in denselben Quellcode inkludiert werden kann.

Danach wird der Aufzählungs-Typ `enum stackError` definiert, in dem Kennnummern für die Rückgabewerte (OK oder Fehler) der folgenden Funktionen festgelegt werden. Damit kann ein anderes Modul auf einen Fehler, der im Modul Stack entsteht, entsprechend reagieren.

Dann werden die exportierten Funktionen `stackPush()` und `stackPop()` als Prototypen deklariert. Damit weiß jedes Modul, welches die Datei `stack.h` inkludiert, welche Funktionen das Modul Stack anbietet. Ein inkorrektter Aufruf einer Funktion (falsche Parameteranzahl, falsche Parameter-Typen, fehlerhafte Verwendung des Rückgabewertes) wird vom Compiler erkannt.

In diesem Beispiel ist zudem eine weitere nützliche Eigenschaft von Header-Dateien zu sehen. Die exportierten Funktionen werden zusätzlich durch Kommentare versehen, welche die Aufgaben der bereitgestellten Funktionen, deren Parameter und die Rückgabewerte beschreiben. Beim Programmieren bieten diese Kommentare eine große Orientierungshilfe, da sie Aufschluss darüber geben, wie genau ein Modul verwendet werden sollte. Viele Entwicklungsumgebungen können diese Kommentare sogar als Hilfe anzeigen, beispielsweise beim Darüberfahren über einen Funktionsaufruf mit der Maus.

In C ist es zudem gängige Praxis, den eigentlichen Namen `push()` und `pop()` der Funktionen des Moduls mit dem Namen des Typs, auf welchem operiert wird, zu ergänzen, also `stackPush()` und `stackPop()`. So wird vermieden, dass es zu Namensüberschneidungen mit anderen Modulen kommt, die denselben Namen für ihre Funktionen brauchen. Das ist vor allem bei „Allerwelts-Funktionsnamen“ wie zum Beispiel `init()`, `close()` und `open()` dringend geboten, wobei die letzteren ja sogar Funktionen aus den Standardbibliotheken sind. Je nach Programmierstil könnten die Funktionen auch `pushStack()` und `popStack()` oder ähnlich heißen.

22.4.2 Arbeitsteilige Entwicklung

Mit der bloßen Kenntnis der Export-Schnittstelle des Moduls Stack kann diese jetzt im Hauptprogramm verwendet werden, ohne weiteres Wissen über die eigentliche Implementierung in der Datei `stack.c` zu haben. Die nun folgende Datei `main.c` kann sogar übersetzt werden, unabhängig davon, ob die Datei `stack.c` überhaupt existiert, fehlerhaft ist oder schon korrekt übersetzt wurde. Die Programmierung dieser beiden Dateien könnte also sogar von zwei unabhängigen Personen erfolgen. Durch die fixe Definition der Schnittstelle `stack.h` würden sie sich dabei nicht gegenseitig stören. Dadurch wird eine arbeitsteilige Programmentwicklung ermöglicht.

Erst beim Versuch, das Programm zu binden, müssen alle vom Hauptprogramm benötigten Dateien und Bibliotheken korrekt übersetzt vorliegen, damit sie zu einem ausführbaren Programm zusammengefügt werden können. Man spricht dann auch davon, dass die einzelnen Module zu einem ablauffähigen System integriert werden.

22.4.3 Implementierung des Moduls main.c

Die ausprogrammierte Datei main.c soll folgendermaßen lauten:

main.c

```
#include <stdio.h>
#include "stack.h"

int main(void) {

    stackPush(5);
    stackPush(18);
    stackPush(23);

    int zahl;
    enum stackError status;
    while ((status = stackPop(&zahl) == STACK_OK)) {
        printf("%d\n", zahl);
    }

    return 0;
}
```

Im Hauptprogramm, das hier als Testrahmen des Moduls Stack dient, werden ein paar Zahlen auf dem Stack abgelegt und daraufhin in umgekehrter Reihenfolge auf dem Bildschirm ausgegeben.

Hier die Ausgabe des Programms:

```
23
18
5
Fehler Nr. 2: Stack ist leer
```

Wie man sieht, wird ein Fehler ausgegeben. Dies deshalb, da dem Stack so lange ein Element entzogen wird, bis er leer ist. Um diese Fehlermeldung zu umgehen, wird normalerweise bei einem Stack eine weitere Funktionalität bereitgestellt, welche zurückgibt, ob der Stack bereits leer ist. Hierfür gibt es eine Übungsaufgabe am Ende dieses Kapitels.

Würden mehr als 5 Werte auf den Stack geschrieben werden, so wird der Stack eine Fehlermeldung ausgeben und den Wert ignorieren.

22.4.4 Implementierung des Moduls `stack.c`

Die Datei `stack.c` schließlich dient der Implementierung des Moduls `Stack`:

`stack.c`

```
#include "stack.h"
#include <stdio.h>

#define MAX 5
static int stack[MAX];
static int topOfStack = 0;

static void printError(const enum stackError errNr) {
    static char * errorStrings [] = {
        "OK",
        "Stack ist voll",
        "Stack ist leer"
    };
    if (errNr < 0 || errNr > STACK_LEER)
        fprintf(stderr, "Undefinierter Fehler\n");
    else
        fprintf(stderr, "Fehler Nr. %d: %s\n", errNr, errorStrings[errNr]);
}

enum stackError stackPush(int wert) {
    if (topOfStack == MAX) {
        printError(STACK_VOLL);
        return STACK_VOLL;
    } else {
        stack[topOfStack] = wert;
        ++topOfStack;
        return STACK_OK;
    }
}

enum stackError stackPop(int* wertPtr) {
    if (topOfStack == 0) {
        printError(STACK_LEER);
        return STACK_LEER;
    } else {
        --topOfStack;
        *wertPtr = stack[topOfStack];
        return STACK_OK;
    }
}
```

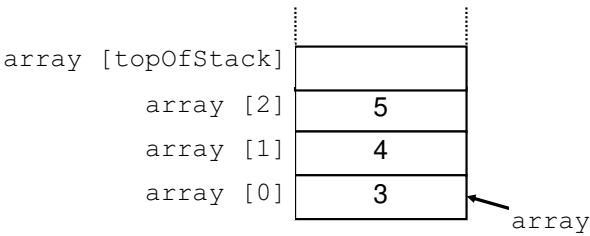
Die Datei enthält als Modul-globale Daten ein Array `array`, welches `int`-Zahlen speichert und einem Füllstandsanzeiger des Stacks (`topOfStack`). Alle diese Daten sind durch das Schlüsselwort `static` gegen versehentlichen Zugriff von außen gesichert. Daher kann hier auch auf das Voranstellen des Präfixes „`stack`“ verzichtet werden. Der C-Linker ist durchaus imstande, mehrere gleich benannte `static`-Variablen aus verschiedenen Modulen parallel konsistent zu verwalten.

Aus anderen Dateien ist der Zugriff auf den Stack nur indirekt, das heißt über die Funktionen `stackPush()` und `stackPop()` möglich. Die Stack-Variablen `array` und `topOfStack` müssen innerhalb der Datei `stack.c` Modul-global sein, da auf sie sowohl von der Funktion `stackPush()` als auch von der Funktion `stackPop()` zugegriffen werden muss. Weiter wird die symbolische Konstante `MAX` vereinbart, die ebenfalls nur innerhalb des Moduls `Stack` bekannt ist.

Die Funktion `printError()` stellt eine Modul-globale, aber nach außen nicht sichtbare Funktion dar. Sie ist ebenfalls durch das Schlüsselwort `static` gegen versehentlichen Zugriff von außen gesichert. Wieder ist es wie bei den Modul-globalen Daten möglich, dass `static`-Funktionen des gleichen Namens in anderen Modulen implementiert werden. Da die auszugebenden Fehlertexte nur innerhalb der Funktion `printError()` gebraucht werden, werden diese als `static`-Array im Funktionsrumpf von `printError()` definiert.

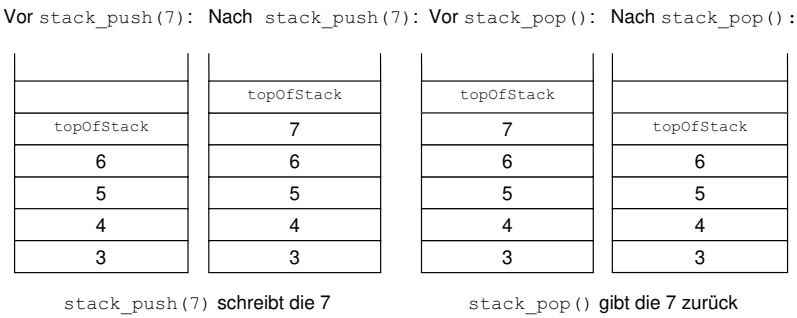
Die Funktionen `stackPush()` und `stackPop()` dienen dazu, mit Hilfe der Variablen `topOfStack` ein neues Datenelement an die oberste Stelle des Stacks zu schreiben oder das oberste Datenelement vom Stack abzuholen.

Der Stack ist programmtechnisch durch ein Array `int array [MAX]` realisiert. `topOfStack` ist ein `int`-Wert, der einen Index für dieses Array darstellt. Der Index `topOfStack` gibt das nächste freie Element des Stacks (also des Arrays) an, wie in folgendem Bild gezeigt:



Schreibt die Funktion `stackPush()` eine `int`-Zahl auf den Stack, so muss sie den Wert der Variablen `topOfStack` erhöhen. Liest die Funktion `stackPop()` vom Stack, muss sie `topOfStack` erniedrigen. Die Variable `topOfStack` zeigt immer auf den nächsten freien Platz.

Diese Funktionsweise wird in folgendem dargestellt:

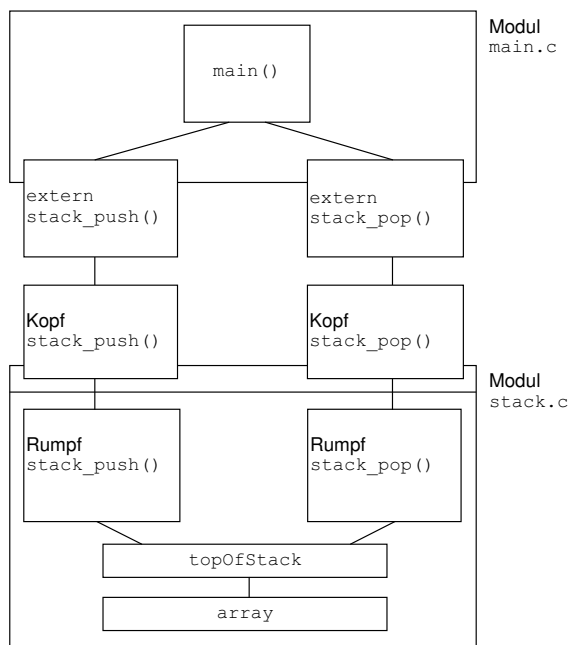


22.4.5 Modulare Betrachtung

Damit liegt hier ein Beispiel für die Umsetzung eines Modularen Designs nach C vor: Das Modul Stack exportiert die Funktionen `stackPush()` und `stackPop()` und verbirgt dabei die eigentliche Datenstruktur. Bei der Verwendung der beiden Funktionen braucht man sich mit der oben skizzierten programmtechnischen Realisierung des Stacks nicht auseinanderzusetzen.

Zu anderen Modulen ist die Implementierung und der Stack selbst nicht sichtbar. So wäre es auch möglich, den Stack statt durch ein Array durch eine verkettete Liste (siehe Kapitel [19.1](#)) zu realisieren. Diese Modul-interne Änderung würde nach außen nicht sichtbar. So muss in diesem Beispiel das Hauptprogramm nicht umgeschrieben werden, da ein Modul, das diesen Stack benutzt, trotz dieser gravierenden Änderung in der Datenhaltung nicht geändert werden muss.

Das folgende Bild zeigt, wie das Modul `main.c` Funktionen des Moduls `stack.c` importiert:



22.5 Übungsaufgaben

Aufgabe 1: Erweiterung des Stack-Moduls

- a) Oft ist es bei Stacks nützlich zu wissen, ob dieser leer ist, beziehungsweise wie viele Elemente dieser gespeichert hat. Erweitern Sie das Modul Stack aus Kapitel [→ 22.4](#) um eine Funktion `stackCount()`, welche die Anzahl der im Stack gespeicherten Elemente als `int`-Zahl zurückgibt.
- b) Schreiben Sie im Hauptprogramm die `while`-Schleife so um, dass die neue Funktion benutzt und somit keine Fehlermeldung mehr ausgegeben wird.

Aufgabe 2: Weitere Nutzung des Stack-Moduls

- a) Nutzen Sie das Modul Stack, um ein Programm aus Kapitel [→ 11.8.3](#) zu vereinfachen. Dort wird eine gegebene `int`-Zahl als Binärzahl ausgegeben, indem drei `for`-Schleifen durchlaufen werden. Durch das Ersetzen des dort verwendeten Arrays mit dem nun zur Verfügung stehenden Stack-Modul sollten zwei `while`-Schleifen ausreichen.
- b) Achten Sie darauf, dass der Stack genügend Platz reserviert hat. Die ursprüngliche Programmierung erlaubt Platz für 8 Bits. Probieren Sie aus, wie einfach eine Erweiterung auf mehr Bits mit dem Stack-Modul ist und wie viel größer die umzuwandelnden Zahlen mit dem Stack-Modul dann werden können.

Aufgabe 3: Nutzung des Heaps für den Stack

Die Beispiel-Implementation eines Stacks aus Kapitel [→ 22.4](#) verwendet ein globales Array mit fixer Grösse, um die Inhalte des Stacks zu speichern. In dieser Aufgabe soll die Implementation umgeändert werden, sodass ein Stack dynamisch mit beliebig vielen Einträgen erzeugt und genutzt werden kann.

- Schreiben Sie eine neue Implementation, welche stattdessen Speicher des Heaps nutzt. Verwenden Sie hierfür eine geeignete Funktion wie `malloc()`.
- Verpacken Sie die benötigten Daten für den Stack in eine Struktur und schreiben Sie eine Funktion `stackErstellen()`, welche einen Pointer auf eine neu erstellte Stack-Struktur mit der gewünschten Stackgröße zurückgibt. Schreiben Sie eine dazu passende `stackAufraeumen()`-Funktion.
- Testen Sie ihre Struktur, indem Sie zwei Stacks erstellen, dann den einen Stack mit Werten füllen, danach diese Inhalte mittels der verfügbaren Funktionen aus dem ersten Stack entfernen und sie direkt in den zweiten Stack übertragen und schlussendlich den zweiten Stack ausgeben und wieder leeren.

23 Multithreading



Dieses Kapitel gibt Einblicke in die Entwicklung von Programmen mit mehreren nebenläufigen Code-Teilen.

Threads sind eine optionale Erweiterung des C11-Standards. Dies bedeutet, dass nicht jeder Compiler, der C11 unterstützt, auch eine Unterstützung der optionalen Threads bietet. Ist das Makro `__STDC_NO_THREADS__` gesetzt, so bietet der Compiler keine Thread-Unterstützung.



23.1 Betriebssystemprozesse und Threads

Als erstes soll ein Betriebssystemprozess charakterisiert werden.

Wird ein lauffähiges Programm auf einem Computer mit einem Betriebssystem ausgeführt, so bezeichnet man dieses Programm als Betriebssystemprozess, oder schlicht „**Prozess**“. Ein laufender Betriebssystemprozess besitzt einen eigenen Adressraum für sein Programm. Er kann auf die ihm zugeordneten Systemressourcen wie Prozessor, Speicher oder Peripheriegeräte zugreifen.



Die Ressourcen eines Betriebssystemprozesses werden als sein Prozess-Kontext bezeichnet.

Eine Ressource, die ein Programm benötigt, damit es laufen kann – wie der Speicher, der Prozessor oder eine benötigte Datei – stellt ein Betriebsmittel für einen Betriebssystemprozess dar.



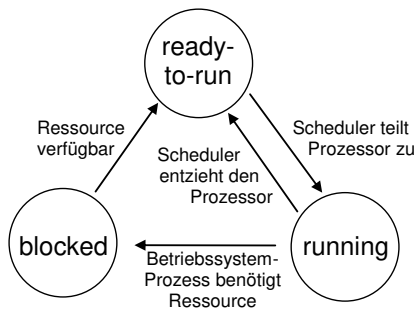
Ergänzende Information Die elektronische Version dieses Kapitels enthält Zusatzmaterial, auf das über folgenden Link zugegriffen werden kann https://doi.org/10.1007/978-3-658-45209-4_23.

In aller Regel sind beim Betrieb eines Computers immer zahlreiche Betriebssystemprozesse gleichzeitig aktiv. Diese werden vom Betriebssystem verwaltet. Betriebssystemprozesse werden auch **Tasks** genannt und das gleichzeitige Abarbeiten mehrerer Tasks wird als Multitasking bezeichnet.

Betriebssystemprozesse können verschiedene Zustände annehmen. Der Zustand eines Betriebssystemprozesses kann je nach Verfügbarkeit der benötigten Betriebsmittel wechseln. Die Verfügbarkeit des Prozessors wird in modernen Betriebssystemen durch einen sogenannten **Scheduler** gesteuert.



Im folgenden Bild wird ein vereinfachtes Zustandsübergangsdiagramm für Betriebssystemprozesse vorgestellt:



Jeder Kreis stellt einen Zustand eines Betriebssystemprozesses dar. Die Pfeile kennzeichnen die Übergänge zwischen den Zuständen. Es gibt hierbei folgende Zustände und Zustandsübergänge:

- Hat ein Betriebssystemprozess bis auf den Prozessor alle Betriebsmittel, die er braucht, um ausgeführt werden zu können, so ist er im Zustand „ready-to-run“ (auf Deutsch „ablaufbereit“).
- Erhält ein Betriebssystemprozess im Zustand „ready-to-run“ vom Scheduler den Prozessor zugeteilt, so geht er in den Zustand „running“ (auf Deutsch „ablaufend“) über.
- Wird einem Betriebssystemprozess im Zustand „running“ der Prozessor entzogen, so geht er wieder in den Zustand „ready-to-run“ über.

- Benötigt ein laufender Betriebssystemprozess weitere Betriebsmittel, die momentan blockiert oder (noch) nicht verfügbar sind, so wechselt er in den Zustand „blocked“ (auf Deutsch „blockiert“). Macht ein laufender Betriebssystemprozess beispielsweise eine I/O-Operation, so verliert er den Prozessor und geht in den Zustand „blocked“ über.
- Sind bei einem Betriebssystemprozess im „blocked“-Zustand die benötigten Betriebsmittel alle verfügbar bis auf den Prozessor, so wechselt der wartende Betriebssystemprozess wieder in den Zustand „ready-to-run“. Ist entsprechend dem vorherigen Beispiel das Gerät für die I/O-Operation verfügbar, so geht der Betriebssystemprozess in den Zustand „ready-to-run“ über.

Das Betriebssystem sorgt dafür, dass die Betriebssystemprozesse, die im Zustand ready-to-run sind, entsprechend einer definierten Strategie des Schedulers abwechselnd den Prozessor erhalten und damit ausgeführt werden.



Jeder Betriebssystemprozess erhält also für eine gewisse Zeit Zugriff auf die CPU und wird anschließend vom Betriebssystem wieder unterbrochen, damit weitere Betriebssystemprozesse ausgeführt werden können.

Der Vorgang des Gewährens beziehungsweise des Entziehens der CPU wird als „Scheduling“ (auf Deutsch „Ablaufplanung“) bezeichnet.



Das Scheduling kann beispielsweise durch die Vergabe definierter Zeitscheiben oder durch Auswertung der Dringlichkeit (Priorität) derjenigen Betriebssystemprozesse, die ablaufbereit sind, erfolgen.

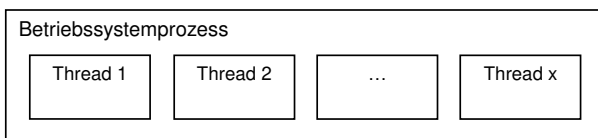
Außerdem stellt das Betriebssystem Möglichkeiten zur Verfügung, wie Betriebssystemprozesse miteinander kommunizieren und Daten austauschen können. Dies wird als Interprozesskommunikation (IPC) bezeichnet.

Es folgt die Erläuterung eines **Threads**.

Als Thread (auf Deutsch „Faden“) bezeichnet man einen nebenläufigen Ausführungsstrang innerhalb eines Betriebssystemprozesses.



Ein Betriebssystemprozess besteht zu Beginn stets aus einem einzigen Thread. In diesem Haupt-Thread wird das Hauptprogramm `main()` gestartet. Indem während des laufenden Betriebssystemprozesses neue Threads erstellt werden, können nebenläufige Tätigkeiten sozusagen „parallel“ ausgeführt werden. So ist es beispielsweise möglich, die grafische Oberfläche von der Berechnungslogik zu trennen, um trotz andauernder Berechnungen parallel auf Eingaben reagieren zu können. Das folgende Bild zeigt symbolisch mehrere Threads, die innerhalb des Kontexts eines Betriebssystemprozesses nebenläufig ausgeführt werden:



23.1.1 Vergleich von Threads und Betriebssystemprozessen

Threads werden also innerhalb eines Betriebssystemprozesses erzeugt und teilen sich dessen Ressourcen. Das heißt, dass alle Threads eines Betriebssystemprozesses gemeinsam auf Daten zugreifen können, die im Adressraum des Betriebssystemprozesses angelegt wurden. Dadurch wird der Datenaustausch zwischen Threads erheblich vereinfacht. Gemeinsam genutzte Daten machen jedoch auch Synchronisationsmechanismen zum Schutz der Daten bei wechselseitigem Zugriff erforderlich. Dies wird in Kapitel [→ 23.3](#) beschrieben.

Threads teilen sich den Adressraum ihres Betriebssystemprozesses und können auf alle Ressourcen dieses Betriebssystemprozesses zugreifen.



Daher ist es wichtig, den Zugriff auf die gemeinsamen Ressourcen durch geeignete Synchronisationsmechanismen zu steuern. So wird verhindert, dass Threads ihre Daten gegenseitig überschreiben und es zu undefinierten Ausführungsergebnissen kommt.



Grundsätzlich besitzt jeder Betriebssystemprozess seinen eigenen Adressraum. Dieser Adressraum wird vom Memory Management (Speichermanagement) verwaltet. Threads unterliegen stets dem Speichermanagement für ihren Betriebssystemprozess. Das Scheduling steuert die Ablaufreihenfolge der verschiedenen Betriebssystemprozesse einschließlich ihrer Threads und weist ihnen gemäß der Strategie des jeweiligen Betriebssystems den Prozessor zu.



Beim Wechsel zwischen Betriebssystemprozessen muss der sogenannte Kontext gesichert werden, damit keine zwei Betriebssystemprozesse sich gegenseitig beeinflussen können. Beim Speichern des Kontexts werden die grundlegendsten Dinge gespeichert, welche für das laufende Programm benötigt werden, wie beispielsweise der verwaltete Speicher, offene Dateizeiger und nicht zuletzt die Inhalte der Register des Prozessors selbst. Threads hingegen arbeiten alle im selben Kontext und greifen somit stets auf dieselben Ressourcen zu. Beim Wechsel zwischen Threads werden somit Zeiger auf Ressourcen nicht gesichert. Einzig die Prozessorregister werden abgelegt, damit ein unterbrochener Thread später im Programmablauf unbeeinflusst fortfahren kann.

Der Vorteil von Threads gegenüber Betriebssystemprozessen liegt in der Geschwindigkeit, mit der es möglich ist, zwischen den Threads zu wechseln.



23.2 Verwendung der Standard-Bibliothek <threads.h>

Für die Unterstützung von Threads existiert die Bibliothek <threads.h>, die Funktionen zur Steuerung der Aktivität von Threads wie beispielsweise zum Erzeugen, Verwalten und Löschen von Threads bereitstellt.



Ferner gibt es Timer-Funktionen sowie Abfrage- und Vergleichsfunktionen.

Zusätzlich bietet diese Header-Datei die Möglichkeit, mit sogenannten Mutexen und Zustandsvariablen zu arbeiten und einen lokalen Thread-Speicher zu definieren.



Mutexe und Zustandsvariablen sind wichtig für die Synchronisation nebenläufiger Vorgänge und werden in Kapitel [→ 23.3](#) beschrieben. Die Umsetzung lokaler Thread-Speicher wird in Kapitel [→ 23.5.3](#) analysiert.

23.2.1 Erzeugen von Threads

Ein Thread lässt sich mittels folgender Funktion erzeugen:

```
int thrd_create(thrd_t* thr, thrd_start_t func, void* arg);
```

Dieser Funktion wird die Referenz auf ein Identifikationsobjekt vom Typ `thrd_t` übergeben. Der Datentyp `thrd_t` erlaubt es, einen Thread zu identifizieren. Bei erfolgreichem Abschluss der Funktion `thrd_create()` werden in diesem Identifikationsobjekt die Daten für eine eindeutige Identifizierung des soeben erzeugten Threads gespeichert. Sie können beispielsweise verwendet werden, um den Thread später wieder zu beenden.

Der Übergabeparameter `func` ist ein Funktionspointer vom Typ `thrd_start_t`. Dieser Datentyp ist als Funktionspointer definiert und zeigt auf die vom Thread auszuführende Funktion mit der Signatur `int funcName(void* arg)`.

Beim letzten Übergabeparameter `arg` handelt es sich um einen Pointer auf `void`, der als Argument an die Threadfunktion `func` dient.

Bei Aufruf der Funktion `thrd_create()` wird ein neuer Thread kreiert, welcher sodann nebenläufig zum aktuellen Thread abgearbeitet wird. Sobald der Thread erfolgreich initialisiert wurde, wird der Thread gestartet, indem er die im Parameter `func` übergebene Funktion aufruft. Bei diesem Aufruf wird derjenige Wert, der dem formalen Parameter `arg` übergeben wurde, als aktueller Parameter an eben diese Funktion übergeben.

Ein einfaches Beispiel soll das Erzeugen von Threads veranschaulichen:

mythread.c

```
#if !defined(__STDC_NO_THREADS) || __STDC_NO_THREADS__
    #error "Threads are not available"
#endif

#include <stdio.h>
#include <threads.h>
#define THREAD_ANZAHL 5

int ausgabethread(void* arg) {
    int threadid = *(int*)arg;
    printf("Thread Nr. %i meldet sich.\n", threadid);
    return thrd_success;
}

int main(void) {
    // Array-Variable zum Speichern der Threads und IDs.
    thrd_t threads[THREAD_ANZAHL];
    int threadids[THREAD_ANZAHL];
    int i;

    for (i = 0; i < THREAD_ANZAHL; ++i) {
        printf("Erzeuge Thread Nr. %i.\n", i);
        // Thread erzeugen und ausführen.
        threadids[i] = i;
        if (thrd_create(&threads[i], ausgabethread, &threadids[i])
            != thrd_success) {
            printf("Thread Nr. %i wurde nicht erzeugt.\n", i);
        }
    }

    for (i = 0; i < THREAD_ANZAHL; ++i) {
        // Warten, bis Threads beendet wurden.
        thrd_join(threads[i], NULL);
    }
}
```

Es werden fünf Threads nacheinander in einer Schleife erzeugt. Alle Threads führen dieselbe Funktion aus. Beim Aufrufen der Funktion `thrd_create()` wird in diesem Beispielprogramm über den Parameter `arg` jedem Thread eine eindeutige Identifikationsnummer übergeben. Innerhalb der Funktion `ausgabethread()` wird dieser Parameter `arg` mittels des korrekten Type-Casts ausgelesen und per `printf()` ausgegeben. Die Threads enden mit dem Rücksprung aus der Funktion `ausgabethread()` durch einen `return`-Befehl.

In der Schleife der `main()`-Funktion wird mithilfe von `thrd_join()` auf das Beenden der Threads gewartet. Dies dient dazu, um sicherzustellen, dass bei Programmende alle geöffneten Threads auch wieder geschlossen sind und der Betriebssystemprozess sauber heruntergefahren werden kann.

Die Programmausgabe ist beispielsweise:

```
Erzeuge Thread Nr. 0.  
Erzeuge Thread Nr. 1.  
Erzeuge Thread Nr. 2.  
Thread Nr. 0 meldet sich.  
Thread Nr. 1 meldet sich.  
Erzeuge Thread Nr. 3.  
Erzeuge Thread Nr. 4.  
Thread Nr. 3 meldet sich.  
Thread Nr. 2 meldet sich.  
Thread Nr. 4 meldet sich.
```

23.2.2 Beenden von Threads

Damit ein Thread beendet werden kann, muss die als Thread aufgerufene Funktion entweder per `return`-Befehl beendet werden oder aber diese Funktion muss die Funktion `thrd_exit()` aufrufen. In beiden Fällen werden die Ressourcen des Threads freigegeben und das Rückgaberesultat an definierter Stelle festgehalten. Dieses Rückgaberesultat kann daraufhin aus einem noch laufenden Thread durch Aufruf der Funktion `thrd_join()` ausgewertet werden. Durch die Funktion `thrd_join()` wird überprüft, ob ein gewünschter Thread beendet wurde. Falls der zu beendende Thread noch ausgeführt wird, wartet die Funktion `thrd_join()` solange, bis dieser Thread beendet ist.

Da ein Thread von der erzeugenden Funktion `thrd_create()` asynchron gestartet wird, kann er beim Beenden nicht mehr zu dieser Funktion zurückkehren, wie sonst bei synchronen Aufrufen üblich.

Mit der Funktion `thr_exit()` existiert ein sogenannter „termination handler“, der für das Freigeben der belegten Thread-Ressourcen sorgt, wenn ein Thread beendet wird.



In der Regel können Threads per `return`-Befehl beendet werden, da der `return`-Befehl implizit die Funktion `thr_exit()` aufruft. Mit der Funktion `thr_exit()` besteht jedoch die Möglichkeit, einen Thread auch aus einer vom Thread aufgerufenen Funktion – also aus einer tieferen Ebene der Aufrufhierarchie – zu beenden.



Oft werden Threads dazu verwendet, zyklische Arbeiten in einem Programmablauf zu erledigen. Dabei sind zyklische Arbeiten häufig als **Endlosschleife** implementiert. Damit ein Thread beendet werden kann, gibt es zwei verschiedene Techniken:

- Das zyklische Abfragen des Werts einer globalen Variablen, die als Abbruchbedingung eines Threads verwendet wird.
- Die ereignisorientierte Benachrichtigung eines Threads über den eingetretenen Wert einer Zustandsvariablen (siehe Kapitel [→ 23.3.4](#)).

Liegt der entsprechende Wert vor, so wird die Endlosschleife verlassen und der Thread beendet.

Liegt ein entsprechender Wert niemals vor, so kann es beim Warten auf das Beenden von Threads mit der Funktion `thr_join()` zu einer Systemblockade (auf Englisch „**Deadlock**“) kommen, wenn der zu beendende Thread in einer Endlosschleife läuft.



Deadlocks werden in Kapitel [→ 23.4](#) behandelt.

Das Beenden eines Threads, der als Endlosschleife implementiert wurde, wird im folgenden Beispiel veranschaulicht:

mythread2.c

```
#include <stdio.h>
#include <stdbool.h>
#include <threads.h>

static bool beenden = false;

int tfunction(void* targ) {
    // Thread initialisieren
    int i = 0;

    // Thread zyklisch ausfuehren.
    while (1) {
        ++i;
        printf("Laufvariable = %i\n", i);
        // Abfragen, ob Thread beendet werden soll.
        if (beenden == true) {
            break;
        }
    }
    return thrd_success;
}

int main(void) {
    thrd_t thread;

    thrd_create(&thread, tfunction, NULL);

    for (int i = 0; i < 10000; ++i) {
        printf("Warten = %i\n", i);
    }

    // Globale Variable fuer Thread-Abbruch auf true setzen.
    beenden = true;
    thrd_join(thread, NULL);
}
```

Die Funktion `main()` startet einen Thread `tfunction()`, der eine Laufvariable `i` in einer Endlosschleife zyklisch ausgibt. Vor Abschluss der Endlosschleife wird bei jedem Schleifendurchgang geprüft, ob die globale Variable `beenden` gesetzt wurde. Trifft dies zu, so wird die Endlosschleife abgebrochen und der Thread beendet. Währenddessen führt die Funktion `main()` eine Berechnung durch und gibt ebenfalls Daten auf der Konsole aus. Nach Abschluss der Berechnung wird im Hauptprogramm `main()` die globale Variable `beenden` gesetzt und mit der Funktion `thrd_join()` darauf gewartet, dass sich der nebenläufige Thread beendet. Würde `beenden` nicht auf `true` gesetzt werden, könnte der Prozess des Programmes nicht beendet werden und müsste manuell über das Betriebssystem terminiert werden, da der Thread `tfunction()` weiterhin aktiv wäre und `thrd_join()` endlos warten würde.

Die Programmausgabe ist:

```
...  
Warten = 9994  
Warten = 9995  
Warten = 9996  
Warten = 9997  
Warten = 9998  
Warten = 9999  
Laufvariable = 20135
```

In dem gezeigten Beispiel wird ein einfacher Synchronisationsmechanismus in Form der wiederholten Abfrage der globalen Variablen zu Veranschaulichungszwecken verwendet. Ereignisorientierte Programme benutzen jedoch anstelle solcher globalen Variablen besser die in <threads.h> zur Verfügung gestellten Synchronisations-Strukturen wie beispielsweise Mutexe. Siehe dazu Kapitel

→ 23.3.1 .

23.2.3 Funktionen zur Verwaltung von Threads

Das Erstellen und Beenden von Threads ist im Ansatz sehr einfach, doch sollte die Information über alle laufenden Threads stets in einem Programm sauber gespeichert werden.

Die folgende Auflistung enthält nebst dem Erstellen und Beenden von Threads weitere in <threads.h> verfügbare Funktionen für das Verwalten von Threads:

```
int thrd_create(thrd_t* thr, thrd_start_t func, void* arg);
```

Erzeugt einen neuen Thread mit dem Funktionsnamen func. An diese Funktion werden mit arg Daten übergeben. Diese Funktion speichert die Identifikation des erzeugten Threads in thr.

```
thrd_t thrd_current(void);
```

Liefert die ID des aufrufenden Threads.

```
int thrd_detach(thrd_t thr);
```

Löst einen Thread von seiner Umgebung. Sobald der Thread beendet wird, werden automatisch alle seine Ressourcen freigegeben.

```
int thrd_equal(thrd_t thr0, thrd_t thr1);
```

Prüft, ob zwei gegebene Thread-Strukturen auf den gleichen Thread zeigen.

```
_Noreturn void thrd_exit(int res);
```

Beendet einen Thread.

```
int thrd_join(thrd_t thr, int* res);
```

Wartet auf das Ende eines Threads und speichert dessen Rückgabewert in res.

```
int thrd_sleep(const struct timespec* duration, struct timespec* remaining);
```

Unterbricht einen Thread für die vorgegebene Dauer. Speichert die Rest-Dauer, falls die Wartezeit durch ein Signal unterbrochen wurde.

```
void thrd_yield(void);
```

Ein Thread unterbricht sich selbst und gibt seine verbleibende Ausführungszeit an andere Threads ab.

23.3 Synchronisation von Threads

Das Synchronisieren von Threads, um Daten kontrolliert parallel verarbeiten zu können, ist ein entscheidender Aspekt für ein fehlerfreies Multithreading.



Alle Threads eines Betriebssystemprozesses haben Zugriff auf dessen Ressourcen und können damit innerhalb des Adressraums des Betriebssystemprozesses untereinander Daten austauschen. Jedoch wird schnell klar, dass dieser Datenaustausch zu Problemen führen kann, wenn mehrere Threads quasi gleichzeitig auf dieselben Daten lesend und schreibend zugreifen möchten. Dies wird als Konflikt (auf Englisch „conflict“ oder auch „clash“) bezeichnet.

Bei einem Konflikt werden die Daten unbrauchbar, da je nach Reihenfolge, wie die Threads lesend und schreibend auf die Daten zugreifen, jeweils ein anderes Resultat herauskommt. In der Fachsprache wird dies als Reader-Writer-Problem bezeichnet. Es ist ein Beispiel für eine sogenannte **Race Condition** (auf Deutsch „Wettlaufsituation“ oder „kritischer Wettlauf“), bei der die Ausführungsreihenfolge über das Ergebnis entscheidet.

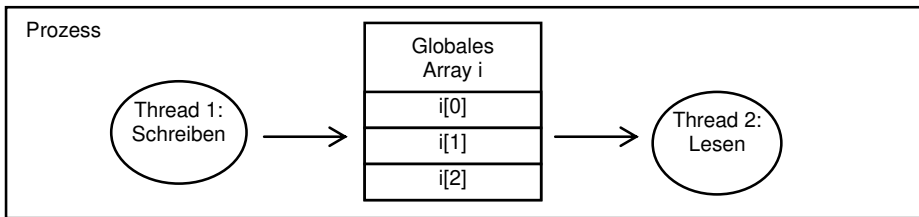
Wenn mindestens ein Thread schreibend auf Daten zugreift, welche auch von anderen Threads genutzt werden, müssen diese Daten vor gleichzeitigem Zugriff geschützt werden. Ein Abschnitt eines Programms, der auf die gemeinsam genutzten und damit zu schützenden Daten zugreift, wird als **kritischer Abschnitt** (auf Englisch „critical section“) bezeichnet.



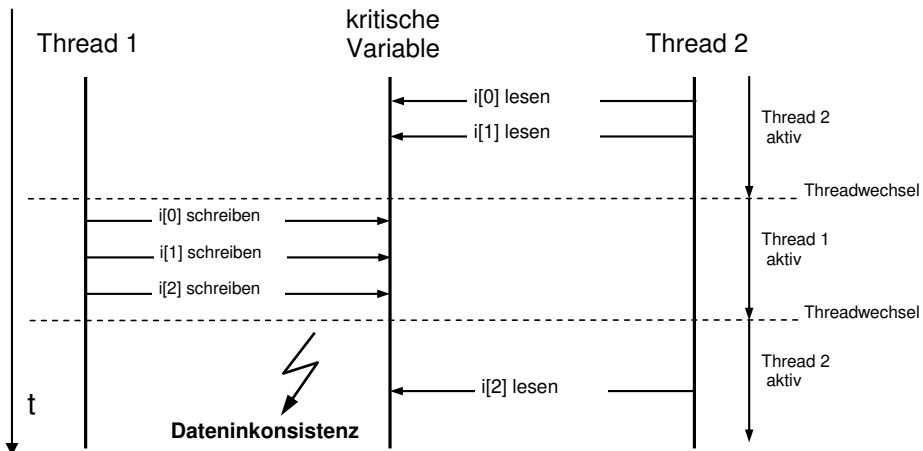
23.3.1 Konzeptionelles Beispiel für den Einsatz von Mutexen

In einem Betriebssystemprozess sollen zwei Threads ausgeführt werden, die beide auf eine globale Array-Variable i der Länge 3 zugreifen sollen. Thread 1 soll das Array mit Daten beschreiben, während Thread 2 die Daten des Arrays lesen soll. Die Datenkonsistenz muss bei dem Zugriff der verschiedenen Threads gewährleistet werden.

Das folgende Bild zeigt einen lesenden und einen schreibenden Thread, die beide dasselbe Array verwenden:



Der Zugriff auf das globale Array soll im Folgenden in einem Gedankenexperiment ohne Synchronisation erfolgen, sodass beide Threads beliebig lesen und schreiben können. Das folgende Sequenzdiagramm veranschaulicht, was passieren kann, wenn der Zugriff auf das Array nicht überwacht wird:



Thread 2 möchte in diesem Beispiel die Daten des Arrays lesen. Nachdem ein Teil der Daten gelesen wurde, führt der Betriebssystem-Scheduler einen Threadwechsel durch. Thread 1 darf nun arbeiten und überschreibt das Array mit neuen Daten. Nun wird Thread 2 fortgesetzt und liest die verbleibenden Datenelemente aus dem Array. Die dabei gelesenen Array-Elemente passen jedoch nicht zu den Elementen des Datensatzes, die zuvor bereits ausgelesen wurden, da die Array-Elemente des ursprünglichen Datensatzes inzwischen von Thread 1 überschrieben wurden. Die geforderte Datenkonsistenz ist damit verletzt.

Es ist also wichtig, den Zugriff auf eine gemeinsam genutzte Ressource zu regeln. Als Lösung für dieses Problem werden sogenannte **Mutexe** (Abkürzung des englischen „mutual exclusion“) zur Zugriffssteuerung verwendet. Ein Mutex wird zum Schutz von gemeinsam genutzten Daten eingesetzt.

Wenn sämtliche Threads, welche auf gemeinsame Daten zugreifen, ihre kritischen Abschnitte durch einen gemeinsamen Mutex sichern, bleiben die Daten konsistent. Der Mutex verhindert, dass kritische Abschnitte anderer Threads auf die Daten zugreifen können, während der aktive Thread seinen kritischen Abschnitt abarbeitet oder gar während dieser Abarbeitung durch den Scheduler unterbrochen wird.

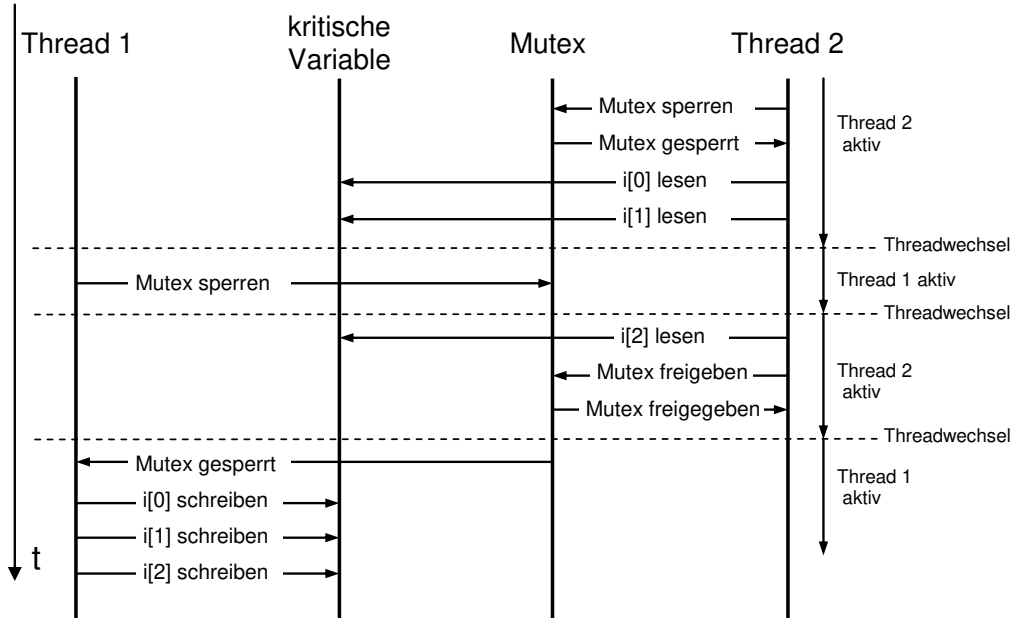


Vor dem Zugriff eines Threads auf die geschützte Ressource muss der zum Schutz der Daten erzeugte Mutex abgefragt werden. Dieser Vorgang wird auch „lock“ (auf Deutsch „sperren“) genannt. Ist der Mutex bereits durch einen Thread gesperrt, so muss gewartet werden, bis der Mutex wieder freigegeben wird.

Nach dem erfolgreichen Sperren des Mutex einer Ressource kann ein Thread auf die geschützte Ressource zugreifen, ohne dass andere Threads diese Ressource verändern können, selbst wenn sie vom Betriebssystem zur Ausführung gebracht werden.

Ist die Bearbeitung der Ressource abgeschlossen, muss der Thread, der die Ressource gesperrt hat, den gesperrten Mutex wieder freigeben, was als „unlock“ (zu Deutsch „freigeben“) bezeichnet wird. Andere Threads haben dadurch wieder die Möglichkeit, den Mutex zu sperren und anschließend selbst auf die Ressource zuzugreifen.

In dem folgenden Sequenzdiagramm soll dieser Zugriffsschutz durch einen Mutex veranschaulicht werden:



Wieder greifen zwei Threads auf ein globales Array zu. Vor dem Zugriff muss der Mutex, der die globalen Daten schützt, gesperrt werden. Im Beispiel versucht Thread 2 erfolgreich, den Mutex zu sperren und kann anschließend das globale Array auslesen. Nach dem zweiten gelesenen Element wird der Thread vom Betriebssystem unterbrochen und Thread 1 wird ausgeführt. Dieser versucht nun ebenfalls, den Mutex zu sperren, um die Elemente des globalen Arrays zu beschreiben. Da der Mutex aber bereits durch Thread 2 gesperrt ist, kann Thread 1 nicht weiter ausgeführt werden und wird wieder unterbrochen. Er muss warten, bis der Mutex wieder freigegeben wurde. Sobald Thread 2 wieder abläuft, werden die verbleibenden Daten aus dem globalen Array gelesen und der Mutex wird wieder freigegeben. Jetzt kann Thread 1 den Mutex sperren und das Array beschreiben. So ist sichergestellt, dass Thread 2 immer konsistente Daten lesen kann.

Der kritische Abschnitt sollte so gewählt sein, dass die Wahrscheinlichkeit eines Konfliktes so gering wie möglich ist. Dementsprechend gibt es Datenstrukturen, welche für Multithreading mehr oder weniger gut geeignet sind.



Entscheidend ist dabei unter anderem auch die sogenannte „Granularität“, sprich, die Feinheit der Sperrung. Wenn wie im soeben gezeigten Beispiel ein gesamtes (potentiell sehr großes) Array gesperrt wird, ist die Wahrscheinlichkeit eines Konfliktes sehr groß, dafür ist die Sperrung sehr einfach zu vollziehen. Wird hingegen beispielsweise in einer linearen Liste ein einzelnes Element gesperrt, so ist die Wahrscheinlichkeit eines Konfliktes sehr gering, dafür aber muss jedes einzelne Element seinen eigenen Mutex besitzen.

23.3.2 Beispielprogramm für den Einsatz von Mutexen

Im folgenden Beispiel greifen zwei Threads auf ein globales Array zu. Der als `writerThread` bezeichnete Thread sperrt den Mutex und inkrementiert alle drei Array-Elemente. Anschließend gibt er den Mutex wieder frei. Der Thread `readerThread` sperrt ebenfalls den Mutex, um aus dem Array zu lesen. Er gibt alle drei Array-Elemente auf der Konsole per `printf()`-Befehl aus. Danach gibt auch er den Mutex wieder frei. Mit der Funktion `thrd_yield()` geben die Threads nach Beenden eines Schleifendurchlaufs ihre verbleibende Ausführungszeit ab und das Betriebssystem führt den nächsten Thread aus. Dies kann auch derselbe Thread sein, der soeben den Threadwechsel eingeleitet hat.

Die verwendeten Funktionen werden in Kapitel [→ 23.3.3](#) etwas detaillierter beschrieben. Hier das Programm:

mutex.c

```
#include <stdio.h>
#include <threads.h>
#include <stdbool.h>

// globale Variable fuer Mutex
mtx_t mutex;

// gemeinsam genutzte Ressource
int i[3] = {0, 0, 0};
int terminate = false;

int writerThread(void* arg) {
    while (!terminate) {
        mtx_lock(&mutex);

        printf("Schreibe Daten\n");
        ++i[0];
        ++i[1];
        ++i[2];
    }
}
```

```
    mtx_unlock(&mutex);
    thrd_yield();
}
return thrd_success;
}

int readerThread(void* arg) {
    while (!terminate) {
        mtx_lock(&mutex);

        printf("lese Daten: %i, %i, %i\n", i[0], i[1], i[2]);

        mtx_unlock(&mutex);
        thrd_yield();
    }
    return thrd_success;
}

int main(void) {
    mtx_init(&mutex, mtx_plain);

    thrd_t reader;
    thrd_t writer;
    thrd_create(&reader, readerThread, NULL);
    thrd_create(&writer, writerThread, NULL);

    for (int i = 0; i < 10000000; ++i) {
        // Threads fuer eine Weile ausfuehren.
    }

    terminate = true;
    thrd_join(reader, NULL);
    thrd_join(writer, NULL);
    mtx_destroy (&mutex);
}
```

Beispiel für einen Programmlauf (Ausschnitt):

```
lese Daten: 0, 0, 0
lese Daten: 0, 0, 0
schreibe Daten
lese Daten: 1, 1, 1
schreibe Daten
lese Daten: 2, 2, 2
schreibe Daten
lese Daten: 3, 3, 3
schreibe Daten
schreibe Daten
schreibe Daten
lese Daten: 6, 6, 6
```

Beim Ausführen des Codes fallen zwei Dinge auf. Erstens ist schon anhand des kurzen Ablaufausschnitts ersichtlich, dass die Threads in beliebiger Reihenfolge vom Betriebssystem ausgeführt werden. Dabei führt das Betriebssystem denselben Thread oft mehrmals hintereinander aus. Zweitens kann beobachtet werden, dass die gelesenen Daten aller drei Array-Felder immer identisch sind, da es sich um eine synchronisierte Ressource handelt. Würde man bei diesem Beispielcode auf das Sperren und Entsperren des Mutex verzichten, käme es früher oder später zu einem Auseinanderlaufen der Zahlenwerte des Arrays.

Der kritische Abschnitt beim Zugriff auf die drei Array-Werte ist sehr kurz, somit ist auch die Wahrscheinlichkeit gering, dass ein Synchronisationsfehler auftritt, wenn im Code keine Synchronisationsmechanismen verwendet werden. Sie steigt aber mit fortschreitender Ausführungsdauer. Außerdem sorgt schon ein einziger Synchronisationsfehler dafür, dass bei allen Folgedurchläufen inkonsistente Daten verwendet werden. Die Erfahrung zeigt, dass selbst die scheinbar unwahrscheinlichsten Synchronisationsfehler beim tatsächlichen Ablauf eines Programmes erstaunlich oft zu Problemen führen können.

Beispiel für einen Programmlauf ohne Mutex (Ausschnitt):

```
lese Daten: 5647, 5647, 5647
schreibe Daten
schreibe Daten
lese Daten: 5649, 5649, 5649
schreibe Daten
lese Daten: 5650, 5650, 5650
schreibe Daten
lese Daten: 5651, 5650, 5650
```

23.3.3 Funktionen zur Mutex-Steuerung

Die Bibliothek `<threads.h>` enthält eine Reihe an Funktionen, die zur Bedienung der Mutexe erforderlich sind. Hier wird auch der Datentyp `mtx_t` für ein Mutex-Objekt festgelegt. Folgendes ist eine Auflistung der bereitgestellten Funktionen:

```
int mtx_init(mtx_t* mtx, int type);
```

Erzeugt einen neuen Mutex.

```
void mtx_destroy(mtx_t* mtx);
```

Löscht einen Mutex.

```
int mtx_lock(mtx_t* mtx);
```

Sperrt einen Mutex. Ist er bereits gesperrt, wird so lange versucht, ihn zu sperren, bis dies wieder möglich ist.

```
int mtx_timedlock(mtx_t* restrict mtx, const struct timespec* restrict ts);
```

Versucht, solange einen Mutex zu sperren, bis er frei ist oder eine vorgegebene Zeit überschritten wird. Nur für Mutexe verfügbar, die Timeouts unterstützen.

```
int mtx_trylock(mtx_t* mtx);
```

Versucht, einen Mutex zu sperren. Ist er bereits gesperrt, passiert nichts.

```
int mtx_unlock(mtx_t* mtx);
```

Gibt einen Mutex wieder frei.

23.3.4 Synchronisation mit Zustandsvariablen

Eine weitere Möglichkeit, Threads zu synchronisieren, stellen die sogenannten **Zustandsvariablen** (auf Englisch „**condition variables**“) dar.

Oftmals muss ein Thread darauf warten, dass bestimmte Bedingungen erfüllt sind beziehungsweise das Programm sich in einem bestimmten Zustand befindet. Beispielsweise warten Threads darauf, dass Daten vollständig geladen werden, bevor sie mit der Verarbeitung beginnen. Oder sie warten darauf, dass eine Warteschlange wieder genügend Platz hat, um neue Elemente einzufügen.

Eine Zustandsvariable erlaubt die Benachrichtigung (auf Englisch „notification“) von Threads über das Eintreffen eines bestimmten Zustandes.



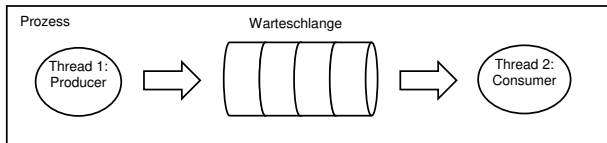
Wenn ein Thread darauf wartet, dass ein Zustand eintritt, hat er zwei Möglichkeiten: Entweder fragt der Thread den gewünschten Zustand in einem sogenannten „busy loop“ ständig ab, bis er eingetreten ist. Dies wird als „**Polling**“ bezeichnet und braucht sehr viel CPU-Zeit, die durch andere Threads oftmals besser genutzt werden könnte. Die zweite Möglichkeit ist, auf eine Zustandsvariable zu warten, bei welcher der Scheduler die Aufgabe übernimmt, dem wartenden Thread mitzuteilen, dass der Zustand eingetreten ist. Während der Wartezeit ist der Thread blockiert und gibt somit seine CPU-Zeit an andere Threads ab. Welcher Thread die Zustandsvariable setzt, ist frei wählbar.

Beim Warten auf das Eintreten eines bestimmten Wertes einer Zustandsvariablen gibt ein Thread seine verbleibende CPU-Zeit ab und lässt andere Threads arbeiten. Seine Arbeit nimmt er erst wieder auf, sobald er über den gewünschten Zustand informiert wurde, oder die Wartezeit abgelaufen ist und der Thread vom Betriebssystem gescheduled wurde. Es wird in diesem Fall also verhindert, dass wartende Threads das Erreichen des gewünschten Zustands pollen müssen.

23.3.5 Konzeptionelles Beispiel ohne Synchronisation

Anhand des sogenannten Producer-Consumer-Problems (zu Deutsch „Erzeuger-Verbraucher-Problem“) wird die Notwendigkeit von Zustandsvariablen erklärt. Zwei Threads sollen in folgendem Beispiel in einem Betriebssystemprozess arbeiten. Ein Thread erzeugt Daten (Producer) und legt sie in eine **Warteschlange** (auf Englisch „**Queue**“) mit einer begrenzten Kapazität, sofern diese noch nicht voll ist. Ein weiterer Thread (Consumer) prüft, ob Daten in der Warteschlange enthalten sind. Solange Daten in der Warteschlange enthalten sind, entnimmt und verarbeitet er diese. Ist die Warteschlange leer, wartet der Consumer-Thread, bis neue Daten verfügbar sind.

Das folgende Bild symbolisiert die Situation beim Producer-Consumer-Problem:



Warteschlangen kommen in Multithreading-Programmen häufig zum Einsatz. C bietet keine spezielle Bibliothek für Warteschlangen an. Warteschlangen werden häufig als sogenannte Ringpuffer implementiert. Hier in diesem Buch wird nicht weiter darauf eingegangen.



Hier der Code-Ausschnitt der beiden Thread-Schleifen mit Polling:

```
// Producer Thread
while (true) {
    erzeugeDaten();

    // Warten, bis Queue nicht mehr voll ist
    while (queueStatus() == VOLL) {}    // Polling (busy loop)

    // Schreibe Daten in die Queue
    queueDatenSchreiben();
}
```

```
// Consumer Thread
while (true) {
    // warten, bis Daten in der Queue verfuegbar sind
    while (queueStatus() == LEER) {}    // Polling (busy loop)

    // lese Daten aus der Queue
    queueDatenLesen();
}
```

Anhand dieses Code-Ausschnitts wird deutlich, dass aufgrund der ständigen Wiederholung der Zustandsabfrage (Polling) Rechenleistung verschwendet wird.

Stellt der Producer-Thread fest, dass die Warteschlange voll ist, führt er solange erneut die Abfrage durch, bis wieder Platz vorhanden ist. Diese Bedingung kann aber erst erfüllt sein, wenn der Consumer-Thread mindestens ein Element aus der Warteschlange entnommen hat. Der Producer-Thread läuft somit unnötig lange, bis das Betriebssystem die Threads wechselt und den Consumer-Thread ausführt.

Dasselbe Problem entsteht im Consumer-Thread, wenn die Warteschlange leer ist. Der Consumer-Thread prüft in einer Schleife, ob neue Elemente verfügbar sind. Auch hier kann die Bedingung für ein Verlassen der Warteschleife erst erfüllt werden, wenn das Betriebssystem die Threads gewechselt und der Producer-Thread wieder neue Daten in die Warteschlange gelegt hat.

Ein zusätzliches Problem zeigt sich, wenn nicht nur jeweils ein Producer- und Consumer-Thread vorhanden ist, sondern mehrere Threads Daten erzeugen und konsumieren. Durch die fehlende Synchronisation kann eine Race Condition entstehen, wenn bei der Abfrage des Warteschlangen-Status oder beim Schreiben und Entnehmen von Daten ein Threadwechsel stattfindet. Dadurch können die Daten der Warteschlange korumpiert werden.

Datenkorruption kann verhindert werden, indem die Warteschlange als gemeinsam genutzte Ressource mit einem Mutex gesichert wird. Dies löst jedoch nicht das Problem, dass sowohl Producer als auch Consumer häufig im Leerlauf sind.

23.3.6 Programmieren mit Zustandsvariablen

Um die zyklische Abfrage, also das Pollen, aus dem vorherigen Beispiel zu umgehen, werden für die Warteschlange zwei Zustandsvariablen `queueNotEmpty` und `queueNotFull` eingeführt, auf die die Threads reagieren können.

Zustandsvariablen werden immer in Kombination mit einer gemeinsam genutzten Ressource verwendet, die durch einen Mutex geschützt ist.



Die Funktionen für Zustandsvariablen, die durch die Bibliothek `<threads.h>` bereitgestellt werden, werden in der folgenden Auflistung kurz beschrieben. Dabei entspricht `cnd_t` dem Typ einer Zustandsvariablen.

```
int cnd_broadcast(cnd_t* cond);
```

Gibt alle Threads frei, die auf einen gegebenen Zustand warten.

```
void cnd_destroy(cnd_t* cond);
```

Löscht eine Zustandsvariable.

```
int cnd_init(cnd_t* cond);
```

Erzeugt eine neue Zustandsvariable.

```
int cnd_signal(cnd_t* cond);
```

Gibt einen Thread frei, der auf einen gegebenen Zustand wartet.

```
int cnd_timedwait(cnd_t* restrict cond, mtx_t* restrict mtx, const  
struct timespec * restrict ts);
```

Nur in Verbindung mit Mutexen zu verwenden. Gibt einen Mutex wieder frei, wenn ein bestimmtes Signal erhalten wird oder eine vorgegebene Zeit abgelaufen ist.

```
int cnd_wait(cnd_t* cond, mtx_t* mtx);
```

Nur in Verbindung mit Mutexen zu verwenden. Gibt einen Mutex wieder frei, wenn ein bestimmtes Signal erhalten wird.

Ein typischer Ablauf funktioniert in etwa folgendermaßen:

Ein Thread, der innerhalb eines kritischen Abschnitts merkt, dass eine notwendige Bedingung nicht erfüllt ist, kann dem Scheduler durch Aufruf der Funktion `cnd_wait()` mitteilen, dass er nun darauf wartet, dass ein gewünschter Zustand eintritt. Der Scheduler blockiert daraufhin diesen Thread und aktiviert ihn erst

dann wieder, wenn dies der Fall ist. Da der Aufruf der Funktion `cnd_wait()` innerhalb eines kritischen Abschnitts stattfindet, erwartet die Funktion den Mutex dazu und gibt ihn automatisch frei, damit andere Threads auf die gemeinsame Ressource zugreifen können.

Wenn ein anderer Thread die gemeinsame Ressource in einen bestimmten Zustand versetzt hat, teilt er die entsprechende Zustandsänderung mittels der beiden Funktionen `cnd_signal()` oder `cnd_broadcast()` dem Scheduler mit. Der Scheduler benachrichtigt daraufhin im nächsten Scheduling-Zyklus diejenigen Threads, welche auf den gewünschten Zustand warten. Bei Aufruf der Funktion `cnd_signal()` wird der Scheduler im nächsten Scheduling-Zyklus genau einen wartenden Thread auswählen, der benachrichtigt wird. Mit der Funktion `cnd_broadcast()` werden sämtliche wartenden Threads nacheinander über die Veränderung des Zustandes benachrichtigt.

Der Scheduler versucht bei beiden Benachrichtigungsfunktionen den bei Aufruf der Funktion `cnd_wait()` oder `cnd_timedwait()` angegebenen Mutex zu sperren. Sobald er dies kann, lässt er den Mutex gesperrt und teilt dem wartenden Thread erneut CPU-Zeit zu, indem er die Funktion `cnd_wait()` des Threads beendet und ihm so erlaubt, weiterzuarbeiten. Der Thread befindet sich durch den erneut gesperrten Mutex genauso wie vor Aufruf der Funktion `cnd_wait()` wieder im kritischen Abschnitt und kann ihn erfolgreich weiterbearbeiten.

23.3.7 Beispiel mit Synchronisation durch Benachrichtigung über Zustände

Im folgenden Programm werden zwei Consumer-Threads, aber nur ein einziger Producer-Thread erzeugt, die wie im vorherigen Beispiel schreibend und lesend auf eine Warteschlange zugreifen. Diese wird hier durch eine gemeinsam genutzte Zählvariable `elementZaehler` symbolisch dargestellt. Der Producer-Thread erhöht beim Zugriff den Wert der Zählvariablen, sofern dieser ein definiertes Limit `QUEUE_SIZE` nicht überschreitet.

Die beiden Consumer-Threads hingegen verringern beim Zugriff auf die Warteschlange den Wert der Zählvariablen `elementZaehler`, was dem Herausnehmen eines Warteschlangenelements entspricht. Der Zugriff auf die Zählvariable `elementZaehler` stellt einen kritischen Abschnitt dar und muss deshalb geschützt werden. Ein Mutex `queueMutex` übernimmt diese Aufgabe und muss von allen Threads vor dem Zugriff auf die Zählvariable `elementZaehler` gesperrt werden. Zusätzlich werden zwei Zustandsvariablen `queueNotEmpty` und `queueNotFull` verwendet, auf die die Threads reagieren können.

Der folgende Code zeigt das vollständige Beispielprogramm:

conditionvar.c

```
#include <stdio.h>
#include <threads.h>
#include <stdbool.h>

cnd_t queueNotEmpty, queueNotFull;
mtx_t queueMutex;
#define QUEUE_SIZE 5
int elementZaehler = 0;
int terminate = false;

int producerThread(void* arg) {
    while (!terminate) {
        printf("Erzeuge Daten\n");
        mtx_lock(&queueMutex);

        while (elementZaehler == QUEUE_SIZE) {
            cnd_wait (&queueNotFull, &queueMutex);
        }
        ++elementZaehler;
        cnd_broadcast(&queueNotEmpty);

        mtx_unlock(&queueMutex);
    }
    return thrd_success;
}

int consumerThread(void* arg) {
    int threadid = *(int*)arg;
    while (!terminate) {
        printf("Verbrauche Daten Thread %d\n", threadid);
        mtx_lock(&queueMutex);

        while (elementZaehler == 0) {
            cnd_wait(&queueNotEmpty, &queueMutex);
        }

        --elementZaehler;
        cnd_broadcast(&queueNotFull);

        mtx_unlock(&queueMutex);
    }
    return thrd_success;
}
```

```
int main(void) {
    thrd_t producer;
    thrd_t consumer1;
    thrd_t consumer2;
    int consumerid1 = 1;
    int consumerid2 = 2;

    cnd_init(&queueNotFull);
    cnd_init(&queueNotEmpty);
    mtx_init(&queueMutex, mtx_plain);

    thrd_create(&producer, producerThread, NULL);
    thrd_create(&consumer1, consumerThread, &consumerid1);
    thrd_create(&consumer2, consumerThread, &consumerid2);
    for (int i = 0; i < 10000000; ++i) {
        // Threads arbeiten lassen
    }

    terminate = true;
    thrd_join(producer, NULL);
    thrd_join(consumer1, NULL);
    thrd_join(consumer2, NULL);

    mtx_destroy(&queueMutex);
    cnd_destroy(&queueNotEmpty);
    cnd_destroy(&queueNotFull);
}
```

Der **Producer-Thread** geht dabei in den folgenden vier Schritten vor:

1. Er versucht, den Mutex `queueMutex` für den Zugriff auf die Warteschlange zu sperren. Wenn der Mutex bereits gesperrt ist, wird der Producer-Thread unterbrochen und erst wieder ausgeführt, wenn der Mutex wieder freigegeben wurde.
2. Konnte der Mutex schlussendlich vom aktuellen Producer-Thread gesperrt werden, liest dieser die Variable `elementZaehler` und prüft, ob diese noch nicht am oberen Limit `QUEUE_SIZE` angekommen und somit noch Platz in der Warteschlange ist. Zwei mögliche Szenarien können eintreten:
 - a) Die Warteschlange ist noch nicht voll. In diesem Zustand ist der Producer-Thread arbeitsfähig und kann neue Elemente in die Warteschlange einfügen.
 - b) Die Warteschlange ist bereits voll. Der Producer-Thread ruft die Funktion `cond_wait()` auf, mit der er auf das Eintreffen des Zustands `queueNotFull` wartet, also darauf, dass andere Threads Elemente aus der Warteschlange entnommen haben. Beim Aufruf der Funktion `cond_wait()` wird die gewünschte Zustandsvariable `queueNotFull` sowie der Mutex `queueMutex` der geschützten Ressource als Parameter übergeben.

Der Mutex `queueMutex` wird von der Funktion `cond_wait()` automatisch freigegeben, damit andere Threads die Warteschlange nutzen können. Die Funktion `cond_wait()` wartet auf die Benachrichtigung über das Eintreten des Zustands, die mittels der Funktionen `cond_signal()` oder `cond_broadcast()` eines anderen Threads erfolgt. Sobald dies geschieht, wird der Mutex automatisch wieder gesperrt und der Thread kann mit der Bearbeitung der geschützten Ressource fortfahren.

3. Der Producer-Thread schreibt Daten in die Warteschlange, indem er die Variable `elementZaehler` symbolisch inkrementiert.
4. Der Producer-Thread benachrichtigt mittels der Funktion `cond_broadcast()` alle Consumer-Threads, die auf das Eintreten des Zustands `queueNotEmpty` warten, also darauf, dass die Warteschlange nicht mehr leer ist. Zuletzt gibt der Producer-Thread den gesperrten Mutex wieder frei, damit andere Threads die Warteschlange nutzen und auf die geteilte Ressource `elementZaehler` zugreifen können.

Auf ähnliche Weise arbeiten die **Consumer-Threads**. Auch sie sperren den Mutex und fragen die Zählvariable `elementZaehler` ab. Dabei gibt es wiederum zwei Möglichkeiten:

1. Die Warteschlange enthält Elemente. Die Consumer-Threads können diese entnehmen und verarbeiten.
2. Die Warteschlange ist leer. Nun wird mit der Funktion `cnd_wait()` auf das Eintreffen des Zustands `queueNotEmpty` gewartet, der vom Producer-Thread signalisiert wird. Auch hier wird der Mutex durch Aufruf der Funktion `cnd_wait()` kurzerhand wieder freigegeben, sodass die Warteschlange für andere Threads verfügbar wird. Ist der gewünschte Zustand eingetreten, wird der Mutex automatisch wieder gesperrt und der Consumer-Thread kann seinen kritischen Abschnitt fortsetzen.

Nachdem die Consumer-Threads die Variable `elementZaehler` abgefragt haben, können sie den Wert der Variablen `elementZaehler` dekrementieren, was im gezeigten Beispiel dem Herausnehmen eines Elements aus der Warteschlange entspricht. Bevor der Mutex wieder freigegeben wird, benachrichtigt die Funktion `cnd_broadcast()` Threads, die auf das Eintreten des Zustands `queueNotFull` warten, also darauf, dass die Warteschlange nun nicht mehr voll ist, da nun mindestens ein Element entnommen wurde. Der Producer-Thread kann so gegebenenfalls wieder neue Elemente erzeugen.

Ein Ausschnitt des Programmablaufs ist:

```
...
Erzeuge Daten
Erzeuge Daten
Erzeuge Daten
Erzeuge Daten
Verbrauche Daten, Thread 0x006b0428
Verbrauche Daten, Thread 0x006b0590
Verbrauche Daten, Thread 0x006b0590
Verbrauche Daten, Thread 0x006b0590
Verbrauche Daten, Thread 0x006b0590
Verbrauche Daten, Thread 0x006b0590
Verbrauche Daten, Thread 0x006b0590
Erzeuge Daten
Verbrauche Daten, Thread 0x006b0428
Erzeuge Daten
...
```

Wird der gezeigte Code ausgeführt, kann ein interessantes Phänomen beobachtet werden: Da zwei Consumer-Threads gleichzeitig auf die Warteschlange zugreifen, aber nur ein einziger Producer-Thread die Warteschlange füllt, kann es vorkommen, dass zwar beide Consumer-Threads ausgeführt werden, jedoch einer davon schon die komplette Warteschlange leert, sobald er aus seinem Wartezustand durch den Aufruf von `cnd_broadcast()` geweckt wird. Aus diesem Grund wird die Bedingung nicht mit `if`, sondern mit einer `while`-Schleife geprüft. In diesem Beispiel spielt dies für den Producer keine Rolle, da nur ein einziger Producer-Thread existiert. Gäbe es jedoch mehrere Producer, so wäre die `while`-Schleife für einen problemlosen Ablauf zwingend notwendig.

Kommt nun der zweite Consumer-Thread zum Zuge, wird er feststellen, dass die Warteschlange schon wieder leer ist. Entsprechend legt er sich sofort wieder schlafen. Man bezeichnet ein solches Verhalten als Starvation (auf Deutsch „Verhungern“). Dieses und weitere Phänomene werden im folgenden Kapitel behandelt.

23.4 Deadlocks, Livelocks und Starvation

Sobald mehrere Programmabläufe parallel stattfinden, können durch Synchronisationsmechanismen unbeabsichtigt Phänomene auftreten, die den Programmablauf behindern, verfälschen oder ihn gar zum Erliegen bringen.

Ein **Deadlock** (auf Deutsch „Systemblockade“ oder „Verklemmung“) ist eine Situation, in der zwei oder mehr Threads sich gegenseitig blockieren, indem sie auf ein Ereignis oder eine Ressource eines jeweils anderen wartenden Threads warten. Die Threads behindern sich gegenseitig und damit steht das System.



Ein einfaches Beispiel hierfür sind zwei sich zankende Kinder, welche sich gegenseitig das Spielzeug weggenommen haben und es erst wieder zurückgeben, wenn das andere seines zurückgibt.

Starvation (auf Deutsch „Verhungern“) bezeichnet eine Situation, in der ein Thread nie zum Laufen kommt, da die benötigten Ressourcen von anderen Threads bereits „verbraucht“ werden. Ohne diese Ressourcen kann der Thread keine Arbeit verrichten. Er verhungert.



Als **Livelock** wird eine Sonderform von Starvation bezeichnet. Wie bei einem Deadlock blockieren sich Threads gegenseitig beim Zugriff auf gemeinsame Ressourcen. Dabei verharren die Threads jedoch nicht wie beim Deadlock in Wartestellung, sondern geben jeweils ihre bereits gesperrten Ressourcen zumindest teilweise wieder frei, worauf sie im nächsten Locking-Versuch jedoch erneut blockiert werden. Für einen Anwender scheinen die Threads zu arbeiten, jedoch gibt es keinen Arbeitsfortschritt mehr.

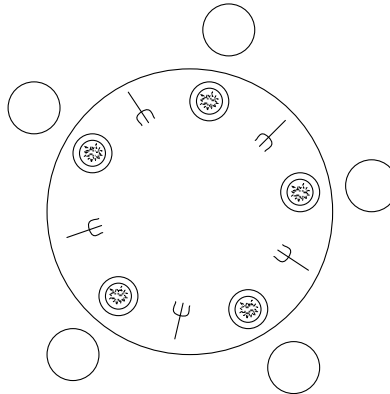


Man stelle sich die Begegnung zweier Personen auf einem engen Flur vor. Jede Person möchte aus Höflichkeit die andere vorübergehen lassen, aber beide wechseln stets zur gleichen Zeit auf die gleiche Seite hin und her, so dass es beiden unmöglich ist, zu passieren.

23.4.1 Die speisenden Philosophen

Am Beispiel des **Dining Philosopher's-Problem**s (auf Deutsch das Problem der „**speisenden Philosophen**“) können diese Phänomene anschaulich aufgezeigt werden [Hoa85]. Dabei stehen die Philosophen symbolisch für Threads, die einer gemeinsamen Tätigkeit nachgehen (speisen) und sich Ressourcen (Gabeln) teilen.

Der Tagesablauf eines Philosophen besteht abwechselnd aus Nachdenken und Essen. Fünf Philosophen sitzen an einem runden Tisch. Jeder Philosoph hat seinen festen Platz am Tisch, vor ihm einen Teller mit Spaghetti und zu seiner Linken und Rechten liegt jeweils eine Gabel, die er mit seinem direkten Nachbarn teilt. Dies soll das folgende Bild symbolisieren:



Ein Philosoph kann nur essen, wenn er zwei Gabeln in den Händen hält. Hierbei darf er jedoch nur die Gabel zu seiner linken sowie zu seiner rechten Seite verwenden. Hat sich ein Philosoph zum Essen beide Gabeln seitlich seines Tellers genommen, können seine direkten Tischnachbarn selbst nichts von ihren Tellern essen, da ihnen mindestens eine Gabel fehlt. Dies bedeutet also, dass es nicht möglich ist, dass zwei nebeneinander sitzende Philosophen zur gleichen Zeit essen. Ist ein Philosoph vorläufig fertig mit dem Essen, so legt er seine beiden Gabeln wieder an die Stelle zurück, von der er sie genommen hat und gibt die Gabeln damit seinen Tischnachbarn frei. Anschließend beginnt der Philosoph erneut zu philosophieren und nachzudenken, bis er wieder Hunger bekommt.

Solange die Philosophen nur gelegentlich hungrig sind, funktioniert das Essen und Nachdenken problemlos. Sobald aber mehrere Philosophen gleichzeitig hungrig sind und essen wollen, kann es zu Problemen kommen. Das Folgende könnte passieren: Wollen alle Philosophen im gleichen Moment essen und zuerst ihre linke Gabel aufnehmen, besitzt zwar jeder Philosoph eine Gabel, jedoch

sind alle Gabeln am Tisch nun verbraucht und keiner kann eine weitere Gabel aufnehmen, um zu essen. Jeder Philosoph wartet stattdessen darauf, dass eine Gabel zu seiner Rechten frei wird, was aber nie eintritt. Man spricht von einem Deadlock.

Eine Möglichkeit, die Situation der Philosophen zu entschärfen, bietet das Einführen einer neuen Tischregel: jeder Philosoph, der eine einzige Gabel in der Hand hält und seit einer gewissen Zeit vergeblich auf die zweite Gabel wartet, muss seine Gabel wieder ablegen. Er darf sie erst nach einiger Zeit wieder erneut aufnehmen und versuchen zu essen. Ein Deadlock kann also dadurch verhindert werden, dass auch wartende Philosophen ihre Gabel wieder freigeben. Allerdings kann dadurch ein neues Problem entstehen. Werden die Zeiten für Ablegen, Denken, Essen und Warten ungünstig gewählt, kann es vorkommen, dass einige Philosophen nie zum Essen kommen. Sie können eine der Gabeln aufnehmen, bekommen aber nie eine zweite Gabel, da ihre Nachbarn immer dann essen, wenn sie auf die zweite Gabel warten. Diese Philosophen verhungern sprichwörtlich am gedeckten Tisch. Daher spricht man in diesem Fall von Starvation.

Es kommt aber noch schlimmer für die Philosophen. Gesetzt den Fall, dass alle Philosophen gleichzeitig eine Gabel aufnehmen, um zu essen, wird durch die neu eingeführte Zeitregel bewirkt, dass die Philosophen ihre Gabeln nach einer gewissen Zeit wieder ablegen. Wenn aber alle Philosophen dies zur gleichen Zeit tun, anschließend gleich lange warten und zugleich wieder zur Gabel greifen, kommt wieder keiner der fünf Philosophen zum Essen. Das System der Philosophen ist ständig in Bewegung und ändert seine Zustände durch das Aufnehmen und Ablegen der Gabeln. Das eigentliche Ziel der Philosophen, nämlich zu essen, wird jedoch nie erreicht. Dieser Sonderfall von Starvation wird als Livelock bezeichnet. Das Wort Livelock ist eine Wortschöpfung als Gegenteil zu Deadlock.

23.5 Fortgeschrittene Programmierung mit Threads

Der Aufbau eines Programmes mit Multithreading ist herausfordernd. Nebenläufig ablaufende Programmteile sind deutlich komplexer zu durchschauen als ein sequenzieller Programmablauf. Durch fehlende oder fehlerhafte Synchronisation können Effekte auftreten, die nicht immer auf den ersten Blick nachvollziehbar sind. Dieses Kapitel beschreibt wichtige Punkte, auf die besonders geachtet werden sollte, sowie welche zusätzlichen Hilfsmittel die Bibliothek `<threads.h>` für die Programmierung von Threads bereitstellt.

23.5.1 Lokale und statische Variablen

Mithilfe von Threads lassen sich verschiedene Tätigkeiten innerhalb eines Betriebssystemprozesses nebenläufig ausführen. Dabei wird beim Erzeugen eines Threads diejenige Funktion übergeben, die nebenläufig ausgeführt werden soll. Im Gegensatz zu der sequenziellen Ausführung von Funktionen innerhalb des Programmablaufs kann dieselbe Funktion über Threads gleichzeitig mehrfach ausgeführt werden.

Threads bieten die Möglichkeit, Code-Segmente in Form von Funktionen nebenläufig auszuführen. Dabei kann ein und dieselbe Funktion mehrfach als Thread instanziiert und somit nebenläufig ausgeführt werden.



Diese Tatsache gilt es zu beachten, vor allem dann, wenn eine Funktion eigene Ressourcen beispielsweise in Form von `static` Variablen besitzt, die durch andere Threads verändert werden können. Da jeder Thread seinen eigenen Stack besitzt, können lokale Variablen nur vom eigenen Thread angesprochen und verändert werden. Im Gegensatz dazu existieren `static` Variablen jedoch ein einziges Mal im Betriebssystemprozess und können von jeder Thread-Instanz angesprochen werden. Wird die **statische Variable** einer Funktion von einem Thread beschrieben, so ist diese Änderung für alle Thread-Instanzen sichtbar, die ebenfalls diese Funktion aufrufen. Dies hat zur Folge, dass Synchronisationsmechanismen für die Verwendung dieser Funktion in mehreren Threads notwendig sind, da sich sonst die Thread-Instanzen die statische Variable gegenseitig überschreiben.

23.5.2 Beispielprogramm

Das folgende Beispielprogramm zeigt eine Funktion `threadfunction()`, die eine statische Variable `count` enthält. In einer Schleife wird `count` inkrementiert und zusammen mit der Threadnummer ausgegeben. Das Inkrementieren und Ausgeben ist durch einen Mutex geschützt. Diese Funktion wird in vier Threads aufgerufen.

staticthreadvar.c

```
#include <stdio.h>
#include <threads.h>
#include <stdbool.h>

int terminate = false;
mtx_t mutex;

int threadfunction(void* arg) {
    int threadid = *(int*)arg;

    static int count = 0;    // Variable ist static!

    while (!terminate) {
        mtx_lock(&mutex);
        ++count;
        printf("count = %03i, Thread %d\n", count, threadid);
        mtx_unlock(&mutex);
        thrd_yield();
    }

    return thrd_success;
}

int main(void) {
    int threadids[4] = {1, 2, 3, 4};
    thrd_t thr1, thr2, thr3, thr4;

    mtx_init(&mutex, mtx_plain);
    thrd_create(&thr1, threadfunction, &threadids[0]);
    thrd_create(&thr2, threadfunction, &threadids[1]);
    thrd_create(&thr3, threadfunction, &threadids[2]);
    thrd_create(&thr4, threadfunction, &threadids[3]);

    for (int i = 0; i < 10000000; ++i) {
        // eine Weile warten...
    }

    terminate = true;
    thrd_join(thr1, NULL);
    thrd_join(thr2, NULL);
    thrd_join(thr3, NULL);
    thrd_join(thr4, NULL);
    mtx_destroy(&mutex);
}
```

Bei der Programmausgabe zeigt sich, dass alle Threads an derselben static Variablen der Funktion weiterzählen. Deren Wert wird kontinuierlich inkrementiert, wobei sich die Threads abwechseln:

```
...  
count = 682, Thread 1  
count = 683, Thread 3  
count = 684, Thread 1  
count = 685, Thread 4  
count = 686, Thread 3  
count = 687, Thread 3  
...
```

Würde man die Funktion `threadfunction()` verändern und die static Variable `count` in eine lokale Variable umwandeln, sähe das Ergebnis beispielsweise folgendermaßen aus:

```
...  
count = 084, Thread 3  
count = 085, Thread 3  
count = 080, Thread 1  
count = 025, Thread 2  
count = 068, Thread 4  
count = 086, Thread 3  
count = 081, Thread 1  
count = 026, Thread 2  
count = 069, Thread 4  
...
```

In diesem Fall besitzt jede Thread-Instanz eine eigene Zählvariable `count`, die unabhängig von den anderen Instanzen inkrementiert wird. Bei lokalen Variablen ist der Mutex somit unnötig.

23.5.3 Thread-lokaler Speicher

Wenn eine Variable lokal innerhalb von Funktionen deklariert wird, besitzt jeder Thread seine eigene Variable, welche von anderen Threads nicht angesprochen werden kann. Dies bedeutet jedoch gleichzeitig, dass die entsprechende Variable nicht als externe Variable existiert und sie somit bei weiteren Funktionsaufrufen innerhalb des Threads stets als Parameter übergeben werden muss, wenn sie denn benötigt wird. Erfahrungsgemäß operieren jedoch gerade Threads oftmals auf mehreren externen Variablen. Dies zum einen, da Threads mit unterschiedlichen Funktionen gestartet werden, welche häufig auf denselben Objekten operieren. Zum anderen wird ein Programm normalerweise zuerst für Singlethreading geschrieben und erst nachträglich für Multithreading erweitert. Eine solche Erweiterung geht am einfachsten durch externe Variablen.

Um die Organisation von lokalen und externen Variablen innerhalb eines Threads einigermaßen in den Griff zu kriegen, muss die Programmierung oftmals rigoros umgestellt werden, weswegen das Thema Multithreading manchmal abschreckend wirkt.

Damit man sich nicht ständig um die Organisation seiner Daten und die Erweiterung seiner Funktionsaufrufe kümmern muss, gibt es die Möglichkeit, auch externe Variablen so zu deklarieren, dass sie für jeden Thread einzeln gespeichert werden.

Wird eine Variable benötigt, die in allen Thread-Instanzen einen eigenen Wert haben soll, so kann die Speicherklasse `_Thread_local` verwendet werden. Eine solche Variable kann innerhalb und außerhalb von Funktionen deklariert werden. Jeder Thread erhält bei seiner Erzeugung eine eigene Instanz dieser Variablen, deren Gültigkeit sich auf die Lebensdauer des Threads beschränkt.



Der Speicherklassen-Bezeichner `_Thread_local` definiert einen sogenannten **thread local storage** (zu Deutsch „**Thread-lokalen Speicher**“), einen lokalen Speicherbereich für Threads. Jeder Thread erhält damit eine eigene Instanz der Variablen, die als `_Thread_local` deklariert wurde. Die Initialisierung des Wertes erfolgt vor dem Starten der Threads, die **Lebensdauer** der Variablen endet gleichzeitig mit dem Ende des jeweiligen Threads. Eine solche Variable verhält sich wie eine globale Variable, deren konkreter Wert jedoch nur innerhalb eines Threads sichtbar ist.

Damit kann man auf globale Variablen zugreifen und muss sich nicht darum kümmern, dass andere Threads diese globalen Variablen möglicherweise ändern könnten. Die gesamte Organisation der Daten wird vom Compiler und vom Laufzeitsystem übernommen. Diese Erweiterung ist jedoch optional und existiert erst ab dem Standard C11.

Nebst dem Schlüsselwort `_Thread_local` wird in der Bibliothek `<threads.h>` auch noch das Makro `thread_local` definiert, welches zu diesem Schlüsselwort auswertet. Der Speicherklassen-Bezeichner `_Thread_local` darf nicht auf Funktionen angewendet werden. Der Standard empfiehlt, eine thread-lokale Variable stets entweder mit `static` oder mit `extern` zu deklarieren. Diese beiden Schlüsselwörter sind Speicherklassen für Variablen und definieren die Bindung der Variablen als intern beziehungsweise extern.

Die interne Bindung mit `static` bedeutet, dass der Speicherplatz der Variablen in der aktuellen Übersetzungseinheit reserviert wird. Durch eine externe Bindung mit `extern` kann der Speicherplatz auch irgendwo in einer anderen Übersetzungseinheit reserviert werden.



Siehe dazu auch Kapitel [→ 15.1.2](#).

Im überarbeiteten Beispiel aus Kapitel [→ 23.5.2](#) könnte somit anstatt der Definition einer lokalen Variablen `int count` innerhalb der Thread-Funktion eine globale Variable außerhalb der Funktion wie folgt definiert werden:

```
thread_local static int count = 0;
```

Damit könnte ebenfalls erreicht werden, dass sich mehrere Thread-Instanzen nicht mehr beim Zugriff auf die Variable `count` beeinflussen.

Mit der Speicherklasse `thread_local` ist es somit möglich, dass beliebige Variablen exklusiv für jeden Thread existieren. Häufig jedoch sind die Thread-Funktionen sehr umfangreich und benötigen sehr viele Variablen, welche allesamt exklusiv sein sollen. Um somit den Programmiercode nicht unnötig mit globalen Variablen aufzufüllen, wurde eine globale Methode zum Ansprechen von Thread-exklusiven Daten geschaffen: Der Thread-spezifische Speicher.

23.5.4 Funktionen für einen Thread-spezifischen Speicher

Eine weitere Möglichkeit zur Definition von Daten, die exklusiv einem Thread zugeordnet sind, bietet C in Form des sogenannten **thread specific storage** (kurz „tss“ und zu Deutsch „**Thread-spezifischer Speicher**“). Hierbei kann ein Thread individuelle Daten und den zugehörigen Schlüssel zum Zugriff auf die Daten hinterlegen. Funktionen, die aus dem Kontext dieses Threads aufgerufen werden, haben die Möglichkeit, über den global verfügbaren Schlüssel an die individuellen Daten des Threads zu gelangen, der die Funktion aufruft. Diese Daten sind für andere Threads nicht zugänglich.



Thread-spezifischer Speicher wird über die folgenden Funktionen bedient, die in der Bibliothek `<threads.h>` definiert sind:

```
int tss_create(tss_t* key, tss_dtor_t dtor);
```

Erzeugt einen Thread-lokalen Speicherschlüssel. Wenn der Schlüssel später gelöscht werden soll, wird die Funktion des übergebenen Funktionspointers `dtor` aufgerufen, um den gespeicherten Wert freizugeben.

```
void tss_delete(tss_t key);
```

Löscht alle Ressourcen, die dem Thread-lokalen Speicher `key` zugeordnet sind.

```
void* tss_get(tss_t key);
```

Liest den Wert des Thread-lokalen Speichers `key` für den aufrufenden Thread.

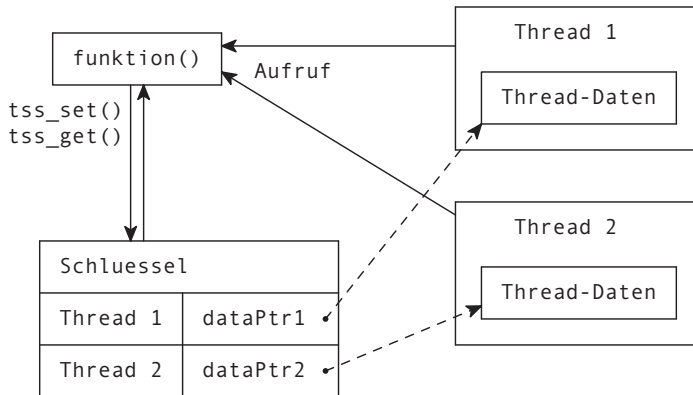
```
int tss_set(tss_t key, void* val);
```

Setzt den Wert des Thread-lokalen Speichers `key` für den aufrufenden Thread.

Ein Beispiel soll den Umgang mit Thread-spezifischem Speicher veranschaulichen. Es werden zwei Threads erzeugt, die dieselbe Funktion `threadFunction()` ausführen. Dieser Funktion wird als Argument jeweils ein Startwert für den jeweiligen Thread übergeben.

Beide Threads halten hierbei lokale Daten und machen diese über einen globalen Schlüssel zugänglich. Wird eine Funktion aus einem der Threads aufgerufen, so kann sie über den globalen Schlüssel an die Daten des aufrufenden Threads gelangen.

Die folgende Grafik veranschaulicht den Sachverhalt:



Jeder Thread muss den gewünschten Speicher lokal innerhalb seiner Funktion anlegen, wie im folgenden Beispiel in der Strukturvariablen `pLocal`. Da `pLocal` eine lokale Variable ist, sind die Daten vor wechselseitigem Zugriff aus anderen Threads geschützt. Durch die Funktion `tss_set()` wird die Adresse von `pLocal` mit einem globalen Schlüssel `tsskey` verknüpft.

In einer Endlosschleife wird der in `pLocal` gespeicherte Wert nun inkrementiert. Zur Ausgabe wird eine weitere Funktion `evaluate()` aufgerufen. Diese hat keinen direkten Zugriff auf die Thread-Daten, aber über den Thread-spezifischen Speicher gelangt sie an die Daten des aufrufenden Threads. Dazu verwendet sie die Funktion `tss_get(tsskey)` und holt sich damit einen Pointer auf den gewünschten Speicher. Das darin enthaltene Datum `data` wird per `printf()`-Befehl ausgegeben.

Hier der Quellcode des Beispielsprogramms:

tss.c

```
#include <stdio.h>
#include <stdlib.h>
#include <threads.h>
#include <stdbool.h>

int terminate = false;

tss_t tsskey;

typedef struct {
    unsigned char data;
} threadLocalData_t;
```



```
void evaluate(void) {
    threadLocalData_t* tld = (threadLocalData_t*)tss_get(tsskey);
    printf("data = %03i, Thread %p\n", tld->data, &(thrd_current()));
}

int threadfunction(void* arg) {
    threadLocalData_t pLocal;

    tss_set(tsskey, &pLocal);

    pLocal.data = *(unsigned char*)arg;
    while (!terminate) {
        ++pLocal.data;
        evaluate();

        if (terminate) {
            break;
        }
        thrd_yield();
    }
    return thrd_success;
}

int main(void) {
    thrd_t thr1;
    thrd_t thr2;

    unsigned char iniVal1 = 0;
    unsigned char iniVal2 = 128;

    tss_create(&tsskey, NULL);
    thrd_create(&thr1, threadfunction, (void*)&iniVal1);
    thrd_create(&thr2, threadfunction, (void*)&iniVal2);

    for (int i = 0; i < 100000000; ++i) {
        // eine Weile warten...
    }

    terminate = true;
    thrd_join(thr1, NULL);
    thrd_join(thr2, NULL);
    tss_delete(tsskey);
}
```

Programmausgabe (Ausschnitt):

```
...  
data = 071, Thread 0x008c0150  
data = 072, Thread 0x008c0150  
data = 129, Thread 0x008c0298  
data = 130, Thread 0x008c0298  
data = 131, Thread 0x008c0298  
data = 132, Thread 0x008c0298  
data = 073, Thread 0x008c0150  
...
```

Wie erwartet, zeigt sich bei der Ausgabe die Trennung der Daten der beiden Threads, obwohl jeweils dieselbe Funktion zur Ausgabe aufgerufen wurde. Der eine Thread beginnt bei 0 zu zählen, während der zweite Thread bei 128 beginnt.

23.5.5 Atomare Operationen

Die Bibliothek `<stdatomic.h>` stellt einen mit Multithreading verwandten Themenkomplex dar und wird daher in diesem Kapitel kurz erläutert.

Die Bibliothek `<stdatomic.h>` stellt Schnittstellen von Makros zur performanten und einfachen Verwendung von **atomaren Operationen** bereit. Aufwendige Synchronisationsmechanismen zum Schutz gegen wechselseitigen Zugriff für einfache Operationen können dadurch vermieden werden.



In den vorhergehenden Kapiteln wurde die Notwendigkeit von Synchronisationsmechanismen beim Arbeiten mit mehreren Threads auf gemeinsamen globalen Variablen gezeigt. Die Beispiele enthielten dabei meist globale Variablen vom Typ `int`. Dass selbst der Zugriff auf diese einfachen Variablen geschützt werden muss, wird beim Betrachten der Prozessorschritte für die Operation `++i` ersichtlich. Der Prozessor führt bei `++i` nicht nur eine, sondern gleich drei Operationen aus:

- Den Wert der Variablen `i` aus dem Speicher holen.
- `i` inkrementieren.
- Den neuen Wert von `i` wieder in den Speicher schreiben.

Führt nun ein Thread T1 die ersten beiden Schritte durch und wird von einem Thread T2 unterbrochen, bevor er den dritten Schritt ausführen konnte, dann liest T2 den alten Wert von `i` ein und erhöht diesen. Insgesamt wird `i` nur um eins erhöht, obwohl zweimal inkrementiert wurde. Um solche Inkonsistenzen zu vermeiden, dürfen diese drei Schritte nicht von einem anderen Thread unterbrochen werden. Mit einem Mutex könnte man diese Operation in einen geschützten Bereich packen und somit den gleichzeitigen Zugriff verhindern. Im Folgenden wird der Lock und der Unlock eines Mutex zum Schutz der Variablen `i` vor dem Zugriff anderer nebenläufiger Threads gezeigt:

```
mtx_lock(&mutex);  
++i;  
mtx_unlock(&mutex);
```

Diese Lösung ist allerdings nicht sehr effizient, da der Aufwand für die Synchronisation durch einen Mutex sehr viel höher ist als die zu schützende Operation selbst. Für einen solchen Fall bietet C den Einsatz von atomaren Operationen an. Sie werden immer am Stück ausgeführt und können nicht unterbrochen werden.

Die Bibliothek `<stdatomic.h>` beinhaltet Funktionen, Makros und Typen für den Umgang mit atomaren Operationen auf Daten, die von mehreren Threads verwendet werden. Da es sich auch hierbei um eine optionale Erweiterung des C-Standards handelt, existiert das Makro `__STDC_NO_ATOMICS__`, über das abgefragt werden kann, ob atomare Operationen implementiert sind.

Mit dem Schlüsselwort `_Atomic` vor einem Typ-Bezeichner lässt sich eine Variable atomaren Typs deklarieren, wie beispielsweise:

```
_Atomic int zahl;
```

Für nahezu alle Integer-Typen werden atomare Datentypen mit einem vorangesetzten `_Atomic` definiert. Wird beispielsweise nach der Initialisierung eines Integers `i` die Operation `++i` ausgeführt, dann ist diese automatisch atomar, kann also nicht unterbrochen werden. Führen mehrere Threads diese Operation aus, kann es also nicht mehr zu Inkonsistenzen kommen.

Elementare Operationen wie Addition und Subtraktion sowie logische Operationen auf Variablen eines atomaren Typs sind ebenfalls atomar. Neben diesen Operationen werden eine Menge weiterer Makros und Funktionen für den Einsatz von atomaren Operationen definiert. Diese aufzulisten würde jedoch den Rahmen dieses Buches sprengen.

A Standardbibliotheksfunktionen



C erlaubt es, durch Einbinden von standardisierten Header-Dateien zusätzliche Funktionalität zu nutzen.

Nachfolgend werden in den einzelnen Unterkapiteln die wichtigsten Funktionen und Makros der folgenden Header-Dateien mit ihrer Deklaration und einer kurzen Beschreibung aufgeführt:

ctype.h	Funktionen zur Klassifizierung und Konvertierung von Zeichen
math.h	Mathematische Funktionen
stdio.h	Funktionen für die Ein- und Ausgabe
stdlib.h	Funktionen zur Stringkonvertierung, Speicherverwaltung, Zufallszahlengenerierung und zum Beenden von Programmen
string.h	Funktionen zur String- und Speicherbearbeitung
time.h	Funktionen für Datum und Uhrzeit

Detaillierte Informationen können auf den üblichen Hilfeseiten nachgelesen werden. Die folgenden Header-Dateien gehören ebenfalls zum Standard, werden jedoch in diesem Buch nicht genauer behandelt:

assert.h	Funktionen zur Fehlersuche
complex.h	Funktionen und Typen für das Rechnen mit komplexen Zahlen
errno.h	Definitionen von Fehler-Codes
fenv.h	Definitionen zur Steuerung von Gleitpunkt-Berechnungen
float.h	Grenzwerte für Gleitpunkt-Typen
inttypes.h	Ein- und Ausgabe-Makros für Integer-Typen
iso646.h	Makros zur Unterstützung älterer Codes
limits.h	Grenzwerte für Integer-Typen
locale.h	Funktionen zur Einstellung von länderspezifischen Darstellungen
setjmp.h	Funktionen für globale Sprünge von einer Funktion in eine andere
signal.h	Funktionen zur Signalbehandlung
stdalign.h	Makros für das Schlüsselwort <code>_Alignof</code>
stdarg.h	Funktionen zur Behandlung einer variablen Parameterliste
stdatomic.h	Definitionen für <code>_Atomic</code> -Behandlung
stdbool.h	Definitionen für einen booleschen Standard-Typ

stddef.h	Allgemeine Definitionen
stdint.h	Definitionen von Integer-Typen mit präziser Bit-Angabe
stdnoreturn.h	Makros für das Schlüsselwort <code>_Noreturn</code>
tgmath.h	Generische mathematische Funktionen
threads.h	Funktionen für Multithreading
uchar.h	Funktionen für die Behandlung von Unicode
wchar.h	Funktionen für Multibyte und wide character Typen
wctype.h	Funktionen für wide character Umwandlungen

A.1 Klassifizierung und Konvertierung von Zeichen (ctype.h)

int **isalnum**(int c)
Testen, ob c alphanumerisch ist

int **isalpha**(int c)
Testen, ob c ein Buchstabe ist

int **iscntrl**(int c)
Testen, ob c ein Steuerzeichen ist

int **isdigit**(int c)
Testen, ob c eine Dezimalziffer ist

int **isgraph**(int c)
Testen, ob c ein druckbares Zeichen ist (ohne Leerzeichen (Blank))

int **islower**(int c)
Testen, ob c ein Kleinbuchstabe ist

int **isprint**(int c)
Testen, ob c ein druckbares Zeichen ist (einschließlich Leerzeichen (Blank))

int **ispunct**(int c)
Testen, ob c ein Sonderzeichen ist

int **isspace**(int c)
Testen, ob c ein Whitespace-Zeichen ist

int **isupper**(int c)
Testen, ob c ein Großbuchstabe ist (ohne Umlaute)

int **isxdigit**(int c)
Testen, ob c eine Hexadezimalziffer ist

int **tolower**(int c)

Umwandlung Groß- in Kleinbuchstaben

```
int toupper(int c)
```

Umwandlung Klein- in Großbuchstaben

A.2 Mathematische Funktionen (math.h)

```
double acos(double x)
```

Berechnet Arcuscosinus

```
double asin(double x)
```

Berechnet Arcussinus

```
double atan(double x)
```

Berechnet Arcustangens

```
double atan2(double y, double x)
```

Berechnet Arcustangens von y/x

```
double ceil(double x)
```

Aufrunden der Zahl x

```
double cos(double x)
```

Berechnet den Cosinus

```
double cosh(double x)
```

Berechnet den Cosinus hyperbolicus

```
double exp(double x)
```

Berechnet Exponentialfunktion e^x

```
double fabs(double x)
```

Berechnet Absolutwert einer Zahl vom Typ double

```
double floor(double x)
```

Abrunden der Zahl x

```
double fmod(double x, double y)
```

Berechnet den Rest von x geteilt durch y

```
double log(double x)
```

Berechnet den natürlichen Logarithmus

```
double log10(double x)
```

Berechnet den Logarithmus zur Basis 10

```
double pow(double x, double y)
```

Berechnet x^y

double **sin**(double x)

Berechnet den Sinus

double **sinh**(double x)

Berechnet den Sinus hyperbolicus

double **sqrt**(double x)

Berechnet die positive Quadratwurzel

double **tan**(double x)

Berechnet den Tangens

double **tanh**(double x)

Berechnet den Tangens hyperbolicus

A.3 Ein- und Ausgabe (stdio.h)

void **clearerr**(FILE* stream)

Fehler- und Dateiende-Flag löschen

int **fclose**(FILE* stream)

Schließen eines Streams

int **feof**(FILE* stream)

Prüfung, ob Streamende erreicht

int **ferror**(FILE* stream)

Prüfen eines Streams auf Fehler

int **fflush**(FILE* stream)

Schreiben des Streampuffers in die Datei

int **fgetc**(FILE* stream)

Lesen eines Zeichens aus einem Stream

int **fgetpos**(FILE* stream, fpos_t*pos)

Ermitteln der aktuellen Position in einer Datei

char* **fgets**(char* s, int n, FILE* stream)

String aus einem Stream auslesen

FILE* **fopen**(const char* path, const char* mode)

Öffnen eines Streams

int **fprintf**(FILE* stream, const char* format, ...)

Formatiertes Schreiben in einen Stream

int **fputc**(int c, FILE* stream)

Schreiben eines Zeichens in einen Stream

int **fputs**(const char* s, FILE* stream)

String in einen Stream schreiben

size_t **fread**(void* ptr, size_t size, size_t nmemb, FILE* stream)

Objekte aus einem Stream lesen

FILE* **freopen**(const char* path, const char* mode, FILE* stream)

Einem offenen Stream wird eine neue Datei zugeordnet

int **fscanf**(FILE* stream, const char* format, ...)

Einlesen formatierter Eingaben aus einem Stream

int **fseek**(FILE* stream, long offset, int whence)

Positionieren eines Dateipositions-Zeigers

int **fsetpos**(FILE* stream, const fpos_t* pos)

Positionieren eines Dateipositions-Zeigers mittels fpos_t

long **ftell**(FILE* stream)

Liefert Position des aktuellen Dateipositions-Zeigers

size_t **fwrite**(const void* ptr, size_t size, size_t nmemb, FILE* stream)

Objekte in einen Stream schreiben

int **getc**(FILE* stream)

Zeichen aus einem Stream lesen

int **getchar**(void)

Zeichen aus stdin lesen

void **perror**(const char* s)

Gibt den String s gefolgt von dem Fehlertext zur Fehlernummer gespeichert in der Systemvariablen errno aus

int **printf**(const char* format, ...)

Formatiertes Schreiben nach stdout

int **putc**(int c, FILE* stream)

Zeichen in einen Stream schreiben

int **putchar**(int c)

Zeichen nach stdout schreiben

int **puts**(const char* s)

String nach stdout schreiben

int **remove**(const char* path)

Löschen einer Datei

int **rename**(const char* old, const char* new)

Umbenennen einer Datei

void **rewind**(FILE* stream)

Dateipositions-Zeiger auf Stream-Anfang setzen

int **scanf**(const char* format, ...)

Formatiertes Lesen aus stdin

void **setbuf**(FILE* stream, char* buf)

Puffer einem Stream zuordnen

int **setvbuf**(FILE* stream, char* buf, int mode, size_t size)

Puffer einem Stream zuordnen

int **sprintf**(char* s, const char* format, ...)

Formatiertes Schreiben in einen String

int **sscanf**(const char* s, const char* format, ...)

Formatiertes Lesen aus einem String

FILE* **tmpfile**(void)

Öffnet eine temporäre Datei im Binärmodus

char* **tmpnam**(char* s)

Erzeugt einen eindeutigen Namen für eine temporäre Datei

int **ungetc**(int c, FILE* stream)

Zurückstellen eines gelesenen Zeichens in einen Stream

int **vfprintf**(FILE* stream, const char* format, va_list arg)

Formatiertes Schreiben in einen Stream mit den zusätzlichen Parametern gegeben als variable Parameterliste

int **vprintf**(const char* format, va_list arg)

Formatiertes Schreiben nach stdout mit den zusätzlichen Parametern gegeben als variable Parameterliste

int **vsprintf**(char* s, const char* format, va_list arg)

Formatiertes Schreiben in einen String mit den zusätzlichen Parametern gegeben als variable Parameterliste

A.4 Allgemeine Funktionen (stdlib.h)

int **abs**(int j)

Absoluter Betrag einer Integer-Zahl

void **abort**(void)

Beenden eines Programms

int **atexit**(void(*func)(void))

Registrierung von Funktionen, die vor dem Programmende aufgerufen werden sollen

double **atof**(const char* nptr)

String in eine Zahl vom Typ double konvertieren

int **atoi**(const char* nptr)

String in eine Integer-Zahl konvertieren

void* **bsearch**(const void* key, const void* base, size_t nmemb, size_t size, int(*compar)(const void*, const void*))

Binäres Absuchen eines Arrays

void* **calloc**(size_t nmemb, size_t size)

Speicher im Heap reservieren und mit '\0' initialisieren

div_t **div**(int x, int y)

Berechnet ganzzahligen Quotienten und Rest von x/y

void **exit**(int status)

Beenden eines Programms

void **free**(void* ptr)

Reservierten Speicher im Heap freigeben

char* **getenv**(const char* name)

Lesen einer System-Umgebungsvariablen

void* **malloc**(size_t size)

Speicher im Heap reservieren

void **qsort**(void* base, size_t nmemb, size_t size, int(*compar)(const void*, const void*))

Sortieren der Elemente eines Arrays nach dem Quicksort-Verfahren

int **rand**(void)

Zufallszahl erzeugen

void* **realloc**(void*ptr, size_t size)

Größe eines reservierten Speicherbereichs im Heap ändern

void **srand**(unsigned seed)
Zufallszahlengenerator initialisieren

int **system**(const char* s)
Aufruf des Kommandointerpreters des Betriebssystems

A.5 String- und Speicherbearbeitung (string.h)

void* **memchr**(const void* s, int c, size_t n)
Puffer nach Zeichen durchsuchen

int **memcmp**(const void* s1, const void* s2, size_t n)
Vergleich zweier Datenblöcke

void* **memcpy**(void* dest, const void* src, size_t n)
Kopieren eines Datenblocks

void* **memmove**(void* dest, const void* src, size_t n)
Verschieben eines Datenblocks

void* **memset**(void* s, int c, size_t n)
Kopiert n mal das Zeichen c in den Datenblock s

char* **strcat**(char* dest, const char* src)
Anhängen des Strings src an den String dest

char* **strchr**(const char* s, int c)
String nach dem Zeichen c durchsuchen

int **strcmp**(const char* s1, const char* s2)
Vergleich zweier Strings

char* **strcpy**(char* dest, const char* src)
Kopieren eines Strings in einen anderen

char* **strerror**(int errnum)
Pointer auf einen Fehlerstring der entsprechenden Nummer

size_t **strlen**(const char* s)
Berechnet die Länge eines Strings

char* **strncat**(char* dest, const char* src, size_t n)
Anhängen eines Teilstrings an einen anderen

int **strncmp**(const char* s1, const char* s2, size_t n)
Vergleich einer Anzahl von Zeichen zweier Strings

char* **strncpy**(char* dest, const char* src, size_t n)

Kopieren einer Anzahl von Zeichen in einen anderen String

char* **strrchr**(const char* s, int c)

Absuchen eines Strings nach dem letzten Vorkommen eines Zeichens

char* **strstr**(const char* s1, const char* s2)

Absuchen eines Strings nach einem Teilstring

char* **strtok**(char* s1, const char* s2)

Absuchen eines Strings nach Begrenzer-Zeichen

A.6 Datum und Uhrzeit (time.h)

char* **asctime**(const struct tm* timeptr)

Ortszeit in String konvertieren

clock_t **clock**(void)

Zeit seit dem Start des Programms

char* **ctime**(const time_t* timer)

Kalenderzeit in String (Ortszeit) konvertieren

double **difftime**(time_t time1, time_t time0)

Differenz zwischen 2 Zeiten in Sekunden berechnen

struct tm* **gmtime**(const time_t* timer)

Kalenderzeit in die Greenwich-Zeit umrechnen

struct tm* **localtime**(const time_t* timer)

Kalenderzeit in Ortszeit umrechnen

time_t **mktime**(struct tm* timeptr)

Ortszeit in die Kalenderzeit konvertieren

size_t **strftime**(char* s, size_t maxsize, const char* format, const struct tm* timeptr)

Ortszeit für Ausgabe formatieren

time_t **time**(time_t* timer)

Kalenderzeit liefern

time_t **timegm**(struct tm* tm)

Ab C23: Umkehrfunktion to gmtime(): Konvertiert Kalenderdaten in einen Zeitstempel um.

B Low-Level-Dateizugriffsfunktionen



In diesem Kapitel sollen die **Low-Level-Dateizugriffsfunktionen** im Gegensatz zu den High-Level-Funktionen (siehe Kapitel [→ 16.4](#)) behandelt werden.

Die Low-Level-Dateizugriffsfunktionen wurden ursprünglich geschrieben, um „System Calls“, also Aufrufe von Betriebssystemroutinen, die normalerweise in Assembler erfolgen, aus einem C-Programm heraus durchführen zu können. Sie haben auch heute noch eine große Bedeutung bei der Systemprogrammierung unter UNIX. Auch für andere Betriebssysteme wurden in der Folge Low-Level-Dateizugriffsfunktionen zur Verfügung gestellt. Diese Funktionen hängen jedoch stark vom jeweiligen Betriebssystem ab.

Die Low-Level-Dateizugriffsfunktionen werden auch als „ungepufferte Dateifunktionen“ bezeichnet, da im Gegensatz zu den High-Level-Dateizugriffsfunktionen keine Struktur FILE mit einem Datenpuffer für die Dateibearbeitung zur Verfügung steht, sondern direkt Betriebssystemroutinen des Kernels zum Dateizugriff aufgerufen werden.

Der Vorteil der Low-Level-Funktionen ist, dass sie auch Zugriffsmöglichkeiten bieten, die es bei den High-Level-Zugriffsfunktionen nicht gibt. Nachteil dieser Low-Level-Funktionen ist, dass diese im ISO-Standard nicht definiert sind und auch nicht werden. Dies kann zu erheblichen Portabilitätsproblemen eines Programms führen, das diese Funktionen nutzt. Wie bereits in Kapitel [→ 16.4.2](#) erwähnt, sind die Low-Level-Dateizugriffsfunktionen für UNIX-Betriebssysteme nach dem IEEE-POSIX-Standard, der die Hersteller-Unabhängigkeit von UNIX zum Ziele hat, standardisiert. Im Folgenden werden die Low-Level-Dateizugriffsfunktionen nach POSIX behandelt.

Unter Windows wird der Zugriff auf Dateien auf einen Windows-spezifischen Typ HANDLE umgeleitet. Unter POSIX erfolgt der Zugriff auf Dateien mit einem sogenannten **Dateideskriptor**, der durch das Dateisystem verwaltet wird. Ein Dateideskriptor ist nichts anderes als ein eindeutiger Integer. Auf den meisten Systemen werden zu Beginn eines Programmes automatisch drei Dateideskriptoren zur Verfügung gestellt:

0	Standard-Eingabe	stdin
1	Standard-Ausgabe	stdout
2	Standard-Fehler-Ausgabe	stderr

Diese Deskriptoren entsprechen den High-Level-Streams stdin, stdout und stderr.

Ergänzende Information Die elektronische Version dieses Kapitels enthält Zusatzmaterial, auf das über folgenden Link zugegriffen werden kann https://doi.org/10.1007/978-3-658-45209-4_25.

High- und Low-Level-Funktionen sollten in einem Programm nicht gemischt werden. Sollte ein solcher Mix erforderlich werden, so sind hierbei die Regeln für das Mischen von High- und Low-Level-Funktionen (siehe [Lew94]) zu beachten.

In den folgenden Unterkapiteln werden die diese Low-Level-Funktionen genauer behandelt. Es werden 4 Header-Dateien benötigt:

sys/types.h	Enthält die Definitionen von POSIX-Datentypen.
sys/stat.h	Enthält unter anderem Konstanten für die Vergabe von Zugriffsrechten auf eine Datei.
fcntl.h	Enthält unter anderem die Prototypen von <code>open()</code> und <code>creat()</code> und Konstanten für das Öffnen von Dateien.
unistd.h	Enthält zahlreiche Prototypen der Low-Level Dateizugriffs-funktionen.

B.1 Operationen auf dem Dateisystem

B.1.1 Die Funktion `open()`

```
int open(const char* path, int oflag, mode_t mode);
```

Zum Öffnen einer Datei, also zur Herstellung der Verbindung eines Handles zu einer Datei, wird die Funktion `open()` benutzt. Der Parameter `path` muss beim Aufruf den relativen oder absoluten Pfad inklusive Dateinamen beinhalten.

Mit dem Parameter `oflag` wird die Art des Zugriffs auf die Datei `path` beschrieben. Folgende Konstanten können angegeben werden:

<code>O_RDONLY</code>	Datei zum Lesen öffnen
<code>O_WRONLY</code>	Datei zum Schreiben öffnen
<code>O_RDWR</code>	Datei zum Lesen und Schreiben öffnen

Die gewünschte Zugriffsart kann unter anderem mit den folgenden Konstanten angegeben werden:

O_APPEND	Setzen des Dateipositions-Zeigers beim Öffnen auf das Ende der Datei.
O_CREAT	Erstellen der Datei, falls diese noch nicht existiert (wenn die Datei bereits existiert, wird dieser Wert ignoriert). Wird O_CREAT gesetzt, so muss auch der Parameter mode, der die Zugriffsrechte regelt, gesetzt werden. Dieser ist implementationsspezifisch und wird hier nicht weiter behandelt.
O_TRUNC	Löschen des Dateiinhaltes, falls die Datei vor dem Öffnen bereits existiert.

Die Konstanten können dabei mit dem bitweisen-ODER-Operator kombiniert werden. Wird die Datei geöffnet, so gibt die Funktion `open()` den Dateideskriptor der gewünschten Datei als einen `int`-Wert größer gleich 0 zurück. Im Fehlerfall wird -1 zurückgegeben.

B.1.2 Die Funktion `creat()`

```
int creat(const char* path, mode_t mode);
```

Die Funktion `creat()` erzeugt eine neue Datei mit dem Dateinamen `path` und den Zugriffsrechten `mode` und gibt einen Filedeskriptor zurück, welcher der dadurch geöffneten Datei entspricht. Entsprechend der Funktion `open()` muss der Parameter `name` den relative oder absoluten Pfad inklusive Dateinamen beinhalten. Existiert eine Datei mit demselben Namen bereits, wird diese überschrieben.

Der Parameter `mode` enthält Zugriffsrechte, die für die Datei vergeben werden sollen. Sie implementationsspezifisch und werden hier nicht weiter behandelt.

Die Funktionen `creat()` kann auch mittels der `open()`-Funktion abgebildet werden. Folgende beiden Codezeilen sind äquivalent:

```
creat(path, mode);  
open(path, O_WRONLY | O_CREAT | O_TRUNC, mode);
```

Wird die Datei korrekt erstellt, so gibt die Funktion `creat()` einen Wert größer gleich 0 zurück, den Dateideskriptor auf die neue Datei. Im Fehlerfall ist der Rückgabewert -1.

B.1.3 Die Funktion `close()`

```
int close(int fildes);
```

Die Funktion `close()` dient zum Schließen der durch `fildes` angegebenen Datei.

Bei erfolgreicher Ausführung gibt die Funktion `0` zurück, im Fehlerfall `-1`.

B.1.4 Die Funktion `unlink()`

```
int unlink(const char* path);
```

Die Funktion `unlink()` bewirkt, dass eine Datei nicht mehr unter dem Namen `path` angesprochen werden kann. Grundsätzlich wird die Datei damit gelöscht. Der Parameter `path` muss den relativen oder absoluten Pfad inklusive Dateiname enthalten.

Die bei den High-Level-Dateizugriffsfunktionen beschriebene Funktion `remove()` (siehe Kapitel [→ 16.5.7](#)) ist äquivalent zu der Funktion `unlink()` und kann für die Low-Level-Dateibehandlung ebenfalls verwendet werden.

Nach korrektem Löschen der Datei liefert die Funktion den Rückgabewert `0`, im Fehlerfall den Wert `-1`.

B.2 Ein- und Ausgabe, Positionieren innerhalb einer Datei

B.2.1 Die Funktion `write()`

```
int write(int fildes, const void* buf, unsigned int nbyte);
```

Die Funktion `write()` schreibt `nbyte` Bytes aus dem Puffer, auf den der Pointer `buf` zeigt, in die mit `fildes` angegebene Datei. Der intern verwaltete Zeiger auf die aktuelle Dateiposition wird nach dem Schreiben um die Anzahl an geschriebenen Bytes weiter bewegt.

Der Rückgabewert der Funktion ist bei korrektem Schreiben gleich der Anzahl an geschriebenen Bytes. Ist der Rückgabewert `-1`, so ist ein Fehler aufgetreten.

B.2.2 Die Funktion `read()`

```
int read(int fildes, void* buf, unsigned int nbyte);
```

Mit der Funktion `read()` werden `nbyte` Bytes aus der durch `fildes` angegebenen Datei gelesen und in den Puffer, auf den `buf` zeigt, geschrieben. Nach dem Lesen wird der Dateipositions-Zeiger um die Anzahl der gelesenen Bytes erhöht.

Der Rückgabewert entspricht der Anzahl der tatsächlich gelesenen Bytes. Diese Anzahl kann kleiner sein, wenn das Ende der Datei erreicht wurde. Tritt ein Fehler auf, so wird `-1` zurückgegeben.

B.2.3 Positionieren in Dateien mit `lseek()`

```
off_t lseek(int fildes, off_t offset, int whence);
```

Die Funktion `lseek()` positioniert den Dateipositions-Zeiger der durch `fildes` angegebenen Datei um `offset` Bytes von der durch `whence` definierten Ausgangsposition. Der Offset kann dabei positiv oder negativ sein. Der Integer-Typ `off_t` ist in `<sys/types.h>` definiert. `whence` kann folgende Werte annehmen:

<code>SEEK_SET</code>	<code>offset</code> ist relativ zum Dateianfang
<code>SEEK_CUR</code>	<code>offset</code> ist relativ zur aktuellen Position
<code>SEEK_END</code>	<code>offset</code> ist relativ zum Dateende

Bei fehlerfreier Ausführung liefert `lseek()` die neue Position des Dateipositions-Zeigers zurück, im Fehlerfall den Wert `-1`.

B.3 Beispiel zur Dateibearbeitung mit Low-Level-Funktionen

In folgendem Beispiel sei eine Datei `source.dat` gegeben, in welcher eine Anzahl Daten vom Typ `struct adresse` gespeichert sind. Diese Daten sollen in umgekehrter Reihenfolge in die Datei `dest.dat` kopiert werden.

lowLevel.c

```
// MSVC meldet fuer veraltete, unsichere C-Funktionen wie
// die low-level-Dateizugriffsfunktionen einen Fehler.
// Mittels folgender Zeile koennen diese Fehler ausgeschaltet werden:
#define _CRT_SECURE_NO_WARNINGS

#include <sys/stat.h>
#include <sys/types.h>
#include <fcntl.h>
#include <stdio.h>

#ifdef _WIN32
    #include <io.h>
    #include <windows.h>
    #define CREATE_MODE _S_IWRITE
#else
    #include <unistd.h>
    #define CREATE_MODE S_IWUSR | S_IRUSR
#endif

#define SRC_FILE "source.dat"
#define DST_FILE "dest.dat"

int main(void) {
    int source_fd;
    int dest_fd;

    struct adresse {
        char name[30];
        char ort[30];
        char strasse[30];
        long plz;
    } adr;

    const int dataSize = sizeof(struct adresse);

    // Quelldatei oeffnen.
    if ((source_fd = open(SRC_FILE, O_RDONLY)) < 0) {
        printf("Fehler beim Oeffnen der source-Datei\n");
    }
```

```
    return 1;
}

// Zielfeile erstellen.
if ((dest_fd = creat(DST_FILE, CREATE_MODE)) < 0) {
    printf("Fehler beim Erstellen der dest-Datei\n");
    return 1;
}

// Ausrechnen, wieviele Adressen sich in der Datei befinden.
off_t size = lseek(source_fd, 0, SEEK_END);
int zaehler = (int)size / dataSize;

// Rueckwaerts durch die Datei gehen und die Adressen auslesen.
for (int i = zaehler - 1; i >= 0; --i) {
    if (lseek(source_fd, i * dataSize, SEEK_SET) < 0) {
        printf("Fehler beim Positionieren in der source-Datei\n");
        return 1;
    }

    // Daten aus Quelldatei auslesen
    if (read(source_fd, &adr, dataSize) < 0) {
        printf("Fehler beim Lesen aus der source-Datei\n");
        return 1;
    }

    // Daten in Zielfeile eintragen
    if (write(dest_fd, &adr, dataSize) < dataSize) {
        printf("Fehler beim Schreiben in die dest-Datei\n");
        return 1;
    }
}

// Schliessen der geoeffneten Dateien
if (close(source_fd) < 0) {
    printf("Fehler beim Schliessen der source-Datei\n");
    return 1;
}

if (close(dest_fd) < 0) {
    printf("Fehler beim Schliessen der dest-Datei\n");
    return 1;
}

return 0;
}
```

C Wandlungen zwischen Zahlensystemen



Im Folgenden wird gezeigt, wie natürliche Zahlen zwischen den Zahlensystemen Dezimalsystem, Binärsystem und Hexadezimalsystem gewandelt werden können.

C.1 Vorstellung der Zahlensysteme

C.1.1 Dezimalsystem

Sei B die Basis eines Zahlensystems, dann sind die möglichen Ziffern 0 bis $B-1$. Das alltägliche Zahlensystem ist das **Dezimalsystem** mit $B = 10$ und den Ziffern $0, 1, 2, 3, 4, 5, 6, 7, 8$ und 9 . Ein Beispiel für eine Zahl im Dezimalsystem ist 3241 . Zur eindeutigen Kennzeichnung wird in der Mathematik eine tiefer gestellte 10 angehängt (3241_{10}), hier in diesem Buch wird darauf verzichtet. Im Folgenden wird diese Zahl in eine Stellenwerttabelle eingetragen:

Stelle	Tausender	Hunderter	Zehner	Einer
Dezimalwert	1000	100	10	1
Zehnerpotenz	10^3	10^2	10^1	10^0
Dezimalsystem	3	2	4	1

Die Zahl lautet dementsprechend:

$$3 \cdot 1000 + 2 \cdot 100 + 4 \cdot 10 + 1 \cdot 1 = 3241$$

C.1.2 Binärsystem

Die Basis B des **Binärsystems** ist 2. Die möglichen Ziffern sind 0 und 1. Ein Beispiel für eine Zahl im Binärsystem ist 10111. Zur eindeutigen Kennzeichnung einer binären Zahl wird in der Mathematik eine tiefer gestellte 2 angehängt: 10111_2 . Im Code kann ab dem Standard C23 eine Binärzahl geschrieben werden, indem ihr ein 0b vor die Zahl angefügt wird, also 0b10111. Im Folgenden wird diese Zahl in eine Stellenwerttabelle eingetragen:

Dezimalwert	16	8	4	2	1
Zweierpotenz	2^4	2^3	2^2	2^1	2^0
Binärsystem	1	0	1	1	1

Die Zahl, umgerechnet ins Dezimalsystem, lautet:

$$1 \cdot 16 + 0 \cdot 8 + 1 \cdot 4 + 1 \cdot 2 + 1 \cdot 1 = 23$$

Im Deutschen wird das Binärsystem auch „Dualsystem“ genannt.

C.1.3 Hexadezimalsystem

Die Basis B im **Hexadezimalsystem** ist 16. Die möglichen Ziffern sind 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, a, b, c, d, e, f. Manchmal werden auch die Großbuchstaben A, B, C, D, E, F anstelle der Kleinbuchstaben verwendet. Dabei haben die Ziffern a bis f folgende Werte:

a	b	c	d	e	f
10	11	12	13	14	15

Ein Beispiel für eine Zahl im Hexadezimalsystem ist: fa6b. Zur eindeutigen Identifikation einer Hexadezimalzahl wird in der Mathematik die Basis 16 tiefer gestellt an die Hexadezimalzahl angehängt: $fa6b_{16}$. Im Code schreibt man für eine Hexadezimalzahl ein 0x vor die Zahl, also 0xfa6b. Im Folgenden wird diese Zahl in eine Stellenwerttabelle eingetragen:

Dezimalwert	4096	256	16	1
Sechzehnerpotenz	16^3	16^2	16^1	16^0
Hexadezimalsystem	f	a	6	b

Die Zahl, umgerechnet ins Dezimalsystem, lautet:

$$15 \cdot 4096 + 10 \cdot 256 + 6 \cdot 16 + 11 \cdot 1 = 64107$$

C.2 Umwandlung von Dezimal in Binär/Hexadezimal

Im Folgenden wird gezeigt, wie eine Dezimalzahl in eine Zahl der Basis B umgerechnet werden kann. Anschließend wird dann an einem Beispiel für die Basis $B = 2$ und einem Beispiel für die Basis $B = 16$ die Umrechnung in das Binär- und das Hexadezimalsystem demonstriert.

C.2.1 Umwandlung einer Dezimalzahl in eine Zahl der Basis B

Gegeben sei eine Dezimalzahl z mit 5 Stellen:

$$z = c_4 \cdot B^4 + c_3 \cdot B^3 + c_2 \cdot B^2 + c_1 \cdot B^1 + c_0 \cdot B^0$$

Die Koeffizienten c_4, c_3, c_2, c_1 und c_0 stellen die Ziffern der fünfstelligen Dezimalzahl dar. Jeder dieser Koeffizienten ist kleiner gleich $B-1$.

Der erste Schritt ist eine Umformung in die folgende Form:

$$z = B (B (B (B (c_4) + c_3) + c_2) + c_1) + c_0$$

Dabei wurden der Übersichtlichkeit wegen die Multiplikationszeichen weggelassen. Durch Ausmultiplizieren kann man sich überzeugen, dass diese Darstellung äquivalent zu der oben gezeigten ist.

Vereinfacht kann man schreiben:

$$z = B \cdot z_1 + c_0$$

Der Ausdruck z_1 entspricht dem geklammerten Ausdruck oben. Damit wird klar: Die Zahl z ist teilbar durch die Basis B bis auf einen Rest c_0 . c_0 ist kleiner als B , da es der Koeffizient von B^0 ist. Also wird geteilt:

$$z / B = z_1 \text{ Rest } c_0$$

Durch einmaliges Teilen der Zahl z durch die Basis B erhält man somit den Wert des Ausdrucks z_1 sowie die Ziffer c_0 .

Nun führt man dasselbe Verfahren mit dem restlichen Ausdruck z_1 aus:

$$z_1 = B \cdot z_2 + c_1$$

Dabei entspricht z_2 der zweiten inneren Klammerung des obigen Ausdrucks.

Also ergibt sich:

$$z_1 / B = z_2 \text{ Rest } c_1$$

Durch weitere Wiederholung des Verfahrens findet man schlussendlich:

$$z_2 / B = z_3 \text{ Rest } c_2$$

$$z_3 / B = z_4 \text{ Rest } c_3$$

$$z_4 / B = z_5 \text{ Rest } c_5$$

Damit sind alle Koeffizienten bestimmt! Durch wiederholte Division mit Bestimmung des Restwertes kann jede Dezimalzahl in ein beliebiges Basissystem umgewandelt werden. Beispielsweise soll die Zahl 53 in das Binärsystem gewandelt werden. Die Berechnung ergibt:

$$53 / 2 = 26 \text{ Rest } 1$$

$$26 / 2 = 13 \text{ Rest } 0$$

$$13 / 2 = 6 \text{ Rest } 1$$

$$6 / 2 = 3 \text{ Rest } 0$$

$$3 / 2 = 1 \text{ Rest } 1$$

$$1 / 2 = 0 \text{ Rest } 1$$

Die Binärzahl lautet somit (von unten nach oben gelesen): 110101

Rechnet man diese Zahl wieder zurück ins Dezimalsystem, so ergibt sich:

$$1 \cdot 2^5 + 1 \cdot 2^4 + 0 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = 32 + 16 + 0 + 4 + 0 + 1 = 53$$

Als weiteres Beispiel soll die Zahl 493 in das Hexadezimalsystem umgerechnet werden. Die Berechnung ergibt:

$$493 / 16 = 30 \text{ Rest } 13 \text{ (d)}$$

$$30 / 16 = 1 \text{ Rest } 14 \text{ (e)}$$

$$1 / 16 = 0 \text{ Rest } 1$$

Die Hexadezimalzahl lautet somit (von unten nach oben gelesen) 0x1ed.

Zurückgerechnet ins Dezimalsystem ergibt sich:

$$1 \cdot 16^2 + 14 \cdot 16^1 + 13 \cdot 16^0 = 493$$

C.3 Umwandlung von Binär in Hexadezimal und umgekehrt

Da 16 eine Zweierpotenz ist, ist die Umwandlung zwischen dem Binärsystem und dem Hexadezimalsystem sehr einfach.

Es werden stets 4 Ziffern des Binärsystems zu einer Ziffer des Hexadezimalsystems umgewandelt und vice versa.

0 0 0 0	=	0
0 0 0 1	=	1
0 0 1 0	=	2
0 0 1 1	=	3
0 1 0 0	=	4
0 1 0 1	=	5
0 1 1 0	=	6
0 1 1 1	=	7
1 0 0 0	=	8
1 0 0 1	=	9
1 0 1 0	=	10 (a)
1 0 1 1	=	11 (b)
1 1 0 0	=	12 (c)
1 1 0 1	=	13 (d)
1 1 1 0	=	14 (e)
1 1 1 1	=	15 (f)

Das Bitmuster 11011011 im Binärsystem ist also beispielsweise äquivalent zu 0xdb im Hexadezimalsystem.

Zu früheren Zeiten wurden vier Bits übrigens auch als eine „Tetrade“, beziehungsweise als ein „Halbbyte“, beziehungsweise als ein „Nibble“ bezeichnet.

D ASCII und Unicode



Eines der fundamentalen Probleme der Informatik ist die Representation von Text. Um als Mensch überhaupt mit dem Computer kommunizieren zu können, mussten schon sehr früh Methoden gefunden werden, um berechnete Inhalte in für den Menschen verständliche Form umzuwandeln, sei dies mittels Zahlen, Buchstaben oder (viel später dann) Bild und Ton.

Um Texte in Computern zu speichern wurden viele **Codierungen** vorgeschlagen (siehe auch Kapitel [→ 6.1](#)). Durchgesetzt haben sich schlussendlich nur wenige, wovon heutzutage insbesondere nur noch zwei wirklich wichtig sind: ASCII und Unicode.

D.1 ASCII

ASCII ist eine 7-Bit-Codierung von den wichtigsten (westlichen) Schriftzeichen und wird heutzutage üblicherweise mit 8 Bit gespeichert, wobei das höchstwertige Bit auf null gesetzt ist. Der ASCII-Zeichensatz entspricht der US-nationalen Ausprägung des ISO-7-Bit-Codes (**ISO 646**). In folgender Tabelle sind die sogenannten „druckbaren Zeichen“ aufgelistet:

Dec	Hex		Dec	Hex		Dec	Hex	
32	20		64	40	@	96	60	'
33	21	!	65	41	A	97	61	a
34	22	"	66	42	B	98	62	b
35	23	#	67	43	C	99	63	c
36	24	\$	68	44	D	100	64	d
37	25	%	69	45	E	101	65	e
38	26	&	70	46	F	102	66	f
39	27	'	71	47	G	103	67	g
40	28	(	72	48	H	104	68	h
41	29	)	73	49	I	105	69	i
42	2a	*	74	4a	J	106	6a	j
43	2b	+	75	4b	K	107	6b	k
44	2c	,	76	4c	L	108	6c	l
45	2d	-	77	4d	M	109	6d	m
46	2e	.	78	4e	N	110	6e	n
47	2f	/	79	4f	O	111	6f	o
48	30	0	80	50	P	112	70	p
49	31	1	81	51	Q	113	71	q
50	32	2	82	52	R	114	72	r
51	33	3	83	53	S	115	73	s
52	34	4	84	54	T	116	74	t
53	35	5	85	55	U	117	75	u
54	36	6	86	56	V	118	76	v
55	37	7	87	57	W	119	77	w
56	38	8	88	58	X	120	78	x
57	39	9	89	59	Y	121	79	y
58	3a	:	90	5a	Z	122	7a	z
59	3b	;	91	5b	[	123	7b	{
60	3c	<	92	5c	\	124	7c	
61	3d	=	93	5d	]	125	7d	}
62	3e	>	94	5e	^	126	7e	~
63	3f	?	95	5f	_			

Die ersten 32 ASCII-Zeichen sind in folgender Tabelle zusammengefasst. Sie stellen **Steuerzeichen** für die Ansteuerung von Peripheriegeräten und die Steuerung einer rechnergestützten Datenübertragung dar. Diese Zeichen haben festgelegte Namen (rechte Spalte) wie beispielsweise FF als Abkürzung für „Form Feed“, also „Seitenvorschub“, oder CR als Abkürzung für „Carriage Return“, dem sogenannten „Wagenrücklauf“, der von der Schreibmaschine her bekannt ist.

0	'\0'	NUL	8	'\b'	BS	16	DLE	24	CAN
1		SOH	9	'\t'	HAT	17	DC1	25	EM
2		STX	10	'\n'	LF	18	DC2	26	SUB
3		ETX	11	'\v'	VT	19	DC3	27	'\e' ESC
4		EOT	12	'\f'	FF	20	DC4	28	FS
5		ENQ	13	'\r'	CR	21	NAK	29	GS
6		ACK	14		SO	22	SYN	30	RS
7		BEL	15		SI	23	ETB	31	US

Einige dieser Steuerzeichen sind auch heute noch in der Programmierung wichtig. Sie sind insbesondere durch spezielle Escape-Zeichen (mittlere Spalte) festgelegt. Siehe dazu Kapitel [→ 6.5.5](#).

D.1.1 Erweiterte ASCII-Codierungen

Insgesamt stehen im ASCII-Zeichensatz nur wenige Zeichen zur Verfügung. Im Zuge der zunehmenden Internationalisierung wollte man in der Informatik jedoch die Möglichkeit schaffen, Zeichen von Landessprachen, beispielsweise aus dem asiatischen Raum, abzubilden. Hierfür werden jedoch weitaus mehr als nur 7 Bits benötigt.

Somit wurden neue Zeichencodierungen geschaffen, welche allesamt dem ASCII-Standard angelehnt sind, ihn jedoch erweitern. Bei diesen erweiterten Zeichensätzen wird zwischen Singlebyte- und Multibyte-Codierungen unterschieden. Eine Singlebyte-Codierung definiert, dass jedes Zeichen genau 1 Byte belegt, wodurch zusätzlich zu den gegebenen 7-Bit-ASCII-Zeichen nochmals 128 weitere Zeichen definiert werden können, indem das letzte verbleibende Bit genutzt wird.

Die heutzutage am häufigsten gesehene Singlebyte-Codierung ist Windows-1252: Der Standard-Zeichensatz des Windows-Systems, welcher oftmals auch fälschlicherweise als ISO-8859-1 angegeben wird. Auf die genauen Unterschiede soll hier nicht weiter eingegangen werden.

Für solche Singlebyte-Codierungen wurden zu früheren Zeiten mehrere sogenannte „code pages“ definiert, beispielsweise für Französisch oder Russisch, welche die zusätzlichen 128 Zeichen für sprachspezifische Zeichen nutzten. Solch erweiterte ASCII-Zeichensätze sind jedoch implementationsspezifisch, gelten heutzutage als veraltet und werden hier nicht weiter behandelt.

D.2 Unicode

Um das Problem der vielen verschiedenen sprachspezifischen Zeichen der Welt zu lösen, wurde gegen Ende des zwanzigsten Jahrhunderts die Entwicklung eines ASCII-Nachfolge-Standards angestrebt, der weit über eine Million verschiedener Zeichen abbilden konnte: Unicode.

Unicode ist ein weit verbreiteter Name für UCS: Universal Character Set. UCS definiert 21 Bits pro Zeichen.



Es werden nicht 32 Bits verwendet, sondern nur deren 21, um verschiedenen Aspekten der Stringverarbeitung auf Computern gerecht zu werden. Es geht dabei um Kompatibilität mit dem bisherigen ASCII-Standard, Erweiterbarkeit für die Zukunft sowie der Möglichkeit, Strings so kompakt wie möglich zu speichern und dennoch eine schnelle Verarbeitung derselben gewährleisten zu können. All diese Eigenschaften zeigen sich bei der sogenannten „Multibyte-Codierung“.

D.2.1 Multibyte-Codierung

Eine Multibyte-Codierung bezeichnet eine spezielle Codierung von Zeichen, welche insbesondere für Unicode häufig angewendet wird.



Traditionell werden Textdateien (Quelldateien, Emails, Websites, Logdateien, Messresultate, ...) in ASCII-Dateien geschrieben, was sehr platzsparend ist. Würde man eine solche Datei in Unicode speichern, so müsste man für jedes Zeichen 21 Bits reservieren, was normalerweise in 32 Bits verpackt wird. Jede Quelldatei würde somit auf ihre vierfache Größe anwachsen. Das ist nicht erwünscht.

Verpackt man jedes Unicode-Zeichen in 32 Bits, so wird dies auch als die sogenannte **UTF-32** Codierung bezeichnet. UTF steht für „Unicode Transformation Format“ und die 32 steht für die Anzahl Bits pro Zeichen. Aufgrund des massiven Speicherbedarfs ist UTF-32 nur von theoretischer Bedeutung.

Weitaus bessere Codierungen werden mit anderen Multibyte-Codierungen erreicht:

Unicode wurde so aufgebaut, dass es möglich ist, Dateien in der existierenden ASCII-Codierung ohne weitere Konvertierung direkt weiterzuverwenden und dennoch auf sämtliche 21 Bits des Unicodes zugreifen zu können. Hierbei fängt ein Zeichen stets mit einem Start-Byte an, woraufhin mehr oder weniger zusätzliche Bytes angefügt werden, was dann aufgrund deren Anordnung und Codierung insgesamt als ein einziges Unicode-Zeichen aufgefasst wird.

Um die vollen 21 Bits des Unicodes nutzen zu können, werden bei einer Multibyte-Codierung je nach Zeichen mehrere Bytes hintereinandergereiht, welche dann zusammen ein vollständiges Zeichen ausmachen.



Multibyte-Zeichen können somit aus einem oder mehreren Bytes bestehen. Dadurch wird es möglich, häufig verwendete Zeichen mit wenigen Bytes und selten verwendete Zeichen mit mehr Bytes zu codieren.

Heutzutage sind insbesondere die Multibyte-Codierung UTF-8 mit 8 Bits pro Zeichen und UTF-16 mit 16 Bits pro Zeichen im Umlauf.

D.2.2 UTF-8, UTF-16 und wchar_t

UTF-8 steht für „Unicode Transformation Format – 8 Bit“ und ist eine Multibyte-Codierung für Unicode. UTF-16 benötigt 16 Bits.



UTF-8 wird hauptsächlich zum Speichern und Übertragen von Text-Dateien verwendet. Insbesondere in der Programmierung haben UTF-8-Dateien einen hohen Stellenwert, da sie kompatibel zu den älteren ASCII-Dateien sind. ASCII-Dateien können ohne weitere Umkonvertierung automatisch als UTF-8-Dateien verarbeitet werden.

UTF-16 wird insbesondere bei der Verwendung von internationalen Strings im Windows- wie auch im MacOS-System verwendet, beispielsweise bei der Steuerung des GUIs. Der Typ `wchar_t` ist auf entsprechenden Systemen oftmals entsprechend dieser Zeichenbreite definiert.

Der Datentyp `wchar_t` ist eine Abkürzung für „wide character type“ und speichert Zeichen einer Codierung, deren Zeichen in einer Variablen vom Typ `char` keinen Platz haben. Die Zeichen einer solchen Codierung werden auch „Breitzeichen“ genannt.

Um im Code einen String zu codieren, der mit dem Typ `wchar_t` gespeichert werden soll, schreibt man ein `L` vor den Anführungszeichen: `L"Hällö Welt"`.

Wie gesagt entspricht die Nutzung von `wchar_t` heutzutage oftmals der UTF-16 Codierung. Die Breite des Datentyps `wchar_t` ist jedoch plattformspezifisch und kann je nach Compilereinstellung variieren.

Es wird davor gewarnt, den Datentyp `wchar_t` für die Speicherung von mit Unicode codiertem Text zu verwenden, da dieser Datentyp plattformspezifisch ist.



Moderne Systeme und Bibliotheken verwenden heutzutage normalerweise UTF-8 als Standard-Codierung.

D.2.3 Beispiel: Abbildung eines Multibyte-Zeichens auf Unicode

Der von den Musiknoten her bekannte Violinschlüssel hat den Unicode U+1d11e. Unicode-Zeichen werden normalerweise als Hexadezimalzahl geschrieben. Zur Kennzeichnung von Unicode wird zudem die Zeichenfolge U+ vorangestellt.

In UTF-32 entspricht das einer einzigen Hex-Zahl 0x0001d11e,
in UTF-16 entspricht das den zwei Hex-Zahlen 0xd834 und 0xdd1e,
in UTF-8 entspricht das den vier Hex-Zahlen 0xf0, 0x9d, 0x84 und 0x9e.

Binär ausgedrückt, bedeutet das:

UTF-32:	00000000	000 00001	11010001	00011110,
UTF-16:	11011000	00110100	11011101	00011110,
UTF-8:	11110000	10011101	10000100	10011110.

Da Unicode insgesamt 21 Bits benötigt, sind bei den hier gezeigten 32 Bits jeweils 11 Bits überschüssig. Diese überschüssigen Bits werden von den Codierungen verwendet, um Start- und Folge-Zeichen zu markieren. Alle drei Codierungen definieren somit, welche Bits als Inhaltsbits und welche als Markierungsbits betrachtet werden sollen. Hier wurden die Inhaltsbits fett hervorgehoben. Setzt man die fettgedruckten Zeichen abstandslos hintereinander, so entstehen folgende 21-Bit-Ketten:

UTF-32:	000011101000100011110,
UTF-16:	000011101000100011110,
UTF-8:	000011101000100011110,

Für alle drei Codierungen entspricht dies genau dem Unicode U+1d11e.

Der Vollständigkeit halber: Bei der UTF-16 Codierung wird die Hex-Zahl 0x10000 vom Unicode-Wert abgezogen, damit nur 20 Bits für die Speicherung benötigt werden. Beim Zurückwandeln muss diese Hex-Zahl 0x10000 wieder hinzuaddiert werden.

E Arithmetische Typen, Wertebereiche und Codierungen



Die im Kapitel → 7 vorgestellten arithmetischen Typen waren nur die wichtigsten, welche in der modernen Programmierung noch verwendet werden. Die Sprache C definiert jedoch in ihrem Kern noch einige zusätzliche Modifikatoren und damit Besonderheiten im Wertebereich und der Codierung von Typen.

Aufgrund der fortlaufenden Standardisierung von C sowie moderner Compiler sind mittlerweile diese „veralteten“ Modifikatoren in Vergessenheit und gar in Verruf geraten. Dennoch finden sie sich ab und zu noch in älterem Code.

E.1 Modifikatoren short und long

Seit dem C99-Standard existiert die Bibliothek `<stdint.h>` mit einer Sammlung an Integer-Typen, wie beispielsweise `int32_t`, für welche klar definiert ist, wie viele Bits ein Integer tatsächlich benötigt. Zu früheren Zeiten musste man, um unterschiedliche Bitgrößen festzulegen, auf den `int`-Typ und seine Modifikatoren zurückgreifen:

- `short int`
- `int`
- `long int`
- `long long int`

Die Schlüsselwörter **short** und **long** gelten dabei als sogenannte Modifikatoren. Sie können jedoch auch für sich alleine stehen, sprich, anstelle `short int` kann man auch einfach schreiben: `short`. Beide Schreibweisen sind äquivalent. Dasselbe gilt für `long int` und `long`.

Auch `long long` gilt als eigenständiger Typ. Das Schlüsselwort `long` darf jedoch nicht mehr als zwei Mal vorkommen. `long long long` ist zu lang.

Die einzelnen Schlüsselwörter können zudem beliebig angeordnet sein. So ist beispielsweise `long int long` dasselbe wie `int long long`.

Wie bereits erwähnt, ist der `int`-Typ plattformabhängig. Das bedeutet, dass dieser Typ je nach System, Prozessor oder Compiler eine unterschiedliche Anzahl Bits benötigt. Damit sind aber auch all seine modifizierten Verwandten plattformabhängig, was den Umgang, wie auch die Beschreibung dieser Typen schwierig macht.

Grundsätzlich soll gelten:

```
long long >= long >= int >= short >= char
```

Salopp ausgedrückt bedeutet dies: Ein `long long` kann potentiell (je nach Plattform) einen größeren Wertebereich speichern als ein `long`, usw.

Des Weiteren wird vom Standard vorgegeben: `short` und `int` müssen wenigstens 16 Bits haben, `long` mindestens 32 Bits, `short` darf nicht länger als `int` und `int` darf nicht länger als `long` sein.

Die Zahl der Bytes für die Darstellung der `int`-Datentypen ist compilerabhängig. Nach Vorschrift des ISO-Standards muss der Compilerhersteller die Wertebereichsgrenzen für Integer-Standard-Datentypen in der Bibliothek `<limits.h>` ablegen.



Auch für den `double`-Typ gibt es den Modifikator `long`. Dies bezeichnet einen Gleitpunkt-Typ mit erweiterter Genauigkeit. Er ist jedoch nicht standardisiert und wird deswegen hier nicht weiter diskutiert.

Heutzutage werden Integer-Typen mit expliziter Bitangabe bevorzugt. Siehe dazu Kapitel [→ 7.2.3](#).

E.2 Suffixe von arithmetischen Literalen

Wird an eine Integer-Konstante der Buchstabe `u` oder `U` als **Suffix** angehängt, beispielsweise `12u`, so ist die Konstante vom Typ `unsigned`, einem Integer-Typ ohne Vorzeichen.

Wird der Buchstabe `l` oder `L` an eine Integer-Konstante angehängt, beispielsweise `123456L`, so ist sie vom Typ `long`.

Es ist auch möglich, einer Integer-Konstanten beide Suffixe anzuhängen, beispielsweise 123ul. Diese Zahl ist dann vom Typ unsigned long.

Das Suffix ll oder LL steht für den Datentyp long long.

Auch für Gleitpunkt-Typen existiert das Suffix L. Ein Konstante wie beispielsweise 12.345L hat den Typ long double.

E.3 Wertebereiche von Integer-Typen und Gleitpunkt-Typen

Hier eine Übersicht über die **Wertebereiche** von Integer-Typen, wie sie auf heutigen modernen Computern **üblicherweise** zu erwarten ist:

Datentyp	Bits	Wertebereich
char	8	-128 bis +127
short	16	-32768 bis +32767
int	32	-2147483648 bis +2147483647
long	32	-2147483648 bis +2147483647
long long	64	-9223372036854775808 bis +9223372036854775807
unsigned char	8	0 bis 255
unsigned short	16	0 bis 65535
unsigned int	32	0 bis 4294967295
unsigned long	32	0 bis 4294967295
unsigned long long	64	0 bis 18446744073709551615

Die tatsächlichen Werte auf einem spezifischen Computer können abgefragt werden durch Makros, welche in der <limits.h>-Bibliothek hinterlegt sind. Beispielsweise bezeichnet INT_MAX den maximalen Wert eines signed int-Typs.

Für Gleitpunkt-Typen gibt es ebenfalls Makros, welche den Wertebereich darstellen. Sie befinden sich in der <float.h>-Bibliothek. Hier beispielsweise die Werte auf dem System des Autors:

Makro	Wert
FLT_MIN	1.175494e-38
FLT_MAX	3.402823e+38
DBL_MIN	2.225074e-308
DBL_MAX	1.797693e+308

Hierbei ist zu beachten, dass der Wertebereich von Gleitpunkt-Typen nicht gleich angegeben werden kann wie bei Integer-Typen. Aufgrund der Exponentendarstellung ist die Angabe des „Minimums“ equivalent zur kleinsten positiven Zahl, welche nicht null ist und das „Maximum“ ist die größte positive Zahl, welche nicht als Unendlich gilt.

Weitere Konstanten werden gegeben wie beispielsweise die Anzahl genauer Dezimalstellen, die Anzahl Bits für den Exponenten und die Signifikante, sowie das Maschinenepsilon. Letzteres bezeichnet die kleinstmöglich darstellbare Distanz von der Zahl 1. Auf eine Auflistung all dieser Angaben wird hier verzichtet.

E.3.1 Bedeutung von typedef für portable Datentypen

Da die obengenannten Typen auf unterschiedlichen Rechnerarchitekturen unterschiedliche Größen aufweisen, war es zu früheren Zeiten notwendig, mittels **typedef** programmspezifische Typen zu deklarieren, welche dann auf unterschiedlichen Architekturen so kompiliert wurden, dass das Programm portabel wurde.

Definiert man eigene Datentypen, so treten die nicht portablen Datentypen nur einmal im Programm auf, nämlich in der typedef-Vereinbarung. Ansonsten kommen sie im Programm nicht vor.

Hierfür ein Beispiel: Auf einem Rechner wird ein eigener Typname INT definiert:

```
typedef int INT;
```

Auf einem anderen Rechner sieht diese Vereinbarung vielleicht so aus:

```
typedef long INT;
```

Diese unterschiedliche Vereinbarung ist möglicherweise notwendig, da auf dem einem Rechner ein int mit 32 Bits, auf dem anderen jedoch nur mit 16 Bits gespeichert wird. Da jedoch das Programm unbedingt Integer-Zahlen mit 32 Bits benötigt, muss auf der zweiten Architektur der Typ long verwendet werden, welcher tatsächlich 32 Bits speichert.

Auf verschiedenen Rechnern wird somit in der typedef-Vereinbarung einfach der passende maschinenabhängige Datentyp des Rechners gesetzt.

Innerhalb des eigentlichen Programmes wird sodann nicht mehr direkt der Typ `int` oder `long` verwendet, sondern stets der Typ `INT`. So kann mittels einer einzigen Änderung im Code der Typ von der einen in die andere Architektur umgeschaltet werden. Mithilfe von vordefinierten Makros und bedingter Compilierung mittels `#if` kann dies sogar automatisiert werden.

Um mit den verschiedenen Bitgrößen besser umgehen zu können, wurden im Standard C99 verschiedene Integer-Typen wie beispielsweise `int32_t` definiert (siehe Kapitel [→ 7.2.3](#)), um die benötigte Anzahl Bits exakt festzulegen. Somit ist die Verwendung von `typedef` für einfache Datentypen heutzutage nicht mehr häufig anzutreffen.

E.4 Zweierkomplement-Codierung

Ganze Zahlen werden auf heutigen Rechnern normalerweise im sogenannten **Zweierkomplement** gespeichert. Ab dem Standard C23 ist diese Codierung sogar zwingend.

Das höchstwertige Bit der Zweierkomplement-Zahl gibt das Vorzeichen an. Ist es null, so ist die Zahl positiv, ist es 1, so ist die Zahl negativ. Zur Erläuterung soll folgendes Beispiel einer Zweierkomplement-Zahl von der Größe 8 Bits dienen:

	MSB				LSB			
Bitmuster	1	0	1	0	0	1	1	1
Stellenwertigkeit	-2^7	$+2^6$	$+2^5$	$+2^4$	$+2^3$	$+2^2$	$+2^1$	$+2^0$
	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0

Beachten Sie, dass Bit 0 das sogenannte least significant bit (LSB) ist. Das höchste Bit wird als most significant bit (MSB) bezeichnet.

Der Wert dieses Bitmusters errechnet sich aufgrund der Stellenwertigkeit:

$$\begin{aligned}
 & -1 \cdot 2^7 + 0 \cdot 2^6 + 1 \cdot 2^5 + 0 \cdot 2^4 + 0 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 \\
 & = -128 + 32 + 4 + 2 + 1 \\
 & = -89
 \end{aligned}$$

Die dem Betrag nach größte positive Zahl in dieser Darstellung ist:

$$0111\ 1111_2 = 64 + 32 + 16 + 8 + 4 + 2 + 1 = 127$$

Die dem Betrag nach größte negative Zahl in dieser Darstellung ist:

$$1000\ 0000_2 = -128$$

Die tief gestellte 2 bedeutet, dass es sich bei den eingeklammerten Ziffern um Binärziffern (Ziffern zur Basis 2), handelt.

Eine andere (äquivalente) Rechenvorschrift zur Berechnung des Wertes negativer Zahlen ist:

- Schritt 1: Da das höchste Bit 1 ist, ist die Zahl negativ.
- Schritt 2: Invertiere alle Bits (daher der Name Zweierkomplement).
- Schritt 3: Addiere die Zahl 1.
- Schritt 4: Berechne die Zahl in der üblichen Binärdarstellung mit den Stellenwerten $2^7 \dots 2^0$ und füge danach das Vorzeichen von Schritt 1 hinzu.

Wendet man diese Rechenvorschrift auf das obiges Beispiel an, so erhält man:

- Schritt 1: Zahl ist negativ
- Schritt 2: Invertieren: 01011000
- Schritt 3: 1 Hinzuaddieren: 01011001
- Schritt 4: $-(2^6 + 2^4 + 2^3 + 1) = -(64 + 16 + 8 + 1) = -89$

E.5 Gleitpunkt-Codierung

Nach IEEE 754 ist die Basis (Radix) eines **Gleitpunkt-Typs** stets 2. Je nachdem, wie viele Bits für **Signifikante** (früher „Mantisse“) und **Exponent** reserviert werden, können reelle Zahlen mit unterschiedlicher Genauigkeit abgebildet werden:

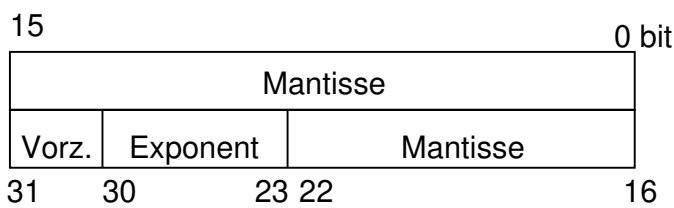
Datentyp float

- 1 Vorzeichenbit (Bit 31)
- 8 Bits für den Exponenten (Bit 23 - 30)
- 23 Bits für die Signifikante (Bit 0 - 22)

Datentyp double

- 1 Vorzeichenbit
- 11 Bits für den Exponenten
- 52 Bits für die Signifikante

Das folgende Bild zeigt die Darstellung einer float-Zahl nach IEEE 754 beziehungsweise IEC 60559:



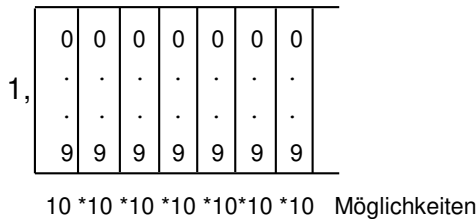
Das Vorzeichenbit hat für negative Zahlen den Wert 1, sonst den Wert 0.

Im Folgenden wird die Darstellung einer float-Zahl betrachtet. Dabei wird die Genauigkeit der Darstellung und die kleinst- beziehungsweise größtmögliche positive Zahl diskutiert.

E.5.1 Signifikante – Genauigkeit der Darstellung

Die Signifikante ist normalisiert, das heißt, die erste Ziffer vor dem Komma ist gleich 1. Die 1 vor dem Komma wird nicht gespeichert. Für die Signifikante stehen 23 Bits zur Verfügung. Für jedes Bit hat man zwei Möglichkeiten (0 oder 1). Daraus resultieren 2^{23} Möglichkeiten. Eine einfache Umformung ergibt:

Damit sind mit 23 Bits fast – aber nicht ganz – 10^7 Zustände darstellbar. In der Dezimaldarstellung hat man für die Signifikante die folgenden Möglichkeiten:



Man sieht, dass 10^7 Zuständen sieben Stellen hinter dem Komma entsprechen. Da es jedoch nur fast 10^7 Zustände sind, können mit 23 Bits nur sechs Dezimalstellen wirklich genau abgebildet werden.

E.5.2 Exponent – Kleinste und größte positive Zahl

Mit acht Stellen kann man die Zahlen 0 bis 255 bilden. Die Rechnerhardware führt dann automatisch eine Verschiebung (ein sogenanntes Bias) dieser Zahl durch, um dem Exponenten sowohl positiv als auch negativ einen möglichst großen Wertebereich zu ermöglichen. Die Zahlen 0 und 255 sind jedoch für spezielle Werte reserviert:

Es muss möglich sein, Sonderfälle anzugeben, wie beispielsweise die Zahlen rund um null oder dass es sich um plus oder minus Unendlich handelt, oder dass versucht wurde, null durch null zu dividieren. Für null geteilt durch null sieht IEEE 754 die Bezeichnung Not-a-Number (**NaN**) vor. NaN ist also bei float-Zahlen beispielsweise das Ergebnis bei der Division $0.0f / 0.0f$.

Gültige Exponenten für normalisierte Zahlen können somit wegen der Sonderfälle nur mit 1 bis 254 beziffert werden. Mit einem Bias von beispielsweise 127 ergeben sich somit die codierten Exponenten -126 bis 127.

Somit kann die kleinste und größte positive von null verschiedene Zahl berechnet werden durch:

kleinste Signifikante * kleinster Exponent

größte Signifikante * größter Exponent

$$= 1.000\dots0000 * 2^{-126} = 2^{-126}$$

$$= 1.111\dots1111 * 2^{127} = (1 + 1/2 + 1/4 + 1/8 + \dots + 1/(2^{23})) * 2^{127} = \text{circa } 2^{128}$$

Die beiden Zahlen sind in der <float.h>-Bibliothek hinterlegt im Makro FLT_MIN = $1.175494\text{e-}38$ und FLT_MAX = $3.402823\text{e}+38$.

Literatur

- [Bac91] M.J. Bach. *UNIX – Wie funktioniert das Betriebssystem*. Hanser, 1991.
- [Bar83] J.P.G. Barnes. *Programmieren in Ada*. Hanser, München, 1983.
- [C11] ISO/IEC 9899 Programming languages – C, Technical corrigendum 1, 2012-07-15.
- [C90] ISO/IEC 9899 Programming languages – C, First edition 1990-12-15, ISO/IEC 9899 Programming languages – C, Technical corrigendum 1, 1994-09-15, ISO/IEC 9899 Programming languages – C, Amendmend 1: C Integrity, 1995-04-01, ISO/IEC 9899 Programming languages – C, Technical corrigendum 2, 1996-04-01.
- [C99] ISO/IEC 9899 Programming languages – C, Second edition 1999-12-01.
- [Dij68] E.W. Dijkstra. Go to statement considered harmful. *Communications of the ACM* 11 (3): pp. 147–148, 1968.
- [Hoa62] C.A.R. Hoare. Quicksort. *Computer Journal* 5(1), pp. 10-16, 1962.
- [Hoa85] C.A.R. Hoare. *Communicating Sequential Processes*. Prentice Hall International, 1985.
- [JTC03] ISO/IEC JTC1/SC22/WG14. Rationale for International Standard – Programming Languages – C, 2003.
- [KR78] B.W. Kerningham and D.M. Ritchie. *The C Programming Language*. Prentice Hall, 1978.
- [Lew94] D.A. Lewine. *POSIX Programmer's Guide*. O'Reilly, 1994.
- [Mey97] B. Meyer. *Object-Oriented Software Construction*. Prentice Hall, 1997.
- [NBB⁺63] P. Naur, J. W. Backus, F. L. Bauer, J. Green, C. Katz, J. McCarthy, A. J. Perlis, H. Rutishauser, K. Samelson, B. Vauquois, J. H. Wegstein, A. van Wijngaarden, and M. Woodger. Revised report on the algorithmic language algol 60. *Comm. ACM* 6(1), pp. 1-17, 1963.
- [Par72] D.L. Parnas. *On the Criteria to be used in Decomposing Systems into Modules*. *Comm. ACM* 15(12), pp. 1053-1058, 1972.

- [Sed92] R. Sedgewick. *Algorithmen in C*. Addison-Wesley, 1992.
- [Wet80] H. Wettstein. *Systemprogrammierung*. Hanser, 1980.

Index

A

Abarbeitungsrichtung 184
Abgeleiteter Typ 120
abort() 485
Abstraktion 12
Abweisende Schleife 50
Adress-Operator 149
Adresse 8, **145**
Aggregats-Initialisierung 160
Aktivitäts-Diagramm 46
Aktueller Parameter 271
Algorithmus 16, **39**
Alignment **208**, 210
Annehmende Schleife 50
Anonyme Struktur 381
ANSI-C 18
Anweisung 3, 23, **40**, 171
argc 478
Argument 25, 36, 78, **272**, 477
argument count 478
argument vector 478
argv 478
Arithmetische Typumwandlung 218
Arithmetischer Typ 121
Array **159**, 305, 393
Array als Parameter 319
Array-Element 159
Array-Element-Operator 161
Array-Index 161
Arrayname 305
ASCII 63, 100, **690**
Assoziativität 184
Atomare Operation 662
Aufrufhierarchie 70
Aufzählungs-Konstante .. **128**, 236
Aufzählungs-Typ 128
Ausdruck 13, 32, **171**
Ausdrucksanweisung 171
Ausführungszeichensatz 100

Auswertungsreihenfolge 184
auto 417
Automatische Initialisierung ... 422

B

Backslash 111
Backtracking 288, **567**
Basiszeichensatz 99
Baum 515
Bedingte Anweisung 48
Bedingte Compilierung 587
Bedingung 31, 48, **230**
Befehlszeiger 72
Bezeichner 105
Bibliothek 4, **70**, 92, 282
Binärer Baum 518
Binärmodus 444
binary 87
binary search 547
Binder 93
Bindung 137, **406**
Binär 108, **684**
Binäres Suchen 547
Bit 8, **60**
Bitfeld 388
Blattknoten 516
Block 24, 51, 229, **259**
Boolescher Wert 130, **194**
bounds checking interfaces ... 471
break 236, 251, **253**
Bubblesort 573
build 97
Byte 8
Byte-Ausrichtung 210

C

C++ 18
C-String 317
C11 19
C17 19

C2319
 C9019
 C9519
 C9919
 call by pointer276
 call by reference275
 call by value275
 Call-Stack289
 case235
 case sensitive105
 cast-Operator215
 char113, **122**
 char-Array**317**, 322
 Code-Segment409
 Codierung**63**, 689
 Compiler4, **87**
 compound literal382
 compound statement260
 condition variable641
 const**142**, 324
 const expression172
 const-safe142, **278**, 376
 continue254
 core dump92
 critical section633

D

Datei428
 Dateideskriptor675
 Dateipositionszeiger446
 Dateipuffer445
 Dateisystem428
 Dateivariablen**432**, 440
 Daten-Segment409
 Datenabstraktion14
 Datenstrom427
 Datenstruktur361
 Datentyp65, **119**
 Deadlock629, **651**
 Debugger98
 default235
 #define30, **581**
 defined588

Definition23, **135**
 Deklaration135
 Dereferenzierung152
 designated initializer366
 Dezimal107, **683**
 Dezimalpunkt109
 Dining Philosophers652
 Direktive577
 divide and conquer535
 double126
 do while249
 Dynamische Datenstruktur505
 Dynamischer Speicher489
 Dynamisches Array500

E

Editor85
 Ein- und Ausgabe27, **427**
 Einfache Selektion36, 48, **230**
 Einfacher Datentyp66
 Einrückung22
 Elementarer Typ120
 #elif588
 Ellipse284
 #else588
 else230
 else if233
 #endif588
 Endlosschleife**250**, 629
 Entwicklungsumgebung4, **97**
 enum128, **236**
 Ersatzdarstellung26, **111**
 escape sequence26, **111**
 Etikett128, **362**, 384
 EVA-Prinzip74
 executable93, **406**
 exit()483
 Exit-Status481
 Explizite Typumwandlung215
 Exponent109, 126, **702**
 Export-Schnittstelle601
 extern411
 Externe Bindung137, **406**

Externe Variable79, **137**, 262

F

Fall234
Fehler-Flag 447
Feld 361
file 428
FILE 440
file system 428
File-Pointer 432, **440**
float 126
flow chart 46
Flussdiagramm 46
for 242
Formaler Parameter 271
Format-String 26, **455**
Formatelement 26, **455**
free() 497
Funktion11, 36, 51, 68, **77**, 259
Funktionsaufruf **78**, 272
Funktionskopf 77, 259, **268**
Funktionspointer ... **350**, 393, 397
Funktionsrumpf 78, 259, **268**

G

Ganzzahl 23
Geordneter Baum 517
Gleitpunkt-Konstante 109
Gleitpunkt-Typ 32, **702**
Gleitpunkt-Zahlen 126
Globale Variable 79, **137**, 262, 404
goto 255
Gültigkeit **263**, 423, 490

H

h-file 27
Hardwarenahe Programmierung . 8
Hashverfahren 552
Hauptprogramm **25**, 68
header file 27
Header-Datei 27, 82, 282, **606**
Heap 410
heap corruption 156

Heap-Segment 410, **489**
Hexadezimal 108, **684**
High-Level-Dateizugriff 438
Höhere Programmiersprache ... 87

I

IDE 4, **97**
identifizier 105
#if 588
if 230
#ifdef 588
#ifndef 588
imperative Sprache 16
Implementation 2
Implizite Typumwandlung . 33, **217**
Import-Schnittstelle 601
#include 4, 27, **579**
indentation 22
Information Hiding . **400**, 601, 606
Initialisierung 43, 65, **140**, 312, 422
Initialisierungsliste 312, **365**
inline 298
Innerer Knoten 516
instruction pointer 72
int 123
int32_t 124
Integer 23
Integer-Konstante 107
Integer-Zahlen 123
Interne Bindung 137, **406**
Interne Variable **137**, 262
Interpunktionszeichen 115
ISO 646 63, 100, **690**
Iteration 51, **287**

K

Kante 515
keyword 103
Klammerung 184
Knoten 515
Kollision 553
Kommentar **29**, 83
Komplexität 540

Konflikt 553
 Konsole **5**, 95
 Konstanter Ausdruck 172
 Konstanter String 110
 Kontrollfluss 44
 Kontrollstruktur 13, **229**
 Kritischer Abschnitt 633

L

L-Wert 178
 label 255
 Lader **95**, 408
 Laufvariable 243
 Laufzeitsystem 92
 Lebensdauer .. **262**, 423, 490, 657
 Leerzeichen 22
 library 70
 Linkage 137
 Linker 4, **93**, 405
 Liste 505
 Listenelement 505
 Literale Konstante 107
 Livelock 651
 loader 95
 Lokale Variable **137**, 262, 404, 417
 long 132, **697**
 Low-Level-Dateizugriff .. 439, **675**
 lvalue 178

M

main() 3, **25**, 69, 477
 Makro 30, **581**
 Makro mit Parameter 583
 malloc() 491
 Mantisse 109, **702**
 Manuelle Initialisierung 140
 Marke 255
 Maschinencode 87
 Maschinensprache 4, **87**
 Mehrdimensionales Array 314
 Mehrfache Selektion 49, **234**, **239**
 Mehrfaches Inkludieren 610
 Member 361

memory leak 499
 Modifikator 132
 Modulares Design 600
 Modularisierung 68
 Mutex 635

N

Namensraum 401
 NaN 704
 Nebeneffekt 174
 newline 26
 NULL **151**, 156, 308
 Nullzeichen 99, **110**, 164, 317

O

O-Notation 540
 Objektdatetei 86, 93, **405**
 Objektorientierung 604
 Oktal 107
 Opaker Typ 400
 Operand 66
 Operation 65
 Operator 66, 115, **180**
 operator precedence **184**, 393
 Operator-Priorität 184
 Optimierung **91**, 143, 166
 overflow 133

P

Parameter 25, 36, 77, **272**
 Parameterliste 269
 Pfeil-Operator 371
 Pipe 436
 Pointer **145**, 305, 393
 Pointer auf Struktur 368
 Pointerarithmetik 308
 Polling 641
 Postfix-Operator 183
 Präfix 113
 Präfix-Operator 183
 Präprozessor 30, 86, **577**
 #pragma 594
 preprocessor 30, **577**

printf() 453
Programm 16, **40**, 59, 86, 93
Programmfluss 71
Prototyp 81, **280**
Prozedur 68, **269**
Prozedurale Programmierung 11, **59**
Prozess 621
Pseudocode 54
Puffer **329**, 441
Punkt-Operator **364**, 371

Q

Qualifikator 141
Quellcode 10, **99**
Quelldatei **79**, 83, 85
Quellzeichensatz 100
Queue 642
Quicksort 535

R

R-Wert 178
Race Condition 633
Referenzierung 152
register 418
Rekursion **287**, 532
restrict 166
return 267, **273**, 482
Rückgabetypp 77, **268**
Rückgabewert 172
Rundungsfehler 127
runtime system 92
rvalue 178

S

Scheduler 622
Schleife 50
Schleifenvariable 30
Schlüssel **545**, 552
Schlüsselwort 103
Schnittstelle 268, **601**
Schrittweise Verfeinerung 73
Segment 409
selection sort 532

Semantik **6**, 90
Semikolon 23
Sentinel 514
Separate Compilierung 607
sequence point 176
Sequenzielles Suchen 546
Sequenzpunkt 176
shadowing 264
short 132, **697**
Sichere E/A-Funktionen 471
Sichtbarkeit 138, **263**
Signatur 14, 259, **268**
signed 132
Signifikante 109, 126, **702**
sizeof 206
size_t 162
Skalarer Typ 121
Sortieren 531
Sortieren durch Auswählen ... 532
source code 10
source file **79**, 85
Speicherklasse 403
Speicherleck 499
Speisende Philosophen 652
Stack (Datenstruktur) **288**, 611
Stack-Segment 289, **410**
Standardausgabe 434
Standardeingabe 435
Standardfehlerausgabe 435
Starvation 651
static 299, **415**, 420, 606
Statische Lokale Variable 420
Statische Variable 654
Status 481
Status-Flag 447
stderr **434**, 441
stdin **434**, 441
stdout **434**, 441
Steuerzeichen 691
Stream **429**, 440
String 5, **164**
String als Parameter 320

String-Konstante **317**, 322
 String-Literal 317
 struct 362
 Strukt. Programmierung 10, **44**
 Struktur 67, 361, **362**
 Strukturiertes Design 599
 Suchbaum 519
 Suchen 531, **545**
 Suffix **108**, 698
 switch 234
 Symbol 105
 Symbolische Konstante ... 30, **582**
 Syntax 6, **87**, 89

T

tag 128, **362**, 384
 Task 622
 Teile und Herrsche **535**, 567
 Text-Segment 409
 Textmodus 444
 Thread 624
 thread local storage 657
 thread specific storage 659
 Thread-lokaler Speicher 657
 Thread-spezifischer Speicher . 659
 top-down 73
 Typ 9, 65, **119**
 Typ-Erweiterung 220
 type promotion 220
 typedef **396**, 700
 Typisierung 119
 Typumwandlung 119

U

Überlauf 133
 UCS 63
 UML 46
 Umlenken 434
 #undef 581
 underflow 133
 Unicode 63, 101, 113, **692**
 Unified Modeling Language 46

Union 384
 union 384
 unsigned 132
 Unterlauf 133
 Unterprogramm 68
 UTF-16 63, 113, **694**
 UTF-32 63, 113, **693**
 UTF-8 63, 101, 113, **694**

V

Variable 9, 23, 42, **64**, 135
 Variable Parameterliste 284
 Vereinbarung 135
 Virtuelle Adresse 405
 void **131**, 268
 void-Pointer **158**, 308
 volatile 143
 Vorrang-Reihenfolge 393
 Vorwärtsdeklaration **280**, 368, 399

W

Wahrheitswert 194
 Warteschlange 642
 wchar_t 113, **694**
 Wert 64
 Wertebereich 119, **699**
 while 241
 Whitespace 22, **102**
 wide character 113
 Wurzel 516

Z

Zählschleife **30**, 243
 Zeichen **59**, 64
 Zeichen-Konstante 110
 Zeichenkette 5
 Zeiger 145
 Zusammengesetzter Typ .. 67, **121**
 Zustandsvariable 641
 Zuweisung 42
 Zweierkomplement 701